Domenico Bianculli · Hassan Sartaj ·
Vasilios Andrikopoulos · Cesare Pautasso ·
Tommi Mikkonen · Jennifer Perez ·
Tomáš Bureš · Martina De Sanctis ·
Henry Muccini · Elena Navarro ·
Mohamed Soliman · Uwe Zdun (Eds.)

LNCS 15982

Software Architecture

ECSA 2025 Tracks and Workshops

Limassol, Cyprus, September 15–19, 2025
Proceedings

Springer

Lecture Notes in Computer Science **15982**

Founding Editors

Gerhard Goos
Juris Hartmanis

Editorial Board Members

Elisa Bertino, *Purdue University, West Lafayette, IN, USA*
Wen Gao, *Peking University, Beijing, China*
Bernhard Steffen, *TU Dortmund University, Dortmund, Germany*
Moti Yung, *Columbia University, New York, NY, USA*

The series Lecture Notes in Computer Science (LNCS), including its subseries Lecture Notes in Artificial Intelligence (LNAI) and Lecture Notes in Bioinformatics (LNBI), has established itself as a medium for the publication of new developments in computer science and information technology research, teaching, and education.

LNCS enjoys close cooperation with the computer science R & D community, the series counts many renowned academics among its volume editors and paper authors, and collaborates with prestigious societies. Its mission is to serve this international community by providing an invaluable service, mainly focused on the publication of conference and workshop proceedings and postproceedings. LNCS commenced publication in 1973.

Domenico Bianculli · Hassan Sartaj ·
Vasilios Andrikopoulos · Cesare Pautasso ·
Tommi Mikkonen · Jennifer Perez ·
Tomáš Bureš · Martina De Sanctis ·
Henry Muccini · Elena Navarro ·
Mohamed Soliman · Uwe Zdun
Editors

Software Architecture

ECSA 2025 Tracks and Workshops

Limassol, Cyprus, September 15–19, 2025
Proceedings

Springer

Editors
Domenico Bianculli
University of Luxembourg
Luxembourg, Luxembourg

Vasilios Andrikopoulos
University of Groningen
Groningen, The Netherlands

Tommi Mikkonen
University of Jyväskylä
Jyväskylä, Finland

Tomáš Bureš
Charles University
Prague, Czech Republic

Henry Muccini
University of L'Aquila
L'Aquila, Italy

Mohamed Soliman
Paderborn University
Paderborn, Germany

Hassan Sartaj
Simula Research Laboratory
Oslo, Norway

Cesare Pautasso
University of Lugano (USI)
Lugano, Switzerland

Jennifer Perez
Technical University of Madrid (UPM)
Madrid, Spain

Martina De Sanctis
Gran Sasso Science Institute
L'Aquila, Italy

Elena Navarro
Universidad de Castilla-La Mancha
Albacete, Spain

Uwe Zdun
University of Vienna
Vienna, Austria

ISSN 0302-9743 ISSN 1611-3349 (electronic)
Lecture Notes in Computer Science
ISBN 978-3-032-04402-0 ISBN 978-3-032-04403-7 (eBook)
https://doi.org/10.1007/978-3-032-04403-7

© The Editor(s) (if applicable) and The Author(s), under exclusive license
to Springer Nature Switzerland AG 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Preface

The European Conference on Software Architecture (ECSA) is the premier European conference aimed at bringing together leading researchers and practitioners to present and discuss the most recent, innovative, and significant findings and experiences in the field of software architecture research and practice.

This volume complements the proceedings of the 19th edition of ECSA, held on September 15–19, 2025 in Limassol. It consists of the refereed and accepted papers from the Tools & Demos, Tutorials, and Doctoral Symposium tracks of the main conference. In addition, it contains the proceedings of the workshops co-located with the conference:

- 1st International Workshop on AI-Assisted Software Architecting (AISA)
- 5th International Workshop on Agility with Microservices Programming (AMP)
- 1st International Workshop on Architecting Digital Twin of an Organization (ArchDTO)
- 12th Workshop on Adaptive and Reconfigurable Systems and Architectures (AROSA)
- 8th International Workshop on Context-Aware, Autonomous and Smart Architectures (CASA)
- 1st International Workshop on Software Architecture for Data-Intensive Systems (SADIS)

A more detailed summary of these workshops is provided below.

In total, we received 56 submissions through the EasyChair system. From these submissions, and after a single-blind review process, we accepted 32 papers for inclusion in this volume, 13 full and 19 short. Each paper was reviewed by at least 3 reviewers. We would like to thank the reviewers for both tracks and workshops for their contributions to this process.

We would like to thank the authors of all these submissions for their contributions.

July 2025

Domenico Bianculli
Hassan Sartaj
Vasilios Andrikopoulos
Cesare Pautasso
Tommi Mikkonen
Jennifer Perez
Tomáš Bureš
Martina De Sanctis
Henry Muccini
Elena Navarro
Mohamed Soliman
Uwe Zdun

Organization

1st International Workshop on AI-Assisted Software Architecting (AISA)

AI-assisted software architecting is transforming how architectural decisions are made, implemented, and maintained. Leveraging artificial intelligence, this paradigm enhances architectural decision making, automates repetitive architecting tasks, and optimizes system designs. The AISA workshop aimed to bring together researchers and practitioners to explore AI's role in various software architecting activities, including architecture analysis, design, decision-making, evolution, and documentation. We also discussed AI-driven architectural pattern recommendations, best practices, and the application of AI in continuous architecting and self-adaptive architectures. Topics included AI-assisted architecture synthesis, automated architecture pattern selection, refactoring, security enforcement, and integrating AI into DevOps and MLOps pipelines. The workshop fostered discussions and collaborations between academia and industry to identify open challenges, research directions, and practical applications.

Organizers

Muhammad Waseem	Tampere University, Finland
Pekka Abrahamsson	Tampere University, Finland
Peng Liang	Wuhan University, China

Program Committee

Nadeem Abbas	Linnaeus University, Sweden
Aakash Ahmad	Lancaster University in Leipzig, Germany
Muhammad Azeem Akbar	LUT University, Finland
Mst Shamima Aktar	Wuhan University, China
Teerath Das	University of Jyväskylä, Finland
Amleto Di Salle	Gran Sasso Science Institute, Italy
Majid Haghparast	University of Jyväskylä, Finland
Muhammad Imran	University of L'Aquila, Italy
Muhammad Taimoor Khan	University of Greenwich, UK
Arif Ali Khan	University of Oulu, Finland
Zengyang Li	Central China Normal University, China
Gastón Márquez	Universidad del Bío–Bío, Chile

Muhammad Nasir	International Islamic University Islamabad, Pakistan
Usman Nasir	Blekinge Institute of Technology, Sweden
Zeeshan Rasheed	Tampere University, Finland
Iflaah Salman	Lappeenranta–Lahti University of Technology, Finland
Malik Abdul Sami	Tampere University, Finland
Hassan Sartaj	Simula Lab, Norway
Mojtaba Shahin	RMIT University, Australia
Kari Systä	Tampere University of Technology, Finland
Zheying Zhang	Tampere University, Finland

5th International Workshop on Agility with Microservices Programming (AMP)

The AMP workshop aimed to collect experiences on microservice adoption and reporting best practices, as well as failure cases, to build community knowledge based on past mistakes and successes. We also emphasized the current trend towards AI-driven software engineering practices and the development of data- intensive applications. The workshop sought solutions and experiences on both AI-assisted tooling, and building data-intensive applications and LLMs using microservices. The workshop was open to experience reports from practitioners and academics. We sought to collect original work on the science and engineering of programming microservices.

Organizers

Jonas Fritzsch	University of Stuttgart, Germany
Sandro Speth	University of Stuttgart, Germany
Thatiane de Oliveira Rosa	IFTO, Brazil

Program Committee

Kyle Brown	IBM, USA
Sebastian Copei	Fraunhofer IEE, Germany
Renato Cordeiro	Universidade de São Paulo, Brazil
Filipe Correia	University of Porto, Portugal
Michał J. Gajda	Migamake Pte Ltd., Poland
Javad Ghofrani	University of Lübeck, Germany
Saverio Giallorenzo	University of Bologna, Italy
Alfredo Goldman	University of São Paulo, Brazil
Eduardo Guerra	Free University of Bozen-Bolzano, Italy
Floriment Klinaku	University of Stuttgart, Germany
Andrea Melis	University of Bologna, Italy
Marco Peressotti	University of Southern Denmark, Denmark
Francisco Poncem	University of Pisa, Italy
Florian Rademacher	RWTH Aachen University, Germany
Larisa Safinam	Inria Lille - Nord Europe, France
Alceste Scalas	Technical University of Denmark, Denmark

1st International Workshop on Architecting Digital Twin of an Organization (ArchDTO)

The Digital Twin of an Organization (DTO) extends the concept of Digital Twins to create virtual replicas of business models, processes, resources, and interactions. DTOs enable optimization of decision-making, increased efficiency, and adaptability to rapid changes by leveraging real-time data, simulations, AI, and predictive analytics. Despite their transformative potential, DTOs are still in early development, facing significant software architecture challenges such as scalability, interoperability, model robustness, and real-time data integration. This workshop explored the architectural frameworks, design patterns, and software engineering principles needed to build robust DTOs. Topics included frameworks for DTO creation, enterprise modeling methodologies, technological enablers like AI and the Internet of Things, and real-world use cases from academia and industry. Given the novelty of the topic, the workshop consisted of two main sessions: (1) presentations of academic and industry papers on DTO architectures, frameworks, and technologies, and (2) an interactive workshop session, where participants engaged in problem-solving, explored case studies, and collaboratively defined challenges, solutions, and best practices for architecting DTOs.

Organizers

Arianna Fedeli	Gran Sasso Science Institute, Italy
Stella Gatziu Grivas	University of Applied Sciences and Arts Northwestern Switzerland, Switzerland
Mauro Caporuscio	Linnaeus University, Sweden
Diego Perez-Palacin	Linnaeus University, Sweden

Program Committee

Vasilios Andrikopoulos	University of Groningen, The Netherlands
Balbir Barn	Middlesex University London, UK
Hugo Bruneliere	IMT Atlantique, France
Carlos E. Cuesta	Rey Juan Carlos University, Spain
Martina De Sanctis	Gran Sasso Science Institute, Italy
Anne Füßl	Technische Universität Ilmenau, Germany
Wolfgang Groher	University of Applied Sciences Eastern Switzerland (OST), Switzerland
Vinay Kulkarni	Tata Consultancy Services Research, India

Emanuele Laurenzi	University of Applied Sciences and Arts Northwestern Switzerland FHNW, Switzerland
Tommi Mikkonen	University of Jyväskylä, Finland
Devid Montecchiari	University of Applied Sciences and Arts Northwestern Switzerland FHNW, Switzerland
Elena Navarro	University of Castilla-La Mancha, Spain
George Angelos Papadopoulos	University of Cyprus, Cyprus
Cesare Pautasso	USI Lugano, Switzerland
Jennifer Pérez	Universidad Politécnica de Madrid, Spain
Luciana Rebelo	Gran Sasso Science Institute, Italy
Ronny Seiger	University of St. Gallen, Switzerland
Victoria Torres Bosch	Universitat Politècnica de València, Spain
Wilfrid Utz	University of Vienna, Austria
Pedro Valderas	Universitat Politècnica de València, Spain
Barbara Weber	University of St. Gallen, Switzerland
Uwe Zdun	University of Vienna, Austria

12th Workshop on Adaptive and Reconfigurable Systems and Architectures (AROSA)

The goal of this workshop was to bring together researchers and practitioners from both Academia and Industry working in the areas of the adaptation and reconfiguration of distributed systems. Different investigation topics were involved, such as: CBSE, Web services, cloud applications, mobile applications, Functional and Non-Functional requirements (QoS, performance, resilience), monitoring, diagnosis, decision and execution of adaptation and reconfiguration. Different research areas were covered: concepts, methods, techniques and tools to design, develop, deploy and manage adaptive and reconfigurable software systems. The concept of adaptive and reconfigurable software systems has been introduced in order to describe architectures which exhibit such properties. An adaptive and reconfigurable software system can repair itself if any execution problems occur, in order to successfully complete its own execution, while respecting functional requirements and quality attributes. In the design of an adaptive and reconfigurable software system, several aspects have to be considered. For instance, the system should be able to predict or to detect degradations and failures as soon as possible and to enact suitable recovery actions.

Organizers

Ismael Bouassida Rodriguez	University of Sfax, Tunisia
Nesrine Khabou	University of Sfax, Tunisia
Khalil Drira	University of Toulouse, France

Program Committee

Imen Abdennadher	University of Sfax, Tunisia
Yamine Aït-Ameur	IRIT/INPT-ENSEEIHT, France
Abdulatif Alabdulatif	Qassim University, Saudi Arabia
Riadh Ben Halima	University of Sfax, Tunisia
Federico Bergenti	University of Parma, Italy
Miriam Capretz	University of Western Ontario, Canada
Carlos E. Cuesta	Rey Juan Carlos University, Spain
Ernesto Exposito	Université de Pau et des Pays de l'Adour, France
Fairouz Fakhfakh	University of Sfax, Tunisia
Bernd Freisleben	University of Marburg, Germany
Wafa Gabsi	University of Sfax, Tunisia

Nikolaos Georgantas	Inria, France
Ian Gorton	Northeastern University, USA
Mohand-Said Hacid	Université Claude Bernard Lyon 1, France
Mohamed Jmaiel	University of Sfax, Tunisia
Slim Kallel	University of Sfax, Tunisia
Mehdi Khouja	University of Gabes, Tunisia
Francisco Moo-Mena	Autonomous University of Yucatán, Mexico
Mohamed Mosbah	University of Bordeaux, France
Henry Muccini	University of L'Aquila, Italy
Elisa Yumi Nakagawa	University of São Paulo, Brazil
Flavio Oquendo	IRISA (UMR CNRS) - Université Bretagne-Sud, France
Mourad Oussalah	University of Nantes, France
Saul Pomares	Autonomous University of Yucatán, Mexico
Claudia Raibulet	University of Milano-Bicocca, Italy
Mouna Rekik	University of Sfax, Tunisia
Philippe Roose	LIUPPA/UPPA, France
Cecilia Rubira	University of Campinas, Brazil
Chouki Tibermacine	University of Montpellier, France
Sami Yangui	University of Toulouse, France
Uwe Zdun	University of Vienna, Austria

8th International Workshop on Context-Aware, Autonomous and Smart Architectures (CASA)

Software is becoming increasingly aware of its current execution context. Decisions that were traditionally made at design time are being postponed until runtime to improve the services that software provides to its users. In addition, software reasons about the execution environments and its internal state to improve the functionality provided to its users. Some of the keywords that characterize software today are: context-aware, dynamic, autonomous, smart, adaptive, self-managed. In this context, the Context-aware, Autonomous, and Smart Architectures (CASA) workshop aimed to address the issues and challenges raised by the design, implementation, and evaluation of software systems characterized by the above keywords. Novel design and development approaches are needed to address the new issues raised by such systems. New solutions are expected to properly address the trade-offs between the various quality attributes that characterize these systems. Or, existing architectural solutions should be adapted and improved to meet the evolving and dynamic requirements of context-aware, autonomous, and intelligent systems. The workshop aimed to bring together software engineers, computer scientists, researchers, practitioners, and educators to discuss and exchange results, case studies, experiences, novel approaches, and visionary ideas on software architectures for context-aware, autonomous, and smart solutions.

Organizers

Luciana Rebelo	Gran Sasso Science Institute, Italy
Alessandro Tundo	TU Wien, Austria

Program Committee

Michel Albonico	Universidade Tecnológica Federal do Paraná, Brazil
Rafael Capilla	Universidad Rey Juan Carlos, Spain
Mauro Caporuscio	Linnaeus University, Sweden
Amleto Di Salle	Gran Sasso Science Institute, Italy
Cédric Eichler	INSA Centre Val de Loire, France
Arianna Fedeli	Gran Sasso Science Institute, Italy
Pasqualina Potena	RISE Research Institutes of Sweden AB, Sweden
Karthik Vaidhyanathan	IIIT Hyderabad, India
Ramon Salvador Vallès	Universitat Politècnica de Catalunya, Spain
Thierry Villemur	LAAS-CNRS, France

1st International Workshop on Software Architecture for Data-Intensive Systems (SADIS)

Modern software systems increasingly rely on data-centric processes to fuel intelligent features, scalable services, and real-time insights. As these systems evolve, designing effective software architectures for data-intensive environments becomes critical yet challenging. These architectures must address complex concerns such as handling high-velocity data streams, ensuring security and privacy, integrating heterogeneous data sources and heterogeneous computation fabrics (including GPUs/TPUs and edge devices), and adapting to changing workloads. The workshop aimed to bring together academic researchers, industry practitioners, and thought leaders to explore cutting-edge approaches for building, deploying, and maintaining robust data-centric solutions. We invited discussions on a range of topics at the intersection of data-intensive systems and software architecture, including machine learning (ML) pipelines, ML Operations (MLOps) frameworks and patterns, domain-specific languages, reference architectures, and visualization techniques. Emphasis was placed on practical experiences, innovative tools and platforms, and theoretical advancements that tackle the ever-growing demands placed on data-intensive systems. By fostering interdisciplinary collaboration and knowledge exchange, the workshop sought to shape the future of data-driven architecture design and empower professionals and researchers to address emerging challenges in this rapidly evolving domain.

Organizers

Ilias Gerostathopoulos	Vrije Universiteit Amsterdam, The Netherlands
Marcela Tuler de Oliveira	Delft University of Technology, The Netherlands
Tomáš Bureš	Charles University in Prague, Czech Republic
Milad Abdullah	Charles University in Prague, Czech Republic

Program Committee

Milad Abdullah	Charles University in Prague, Czech Republic
Faezeh Amou Najafabadi	Vrije Universiteit Amsterdam, The Netherlands
Steffen Becker	University of Stuttgart, Germany
Justus Bogner	Vrije Universiteit Amsterdam, The Netherlands
Ivan Compagnucci	Gran Sasso Science Institute, Italy
Martina De Sanctis	Gran Sasso Science Institute, Italy
Giorgos Giannopoulos	Athena Research Center, Greece

Petr Hnetynka	Charles University in Prague, Czech Republic
Henry Muccini	University of L'Aquila, Italy
George Papastefanatos	Athena Research Center, Greece
Keerthiga Rajenthiram	Vrije Universiteit Amsterdam, The Netherlands
Karthik Vaidhyanathan	IIIT Hyderabad, India
Yiannis Verginadis	Athens University of Economics and Business, Greece
Eleni Zarogianni	Intracom Telecom, Greece

ECSA 2025 Organization

Steering Committee

Nour Ali	Brunel University of London, UK
Vasilios Andrikopoulos	University of Groningen, The Netherlands
Paris Avgeriou	University of Groningen, The Netherlands
Thais Batista	Federal University of Rio Grande do Norte, Brazil
Stefan Biffl	TU Wien, Austria
Tomáš Bureš	Charles University, Czech Republic
Laurence Duchien	University of Lille, France
Carlos E. Cuesta	Rey Juan Carlos University, Spain
Matthias Galster	University of Canterbury, New Zealand
Ilias Gerostathopoulos	Vrije Universiteit Amsterdam, The Netherlands
Paola Inverardi	Gran Sasso Science Institute, Italy
Andrea Janes	Free University of Bozen-Bolzano, Italy
Patricia Lago	Vrije Universiteit Amsterdam, The Netherlands
Grace Lewis	Carnegie Mellon University, USA
Ivano Malavolta	Vrije Universiteit Amsterdam, The Netherlands
Raffaela Mirandola	Karlsruhe Institute of Technology, Germany
Henry Muccini	University of L'Aquila, Italy
Elisa Yumi Nakagawa	University of São Paulo, Brazil
Elena Navarro	University of Castilla-La Mancha, Spain
Flavio Oquendo	University of Southern Brittany, France
Ipek Ozkaya	Carnegie Mellon University, USA
Cesare Pautasso	Università della Svizzera italiana, Switzerland
Patrizia Scandurra	University of Bergamo, Italy
Jacopo Soldani	University of Pisa, Italy
Davide Taibi	University of Oulu, Finland
Bedir Tekinerdogan	Wageningen University and Research, The Netherlands
Chouki Tibermacine	University of Southern Brittany, France
Catia Trubiani	Gran Sasso Science Institute, Italy

Organizing Committee

General Chairs

Vasilios Andrikopoulos	University of Groningen, The Netherlands
Cesare Pautasso	USI Lugano, Switzerland

Research Track Program Chairs

Nour Ali	Brunel University of London, UK
Jacopo Soldani	University of Pisa, Italy

Industry Track Program Chair

Xiwei Xu	CSIRO Data61, Australia

Workshops Chairs

Tommi Mikkonen	University of Jyväskylä, Finland
Jennifer Perez	Universidad Politécnica de Madrid, Spain

Tools and Demos Chairs

Mohamed Soliman,	Paderborn University, Germany
Uwe Zdun	University of Vienna, Austria

Tools and Demos Program Committee

Humberto Cervantes	UAM Iztapalapa, Mexico
Daniel Feitosa	University of Groningen, The Netherlands
Anton Jansen	Philips Innovation Excellence, The Netherlands
Anne Koziolek	Karlsruhe Institute of Technology, Germany
Elisa Yumi Nakagawa	University of São Paulo, Brazil
Evangelos Ntentos	University of Vienna, Austria
Chouki Tibermacine	IRISA, CNRS and University of Southern Brittany, France

Tutorials Chairs

Tomáš Bureš	Charles University, Czech Republic
Martina De Sanctis	Gran Sasso Science Institute, Italy

Doctoral Symposium Chairs

Henry Muccini	University of L'Aquila, Italy
Elena Navarro	University of Castilla-La Mancha, Spain

Doctoral Symposium Program Committee

Roberta Capuano	University of L'Aquila, Italy
Carlos E. Cuesta	Rey Juan Carlos University, Spain
Martina De Sanctis	Gran Sasso Science Institute, Italy
Valentina Lenarduzzi	University of Oulu, Finland
Silverio Martínez-Fernández	UPC-BarcelonaTech, Spain
Nabor Mendonca	University of Fortaleza, Brazil
Raffaela Mirandola	Karlsruhe Institute of Technology, Germany
Patrizia Scandurra	University of Bergamo, Italy
Mahyar T. Moghaddam	University of Southern Denmark, Denmark
Karthik Vaidhyanathan	IIIT Hyderabad, India
Uwe Zdun	University of Vienna, Austria

Journal-First Chairs

Jesper Andersson	Linnaeus University, Sweden
Romina Spalazzese	Malmö University, Sweden

MIP Chairs

Raffaela Mirandola	Karlsruhe Institute of Technology, Germany
Remco de Boer	ArchiXL & Vrije Universiteit Amsterdam, The Netherlands

Open Science Chair

Valentina Lenarduzzi	University of Oulu, Finland

Proceedings Chairs

Domenico Bianculli	University of Luxembourg, Luxembourg
Hassan Sartaj	Simula Research Laboratory, Norway

Web Chair

Christos Therapontos	Easyconferences, Cyprus

Local Chair

George Angelos Papadopoulos	University of Cyprus, Cyprus

Contents

Tools and Demos

SAVAT: A Tool for Visualizing the Impact of Changes in Microservices 3
Adam Kattan Rahmani, Gabriel Elijah Goulis, David Kozák, Tomas Cerny, and Tomáš Vojnar

Infragenie: Living Software Architecture Diagrams From Docker Compose Files 12
Ricardo Ferreira, Filipe F. Correia, and Paulo G. G. Queiroz

ServLessSense: Serverless Smell Detection Tool 21
Hasini Sumalee Perera, Zadia Codabux, and Fabio Palomba

CUBEX: A CubeSat Exemplar for Teaching Software Architecture Principles 29
Angelos Motsios, Timo Kehrer, and Christos Tsigkanos

Tutorial

Software Architecture in the Age of Agentic AI 41
Karthik Vaidhyanathan and Henry Muccini

Doctoral Symposium

Requirements-Compliant IT System Deployment Architectures for IoT Systems 53
Florian Schmalriede

GreenDevLoop: Power-Aware Software Development with CI/CD, AI, and Feedback 62
Alessandro Aneggi

Architecting Carbon-Aware Software-as-a-Service 70
Samuele Giussani

Temporal Network Analysis of Microservice Architectural Degradation 78
Alexander Bakhtin

AISA Workshop

The Impact of AI-Generated Solutions on Software Architecture and Productivity: Results from a Survey Study 89
Giorgio Amasanti and Jasmin Jahić

Autonomic Microservice Management via Agentic AI and MAPE-K Integration 105
Matteo Esposito, Alexander Bakhtin, Noman Ahmad, Mikel Robredo, Ruoyu Su, Valentina Lenarduzzi, and Davide Taibi

Application of AI in Software Architecture Development at Volvo Construction Equipment 119
Nils Johansson and Mauro Caporuscio

Towards Benchmarking Design Pattern Detection Under Obfuscation: Reproducing and Evaluating Attention-Based Detection Method 127
Manthan Venkataramana Shenoy and Andreas Rausch

LLM-Based Explainability at Design Time: Detecting Elasticity Antipatterns in Software Architectures 141
Floriment Klinaku, Jonas Lammert, and Steffen Becker

AI-Powered Architecting for Industry 4.0 Cyber-Physical Production Systems: A Novel Approach, Research Problems and Challenges 155
Stephen John Warnett and Uwe Zdun

AMP Workshop

Assessing the Impact of Asynchronous Communication on Resilience and Robustness: A Comparative Study of Microservice and Monolithic Architectures 171
Nathanael Bosilia, Gerald Weinberger, and Philipp Haindl

Exploring Micro Frontends: A Case Study Application in E-Commerce 187
Ricardo Hideki Hangai Kojo, Luiz Fernando Corte Real, Renato Cordeiro Ferreira, Thatiane de Oliveira Rosa, and Alfredo Goldman

Industrial Views on DevOps Adoption Before and After Implementation: A Qualitative Comparison 202
Sebastian Copei, Oliver Hohlfeld, and Jens Kosiol

The (C)omprehensive (A)rchitecture (P)attern (I)ntegration Method: Navigating the Sea of Technology 212
Sebastian Copei, Oliver Hohlfeld, and Jens Kosiol

ArchDTO Workshop

Requirements for a Digital Twin of Organizations for Advancing Cybersecurity Risk Management 231
Bennet Puthuparambil and Claudio Giovanoli

AROSA Workshop

Comprehensive Exploration of Resilience and Adaptability in IoT-Based System-of-Systems: Bibliometric and Full-Text Literature Analysis 243
Aymen Abdelmoumen, Zakaria Benzadri, and Ismael Bouassida Rodriguez

Recent Literature Review on Modeling Explainable Medical Cyber-Physical Systems: A Bibliometric Analysis 259
Benina Feryel, Zakaria Benzadri, Faiza Belala, and Ahmed Hadj Kacem

Silver Box Testing for Scaling Software Development Fidelity: Enhancing Quality and Efficiency 273
Natali Levi-Soskin and Naomi Unkelos-Shpigel

CASA Workshop

The Lifecycle Workbench - A Configurable Framework for Digitized Product Maintenance Services 285
Dominique Briechle, Mohammed Fahad Ali, Marit Briechle-Mathiszig, Tobias Geger, Robert Werner, and Andreas Rausch

Exploring Static Analysis for Understanding Self-Adaptive Systems 295
Claudia Raibulet and Michele Pugno

SADIS Workshop

AutoML: A Tertiary Study of Phases, Methods, Tools, and Frameworks 313
Keerthiga Rajenthiram, Pauline Delporte, and Patricia Lago

FAIR-ER Data: Proposing a Data Model for Data Cooperatives 329
Francisco Mendonça, Nabil Abdennadher, and Giovanna Di Marzo Serugendo

GLOVES 2.0: Global Counterfactual-Based Visual Explanations 337
Nikolas Theologitis, Panagiotis Gidarakos, Stavros Maroulis, Loukas Kavouras, Giorgos Giannopoulos, and George Papastefanatos

Temporal Harmonization of Heterogeneous Software Logs: A Unified Model for Time-Series Analysis 345
Milad Abdullah, Petr Hnětynka, and Jiyan Salim Mahmud

Making a Pipeline Production-Ready: Challenges and Lessons Learned in the Healthcare Domain 354
Daniel Angelo Esteves Lawand, Lucas Quaresma Medina Lam, Roberto Oliveira Bolgheroni, Renato Cordeiro Ferreira, Alfredo Goldman, and Marcelo Finger

DQGen: Scalable Metadata-Driven Automation for Data Quality Validation in Data-Intensive Applications 363
Moamin Abughazala and Henry Muccini

A Visionary Architecture for Adaptive Data Contracts in Behavior-Driven IoT Systems 381
Moamin Abughazala and Henry Muccini

Author Index 389

Tools and Demos

SAVAT: A Tool for Visualizing the Impact of Changes in Microservices

Adam Kattan Rahmani[1], Gabriel Elijah Goulis[2], David Kozák[3,4(✉)], Tomas Cerny[2], and Tomáš Vojnar[1,3]

[1] FI, Masaryk University, Brno, Czechia
[2] SIE, University of Arizona, Tucson, USA
[3] FIT, Brno University of Technology, Brno, Czechia
ikozak@fit.vutbr.cz
[4] Oracle Labs, Brno, Czechia

Abstract. Change impact analysis is a vital process for managing changes in complex systems. This becomes especially important in decentralized systems like those based on microservices. The paper presents a SAVAT tool designed to facilitate change impact analysis in microservice systems. SAVAT enables practitioners to interact with visual exploration of microservice systems, both from two architectural views (service and domain views), using the established notions of context maps and service dependency graphs, along with a novel notion of inter-microservice call graphs. Our tool can also automatically highlight changes in different versions of microservice systems and provide support to explore how the changes propagate through the system.

1 Introduction

Microservices have gained substantial popularity in the industry due to their advantages in scalability, flexibility, and reliability. Decomposing monolithic applications into independently evolvable units offers significant flexibility to developers. However, this architectural shift makes it more difficult to maintain a comprehensive, holistic view of the system [8]. Even minor changes in one part of the system can trigger ripple effects, potentially leading to major failures [5].

A similar kind of issue appears also in object-oriented programs where the *change impact analysis* (CIA) approach has been proposed to provide feedback on the semantic impact of program modifications [15]. More recently, CIA techniques specifically tailored for microservice systems have emerged [6]. However, to the best of our knowledge, currently there is no general-purpose tool that offers an intuitive interface to holistically explore the system structure.

This paper introduces SAVAT (Software Architecture Visualization and Analysis Tool), a tool for visualizing the impact of changes in microservice systems[1]. SAVAT

The work was supported by the Czech Science Foundation project 23-06506S, the FIT BUT project FIT-S-23-8151, and the FI MU project MUNI/A/1600/2024. This material is also based upon work supported by the National Science Foundation under Grant No. 2409933.

[1] The SAVAT showcase video can be found at https://youtu.be/yR_YqUXRKjs.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 3–11, 2026.
https://doi.org/10.1007/978-3-032-04403-7_1

enables practitioners to interactively explore both the *service* and the *domain* architectural views, represented by the established notions of *service dependency graphs* (SDG) and *context maps* (CM), respectively. In addition, SAVAT introduces *inter-microservice call graphs* (IMCGs), an extended form of classical call graphs that captures both local method invocations within a single microservice and remote calls between methods in different microservices. This allows for a more detailed exploration of the dependency chains compared to the SDG alone. Furthermore, SAVAT offers automated CIA capabilities in all its views, enabling users to compare two versions of a system, identify differences, and analyze how local changes will affect the overall system.

The visualization offered by SAVAT provides stakeholders with multiple views on both the overall architecture of the system and the results of the analyses carried out on it. This allows stakeholders to identify architectural antipatterns, such as low cohesion, high coupling, or cyclic dependencies. Addressing these issues, stakeholders can propose architectural changes that eliminate potential antipatterns and reduce the risk of bugs or the reappearance of antipatterns in future versions of the microservice system.

2 Preliminaries

Fundamental concepts and notation include microservices, software architecture reconstructions, and change impact analysis.

2.1 Microservices Architecture

Systems designed using *microservice architecture* (MSA) consist of multiple autonomous services. Each of these services, known as a microservice, communicates with others via various types of API [7]. A single microservice can be described as a distinct deployment unit. Ideally, microservices should have a single responsibility and be contained within a specific context [13]. Their size may also be influenced by the technologies used, the types of API communication, and the deployment strategies [14].

2.2 Software Architecture Reconstruction

Software architecture reconstruction (SAR) is an approach that can be used to extract key knowledge about a given microservice system, which can later be used to analyze the impact of changes in the system [11]. In particular, we use this process to generate multiple views on the given microservice system, including its context map (CM), service dependency graph (SDG), and inter-microservice call graph (IMCG).

For our purposes, we use a tool called Prophet [3], which employs GraalVM Native Image [17], an ahead-of-time optimizing compiler. This tool is capable of extracting the necessary knowledge from each microservice for the construction of the views mentioned above. The extracted knowledge includes information on REST calls, endpoints, entities, and call graphs (CGs) that characterize the given microservice system. Prophet consists of two parts: the first part is a plugin within GraalVM Native Image that performs the extraction, while the second is a dedicated Java application that runs

GraalVM, along with the plugin, on top of each microservice to create multiple views from the previously extracted knowledge.

The results of SAR can be presented from various perspectives. These perspectives may include views centered on entities and bounded contexts (CMs), views that focus on communication between different microservices (SDGs, IMCGs), and views that examine the internal implementation of a specific microservice (microservice CGs).

To illustrate each type of SAR result, we begin with *SDGs*. These graphs demonstrate how microservices communicate with each other through API calls. The edges of the graph represent every pair of REST calls and endpoints between the microservices. These pairs are formed by matching the target URI of the REST call with the URI of the endpoints, as well as by matching the HTTP method of the REST call with the HTTP method of the endpoint. We focus on REST calls at the moment as they are provided by Prophet out of the box, but SAVAT is extensible in this regard.

CMs provide a perspective that emphasizes the relationships between entities within a microservice system. An entity is defined as an object that contains data and revolves around a specific use case. Each entity has a unique identifier that sets it apart from other entities in the CM. These entities can persist over time, and their structure may be based on a template, like a class in Java.

A *CG* is a directed graph whose nodes represent the methods and the edges indicate their caller-callee relationships [9]. Visualizing CGs provides users with an efficient way to navigate the intricate relationships between calls. Below, we distinguish between two types of CGs: *individual microservice CGs* and the *IMCG* that combines individual microservice CGs with the SDG. This graph not only illustrates the caller-callee relationships among all methods in the microservice system but also highlights the API calls between different microservices, linking them to their specific methods.

2.3 Change Impact Analysis

Minor changes in projects written in object-oriented languages may have major and global effects. Similar problems arise in microservice systems, made even worse by their distributed nature. Indeed, a change in one microservice may break another distant microservice and cause a ripple effect [6]. The goal of the CIA is to provide feedback on the semantic impact of a set of program changes [15], enabling stakeholders to address unwanted changes caused by the ripple effect.

The first step in CIA is to identify changes within a defined artifact. According to [15], in object-oriented programming, changes can be categorized into various types of atomic changes. These atomic changes can be classified into three main categories: changes made to classes (such as adding or removing methods or fields), modifications to the body of methods, or the behavior of method lookups, and changes that involve the creation or deletion of a class. This categorization can naturally be extended to microservice systems too.

3 Capabilities of SAVAT

We now describe the key capabilities of SAVAT that allow practitioners to assess the impact of changes in microservice systems. The figures presented are based on data obtained from running SAVAT on the well-established Train Ticket benchmark [18].

3.1 Change Impact Analysis in SAVAT

In SAVAT, CIA is performed only on CMs and IMCGs. We do not perform CIA on SDGs since they are subsumed by IMCGs. Performing CIA in SAVAT involves multiple steps. At first, we collect the changed artifacts. A changed artifact is either a method whose bytecode hash differs or an entity with a modified field or field annotation.

After assessing the changes, the CIA begins to examine each change and trace its propagation. This tracing relies on connections defined by the model being used. For example, when tracking changes made to methods, the process follows the caller-callee relationships within the call graph. In this case, it makes sense to perform a type of reverse graph traversal starting from the initially changed method and collect all the methods that are traversed, as each of them may eventually call the initially changed method and therefore might be influenced by its change.

3.2 Links Difference Analysis in SAVAT

Unlike CIA, *links difference analysis* (LDA) operates exclusively with edges. The visualization highlights edges that have been added (shown in green), modified (shown in blue), and deleted (shown in red). This visualization is available only for CMs and SDGs. In SAVAT, users can easily access the LDA visualization using a utility bar located above the visualized graph.

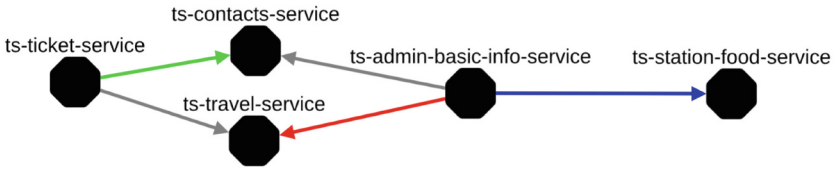

Fig. 1. An example of visualization of the LDA conducted on two different SDGs.

Figure 1 shows an example of what the visualized result of LDA can look like:

- The green edge indicates that a new connection has been added between *ts-ticket-service* and *ts-contacts-service*, which means that at least one REST call was added to *ts-ticket-service* that invokes an endpoint of *ts-contacts-service*.
- The red edge shows that the connection between *ts-adminbasic-info-service* and *ts-travel-service* has been completely removed, meaning that all REST calls from *ts-admin-basic-info-service* that invoke endpoints from *ts-travel-service* were deleted.

– The blue edge indicates a modified connection between *ts-adminbasic-info-service* and *ts-station-food-service*, showing that at least one REST call has been changed between them.

3.3 Antipatterns and Metrics

SAVAT is capable of not only demonstrating the impact of changes introduced in a microservice system, but also of identifying and visualizing various types of *antipatterns* and *metrics*. The antipatterns that SAVAT can visualize include *high coupling* and *cyclic dependencies*. The high coupling antipattern identifies which microservices have a number of connections exceeding a user-defined threshold. The cyclic dependencies antipattern highlights the microservices that are part of a cycle in the visualized graph.

3.4 Dependency Analysis in SAVAT

A rendered SDG can provide key insight into the dependency network drawn from the architecture and how service dependencies evolve over time. The graph structure can be traversed manually. During that, selecting nodes will highlight direct dependencies and provide context on how the services are connected. Since changes to these dependencies may induce breaking changes, our visualization brings attention to modifications made specifically to the rest calls that form the dependencies alongside endpoints targeted by rest calls. In addition to highlighting direct connections, SAVAT provides the ability to visualize the communication graph up to the depth provided. This makes the impact of modifications on downstream dependencies and on the overall system more obvious.

Furthermore, SAVAT provides automated *reachability analysis*, as depicted in Fig. 2. When a user clicks on a given node, the node itself and all transitively reachable nodes are highlighted in yellow.

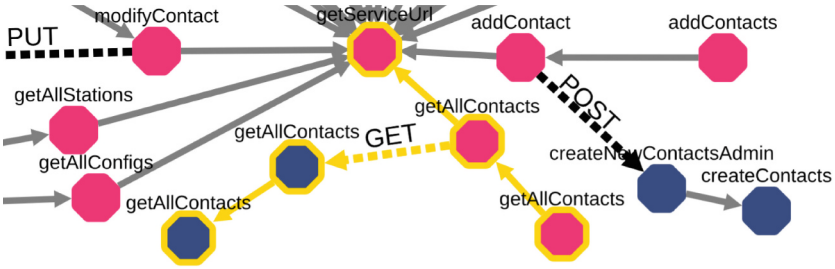

Fig. 2. Highlighted reachability of *getAllContacts* methods.

3.5 Inter-Microservice Call Graph

Visualizing connections between microservices typically offers a broad and holistic perspective. However, to gain deeper insights, it is often more effective to visualize the interactions between individual methods. The real challenge lies in accurately representing the *caller-callee relationships* between the methods located in different microservices. To address this, we propose a more detailed view, called the inter-microservice call graph (IMCG).

In this directed graph, nodes encode individual methods, distinguished by color according to their microservice. The edges denote caller-callee relationships that are either within a single microservice or across microservices via API calls, capturing the precise connections between methods in a way reflecting real inter-service communication.

SAVAT also supports CIA on IMCGs. The resulting graph visually displays nodes with green, blue, or red dotted borders, indicating the type of change that happened: in particular, green is used for addition, blue for modification, and red for deletion. The graph also includes other nodes and edges that depend on the changed node. Figure 3 shows the result of a CIA conducted on two versions of IMCG where all three types of changes are visible:

- *Modification* – indicated by the blue dotted border around the *queryForTravel*, *queryById*, and *queryByIdBatch* methods. The modification of *queryForTravel* affected another method of the same name that serves as an endpoint handler. This, in turn, affected methods in different microservices that invoke the endpoint via a REST call using the POST HTTP method. For example, among the impacted methods are the *preserve* methods.
- *Addition* – indicated by the green dotted border around the *createEvent* methods. The *queryByIdBatch* method, which was modified due to calling the newly added *createEvent* method, in turn impacted its direct caller, the *queryForNameBatch* method.
- *Deletion* – indicated by the red dotted border around the *updatePriceConfig* method. The deletion impacted not only methods within the same microservice, but also methods in other microservices.

This CIA result clearly shows that adding, deleting, or modifying a method can impact many other methods, not only within the same microservice but also across multiple microservices. Another observation is that the modified method *queryById* is not called by any other method.

4 Architecture and Implementation of SAVAT

This section details the architecture of SAVAT in two parts: the first part describes the extraction of preliminary data for visualization, and the second part explains how the visualization process subsequently processes and displays this data.

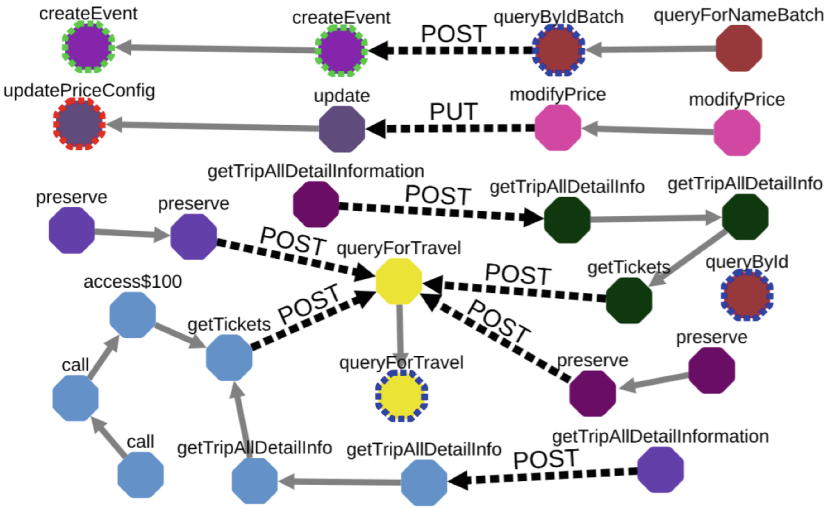

Fig. 3. A visualization of the CIA conducted on two different versions of IMCGs.

4.1 Intermediate Representation

Before starting with visualization, it is necessary to extract relevant data from the analyzed projects and store them in a form that allows efficient automated analysis. SAVAT uses a hierarchical JSON-based intermediate representation (IR) originally introduced in [1]. It is extracted from the source code and describes holistically the structure of the microservice system. It is the foundation for visualizing how the system evolves by tracking how the source code changes over time. It can be broken up into code components, which are source code structures, e.g. method declarations or method calls, and an additional layer of context, which provides information on the microservices present along with metadata on the overall system like the commit id.

The IR is created by iterating over project files and mapping source code to code components. Contextual information like the individual microservices is based on package management files such as Gradle or Maven files. This is done over a series of commits, forming a temporal data set that tracks changes in architecture and implementation over time. The extraction process leverages the JavaParser library that forms AST structures from source code based on the Java language syntax (https://javaparser.org). The JGit library (https://github.com/eclipse-jgit/jgit) is used to interface with Git version control within Java, to locally clone projects, and to iterate over their version history.

4.2 Visualization

The related tools for visualizing the SAR data, which we present below, use libraries that render the data in a 3D space. In contrast, SAVAT uses a different library called Cytoscape.js, which focuses on graph visualization in a 2D space [16]. This library provides a variety of layout algorithms, of which SAVAT specifically uses the fCoSE

layout algorithm [2]. This algorithm effectively calculates the positions of the nodes and provides various attributes that allow further customization of the layout structure.

Each input or analysis output graph is managed through its own Cytoscape instance, which contains information about: (1) Graph elements objects (nodes and edges) with their metadata. (2) The container component in which the graph is visualized. (3) Styles that define colors, shapes, and labels for graph elements. (4) Layout options, including the layout algorithm and its attributes. These instances facilitate effective and customizable visualizations of graphs. They are then used in React.js (https://react.dev/) components with embedded data and a selected layout algorithm.

SAVAT utilizes several modern tools to create an intuitive user interface (UI). This user interface facilitates graph data visualization and offers users robust data management options, including the ability to add new graph data sources, remove existing ones, filter the data, and perform analyses.

To implement the described functionality, SAVAT uses several helper tools. Axios (https://axios-http.com/docs/intro) is a promise-based HTTP client that allows SAVAT to make calls to the back-end REST API. Shadcn (https://ui.shadcn.com/docs) is a library of UI components that provides various ready-to-use components. Tanstack Query (https://tanstack.com/query/latest/docs/framework/react/overview) is a library that facilitates asynchronous fetching, caching, and the creation of state or mutation hooks for making REST API calls to the back-end.

5 Related Works

Microvision [4] is a static analysis tool that brings visualization and observability to microservice architectures by using augmented reality. It leverages augmented reality to make viewing densely packed systems easier to manage. This seeks to address a well-known concern in visualizing microservice systems: as the system expands, it degrades visibility and makes navigating the system hard. Microvision functions by extracting a model of parsed source code and then, through SAR, in multiple steps of extraction, parsing, and manipulation. The visualization can be manipulated and traversed in 3D space and provides pop-up displays of relevant information while navigating the graph.

Harris et al. [10] designed a prototype tool that compares 2D and 3D visualization techniques with static analysis. SAR is used to generate an intermediate model from source code that is visualized by the tool. The visualization features intractable nodes that pop up relevant information and search features to navigate the system. A survey of users revealed that 3D scenes may be hard to traverse due to scale and could unnecessarily prolong observation of the system.

Another tool capable of SAR is described in [12], which employs profiling tactics in order to extract knowledge about a given microservice system.

6 Conclusion and Future Work

Although microservice systems provide significant advantages and have gained traction, managing architectural degradation in these systems has become challenging. With SAVAT, we bridge the gap between what developers do and the consequences of their

changes. SAVAT is capable of providing extensive impact analysis in distributed MSA's by visualizing structural changes to the system.

In future versions of SAVAT, we aim to provide customization of the user interface and plan to incorporate more data into the existing visualization.

Data Availability Statement. SAVAT is open source and is available in the following git repository https://github.com/katyadam/SAVAT. Moreover, all the examples presented are based on the openly available train ticket benchmark, as mentioned above.

References

1. Adams, L., et al.: Evolution and anti-patterns visualized: MicroProspect in microservice architecture. In: Proceeding of the ECSA 2023. LNCS, vol. 14590. Springer (2023)
2. Balci, H., Dogrusoz, U.: fCoSE: a fast compound graph layout algorithm with constraint support. Trans. Vis. Comput. Graph. **28** (2021)
3. Cerny, T., Abdelfattah, A.S., Bushong, V., Al Maruf, A., Taibi, D.: Microservice architecture reconstruction and visualization techniques: a review. In: In: Proceeding of the SOSE 2022. IEEE (2022)
4. Cerny, T., Abdelfattah, A.S., Bushong, V., Al Maruf, A., Taibi, D.: Microvision: static analysis-based approach to visualizing microservices in augmented reality. In: Proceeding of the SOSE 2022. IEEE (2022)
5. Cerny, T., Chy, M., Abdelfattah, A., Soldani, J., Bogner, J., et al.: On Maintainability and microservice dependencies: how do changes propagate? In: CLOSER 2024. Scitepress (2024)
6. Cerny, T., Goulis, G., Abdelfattah, A.S.: Towards change impact analysis in microservices-based system evolution (2025), https://arxiv.org/abs/2501.11778
7. Dragoni, N., et al.: Microservices: yesterday, today, and tomorrow. Springer (2017)
8. Granchelli, G., et al.: MicroART: a software architecture recovery tool for maintaining microservice-based systems. In: Proceeding of the ICSAW 2017. IEEE (2017)
9. Grove, D., DeFouw, G., Dean, J., Chambers, C.: Call graph construction in object-oriented languages. In: Proceeding of the OOPSLA 1997. ACM (1997)
10. Harris, P., Gortney, M., Abdelfattah, A.S., Cerny, T., Rivas, P.: Designing a system-centered view to microservices using service dependency graphs: elaborating on 2D and 3D visualization. In: Proceeding of the ICECCME 2024. IEEE (2024)
11. Hutcheson, R., Blanchard, A., Lambaria, N., Hale, J., Kozak, D., et al.: Software architecture reconstruction for microservice systems using static analysis via GraalVM native image. In: Proceeding of the SANER 2024. IEEE (2024)
12. Nakazawa, R., Ueda, T., Enoki, M., Horii, H.: Visualization tool for designing microservices with the monolith-first approach. In: Proceeding of the VISSOFT 2018. IEEE (2018)
13. Newman, S.: Building Microservices: Designing Fine-Grained Systems. O'Reilly (2021)
14. Pahl, C., Jamshidi, P.: Microservices: a systematic mapping study. In: Proceeding of the CLOSER 2016. ACM (2016)
15. Ryder, B.G., Tip, F.: Change impact analysis for object-oriented programs. In: Proceeding of the PASTE 2001. ACM (2001)
16. Shannon, P., et al.: Cytoscape: a software environment for integrated models of biomolecular interaction networks. Genome Res. **13**(11) (2003)
17. Wimmer, C., et al.: Initialize once, start fast: application initialization at build time. In: Proceeding of the OOPSLA 2019. ACM (2019)
18. Zhou, X., et al.: Benchmarking microservice systems for software engineering research. In: Proceeding of the ICSE 2018. ACM (2018)

Infragenie: Living Software Architecture Diagrams From Docker Compose Files

Ricardo Ferreira[1], Filipe F. Correia[1(✉)], and Paulo G. G. Queiroz[1,2]

[1] INESC TEC, Faculty of Engineering, University of Porto, Porto, Portugal
up200305418@edu.fe.up.pt, filipe.correia@fe.up.pt, pgabriel@ufersa.edu.br
[2] Computing Department, Federal University of the Semi-Arid Region, Mossoró, Brazil

Abstract. Software architecture is reflected across multiple artifacts, making it difficult to communicate without proper documentation, which often becomes outdated or unreliable. We propose an approach to support Living Documentation by generating architectural diagrams from Docker Compose files. We implement our approach as a prototype tool that we name *Infragenie* and conduct an empirical study to show the viability of the approach. The study involved sending questionnaires to maintainers of 378 GitHub repositories. We received 36 responses. *Infragenie*-generated diagrams were rated as better or much better for most of the 12 projects with previous diagrams. Over 70% of the respondents agreed that our approach improved documentation completeness, consistency, and accessibility, and more than 90% recognized its effectiveness in capturing key architectural elements. We conclude that by using Docker Compose files we were able to provide useful architectural diagrams.

Keywords: Software architecture · Living documentation · Docker compose

1 Introduction

The architecture of a software system defines its structure, relationships, and alignment with business goals. Maintaining this alignment during development and maintenance is a persistent challenge. *Good* architecture documentation supports this process, aiding reuse, deployment, and maintenance [4]. It can include semi-structured text (*e.g.*, Architecture Decision Records [9], Pattern Instances [7,11]) and diagrams [2], which enhance communication and foster a shared understanding across development teams. Unfortunately, though, manually created architecture documentation often becomes outdated or hard to maintain as software evolves, especially in projects involving multiple teams, new developers, diverse backgrounds, or insufficient expertise [1]. This challenge is exacerbated by the need for architectural changes driven by business demands, which often favor distributed designs to support scalability across users and

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 12–20, 2026.
https://doi.org/10.1007/978-3-032-04403-7_2

teams [14,16]. Such designs can be realized as service-based applications, where services—often within containers [20]—communicate via lightweight protocols.

Technologies like Docker enable containerization [15], while orchestrators such as Docker Compose define and run multi-container systems [5,19]. Docker Compose uses declarative YAML files, supporting versioning, replication, and deployment in line with *Infrastructure as Code* principles [8]. However, the complexity of these distributed systems can make their architecture harder to understand and document [10]. Traditional practices often fail to keep documentation accurate as distributed systems evolve, making *Living Documentation* increasingly important [13]. This agile approach promotes dynamic, up-to-date documentation that remains accurate, rigorous, and accessible.

Considering that *good* software architecture documentation is complete, consistent, precise, easy to consult, understand, maintain, and update, we postulate that *automatically recovering* architecture documentation can make it substantially better, and the use of *Infrastructure as Code* artifacts provides a little-explored yet viable and valuable approach to achieving *Living Documentation* for software architecture. This work explores this hypothesis by presenting and evaluating a tool to recover the software architecture by analysing Docker Compose files.

While tools exist to generate architecture models and diagrams they can hardly provide a fully accurate view [22], only two prior works, to our knowledge, have attempted to recover software architecture using Docker Compose. Rademacher et al. [18] propose a reconstruction method that relies on specialized modeling languages, requires manual steps, and was evaluated on a single example system. In contrast, our approach is lightweight, fully automated, focused solely on Docker Compose, and empirically evaluated by maintainers across dozens of open source projects. Quéval et al.'s tool [17] also extracts microservice architectures using Docker Compose files among other artifacts, but its focus is architectural analysis rather than documentation, while we concentrate on diagram utility and automation to support living documentation for project maintainers.

2 Approach and Prototype of *Infragenie*

Our approach uses models and model transformations to ensure a reproducible process for generating architectural diagrams. It is data-driven, focuses on IaC files, and produces UML-compliant output. We implemented it as a prototype tool called ***infragenie***, available as a web application at https://infragenie.eu. The tool's operation follows the steps outlined below.

Entrypoint. The user interface of the application's entrypoint is shown in Fig. 1. Users provide the GitHub repository address and branch to analyze.

Gather Architectural Information. Leveraging IaC tools that use filename conventions, we identify and extract information from architecturally-significant files. Different IaC files can capture distinct architectural elements. In this work,

Fig. 1. User interface of *Infragenie* main form.

we focus on *docker-compose.yml* files, which can describe services and containers. The different elements we extract from IaC files are used to construct an integrated view of the software components, based on a defined domain model of the architecture. In specific terms, our current prototype uses the *GitHub* API to scan a repository and gather *docker-compose.yml* files, and is designed for easy extension to other IaC files, such as Kubernetes manifests in future iterations.

Build the Architectural Model. We transform Docker Compose files into a domain model that describes the recovered infrastructure (see Fig. 2). This intermediate model is expressive yet simple, and aligned with standard UML deployment and component diagrams. It supports a systematic model transformation process. In practice, our prototype maps *docker compose* constructs to domain model elements: service definitions become `Entity` instances, *service* names map to *Node* names, *container* names to *Component* names, and service properties to *Component* attributes.

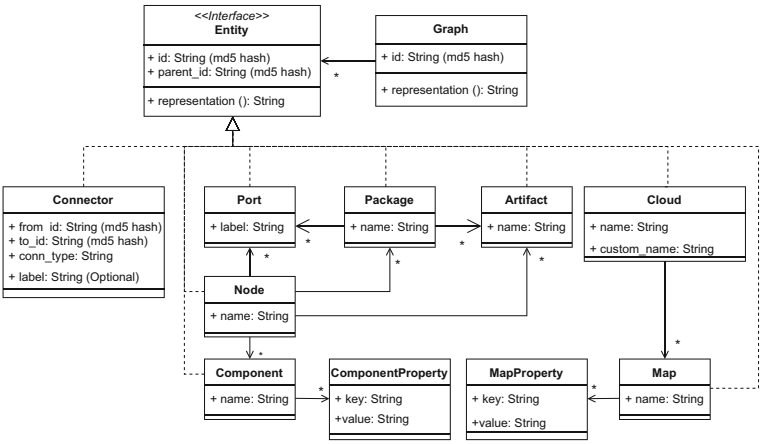

Fig. 2. Architectural domain model. Most elements realizing *Entity* can also be found in UML deployment diagrams. The *Cloud* element allows representing computer networks.

Provide a Visual Representation of the Architecture. Once the domain model is created, we apply a second transformation to generate PlantUML code directly representing the domain model, with software components and infrastructure nodes. We aim to support different types of elements, with the ability to support more in the future or change how they are visually represented. After this step, the tool applies a PlantUML renderer to create the actual diagram (see Fig. 3) and creates a pull request (PR) to add the diagram to the repository, optionally updating the README to show the diagram image there.

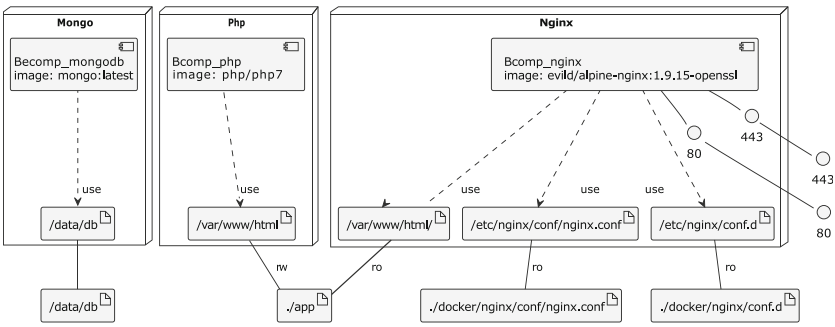

Fig. 3. Example of an architectural diagram generated by *Infragenie*.

3 Research Questions and Methods

Our initial hypothesis (see Sect. 1) leads us to the following research questions.

- RQ1: To what extent does *Infragenie* support better documentation compared to pre-existing documentation?
- RQ2: To what extent does *Infragenie* provide documentation that is complete, consistent, precise, easy to consult, understand, maintain, and update?
- RQ3: To what extent does *Infragenie* provide the most relevant elements of the architecture component view?

To answer these questions, we used *Infragenie* to automatically reconstruct the architecture and generate a diagram for 378 GitHub repositories. We then opened a PR containing the diagram and a link to an online questionnaire inspired by the Technology Acceptance Model [21] and designed to capture user perceptions about the tool, as presented below.

Sampling. We used purposive sampling to define relevant projects and random sampling to select a manageable subset [3]. Maintainers of open source GitHub projects were shown a diagram of their repository generated by our tool and invited to complete an online questionnaire. To boost response rates and ensure

project-tool fit, we used GitHub's advanced search to find active repositories (created since 2018, active since Nov 2021) containing a *docker-compose.yml* file. This returned 22,447 repositories, from which we randomly selected 378.

Instruments Design. To later assess sample diversity and provide context for interpreting responses, the questionnaire starts by collecting background data such as *academic background*, *years of professional experience*, *UML knowledge*, and *experience with text-based diagram tools*.

The questionnaire then includes a short introduction of *Infragenie*, which is followed by a few questions. The close-ended questions are mandatory and address user perceptions of diagram quality, using single-choice formats or a five-point Likert scale [12] (*Strongly disagree*, *Disagree*, *Neutral*, *Agree*, *Strongly agree*). The open-ended questions are optional and support additional qualitative insights, providing context to the quantitative results. Questions are grouped into four categories: *a) participant characterization*, *b) Infragenie usefulness*, *c) diagram completeness*, and *d) suggestions and open comments*. We got in touch with project maintainers through pull requests (PRs). Each PR would describe its purpose, and contribute a diagram generated from the repository with *Infragenie*, and include a direct link to the questionnaire.

4 Results

Of the 378 PRs submitted, 197 (52.11%) remained open, 116 (30.68%) were closed, and 65 (17.19%) were merged. We received 36 completed questionnaires in total. Some PRs remained open due to project requirements regarding the PR format, or were closed by maintainers who took them for advertisements.

Participants Characterization. Respondents, from 24 countries, were fairly evenly distributed, with slightly more from China (11%), Germany (8%), and the USA (8%). Most (78%) had over five years of professional experience, indicating diverse backgrounds and technical proficiency likely to value good architectural documentation. We also asked maintainers how much they agreed with the following statements.

Q1: *"I have experience with tools that allow the creation of diagrams from plain text or code (e.g. PlantUML, Mermaid.JS, others)"*
Q2: *"I am familiar with the Unified Modeling Language (UML)."*
Q3: *"I am familiar with UML component diagrams."*
Q4: *"I am familiar with UML deployment diagrams."*

Results show respondents as generally comfortable with UML and text-based diagramming tools (see Fig. 4). Notably, no one selected *Strongly disagree* for any question, indicating at least a basic level of familiarity across the group.

Diagram Comparison. To address RQ1, we asked maintainers whether their project already had an architectural diagram before receiving *Infragenie*'s PR. Those who answered *yes* were then asked to compare it with *Infragenie*'s. Among

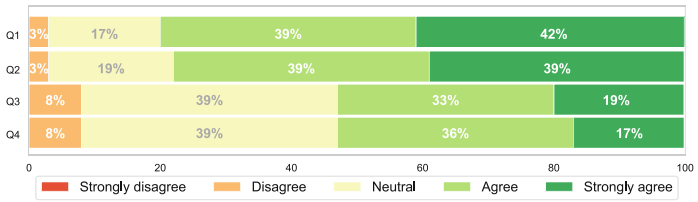

Fig. 4. Respondents knowledge levels.

the 12 (33%) repositories with a prior diagram, the maintainers of 9 of them (78%) found *Infragenie*'s version better or much better. In open responses, many described existing diagrams as outdated, incomplete, or non-compliant with standards. One noted their existing documentation was *"not comprehensive and is old"*, showing how manual or non-co-evolving tools often lead to obsolescence. Participants also valued the detail and clarity of *Infragenie*'s diagrams, citing improvements such as the explicit depiction of service properties and clearer port mappings. As one respondent observed, the diagram *"shows more details like properties of services and the ports are easier to follow"*. Others highlighted the advantage of UML compliance, noting that previous diagrams were overly simplistic or lacked standardization.

Documentation Usefulness. To address RQ2, we asked project maintainers to rate their agreement with statements beginning with *"I think that infragenie is useful to help ensure that architectural documentation is..."*:

Q1: *"... complete."*
Q2: *"... consistent."*
Q3: *"... precise."*
Q4: *"... easier to consult."*
Q5: *"... easier to understand."*
Q6: *"... easier to maintain."*
Q7: *"... easier to update."*

Figure 5 shows over 70% of participants *agreed* or *strongly agreed* that *Infragenie* supports completeness, consistency, precision, and ease of use of architectural documentation. Over 50% expressed that it helps with maintenance and updates.

Documentation completeness. To help us answer RQ3 we asked project maintainers the extent to which they agreed with statements starting with the phrase *"I think that the diagram provided by infragenie shows..."*:

Q1: *"... a complete view of the architecture."*
Q2: *"... the most relevant elements of the architecture."*

The results (see Fig. 6) show that over 60% of the respondents *agreed* or *strongly agreed* that *Infragenie* offered a complete architectural view, and over 90% *agreed* or *strongly agreed* it provided the most relevant elements.

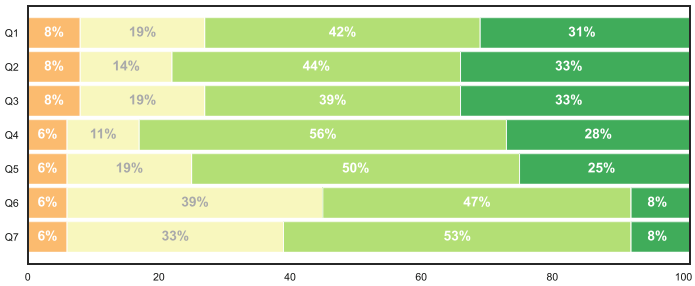

Fig. 5. Perceptions of the quality of the diagrams generated by *Infragenie*.

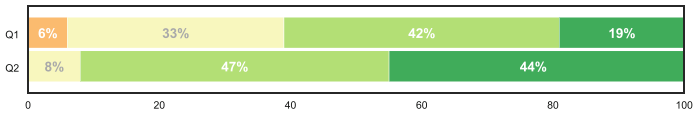

Fig. 6. Perceived completeness of the diagrams generated by *Infragenie*.

5 Discussion and Conclusion

Our results show that *Infragenie* significantly enhances architectural documentation. Namely, 78% of participants rated its diagrams as better or much better than existing documentation (RQ1). Over 70% of respondents agreed that *Infragenie* improves documentation in terms of completeness, consistency, precision, and ease of consultation and understanding (RQ2). Regarding completeness and relevance, most participants felt the diagrams provided a comprehensive architectural overview and clearly highlighted key components (RQ3).

Responses to open-ended questions further supported the notion that automatically generating UML-compliant diagrams from Docker Compose files leads to more accurate and up-to-date architectural views. This reinforces our hypothesis that Docker Compose files are a valuable source for recovering architectural information. They offer a declarative blueprint of a system's runtime composition, effectively capturing the primary services, their deployment configurations, and interdependencies (*e.g.*, through networks, shared volumes, or service links). These services typically correspond to bounded deployment units, often mapping to a microservice, databases, or a message broker.

Although *Infragenie* currently supports only Docker Compose files, it was designed for extensibility. Future work will focus on expanding support to additional IaC artifacts, particularly configuration files for other container orchestration tools (*e.g.*, Kubernetes) and infrastructure provisioning tools (*e.g.*, Terraform, CloudFormation). Additionally, we want to investigate the extent to which large Docker Compose files might affect the readability and utility of generated diagrams when the number of services and interconnections significantly increases, and explore strategies to maintain diagram clarity in such scenarios.

The use of a widely recognized visual language such as UML may have positively influenced participants' perceptions. Familiar notations can improve the readability and accessibility of architectural diagrams. However, UML's expressiveness may prove limiting in the future, especially when capturing dynamic behaviors or concepts not originally addressed by standard UML. In such cases, we may need extensions or complementary notations.

Acknowledgments. This work is co-financed by Component 5—Capitalization and Business Innovation, integrated in the Resilience Dimension of the Recovery and Resilience Plan within the scope of the Recovery and Resilience Mechanism of the European Union, for the period 2021–2026, within project HfPT, with reference 41.

Data Availability Statement. The artifacts we generated, including the questionnaire, the raw data of responses, and the python scripts to process such data, are available and described as a Zenodo record with the DOI 10.5281/zenodo.15496366 [6].

References

1. Aghajani, E., Nagy, C., Lucero, O., Márquez, V., Vásquez, M., Moreno, L., Bavota, G., Lanza, M.: Software documentation issues unveiled. 41st International Conference on Software Engineering (ICSE) (2019)
2. Anderson, M., Meyer, B., Olivier, P.: Diagrammatic representation and reasoning. Springer Science & Business Media (2011)
3. Baltes, S., Ralph, P.: Sampling in software engineering research: A critical review and guidelines. Empir. Softw. Eng. **27**(4), 94 (2022)
4. Chen, J.C., Huang, S.J.: An empirical analysis of the impact of software development problem factors on software maintainability. The Journal of Systems and Software (2009)
5. Eng, K., Hindle, A., Stroulia, E.: Patterns of multi-container composition for service orchestration with docker compose. Empirical Soft. Eng. **29**(3), 65 (2024)
6. Ferreira, R., Correia, F.: SoftwareForHumans/infragenie-experimental-package:v1.2 (May 2025). 10.5281/zenodo.15496366
7. Haitzer, T., Zdun, U.: Semi-automatic architectural pattern identification and documentation using architectural primitives. J. Syst. Softw. **102**, 35–57 (2015)
8. Hüttermann, M.: Infrastructure as code. In: DevOps for Developers, pp. 135–156. Springer (2012)
9. Keeling, M.: Love unrequited: The story of architecture, agile, and how architecture decision records brought them together. IEEE Softw. **39**(4), 90–93 (2022). https://doi.org/10.1109/MS.2022.3166266
10. Kleehaus, M., Matthes, F.: Challenges in documenting microservice-based IT landscape: A survey from an enterprise architecture management perspective. In: 2019 IEEE 23rd International Enterprise Distributed Object Computing Conference (EDOC). pp. 11–20. IEEE (2019)
11. Lemos, F., Correia, F.F., Aguiar, A., Queiroz, P.G.: Live software documentation of design pattern instances. PeerJ Computer Science **10**, e2090 (2024)
12. Likert, R.: A technique for the measurement of attitudes. Archives of Psycology, No. 140, vol 22, pp 5-55 (1932)

13. Martraire, C.: Living Documentation by design, with Domain-Driven Design. Addison-Wesley Professional, First edn. (2019)
14. Matias, T., Correia, F.F., Fritzsch, J., Bogner, J., Ferreira, H.S., Restivo, A.: Determining microservice boundaries: A case study using static and dynamic software analysis. In: 14th European Conference on Software Architecture (ECSA). pp. 315–332. Springer (2020)
15. Merkel, D., et al.: Docker: lightweight linux containers for consistent development and deployment. Linux j **239**(2), 2 (2014)
16. de Oliveira Rosa, T., Guerra, E.M., Correia, F.F., Goldman, A.: Charm–evaluating a model for characterizing service-based architectures. J. Syst. Softw. **206**, 111826 (2023)
17. Quéval, P.J., Zdun, U.: Extracting the architecture of microservices: An approach for explainability and traceability. In: European Conference on Software Architecture. pp. 346–353. Springer (2023)
18. Rademacher, F., Sachweh, S., Zündorf, A.: A modeling method for systematic architecture reconstruction of microservice-based software systems. In: Enterprise, Business-Process and Information Systems Modeling. pp. 311–326. Springer (2020)
19. Reis, D., Piedade, B., Correia, F.F., Dias, J.P., Aguiar, A.: Developing docker and docker-compose specifications: A developers' survey. IEEE Access (2021)
20. Sousa, T.B., Correia, F.F., Ferreira, H.S.: Patterns for software orchestration on the cloud. In: 22nd Conference on Pattern Languages of Programs. pp. 1–12 (2015)
21. Venkatesh, V., Bala, H.: Technology acceptance model 3 and a research agenda on interventions. Decis. Sci. **39**(2), 273–315 (2008)
22. Zhou, Z., Zhi, Q., Morisaki, S., Yamamoto, S.: A systematic literature review on enterprise architecture visualization methodologies. IEEE Access **8**, 96404–96427 (2020). https://doi.org/10.1109/ACCESS.2020.2995850

ServLessSense: Serverless Smell Detection Tool

Hasini Sumalee Perera[1], Zadia Codabux[1(✉)], and Fabio Palomba[2]

[1] University of Saskatchewan, Saskatoon, Canada
hasinis.perera@usask.ca, zadiacodabux@ieee.org
[2] University of Salerno, Salerno, Italy
fpalomba@unisa.it

Abstract. Serverless computing has gained widespread adoption due to its scalability, cost-efficiency, and abstraction of infrastructure management. However, the shift toward event-driven, function-based architectures introduces new code quality challenges and development practices that differ from traditional paradigms. While recent research has identified serverless-specific bad practices commonly referred to as "smells," there remains a lack of automated tools to support their detection and remediation. This paper presents ServLessSense, a tool designed to detect code smells automatically in serverless applications written in JavaScript and TypeScript. Built using a custom ESLint plugin, ServLessSense identifies five serverless-specific smells, provides visualizations through an interactive dashboard, and integrates Large Language Models to offer automated refactoring suggestions. We evaluated the precision and recall of the tool using five open-source serverless applications and conducted a pilot survey study to assess its potential usefulness from the practitioners' perspective. The results indicate that ServLessSense is helpful in detecting serverless-specific smells and generating refactoring suggestions. The survey participants showed an overall favorable perspective towards ServLessSense.

Tool & Data: https://doi.org/10.5281/zenodo.15477162
Demo Video: https://youtu.be/3WDCiqBpQ9c

Keywords: Serverless Computing · Code Smell Detection · Refactoring

1 Introduction

Serverless computing allows developers to build and deploy applications without managing the underlying infrastructure, allowing them to focus on application logic [3]. Function-as-a-Service (FaaS) is the most adopted technology of this model, supported by platforms such as AWS Lambda, Azure Functions, and Google Cloud Functions [5]. As software architecture evolves from monoliths to microservices and serverless functions, organizations are increasingly adopting serverless to improve scalability, maintainability, and adaptability [10].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 21–28, 2026.
https://doi.org/10.1007/978-3-032-04403-7_3

Despite the numerous benefits of serverless computing, it also introduces new challenges, in terms of programming, pricing, scheduling, networking, workflows, caching, and security [8]. Leitner et al. [5] conducted a mixed-method study to investigate the adoption of serverless computing in an industrial setting, identifying common patterns, benefits, and challenges associated with serverless architectures. Taibi et al. [9] identified five bad practices, or smells that affect maintainability, based on interviews with 91 practitioners. While this research provides awareness, it is the developer's responsibility to manually identify and address these smells, which can be overlooked under tight deadlines.

To address the issue of automatically identifying smells in serverless applications, we developed ServLessSense. It automatically detects and visualizes serverless-specific smells and generates refactoring suggestions. ServLessSense implements the serverless-specific smells identified in previous research [9] using pattern matching and a custom ESLint[1] plugin. Our tool analyzes source code, extracts serverless code smells, and visualizes the distribution of smells using a dashboard. Beyond smell detection, ServLessSense integrates OpenAI's[2] Large Language Models (LLMs) to provide automated refactoring suggestions for select smells using prompt engineering. As 72% of serverless projects on GitHub use JavaScript or TypeScript [4], ServLessSense is built for these languages to ensure relevance and applicability to real-world projects.

ServLessSense is the first tool to detect code smells specific to serverless applications. While motivated by similarities with microservices and static analysis tools [2,6,11], our focus is on identifying smells unique to serverless systems. As shown in Fig. 1, ServLessSense architecture is designed around a three-layer workflow. The source code and the OpenAI API key are provided as input to the tool. The analysis layer features a smell detection component, which is explained in further detail in Sect. 2, and a refactoring suggestion generation component, as described in Sect. 3. The output layer is explained in detail in Sect. 4).

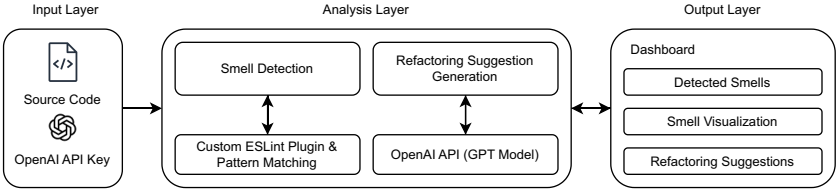

Fig. 1. Methodology

2 Smell Detection

ServLessSense uses a combination of regular expressions and a custom ESLint plugin for smell detection. *Sync & Async calls* used regular expressions and

[1] https://eslint.org/.
[2] https://openai.com/api/.

Abstract Syntax Trees (ASTs) to generate a tree structure that can be traversed for analyzing purposes by parsing source code. The detection algorithms for the other four smells were written using the ESLint plugin. ESLint is a customizable environment where developers use to create their own unique plugin for extracting unique data from the source code [7]. It also uses ASTs for tree traversal. Next, we discuss the detection techniques of each smell.

Sync & Async calls is defined as there is a high usage of synchronous function (functions that execute sequentially) calls instead of asynchronous functions (functions that allow the program to continue executing other tasks while waiting for long-running tasks to complete) [9]. Their detection strategy uses a hybrid approach that combines AST and pattern matching to identify types of functions. It traverses the AST of JavaScript and TypeScript files to explicitly identify functions marked as "async," while simultaneously using regular expression-based patterns to identify synchronous functions and promises. The algorithm then outputs a JSON file with all the functions in the application, labeling them as asynchronous, synchronous, or a promise. In the dashboard, synchronous functions are presented to the user.

Shared Code suggests that there is code shared across multiple functions, making a strong coupling between microservices and tying new releases together [9]. The detection strategy uses AST traversal to identify and track named function imports across the codebase. By focusing on local module imports (denoted by relative paths), the analysis builds a function-to-file mapping that identifies potential functional dependencies between serverless components. The algorithm first creates a graph that represents how files are shared across files. Then, it looks for functions that are referenced by multiple files, which can indicate instances of *Shared Code* in the source code.

Too Many Libraries indicates that there are libraries imported, but are not being used, or only a small part of a large library is being used [9]. The detection algorithm for *Too Many Libraries* identifies unused imports and over-imports. First, the algorithm identifies all module imports through AST traversal while building maps to track import bindings. Then, each identifier and member expression in the code is checked against the import maps to determine which imports are used. Finally, the rule calculates an import efficiency score by comparing the ratio of used exports to total available exports for each module. It applies thresholds to detect modules as over-imported when usage falls below a critical value. We considered a minimum import threshold of 3 and a maximum percentage threshold of 15%. We selected these values to balance trade-offs between performance and the additional code complexity. However, these thresholds can be configured according to the user's needs.

Too Many Technologies suggests that there are technologies not being used [9]. The detection rule builds a dependency usage map by parsing ASTs to identify both explicit and implicit dependency references across the codebase. First, the detection strategy captures explicit dependency usage by analyzing import statements. Next, it uses pattern recognition to identify implicit dependencies by analyzing build configuration files, package.json scripts, and

framework-specific patterns, recognizing that many dependencies may be utilized without direct imports. Finally, it uses regular expressions to identify various dynamic loading techniques, templated imports, and framework-specific component usage. The detection algorithm also considers modern JavaScript ecosystems, handling monorepos, JSX components, TypeScript path aliases, and CSS preprocessors. The algorithm excludes dependencies used by build tools, test frameworks, and browser polyfills based on project structure and configuration files.

Too Many Functions states that there are functions that have duplicate code snippets that are performing the same task [9]. The detection strategy uses a hash-based signature comparison to identify duplicate code fragments within the source code. The strategy uses AST traversal to isolate function declarations, expressions, and arrow functions, while using a multi-stage normalization to ensure semantic equivalence detection despite syntactic variations. First, the strategy extracts function bodies and normalizes them by removing comments and standardizing white spaces to create comparable code representations. Next, these normalized code blocks are processed through the MD5 hash function to create unique signatures, which help detect duplicate code. The detection strategy uses a minimum threshold of 5 lines per function and filters out insignificant functions such as setters, getters, and one-line property assignments. This threshold can be adjusted to the user's preference, and helps focus on significant duplication and avoid functions that often appear similar but are not problematic.

3 Refactoring Suggestion Generation

ServLessSense integrates OpenAI's LLMs to generate automated refactoring suggestions for selected smells. Among the five smells detected by the tool, refactoring support is available for *Sync & Async Calls*, and *Too Many Functions*. Users can select from three versions of OpenAI's Generative Pre-trained Transformer (GPT) model, GPT-3.5, GPT-4, and GPT-4o.

For the *Sync & Async Calls* smell, ServLessSense extracts the function and its corresponding line number, then sends this information to the OpenAI API with a prompt. The model then returns two refactoring suggestions to assist developers in refactoring their code. For the smell *Too Many Functions*, the duplicate functions in the code are sent to the API along with a prompt that instructs the model to suggest approaches for abstraction. In both cases, prompt engineering techniques were applied to compose effective input prompts that maximize the relevance and clarity of the output. Our tool provides self-explanatory and straightforward messages for the remediation of *Too Many Libraries* and *Too Many Technologies* smells. Therefore, refactoring suggestion generation is not required for those two smells. Refactoring suggestion generation is not yet available for *Shared Code* smells, and would require advanced logic and prompt engineering to generate meaningful and context-aware recommendations. This feature will be incorporated in the future. Although LLMs like GPT enhance the

tool's usefulness, human involvement remains vital. Developers must interpret the suggestions and ensure that any changes align with their applications' goals.

4 ServLessSense Dashboard

ServLessSense provides users with an overview of code quality issues through three primary features: smell detection, data visualization, and refactoring suggestions. Together, these features promote both the identification and resolution of serverless-specific smells. ServLessSense supports viewing results by file and smell. In the *view by file* mode, the tool presents the results by file, assigning a severity level of low (1–5 smells), medium (6–10), and high (> 10) per file. Users can search for files by name and apply filters based on severity to narrow down their focus. Users can search for smells by file name and filter by severity. In the *view by smell* mode, users can view all the smells of a specific smell type across the entire codebase.

ServLessSense includes interactive heatmaps that provide intuitive graphical summaries by supporting two visualization modes. The project-level heatmap visualizes the overall distribution and severity of smells across the project, while smell-specific heatmaps show how each of the five smell types is distributed within the codebase. In addition to detecting and visualizing smells, our tool also provides actionable guidance. Users can generate refactoring suggestions in either view mode by selecting a preferred GPT model (GPT-3.5, GPT-4, or GPT-4o). The tool sends the relevant code context and an engineered prompt to OpenAI's API, and the resulting refactoring suggestions are displayed in a pop-up window for developer review.

5 Evaluating the Precision and Recall of the Tool

We validated ServLessSense using five projects selected from a collection of open-source serverless projects[3] on GitHub. Projects were selected based on the criteria that they were implemented using serverless architecture, should utilize JavaScript or TypeScript, and had the highest number of stars. Table 1 summarizes the analysis results, and precision and recall of each smell detection strategy. To validate the accuracy, we manually analyzed the projects to identify instances of all smells except for the *Too Many Functions* smell, as manual detection of duplicate code in a codebase is tedious and impractical. Then, each detected smell instance by the tool was located in the codebase and reviewed to assess its correctness. This verification process ensured that the tool's detections were consistent with the definitions of the respective smells.

As shown in Table 1, the precision of *Sync & Async Calls*, *Shared Code*, and *Too Many Functions* is 1.00 as the respective algorithms identified all the respective smells in the projects. Only 15 out of 26 *Too Many Libraries* smells detected by the tool were identified as true positives, resulting in a precision of

[3] https://serverlesstalent.com/projects/all.

Table 1. Precision and Recall of Smell Detection Strategies by ServLessSense

Project	Sync. & Async. Calls			Shared Code			Too Many Libraries			Too Many Technologies			Too Many Functions		
	Tot.	Det.	TP	Tot.	Det.	TP	Tot.	Det.	TP	Tot.	Det.	TP	Tot.	Det.	TP
Pipedream[a]	1701	1701	1701	43	43	43	0	2	0	0	0	0	–	10	10
Serverless webpack[b]	836	836	836	4	1	1	13	13	13	0	0	0	–	5	5
Middy[c]	1223	1223	1223	38	38	38	0	0	0	0	0	0	–	96	96
SST[d]	2744	2744	2744	7	7	7	2	7	2	0	0	0	–	25	25
Chromeless[e]	85	85	85	0	0	0	0	4	0	0	0	0	–	0	0
Total	6589	6589	6589	92	89	89	26	26	15	0	0	0	–	136	136
Precision			**1.00**			**1.00**			**0.58**			–			**1.00**
Recall			**1.00**			**0.96**			**1.00**			–			–

Note: Tot. = Total Smells, Det. = Total Detected Smells, TP = True Positives.
[a] https://github.com/PipedreamHQ/pipedream
[b] https://github.com/serverless-heaven/serverless-webpack
[c] https://github.com/middyjs/middy
[d] https://github.com/sst/sst
[e] https://github.com/schickling/chromeless

0.58. Although the detection strategy is complex and accounts for contextual awareness, it still has some flaws when detecting the use of imported libraries in complex code. We did not identify any *Too Many Technologies* smells in the selected projects. The recall of *Sync & Async Calls* and *Too Many Libraries* smells was 1.00 as the tool detected all the respective smells in the project. The recall of *Shared Code* smell was 0.96, as it was not able to detect dynamically shared files.

6 Practitioner Evaluation of the Tool

We conducted a pilot user study using a structured survey to assess the usefulness and usability of ServLessSense. The goal of the pilot study is to test the clarity of the questions, the length of the survey, and the overall design before distributing it to developers of open-source serverless projects. The survey was shared with experienced industry practitioners. It consisted of four main sections as discussed below and included follow-up questions to ensure clarity.

Participant Demographics. The pilot study involved eight participants from diverse roles, which included five software developers, two DevOps engineers, and one technical lead. Their professional experience ranged from 1 to 10 years, and their serverless experience ranged from less than 1 to 10 years. Participants had experience working with AWS Lambda, Azure Functions, Google Cloud Functions, and OpenWhisk, and worked in projects of varying sizes, including small, medium, and large. All had experience with JavaScript or TypeScript, aligning with ServLessSense's language support.

Perception of Using Static Code Analyzers. Five out of the eight participants reported using static code analysis tools. Among those users, four (80%)

were satisfied with their tools, while one (20%) remained neutral. When asked whether their current tools are capable of detecting serverless-specific smells, three (60%) agreed, one (20%) was neutral, and one (20%) disagreed. All participants agreed on the importance of writing clean code for long-term maintainability and overall code quality.

Usefulness of the Serverless Specific Smells Detected by ServLessSense. Seven participants (87.5%) agreed that the smells detected by ServLessSense might lead to potential Technical Debt (compromises made by developers to meet deadlines [1]), while one was neutral. All participants agreed that early detection of these smells is essential to facilitate refactoring. Additionally, seven participants (87.5%) agreed that detecting smells could help them write cleaner code, while one remained neutral. Participants ranked *Sync & Async Calls* and *Shared Code* as the most frequent and severe smells, while *Too Many Technologies* was considered the least frequent and severe.

Evaluation of ServLessSense. A total of seven participants (87.5%) reported that they would find ServLessSense useful in their work and believed it would help them in writing cleaner code; the remaining participant was neutral. Similarly, seven (87.5%) agreed that the tool would support easier code refactoring, while one disagreed. In terms of productivity, six participants (75%) believed ServLessSense would improve their efficiency, while the remaining two (25%) were neutral. Regarding usability, seven participants (88%) found the tool easy to use, and six (75%) indicated they had the necessary resources to use it. Regarding tool adoption, six participants (75%) expressed their intention to use ServLessSense in the future, while the remaining participants remained neutral.

The pilot study results show strong user interest and a positive perspective towards ServLessSense. Participants appreciated the tool's ability to detect serverless-specific smells and found the LLM-based refactoring suggestions to be beneficial. The results demonstrate that practitioners recognize a clear value in automated support for smell detection in serverless applications. In the pilot study, practitioners identified the *Sync & Async Calls* as the most frequent and severe smell, which aligned with findings from the analysis of five projects. Conversely, the *Too Many Functions* was reported as the least frequent and severe smell in both the pilot study and the project analysis, suggesting it may pose a minimal threat to the maintainability of serverless applications.

7 Conclusion and Future Work

We introduced ServLessSense, a tool designed to detect code smells specific to serverless applications, which is the first tool developed to detect serverless smells, to the best of our knowledge. It identifies five empirically validated smells in JavaScript/TypeScript projects and supports LLM-based refactoring suggestions for selected smells. Our method includes defining detection rules, integrating LLM support, and validating the tool on real-world projects. A pilot study with industry practitioners confirmed its usefulness, especially in terms of

maintainability. For future work, we plan to improve detection accuracy, expand refactoring support, and validate our methods on a broader set of projects. We will also conduct a large-scale survey study evaluating the tool.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Brown, N., et al.: Managing technical debt in software-reliant systems. In: Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research, pp. 47–52 (2010)
2. Cerny, T., Abdelfattah, A.S., Bushong, V., Al Maruf, A., Taibi, D.: Microvision: static analysis-based approach to visualizing microservices in augmented reality. In: IEEE International Conference on Service-Oriented System Engineering, pp. 49–58. IEEE (2022)
3. Cinque, M.: Real-time faas: serverless computing for industry 4.0. Serv. Oriented Comput. Appl. **17**(2), 73–75 (2023)
4. Eskandani, N., Salvaneschi, G.: The wonderless dataset for serverless computing. In: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories, pp. 565–569 (2021)
5. Leitner, P., Wittern, E., Spillner, J., Hummer, W.: A mixed-method empirical study of function-as-a-service software development in industrial practice. J. Syst. Softw. **149**, 340–359 (2019)
6. Pigazzini, I., Fontana, F.A., Lenarduzzi, V., Taibi, D.: Towards microservice smells detection. In: Proceedings of the 3rd International Conference on Technical Debt, pp. 92–97 (2020)
7. Saboury, A., Musavi, P., Khomh, F., Antoniol, G.: An empirical study of code smells in javascript projects. In: International Conference on Software Analysis, Evolution and Reengineering, pp. 294–305. IEEE (2017)
8. Shafiei, H., Khonsari, A., Mousavi, P.: Serverless computing: a survey of opportunities, challenges, and applications. ACM Comput. Surv. **54**(11s), 1–32 (2022)
9. Taibi, D., Kehoe, B., Poccia, D.: Serverless: from bad practices to good solutions. In: IEEE International Conference on Service-Oriented System Engineering, pp. 85–92. IEEE (2022)
10. Taibi, D., Lenarduzzi, V., Pahl, C.: Processes, motivations and issues for migrating to microservices architectures: an empirical investigation. IEEE Cloud Comput. **4** (2017)
11. Walker, A., Das, D., Cerny, T.: Automated code-smell detection in microservices through static analysis: a case study. Appl. Sci. **10**(21), 7800 (2020)

CUBEX: A CubeSat Exemplar for Teaching Software Architecture Principles

Angelos Motsios[1(✉)], Timo Kehrer[2], and Christos Tsigkanos[1,2]

[1] Space Software Group, University of Athens, Athens, Greece
aggelosmots@di.uoa.gr
[2] Software Engineering Group, University of Bern, Bern, Switzerland

Abstract. Teaching software architecture effectively requires bridging the gap between theoretical concepts and practical application, particularly in complex domains like software for aerospace systems. This paper presents CUBEX, an exemplar developed for teaching software architecture to master's level university students. The project utilizes the JPL's F′ flight software framework within the context of a simulated CubeSat mission focused on orientation monitoring. Students are tasked with developing key flight software components, specifically an Inertial Measurement Unit driver and a Payload processing component, adhering to specified mission requirements. Through this process, students gain practical experience with core software architecture principles, including component-based design, interface definition, telemetry and event handling, and system integration. We detail the project's technical foundations, pedagogical structure, and its value as a readily available, reusable educational artifact designed to facilitate teaching of contemporary software architecture principles.

Artifact: software.aerospace.uoa.gr/cubex
Video: software.aerospace.uoa.gr/cubex/overview.mp4

Keywords: Flight Software Architectures · Architecture Education

1 Introduction

Software architecture stands as a critical discipline within software engineering, dictating the fundamental structure, interactions, and properties of complex systems. Educating future software architects [10] –particularly at the master's level – necessitates pedagogical approaches that move beyond abstract lectures towards tangible, hands-on experiences [1]. Project-based learning, especially when grounded in realistic domains like aerospace flight software (FSW), offers a powerful mechanism for students to grapple with architectural trade-offs, design patterns, and implementation challenges. However, educators often face a scarcity of well-structured and accessible exemplars that utilize modern frameworks representative of industry practice; the significant effort required to create comprehensive teaching material, including setup guides, tutorials, and realistic requirements, often presents a barrier. Architecture education [7] often struggles to bridge the gap between theoretical concepts and practical application.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 29–37, 2026.
https://doi.org/10.1007/978-3-032-04403-7_4

A real-world domain highlighting that a practical skillset is instead required is the engineering of flight software for on-board space systems, which involves a constellation of complex architectural challenges [6]. Those include constraints imposed by limited computational resources, the imperative for high reliability as well as intricate operational demands of particular missions; sophisticated and well thought-out architectural designs are desired [5]. Our overall objective being teaching advanced software engineering concepts, in line with notable efforts in the field [9]. To this end, this paper introduces the CUBEX exemplar, designed to address this need from the perspective of architecture education. It serves as a laboratory exercise project for master's students, focusing on teaching architecture fundamentals within a simulated context. The core task involves building FSW for a simulated CubeSat mission, monitoring the satellite's orientation using real-time accelerometer data.

CUBEX leverages F′ (F Prime [2]), a component-based software framework developed at the Jet Propulsion Laboratory (JPL) and increasingly adopted for small satellite missions and robotics – and famously on the Mars Helicopter. F′ provides a structured environment that inherently promotes key architectural principles like modularity, well-defined interfaces, and separation of concerns, making it an excellent vehicle for teaching software architecture concepts. The exercise context in a space mission adds a layer of realism and relevance for students. The contribution of this paper is the presentation of CUBEX as an educational tool for the software architecture community. It details the project's technical architecture, the specific software architecture concepts inehrent in it, the pedagogical approach embodied in its structured tutorials and requirements, and its availability as an open resource. The subsequent sections cover background on FSW development and the context of the exemplar (Sect. 2), a detailed description of the architecture, requirements (Sect. 3), and how it teaches specific concepts with an overview of the pedagogical structure and learning experience (Sect. 4), and concluding remarks (Sect. 5).

2 Background: FSW Development and Exemplar Context

Understanding the context of CUBEX requires familiarity with both the F′ framework and the specific CubeSat mission scenario employed.

2.1 The JPL F′ Flight Software Framework

F′ is an open-source framework designed for the rapid development and deployment of flight software systems. Originating from NASA/JPL, it rigorously follows a component-based architecture. Its key tenets, in brief, entail:

- **Components:** The fundamental units of design and modularity in F′, components are analogous to C++ classes – they encapsulate specific functionalities and interact with each other through well-defined interfaces.
- **Ports:** Typed interfaces used by components for communication, defining the data types and directionality of interactions.

- **Topology:** A model representing the overall system architecture, specifying component instances and the connections between their ports.
- **Commands, Telemetry, Events :** Built-in mechanisms for space-ground interaction, data reporting, significant event logging, and configuration management, respectively are key elements foundational in the space context.
- **Modeling and Autocoding:** F′ utilizes a domain-specific modeling language (FPP – F Prime Prime [3]) to define components, ports, types, and topology. This model is then used by autocoding tools to generate significant portions of target C++ code, including serialization, connection logic, etc.

The suitability of F′ for teaching software architecture stems directly from these features. F′ enforces modular design through components, requires explicit interface definition via ports and the FPP topology, promotes separation of concerns (e.g., application logic versus communication), and provides concrete mechanisms for common FSW patterns like telecommand handling and telemetry. The C++ implementation base provides exposure to a language widely used in embedded systems, while the autocoding aspect introduces students to model-driven development practices. Furthermore, its provenance and use in real missions enhances student engagement.

2.2 CubeSat Simulated Mission Scenario

The project is situated within a specific – albeit simplified – CubeSat mission scenario. This context provides a narrative and concrete goals.

Mission Objective: The primary goal entails monitoring a CubeSat's orientation and movement in space using data from an on-board accelerometer. Software must acquire acceleration sensor data along three axes in real-time and transmit it to the ground station. The FSW is required to continuously acquire accelerometer data, compute average acceleration over the last second for each axis, determine the dominant axis of acceleration and provide real-time feedback to ground facilities on the CubeSat's operational state and orientation.

The target CubeSat scenario serves a crucial pedagogical purpose – it transforms abstract architectural concepts into tangible requirements (outlined later in Sect. 4). For instance, "telemetry" becomes specifically accelerometer data, "events" become dominant axis change notifications, and component interactions are driven by the need to process sensor data and control physical actuators. For the latter, LEDs are used in place of more advanced alternatives used in actual missions – a decision also to enable breadboarding as an initial electronics bootstrapping exercise. The grounding in a relatable mission makes the architectural constructs easier for students to understand and motivates the design choices needed to fulfill the mission objectives, as explicitly captured in requirements.

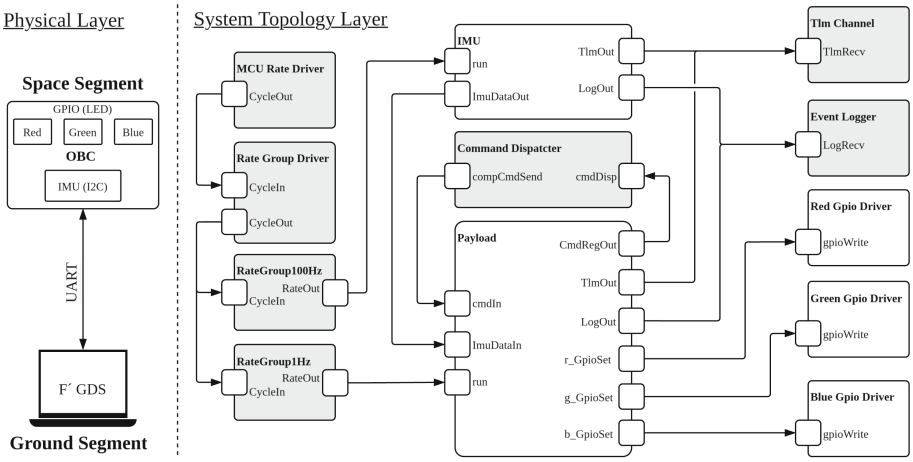

Fig. 1. CUBEX architecture in two layers: (a) Physical Layer demonstrating integration of the IMU sensor with the MCU for data acquisition, with LED outputs for status indication and the Ground Data System. (b) System Topology Layer demonstrating the target architecture. Marked in gray are F′ core framework components.

3 CubeSat Examplar for Teaching Architectural Concepts

CUBEX intends to provide a structured environment and materials for students to learn software architecture principles through practical implementation.

3.1 CUBEX Flight Software Architecture

The central goal for students undertaking the exercise is to develop and integrate specific components of the CubeSat's flight software using the F′ framework. A detailed walkthrough is provided to students, along with drivers and example code that does not address core tenets of the project, to lower the on-boarding overhead of dealing with low-level embedded code such as device drivers.

The overall system architecture consists of multiple interacting F′ components deployed onto a microcontroller platform utilizing a dual-core ARM Cortex-M0+ MCU at 133 MHz, featuring 264 KB of on-chip SRAM, augmented with an MPU6050 accelerometer sensor connected via I2C, and according supporting electronics. MCUs at this class typically serve the capacity of the On-Board Computer (OBC) in a space mission and are responsible for core FSW tasks. Within CUBEX, students primarily focus on understanding the functionality and developing two discrete components:

IMU Component: This component is responsible for interfacing with an Inertial Measurement Unit (IMU) sensor, by providing data regarding the spacecraft's linear acceleration at a specific sampling rate. Students learn to integrate

drivers into the F's build system to achieve communication with the IMU and interpret raw sensor data. Such data will later be processed and transformed into telemetry and subsequently passed through a defined F′ port interface into the next processing component within the FSW architecture.

Payload Component: Building upon the reception of linear acceleration data from the IMU component, the Payload component is specifically designed to first read and then store incoming data. The data for the X, Y, and Z axes is organized and held within a vector structure accessible through the component port. A key student task consists to implement a running average algorithm that operates on these stored acceleration values for each axis over a one-second window with a specific sampling rate. The resulting average acceleration values is then formatted and transmitted as telemetry. Following this, the component analyzes these averaged values to identify the dominant axis of acceleration.

Observe that the division of responsibilities between the IMU and Payload components directly illustrates architectural principles like separation of concerns (hardware interaction vs. business logic) and modularity, while the interaction mode between them exclusively via F′ ports highlights interface-based design.

3.2 Software Requirements

A key element of CUBEX as an educational artifact, is the provision of clear and itemized requirements that need to be satisfied by a design – in line with software requirements in the aerospace domain. These requirements bridge the gap between the high-level mission objectives and the specific functionalities to be implemented in software by student teams. Requirements follow a structured naming convention (Module-Component-Number), hinting at traceability practices as illustrated in Table 1[1].

Table 1. (Fragment of) Software requirements of CUBEX [1].

Requirement ID	Description
Comp-IMU-3	Components::Imu shall be able to produce telemetry and events for the IMU's I2C status.
Comp-IMU-4	Components::Imu shall produce telemetry of accelerometer data at 100Hz.
Comp-Payload-2	Components::Payload shall compute the average acceleration for the X, Y, and Z axes over the last 1 second and produce telemetry.
Comp-Payload-3	Components::Payload shall determine the dominant axis of acceleration based on the computed average values and emit an event.

Observe that these detailed requirements serve as essential *pedagogical scaffolding*. Instead of providing vague instructions, they break down the overall task

[1] The complete requirements can be found in accompanying material.

into smaller, verifiable units of work of increasing difficulty. For example, requirement Component-IMU-3 directly tasks the student with implementing I2C communication, while Comp-IMU-4 specifies the required data rate for telemetry. This structure guides the students' design and implementation efforts, focusing their learning on the specific F′ mechanisms needed to satisfy each requirement and reducing ambiguity (also in evaluation by instructors). It provides clear targets against which students can verify their implementation.

3.3 Illustrating Architecture Concepts

CUBEX is designed as a reusable educational artifact designed to facilitate teaching of contemporary software architecture principles, utilizing a real-world FSW framework. In the following, we highlight key such concepts:

Component Modeling and Interface Definition: The exemplar emphasizes the use of high level modeling (here, in the FPP DSL) as the starting point; students learn to define custom data types such as the Vector array used for 3-axis data, or ports to carry acceleration data. This act of defining types and ports in FPP forces students to think about the component's external contract – its interface – before implementation. The tutorial structure guides students through creating components, defining these elements, updating the build system (CMake), and running the appropriate utility to generate corresponding C++ code. This process highlights the model-driven nature of F′ development and pedagogically reinforces the practice of defining architecture and interfaces first, promoting interface-based design and separation of concerns.

Commands, Telemetry and Events: The concepts of telemetry and event logs are central to FSW and are explicitly treated. Within the exemplar, students define commands, telemetry channels and event logs in the FPP model, including their types, severity levels and format strings. Such data are visualized using the built-in F′ Ground Data System (Fig. 1), demonstrating how architectural choices enable system monitoring and debugging. In F′ commands, telemetry and events are provided and can be seamlessly integrated within a custom component. Command Dispatcher is responsible receiving encoded command packets, decode them and look up the opcode of the command in a table build by component registrations. Event Logger is responsible for storing and filtering events based on their severity level. Telemetry Chanel is responsible for storing telemetry values in a serialized form in a set of telemetry channels in a table.

Hardware Abstraction and Interaction: Development of the IMU component directly addresses hardware abstraction by encapsulating MPU6050 sensor interaction via I2C using an external library. The provided library includes examples that students can examine to understand the expected behavior. This demonstrates how architectural components can isolate hardware dependencies, making the system portable and testable.

Component Logic: Component's runtime behavior is implemented within C++ handlers, making use of features generated from the model files. For

instance, for the Comp-Payload-3 requirement, students have to adopt event methods which will confirm the dominant axis of the IMU. Similarly, Comp-IMU-5 and Comp-Payload-1 (found in accompanying material) target checks of communication between component ports. The explicit connection between the architectural model (in FPP) and the C++ implementation reinforces the benefits of model-driven development and code generation, showing how the framework simplifies implementation by providing hooks consistent with the design.

Topology and Integration: After developing individual components, students learn how to assemble them into a complete system using F's topology mechanism. In this crucial step, students must carefully consider the connections defined in the topology, particularly in relation to requirements Comp-IMU-4, and Comp-Payload series 5 to 7 (found in supplementary material). These requirements dictate the execution rate of the IMU component and the connection of the Payload component with the GPIO component driver. The IMU execution rate, as defined by Comp-IMU-4's 100 Hz execution, is bound to a RateGroup component which determines a specific period at which the components connected will execute. A key highlight is the reusability of the RateGroup component itself. Multiple components, not just the IMU, that require the same execution frequency can be connected to the same RateGroup instance. The RateGroup component takes the Cycle rate from the RateGroupDriver component, which acts as a divider of the hardware's tick rate in order to adjust the triggering of different components, further enhancing reusability by allowing a single RateGroupDriver to serve the timing needs of numerous RateGroup instances operating at various frequencies. Integration shows how independently developed components are composed and how system-wide concerns like timing are managed architecturally, forcing students to consider how components interact to achieve overall mission goals.

4 Pedagogical Structure and Experience

The exemplar is designed not just as a technical exercise but with a clear pedagogical structure aimed at maximizing learning outcomes for master's level students, assuming CS background[2].

Learning Objectives. Upon successful completion of the exercise, students are expected to achieve the following learning objectives:

- *Understand fundamental architectural concepts* of the F′ framework, including components, ports, topology, commands, telemetry, and events.
- Gain practical experience in modeling F′ components using FPP and implementing their behavior in C++.
- *Apply core software architecture principles* such as modularity, interface-based design, and separation of concerns within a realistic project context.

[2] CUBEX has been adopted at the MSc-level "Space Software" course at the University of Athens and is planned for "Dependable Cyber-Physical Systems" within the JMCS MSc program at the University of Bern.

- *Develop an understanding of how software components interact* with hardware peripherals (sensors and actuators) within a structured framework.
- *Learn the importance of requirements* and gain experience in implementing software to meet specific, documented requirements.
- *Become familiar with the development workflow* typical in embedded and flight software, involving modeling, code generation, implementation, system integration and testing.

Mapping Exemplar Features to Architectural Concepts. To provide a clear link between hands-on experience and the underlying architectural knowledge being taught, Table 2 maps specific practical features (or activities) within CUBEX to corresponding, broader theoretical architectural principles and practices that these features illustrate.

Table 2. Mapping features to taught architectural concepts.

Features of exemplar	Teaching architectural concepts
Project Requirements	Requirements Engineering, Requirements Traceability
Defining ports and types	Interface Definition, Custom Data Types, Architectural Modeling (FPP), Build System
Defining telemetry and events	Observability Architectural Modeling (FPP), Hardware Abstraction
Component logic development	C++ implementation, Hardware Interaction, Framework API Usage
Full System Integration	System Assembly, Component Instantiation, Scheduling, Component Reusability

5 Conclusions and Future Work

The paper introduced CUBEX, an exemplar designed for teaching software architecture principles to master's-level university students. The exemplar uses the JPL's F′ [2] framework within a simulated CubeSat mission, where students develop key flight software components, gaining practical experience with core software architecture principles. We plan to integrate more advanced software engineering concepts as elective extensions to the exemplar such as formalization of requirements [4], integrating runtime verification and goal modeling [8], as well as scaffolding for accommodating other payloads such as sensors, motors etc., covering wider architecturally-backed software engineering aspects [6], and reporting on the respective educational experience in depth.

Acknowledgements. Partially supported by HFRI/Greece Project 15706 and SNSF/Switzerland Project 220875.

Data Availability Statement. Supplementary material can be accessed at: software.aerospace.uoa.gr/cubex.

References

1. ACM/IEEE/AAAI joint task force on computer science curricula: computer science curricula 2023, January 2024
2. Bocchino, R., Canham, T., Watney, G., Reder, L., Levison, J.: F prime: an open-source framework for small-scale flight software systems. In: 31st AIAA Conf. (2017)
3. Bocchino, R.L., Levison, J.W., Starch, M.D.: Fpp: a modeling language for f prime. In: 2022 IEEE Aerospace Conference (AERO), pp. 1–15 (2022)
4. Bögli, R., Rohani, A., Studer, T., Tsigkanos, C., Kehrer, T.: Temporal logics meet real-world software requirements: a reality check. In: 13th International Conference on Formal Methods in Software Engineering (FormaliSE) (2025)
5. Dvorak, D.: NASA study on flight software complexity. In: AIAA Infotech@Aerospace. American Institute of Aeronautics and Astronautics (2009)
6. European Cooperation for Space Standardization: ECSS-E-ST-40C: Space Engineering - Software (2009)
7. Kiwelekar, A.W., Wankhede, H.S.: Learning objectives for a course on software architecture. In: Software Architecture (2015)
8. Li, J., Tsigkanos, C., Li, N., Tei, K.: Instrumenting runtime goal monitoring for f' flight software. In: 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), pp. 1300–1309 (2024)
9. Rozier, K.Y.: On teaching applied formal methods in aerospace engineering. In: Formal Methods Teaching. Springer (2019)
10. Vogel, O., Arnold, I., Chughtai, A., Kehrer, T.: Software architecture: a comprehensive framework and guide for practitioners. Springer (2011)

Tutorial

Software Architecture in the Age of Agentic AI

Karthik Vaidhyanathan[1(✉)] and Henry Muccini[2]

[1] SERC, IIIT-Hyderabad, Hyderabad, India
karthik.vaidhyanathan@iiit.ac.in
[2] FrAmeLab, University of L'Aquila, L'Aquila, Italy
henry.muccini@univaq.it

Abstract. The increasing use of generative AI and agent-based systems in software development has led to new challenges and opportunities in software architecture. Two broad areas of research are emerging: i) software architecture (SA) for building AI-enabled systems, including those with LLMs and autonomous agents; and ii) the use of AI techniques, especially generative models, to support architectural activities such as decision-making, documentation, etc. While both directions are gaining interest, contributions are scattered across different research communities including software architecture, AI/ML, and self-adaptive systems.

This half-day tutorial aims to bring together ideas and experiences from both research and practice in a structured way. It is divided into three parts: i) a brief introduction to SA for ML-enabled systems (SAML), ii) LLMs for supporting software architectural processes and practices (LLM4SA), and iii) Architecting LLM-based multi-agent systems (Agentic AI).

Each part will include key terminology, research themes, example systems/demos/case studies, and hands-on/discussion activities. The goal is to help participants understand the current state of research, share practical tools and frameworks, and identify future research directions in this fast-evolving area.

Keywords: Software Architecture · Agentic AI · Generative AI · Large Language Models · Machine Learning · LLM

1 Tutorial Description

The rapid advancements in generative AI—especially large language models (LLMs)—has introduced new possibilities in how software systems are designed, developed, and evolved [6,9,10]. These systems are no longer static or rule-driven but are increasingly dynamic, learning-enabled, and often composed of intelligent components capable of autonomous decision-making. At the same time, the growing levels of abstraction also implies that the role of software architecture as well as architects (as a role) is becoming more critical than ever.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 41–49, 2026.
https://doi.org/10.1007/978-3-032-04403-7_5

Architecting modern AI-enabled software systems is not just about considering traditional quality attributes like performance, maintainabililty etc. but it is also about considering other key quality attributes like observability, uncertainty, explainability, trustworthiness, data eccentricity, evolovability of agentic behaviour, etc. [7].

This shift has led to two parallel and fast-growing areas of interest: (i) software architecture for AI-enabled systems—where traditional architectural methods are being extended to handle the complexities of ML, LLMs, and multi-agent intelligence [8,11]; and (ii) the use of AI, particularly generative models, to support software architecture practice itself—through automation, design assistance, and decision support [1–4]. While there are works emerging in both directions, the research and tooling are fragmented across different communities. Moreover, there is also a need to create awareness in these emerging areas among the software architecture research and practitioner communities.

To this end, this tutorial aims to explore the growing connection between software architecture and AI, with a dual focus: i) On one hand, we look at how to architect systems that include AI components, such as machine learning models, large language models (LLMs), or AI agents. This is about adapting and extending software architecture practices to handle the complexity, adaptability, and unique challenges of AI-enabled systems [11]; ii) On the other hand, we also explore how AI itself can help with software architecture—for example, using generative AI to support architectural decision-making, automate documentation, or guide design trade-offs.

Covered Topics. While both of the directions of using AI for software architecture and Architecting AI-enabled systems are becoming active research areas, they are still developing in parallel, often across different research communities. This tutorial brings them together in a single, structured session that blends research insights with practical examples and hands-on activities.

The tutorial is divided into three connected parts:

Part 1: Software Architecture for ML-enabled Systems (SAML)
In this part, we look at how traditional software architecture needs to change when systems include ML components. Unlike typical software, ML-enabled systems require handling data pipelines, training workflows, inference pipelines, and feedback loops. This part will provide the foundations of software architecture for ML-enabled systems by setting some background and context.
This segment will help participants build a solid foundation in thinking architecturally about ML-enabled systems.

Part 2: LLM for Software Architecture (LLM4SA)
This part focuses on the reverse direction: how GenerativeAI, and specifically LLMs, can assist software architects. With recent advances, LLMs are now being used as co-pilots that help in writing architectural documentation, generating ADRs, analyzing trade-offs, and even recommending patterns. In this part, the tutorial will:

- Introduce terminology such as prompts, context windows, and model chaining.
- Explore practical use cases where LLMs are embedded into architecture workflows.
- Discuss how to evaluate and validate the outputs of LLMs in this context.
- Include a short hands-on session using an open source tool developed by one of the tutorial presenters based on the work done by Dhar et al. [2].

The goal is to help participants see the potential of using AI to support architectural thinking and not just system functionality.

Part 3: Architecting Agentic AI Systems (Agentic AI)
This final part focuses on agentic AI systems—those made up of multiple autonomous AI agents that can plan, reason, and work together. These systems require a different way of thinking about architecture, especially when it comes to coordination, communication, and adaptability. This part of the tutorial will introduce:

- The concept of agentic systems using examples from current research and practice.
- Show how multi-agent systems (MAS) are evolving with the rise of LLMs and orchestration frameworks (like AutoGen, CrewAI, Google ADK, etc.).
- Explain how architecture supports autonomy, memory, message passing, and dynamic decision-making.
- Include a hands-on case study where participants design the architecture of a simple multi-agent system (e.g., for autonomous CloudOps using MOYA [11]).

This part brings together many of the earlier ideas and prepares participants to think systemically about the future of architecting Agentic AI systems.

1.1 Learning Objectives

This tutorial is designed to help participants build both foundational understanding and practical skills at the intersection of software architecture and Generative AI. Using Bloom's Taxonomy as a reference [5], the following learning objectives are addressed:

1. Recall key terminology and concepts related to AI-enabled software systems.
2. Understand how LLMs can act as co-pilots in architecture-related tasks like architecture design decisions.
3. Evaluate the fitness of architectural decisions produced by LLMs for given scenarios.
4. Critically assess the role of LLMs in software architecture practices—considering reliability, reproducibility, explainability, and sustainability.
5. Analyze the architectural implications of including LLMs or agentic AI components within an existing software system.
6. Design a multi-agent architecture for a real-world case study, integrating generative AI components.

1.2 Key Takeaways for the Audience

This tutorial aims to provide both conceptual understanding and hands-on skills. Below are the key takeaways that participants can expect:

- **A clear understanding of the dual role of AI and Software Architecture**
  Participants will gain a solid understanding of both directions:
  - Architecting AI-enabled systems, including ML, LLMs, and agentic systems.
  - Using AI (especially LLMs) to support software architecture practices.
- **Familiarity with ML-specific architectural concepts and patterns**
  Participants will learn about:
  - ML pipeline components and their architectural implications.
  - Architecting multi-agent systems with coordination and autonomy.
- **Hands-on experience with ML-enhanced architecture design**

  Through interactive sessions, participants will:
  - Use an existing LLM based co-pilot to generate architecture design decisions.
  - Design a multi-agent architecture for a real-world case study.
- **A shared vocabulary across SA and LLM communities**

  The tutorial will help participants bridge terminology gaps by introducing:
  - Key terms such as prompts, chaining, Agentic AI, orchestration, memory, agentic AI protocols like MCP, A2A, etc. and tool calling.
  - Concepts that promote interdisciplinary communication.
- **Awareness of current tools, frameworks, and practices**

  Participants will be introduced to:
  - Agent orchestration frameworks like CrewAI, MOYA, Google ADK, etc.
  - LLM-based co-pilot tools for architecture support.
  - References to datasets, benchmarks, and research tools.
- **Exposure to open research questions and challenges**

  The tutorial will highlight:
  - Key gaps in AI-for-software architecture and software architecture-for-AI research.
  - Challenges in using LLMs for software architecture specific tasks.
  - Open problems with respect to architecting agentic AI systems.

1.3 Relevance of Topic for ECSA

More and more industries as well as research communities have started embracing Generative AI as part of their business strategies and software systems.

The topic of this tutorial is highly relevant to ECSA 2025, as it addresses the growing need to understand and evolve software architecture in response to advances in AI—particularly generative and agentic AI. As AI becomes an essential part of modern software systems, software architects are facing new design challenges related to adaptability, coordination, explainability, and system evolution. Furthermore, a substantial share of existing SA knowledge related to design, development, deployment, and evolution can be applied or adapted to Agentic AI systems. The SA community can take advantage of this existing knowledge and identify concrete gaps that require further research and development to support the design of AI-enabled systems. At the same time, AI techniques such as large language models (LLMs) are being used to support architectural tasks, influencing how architecture is practiced in both industry and research. Despite this dual shift, research and experience remain scattered across multiple communities, including software engineering, AI/ML, and software architecture. This tutorial brings these threads together, offering a structured view of the state of the art (in bother research and practice) while identifying open challenges and opportunities for future research. It directly supports the ECSA 2025's focus on impactful architecture and the role of architecting principles in AI-driven systems.

1.4 Intended Target Audience

We expect the tutorial to target several different types of participants:

1. researchers working at the intersection of SA and AI,
2. researchers working on improving architecting practices for AI-enabled systems,
3. researchers working on the adoption of LLMs for SA research,
4. practitioners involved in architecting agentic AI systems,
5. practitioners wanting to explore how to better adopt Generative AI for SA practices.

Concerning the necessary expertise, we believe that any researcher or practitioner with a background in either SA, Generative AI/Machine Learning, or both fields will be able to actively participate in the workshop. The tutorial is structured in a manner to provide adequate background to researchers or practitioners who are also trying to explore this emerging space of software architecture and generative AI.

Considering the increasing use of Generative AI and the rapid evolution of tools/frameworks in this space, the potential interest of the practitioner community, the increasing interest in the research community and the interest displayed by the participants during a working session on "GenAI and Software Architects:

What Skills Will Architects Need in the Age of LLMs?" at ICSA 2025, we expect to have around 35–40 participants in the tutorial.

Tutorial Publicity. We plan to disseminate the Tutorial plan and attract a wider audience by spreading information on the tutorial via:

- the personal SA practitioner and research networks of the tutorial presenters,
- social media groups and channels of SA and ML communities in LinkedIn, Facebook, and Twitter,
- the dedicated social media account of SAML workshop (co-located with ICSA),
- the ECSA, ICSA, ICSE, and ISAPS social networks,
- the list of participants of Dagstuhl Seminar 23302 *Software Architecture and Machine Learning*,

2 Tutorial Implementation

2.1 Duration of the Tutorial

This tutorial is designed as a half-day session with a total duration of 3 h (180 min). It is divided into three main modules, each focusing on a key aspect of the intersection between software architecture and AI: (i) a brief introduction to Software Architecture for Machine Learning (SAML), (ii) Large Language Models for Software Architecture (LLM4SA), and (iii) Architecting Agentic AI Systems (Agentic AI). Each module includes a mix of presentation, interactive discussion, and hands-on activities. Discussions with audience, Q&A, hands-on session and case study based examples are integrated to maintain engagement and allow reflection. Moreover, the participants will also be provided with all the tutorial presentation materials including the slides, source code developed and a repository of agentic AI systems through a dedicated tutorial website.

2.2 Preliminary Schedule

Assuming that the workshop begins at 10 AM, a preliminary schedule is given below:
09:00–09:05: Welcome
09:05–09:25: Session 1, Introducing Software Architecture for ML (SAML)

1. Introduction to basic concepts in ML-enabled systems (10 min)
2. Challenges in architecting AI-enabled systems through a case study (10 min)

09:25–10:00: Session 2, LLMs for Software Architecture (LLM4SA)

1. LLM for Software Architecture (and terminology) (35 min)

10:00–10:30: Coffee break
10:30–11:30: Session 2, LLMs for Software Architecture - continuation (LLM4SA)

1. Demo of a tool, ArchCoPilot (10 min)
2. Discussions with the audience (20 min)
3. Hands-on with ArchCoPilot (20 min)

11:20–12:25: Session 3, Architecting Agentic AI systems

1. Introduction to Agentic AI systems (15 min)
2. Architecting Agentic AI systems (20 min)
3. Case study of Agentic AI framework for autonomous CloudOps (10 min)
4. Hands-on activity for the audience on architecting an agentic AI system (20 min)

12:25–12:30: Tutorial Closing

3 Background of Presenters

The presenters are well-connected in the software architecture community, covering both academia and industry, and have participated in Dagstuhl Seminar 23302 *Software Architecture and Machine Learning*[1], as organizers. Further, the presenters have been the co-organizers of the Software Architecture and Machine Learning Workshop (SAML)[2], which has been co-located with ECSA/ICSA over the last four years.

Karthik Vaidhyanathan is an Assistant Professor at the Software Engineering Research Center, IIIT-Hyderabad, India, where he is also associated with the leadership team of the Smart City Living Lab. He obtained his PhD from the Gran Sasso Science Institute, Italy, and did his postdoc at the University of L'Aquila, Italy. His main research interests lie in the intersection of software architecture and artificial intelligence, specifically focusing on building sustainable and self-adaptive software systems. Over the last two years, he has been working on using Generative AI for improving software architectural practices and further contributed to architecture and development of an open source agentic framework. Karthik also possesses more than five years of industrial experience and serves as a reviewer for various conferences and journals. Karthik is/was also part of the organizing committee of I-Cities 2018, ECSA 2020, SAML 2021, SAML 2022, ICSA 2023, CAIN 2023, CompEd 2023, Dagstuhl Seminar 23302, ICSA 2024, and CAIN 2025. He is also an editorial board member of IEEE Software.

Henry Muccini is a Professor in Computer Science at the University of L'Aquila (Italy). His main research interests are Software Architecture, Model-based/-driven Engineering, and Quality Assurance. He is investigating how ML can be used for better architecting software systems (for supporting architectural self-adaptation, microservices selection and modernization, and green AI), and how to architect ML applications. He is currently highly involved in Agentic AI

[1] https://www.dagstuhl.de/23302.
[2] https://sa-ml.github.io/saml2025.

architectures research. He has been co-organizing the Dagstuhl Seminar 23302, and has been the general co-chair of ICSA 2023, ECSA 2020, I-Cities 2018, and MOBILESoft 2017. He is the SC chair of ICSA, and a member of the ECSA steering committee.

4 Previous Editions

This is the first official edition of this tutorial in its current form. However, the content has been developed and refined over time through various related events/forums. Elements of the tutorial have been presented in invited talks, conferences (such as Conference in AI Engineering - CAIN[3]), workshops (such as SAML workshops), seminars such as the Dagstuhl (23302) seminar on *Software Architecture and Machine Learning* graduate-level course modules, and research group sessions (such as the working session on "What Skills do Architects needs in the age of Generative AI". Moreover, the tutorial will also serve as a bridge between SAML 2025 and the next edition of SAML (SAML 2026) which is being planned to be organized with ICSA 2026.

References

1. Arun, S., Tedla, M., Vaidhyanathan, K.: LLMs for generation of architectural components: an exploratory empirical study in the serverless world. In: 2025 IEEE 22nd International Conference on Software Architecture (ICSA). IEEE (2025)
2. Dhar, R., Vaidhyanathan, K., Varma, V.: Can LLMs generate architectural design decisions? An exploratory empirical study. In: 2024 IEEE 21st International Conference on Software Architecture (ICSA), pp. 79–89. IEEE (2024)
3. Dhar, R., Vaidhyanathan, K., Varma, V.: Leveraging generative AI for architecture knowledge management. In: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), pp. 163–166. IEEE (2024)
4. Esposito, M., et al.: Generative AI for software architecture. applications, trends, challenges, and future directions. arXiv preprint arXiv:2503.13310 (2025)
5. Forehand, M.: Bloom's taxonomy. Emerg. Perspect. Learn. Teach. Technol. **41**(4), 47–56 (2010)
6. Hou, X., et al.: Large language models for software engineering: a systematic literature review. ACM Trans. Softw. Eng. Methodol. **33**(8), 1–79 (2024)
7. Lewis, G.A., Muccini, H., Ozkaya, I., Vaidhyanathan, K., Weiss, R., Zhu, L.: Software architecture and machine learning (Dagstuhl Seminar 23302). Dagstuhl Rep. **13**(7), 166–188 (2024)
8. Liu, Y., et al.: Agent design pattern catalogue: a collection of architectural patterns for foundation model based agents. J. Syst. Softw. **220**, 112278 (2025)
9. Muccini, H., Vaidhyanathan, K.: Software architecture for ML-based systems: what exists and what lies ahead. In: 2021 IEEE/ACM 1st Workshop on AI Engineering-Software Engineering for AI (WAIN), pp. 121–128. IEEE (2021)

[3] https://conf.researchr.org/home/cain-2025.

10. Ozkaya, I.: Application of large language models to software engineering tasks: opportunities, risks, and implications. IEEE Softw. **40**(3), 4–8 (2023)
11. Parthasarathy, K., et al.: Engineering LLM powered multi-agent framework for autonomous CloudOps . In: 2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN), pp. 201–211. IEEE Computer Society, Los Alamitos (2025)

Doctoral Symposium

Requirements-Compliant IT System Deployment Architectures for IoT Systems

Florian Schmalriede(✉)

Carl von Ossietzky Universität, 26129 Oldenburg, Germany
florian.schmalriede@uol.de

Abstract. IT systems in IoT systems support interactions between differently and distributed things to solve application-specific problems. To this end, IT systems must be deployed in IoT landscapes on software and hardware level. Due to the heterogeneous and distributed nature of IoT systems, various IT system deployment architectures are conceivable, which either meet or not application-specific requirements. To develop required deployment architectures, many interrelated decisions must be considered holistically. Thus, this is complex and error prone.

This paper summaries a running PhD project addressing the development of IT system deployment architectures for IoT systems with a process. The process should enable to develop requirements-compliant ones targeted. Acceptance criteria concretize the expected process. The planned evaluation, involving applications along three application examples, is derived. Related work shows, that different disciplines' focus on deployment architectures do not meet the holistic need. The methodology, in which problems regarding the development of deployment architectures are captured in detail and then a solution is derived, is presented. An iterative process, in which deployment architectures are supplemented and evaluated against requirements, outlines a solution idea. Results achieved are reflected and used to motivate discussions about practical relevance and acceptance of the intended process.

Keywords: IoT · systems engineering · deployment architectures

1 Introduction

In many domains, like healthcare, logistics, and agriculture, things, like physical objects, living beings and processes, must be supported in interacting with each other in order to adequately solve application-specific problems. For example, in healthcare, cardiologists must continuously examine patients' cardiac activities regarding atrial fibrillation (AF) over two years to adapt treatments [2]. They must do this economically while maintaining patients' quality of life. Cardiologists manually examine all cardiac activities or restrict patients are not practicable options. Long-term AF detection systems (LTAFDS) that automatically capture cardiac activities, examine them, and provide examination results without

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 53–61, 2026.
https://doi.org/10.1007/978-3-032-04403-7_6

restricting patients [2] are required. In general, systems that capture information about things, process information, communicate information to other things, and influence things in application-specific automated manner are needed.

Distributed and heterogeneous IT systems can support interactions between things as required. To this end, distributed and differently designed compositions of hardware and software, called nodes, cooperate across various communication technologies. In case of LTAFDS, cardiac monitor nodes (CMN) at patients, gateway nodes in patients' houses, server nodes in server farms, and client nodes in cardiologists' offices cooperate across wireless and wired network technologies. Together, such IT systems and things form Internet of Things (IoT) systems in which things are connected via nodes and their networks. For example, LTAFDS, cardiologists, and AF patients form Internet of cardiologists and AF patients (IoCAFP) systems. Such systems automate interactions partially or fully.

1.1 Problem Statement

Before IT systems support interactions of things, deployer must deploy them. IT system providers, customers, and third-parties must integrate instances of them in landscapes and put them into running states. To this end, deployer must compose, parameterize, mount, network, and startup nodes.

Since such IT systems are heterogeneous and distributed, there are various deployment decisions regarding how to compose, parameterize, mount, network and startup nodes. They lead to different functional and non-functional feature values of IT systems and non-functional feature values of deployment processes. This, in turn, leads to different feature values of IoT systems.

Example 1 (CMN Mounting): CMN can either be implanted in patients' chests or attached to patients' chests [2]. With implanted CMN, patients can continue daily activities. However, implantations lead to health risks and require surgeons for longer periods of time. Chest-mounted cardiac monitors restrict patients' everyday activities. However, mounting on chest does not lead to mentionable health risks and can be done by assistants in a short time. Resulting IoCAFP systems differ in maintaining patients' quality of life and patient integration health risks, time, and costs.

Example 2 (CMN Software Composition): CMN could either be composed by firmware that only sends AF affected cardiac activities, or firmware that sends all cardiac activities. In the first case, the firmware must detect AF through calculations, but typically less data must be transferred than in the second case. Depending on the hardware, the communication technology, the algorithms and the frequency of AF events, the cases would lead to different energy demand of CMN [10]. Energy sources would have to be replaced or recharged correspondingly. Especially when CMN are implanted, this would not be justifiable with frequencies less than two years due to health risks. Resulting IoCAFP systems differ in terms of operating effort and, in case of implanted CMN, in terms of patient integration health risks.

Example 3 (CMN Hardware Composition): CMN could be composed by hardware that enables wireless communication with either low or long range. Low range technologies typically requests less energy for transmissions than long range technologies. But, in low range cases additional network infrastructure, like gateway nodes are required. Resulting IoCAFP systems differ in terms of deployment and operating effort and again, in case of implanted CMN, in terms of patient integration health risks.

Beyond this deployment decision examples, there are many other that, alone or in combination, lead to different feature values of IoT systems.

In order to adequately solve application-specific problems, deployment alternatives that meet application-specific requirements are needed. Deployer must deploy IT systems so that resulting IoT system feature values are in line with expected feature values specified by requirements. Since there are various alternatives, due to possible decisions, and not all are requirements-compliant, appropriate ones must be identified. System architects must develop application-specific requirements-compliant IT system deployment architectures for IoT systems.

Since individual and combined deployment decisions affect requirements compliance, a holistic view is necessary. System architects must decide together about how nodes are composed, parameterized, mounted, networked, and started. Together with the variety of individual decisions, this is a complex task which can lead to deployment architectures that are not requirements-compliant. Such IoT systems will not adequately solve application-specific problems.

1.2 Objective

Reducing the complexity of developing requirements-compliant IT system deployment architectures for IoT systems and thus to ensure higher success rates of IoT projects, requires a process to develop such architectures. System architects can apply such a process on application-specific level to develop requirements-compliant deployment architectures. IoT systems, formed by IT systems deployed in line with resulting deployment architectures, meet respective requirements and thus provide adequately solutions to application-specific problems. By answering the following research question, this should be achieved.

How must a process to develop application-specific IT system deployment architectures for IoT systems be designed so that requirements-compliant IT system deployment architectures will result?

2 Expected Results

Acceptance criteria *AC1* to *AC7* concretize the expected result. Section 3 specifies verifications of criteria and impacts of results on research question answers.

The expected result is a process, referred as requirements-driven planning of IT system deployment architectures (RePloID), which system architects can

apply to develop IT system deployment architectures for IoT Systems. To enable system architects to focus on developing such architectures without having to deal with application-specific developing or acquiring processes to develop them, RePloID must be applicable to different application cases. This reduces the complexity of developing such architectures across application cases.

AC1 RePloID must lead to IT system deployment architectures for IoT systems.
AC2 RePloID must be applicable to different application cases.

IoT systems, formed with IT systems deployed in line with resulting deployment architectures, must meet application-specific requirements to be adequate problem solutions. But since system architects develop IT system deployment architectures before deployer deploy IT systems accordingly, it is impossible to make this sure. IoT system feature values can only be predicted with degrees of reliability. Thus manager have to identify and specify minimum degrees of reliability which must be given to consider requirements being met. Deployment architectures are considered to be requirements-compliant when they meet given requirements with corresponding degree of reliability. If, after deployment, it turns out that specified degree of reliability were insufficient, RePloID can be applied again to develop new deployment architectures based on existing ones with adjusted degree of reliability or application-specific information (see *AC7*).

AC3 RePloID must lead to requirements-compliant deployment architectures.

If one or more requirements-compliant deployment architectures exists, then RePloID must lead to at least one of them. It is sufficient to develop at least one of them, as any requirements-compliant one can be selected. If there are not any deployment architectures that is requirements-compliant, RePloID must indicate this. System architects can conclude that specified requirements are not realizable. Given requirements must be adapted before RePloID is applied again.

AC4 When one or more requirements-compliant deployment architectures exists, then RePloID must lead to at least one of them.
AC5 When no requirements-compliant deployment architectures exists, then RePloID must indicate this.

Situations where IT systems are present or not can occur. RePloID must cover both cases so that new requirements-compliant deployment architectures can be developed and established deployment architectures can be transformed into requirements-compliant ones. The latter is necessary when requirements changes or established deployment architectures are not requirements-compliant.

AC6 RePloID must in cases without present IT systems be applicable.
AC7 RePloID must in cases with present IT systems be applicable.

3 Planned Evaluation

In order to determine when the research question is answered, RePloID will be applied to the three application examples long-term AF detection, cold chain monitoring, and plant cultivation. The applications' results must be IT system deployment architectures for IoT systems in order to meet *AC1*. The broad distribution across domains is intended to show that RePloID applicable to different application cases and thus meet *AC2*. The acceptance criteria are met when deployment architectures can be developed using RePloID.

Resulting deployment architectures from applications are verified with application example-specific realizable requirements. If deployment architectures comply with requirements, then the acceptance criteria *AC3* and *AC4* are met.

In order to cover presents of unrealizable requirements, thus are collected for each application example. If RePloID indicates that it is not possible to develop requirements-compliant deployment architectures for these, then *AC5* is met.

Additionally, initial situations with and without present IT systems are considered for each application example. If RePloID is applicable to both cases, the acceptance criteria *AC6* and *AC7* are met.

In total, the compliance with acceptance criteria *AC1* to *AC7* will be verified on twelve different initial situations, formed by combinations of application examples, application example-specific requirements and presence of IT systems. When all acceptance criteria are met for each initial situation, RePloID is accepted as an answer to the research question. However, if it is proven that not all acceptance criteria can be met, then this is also accepted as an answer. An answer is not accepted when neither the first nor the second case is given.

4 Related Work

Due to the heterogeneity at software and hardware level of IT systems in IoT systems and established separations of software and hardware development, the development of such systems is considered from different disciplines, like software engineering and embedded systems. These disciplines focus differently on the development of deployment architectures. Software engineering focuses on software deployment architectures in which decomposable software systems are distributed to complex hardware systems such as cloud and fog nodes [1,4,6]. Embedded systems, like in [5,7], considers hardware and software systems together. However, the deployment architectures here focus on less complex hardware systems, such as sensor and actuator nodes. Experience reports, such as [3,8], show that physical distributions in IoT landscapes is relevant beyond focuses of considered disciplines. Although this is partially covered by software engineering and embedded systems, the focus is not general enough for IoT systems. For the individual disciplines, the respective focus makes sense, as it enables disciplines' objectives to be met. But, there is a lack of holistic considerations when developing IT system deployment architectures for IoT systems.

5 Methodology

The research question is tackled with an engineering methodology. First, the problem is examined in depth through literature research and a physical model of a LTAFDS, with which different deployment architectures are testable, as a running application example. Based on this, a domain model is developed. Using the domain model, the problem is formalized. Starting from the problem, the objective and *AC1* to *AC7* will be concretized by requirements. Existing approaches are analyzed in comparison to specified requirements. Currently, it is not expected that existing approaches will meet the requirements (see Sect. 4), but suitable sub-approaches can be identified. Consequently, they can be considered during the development of a new approach. The new approach is evaluated as described in Sect. 3. Within the evaluation, compliance with captured requirements and thus compliance with *AC1* to *AC7* are verified.

6 RePloID Idea

To capture all deployment decisions and thus IT system deployment architectures (*AC1*), applications of RePloID will lead to deployment plans that schedule deployment activities. For example, deployment plans summarize that during patient integration, surgeons must implant specified CMN in patients' chests, and this only takes place after specified servers are put into operation.

Since initial states and requirements are application-specific and RePloID must be applicable to different application cases (*AC2*), descriptions of these are required. Similar to [1,6,7], models are used for this. State models describe states of deployers, landscapes, things, IT systems (*AC6* and *AC7*), and relationships between them. Requirement models describe which feature values of IoT systems must be met under which conditions and with which degree of reliability. Conditions will include validity time periods, since undermined requirements cannot be realized. As shown later, this also enables to terminate RePloID.

Temporal aspects are addressed with behavioral models that describe behaviors of deployers, things, and nodes and estimation models that estimate IoT system feature values. Starting from initial states, subsequent ones are determined over time and resulting IoT system feature values are estimated. Requirements-compliance is determined by estimations and there degree of reliability (*AC3*).

Deployment plans are explored systematically to address the variety of possibilities. Based on requirements compliance of considered deployment plans, proposal models will propose supplemented ones. These includes deployment plans with more activities than previous ones and assessments about their requirements-compliance potential. Assessments prioritize further considerations.

As shown in Fig. 1, RePloID envisages that, starting from an initial deployment plan that does not include any activities, possible deployment plans are successively considered. Initial states are supplemented with the deployment plan. Supplemented states are used together with behavior models and estimation models to determine IoT system feature values across time periods and

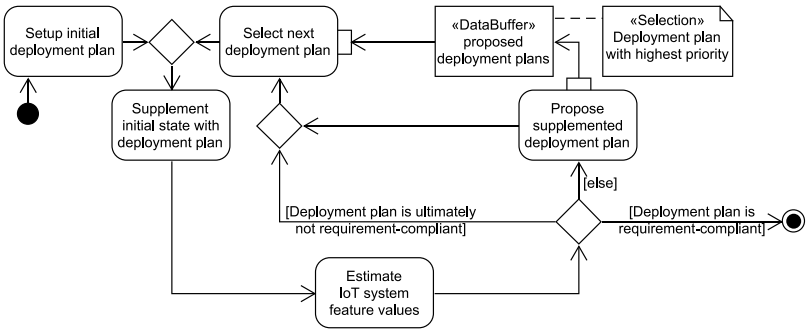

Fig. 1. RePloID deployment plan exploration

features specified by requirements. Estimated IoT system feature values are evaluated against requirements. If requirements are not met and there validity start before deployment activities are done, they are considered as ultimately not met. The deployment plan will not be considered further. If all requirements are met, RePloID ends with the deployment plan as result. Otherwise, results are incorporated into proposal models. This results in proposals of supplemented deployment plans that are used as next ones to be considered according their prioritization in all cases in which deployment plans are not considered further.

Since considering all possible behaviors, estimates, and proposals across all possible states is not realistic due to the variety, comparable to [9], only those that are required are considered. RePloID requests specifications on demand when they are missing. Passed models are supplemented accordingly. Only required specifications are captured and subsequent applications can rely on them.

Since deployment plans are successively supplemented, deployment periods become increasingly longer. Requirements cannot be met after a certain point. Affected deployment plans do not need to be considered further. Together with considered alternative deployment plans from proposals, all potential ones are considered in worst-cases, assuming that all possible proposals are captured. RePloID terminates in worst-cases with the result that no required deployment architecture exists ($AC5$). But, if one requirements-compliant deployment architecture is found, RePloID terminates immediately ($AC4$).

7 Reflection

The intended PhD thesis was discused with three scientific experts from the fields software engineering, distributed systems and embedded systems. This discussion showed that the research topic is scientifically relevant. However, it was not yet discussed with system architects in detail. Assessments from this perspective regarding the practical relevance and acceptance would be useful.

It can be deduced that developments of non-requirements-compliant deployment architectures can lead to failed projects. But, it is not confirmed that this happens in practice. In the best-case, concrete examples can be provided.

Closely linked to the relevance is the acceptance of a holistic approach to develop deployment architectures, as intended here. It needs to be discussed whether such a broad approach is necessary and whether it will be accepted.

Additionally, the practical acceptance of RePloID that is evaluated as intended in Sect. 3 should be adressed. It should be discussed whether the evaluation and acceptance criteria *AC1* to *AC7* are appropriate.

7.1 Current State

The problem was captured in depth. As planned, a physical model of LTAFDS, which enables different deployment architectures, was realized and used to deepen the problem. For example, the impact of different deployment architectures on energy demands of CMN was considered [10]. The LTAFDS will be used as running application example and in the evaluation to reflect long-term AF detection.

Currently, the idea outlined in Sect. 6 is being refined. Steps are being specified in detail and languages to describe required models are being developed. Applications to realized LTAFDS are used to continuously evaluate refinements.

The next steps are to finalize RePloID refinement and to design the application examples cold chain monitoring and plant cultivation in detail.

7.2 Data Availability

Data supporting the findings of this summary are available within the article.

References

1. Brogi, A., Forti, S., Guerrero, C., Lera, I.: How to place your apps in the fog: state of the art and open challenges. Softw. Pract. Exper. **50** (2020)
2. Giancaterino, S., Lupercio, F., Nishimura, M., Hsu, J.C.: Current and future use of insertable cardiac monitors. JACC Clin. Electrophysiol. **4**(11), 1383–1396 (2018)
3. Hartkens, J., et al.: Detecting effects on soil moisture with guerilla sensing. In: Wohlgemuth, V., Kranzlmüller, D., Höb, M. (eds.) ENVIROINFO 2023. Progress in IS, pp. 201–218. Springer, Cham (2024)
4. Jha, D.N., Michalak, P., Wen, Z., Ranjan, R., Watson, P.: Multiobjective deployment of data analysis operations in heterogeneous IoT infrastructure. IEEE Trans. Industr. Inf. **16**(11), 7014–7024 (2020)
5. Koopman, P.: Embedded system design issues (the rest of the story). In: Proceedings International Conference on Computer Design. VLSI in Computers and Processors, ICCD 1996, pp. 310–317. IEEE Comput. Soc. Press
6. Krishna, A., Pallec, M.L., Mateescu, R., Noirie, L., Salaun, G.: IoT composer: composition and deployment of IoT applications. In: 41st International Conference on Software Engineering: Companion Proceedings. IEEE (2019)

7. Kugele, S., Pucea, G., Popa, R., Dieudonne, L., Eckardt, H.: On the deployment problem of embedded systems. In: 2015 ACM/IEEE International Conference on Formal Methods and Models for Codesign (MEMOCODE). IEEE (2015)
8. Nundloll, V., et al.: The design and deployment of an end-to-end IoT infrastructure for the natural environment. Future Internet **11**(6), 129 (2019)
9. Ring, M., Bornebusch, F., Luth, C., Wille, R., Drechsler, R.: Better late than never: verification of embedded systems after deployment. In: Design, Automation and Test in Europe, pp. 890–895. IEEE (2019)
10. Schmalriede, F., Winter, A.: Local energy demand of functionality distributions. In: INFORMATIK 2024. Lecture Notes of Informatik, vol. 352, pp. 1239–1248. Gesellschaft für Informatik e.V. (GI), Bonn (2024)

GreenDevLoop: Power-Aware Software Development with CI/CD, AI, and Feedback

Alessandro Aneggi(✉)

Free University of Bozen-Bolzano, Bolzano, Italy
aaneggi@unibz.it

Abstract. Current energy-aware development approaches overlook architectural inefficiencies. We address this gap with a real-time feedback loop combining power measurement and GenAI analysis in Docker-based CI/CD pipelines. A key novelty is using fine-tuned generative models to detect architectural anti-patterns, validated through empirical studies across software versions and real-world scenarios.

Keywords: Sustainable software · Software architecture · CI/CD pipelines

1 Research Problem

In this section, first, we will introduce the topic and express why this is important. Second, we are going to present some relevant literature. Last, research questions will be defined with a GQM methodology. In 2022, data centres consumed 240–340 TWh—comparable to Italy's national power output IEA [1]. Part of this power is used by software, for this reason software industry is actively working toward carbon neutrality, yet it faces significant challenges due to the lack of actionable guidelines and quality standards for sustainable deployment [2]. For example, data processing activities within this industry contribute to approximately 3.9% of global CO2 emissions [3], underscoring the need for more energy-efficient practices. Also, automatic code generation—concerns are mounting about the sustainability of these tools, as they are not typically optimised for energy efficiency [4]. Engineering sustainable software is inherently complex, involving technical, environmental, social, and economic factors [5]. Yet many developers remain unaware of the energy impact of their design decisions [6], which limits the adoption of sustainability practices. This challenge is even more pressing as software now runs not only in data centres, but across a wide range of devices and embedded systems. Developers, however, often face competing demands such as tight deadlines, performance goals, and functional requirements, which can lead to a lack of awareness about the energy efficiency of their work. As noted by Pang [6], this lack of awareness is a significant barrier to sustainable development practices. By creating an automatic and understandable

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 62–69, 2026.
https://doi.org/10.1007/978-3-032-04403-7_7

feedback loop regarding the energy footprint of software, we can help developers reduce the environmental impact of their creations. Uniquely, this feedback loop aims to automatically identify energy-related architectural anti-patterns using AI—a novel contribution that extends beyond traditional code-level analysis and supports more sustainable design decisions.

1.1 State of the Art

As the state of the art, we consider relevant works from various areas: software architectures, software evolution, the use of GenAI in software engineering, and energy consumption measurement. Below, we provide a first set of works we have taken into consideration so far.

Architectural tactics serve as reusable design strategies for building high-quality software systems; however, their adaptation to energy-aware and sustainable software architecture remains limited due to context-specific trade-offs and insufficient empirical validation. Building on this foundation, this discussion examines how generative AI (GenAI) bridges these gaps by enabling cross-domain knowledge integration for sustainable software design, while also addressing energy-aware maintenance practices, quality assurance, and development processes.

Architectural tactics are effective, reusable solutions for building high-quality complex software systems [7]. Recently, strategies for energy-efficient software have been proposed and studied, particularly in machine learning systems, demonstrating energy consumption reductions ranging from 21% to 99% [8]. However, these tactics are often domain- and context-specific, and may come with trade-offs and side effects [8]. GenAI has the potential to support architectural decision-making by encoding cross-domain knowledge and enabling automated reasoning over sustainable design choices, thus assisting architects in integrating energy efficiency into early-stage architectural models. Nevertheless, there is still limited research on the use of GenAI in the software design and architecture phases [9].

In maintenance and evolution, GenAI models have been successfully applied to tasks like code summarisation, bug fixing, and automatic code reviews [10]. These activities, which involve compiling, testing, and deploying code changes, inherently consume significant energy. GenAI could help reduce the ecological footprint of software by offering energy-efficient code recommendations or identifying unused resources [11]. Recommendations should align with quality goals for sustainable software [12]. The use of GenAI in this area is underexplored and needs further research. Additionally, GenAI models require significant energy, so their savings must outweigh deployment costs.

As the software industry addresses its environmental impact, there is a growing need to integrate sustainability into Software Quality Assurance (SQA). Traditionally, SQA focuses on functional correctness and performance, often overlooking energy consumption. While QA tools are chosen for defect detection, they can be energy-intensive, especially during heavy testing [13]. Recent research has introduced energy-efficient testing frameworks that reduce consumption without

compromising test coverage [14]. GenAIs could automate the creation of energy-efficient test cases, optimise testing, and suggest refactoring opportunities to reduce energy use without impacting functionality. Sustainable CI/CD pipelines that reduce energy use during builds and deployments are a key focus in green software development [15].

Research has explored measuring power consumption at various levels. Zhang 2020 [16] studied virtual machine power consumption, noting the difficulty in mapping software execution to power use, while Warde 2023 [17] proposed methods for measuring container power consumption. Despite these advances, determining the overall environmental footprint remains complex. Protocols for experimentation, such as those by Guldner 2024 [18], have also been proposed. Emerging tools now enable real-time monitoring of energy consumption and carbon footprints, promoting more sustainable development practices [19]. Furthermore, Peng 2024 [20] shows that an improvement in energy consumption is also possible throughout LLMs.

1.2 Research Challenges and Methodological Approach

Despite growing interest in software energy use, many research gaps remain. First, existing power measurement methods are often too complex or impractical for everyday development, which contrasts with the need for developer-friendly tools emphasized in the Energy-Aware Software Manifesto [21].

Also, even though we know a lot about code patterns that hurt performance, **we do not yet understand well which patterns cause high power usage**—and how to detect them in real code. Right now, **most tools that try to help developers save energy are too hard to use or require special setups**, so they do not really fit into the normal workflow of building software. On top of that, **most research looks at energy usage at a single point in time and executions**, without considering how it changes as the software grows or gets updated. Despite progress in energy measurement and code-level optimizations, there is still a gap in identifying energy-related architectural patterns and anti-patterns. Existing approaches are often domain-specific and lack automation. Properly fine-tuned GenAI models could help bridge this gap by learning from architectural and sustainability data, offering insights beyond performance to support energy-aware design decisions.

Finally, there is almost no research on **how to measure the energy used by external systems—like devices or machines**—which are controlled by the software. This broader view of the energy consumption of dependent devices is important, especially in areas where code controls physical machines. Altogether, these gaps show that we need better, more practical methods to measure and reduce energy use in software—methods that work over time, fit into real development practices, and look beyond just the software itself.

To address these gaps, we adopt an empirical approach based on scientific experimentation and measurement theory [22]. We aim to design developer-friendly tools, conduct real-world case studies, and analyse the impact of code and architectural decisions on power consumption.

Having outlined the topic, reviewed relevant literature, formulated the research question and underlined research approach, this paper now turns to a more practical plan and explains the proposed solution. Based on the challenges outlined, in the next section, we define our research questions using the Goal-Question-Metric (GQM) approach.

1.3 Research Questions and Methodologies

Software applications show diverse utilisation patterns, which complicates accurate power consumption estimation. Standalone applications often have stable energy usage, while service-oriented applications, such as web APIs and microservices, are highly dynamic, with consumption varying based on user load and resource access patterns. In such contexts, even small energy improvements can have a significant impact due to high-frequency usage, often outweighing the energy cost of applying GenAI analysis. Therefore, we aim to measure not only real-time power usage—including the utilisation profile—but also the energy consumed by devices connected to and dependent on software behaviour (e.g., actuators, motors, etc.). To do this, we focus on Docker-based environments and analyse entire software versions rather than relying on a static baseline. Instead of comparing against an external reference, we evaluate different versions of the same software to identify regressions or improvements in energy efficiency.

We use the GQM framework [23] to define goals, questions, and metrics for improving sustainability in service-oriented applications.

> **Analyse** the power consumption of microservice applications **for the purpose of** improvement **with respect to** the software development process **from the viewpoint of** software engineers **in the context of** different execution environments, architectures, and programming languages.

We defined the following subquestions with relative example metrics below.

- **Q.1:** How can we measure the power consumption? **Rationale:** We find this an obvious starting point: first, we need to measure the problem, before we try to solve it. **Metrics:** Total Energy Consumption (Joules and Wh), CPU Utilisation (%), etc.
- **Q.2:** How can we identify anti-patterns of source code leading to high energy consumption? **Rationale:** Once the problem is identified (see Q.1), we ask ourselves if a) there are frequent antipatterns leading to high energy consumption and b) how to identify them. **Metrics:** Code Complexity Metrics (Cyclomatic Complexity, Maintainability Index)
- **Q.3:** How can we contribute to helping developers reduce power consumption in their systems and applications? **Rationale:** After identifying frequent misbehavior (see Q.2), we ask ourselves, how these anti-patterns can be removed (also automatically). **Metrics:** Power Savings Ratio, Probability power saving

- **Q.4:** How can the power consumption evolve and change during the lifecycle of the software? **Rationale:** With this question we want to go from a product perspective (see Q.1–Q.3) to a process perspective and see how the problem evolves over time. **Metrics:** Power Consumption Trend (Joules), proxy measures by CPU utilization, memory consumption, hard disk I/O, network traffic
- **Q.5:** How can we measure the power consumption of both software and the external systems it controls? **Rationale:** With this question we want to go from a pure software perspective (see Q.1–Q.4) to a perspective that includes also physical systems *controlled* by the software. **Metrics:** Energy Cost of New Features (Joules per Feature)

We refer to this methodology as the **GreenDevLoop**, a continuous feedback model designed to embed energy-awareness into the software development lifecycle via CI/CD pipelines and GenAI support. This naming facilitates the identification and discussion of our approach in future research and tool development.

2 Proposed Solution

Imagine a developer committing code that triggers a CI/CD pipeline. The pipeline not only runs functional tests but also detects an inefficient loop consuming high CPU power. The system then suggests a low-energy refactoring, automatically proposed by a GenAI model trained on past optimisations. This approach aligns with the principles outlined in the Energy-Aware Software Manifesto [21].

GreenDevLoop is designed to integrate into the DevOps pipeline, particularly during code commits, testing, and monitoring points where energy metrics and developer feedback can most effectively influence behavior. Energy measurements will be embedded into the pipeline using tools like Scaphandre or PowerLetrics. Git data will be used to identify code changes, which are then passed to the GenAI model to evaluate usage and suggest more energy-efficient alternatives. We will fine-tune a general-purpose pretrained LLM on labeled examples of code patterns associated with energy inefficiency. The model will be integrated into the CI/CD pipeline to analyze code changes during commits and testing phases, providing feedback and refactoring suggestions. Validation will include accuracy on known anti-patterns and developer feedback in real-world use.

The following section below outlines the proposed steps necessary to achieve the objective of this research and provides an overview of the expected results.

First, during the **initial phase** of the research, focus on a mapping study to determine which aspects of power consumption can be measured and analysed. This will lead to the development of a proof-of-concept feedback loop, integrated into the CI/CD pipeline, enabling estimation and monitoring of software energy consumption. **Expected results:** Define a developer-friendly methodology to collect and present power consumption data. Success will be measured by usability (via developer surveys) and integration ease into CI/CD workflows.

Second, we will perform an empirical analysis of power consumption across different software versions and workloads. This will involve deploying a measurement framework in real-world settings, providing developers with energy consumption feedback while collecting insights from their experiences. To achieve this, we will extend the PPTAM [24], which currently supports performance testing, by adding energy consumption measurement features. This will enable developers to assess their software's environmental footprint in a short time. **Expected results:** Compare energy use across software versions to identify code segments likely affecting consumption. Effectiveness will be evaluated by the accuracy of detected hotspots and developer validation.

Third, we aim to introduce new functionalities that will identify specific sections of the software responsible for excessive power consumption. With this new point of view, we will identify code which have the same objective from different refactoring, which allows us to limit the energy consumption during the load testing. Moreover, providing actionable feedback to developers will be part of the study and increase the chance to have possibility to test our findings also in industry. **Expected results:** Build a labelled dataset of code versions and energy metrics to detect improvement opportunities and antipatterns. Success is measured by the dataset's precision in predicting high-consumption patterns.

The fourth and final phase will explore power consumption anti-patterns, investigating how GenAI can assist in automatically identifying inefficient code and suggesting refactoring strategies to optimize energy efficiency. The dataset created in the previous phase will serve as a methodology to evaluate GenAI's ability to detect and suggest fixes for anti-patterns. Moreover, in this phase, we plan to explore the possibility of analyzing devices connected to the software, as mentioned in previous sections. **Expected result:** Provide developers with automatic energy-saving suggestions and test the approach in real environments. Measurable outcomes include developer adoption rate and observed power reduction during feedback-loop usage.

By embedding power measurement and analysis within the architectural feedback loops of CI/CD pipelines, this research promotes a shift toward architectures that are performance, scalability, and sustainability aware.

3 Critical Reflection

This research originated from an idea proposed by my supervisor and has since evolved through our shared discussions and my growing personal interest in several interconnected topics. We want to contribute by integrating power measurement tools directly into development workflows. This research aims to bridge that knowledge gap, offering concrete, actionable insights that empower developers to create more energy-efficient software through code refactoring. Power consumption depends not only on code but also on real usage. Modelling this remains a challenge and requires further experimentation.

Nevertheless, a central question we revisit is: **How will we know when the research goal has been achieved?** The core objective of this research is to

evaluate whether the proposed system effectively supports developers in adopting more power-aware development practices. Success will be defined by empirical evidence that such improvements are achievable through the integration of specific tools, architectural decisions, and GenAI support within real-world development workflows.

Validation will be conducted in collaboration with an industrial partner by deploying the system in a live CI/CD environment. We will evaluate its impact through two key aspects: (1) developer feedback, guided by the UTAUT framework to assess perceived usefulness, ease of integration, and behavioral intention, and (2) qualitative analysis of Git commit histories to observe whether energy-aware suggestions were considered over time.

Accurately **identifying energy-related antipatterns and the specific code segments responsible for excessive power consumption is crucial**. Incorporating analysis of the devices interacting with the software can provide deeper insights into both code-level and architectural energy usage. This enables precise localization of energy-intensive components, with effectiveness measured by reduced energy consumption. To this end, we aim to demonstrate the feasibility of these techniques and gather sufficient data to support their effectiveness in real-world scenarios. We envision this work as a foundation for a broader research agenda on sustainable development workflows. We invite fellow researchers to experiment with and extend the GreenDevLoop in different contexts, including mobile systems, IoT, and industrial CI/CD environments.

References

1. Thomas Spencer, S.S.: What the data centre and AI boom could mean for the energy sector. https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks/
2. Noman, H., Mahoto, N., Bhatti, S., Rajab, A., Shaikh, A.: Towards sustainable software systems: a software sustainability analysis framework. Inf. Softw. Techn. **169** (2024)
3. Sanjay Podder, S.K.S., Burden, A., Maruca, R.: How Green Is Your Software? (2020). https://hbr.org/2020/09/how-green-is-your-software
4. Sikand, S., Mehra, R., Sharma, V.S., Kaulgud, V., Podder, S., Burden, A.P.: Do generative AI tools ensure green code? An investigative study. In: Proc. 2nd Int. WS Responsible AI Eng. (2024)
5. Venters, C.C., Capilla, R., Nakagawa, E.Y., Betz, S., Penzenstadler, B., Crick, T., Brooks, I.: Sustainable software engineering: Reflections on advances in research and practice. Inf. Softw. Technol. **164** (2023)
6. Pang, C., Hindle, A., Adams, B., Hassan, A.E.: What do programmers know about software energy consumption? IEEE SW **33**(3) (2016)
7. Bass, L., Clements, P., Kazman, R.: Software Architecture in Practice, 4th edn. (2021)
8. Funke, M., Lago, P., Adenekan, E., Malavolta, I., Heitlager, I.: Experimental evaluation of energy efficiency tactics in industry: results and lessons learned. In: Proceedings of the 47th International Conference on Software Engineering and Software Architecture (ICSA 2024) (2024)

9. Fan, A., et al.: Large language models for software engineering: survey and open problems. In: Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE) (2023)
10. Du, X., et al.: Evaluating large language models in class-level code generation. In: Proceedings IEEE/ACM 46th International Conference on Software Engineering (ICSE) (2024)
11. Procaccianti, G., Fernández, H., Lago, P.: Empirical evaluation of two best practices for energy-efficient software development. J. Syst. Softw. **117** (2016)
12. Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for modern deep learning research. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34 (2020)
13. Hort, M., Grishina, A., Moonen, L.: An exploratory literature study on sharing and energy use of language models for source code. In: 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (2023)
14. Pereira, R., Carçao, T., Couto, M., Cunha, J., Paulo Fernandes, J., Saraiva, J.: SPELLing out energy leaks: aiding developers locate energy inefficient code. J. Syst. Softw. **161** (2020)
15. Bouzenia, I., Pradel, M.: Resource usage and optimization opportunities in workflows of GitHub actions. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE) (2024)
16. Zhang, X., Shen, Z., Xia, B., Liu, Z., Li, Y.: Estimating power consumption of containers and virtual machines in data centers. In: 2020 IEEE International Conference on Cluster Computing (CLUSTER) (2020)
17. Warade, M., Lee, K., Ranaweera, C., Schneider, J.-G.: Monitoring the energy consumption of docker containers. In: 2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC) (2023)
18. Guldner, A., et al.: Development and evaluation of a reference measurement model for assessing the resource and energy efficiency of software products and components—Green Software Measurement Model (GSMM). Future Gener. Comput. Syst. **155** (2024)
19. Brunnert, A.: Green software metrics. In: Companion of the 15th ACM/SPEC International Conference on Performance Engineering (ICPE) (2024)
20. Peng, H., Wan, Y., Fan, L., Jin, Z., Yu, P.S.: Large language models for energy-efficient code: emerging results and future directions. arXiv preprint arXiv:2410.09241, 2024
21. Fonseca, A., Kazman, R., Lago, P.: A manifesto for energy-aware software (2019)
22. Wohlin, C., Runeson, P., Hst, M., Ohlsson, M.C., Regnell, B., Wessln, A.: Experimentation in Software Engineering (2012)
23. Basili, V.R., Caldiera, G., Rombach, D.H.: The Goal Question Metric Approach, vol. 1 (1994)
24. Avritzer, A., et al.: PPTAM: production and performance testing based application monitoring. In: Companion Proceedings 2019 ACM/SPEC International Conference on Performance Engineering (2019)

Architecting Carbon-Aware Software-as-a-Service

Samuele Giussani(✉)

Linnaeus University, Växjö, Sweden
samuele.giussani@lnu.se

Abstract. Software-as-a-Service solutions are increasingly being adopted for delivering software applications, as they are scalable, cost-effective, and facilitate rapid deployment while providing high availability and flexibility. However, ensuring expected performance while tackling carbon emission concerns poses a significant challenge for the sustainable design and operation of software distributed over the cloud. The proposed research outline investigates Software-as-a-Service carbon impact throughout the main software development stages. As a first step, insights on the modeling and simulation of carbon-related metrics were derived from the lifecycle of a software system. The resulting knowledge led to the development of a reasoning framework enabling carbon awareness during the design phase of cloud-based software architectures. Future steps aim to integrate data visualization capabilities into the reasoning framework to understand the importance of the designed application features in the evaluation process and favor enhanced decision support. Furthermore, an adaptation framework is envisioned to handle uncertainty during the software runtime and keep the balance between carbon impact and performance under control. Together, these steps form a cohesive path toward carbon-aware Software-as-a-Service solutions design and operation.

Keywords: Software-as-a-Service · Carbon Awareness · Model-driven Engineering · Data Visualization · Reasoning Framework

1 Introduction

Software-as-a-Service (SaaS) is the most common among the cloud computing paradigms adopted for deploying modern digital services, offering flexibility and scalability while maintaining a lower cost compared to other software deployment options [17]. SaaS allows application software providers to retain control over their runtime environment and physical resources, often delegated further to a Platform-as-a-Service (PaaS) or Infrastructure-as-a-Service (IaaS) supplier, offering their customers the application use through the internet. This deployment solution is suitable for a wide range of domains and optimal for scenarios with highly variable workloads and user demands. In particular, various industrial stakeholders, including small- and medium-sized enterprises (SMEs), benefit from the adoption of the SaaS paradigm thanks to its ease of use, high degree

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 70–77, 2026.
https://doi.org/10.1007/978-3-032-04403-7_8

of security and privacy, and reduced cost for launching a new application on the market compared to self-hosted solutions [8]. Moreover, the existence of a single shared environment for all the application's customers, often referred to as tenants, greatly simplifies the software management operations [7]. However, all the mentioned positive aspects come at a price of increased architectural complexity and introduce challenges in performance [11], design [20], and resource monitoring [13].

Growing awareness of the substantial energy and carbon footprint of cloud data centers hosting SaaS applications has recently intensified research efforts to optimize energy use and carbon emissions alongside other key performance metrics [18]. These sustainability aspects have attracted the interest and the commitment of leading business organizations in a collective effort for the decarbonization of software. As a notable example, Google, Microsoft, and several other big tech companies figure as the steering members of the Green Software Foundation[1], whose goal is to minimize the global carbon emissions associated with software. In particular, the carbon impact of a data center has a twofold nature: *operational emissions* refer to the amount of CO_2 released into the environment to provide electrical energy to the hardware, while *embodied emissions* refer to the portion of carbon produced for hardware manufacturing and transport [22]. It is not sufficient, then, to develop energy-efficient applications to optimize the software's carbon intensity. The source of power coming from the electrical grid represents a fundamental factor in the assessment of operational emissions, making carbon-inefficient those applications that heavily rely on non-renewable energy, regardless of their energy efficiency, and carbon-efficient those which are powered by clean energy, despite possible inefficiencies in the energy usage [19]. Moreover, focusing uniquely on the reduction of operational emissions might not lead to a reduction of the total carbon footprint if the optimization efforts increase the embodied emissions as a drawback [9].

The proposed research project aims to solve the problem of architecting carbon-aware SaaS applications, with the goal of methodically integrating carbon footprint concerns into all stages of their development lifecycle. To this purpose, the following research questions were formulated:

- **RQ1:** How can carbon impact considerations be integrated in the architecture design of SaaS applications?
- **RQ2:** How can carbon-related insights contribute to improving the architecture design and deployment of SaaS applications?
- **RQ3:** How can carbon emissions of a SaaS application remain optimal given the unpredictability of the system's runtime behavior?

2 Related Work

The architectural and engineering solutions found in the literature for designing and developing carbon-aware software applications predominantly consider

[1] https://greensoftware.foundation.

energy consumption as the main optimization target to reduce emissions, overlooking the distinction between operational and embodied emissions [3]. In the domain of SaaS applications, and more broadly considering contributions related to cloud systems, various recent research efforts distinguish carbon efficiency from energy efficiency, primarily aimed at improving the environmental sustainability of the software following its deployment. Existing solutions include the use of scheduling delay techniques for executing low-priority tasks when the power grid's carbon intensity is low [2,16], complementary battery storage charged with renewable energy to be deployed when the grid's intensity is high [1], and algorithms that resize or shut down idle or partially idle resources [10]. These approaches, acting on SaaS applications at runtime, provide only one of the two individuated layers of carbon awareness, lacking the ability to influence embodied emissions.

Most of the existing solutions considering the entire lifecycle of software are high-level guidelines that are not well-suited for specific application domains [3]. For instance, Patel et al. [14] proposed the instantiation of a shared carbon knowledge repository to collect standardized cloud configurations, to be used for informed decisions when deploying new services. This and related efforts need, however, further investigation to be concretely exploited for carbon awareness. The work of Forti et al. [4], instead, introduced a novel framework covering both software design and runtime adaptation strategies focused on reducing carbon emissions. The contribution, conceptually close to the objectives of this research project, addresses the design perspective through code patterns, while this project's objectives plan to employ architectural design models. It also tackles the adaptation challenges through varying strategies depending on the hour of the day, with this project envisioning instead self-adaptation feedback loops.

3 Problem Decomposition and Research Methods

To address the proposed research problem, this work adopts the Design Science Research Method (DSRM) [15]. DSRM is well-suited when the identified problem requires the implementation of one or more artifacts, such as frameworks or models, to be adequately addressed. In particular, purposeful artifacts are obtained by iterating design, implementation and evaluation cycles.

The main research problem was subdivided into four sub-problems, as follows:

- **P-I:** Modern software systems are characterized by an elevated degree of complexity, and carbon impact evaluation, especially in early-stage development, is often challenging.
- **P-II:** Cloud-based SaaS application design seldom includes carbon emissions considerations.
- **P-III:** The influence of design parameters on carbon impact in SaaS applications is often difficult to interpret. This can undermine the trustworthiness and actionable value of evaluation results.

– **P-IV:** SaaS applications' behavior at runtime differs, sometimes greatly, from the expectations gathered during the design-time evaluation, mainly due to the inherent uncertainty of complex systems.

P-I, **P-II**, and **P-IV** were formulated adopting a problem-centered approach, mainly driven by the gaps existing in research. **P-III**, instead, was the result of an objective-centered process aimed at improving the presentation layer and explainability of the carbon footprint assessment performed at design time.

Following the sub-problems formulation, the related sub-objectives were identified:

– **O-I:** Establishing a model-driven procedure to evaluate carbon impact and performance dynamics, forming the foundation of a carbon-aware framework for complex systems.
– **O-II:** Enabling carbon awareness into the design stage of SaaS cloud-based applications by adopting a Reasoning Framework.
– **O-III:** Providing the Reasoning Framework with advanced data visualization capabilities, enabling feature importance analysis of the system's design parameters and supporting more informed decision-making or redesign.
– **O-IV:** Designing an Adaptive Framework that monitors carbon-related metrics at runtime and acts on the system to keep it aligned with the targets identified at design time.

The research objectives are designed to directly address the key challenges identified across the four problem areas. **O-I** envisions the creation of a model-driven procedure to handle the evaluation of carbon impact in consideration of the system's complexity, as stated in **P-I**, making use of the software design models available in the initial development phases. **O-II** aims at establishing a structured Reasoning Framework, reusing the knowledge acquired from the exploration of the previous sub-problem, to offer cloud software developers robust support for carbon intensity evaluation at design time. Through the employment of the Software Carbon Intensity (SCI) indicator, recognized as a standard score metric [12], this goal effectively satisfies the gap identified in **P-II**. **O-III** works towards the integration of a visual analysis tool into the prospective artifact for the previous step, positioning its scope as a support for decision-making stakeholders. Established data visualization techniques contribute to reaching the objectives of interpretability and trustworthiness in the evaluation results included in **P-III**. **O-IV**'s goal is to keep carbon emissions under control in the runtime stage of the software lifecycle, aiming at implementing an adaptive mechanism that reacts to unexpected changes in the system's behavior. This adaptive capability directly addresses **P-IV** by mitigating the effects of the system's inner uncertainty.

4 Proposed Solution and Expected Results

Design and development of artifactual solutions to address the identified objectives represents the next step of the DSRM approach. The implementation of

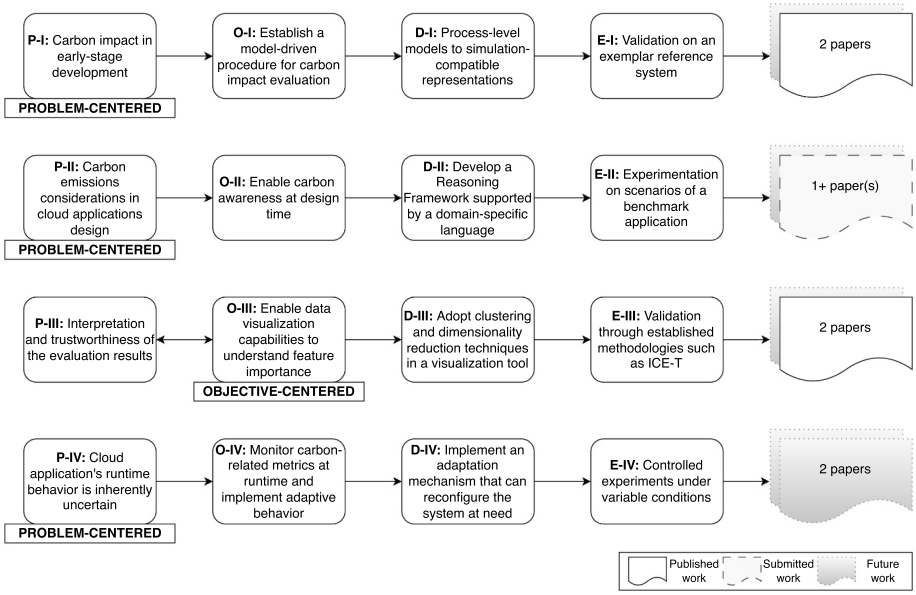

Fig. 1. Outline of the presented research project following the DSRM model

suitable prototypes leads to the demonstration of their applicability in solving the problems they relate to. In particular, the expected results include:

- **D-I:** Implementation of model-driven transformations to derive simulation-compatible representations from process-level models. Application to multiple scenarios of the system lifecycle.
- **D-II:** Development of the fundamental elements of a Reasoning Framework, including a domain-specific language to include carbon-relevant data in the model-driven evaluation.
- **D-III:** Implementation of dimensionality reduction and clustering techniques, combined with intuitive data visualization tools, to present evaluation results effectively.
- **D-IV:** Development of an adaptive mechanism that reacts to deviations from the intended behavior and adjusts the system to realign performance and carbon emissions to the desired values.

Following the demonstration of the artifacts' efficacy, an assessment of how well they support the problem's solution is required. Hence, the following evaluation steps were formulated:

- **E-I:** Assessment of the procedure's ability to capture the carbon emission patterns using a reference system as a validation exemplar.
- **E-II:** Experimentation of the Reasoning Framework across multiple design scenarios of a benchmark cloud application delivered with the SaaS model.

- **E-III:** Application of the developed data visualization on a benchmark application and evaluation through established validation methodologies, like the ICE-T methodology [21].
- **E-IV:** Comparison between static configurations and the implemented adaptive mechanism under different conditions in controlled experiments.

E-I and **E-II** are performed by observing and comparing the produced solution with the set objectives. In particular, **D-I**'s simulation outcomes are tested against the expected carbon emissions of a reference system, while the Reasoning Framework resulting from **D-II** is applied to multiple designs of a benchmark application to ensure consistency in the obtained results. **E-III** and **E-IV**, instead, rely on qualitative and quantitative measurements for their validation, respectively. **D-III**'s effectiveness is inferred through a value-driven methodology, ICE-T, spread across four value components and supported by a dedicated survey data collection process. Controlled experiments for **D-IV** are needed instead to verify whether the adaptive experimental setup manages carbon emissions more optimally than a non-adaptive control setup.

Figure 1 summarizes the outlined research project and includes the planned scientific communication strategy that will be the foundation of the thesis work. The contribution [5] partially addresses **P-I** and further contributions are envisioned to cover its shortcomings, while **P-II** is explored in a manuscript currently under review for publication, with possible follow-up work to come. The paper [6] mainly focuses on the development of the artifact to be used in **D-III**, and a further publication is planned to expand its derived knowledge and solve **P-III**. Finally, **P-IV** is part of future work and is expected to produce a further contribution and a comprehensive article, including the findings of the previous problems in a holistic perspective.

5 Critical Reflection

The presented research project relies on the adoption of multiple established scientific methodologies and requires knowledge from various domains, such as model-driven engineering, data visualization, and self-adaptive systems. The intersection of all these elements exposes several challenges in effectively evaluating carbon impact at all stages of a SaaS software application's lifecycle.

In **P-I**, the problem focus is set to finding a technique that is effective for carbon footprint evaluation at design time, being interested in obtaining structured and reusable knowledge, not restricting the software's domain to cloud systems or SaaS solutions. Thus, the obtained results represent indeed valuable foundational insights, but they are exposed to validity and transferability concerns due to their demonstration not being tied to the cloud domain.

The development of a Reasoning Framework to address **P-II** deals with common problems found in modeling and simulation. In particular, the abstraction of carbon emissions in software design models might lead to oversimplification or distortions in the representation of the system behavior, and the transition to the

simulation domain operated by a model-driven transformation might generate inconsistencies between input and output models.

For **P-III**, instead, the adoption of clustering and dimensionality reduction techniques in a visual analysis tool suffers from a trade-off between explainability and accuracy of the presented data. For instance, reducing the dimensionality of a complex simulation output might be beneficial for lowering visual cluttering and overload in the results presentation and favoring data immediacy, but it also results in a loss of semantic meaning for the obtained new dimensions, making it harder to trace the original feature's contribution in the explored data.

The adaptation framework envisioned for the solution of **P-IV** must be able to monitor the changes in carbon emissions patterns occurring at runtime to properly analyze the available reconfiguration options for the system under analysis and initiate an adaptation cycle. However, most cloud server providers do not offer full access to their data nodes, preventing access to energy consumption monitoring tools, used for deriving operational carbon emissions, installed on their devices. This limitation represents a significant challenge in the framework's development and may require indirect estimations of values not directly inferable from the system, reducing the accuracy and, possibly, the effectiveness of adaptation cycles.

References

1. Acun, B., et al.: Carbon explorer: a holistic framework for designing carbon aware datacenters. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2023, vol. 2, pp. 118–132. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3575693.3575754
2. Claßen, H., Thierfeldt, J., Tochman-Szewc, J., Wiesner, P., Kao, O.: Carbon-awareness in CI/CD. In: Monti, F., et al. (eds.) Service-Oriented Computing - ICSOC 2023 Workshops, pp. 213–224. Springer, Singapore (2024). https://doi.org/10.1007/978-981-97-0989-2_17
3. Danushi, O., Forti, S., Soldani, J.: Carbon-efficient software design and development: a systematic literature review. ACM Comput. Surv. **57**(10) (2025). https://doi.org/10.1145/3728638
4. Forti, S., Soldani, J., Brogi, A.: Carbon-aware software services. In: Pahl, C., Janes, A., Cerny, T., Lenarduzzi, V., Esposito, M. (eds.) Service-Oriented and Cloud Computing, pp. 65–80. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-84617-5_6
5. Giussani, S., Perez-Palacin, D., Caporuscio, M., Edrisi, F.: Business process lifecycle enhancement via digital twin and model-driven engineering. In: 2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C), pp. 300–309 (2025). https://doi.org/10.1109/ICSA-C65153.2025.00050
6. Giussani, S., et al.: Visualizing feature importance of time series data in discrete-event simulations using Shapley additive explanations. In: Proceedings of the 39th ACM SIGSIM Conference on Principles of Advanced Discrete Simulation, pp. 65–69 (2025). https://doi.org/10.1145/3726301.3728401
7. Golding, T.: Building Multi-Tenant SaaS Architectures. O'Reilly (2024)

8. Gupta, P., Seetharaman, A., Raj, J.R.: The usage and adoption of cloud computing by small and medium businesses. Int. J. Inf. Manage. **33**(5), 861–874 (2013)
9. Gupta, U., et al.: Chasing carbon: the elusive environmental footprint of computing. IEEE Micro **42**(4), 37–47 (2022). https://doi.org/10.1109/MM.2022.3163226
10. Huang, N., Li, A., Zhang, S., Zong, Z.: Reducing cloud expenditures and carbon emissions via virtual machine migration and downsizing. In: 2023 IEEE International Performance, Computing, and Communications Conference (IPCCC), pp. 74–81 (2023). https://doi.org/10.1109/IPCCC59175.2023.10253871
11. Ibrahim, A.M.A., Abdullah, N.S., Bahari, M.: Software as a service challenges: a systematic literature review. In: Arai, K. (ed.) Proceedings of the Future Technologies Conference (FTC) 2022, vol. 3, pp. 257–272 (2023)
12. ISO/IEC 21031:2024: Information technology – Software Carbon Intensity (SCI) specification. Standard, International Organization for Standardization, Geneva, CH (2024). https://www.iso.org/standard/86612.html
13. Lee, J., Hur, S.J.: Level 2 SaaS platform and platform management framework. In: 13th International Conference on Advanced Communication Technology (ICACT 2011), pp. 1177–1180 (2011)
14. Patel, P., Gregersen, T., Anderson, T.: An agile pathway towards carbon-aware clouds. SIGENERGY Energy Inform. Rev. **4**(3), 10–17 (2024). https://doi.org/10.1145/3698365.3698368
15. Peffers, K., Tuunanen, T., Rothenberger, M.A., Chatterjee, S.: A design science research methodology for information systems research. J. Manag. Inf. Syst. **24**(3), 45–77 (2007)
16. Radovanović, A., et al.: Carbon-aware computing for datacenters. IEEE Trans. Power Syst. **38**(2), 1270–1280 (2023). https://doi.org/10.1109/TPWRS.2022.3173250
17. Rosati, P., Lynn, T.: Measuring the business value of infrastructure migration to the cloud. In: Lynn, T., Mooney, J.G., Rosati, P., Fox, G. (eds.) Measuring the Business Value of Cloud Computing. PSDBET, pp. 19–37. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-43198-3_2
18. Sharma, P., Fuerst, A.: Accountable carbon footprints and energy profiling for serverless functions. In: Proceedings of the 2024 ACM Symposium on Cloud Computing, SoCC 2024, pp. 522–541. Association for Computing Machinery, New York (2024). https://doi.org/10.1145/3698038.3698531
19. Souza, A., et al.: Ecovisor: a virtual energy system for carbon-efficient applications. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2023, vol. 2, pp. 252–265. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3575693.3575709
20. Vidhyalakshmi, R., Kumar, V.: Design comparison of traditional application and SaaS. In: 2014 International Conference on Computing for Sustainable Global Development, INDIACom 2014, pp. 541–544 (2014)
21. Wall, E., et al.: A heuristic approach to value-driven evaluation of visualizations. IEEE Trans. Visual Comput. Graphics **25**(1), 491–500 (2019). https://doi.org/10.1109/TVCG.2018.2865146
22. Wang, J., Gupta, U., Sriraman, A.: Peeling back the carbon curtain: carbon optimization challenges in cloud computing. In: Proceedings of the 2nd Workshop on Sustainable Computer Systems, HotCarbon 2023. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3604930.3605718

Temporal Network Analysis of Microservice Architectural Degradation

Alexander Bakhtin(✉)

University of Oulu, Oulu, Finland
alexander.bakhtin@oulu.fi

Abstract. Microservice architecture can be modeled as a network of microservices making calls to each other, commonly known as the service dependency graph. Network Science can provide methods to study such networks. In particular, temporal network analysis is a branch of Network Science that analyzes networks evolving with time. In microservice systems, temporal networks can arise if we examine the architecture of the system across releases or monitor a deployed system using tracing.

In this research summary paper, I discuss the challenges in obtaining temporal networks from microservice systems and analyzing them with the temporal network methods. In particular, the most complete temporal network that we could obtain contains 7 time instances and 42 microservices, which limits the potential analysis that could be applied.

Keywords: microservice · architectural degradation · network science · temporal networks

1 Introduction

Microservice architecture is a paradigm promoting the decomposition of a coupled software system into small, independent components, called microservices [24]. Many patterns and anti-patterns were proposed in the literature to improve the design and architecture of microservice systems [13]. Over time, the architecture of a microservice system can decay. Such a phenomenon is known as microservice architectural degradation.

We can model the composition of microservices, i.e., their architecture and their interactions, as networks. Such networks can be studied as static or evolving over time, i.e., so-called *temporal* networks, leveraging techniques and methods from Network Science [3,7]. In the context of microservice systems, temporal networks arise when considering either a network of a microservice system taken for each release or commit in the project history [6], or a real-time network of monitoring traces collected from a running system [3]. Our aim is to leverage such representation to detect architectural degradation and provide feedback to the architects.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 78–85, 2026.
https://doi.org/10.1007/978-3-032-04403-7_9

2 Related Work

In this section, I discuss the related work leveraging network methods for the analysis of microservice architecture and its degradation, although such methods have not been widely utilized for this problem. Moreover, most of the current approaches do not leverage the temporal dimension of the data.

For instance, Cerny et al. [13] conducted a tertiary study on architectural anti-patterns for microservices and provided a comprehensive catalog. The *topology* sub-category represents the only case where anti-patterns may be detected leveraging the microservice architectural network. Current approaches for detecting such anti-patterns either consider individual call chains, or utilize the aggregated, static network, where the order and frequency of the calls are ignored [3]. Analyzing the entire temporal network of traces could provide a more detailed and fine-grained insight into these anti-patterns [20]. For instance, Saarimäki et al. [31] highlight the importance of incorporating time as a critical yet underexplored dimension in software analysis.

Furthermore, Cerny et al. [14] have argued for adopting an evolutional study of microservice systems to assess change impact, which is currently done in monolithic projects. They argue that current approaches and tools focus on studying each microservice individually, which can miss instances when a change in one service can have an impact on other services in the system [30]. Notably, they identified works that deal with co-evolution analysis and evolution graphs [23] and showcased an evolution analysis of the *train-ticket* benchmark system [1] However, the authors did not perform any kind of network analysis on the reconstructed microservice architectures, while Abufouda and Abukwaik [2] argued for properly adopting temporal network methods in Empirical Software Engineering. In our recent paper, accepted in the current edition of ECSA [6], we demonstrated the temporal analysis of *train-ticket* leveraging temporal centrality, while the paper from the previous edition [10] leveraged temporal community detection for developer collaboration networks.

3 Proposed Methodology

We can leverage diverse types of temporal network methods for analysis of microservice systems with different goals. In this section, I consider how we can analyze two types of microservice networks presented in the introduction: microservice architectural evolution and microservice real-time tracing.

3.1 Microservice Architectural Evolution

To be able to perform temporal architectural analysis, we require a suitable microservice system and reconstructed architecture [9,15,32] available for many versions of the system [8].

Temporal centrality analysis [6,35] can provide a metric indicating the importance of each node temporally. If certain nodes become too important and central, this can be an indication that they are becoming bottlenecks and critical

points in the system [3,13]. We have recently performed such kind of analysis on the *train-ticket* microservice benchmark [6] and found that the centrality metric does not correlate with size, complexity, or quality metrics, thus being potentially a novel metric to consider [7], or an orthogonal perspective.

Moreover, temporal community detection [19] can uncover groups of nodes that are tightly connected to each other, potentially highlighting services that exchange too many calls and are too highly coupled. For instance, Gauvin et al. [19] proposed a community detection method that can demonstrate in which time instances such a group was present, even if the time instances are not consecutive, thus shedding light on the re-emergence of highly coupled groups. Indeed, we have previously applied their method on a network of developer collaboration in an open-source (OSS) microservice benchmark [10]. We discovered that each release was prepared by a single tightly coupled group of people, with only one core developer being present during the entire development and a couple of others for most of the development.

Finally, we also aim to investigate if disease spreading simulations [28], such as Susceptible-Infected-Susceptible [22], can be applied to the temporal network [21] of microservice architecture versions given some quality attributes, such as the presence of bugs, code smells, or violations [17], to see if their presence spreads like a disease once they start affecting one service [16,30].

3.2 Microservice Real-Time Tracing

Microservice tracing data is an important aspect of monitoring microservice systems [11]. Tracing data allows the practitioner/researcher to see invocation chains of services and analyze how different services and their latency or stability affect other services and the health of the overall microservice system [3]. Tracing data can be analyzed leveraging temporal network methods developed for online processing, i.e., consuming and analyzing incoming data in real-time.

In particular, temporal centrality has the potential to be calculated and updated in real-time. Unlike the algorithm by Taylor et al. [35], which requires the availability of the entire temporal network at once, Beres et al. [12] provided an algorithm that computes the centrality scores of nodes in a temporal network given as a stream of connections. The scores are updated whenever a new connection is registered. Therefore, we aim to investigate if the real-time centrality score can function as a novel metric for microservice monitoring by correlating microservice centrality with latency or other metrics [7]. The centrality score could highlight problematic services that receive too many requests and thus need to be scaled-up or potentially refactored into several services.

Similar to temporal community detection, it is possible to detect different system states to identify anomalies. Masuda and Holm [27] provided a method to cluster the different time intervals of a temporal network to identify intervals when a network appears to be in a certain state and when this state changes. A possible research direction would be to observe whether the detected states of a microservice system can be identified with the anomalies in the system given the ground truth of the induced anomalies, or correlate the state with some

other metrics observed in the system. Leveraging this method could provide a light-weight, purely trace-based method for microservice anomaly detection.

Finally, we can focus on detecting groups of nodes that stay tightly connected for a certain interval of time, known as cliques. Thus, mining cliques in the temporal network is a process similar to community detection. Lin et al. [25] provide an algorithm to detect such structures that we can leverage for a network of microservice traces. In a network of microservice monitoring, such a clique would indicate a group of services that exchange a lot of communication consistently, thus indicating the potential presence of architectural anti-patterns such as Inappropriate Service Intimacy, Wrong Cuts, or Microservice-greedy system [13].

4 Encountered Challenges

In the current stage of my PhD work and research, I have encountered challenges when pursuing both of the directions outlined in this work. The challenges are related to the availability of Industrial and OSS microservice systems and their data. They are detailed below.

4.1 Microservice Architectural Evolution

As part of our group's research work, I investigated the availability of OSS microservice systems suitable for analysis [15] and tools for microservice architecture reconstruction [9], as well as comparison and evaluation of such tools leveraging static analysis [32,33]. However, an attempt to combine the most promising tool prototype, *Code2DFD* [34], and all applicable projects from the identified dataset, i.e., Java Spring Projects, resulted in only 24 projects having a reliably reconstructed architectural network [7]. This result, together with the results of the tool evaluation [32], poses the following challenge:

Challenge 1. *Lack of microservices systems with a provided architectural diagram or which can be reconstructed in detail with a reliable tool.*

After identifying the 24 projects for which reliable architecture was obtained for at least the latest version [7], I also attempted to reconstruct the previous versions of the projects in order to obtain the temporal architectural network. However, since most of the projects are small example benchmarks, they do not show variations in architecture [8]. The only suitable project, i.e., the project with enough components, connections, and clear architectural evolution, turned out to be *train-ticket* [38], which is the de facto standard subject of research on microservices anyway [3–5,36,37]. Thus, it was the only project we could analyze in our most recent paper [6], inevitably affecting the validity and generalizability of the study. Furthermore, even the temporal network of *train-ticket* only contains 7 time instances and 42 microservices. This poses the following challenge:

Challenge 2. *Lack of microservice systems for which variations in architecture over time are known.*

Access to a big industrial system would, of course, provide a system that has evolved significantly. However, it is not guaranteed that its architecture has been precisely documented in all releases or that it could be reconstructed across the history.

4.2 Microservice Real-Time Tracing

Collecting tracing data through dynamic analysis of the system is a laborious and time-consuming task, which requires some load scenarios to be generated and executed on the running system deployed with sufficient resources [3]. As such, data of the necessary scale and with realistic scenarios and distributions is only in the hands of industrial companies with massive microservice systems that do not publish it openly.

Several "tech giants", such as eBay [20], Alibaba [26], and IBM [29], published research on analysis and monitoring of their proprietary microservice systems; however, due to industry practices, they do not share the analyzed data. This situation by itself poses a certain challenge:

Challenge 3. *Lack of access to Industrial-scale datasets of microservice traces.*

Conversely, most research leveraging OSS benchmark projects [3–5,36,37] seems to rely on either *train-ticket* [38] or *DeathStarBench* [18] projects, which appear to be the two biggest available projects in terms of the number of microservices. Others include *SockShop*[1], which is significantly smaller in scope, and *OnlineBoutique*[2], which requires deployment onto the Google Cloud platform. Such concentrated attention on a handful of benchmarks created by researchers poses a threat to validity and generalizability of works leveraging them, thus posing a challenge:

Challenge 4. *Lack of diverse OSS microservice benchmarks that can be efficiently leveraged for dynamic trace analysis.*

As part of the *Multimodal Fusion-based Anomaly Detection in Microservices* (MuFAno) project, we require a dataset of microservice monitoring data comprised of logs, metrics, and traces, to be studied jointly. Given the challenge 3, we attempted to tackle the challenge 4 by leveraging the dataset of microservice projects [15] to identify a non-benchmark OSS microservice system to perform dynamic analysis and reconstruction, obtaining logs, metrics, and traces, thus providing a novel source of microservice monitoring data. We performed dynamic analysis and testing of the seven microservices of the *light-oauth2*[3] project by *networknt* [11]. However, authors of the project did not rely on an existing OpenTelemetry implementation which required us to inject the Jaeger agent to gather the traces, resulting in almost null output.

Our attempts to find and describe the related work on microservice monitoring datasets also highlighted the fact that it is hard to find reliable datasets

[1] https://github.com/microservices-demo/microservices-demo.
[2] https://github.com/GoogleCloudPlatform/microservices-demo.
[3] https://github.com/networknt/light-oauth2.

featuring all three modalities of monitoring data, i.e., logs, traces, and metrics [11]. This presents yet another challenge:

Challenge 5. *Lack of substantial datasets of microservice monitoring data featuring traces, logs, and metrics.*

All in all, most encountered challenges are related to the lack of availability of reliable and representative microservice systems and related datasets.

5 Conclusion

In this research summary paper, I presented the ideas serving as the foundation of my research on applying temporal network methods for microservice architectural degradation. Moreover, I summarized the current status of its implementation and described several encountered challenges, most of which are due to the closed nature of big industrial players and the limited availability of OSS microservice benchmarks.

References

1. Abdelfattah, A.S., Cerny, T., Yero Salazar, J., Li, X., Taibi, D., Song, E.: Assessing evolution of microservices using static analysis. Appl. Sci. **14**(22), 10725 (2024)
2. Abufouda, M., Abukwaik, H.: On using network science in mining developers collaboration in software engineering: a systematic literature review. Int. J. Data Mining Knowl. Manag. Process (IJDKP) **7**(5/6), 17–34 (2017)
3. Al Maruf, A., Bakhtin, A., Cerny, T., Taibi, D.: Using microservice telemetry data for system dynamic analysis. In: 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE), pp. 29–38. IEEE (2022)
4. Avritzer, A., et al.: Architecture and performance anti-patterns correlation in microservice architectures. In: 2025 IEEE 22nd International Conference on Software Architecture (ICSA), pp. 60–71. IEEE (2025)
5. Bakhtin, A.: Microservice API pattern detection: using business processes and call graphs (2022)
6. Bakhtin, A., Esposito, M., Lenarduzzi, V., Taibi, D.: Centrality change proneness: an early indicator of microservice architectural degradation. In: 19th European Conference on Software Architecture (ECSA) (2025)
7. Bakhtin, A., Esposito, M., Lenarduzzi, V., Taibi, D.: Network centrality as a new perspective on microservice architecture. In: 2025 IEEE 22nd International Conference on Software Architecture (ICSA), pp. 72–83 (2025)
8. Bakhtin, A., Esposito, M., Taibi, D.: Challenges in constructing temporal architecture networks for microservice systems. Arctic AI 2024 (2024)
9. Bakhtin, A., Li, X., Soldani, J., Brogi, A., Cerny, T., Taibi, D.: Tools reconstructing microservice architecture: a systematic mapping study. In: Tekinerdoğan, B., Spalazzese, R., Sözer, H., Bonfanti, S., Weyns, D. (eds.) ECSA 2023 Tracks, Workshops, and Doctoral Symposium, ECSA 2023. LNCS, vol. 14590, pp. 3–18. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-66326-0_1
10. Bakhtin, A., Li, X., Taibi, D.: Temporal community detection in developer collaboration networks of microservice projects. In: Galster, M., Scandurra, P., Mikkonen, T., Oliveira Antonino, P., Nakagawa, E.Y., Navarro, E. (eds.) ECSA 2024. LNCS, vol. 14889, pp. 174–182. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-70797-1_11

11. Bakhtin, A., et al.: LO2: microservice API anomaly dataset of logs and metrics. In: 21st International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE) (2025)
12. Béres, F., Pálovics, R., Oláh, A., Benczúr, A.A.: Temporal walk based centrality metric for graph streams. Appl. Netw. Sci. **3**(1), 1–26 (2018). https://doi.org/10.1007/s41109-018-0080-5
13. Cerny, T., Abdelfattah, A.S., Al Maruf, A., Janes, A., Taibi, D.: Catalog and detection techniques of microservice anti-patterns and bad smells: a tertiary study. J. Syst. Softw. **206**, 111829 (2023)
14. Cerny, T., Goulis, G., Abdelfattah, A.S.: Towards change impact analysis in microservices-based system evolution, January 2025. https://doi.org/10.48550/arXiv.2501.11778
15. d'Aragona, D.A., et al.: A dataset of microservices-based open-source projects. In: Proceedings of the 21st International Conference on Mining Software Repositories, pp. 504–509 (2024)
16. Esposito, M., Falessi, D.: Uncovering the hidden risks: the importance of predicting bugginess in untouched methods. In: 2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM), pp. 277–282. IEEE (2023)
17. Esposito, M., Falessi, D.: VALIDATE: a deep dive into vulnerability prediction datasets. Inf. Softw. Technol., 107448 (2024)
18. Gan, Y., et al.: An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 3–18 (2019)
19. Gauvin, L., Panisson, A., Cattuto, C.: Detecting the community structure and activity patterns of temporal networks: a non-negative tensor factorization approach. PLoS ONE **9**(1), e86028 (2014)
20. Guo, X., et al.: Graph-based trace analysis for microservice architecture understanding and problem diagnosis. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 1387–1397 (2020)
21. Holme, P.: Temporal network structures controlling disease spreading. Phys. Rev. E **94**(2), 022305 (2016)
22. Lee, H.K., Shim, P.S., Noh, J.D.: Epidemic threshold of the susceptible-infected-susceptible model on complex networks. Phys. Rev. E-Stat. Nonlinear Soft Matter Phys. **87**(6), 062812 (2013)
23. Lelovic, L., et al.: Change impact analysis in microservice systems: a systematic literature review. J. Syst. Softw., 112241 (2024)
24. Lewis, J., Fowler, M.: Microservices: a definition of this new architectural term (2014). https://martinfowler.com/articles/microservices.html
25. Lin, L., Yuan, P., Li, R.H., Wang, J., Liu, L., Jin, H.: Mining stable quasi-cliques on temporal networks. IEEE Trans. Syst. Man Cybern. Syst. **52**(6), 3731–3745 (2021)
26. Luo, S., et al.: Characterizing microservice dependency and performance: Alibaba trace analysis. In: Proceedings of the ACM Symposium on Cloud Computing, pp. 412–426 (2021)
27. Masuda, N., Holme, P.: Detecting sequences of system states in temporal networks. Sci. Rep. **9**(1), 795 (2019)
28. Newman, M.E.: Spread of epidemic disease on networks. Phys. Rev. E **66**(1), 016128 (2002)

29. Pourmajidi, W., Miranskyy, A., Steinbacher, J., Erwin, T., Godwin, D.: Dogfooding: using IBM cloud services to monitor IBM cloud infrastructure. In: Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering, pp. 344–353 (2019)
30. Robredo Manero, M., Esposito, M., Palomba, F., Peñaloza, R., Lenarduzzi, V.: Analyzing the ripple effects of refactoring. a registered report. This Registered Report has been accepted at ICSME (2024)
31. Saarimäki, N., Moreschini, S., Lomio, F., Penaloza, R., Lenarduzzi, V.: Towards a robust approach to analyze time-dependent data in software engineering. In: 2022 SANER, pp. 36–40. IEEE (2022)
32. Schneider, S., et al.: Comparison of static analysis architecture recovery tools for microservice applications: Preprint. arXiv preprint arXiv:2412.08352 (2024)
33. Schneider, S., et al.: Comparison of static analysis architecture recovery tools for microservice applications: Registered report. arXiv preprint arXiv:2403.06941 (2024)
34. Schneider, S., Scandariato, R.: Automatic extraction of security-rich dataflow diagrams for microservice applications written in Java. J. Syst. Softw. **202**, 111722 (2023)
35. Taylor, D., Myers, S.A., Clauset, A., Porter, M.A., Mucha, P.J.: Eigenvector-based centrality measures for temporal networks. Multiscale Model. Simul. **15**(1), 537–574 (2017)
36. Zhang, C., et al.: TraceCRL: contrastive representation learning for microservice trace analysis. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 1221–1232 (2022)
37. Zhao, Y., De Matteis, T., Bogner, J.: How does microservice granularity impact energy consumption and performance? A controlled experiment. In: 2025 IEEE 22nd International Conference on Software Architecture (ICSA) (2025)
38. Zhou, X., Peng, X., Xie, T., Sun, J., Xu, C., Ji, C., Zhao, W.: Benchmarking microservice systems for software engineering research. In: Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings, pp. 323–324 (2018)

AISA Workshop

The Impact of AI-Generated Solutions on Software Architecture and Productivity: Results from a Survey Study

Giorgio Amasanti and Jasmin Jahić(✉)

University of Cambridge, Cambridge, UK
jj542@cam.ac.uk

Abstract. AI-powered software tools are widely used to assist software engineers. However, there is still a need to understand the productivity benefits of such tools for software engineers. In addition to short-term benefits, there is a question of how adopting AI-generated solutions affects the quality of software over time (e.g., maintainability).

To provide some insight on these questions, we conducted a survey among software practitioners. We conclude that AI tools significantly increase the productivity of software engineers. However, the productivity benefits reduce as projects become more complex. The results also show that there are no significant negative influences of adopting AI-generated solutions on software quality, as long as those solutions are limited to smaller code snippets. However, when solving larger and more complex problems, AI tools generate solutions of a lower quality, indicating the need for architects to perform problem decomposition and solution integration.

Keywords: Software Architecture · AI · Productivity · Quality

1 Introduction

According to a study conducted by Stack Overflow in May 2024 [24], 76.6% developers are using or planning to use AI tools. Engineers use AI tools for various tasks in software engineering [7,28] (e.g., code generation [6], assistance with software architecture [12]). However, adopting AI in software engineering also presents challenges, as AI is a disruptive technology [13]. Therefore, many practitioners (e.g., architects and managers) need to be convinced that adopting AI will bring concrete benefits without hidden "costs". E.g., while some AI tools certainly can provide short-term benefits (e.g., coding assistance), adopting AI-generated solutions might have negative long-term effects (e.g., adopting suboptimal AI-generated code). In software architecture, we label suboptimal solutions as antipatterns and aim to avoid them. Hence, there is a need to understand how the adoption of AI tools (and solutions created with or generated by AI tools)

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 89–104, 2026.
https://doi.org/10.1007/978-3-032-04403-7_10

influences software architecture in terms of maintainability, extendability, understandability, modifiability, cohesion, coupling, performance, and overall software partitioning. Therefore, we created a study driven by these two main research questions: i) **how AI influences the productivity of software engineers**, and ii) **does adopting AI-generated solutions have significant negative effects on the quality of software architecture**.

To gather data that can help us answer these questions, we conducted a survey among 40 practitioners who use AI tools. The data gathered shows that using AI tools significantly boosts productivity in the early stages of software projects and that adopting AI generated solutions does not affect software architecture in a significantly negative way. However, once there is an established code base, adopting solutions generated by AI (e.g., changing code, new design solutions) is not straightforward and can introduce significant overhead. Despite these challenges, the overwhelming majority of the surveyed professionals reported a significant increase in productivity (over 35%) when using AI tools (for various tasks, not just coding). Therefore, **if you do not use AI tools in software engineering, you are becoming less competitive** because individuals using AI tools ***significantly outperform*** those who do not use such tools for the variety of tasks in Software Development Life Cycle (SDLC).

This paper is organised as follows. In Sect. 2, we discuss related work. In Sect. 3, we present our research methodology. Section 4 presents the results of the industrial survey. We conclude in Sect. 5.

2 Related Work

There exist several studies that investigate how effective AI tools are for code generation [1,6,16]. Engineers also use AI tools for other purposes: program repair [14,18,29], code summarization [2,11], and generating software architecture solutions [12,21]. In addition, there are several studies that investigate quality of solutions that these tools provide [20], indicating that these tools suffer from low accuracy and low reliability [8,9,17,26]. For example, how ChatGPT struggles with lengthy descriptions of problems [25].

However, more recent studies [19,22], such as the one performed by Stack Overflow [24] offer different perspective and indicate that using AI tools in software engineering offers productivity boosts. For example, one of the highest-performing recent AI tools, LDB (a novel AI code debugging framework using GPT-4o), can solve up to 98.2% of the problems in HumanEval benchmark after one attempt (Pass@1) [30]. Bencheikh et al. [5] conducted a study comparing the quality of AI and human-written software requirements, and concluded that the AI "performs reasonably well" as a software requirements engineer. As outlined by He et al. [10], the subject of much research on AI tools for software development has been focussed on the implementation of multi-agent systems. This involves utilising multiple AI entities to work together and perform separate tasks within the software engineering process, much like a human engineering team. This framework is the methodology adopted by many dedicated AI software development tools including ChatDev and MetaGPT, with a seemingly

positive impact on performance in the software development process. Waseem et al. [27] conducted a case study similar to ours investigating the viability of ChatGPT as a software engineering tool. Their study reported that AI was effective at: identifying requirements, streamlining development via code generation, bug detection and architectural decision making. Compared to the existing work, we focus on productivity and quality of software design, when adopting AI-generated solutions.

3 Research Study Design

Our study is composed out of two substudies. In the first sub-study, we created an instance of the classic Tetris videogame with the help of AI tools. We used this sub-study to scan the market for available AI tools and to get initial results of how these tools behave when integrated into Software Development Life Cycle (SDLC). We used the conclusions and experiences from the first substudy to setup a survey with industry professionals (second substudy). To structure our study, we used Goal-Question-Metric (GQM) approach [4,23]. For the design of the survey, we followed the established guidelines [15]. To analyse the data gathered in the survey, we applied the principles of grounded theory using open and axial coding.

3.1 Tetris Project Study: Design

Table 1 presents the detailed design (using the GQM approach) for our substudy in which we created an instance of a classical Tetris game. In this substudy, we aimed to investigate how AI tools behave when integrated in SDLC compared to the work done without using AI tools. To conduct the study, we sought a junior *software engineer* with proficiency in at least one programming language, and an experienced software engineering professional to serve as the *project commissioner*, responsible for defining the business architecture and high-level requirements. The development was organised using the Scrum framework[1].

3.2 Tetris Project Study: Results

The substudy was conducted over a period of six weeks. In the first half of the study, the *software engineer* scanned the market for available AI-tools (Google search, systematic literature reviews, case studies, and reports on using AI in software engineering) and used those tools to experiment with setting up tool chains to cover stages of the Software Development Life Cycle (SDLC). In the second half of the study, over a period of the next three weeks (comprising three one-week sprints), the *software engineer*, guided by instructions from the *project commissioner*, developed three parallel versions of the Tetris game, all three based on the same set of requirements. Each version followed a different technical

[1] https://www.scrum.org/resources/what-scrum-module.

Table 1. GQM Mapping: Tetris Project Study

Goal	Question	Metric
G1: Identify AI tools for SDLC	**Q1.1:** Which AI tools are currently available across SDLC?	**M1.1:** Tool name and SDLC stage(s) supported; context of use
	Q1.2: Which SDLC tools have AI-enabled features?	**M1.2:** Tool name and vendor; list of AI subcomponents/features
	Q1.3: Are there integrated AI-powered SDLC toolchains?	**M1.3:** Toolchain structure; SDLC stages covered
G2: Analyze suitability of AI for SDLC	**Q2.1:** What tasks are commonly assisted or automated by AI?	**M2.1:** Task–tool mapping; automation level
	Q2.2: Which SDLC stages are difficult to delegate to AI?	**M2.2:** Stages flagged as unsuitable; challenges reported per stage
G3: Assess productivity impact of AI tools	**Q3.1:** Does AI reduce time required for specific tasks?	**M3.1:** Estimated time saved per task/project
	Q3.2: How often do AI outputs require rework?	**M3.2:** Percentage of AI outputs needing edits; rework frequency
	Q3.3: How do developers assess productivity with AI?	**M3.3:** Self-rated productivity scores with/without AI
G4: Examine architectural implications of adopting AI-generated solutions	**Q4.1:** How correct and adequate are AI solutions?	**M4.1:** Percentage of AI outputs used without modification; number of bugs
	Q4.2: Are AI-generated solutions harder to modify or test?	**M4.2:** Time to understand/edit/integrate AI code; difficulty rating
	Q4.3: How does AI-generated code affect quality of architecture?	**M4.3:** System consistency; integration issues; maintainability issues; extendibility issues; understandability issues; cohesion and coupling issues; logical partitioning issues

setup. In **version a)**, the *software engineer* did not use any AI tools. In **version b)**, the engineer used GPT Pilot—a terminal-based code planning and generation tool leveraging GPT-4o-mini-2024-07-18 via OpenAI's API—and ChatGPT running on GPT-4o-2024-05-13. In **version c)**, the *software engineer* employed multiple ChatGPT instances (GPT-4o-2024-05-13) to simulate distinct roles: (i) project manager—configures other roles; (ii) requirements engineer—generates refined requirements; (iii) software architect—designs the base software architecture; (iv) developer agent—writes and debugs code; and (v) quality assurance (QA)/quality control (QC) engineer—evaluates and provides feedback on code quality. The *software engineer* manually copied inputs and outputs between these roles to advance development. The results of the study are summirized in Table 2, and the full and raw results, as well as the code, are available online[2345].

[2] https://github.com/AItoolsSE/CAIN-AI-documents.
[3] https://github.com/AItoolsSE/CAIN-AI-approach-A.
[4] https://github.com/AItoolsSE/CAIN-AI-approach-B.
[5] https://github.com/AItoolsSE/CAIN-AI-approach-C.

Table 2. Mapping GQM Structure to Tetris Project Results

G	Q	M	Finding/Observation
G1	Q1.1	M1.1	Identified: ChatGPT, StarCoder, Llama; GitHub Copilot, Cursor IDE; Snyk, Harness; Datadog, Splunk, Logz.io, Moogsoft
	Q1.2	M1.2	Features observed in tools like GPT Pilot, with multi-stage AI support
	Q1.3	M1.3	Tools like GPT Pilot and ChatDev integrate multiple SDLC stages including requirements, coding, and review
G2	Q2.1	M2.1	AI is effective in early implementation and boilerplate generation. Less effective in bug fixing and integration
	Q2.2	M2.2	Bug fixing and code modification stages require multiple prompts; productivity drops as complexity increases
G3	Q3.1	M3.1	AI improves productivity by up to 35%, especially during early-stage development. Long response times from AI tools have a significant negative impact on productivity. As code complexity increases, AI response times also increase, reducing productivity gains
	Q3.2	M3.2	Typically 2–3 prompt iterations are needed; the need for rework increases with code complexity (to resolve introduced bugs)
	Q3.3	M3.3	Asking for AI assistance on small, focused problems maximizes productivity. Attempting to solve large problems in a single prompt often leads to errors. Manual editing is often faster for integration and bug fixes (AI often requires multiple prompts). Humans comprehend the code better than AI
G4	Q4.1	M4.1	AI-generated code is almost always syntactically correct. AI rarely produces functionally adequate results on first attempt and the functional adequacy of generated code decreases as complexity grows
	Q4.2	M4.2	The logical style of AI-generated code—such as naming consistency, modifiability, and line length—is generally comparable to human-written code, contributing to good readability and understandability. AI requires more time than humans for editing and debugging as it often requires multiple prompts. Integrating AI-generated code into an existing codebase can require substantial manual effort, such as renaming variables or adapting function signatures and module interfaces
	Q4.3	M4.3	Large AI-generated code has poor modularity, maintainability, extensibility, and understandability due to illogical structure. Small snippets yield high cohesion, low coupling and a clear separation of concerns, resembling human-written code in architectural quality. Smaller AI-generated code snippets also match human-written code in terms of maintainability, including structural quality and clarity. AI-generated code demonstrates performance characteristics comparable to those of human-written code, with no significant overhead observed

3.3 Survey with Industry Professionals: Design

We used conclusions from the Tetris Project Study (Sect. 3.2) to create a structure of the survey with industry professionals. Table 3 presents the detailed study design using the GQM approach. To conduct the study, we sought industry professionals who are already using some AI tools in their daily work. We aimed for 30 participants, with a variety considering the institution size, years of experience, roles in SDLC, and geography locations. We invited professionals from our network and shared the survey on LinkedIn.

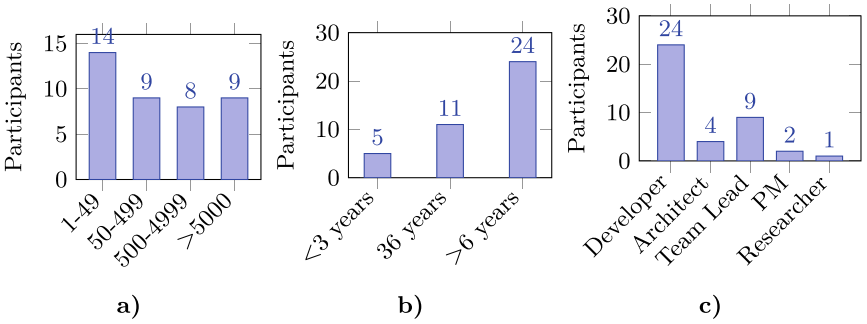

Fig. 1. a) institution size, b) experience (PM - Project Manager), c) role

4 The Impact of AI-Generated Solutions on Software Architecture and Productivity

4.1 Demographics

We conducted the survey in November 2024 and received responses from ***40 participants*** (***the raw survey results are available online***[6]) from 13 different countries and companies of various sizes (Fig. 1).

4.2 Results

As the first step in grounded theory, we performed open-coding analysis on the raw qualitative data, breaking them into discrete parts and labelling them with codes that represent the underlying concepts, actions, and concerns. To build

[6] https://github.com/AItoolsSE/CAIN-AI-documents.

Table 3. GQM: AI Tools in Software Engineering – Productivity and Architecture

Goal	Question	Metric
G1: Identify AI tools in the SDLC	**Q1.1:** Which tasks in the SDLC are supported by AI?	**M1.1:** List of SDLC tasks (e.g., code generation, test generation, CI/CD, bug fixing)
	Q1.2: Which AI tools are used in daily development?	**M1.2:** Tool names selected (e.g., Copilot, ChatGPT)
G2: Assess the productivity benefits of using AI tools	**Q2.1:** How do software engineers perceive productivity improvements with AI?	**M2.1:** Self-reported productivity impact relative to a reference of up to 35% improvement (ordinal scale: much lower to much higher)
	Q2.2: At which SDLC stage is AI usage most beneficial?	**M2.2:** Agreement on usefulness in early implementation (e.g., generating skeleton code)
	Q2.3: What is the most effective way to apply AI tools during development?	**M2.3:** Agreement on effectiveness of decomposition (e.g., prompts on smaller problems yield better results)
	Q2.4: What factors hinder productivity when using AI tools?	**M2.4:** Processing time; increased latency with code size; bug frequency in larger outputs; average number of prompt repetitions; time to fix issues; manual integration effort
G3: Examine architectural challenges of adopting AI-generated code	**Q3.1:** How syntactically correct is generated code?	**M3.1:** Frequency of syntactic correctness (Always, Often, Sometimes, Rarely)
	Q3.2: How functionally adequate is generated code?	**M3.2:** Self-reported success rate; prompt repetition; influence of complexity
	Q3.3: Does large-scale AI-generated code reduce logical organisation and architectural quality?	**M3.3:** Agreement with poor structure in large AI-generated blocks (low maintainability, extendability)
	Q3.4: Does the use of AI increase architectural erosion (e.g., cohesion, coupling)?	**M3.4:** Agreement on architectural erosion; depends on size and structure of AI-generated code
	Q3.5: Is AI-generated code comparable in performance to human-written code?	**M3.5:** Agreement with performance quality (e.g., minimal overhead)
	Q3.6: Does AI-generated code affect maintainability and modifiability?	**M3.6:** Agreement with difficulty updating/extending AI-generated code
	Q3.7: What kinds of tests are generated by AI tools?	**M3.7:** Presence of advanced testing patterns (e.g., mocks, pixel-level testing)
	Q3.8: How reliable are AI-generated tests?	**M3.8:** Initial test failure rate

Table 4. Survey results: Mapping of analysis results to GQM elements

G	Q	M	Finding/Observation
G1	Q1.1	M1.1	AI tools are used across SDLC for code generation, bug fixing, test generation, CI/CD, and documentation support
	Q1.2	M1.2	Tools reported include ChatGPT, Claude, Copilot, and Cursor IDE, with varying preferences based on user experience
G2	Q2.1	M2.1	Productivity gains range from much higher (considering 35% improvement reference) to none, depending on the SDLC task, developer experience, and task complexity. A significant increase in engineering performance productivity
	Q2.2	M2.2	AI is considered most beneficial in early implementation stages (e.g., boilerplate, skeleton code)
	Q2.3	M2.3	Smaller, well-scoped prompts are more effective; prompting skill is a key enabler of productivity
	Q2.4	M2.4	Manual integration effort, low test reliability, prompt iteration overhead, and poor handling of complex tasks. Minimal challenges for smaller code snippets and smaller problems
G3	Q3.1	M3.1	AI-generated code is often syntactically correct but varies in quality depending on prompt clarity and task complexity
	Q3.2	M3.2	First-attempt functional adequacy is inconsistent; requires iterative prompting, especially for complex problems
	Q3.3	M3.3	Larger AI-generated blocks reduce architectural clarity and maintainability; structure quality degrades
	Q3.4	M3.4	Signs of architectural erosion (lower cohesion) in AI-generated code for larger problems and code bases. Minimal negative effect for smaller problems and small code snippets
	Q3.5	M3.5	Performance of AI-generated code is acceptable for simpler cases. It can be affected as the problem size increases
	Q3.6	M3.6	Maintainability can suffer due to lack of structure, contextual gaps, and ambiguous logic
	Q3.7	M3.7	Tests generated are often superficial; lack coverage based on requirements or edge cases
	Q3.8	M3.8	High initial failure rate of tests noted; AI tends to generate tests to pass existing code, not validate correctness

meaningful connections between data fragments in the survey, we applied axial coding on the results of the open-coding analysis. Finally, we mapped these insights to our GQM table (Table 4).

From the results obtained by the application of grounded theory, we conclude the following. The most common uses of AI considering SDLC are to generate small code snippets, modify existing code, fix bugs, and generate tests cases (Fig. 2). The most popular AI tools in the industry are ChatGPT and Copilot (Fig. 3). We observed that 50% of the survey participants who use ChatGPT also use Copilot.

Measuring ***productivity*** in software engineering is not easy [3]. Therefore, we decided to ask for overall ***subjective feeling of productivity*** that each survey participant has when using AI tools. In that subjective context, we did try to understand if AI is more helpful with some tasks and if it introduces overhead in others. Based on our experience from the test project and based on the experiences reported in related work, we decided to differentiate between i) applying AI tools to create a fresh code base and ii) applying AI to an already established project. The results of the survey are presented in a way that reflects this differentiation.

When considering 35% increase in productivity as a reference value (as observed in our test project), the results (Fig. 4) show that 79% of the surveyed participants experience the same, higher or much higher productivity when using AI tools. It is important to note that 21% of the participants experienced lower improvements in productivity or no improvements in productivity at all. We analysed the results of those that indicated lower productivity, but could not find any correlation with other answers. These participants use ChatGPT and Copilot (the same as those who experienced increase in productivity), and use these tools for similar tasks. The only interesting thing here might be that all the juniors (below 3 years of work experience) said that AI tools increase their productivity (none of the juniors was among 21% of the participants who experienced lower improvements in productivity or no improvements in productivity). Majority of the survey participants agree that the highest productivity increase with AI tools can be achieved to generate a skeleton of an application and generate the basic features of the application (Fig. 5).

The overwhelming majority of the survey participants agree that the best way to use AI tools to improve productivity is to break a problem into small chunks and ask AI to implement it (Fig. 6-a). As indicated in Fig. 6-b, 40% survey participants observed increase in processing time of AI tools as their code size increases, affecting productivity in a negative way. Most of the survey participants (more than 67%) observed that AI tools are more likely to break the existing code, hindering the productivity, as the size of the code base increases (Fig. 6-c).

Although 65% of the survey participants agree that AI can take more time than humans to change the code (Fig. 7-a), there is a strong indication from 15% of the participants who disagree with this claim. This is an interesting result, as it shows that AI in certain scenarios can be faster than humans when it is necessary to change a code base. When it comes to fixing bugs, the results of the survey (Fig. 7-b) show that 20% of the participants experienced that AI is faster than humans. However, 60% of the participants still experience that humans are

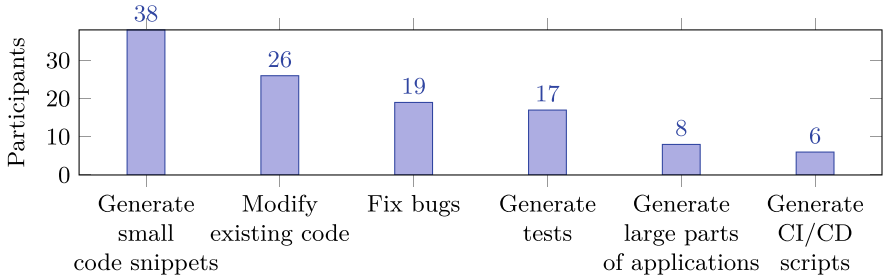

Fig. 2. Usage of AI for Coding Tasks

(much) faster when it comes to fixing bugs. The survey results (Fig. 7-c) show that there is a notable number of people who expressed that they required a significant effort when integrating AI-generated code into existing code. However, there are also those that consider this not to be a challenge at all (over 35% of the surveyed participants).

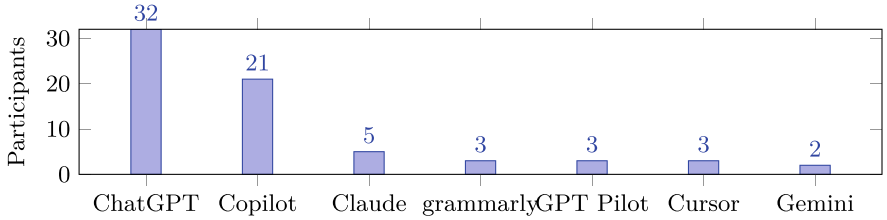

Fig. 3. AI tools used by the survey participants (there were also single users of Amazon Q, CodeWhisperer, Open-source projects, in-house tools)

Some survey participants claim challenges with the code organization of the generated code (Fig. 8-a). Although, it seems that there were few (15%) who had positive experiences with AI partitioning the code (from the answers we had in the survey, we could not find any correlation between these results and any other answers - except that all those who answered in this way have over 3 years of experience in software engineering). The survey participants agree that generating small snippets of code with AI tool and their adoption does not lead to architecture erosion in terms of cohesion, coupling, and logical code partitioning (Fig. 8-b). It seems that over 50% of the survey participants agree that AI generated code is performant (no significant unnecessary overheads in the code that humans would avoid) in a similar way that it would have been if it was written by humans (Fig. 8-c). It is important to emphasize here that 17.5% of the survey participants have different experience and disagree with this claim, indicating that AI tools still have some work to do when it comes to code performance. However, from the answers we gathered, we could not find any correlation with other parameters that would indicate why this is the case

(similar tools, years of experience, company size, and roles as for those that have experienced the opposite).

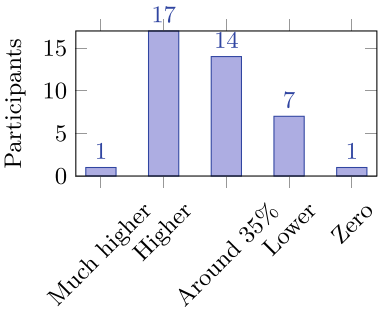

Fig. 4. Perceived productivity improvements (reference: 35%)

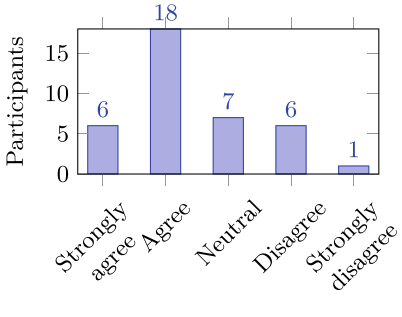

Fig. 5. The best moment to apply AI is in the early implementation

It seems to be a common experience in industry that first shot answers provided by AI tools rarely fulfill desired functionality, and therefore prompts have to be tuned (Fig. 9-a). Over 50% of the survey participants say that AI-generated code is almost always syntactically correct. But, there is a significant number of participants who disagree (Fig. 9-b). However, from the answers we gathered, we could not find any correlation with other parameters that would indicate why this is the case (similar tools, years of experience, company size, and roles as for those that have experienced the opposite). When it comes to the tests, some survey participants agree that AI generated tests use more complex testing techniques (around 20%), while the same number of the survey participants disagree with this claim. The majority was neutral with respect to this question (Fig. 9-e).

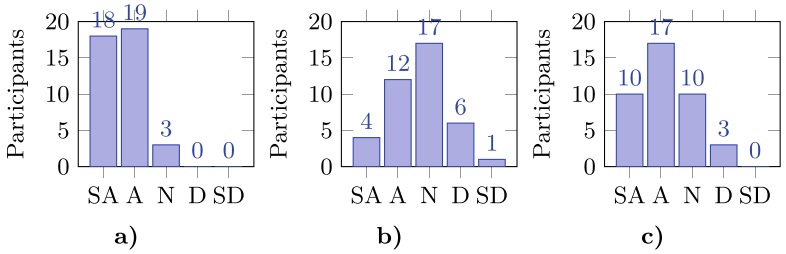

Fig. 6. Participant agreement levels: (a) The best way to use AI tools for improving productivity is to break a problem into small chunks and ask AI to implement it; (b) As code size and code complexity increase, productivity gains decrease because AI tools take a longer time to process and return the larger code segments; (c) As code size and code complexity increase, productivity gains decrease because AI tools are more likely to break existing code. Response scale: SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, SD = Strongly Disagree.

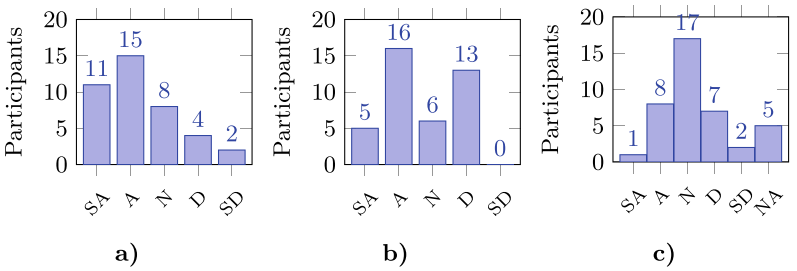

Fig. 7. Participant agreement levels on AI limitations for code understanding and integration: (a) Using AI to make code changes can be more time-consuming than doing it manually; (b) AI can take significantly longer to fix bugs than humans; (c) AI-generated code requires significant integration effort. Response scale: SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, SD = Strongly Disagree.

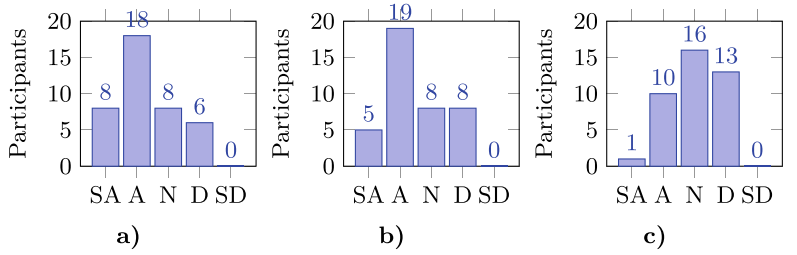

Fig. 8. Perceived quality of AI-generated code: (a) When generating full applications or large portions of code involving multiple functions/classes at once, partition and organisation of AI-generated code can be of a significantly lesser quality to that of human-written code and therefore not easy to maintain; (b) Code snippets (smaller code chunks) generated by AI are of comparable maintenance quality (high cohesion, low coupling) to that of human-written code; (c) Code generated by AI is of comparable performance quality to that of human-written code. Response scale: SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, SD = Strongly Disagree, NA = Not Applicable.

When it comes to **individual comments** we received about **positive effects of AI tools on productivity**, the survey participants listed several interesting statements (we tried to merge similar ones). For example: i) use LLMs to quickly sum up and recommend the latest cutting-edge research that's relevant to the daily work, which really speeds up getting new tech into software. They claim that this has been more useful than help with coding; ii) Quickly access information from design documents and other company-specific resources (much faster than standard searching); iii) Brainstorm ideas, ask questions like "how do I do this?" rather than asking the tool to output code; iv) Ask AI tools for example implementation with some libraries is faster than reading documentation; v) Generate boilerplate code.

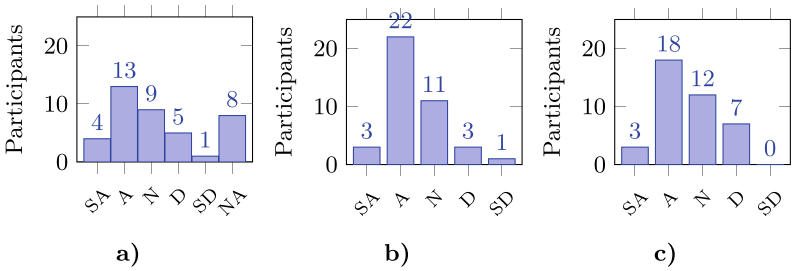

Fig. 9. Participant responses on characteristics of AI-generated code: (a) AI-generated code rarely fulfils desired functionality in the first shot. As the problem size increases, so does the need for fixing the generated code, (b) AI-generated code is almost always syntactically correct, (c) Tests generated by AI tend to use more complex testing techniques (e.g., mock objects, pixel colour testing). Response scale: SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, SD = Strongly Disagree, NA = Not Applicable.

4.3 Summary

Here are the conclusions based on the survey analysis:

i) About 45% of the survey participants experienced an increase in productivity of OVER 35% when using AI tools. This fact cannot be ignored, and it is clear that anyone working today in the software industry needs to use AI tools if they want to remain competitive. Otherwise, with the best of their efforts, they will struggle to keep up with the productivity that those who use AI tools can achieve.

ii) Adoption of AI-generated code snippets does NOT lead to a significant erosion of software architecture (in terms of cohesion, coupling, partitioning of logical parts). The code is performant and easy to maintain. AI-generated code is almost always syntactically correct and the quality of the logical syntax style is adequate.

iii) AI tools can suggest adequate architectural designs. However, this conclusion should be taken carefully, as it depends on many factors, including how we formulate the problem and the experience of those that consume the AI generated solutions, as solutions often require tweaks. Junior software architects would certainly struggle to assess whether AI-generated solutions are adequate.

iv) However, AI tends to suggest inadequate solutions when faced directly with large and complex problems (without prior decomposition). In addition, AI tends to suggest inadequate solutions when changing complex and large code bases. When applied on large and complex problems, AI-generated code is often of a significantly lesser quality than human-written code in terms of partition and organisation and, therefore, not easy to maintain and integrate. Furthermore, using AI to change complex and large code bases leads to the breaking of existing code.

Although engineers tend to profit from AI in terms of higher productivity, it is also clear that they still need to perform a decomposition of problems to the level that AI can handle. Furthermore, it is also clear that the job of software engineers is to synthesise the generated solutions into well-architected systems. Therefore, it is fair to say that architecture skills are highly needed in the age of AI.

5 Conclusion and Future Work

Generating code and tests is only one of the use cases where the adoption of AI tools increases productivity. Others include using AI for more conceptual work (software architecture) and using AI to help with side tasks (e.g., summarising information). Despite all the challenges associated with the use of AI in software engineering, it is clear that AI is not a hype and that AI tools in software engineering are here to stay. The increase in productivity of over 35% is a clear and strong sign that individuals using AI tools significantly outperform those who do not use AI tools. The questions that now open are related to the use cases where we could apply AI, how to reduce friction when adopting AI tools in existing workloads and code bases, and what is it that we can now achieve with the increase in productivity. It is also clear that not everyone equally enjoys increases in productivity, and there is a strong concern that junior engineers might bring more overhead than benefit when adopting solutions that they fully do not understand.

As code size and code complexity increase (in general, as a problem complexity increases), productivity gains decrease because AI tools are a) more likely to break existing code; b) demonstrate increased processing time; c) software shows the sign of architectural erosion. Therefore, it is more challenging to change code using AI than to create new code. We have to observe that today this is where most of the engineering time is invested, and it would be great if AI could support these activities in a smoother way. That will also be one of the topics of our future work.

References

1. OpenAI, Pilipiszyn, A.: https://openai.com/blog/gpt-3-apps/. Accessed 8 Jan 2023
2. Ahmed, T., Devanbu, P.: Few-shot training LLMs for project-specific code-summarization. In: Automated Software Engineering, 2022, NY, USA (2023)
3. de Barros Sampaio, S.C., Barros, E.A., de Aquino, G.S., e Silva, M.J.C., de Lemos Meira, S.R.: A review of productivity factors and strategies on software development. In: 2010 Fifth International Conference on Software Engineering Advances, pp. 196–204 (2010). https://doi.org/10.1109/ICSEA.2010.37
4. Basili, V.R., Weiss, D.M.: A methodology for collecting valid software engineering data. IEEE Trans. Softw. Eng. **SE-10**(6), 728–738 (1984). https://doi.org/10.1109/TSE.1984.5010301

5. Bencheikh, L., Höglund, N.: Exploring the efficacy of ChatGPT in generating requirements: an experimental study (2023)
6. Bucaioni, A., Ekedahl, H., Helander, V., Nguyen, P.T.: Programming with ChatGPT: how far can we go? Mach. Learn. Appl. **15**, 100526 (2024). https://doi.org/10.1016/j.mlwa.2024.100526. https://www.sciencedirect.com/science/article/pii/S2666827024000021
7. Chang, Y., et al.: A survey on evaluation of large language models. ACM Trans. Intell. Syst. Technol. **15**(3) (2024). https://doi.org/10.1145/3641289
8. Ciniselli, M., Cooper, N., Pascarella, L., Poshyvanyk, D., Penta, M.D., Bavota, G.: An empirical study on the usage of BERT models for code completion. CoRR abs/2103.07115 (2021). https://arxiv.org/abs/2103.07115
9. Fan, Z., Gao, X., Mirchev, M., Roychoudhury, A., Tan, S.H.: Automated repair of programs from large language models (2023)
10. He, J., Treude, C., Lo, D.: LLM-based multi-agent systems for software engineering: vision and the road ahead (2024). https://arxiv.org/abs/2404.04834
11. Hu, X., Li, G., Xia, X., Lo, D., Jin, Z.: Deep code comment generation with hybrid lexical and syntactical information. Emp. Softw. Engg. **25**(3), 2179–2217 (2020)
12. Jahic, J., Sami, A.: State of practice: LLMs in software engineering and software architecture. In: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), pp. 311–318. IEEE Computer Society, Los Alamitos, CA, USA, June 2024. https://doi.org/10.1109/ICSA-C63560.2024.00059. https://doi.ieeecomputersociety.org/10.1109/ICSA-C63560.2024.00059
13. Jahić, J., Roitsch, R., Grzymkowski, L.: Knowledge-based adequacy assessment approach to support AI adoption. In: 2021 IEEE 18th International Conference on Software Architecture Companion (ICSA-C), pp. 8–14 (2021). https://doi.org/10.1109/ICSA-C52384.2021.00008
14. Jain, N., et al.: Jigsaw: large language models meet program synthesis. In: Proceedings of the 44th International Conference on Software Engineering, ICSE 2022, New York, NY, USA, pp. 1219–1231 (2022)
15. Kasunic, M.: Designing an effective survey. Technical report (2005)
16. Kim, S., Zhao, J., Tian, Y., Chandra, S.: Code prediction by feeding trees to transformers. In: ICSE, pp. 150–162 (2021)
17. Mastropaolo, A., et al.: Studying the usage of text-to-text transfer transformer to support code-related tasks. CoRR abs/2102.02017 (2021). https://arxiv.org/abs/2102.02017
18. Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M.C., Jiang, Z.M.J.: GitHub Copilot AI pair programmer: asset or liability? J. Syst. Softw. **203**, 111734 (2023)
19. Nguyen-Duc, A., Abrahamsson, P., Khomh, F. (eds.): Generative AI for Effective Software Development. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-55642-5
20. Pearce, H.A., Tan, B., Ahmad, B., Karri, R., Dolan-Gavitt, B.: Can OpenAI codex and other large language models help us fix security bugs? ArXiv abs/2112.02125 (2021)
21. Ramachandran, R.: Transforming software architecture design with intelligent assistants-a comparative analysis. In: SoutheastCon 2025, pp. 1446–1454 (2025). https://doi.org/10.1109/SoutheastCon56624.2025.10971683
22. Sauvola, J., Tarkoma, S., Klemettinen, M., Riekki, J., Doermann, D.: Future of software development with generative AI. Autom. Softw. Eng. **31**(1), 26 (2024). https://doi.org/10.1007/s10515-024-00426-z

23. van Solingen, R., Basili, V., Caldiera, G., Rombach, H.D.: Goal Question Metric (GQM) approach. John Wiley and Sons, Ltd. (2002). https://doi.org/10.1002/0471028959.sof142. https://onlinelibrary.wiley.com/doi/abs/10.1002/0471028959.sof142
24. StackOverflow: 2024 developer survey insights for AI/ML (2024). https://stackoverflow.co/labs/2024-developer-survey-insights-for-ai-ml/. Accessed 08 Sept 2024
25. Tian, H., et al.: Is ChatGPT the ultimate programming assistant – how far is it? (2023). https://arxiv.org/abs/2304.11938
26. Tufano, M., Watson, C., Bavota, G., Penta, M.D., White, M., Poshyvanyk, D.: An empirical study on learning bug-fixing patches in the wild via neural machine translation. ACM Trans. Softw. Eng. Methodol. **28**(4) (2019)
27. Waseem, M., Das, T., Ahmad, A., Liang, P., Fehmideh, M., Mikkonen, T.: ChatGPT as a software development bot: a project-based study (2024). https://arxiv.org/abs/2310.13648
28. Yang, J., et al.: Harnessing the power of LLMs in practice: a survey on ChatGPT and beyond. ACM Trans. Knowl. Discov. Data **18**(6) (2024). https://doi.org/10.1145/3649506
29. Zhang, J., et al.: Repairing bugs in Python assignments using large language models (2022)
30. Zhong, L., Wang, Z., Shang, J.: Debug like a human: a large language model debugger via verifying runtime execution step-by-step (2024). https://arxiv.org/abs/2402.16906

Autonomic Microservice Management via Agentic AI and MAPE-K Integration

Matteo Esposito(✉), Alexander Bakhtin, Noman Ahmad, Mikel Robredo, Ruoyu Su, Valentina Lenarduzzi, and Davide Taibi

University of Oulu, Oulu, Finland
{matteo.esposito,alexander.bakhtin,noman.ahmad,mikel.robredomanero, ruoyu.su,valentina.lenarduzzi,davide.taibi}@oulu.fi

Abstract. While microservices are revolutionizing cloud computing by offering unparalleled scalability and independent deployment, their decentralized nature poses significant security and management challenges that can threaten system stability. We propose a framework based on MAPE-K, which leverages agentic AI, for autonomous anomaly detection and remediation to address the daunting task of highly distributed system management. Our framework offers practical, industry-ready solutions for maintaining robust and secure microservices. Practitioners and researchers can customize the framework to enhance system stability, reduce downtime, and monitor broader system quality attributes such as system performance level, resilience, security, and anomaly management, among others.

Keywords: Agentic AI · Large Language Model · Autonomous · Anomalies · Microservices · MAPE · Remediation · Human-Machine Teaming

1 Introduction

Microservice architecture is an architectural style that structures an application as a collection of small, autonomous services modelled around a business domain [37]. Each microservice is designed to perform a specific function and can be developed, deployed, and scaled independently of other services. This approach contrasts with the traditional monolithic architectures, where the entire application is built as a single, interconnected unit. Nonetheless, according to [32], "microservices" have become an IT buzzword for large enterprise firms. Therefore, tech giants such as Amazon, Azure, and Google promote various services to enable practitioners and researchers to develop, efficiently deploy, and test microservices [33]. However, microservices are likely to have anomalies [36]. The current state-of-the-art solutions focus on anomaly detection of a distributed architecture by combining trace, logs, and metrics, and performing graph-based deep learning for root cause analysis [16,30,40,41].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 105–118, 2026.
https://doi.org/10.1007/978-3-032-04403-7_11

Nonetheless, practitioners still need to spend a lot of manual effort. Moreover, specialized software can suddenly fail, sometimes with far-reaching and chaotic consequences, such as in the recent event involving crowd strike outage[1]. Amidst state-of-the-art, no autonomous solutions exhibit actual agentic behavior [7] that can help practitioners to alleviate from their shoulders the weight of managing such complex systems.

Our approach significantly enhances the resilience and adaptability of microservices environments, addressing faults, emerging threats, and operational challenges. Therefore, our framework can be used to monitor the broader system quality attributes, e.g. performance, resilience, security and anomaly management. Practitioners and researchers can benefit from the following key contributions: (1) We present the first framework that integrates MAPE-K and agentic AI (AAI) concepts, aiming for a robust solution for microservices anomaly management. (2) Introducing the novel concept of an "autonomic threshold", our approach ensures human oversight of high-risk actions in AI-managed systems, balancing autonomy with control [4,39].

Paper Structure. In Sect. 2, we provide a theoretical background for the study. In Sect. 3, we provide motivations for our framework, and in Sect. 4, we present its design. In Sect. 5, we discussed the limitations, and in Sect. 6, the ethical implications. In Sect. 7, we conclude.

2 Background

An AI Agent is a system that can make decisions independently and decide a course of action. In many cases, it emulates human agency [34]. An Agentic AI system can sense its environment, reason through different courses of action, and make decisions to reach specific goals. Furthermore, it should adapt to new information, learn from experience, and respond to changing conditions without the need for human intervention [34]. AAI is proactive; in other words, it initiates actions and does not merely react to explicit commands or input. This is a highly desirable property in many applications where the environment is complex and dynamic, and real-time decision-making and adaptation to constantly changing conditions are required [34].

Recent studies have addressed the daunting question of liability involving AAIs [11], or instead, its sense of agency. More specifically, [11] acknowledges that the agency of algorithmic systems does not alleviate or shift human responsibility for potential algorithmic harm.

In the same vein, [22] highlights that understanding how humans' sense of agency (SoA) might be influenced by perceiving control by an AI and thus may be very useful for AI development in general and the spread of human attitudes more positively toward AI, in particular. More precisely, they highlight the research gap related to AI that adapts to changing human SoA in dynamic settings due to the difficulty in modelling and responding to human SoA in intricate settings.

[1] https://blogs.microsoft.com/blog/2024/07/20/helping-our-customers-through-the-crowdstrike-outage/.

MAPE-K cycle is a framework in autonomic computing for self-managing systems [38]. Its acronym stands for the following words: Monitor, Analyze, Plan, Execute, and Knowledge. The two major subsystems involved are the manag*ed* and manag*ing* systems. In this context, the AAI implements the framework as the managing system. The second, managed system, is an analyzed system composed of microservices. These two systems interact through sensors and actuators. Sensors collect relevant data that serves as input for the monitoring step. On the other hand, actuators modify the managed system based on the instructions from the execution step. The cycle itself includes the following steps:

1. **Monitor:** Collect system and environment data.
2. **Analyze:** Process and interpret the monitored data to identify patterns, trends, and anomalies.
3. **Plan:** Develop strategies or actions to address the issues identified in the analysis phase.
4. **Execute:** Implement the planned actions to manage the system's behaviour.

(*) **Knowledge:** Maintain a repository of data, models, and policies that support the other four activities. Also known as the Knowledge Base (KB).

Donakanti et al. [15] previously demonstrated the feasibility of incorporating LLMs into the MAPE-like cycle. However, their approach has several limitations: the authors merged Analyze and Plan stages into a *Synthesize* stage, thus blurring the boundary between the results of the analysis and planning performed based on these results; they allow the framework to execute actions without human oversight; and lastly, they use a monolithic benchmarking system as the evaluation study, while we envision a more distributed and diverse microservice system as the managed system.

Layers of Microservice-Based Systems. Anomalies in microservices can occur on the different layers of microservice-based systems, each requiring adequate reconstruction [8]. There are typically three different layers where the potential in-time anomaly detection and remediation actions are required.

1. Static Layer. This layer concerns the static components of a microservice, e.g., source code. Static analysis is the analysis of the code base of any particular software project to formally verify the target system's correctness, especially when a set of technical reasoning practices is required [8]. It has long been adopted to support a high-level understanding of legacy monolithic systems in terms of their maintenance and replacement [29]. For microservice-based systems, static analysis can also help the effectiveness of anti-pattern and bad smell detection [10]. In addition, the visualization of the microservice structure based on static analysis can also help in terms of anti-pattern identification [9]. On the other hand, many studies also contributed to the root cause identification of anomalies in microservice architecture [26]. Moreover, continuous delivery and explainability are considered the main challenges of anomaly detection and failure root causes analysis for microservice [35], which can gain support from Agentic AI.

2. Dynamic Layer. Furthermore, dynamic analysis can help reconstruct microservice-based systems on different layers. For example, using the telemetry

data for system dynamic analysis, it is possible to detect the service dependencies and identify potential architecture smells [2,8]. Furthermore, the combination of static and dynamic analysis can facilitate the decomposition of monolithic systems into microservices and detect the anomalies therein [21]. The challenges of adopting dynamic analysis for microservice anomaly detection include the compliance and integration of business logic and the interweaving anti-patterns [2]. Though machine learning techniques have been applied for such a purpose [23], Agentic AI can facilitate the practice of improving accuracy and reducing human labour involvement.

3. Organizational Layer. Moreover, the human aspect, i.e., the organizational structure of microservice projects, is another critical layer where anomalies can occur, especially since, according to "Conway's law" [13], the system architecture can shift, mirroring the project teams' structure [24]. Therefore, the organizational coupling between microservices and between the corresponding teams can be considered as typical anomalies on the organizational layer since the ideal "one microservice per team" status is hard to achieve [3]. To such an end, various context information can facilitate the analysis of developer collaboration and organizational structure optimization [6,25]. However, this aspect is still limitedly explored, while agentic AI can help evaluate the organizational coupling, identify and recommend the optimal way to decouple, and proactively suggest high-performing collaboration relationships. Therefore, we can leverage AAI to facilitate anomaly detection practice in different layers with a specialized MAPE-K loop adopted on each. Introducing new layers, e.g., the energy consumption layer [5], is also possible in the existing framework by adding a corresponding new loop that targets such anomalies. Previous empirical work has demonstrated the potential benefits on the use of MAPE-K in combination with AI, with techniques such as Markov decision processes [27], artificial neural networks [28] and Long-Short Term Memory (LSTM) models [14]. However, research on the use of AI in microservice-based systems remains in early stages [31].

Our work extends MAPE-K with AAI and a human-in-the-loop (HITL). Cleland-Huang et al. [12] proposed the first attempt at inserting HITL in a MAPE-K to address the Human Machine Teaming (HMT) challenge. Their approach details which tasks in each MAPE-K step the human agent can cooperate with the machine. Conversely, our approach focuses on HMT specifically in the execution plan. Our choice is aimed at averting critical failures of the system, but keeping the managing system as autonomous as possible in handling the managed system.

2.1 Ansible

An appropriate number of independent services in a microservices architecture necessitates a corresponding set of automation tools to manage them effectively [37]. Among the most critical automation tasks are **container orches-**

tration (e.g., *Kubernetes*[2]), **cluster management** (e.g., *Docker Swarm*[3]), and **system state management** (e.g., *Ansible*[4]).

Ansible, introduced in 2012 as an open-source automation platform, leverages YAML-based playbooks to declaratively define system configurations. It operates in an **agentless**, **push-based** model over SSH, simplifying deployment across diverse environments. Ansible includes a rich set of built-in modules for managing containers, networks, and services, and also supports the development of custom modules tailored to specific use cases.

3 Motivation

We consider the point of view of a practitioner tasked with managing a complex system composed of multiple microservices in a highly distributed environment. Such microservices would produce a high volume of execution logs, performance metrics, and service call traces made across the system. Already overwhelmed by the amount of data, the practitioner needs to cope with unexpected failures of different components that can occur, rendering the system unstable. To avert such disastrous failures, there is a need for complexity to be handled in a stronger and automated way [15].

Incorporating our new framework into the organization's workflows could ease work and enhance the practitioner's performance. Our idea can provide a stable, self-adaptive and **proactive** framework for managing anomalies in microservices.

Monitoring. The AAI continuously monitors the microservices environment, collecting data from execution logs, performance metrics, and service calls. Using advanced AI algorithms, it analyzes this data in real-time to detect anomalies and potential threats before they escalate into major issues, thus anticipating the failure of the system.

Proactive Fault Management. When anomalies are detected, the Agent not only alerts the practitioner but also provides actionable insights and recommended solutions. This proactive fault management **reduces the time and effort** required to detect and remediate issues by anticipating their occurrence, thus allowing the practitioner to focus on more strategic tasks.

Customizable Solution. Our framework provides a flexible foundation that practitioners can build upon, considering their specific needs. Such adaptability means that the framework can change with the company's needs: whether it is adapting to new services, changing performance standards or new threats, **all without expensive re-engineering**.

Overall, integrating our framework into the company's practices will **transition** the role of a practitioner from **a reactive firefighter** to a **proactive strategist**. Now, the practitioner will primarily be directed by this framework in

[2] https://kubernetes.io/.
[3] https://docs.docker.com/engine/swarm/.
[4] https://www.ansible.com/.

daily routine monitoring and anomaly detection, which leads them to pay attention to higher-level planning and optimization with consequent improvements in resilience and efficiency.

4 Design

This section presents the envisioned framework for autonomous anomaly detection and remediation leveraging Agentic AIs. We designed our framework around the MAPE-K cycle. We describe for each step of the cycle how we contextualize it to address the challenge of autonomous anomaly detection and remediation. Moreover, for each step, we propose the implementation methodology and the relationship with the knowledge base, and summarize the contribution of AAI. Figure 1 displays the diagram showing the interaction between AAI and MAPE-K components.

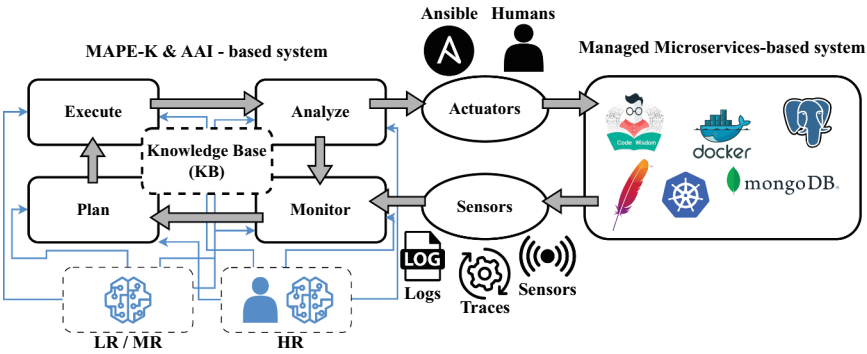

Fig. 1. Diagram on the interaction between AAI and MAPE-K components.

4.1 Monitor

Context. The monitoring stage aims to supervise the development and operation activities of the microservice-based systems by continuously collecting data from multiple layers, e.g., source code, system operation, and organizational structure. The whole process should be conducted efficiently when sufficient data is collected.

Method. Changes in the source code as measured by the critical metrics can be monitored through the updates in the repositories of version control systems by continuously crawling data via public APIs and using customized tools or via CI/CD actions. Open tracing tools can facilitate logging and tracing of the system operations on the dynamic layer.

Knowledge. The knowledge gathered in this stage consists of a large volume of raw data from the different layers of microservice-based systems. Such data

shall be further processed and analyzed for input by the AAI in the later stages. Meanwhile, the output of AAI shall also be continuously monitored.

Agentic AI. The AAI shall facilitate the monitoring stage by selectively collecting useful data and automatically categorizing the data, targeting potential tasks on different layers. Furthermore, the AAI shall also facilitate the coordination of all integrated tools adopted. Human interaction shall be allowed when new metrics are introduced and corresponding monitoring instructions are deployed.

4.2 Analyze

Context. The analysis stage consists of handling the data collected in the monitoring stage, making it processable, producing interpretable outcomes to be added to the KB, and, therefore, considering it in the planning stage to make data-driven decisions. Similarly, the entire process shall maintain the data pipelines of the three layers generated in the previous stage.

Method. The analysis stage initially pre-processes the raw data collected through the monitoring stage. The framework will consider diverse data preprocessing techniques stored in the KB to make the raw data obtained through the suggested three layers analyzable. For instance, we can detect the impact of the implemented changes in the source code through a wide variety of methods based on anomaly detection, such as change point detection techniques for CI performance control, as well as multiple ML algorithms for defect prediction. Furthermore, ML ensemble learning techniques and deep learning techniques can help in fault prediction and localization tasks involving time-series-based continuous data flow, such as log data generated through log tracing tools. In the context of the organizational layer, automated analysis of collaboration relationships through Social Network Analysis has proven suitable for showing developers' collaboration patterns [6,24].

Knowledge. Within the analysis stage, the KB is crucial to obtaining data-driven results. The KB provides the monitored raw data, which is further preprocessed and analyzed. Similarly, the KB is the source from which the AAI chooses and adopts the most suitable analysis techniques based on their goodness of fit for each layer. Furthermore, the KB is the storage where the AAI stores the detected anomalies and potential future predictions to be interpreted in the planning stage. The Agent's continuous learning mechanism should leverage the knowledge stored in the KB in this stage to provide exhaustive data-driven decisions and robust reinforcement learning of the models used in the KB for future analysis cycles.

Agentic AI. The AAI deployed in this stage shall provide the most efficient data analysis pipelines existing in the KB, given the nature of the metrics identified in the data categorization of the monitoring stage. The AAI should calculate the best analysis methods to cope with the potential insights extracted from the data, such as anomaly detection or collaboration optimization. Human interaction should be included when AAI presents low performance on the chosen models and detects anomalies in the integrated analysis pipelines' accuracy results.

4.3 Plan

Context. The planning step is the core of the MAPE-K cycle [20]. In our context, we tailored the tasks for which the plan step is responsible as follows: *Goal Setting*: Define the goals and desired state of the managed system, excluding strategy development in this context; *Resource Allocation*: Decide on resource allocation, scheduling, and roles for plan implementation; *Risk Assessment*: Assess potential risks and impacts, and develop mitigation strategies; *Action Plan Formulation*: Formulate an action plan, often involving human input, as directives for actuators; *Feedback Integration*: Gather feedback from previous cycles or external sources to ensure continuous improvement and adaptation.
Method. The AAI will collect the results of the analysis step, particularly the identified anomalies, from the knowledge base to formulate the action plan. To allow autonomous planning and execution, we design this step, specifically the AI agent, to formulate the plan in a machine-understandable format. For simplicity, we can refer to technologies such as Ansible Playbooks. In our context, risk assessment is crucial to avoid a catastrophic failure. Previous research showed that AI models such as LLMs can perform preliminary security risk analysis [17,18]; therefore, AAI can leverage LLM to evaluate the risk associated with the formulated plan. Moreover, this evaluation is pivotal for the execution step and for selecting the correct type of actuators.

The risk associated with the threshold does not have a consequence on the overall design of the framework. For presentation purposes, we define the following risk level:

Low Risk (LR): High-level actions like parameter tuning and log cleanups that, if incorrect, don't cause system instability.

Medium Risk (MR): Actions affecting system performance, such as increasing virtual memory, reinstating services, and backups. Incorrect actions may cause minor instability but can be recovered.

High Risk (HR): Low-level actions like key distribution, certificate management, service reinstatement, and system upgrades. Incorrect actions can cause significant instability.

According to our proposed sample risk levels, an AAI can perform LR and MR actions freely, directly issuing commands through the CLI actuators. Conversely, we should consider a different approach to HR actions. Allowing an AAI to control the managed system without any "human-in-the-loop" for such actions is risky. Therefore, we should define an"autonomic threshold" (α). Previous works have highlighted the importance of categorizing the"degree of autonomy" within a software system [39], as well as the measurement on the"level of autonomy" of an autonomous system [4]. We propose a threshold based on how many HR actions are associated with each action formulated in the plan. We can also define subtypes of actions and compute a weighted sum of the HR actions based on these subcategories. When the value α is reached, AAI should stop taking actions and alert a human agent to take those on its behalf (Fig. 2).
Knowledge. In the planning step of MAPE-K, the knowledge base is instrumental in providing the relevant information. AAI extracts data points regard-

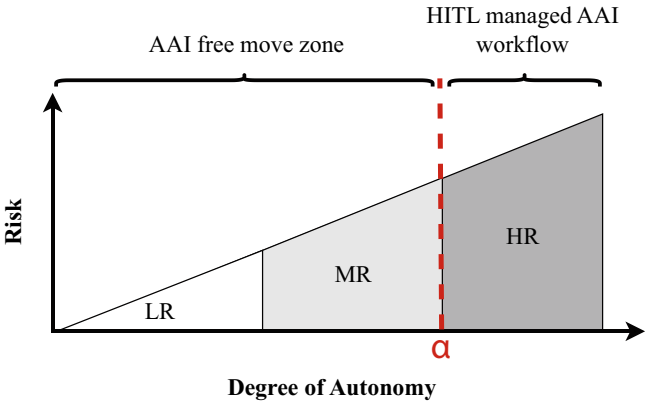

Fig. 2. Overall risk associated framework design.

ing goal setting, resource allocation, risk assessment, action plans, and feedback integration from the KB. In particular, the knowledge base contains data from the analysis stage, which is vital to obtaining information regarding the anomalies and ensuring the action plan is oriented toward efficient remediation of the detected anomalies.

Agentic AI. AAI is vital in considering and testing more than one strategy or action plan. This is done using optimization algorithms and scenario simulations during the planning process. The AAI could rate them according to their potential impact and the likelihood of their success in addressing current and future objectives.

4.4 Execute

Context. The execution stage is where the action plans developed at the planning stage are executed. In this phase, the managed systems are modified to remediate the detected anomalies and bring them to the desired state defined by the KB goals. This phase uses the software actuators to execute the planned actions, and potentially requests inputs and decisions from the human user depending on the value of α.

It is worth noticing that, in our context, **actuators** are of two types: human and software. The framework is designed to be as autonomous as possible, but no current definition of'autonomic threshold' exists, i.e., how to decide when the HITL is needed. Finally, allowing an AAI to fully control the cloud architecture has non-negligible risks that we must address.

Method. The execution step fetches the action plan from the knowledge base. The AAI coordinates the planned actions based on the identified risk levels. The AAI generates the appropriate plan instruction, e.g., the Ansible playbooks, and sends the action commands via the CLI actuators for executing low and medium-risk actions. For high-risk actions, the AAI will alert human operators

when it reaches the autonomic threshold, α, and requires human intervention to execute such actions. Moreover, execution logs and outcomes are monitored and recorded in real-time to provide feedback and populate the knowledge base for the next cycle. We note that, while we describe Ansible within our framework, any comparable automation tool can be used either as a substitute for or in conjunction with Ansible, depending on the system requirements and context.
Knowledge. The knowledge base is the source of action plans and historical data on past executions. The ongoing execution stage updates the KB with logs of success and failure, performance metrics, and deviations compared to expected results. This information is critical to shape future MAPE rounds and, therefore, enhance the autonomic capabilities of the system.
Agentic AI. The AAI must ensure the correct execution of the planned actions, infer the risk level, and support human interaction in the case of HR actions to assess critical decisions before implementation.

5 Limitations

This section presents our approach's challenges, limitations, and possible solutions.

5.1 Monitor

Limitation 1M. Data collected from different layers, such as source code, the system operation, and the organizational structure, is very high in volume and diverse in structure; it may easily overwhelm the system and render the data processing inefficient, leading to a bottleneck. **Solution:** We will provide data filtering and aggregation algorithms during the data collection and KB storage to reduce the amount of data sent for further processing, and use distributed data processing frameworks to handle big data.

5.2 Analyze

Limitation 1A. The produced data volumes are big, which may compromise the accuracy of the anomaly detection and fault prediction models, hence having a high false positive or negative rate. **Solution:** We can use ensemble techniques by combining multiple models to detect anomalies via majority voting. Continually update and retrain such models with new data to enable them to continue learning and responding to the system changes.
Limitation 2A. The analyzed data is usually of a very high dimension, so it is cumbersome to handle and understand; hence, it is very likely to skew the analysis results. **Solution:** Dimensionality Reduction Techniques, such as Principal Component Analysis (PCA) or t-distributed Stochastic Neighbour Embedding (t-SNE), can be used to reduce the dimensionality of the data while retaining relevant information.

5.3 Plan

Limitation 1P. It is hard to know which part of the system should be automated and which would still require human intervention, especially in the case of high-risk actions that may affect the reliability and safety of the system. **Solution:** We propose the adjustable autonomic thresholds that depend on the system performance and the confidence level of the AAI.

5.4 Execute

Limitation 1E. System errors or conflicts can lead to unpredictable outcomes due to the automation at the 'execute' stage, which can make the system unstable. **Solution:** The AAI must be able to perform rollbacks to restore the previously known stable point of the system, or request such an action from the human user.

5.5 General Considerations

Limitation 1G. Catastrophic AAI actions can occur due to unforeseen events and result in critical system failures. Such is the situation when **AAI can keep remediating an issue caused by itself**, causing a degrading feedback loop. **Solution:** Our implementation will include checks for such degradation loops. Moreover, the AAI should create an incident response plan that covers automatic rollback procedures and human intervention protocols for detecting and mitigating such events by restoring the system to its former stable condition. **Limitation 2G** There is no historical data on autonomic AAI-based anomaly detection and remediation systems for microservices. **Solution:** Our proposed approach faces novel challenges no prior study has faced. Thus, the future empirical validation will lay the cornerstone of AAI for microservice anomaly detection and remediation.

6 Ethical Implications

We live in a fast-changing world. Until recently, AI mostly categorized things, but now it enhances human imagination. According to [19], SE researchers and practitioners should not "look away." Leveraging AI in the context of software engineering, although not involved with human data directly, still carries ethical risks and implications [19]. We identified the following ethical considerations that we should address when implementing and deploying the proposed framework:
Accountability and Transparency. With AAI making autonomous decisions, clear accountability for errors or unintended consequences is crucial. We will consider implementing transparency mechanisms like detailed logging and decision tracing to allow human agents to understand AI decisions.
Bias and Fairness. AI decisions are only as fair as the data and algorithms used within. We will design the feedback loop in the KB to periodically audit

data and performance to flag and reduce any biases. This issue links back to the scarce data constraint we highlighted earlier.

Privacy and Data Security. Continuous data collection and analysis raise privacy and security concerns, especially when considering the organizational layer. We will consider robust data encryption and access control measures and obey data protection regulations to prevent privacy and security breaches on both the managing and managed systems.

7 Conclusions

In this vision paper, we presented our framework, which is the first to unify the autonomic computing theory and the MAPE-K cycle with the novel scenarios enabled by the agentic AI in the context of microservice architecture. Our framework allows practitioners to focus on higher-level microservice system design and management while only being in the loop of monitoring and remediation for making critical decisions which cannot be responsibly allocated to agentic AI. On the other hand, researchers can build on our pioneering study and the results of our future empirical validation to enhance current autonomic systems. Our future research effort will focus on implementing the proposed framework, empirically validating it, and overcoming the highlighted limitations.

Acknowledgments. This work has been funded by the Research Council of Finland (grants n. 359861 and 349488 - MuFAno), by Business Finland (grant 6GSoft [1]), and FAST, the Finnish Software Engineering Doctoral Research Network, funded by the Ministry of Education and Culture, Finland.

References

1. Akbar, M.A., et al.: 6GSoft: software for edge-to-cloud continuum. In: 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 499–506. IEEE Computer Society, Los Alamitos, CA, USA, August 2024
2. Al Maruf, A., et al.: Using microservice telemetry data for system dynamic analysis. In: International Conference on Service-Oriented System Engineering (SOSE), pp. 29–38 (2022)
3. Amoroso d'Aragona, D., Li, X., Cerny, T., Janes, A., Lenarduzzi, V., Taibi, D.: One microservice per developer: is this the trend in OSS? In: Papadopoulos, G.A., Rademacher, F., Soldani, J. (eds.) ESOCC 2023. LNCS, vol. 14183, pp. 19–34. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-46235-1_2
4. Antsaklis, P.: Autonomy and metrics of autonomy. Annu. Rev. Control. **49**, 15–26 (2020)
5. Araújo, G., et al.: Energy consumption in microservices architectures: a systematic literature review. IEEE Access (2024)

6. Bakhtin, A., Li, X., Taibi, D.: Temporal community detection in developer collaboration networks of microservice projects. In: Galster, M., Scandurra, P., Mikkonen, T., Oliveira Antonino, P., Nakagawa, E.Y., Navarro, E. (eds.) ECSA 2024. LNCS, vol. 14889, pp. 174–182. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-70797-1_11
7. Brumani, T.: Microservices-based autonomous anomaly detection for mobile network observability (2022)
8. Cerny, T., et al.: Microservice architecture reconstruction and visualization techniques: a review. In: 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE), pp. 39–48. IEEE (2022)
9. Cerny, T., et al.: Microvision: static analysis-based approach to visualizing microservices in augmented reality. In: 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE), pp. 49–58. IEEE (2022)
10. Cerny, T., et al.: Catalog and detection techniques of microservice anti-patterns and bad smells: a tertiary study. J. Syst. Softw. **206**, 111829 (2023)
11. Chan, A.E.A.: Harms from increasingly agentic algorithmic systems. In: Conference on Fairness, Accountability, and Transparency, FAccT 2023, pp. 651–666 (2023)
12. Cleland-Huang, J., et al.: Extending MAPE-K to support human-machine teaming. In: Symposium on Software Engineering for Adaptive and Self-Managing Systems, pp. 120–131 (2022)
13. Conway, M.E.: How do committees invent. Datamation **14**(4), 28–31 (1968)
14. De Sanctis, M., Muccini, H., Vaidhyanathan, K.: Data-driven adaptation in microservice-based IoT architectures. In: 2020 IEEE International Conference on Software Architecture Companion (ICSA-C), pp. 59–62. IEEE (2020)
15. Donakanti, R., Jain, P., Kulkarni, S., Vaidhyanathan, K.: Reimagining self-adaptation in the age of large language models. In: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), pp. 171–174. IEEE (2024)
16. Esposito, M., et al.: Generative AI for software architecture. Appl. Trends Challenges Future Dir. arXiv preprint arXiv:2503.13310 (2025)
17. Esposito, M., Palagiano, F.: Leveraging large language models for preliminary security risk analysis: a mission-critical case study, pp. 442–445 (2024)
18. Esposito, M., Palagiano, F., Lenarduzzi, V., Taibi, D.: On large language models in mission-critical it governance: are we ready yet? In: ICSE-SEIP 2025 (2024)
19. Johnson, B., Menzies, T.: Ethics: why software engineers can't afford to look away. IEEE Softw. **41**(1), 142–144 (2024)
20. Kephart, J., et al.: The vision of autonomic computing. Computer **36**(1), 41–50 (2003)
21. Krause, A., et al.: Microservice decomposition via static and dynamic analysis of the monolith. In: 2020 IEEE International Conference on Software Architecture Companion (ICSA-C), pp. 9–16. IEEE (2020)
22. Legaspi, R., et al.: The sense of agency in human-AI interactions. Knowl.-Based Syst. **286**, 111298 (2024)
23. Li, M., et al.: Microservice anomaly detection based on tracing data using semi-supervised learning. In: 4th International Conference on Artificial Intelligence and Big Data, pp. 38–44 (2021)
24. Li, X., et al.: Analyzing organizational structure of microservice projects based on contributor collaboration. In: 2023 IEEE International Conference on Service-Oriented System Engineering (SOSE), pp. 1–8. IEEE (2023)

25. Li, X., et al.: Toward collaboration optimization in microservice projects based on developer personalities. In: International Conference on Software Architecture (2024)
26. Ma, M., et al.: ServiceRank: root cause identification of anomaly in large-scale microservice architectures. IEEE Trans. Dependable Secure Comput. **19**(5), 3087–3100 (2021)
27. Magableh, B., Almiani, M.: A self healing microservices architecture: a case study in docker swarm cluster. In: Barolli, L., Takizawa, M., Xhafa, F., Enokido, T. (eds.) AINA 2019. AISC, vol. 926, pp. 846–858. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-15032-7_71
28. Nguyen, P., Nahrstedt, K.: MONAD: self-adaptive micro-service infrastructure for heterogeneous scientific workflows. In: 2017 IEEE International Conference on Autonomic Computing (ICAC), pp. 187–196. IEEE (2017)
29. Papotti, P.E., et al.: Reducing time and effort in legacy systems reengineering to MDD using metaprogramming. In: 2012 ACM Research in Applied Computation Symposium, pp. 348–355 (2012)
30. Pham, L., Zhang, H., Ha, H., Salim, F., Zhang, X.: RCAEval: a benchmark for root cause analysis of microservice systems with telemetry data. arXiv:2412.17015, February 2025
31. Pimentel, E., Pereira, W., Maia, P.H.M., Cortés, M.I., et al.: Self-adaptive microservice-based systems-landscape and research opportunities. In: 2021 International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp. 167–178. IEEE (2021)
32. Raj, V., et al.: Assessing the impact of migration from SOA to microservices architecture. SN Comput. Sci. **4**(5), 577 (2023)
33. Sampaio, A.R., et al.: Supporting microservice evolution. In: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 539–543. IEEE (2017)
34. Shavit, Y., et al.: Practices for governing agentic AI systems. Research Paper, OpenAI, December 2023
35. Soldani, J., et al.: Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: a survey. ACM Comput. Surv. (CSUR) **55**(3), 1–39 (2022)
36. Souppaya, M., Morello, J., Scarfone, K.: Application container security guide. Technical report. SP 800-190, National Institute of Standards and Technology (NIST) (2017)
37. Taibi, D., et al.: On the definition of microservice bad smells. IEEE Softw. **35**(3), 56–62 (2018)
38. Weyns, D., et al.: On patterns for decentralized control in self-adaptive systems. In: de Lemos, R., Giese, H., Müller, H.A., Shaw, M. (eds.) Software Engineering for Self-Adaptive Systems II. LNCS, vol. 7475, pp. 76–107. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-35813-5_4
39. Wright, S.A.: Measuring DAO autonomy: lessons from other autonomous systems. IEEE Trans. Technol. Soc. **2**(1), 43–53 (2021)
40. Yu, G., Chen, P., Li, Y., Chen, H., Li, X., Zheng, Z.: Nezha: interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 553–565 (2023)
41. Zhang, C., et al.: DeepTraLog: trace-log combined microservice anomaly detection through graph-based deep learning. In: 44th International Conference on Software Engineering, pp. 623–634 (2022)

Application of AI in Software Architecture Development at Volvo Construction Equipment

Nils Johansson[1(✉)] and Mauro Caporuscio[2]

[1] SW Systems AAH Tech Development, Volvo Construction Equipment, Braås, Sweden
nils.johansson.3@volvo.com
[2] Department of Computer Science and Media Technology, Linnaeus University, Växjö, Sweden
mauro.caporuscio@lnu.se

Abstract. As software systems grow in complexity, traditional development methods increasingly struggle to meet demands for scalability, agility, and efficiency. In this context, software architecture plays a central role in maintaining system quality and long-term adaptability. However, SA development faces persistent challenges such as bridging loosely defined requirements with design, managing architectural knowledge across distributed teams, and preserving structural integrity in agile settings. At Volvo Construction Equipment, Artificial Intelligence is being investigated as a means to support and enhance architectural work. Specifically, AI is applied to automate labour-intensive tasks, support design decision-making, and document architectural rationale to mitigate erosion. This paper presents four use cases at Volvo Construction Equipment, including generating Architecture Decision Records, mapping requirements to modules, supporting component design, and evaluating generative AI for module classification. These efforts illustrate AI's potential to act as a co-pilot in the software architecture process—improving consistency, traceability, and decision support. By sharing our approach and early experiences, this paper contributes to the broader discourse on integrating AI into industrial software architecture practice.

Keywords: Software Architecture · AI · AI in Software Engineering

1 Introduction

As software systems grow in complexity, the role of software architecture (SA) becomes increasingly critical—not only in ensuring maintainability and scalability, but also in enabling rapid, iterative development [12]. At Volvo Construction Equipment (VCE), agile and distributed development adds complexity to the architecting process. Architects must often make decisions with incomplete or evolving information, which increases the risk of architectural erosion—i.e., the gradual degradation of the system's intended structure and quality over time [4].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 119–126, 2026.
https://doi.org/10.1007/978-3-032-04403-7_12

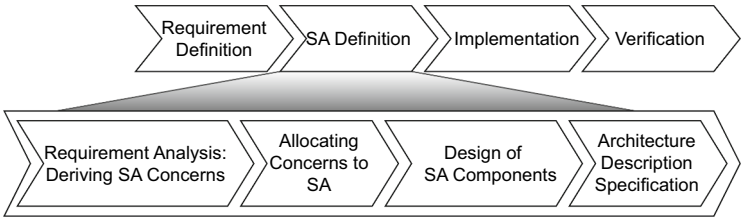

Fig. 1. Outline of software development process at VCE.

Recent advances in Artificial Intelligence (AI), particularly in natural language processing and generative models, offer new opportunities to support these architectural tasks. At VCE, we are exploring how AI can help mitigate known challenges by enhancing decision-making, generating design artifacts, and supporting knowledge management.

This position paper presents our experiences, challenges, and preliminary results in applying AI to SA development. The layout of the paper is as follows: In Sect. 2, we outline the software development process at VCE. In Sect. 3 we describe observed key challenges of SA development at VCE. In Sect. 4, we review opportunities of AI in SA development. Section 5 describes how we aim to utilize AI for SA at VCE. Finally, we state future plans in Sect. 6.

2 The Software Development Process at VCE

At VCE in Braås, Sweden, the development of embedded software for the mechatronic system of articulated haulers involves over 60 personnel, including system and software engineers, as well as Hardware-in-the-Loop (HIL) rig and in-machine testers. Many work in Braås, while others work remotely, requiring virtual communication across time zones. Software development is feature-based, with each feature being defined, implemented, and verified. The process follows the Scaled Agile Framework (SAFe)[1], with 10-week iterations comprising three-week sprints and a planning week. Figure 1 illustrates how feature-based development is structured at VCE. Cross-functional scrum teams, including system engineers, software engineers, and testers, work concurrently on requirements, software, and test specifications. Each team includes a software architect responsible for developing and specifying the software architecture of features. Development often begins before requirements are fully defined, relying on feature scopes. The development process starts with software architecture definition, followed by implementation and integration, and concludes with verification through HIL rig and in-machine testing.

To support software architecting, a product-line reference architecture is utilized [10]. This includes architectural modules and components with defined dependencies, ensuring quality demands such as reusability, modifiability, and

[1] https://framework.scaledagile.com/.

modularity are met. Each module and component is detailed in text, outlining its purpose, scope, and design considerations. Architecture development begins with requirement analysis, where architects enrich functional requirements to identify architectural concerns. These concerns are mapped to architectural modules within the current or reference architecture. If the reference architecture lacks support for a feature, new solutions are developed. Once concerns are mapped, component interactions are designed, often using Unified-Modelling-Language sequence diagrams, and the updated software component structure is specified, such as by class diagrams.

3 Challenges

This section describes two significant challenges we face during SA development at VCE.

3.1 Bridging Requirements and Software Architecture

Feature development begins with loose functionality descriptions, with requirements defined concurrently. To derive architectural concerns, architects rely on specialized knowledge, which is scarce within the organization, and while documentation is available, it requires extensive manual analysis. Documentation that may be useful includes laws and regulations, an operator's manual, and descriptions of "Hauler functions," detailing the mechatronic systems of the products. Mapping concerns to modules is complex due to limited architectural descriptions, often requiring code as documentation or comparison with reference architecture modules. Interpreting functional requirements into architectural concerns and defining object-oriented properties is challenging. Key object-oriented principles, such as suitable abstractions and encapsulation, must be followed, raising questions like "What information should the interface functions expose?" for a given concern.

Related Work – These internal challenges are mirrored in the literature, which highlights similar issues faced across the software industry of translating loosely defined requirements into architectural concerns. Hofmeister et al. [8] highlight the lack of systematic methods, noting architects often rely on intuition. Our reliance on internal knowledge and manual documentation analysis aligns with their observations about the hidden nature of architectural information and varying stakeholder interpretations. Mapping concerns to existing modules is difficult, especially with limited documentation, mirroring the challenge of identifying Architecturally Significant Requirements as noted by Capilla et al. [7]. Additionally, expressing object-oriented principles based on functional concerns relates to broader issues in handling non-functional requirements like performance and security [8]. Our experience of requirements evolving alongside architectural modelling reflects the co-evolution of requirements and architecture, a common reality in modern software development [8].

3.2 Software Architecting in a Global and Agile Environment

We advocate for planning architecture development well ahead of software delivery deadlines, ideally during the sprint before implementation begins. However, plans often change mid-increment, necessitating last-minute architectural work. Short deadlines force architects into rapid, potentially flawed design decisions, leading to architectural erosion, often manifesting as additional dependencies, risking a "big-ball-of-mud" structure. The urgency to deliver software often results in neglected documentation of design rationales, complicating future feature development as architects struggle to understand past decisions. In globally distributed teams, constrained by time and virtual communication, this can lead to improper assumptions and uninformed design choices. These challenges stem from the short deadlines inherent in agile development, which prioritizes fast delivery and communication over documentation. Integrating a comprehensive architecture development phase within agile practices remains challenging.

Related Work – The tension between traditional software architecture and agile methods is well-documented. Agile's focus on quick delivery and minimal documentation often conflicts with the need for upfront planning and rationale capture [13]. Our experiences of architectural erosion and undocumented decisions reflect broader concerns about maintaining architectural integrity under agile pressures [13]. Architects face challenges in conveying architectural ideas within distributed, autonomous teams, particularly with virtual and asynchronous communication [1]. This supports our observations of uninformed decisions due to limited interactions. Additionally, documentation is often deprioritized in agile settings, leading to knowledge gaps in future development [11,13]. These challenges highlight the need for improved architectural knowledge sharing, and lightweight yet effective documentation practices in complex software development using agile methods.

4 Opportunities of AI in Software Architecture Development

As the challenges outlined in the previous section illustrate, the complexities of software architecture development require innovative solutions to enhance efficiency and effectiveness. AI technologies offer promising approaches to these challenges. This section presents key applications in SA development.

AI-Assisted Decision Making – AI, especially Large Language Models (LLMs), supports architects by generating design solutions and recommending patterns based on requirements. These models serve as co-pilots during the initial design phase, helping to produce architectural patterns and propose system decompositions, thereby reducing cognitive load and enhancing the quality and maintainability of systems [2,5,14].

Architectural Evaluation and Trade-Off Analysis – Beyond generation, AI contributes significantly to decision analysis through architectural evaluation and trade-off analysis. Techniques such as knowledge graphs and multi-agent reasoning systems are employed to analyse Architecture Decision Records (ADRs) [9]. Multi-objective reinforcement learning has emerged as a powerful method to evaluate competing design objectives—such as performance, maintainability, cost, and security—when selecting or refining architectural options [2]. These techniques enable a quantitative comparison of alternatives, guiding architects toward more informed and optimized decisions.

Architectural Knowledge Management – AI enhances Architecture Knowledge Management (AKM) by automating the extraction and documentation of architectural knowledge dispersed across various sources, such as source code and documentation [15]. This automation supports continuous generation and documentation of architectural knowledge, significantly reducing manual effort. AI-powered systems also improve querying and traceability, leveraging information retrieval and generative models to answer architecture-related queries and establish links between decisions and implementation artifacts. This fosters a more agile, informed, and sustainable architecture development process.

Potential Pitfalls of Applying AI in SA – While AI brings substantial promise to software architecture, it also introduces critical challenges that must be addressed to ensure reliable and trustworthy integration. A primary concern is the quality and trustworthiness of AI-generated output [3]. Many AI models, particularly LLMs, are prone to hallucinations—producing plausible-sounding but incorrect or irrelevant architectural suggestions [6]. Such outputs require manual validation, often negating the intended productivity gains and risking flawed designs if adopted uncritically [2]. Another key issue lies in the relationship between human users and AI systems. Architectural decision-making remains highly contextual, requiring expert interpretation and judgement. However, over-reliance on AI tools may lead users to accept suggestions without sufficient scrutiny, undermining critical evaluation [6]. Crafting effective prompts and reviewing AI output demands significant expertise, yet this process can be opaque and inconsistent. Moreover, there is a risk that reliance on AI could erode architectural skills, limiting a team's long-term ability to maintain or evolve systems independently [6]. These concerns underline the necessity of human oversight, robust evaluation frameworks, and explainable AI mechanisms to ensure the responsible and effective use of AI in architecture development.

5 Utilizing AI for SA at VCE

Building on the challenges and opportunities identified in the previous sections, we at VCE are committed to leveraging AI to enhance our software architecture development process. We outline four use cases where AI supports architects and improves development practices. These initiatives aim to streamline our processes and maintain high-quality architectural standards.

Use Case 1: Generating ADRs from Development Data – To support AKM, we aim to leverage AI to generate ADRs from both historical and upcoming development efforts. This aligns with the AI-enhanced AKM discussed in Sect. 4, where AI automates the extraction and documentation of architectural knowledge, reducing manual effort [15]. By generating documentation of past decisions, AI can help mitigate the challenge of architectural erosion and undocumented decisions described in Sect. 3.2, ensuring that design rationales are preserved for future reference.

Use Case 2: Deriving Architectural Concerns from Functional Requirements – AI will aid in deriving architectural concerns from functional requirements by utilizing available documentation, which includes sources such as hauler function descriptions, laws and regulations, the operator's manual, and past decisions of similar features using ADRs. As highlighted in Sect. 4, AI technologies can generate design solutions from textual requirements, thereby streamlining the process of identifying architectural concerns [2,5,14]. This approach addresses the challenge of bridging loosely defined requirements with architectural design, as noted in Sect. 3.1, by providing a systematic method to interpret and map requirements to architectural concerns.

Use Case 3: Mapping Concerns to Modules – The architectural concerns shall be mapped to architectural modules using proposals by AI. Section 4 discusses AI's role in architectural evaluation and trade-off analysis, which can assist in mapping concerns to modules by analysing module descriptions and proposing optimal mappings [9]. This use case helps overcome the complexity of mapping concerns to modules, a challenge identified in Sect. 3.1, by leveraging AI to automate and optimize the mapping process.

We perform a preliminary evaluation of the capabilities of a GenAI model to complete this task. Using the model *meta-llama-3-3-70b-instruct* through prompting, we input five functional text requirements and instructions to find which of 40 available modules of the reference architecture should be involved in realizing the requirement. Figure 2 shows an application where the interface to the AI model is implemented. The five functional requirements evaluated include emergency stopping, acceleration control, headlight control, dump body control, and wiper control—selected as representative cases of articulated hauler functionality. For these requirements, we can determine a ground truth based on past feature development. Each requirement should be mapped to several modules. Figure 2 shows an excerpt of a proposal made by the AI model, after being prompted to find modules that should be involved in realizing a functional requirement. Table 1 describes the result of allocating each requirement in terms of classification accuracy, as well as the mean accuracy for all requirements.

Use Case 4: Developing Component Designs – AI will utilize components in the reference and current architecture to propose designs that fulfil input requirements. As described in Sect. 4, AI can assist in developing component designs by integrating architectural principles and object orientation, enhancing the quality

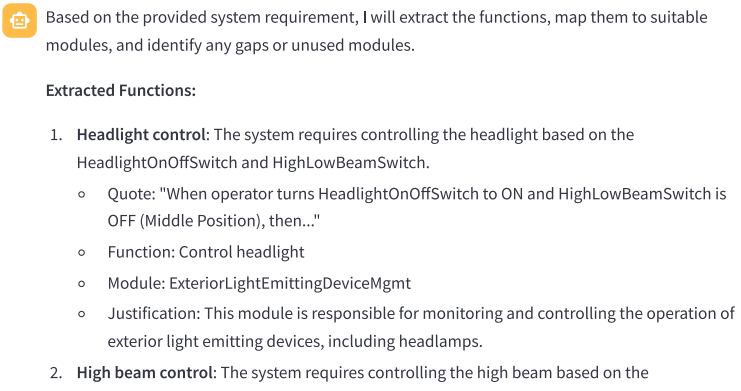

Fig. 2. Allocating architectural concerns to modules using AI.

Table 1. Preliminary results of mapping requirements to modules.

Requirement	Headlight	Dump Body	E-stop	Accel. Ctrl.	Wiper Ctrl.
Accuracy	1.0	1.0	1.0	0.5	0.5
Mean Accuracy	0.8				

and maintainability of systems [2,5,14]. This use case addresses the challenge of rapid design decisions and architectural erosion in a global and agile environment, as mentioned in Sect. 3.2, by providing AI-generated insights that support informed and sustainable design choices.

6 Future Work

A prototype software application is under development that shall integrate all use cases, with the exception of generating ADRs from historical data, which will be handled as a background process and does not require a dedicated interface. We especially focus on developing a user interface that facilitates effective collaboration between the user and AI. Each use case is designed to integrate AI-generated insights transparently, ensuring that decisions are justified and traceable. This approach mitigates the risk of deskilling but fosters a learning environment for architects.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Angelov, S., Meesters, M., Galster, M.: Architects in scrum: what challenges do they face? In: Tekinerdogan, B., Zdun, U., Babar, A. (eds.) Software Architecture,

pp. 229–237. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-48992-6_17
2. Bucaioni, A., Weyssow, M., He, J., Lyu, Y., Lo, D.: Artificial intelligence for software architecture: literature review and the road ahead. In: 2030 Software Engineering - 2025, June 2025
3. Chen, J., Zacharias, J.: Design principles for collaborative generative AI systems in software development. In: Mandviwalla, M., Söllner, M., Tuunanen, T. (eds.) Design Science Research for a Resilient Future, pp. 341–354. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-61175-9_23
4. de Silva, L., Balasubramaniam, D.: Controlling software architecture erosion: a survey. J. Syst. Softw. **85**(1), 132–151 (2012). Dynamic Analysis and Testing of Embedded Software
5. Díaz-Pace, J.A., Tommasel, A., Capilla, R.: Helping novice architects to make quality design decisions using an LLM-based assistant. In: Galster, M., Scandurra, P., Mikkonen, T., Oliveira Antonino, P., Nakagawa, E.Y., Navarro, E. (eds.) Software Architecture, pp. 324–332. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-70797-1_21
6. Esposito, M., et al.: Generative AI for software architecture. Applications, Trends, Challenges, and Future Directions. ArXiv abs/2503.13310 (2025)
7. Gillani, M., Niaz, H.A., Ullah, A.: Integration of software architecture in requirements elicitation for rapid software development. IEEE Access **10**, 56158–56178 (2022)
8. Grünbacher, P., Egyed, A., Medvidovic, N.: Reconciling software requirements and architectures with intermediate models. Softw. Syst. Model. **3**(3), 235–253 (2004)
9. Narváez, D., Battaglia, N., Fernández, A., Rossi, G.: Designing microservices using AI: a systematic literature review. Software **4**(1) (2025)
10. Schroeder, J., Holzner, D., Berger, C., Hoel, C.J., Laine, L., Magnusson, A.: Design and evaluation of a customizable multi-domain reference architecture on top of product lines of self-driving heavy vehicles - an industrial case study. In: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering. vol. 2, pp. 189–198 (2015)
11. Sievi-Korte, O., Richardson, I., Beecham, S.: Software architecture design in global software development: an empirical study. J. Syst. Softw. **158**, 110400 (2019)
12. Wan, Z., Zhang, Y., Xia, X., Jiang, Y., Lo, D.: Software architecture in practice: challenges and opportunities. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, pp. 1457–1469. Association for Computing Machinery, New York, NY, USA (2023)
13. Yang, C., Liang, P., Avgeriou, P.: A systematic mapping study on the combination of software architecture and agile development. J. Syst. Softw. **111**(C), 157–184 (2016)
14. Zhang, Y., Li, R., Liang, P., Sun, W., Liu, Y.: Knowledge-based multi-agent framework for automated software architecture design. ArXiv abs/2503.20536 (2025)
15. Zhou, X., et al.: Using LLMs in generating design rationale for software architecture decisions. ACM Trans. Softw. Eng. Methodol. (2025)

Towards Benchmarking Design Pattern Detection Under Obfuscation: Reproducing and Evaluating Attention-Based Detection Method

Manthan Venkataramana Shenoy(✉) and Andreas Rausch

Institute for Software and Systems Engineering, Technische Universität Clausthal, Clausthal-Zellerfeld 38678, Germany
{manthan.venkataramana.shenoy,andreas.rausch}@tu-clausthal.de

Abstract. This paper investigates the semantic robustness of attention-based classifiers for design pattern detection, particularly focusing on their reliance on structural and behavioral semantics. We reproduce the state-of-the-art attention-based design pattern detection approach using learning based classifiers–and evaluate its performance under obfuscation. To this end, we curate an obfuscated version of the original Java Corpus used in the attention-based approach, where the name-based syntactic identifiers such as class names, method names and string literals in print statements and comment blocks are replaced while preserving control flow, inheritance and code logic. Our findings reveal that these trained classifiers depend significantly on superficial syntactic features, leading to substantial misclassification when such cues are removed through obfuscation. This work highlights the need for more robust detection tools capable of capturing deeper semantic meanings in source code, as well as the need for semantically grounded benchmark. We propose our curated obfuscated java subset (containing 34 Java source files) as a reusable proof-of-concept benchmark for evaluating state-of-the-art design pattern detectors on their true semantic generalization capabilities.

Keywords: Design Pattern Detection · Obfuscation · Classification · Transformer Architecture · Machine Learning

1 Introduction

Software quality—particularly maintainability and understandability—is a core objective of software engineering. Rana et al. [12] performs an empirical analysis demonstrating that the introduction of Gang of Four (GoF) [8] design patterns yields significant gains in maintainability metrics compared to pattern-free code, while Al-Obeidallah et al. [5] report that classes embodying design patterns exhibit maintainability but also higher understandability, thereby reducing code-comprehension effort relative to non-pattern classes.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 127–140, 2026.
https://doi.org/10.1007/978-3-032-04403-7_13

Despite these benefits, production code rarely contains explicit pattern documentation, forcing developers to spend up to 58% of their time on program comprehension alone in [23] and contributing to software maintenance effort that can consume up to 70% of lifecycle costs [17]. To bridge this gap between pattern theory and practice, the research community has developed several detection techniques that recover architectural intent from legacy systems without relying on explicit documentation.

Early automated detection methods relied on structural matching of code artifacts against rigid templates. Techniques such as similarity scoring between program and pattern graphs in [20] and metrics-based detection techniques in [13] potentially located canonical pattern shapes but proved brittle under minor code variations, yielding high false-positive rates. These structure-focused methods lacked the capacity to capture the semantic or behavioral nuances that distinguish, for example, two patterns with identical class hierarchies but differing runtime interactions.

To handle implementation variants as present in real-world systems, reasoning-based frameworks incorporate fuzzy and constraint inference: Niere [16] employ fuzzy graph-rewrite rules to allow inexact matches, Rasool et al. [18] recover patterns via annotations, regular expressions, and SQL queries over intermediate code representations, and Guéhéneuc et al. [10] uses explanation-based constraint programming to recognize design-motif variants semi-automatically using their tool called DeMIMA. While these approaches increase recall on variant implementations, they depend on expert-tuned parameters and manual effort.

To reduce manual rule-crafting, the field shifted toward machine-learning classifiers. Feature-based detectors such as DPD_F [15] combine Word2Vec embeddings with ensemble classifiers to achieve over 80% precision and recall, requiring substantial labeled data.

Recently, attention-based transformer [21] architectures were proposed to capture deeper contextual semantics and behavioral logic. Mzid et al. [14] introduce DPD_{Att} detector, which first generates contextualized embeddings of code tokens and then applies multi-head attention to identify semantically salient relationships among classes, methods, and fields. This context-aware mechanism enables DPD_{Att} detector to generalize over real world software systems which they used from [15] and augmented to having a 1645 Java files spanning 13 GoF patterns, achieving 86% precision and recall.

In our analysis of the *DPD_{Att} Corpus*, we identified pervasive **label leakage**: syntactic cues i.e., pattern names appear directly in class names (e.g., "SingletonManager", "ObserverClient"), as well as in package names, comment blocks, and string literals. Prior work has shown that such syntactic cues can dominate model decisions—Gao et al. [9] demonstrate that renaming as few as one identifier in source code can cause neural code comprehension models to produce completely irrelevant outputs, despite unchanged program semantics. Similarly, Dunlop et al. [7] observe that deep-learning detectors for design patterns misclassify non-pattern code when pattern terms appear in identifiers.

Despite these concerns, current evaluation methods in pattern detection often overlook this weakness. Performance is typically measured using datasets that include these syntactic signals, leaving unanswered whether detectors like DPD_{Att} detector truly understand the architectural semantics of design patterns. This emphasized the importance of developing a benchmark that maintains structural and behavioral logic while removing these syntactic signals. An effective benchmark should test if models can generalize across variations in naming and documentation, as commonly found in real-world codebases.

To address this, we propose a validation framework focused on semantic robustness. Our method involves systematically obfuscating a portion of the DPD_{Att} corpus, eliminating all identifier-based signals while keeping the executable logic intact. This framework offers a structured way to determine if detection models genuinely learn the semantics of design patterns or simply overfit to token-level features.

To guide our research, we propose the following hypothesis and questions:

Research Hypothesis: Systematic code obfuscation that removes syntactic signals while preserving structural and behavioral semantics can reveal whether design pattern detectors, such as DPD_{Att}, truly capture semantic understanding.

- **RQ1**: How does the performance of machine learning–based detectors such as DPD_{Att} degrade when syntactic signals are removed?
- **RQ2**: To what extent can a systematically obfuscated, 13-pattern subset of the *DPD_{Att} Corpus* serve as a valid and reusable benchmark for testing the semantic robustness of design pattern detectors?

The remainder of this paper is organized as follows: Sect. 2 outlines the methodology, Sects. 3–6 describe the experimental setup and results, and Sect. 7 presents the analysis and discussion, followed by conclusion and future work in Sect. 8.

2 Methodology

This section presents the methodology adopted in this paper, starting with a recap of the DPD_{Att} [14] approach described in Subsect. 2.1 and then providing a high-level overview of the experimental workflow shown in Fig. 1 to perform comparative analysis of the pre-trained DPD_{Att} classifiers under original and systematic name-based code obfuscation approach as outlined in Subsect. 2.2.

2.1 Intro to the DPD_{Att} Approach

The DPD_{Att} (Attention-based method for design pattern detection) [14] approach employs supervised learning classifiers designed to automatically detect the design patterns at a Java class-level granularity. It leverages attention-based code embedding model to automatically generate the feature vectors from the Java source files. These feature vectors are then used to train different classifiers for pattern classification.

The original DPD_{Att} workflow involved:

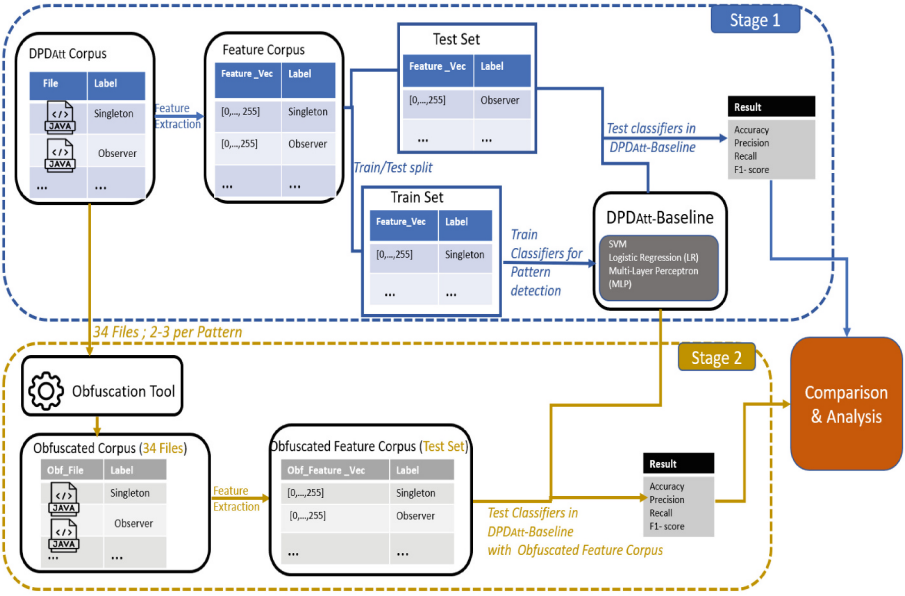

Fig. 1. Overview of our experimental workflow

- Curating a large Java corpus called *DPD$_{Att}$ Corpus* (containing 1645 Java source files) that covers 13 Gang of Four (GoF) [8] design pattern labels across the Creational (e.g., Singleton, Factory Method,etc.), Structural (e.g., Adapter, Decorator,etc.), and Behavioral (e.g., Observer, Strategy,etc.) categories and also includes non-pattern files with "Unknown" labels to assess classifier generalization.
- Automatic Transformation of each Java source file into fixed-length feature vector (300-dimensional) using transformer-based code Large language Model(LLM), they specifically use codeT5+ (110M parameter) [22] an attention-based encoder that captures statically syntactic and semantic code features.
- Training of multiple classifiers (such as SVM, Logistic Regression, Multi Layer Perceptron) for pattern classification using these feature vectors.
- Evaluating these trained classifiers to compare with state-of-art-approaches using classification metrics such as precision, recall and F1-score.

2.2 Description of Our Experimental Workflow

Figure 1 illustrates the complete workflow of our experimental setup, which is structured into **two main stages**: (1) training and evaluating the baseline design pattern classifiers mentioned in Subsect. 2.1 (we refer as DPD$_{Att}$-Baseline), and (2) evaluating the semantic robustness of these classifiers under systematic code obfuscation.

In **Stage (1)**, We begin with the reusing the DPD_{Att} *Corpus* mentioned in Subsect. 2.1, is a Java corpus consisting of 1645 Java source files, each annotated with one of the 13 GoF design patterns labels and "Unknown" label. In the feature extraction step, we use the CodeT5+ embedding model to generate fixed-length vector representations for each source files that capture both syntactic and semantic properties of the code. This yields a *Feature Corpus*, a dataset of feature vectors labeled with their corresponding pattern labels. This corpus is split into *Train set* and *Test set*. The training portion is used to train a set of supervised classifiers—SVM, Logistic Regression, and MLP—constituting the DPD_{Att}*-Baseline* is explained in Sect. 4. The evaluation of these classifiers on the *Test set* using standard metrics such as accuracy, precision, recall and F1-score is discussed in Sect. 5.

In **Stage (2)**, to evaluate the semantic robustness, We randomly collect a subs-set of 34 Java source files from the DPD_{Att} *Corpus*, sampling 2–3 files per pattern and 2 files with "Unknown" labels. These files are then passed through the obfuscation tool (described in detail in Sect. 3), to perform systematic name obfuscation of the syntactic signals in these randomly collected source files preserving code structure and behavioral logic. Which is then collected as *Obfuscated Corpus* having these source files, with each retaining its original design pattern label from DPD_{Att} *Corpus*. We again use the code T5+ embedding model to perform feature extraction to get a fixed-length feature vector for every obfuscated file. This is then collected in the *Obfuscated Feature Corpus*. This corpus is used as *Test Set* in similar fashion to stage (1) and passed through the pre-trained classifiers in DPD_{Att}*-Baseline* from stage(1) under for evaluation. The prediction results are reported in Sect. 6.

Finally the Comparison & Analysis between the original and obfuscated results from stage(1) and stage (2) is presented in Sect. 7.

3 Obfuscation Tool Description

This section outlines systematic procedure to remove syntactic signals from the Java source files while preserving the code integrity. The outline for the tool selection process and what the tool requires and how the end result would look would be described in the subsections sequentially as follows:

3.1 Obfuscation Strategy and Objectives

Our primary goal is to determine whether the DPD_{Att}*-Baseline* (i.e., the pre-trained design-pattern classifiers) relies on surface-level syntactic signals such as (class, method, and variable names) when detecting design patterns. To do so, we apply a systematic name-based obfuscation that:

- Renames all identifiers in code (such as classes, methods, variables, etc.) to name-neutral placeholders.
- Preserves control-flow, method bodies, inheritance hierarchies, and overall program logic.

– Removes semantic hints embedded in print statements and comments via manual replacement.

By preserving the program logic while removing surface-level syntax, we isolate the impact of syntactic cues on classification performance.

3.2 Obfuscation Tool Selection and Comparison

To implement the obfuscation, we surveyed multiple Java obfuscation tools with varying capabilities. Table 1 summarizes the comparison:

Table 1. Comparison of Obfuscation Tools

Tools	License Type	Level	Mapping File
Jobfuscator [1]	Closed	Source	No
yGuard [3]	Open	Byte-code	No
ProGuard [2]	Open	Byte-code	Yes
Zelix KlassMaster [4]	Closed	Byte-code	Yes

Considering the objectives mentioned in Subsect. 3.1, we initially explored Jobfuscator [1] which does not obfuscate class names and does not handle inheritance properly. Since it cannot process multiple interdependent source files in a single pass. Hence we select ProGuard [2] due to its open-license, support for reliable bytecode-level obfuscation, and generation of detailed mapping files necessary for aligning obfuscated files with original pattern labels of the DPD_{Att} *Corpus*. However, ProGuard does not obfuscate string literals, which we addressed manually during manual code stubbing mentioned in Subsect. 3.3

3.3 Obfuscated Corpus Preparation, Manual Stubbing, and Compilation

ProGuard [2] tool requires compiled ".jar" file as input, necessitating successful compilation of Java source files. This proved non-trivial due to the nature of the DPD_{Att} *Corpus*: each project folder contains only those Java classes that are a part of design pattern, omitting external dependencies.

While the DPD_{Att} *Corpus* was built by augmenting and refining the original DPD_F-Corpus [15], the cleaned version of DPD_F-Corpus which was derived from large open-source repository named GitHub Java Corpus (GJC) [6] still suffers from the outdated Java features, unresolved imports making difficult to compile. The augmented part having additional files in DPD_{Att} *Corpus* crawled from GitHub repositories, does not document the specific project versions or commit hash used, making it difficult to reproduce those project structure reliably. The authors in [11] also note the inability for execution of the projects from DPD_F [15] and in turn same issues for DPD_{Att} [14] being based on DPD_F.

To overcome this, we performed manual stubbing using Eclipse IDE (as shown in Fig. 2):

- For each selected file, we created a minimal Java Project in Eclipse IDE.
- We manually implement and use IDE suggestions to resolve missing dependencies with stub classes or dummy methods to enable successful compilation.
- String literals and semantically revealing content (e.g., in print statements or comments) were obfuscated or removed to eliminate detection bias.

This manual process was resource-intensive and limited our scope to a representative subset of 34 Java files (2–3 design patterns). Selection was random to avoid bias, and each file was mapped to its corresponding label using the pattern annotation provided by the CSV file of *DPD$_{Att}$ Corpus*.

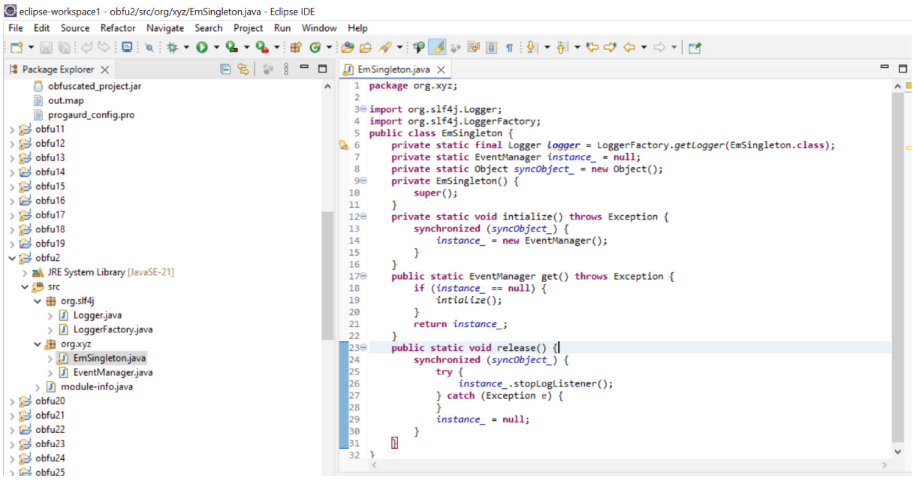

Fig. 2. One of the 34 original file sample `EmSingleton.java` loaded and compiled in Eclipse IDE (prior to obfuscation).

3.4 Obfuscation and De-compilation

After stubbing and compilation, each project was passed through the ProGuard [2] tool. This results in a output ".jar" and a mapping file (e.g., containing "org.xyz.EmSingleton → a.b.a") to trace the obfuscated class names. After identifying the obfuscated ".class" file (e.g. a.class), we de-compiled it using the FernFlower de-compiler plugin from the Visual Studio Code.

Listing 1.1. Obfuscated code sample labelled as Singleton from the collected subset

```
import a.a.b;

public class a {
    private static final a.a a = b.a(a.class);
    private static b.b a = null;
    private static Object a = new Object();

    private a() {
    }

    private static void b() {
        synchronized(a) {
            a = new b.b();
        }
    }

    public static b.b a() {
        if (a == null) {
            b();
        }

        return a;
    }

    public static void a() {
        synchronized(a) {
            try {
                a.a();
            } catch (Exception var2) {
            }

            a = null;
        }
    }
}
```

To illustrate, Listing 1.1 shows the de-compiled obfuscated version of "EmSingleton.java". Despite the identifier renaming, method structure, synchronization blocks, and instance management remain intact, preserving the Singleton pattern's intent.

3.5 Final Test Set: Obfuscated Corpus

We collect these 34 de-compiled Java files resulting in an *Obfuscated Corpus* with preserved pattern labels from DPD_{Att} *Corpus*. The mapping file provided by ProGuard [2] tool ensured traceability between original and obfuscated class file, enabling a valid *Test Set* for classifier inference used in Sect. 6.

4 Training the Classifiers of DPDAtt-Baseline for Pattern Detection

This section describes the setup and training of the DPD_{Att}-Baseline classifiers using the feature corpus derived from the *DPD_{Att} Corpus*. It outlines the feature extraction pipeline and the supervised learning strategy employed to train the baseline models.

4.1 Dataset and Pre-processing

The *DPD_{Att} Corpus* consists of 212 project folders comprising 1645 Java source files. But the corpus annotations with 13 GoF design pattern labels and with "Unknown" labels was available only for 1554 files. Also the authors of DPD_{Att} used the same set for embedding generation pipeline, using the CodeT5+ model to generate 256-dimensional feature vectors for each file. The configuration was kept consistent with the original setup of authors. The resulting *Feature Corpus*, consisting of labeled code embeddings, serves as the input for training classifiers.

4.2 Classifier Setup

The authors of DPD_{Att} originally used three machine learning models for classification: Support Vector Machine (SVM), Logistic Regression, and Multi-Layer Perceptron (MLP). In our setup, we adopted the same classifiers using the scikit-learn (sklearn) library with default hyper-parameters and for MLP (max_iter=1000) was set for convergence. These classifiers were trained in a supervised fashion using the "Train Set" which has fixed-length feature vector and pattern labels for it in the *Feature Corpus*.

4.3 Training Procedure

The authors of DPD_{Att} applied K-fold cross-validation (k=5), splitting the *Feature Corpus* into 80% training and 20% testing sets across folds. Each classifier was trained and validated across five folds to ensure robustness and generalization. During each fold, the trained models were evaluated using standard classification metrics—accuracy, precision, recall, and F1 score—and subsequently saved from the last fold to be used later for evaluation in Sects. 6 and 7 respectively.

5 Reproducing the DPDAtt-Baseline Classifiers

This section presents reproducing the baseline classification performance originally reported in DPD_{Att} [14] As outlined in Subsect. 2.2 and implemented in Sect. 4, we trained three classifiers (SVM, Logistic Regression, and Multi-Layer Perceptron) on the *Feature Corpus* generated from the DPD_{Att} Java Corpus.

While the original DPD$_{Att}$ study reported using 300-dimensional embeddings, we were unable to replicate this setup due to the lack of reproducibility details in the configuration. The CodeT5+ encoder used in the embedding pipeline does not support an explicit configuration for a 300-dimensional output. Instead, we followed the default CodeT5+ configuration provided in [14], which produces 256-dimensional feature vectors. This is consistent with the behavior of the provided token configuration file by the authors.

To ensure the use of 256-dimensional vectors did not degrade performance, we trained the classifiers using the same cross-validation approach (5-fold) described in the original paper. Table 2 shows a side-by-side comparison of the classification metrics obtained from the original 300 dimension embedding setup (as reported by authors in [14]) and our 256 dimension replication.

Table 2. Classifier performance comparison on 300 dimension vs 256 dimension embeddings with cross-validation k=5

Classifier	Dim	Accuracy	Precision	Recall	F1 Score
SVM	300	0.86	0.87	0.86	0.86
Logistic Regression	300	0.81	0.82	0.81	0.81
MLP	300	0.84	0.85	0.84	0.84
SVM	256	0.86	0.86	0.86	0.86
Logistic Regression	256	0.81	0.82	0.82	0.81
MLP	256	0.84	0.85	0.84	0.84

6 Testing the Trained DPD$_{Att}$-Baseline Under the Obfuscated Approach

This section evaluates how the classifiers trained in Sect. 4 perform on systematically obfuscated Java source files. The purpose of evaluation is to investigate whether these classifiers can maintain detection performance when syntactic signals are obscured, thereby testing their semantic generalization capability.

6.1 Obfuscated Corpus and Feature Extraction

As described in Sect. 3, we selected a representative subset of 34 Java files from the original *DPD$_{Att}$ Corpus*, with 2–3 files per GoF design pattern labels and also "Unknown" labels to files that are not a part of pattern, similar to the original corpus and call it *Obfuscated Corpus*. These files were manually obfuscated to systematically rename syntactic signals (e.g., class names, method names, and variable names, etc.) while preserving the structural and behavioral logic of the source code.

To create feature vectors for this new test set, we used the same embedding process described in Sect. 4—the CodeT5+ encoder—to generate 256-dimensional vector representations. This resulted in the creation of *Obfuscated Feature Corpus*, which retains the pattern labels for the corresponding feature vectors from the *Obfuscated Corpus* in similar fashion to original *Feature Corpus* creation from DPD_{Att} *Corpus* mentioned in Subsect. 4.1.

6.2 Evaluation of Trained DPD_{Att}-Baseline Under Obfuscation

The trained classifiers in DPD_{Att}-Baseline descibed in Sect. 4 were evaluated on the *Obfuscated Feature Corpus*, which is used as a Test Set. Enabling a direct evaluation of the trained model's predictions under syntactic perturbation. The classifier metric evaluation result is shown in Table 3.

Table 3. Classifier evaluation metrics under obfuscated test set (34 files)

Classifier	Accuracy	Precision	Recall	F1 Score
SVM	0.176	0.182	0.167	0.128
Logistic Regression	0.147	0.078	0.143	0.081
MLP	0.147	0.190	0.119	0.127

7 Analysis and Discussion

Testing DPD_{Att}-Baseline on the original *Feature Corpus* confirms the high classification scores, as after running and validating with 256 embeddings– e.g., their best pefroming classifer SVM achieves F1=0.86 (in Sect. 5; Table 2). However, when exactly same models are tested on the *Obfuscated Feature Corpus* (Sect. 6), performance collapses across all metrics. For instance, SVM drops from F1-score=0.86 to F1-score=0.128 (Sect. 6; Table 3). This drastic drop can be attributed to the superficial syntactic signals–such as identifier names and string literals, since control flow, inheritance and code logic were preserved during obfuscations–this answers **RQ1**.

Although the values are quite small and differ marginally in Table 3. They still reflect meaningful distinction. While SVM achieves the highest accuracy (≈ 0.18) and recall (≈ 0.17), MLP delivers slightly better precision (≈ 0.19) and a comparable F1 score (0.127) as compared to SVM's (0.128). This indicates that SVM retrieves more true positives but with a slight trade-off in precision, whereas MLP is more conservative yet balanced in its predictions. This may be due to its nonlinear capacity to model latent features, though the advantage remains marginal. Logistic Regression underperforms across all metrics, with poor precision and lowest F1 score, reflecting an overgeneralizing behavior.

Collectively, these results reinforce that all models primarily exploit syntactic artifacts and possess only limited semantic generalization.

Pattern-wise confusion matrix (omitted for space) for Sect. 6 reveals a consistent drift toward Prototype and Builder pattern predictions across classifiers. These patterns likely function as semantic defaults, reflecting the model's inability to disentangle structural intent or behavioral logic once name-level syntactic signals are removed. This suggests that the models fail to internalize deeper class-level semantics, leading to overgeneralized misclassification.

The 34-file obfuscated subset therefore fulfills its role as a "stress test". Because (i) it preserves code logic and structure, (ii) covers all 13 GoF patterns, and (iii) exposes a reproducible mapping file, the set offers a controlled proof-of-concept benchmark for any detectors that claims semantic awareness. Although larger benchmarks are desirable, this subset already discriminates strongly between genuine semantic modeling and mere name-based memorization. Affirming **RQ2** as a viable benchmark for probing semantic robustness and can be reused for future work.

Finally, we note that although DPD$_{Att}$ compares its approach with other detection tools such as FeatureMaps [19] and MARPLE-DPD [24] and DPD$_F$ [15], their implementation details were not provided in software artifact of DPD$_{Att}$ paper, preventing a like-for-like comparison on our corpus. Their omission, however, does not affect the primary finding: DPD$_{Att}$'s performance depends on syntactic cues and degrades when those cues are erased.

8 Conclusion and Future Work

In this paper, we reproduced and extended the DPD$_{Att}$ design pattern detection by systematically evaluating the semantic robustness under code obfuscation. While the baseline classifiers (SVM, Logistic Regression, MLP) demonstrated strong performance on the original *DPD$_{Att}$ Corpus*, their effectiveness dropped drastically when tested on an obfuscated subset in which identifier-based signals were removed but structural and behavioral semantics were preserved. This degradation confirms the current attention-based detectors like DPD$_{Att}$ fail to capture the code structure and code functionality.

To support reproducible evaluation of such robustness, we introduced a manually constructed, label preserved benchmark that consists of 34 obfuscated Java files spanning 13 GoF design patterns. This small benchmark serves as an effective diagnostic tool to evaluate deep semantic understanding of pattern detectors.

Future work will focus on using this benchmark to define suitable criteria for selecting detection tools that are compatible with our dataset, enabling comparative evaluation against other state-of-the-art detectors.

Data Availability. The data that support the findings of this study are available at: https://github.com/manthan410/Benchmark_dpd_att of Interests.

Disclosure of Interests. The authors have no competing interests to declare.

References

1. Jobfuscator: Java obfuscation tool. https://www.pelock.com/products/jobfuscator. Accessed 26 May 2025
2. ProGuard: Open-source java class file shrinker, optimizer, obfuscator, and preverifier. https://www.guardsquare.com/products/proguard. Accessed 26 May 2025
3. yGuard: The bytecode obfuscator for java. https://github.com/yWorks/yGuard. Accessed 26 May 2025
4. Zelix KlassMaster: Java obfuscator. https://www.zelix.com/klassmaster/. Accessed 26 May 2025
5. Al-Obeidallah, M.G.: The impact of design patterns on software maintainability and understandability: a metrics-based approach. ICIC express letters. Part B, Appl. Int. J. Res. Surv. **12**(12), 1111–1119 (2021)
6. Allamanis, M., Sutton, C.: Mining source code repositories at massive scale using language modeling. In: The 10th Working Conference on Mining Software Repositories, pp. 207–216. IEEE (2013)
7. Dunlop, N.: Investigating the accuracy of metric-based versus machine learning approaches in detecting design patterns (2023)
8. Gamma, E.: Design patterns: elements of reusable object-oriented software. Pearson Education India (1995)
9. Gao, S., Gao, C., Wang, C., Sun, J., Lo, D., Yu, Y.: Two sides of the same coin: exploiting the impact of identifiers in neural code comprehension. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 1933–1945. IEEE (2023)
10. Guéhéneuc, Y.G., Antoniol, G.: DeMIMA: a multilayered approach for design pattern identification. IEEE Trans. Software Eng. **34**(5), 667–684 (2008). https://doi.org/10.1109/TSE.2008.48
11. Houichime, T., El Amrani, Y.: Context is all you need: a hybrid attention-based method for detecting code design patterns. IEEE Access (2025)
12. Jun, H.K., Rana, M.E.: Evaluating the impact of design patterns on software maintainability: an empirical evaluation. In: 2021 Third International Sustainability and Resilience Conference: Climate Change, pp. 539–548 (2021). https://doi.org/10.1109/IEEECONF53624.2021.9668025
13. Marinescu, R.: Detection strategies: metrics-based rules for detecting design flaws. In: 20th IEEE International Conference on Software Maintenance, 2004. Proceedings, pp. 350–359 (2004). https://doi.org/10.1109/ICSM.2004.1357820
14. Mzid, R., Rezgui, I., Ziadi, T.: Attention-based method for design pattern detection. In: European Conference on Software Architecture, pp. 86–101. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-70797-1_6
15. Nazar, N., Aleti, A., Zheng, Y.: Feature-based software design pattern detection. J. Syst. Softw. **185** (2022). https://doi.org/10.1016/j.jss.2021.111179
16. Niere, J.: Fuzzy logic based interactive recovery of software design. In: Proceedings of the 24th International Conference on Software Engineering, pp. 727–728. ICSE '02, Association for Computing Machinery, New York, NY, USA (2002). https://doi.org/10.1145/581339.581473
17. O'Brien, M.P., Buckley, J.: Inference-based and expectation-based processing in program comprehension. In: Proceedings 9th International Workshop on Program Comprehension. IWPC 2001, pp. 71–78. IEEE (2001)
18. Rasool, G., Philippow, I., Mäder, P.: Design pattern recovery based on annotations. Adv. Eng. Softw. **41**(4), 519–526 (2010)

19. Thaller, H., Linsbauer, L., Egyed, A.: Feature maps: a comprehensible software representation for design pattern detection. In: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 207–217. IEEE (2019)
20. Tsantalis, N., Chatzigeorgiou, A., Stephanides, G., Halkidis, S.T.: Design pattern detection using similarity scoring. IEEE Trans. Softw. Eng. **32**(11), 896909 (2006). https://doi.org/10.1109/TSE.2006.112
21. Vaswani, A., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30 (2017)
22. Wang, Y., Le, H., Gotmare, A.D., Bui, N.D., Li, J., Hoi, S.C.H.: CodeT5+: open code large language models for code understanding and generation. arXiv preprint (2023)
23. Xia, X., Bao, L., Lo, D., Xing, Z., Hassan, A.E., Li, S.: Measuring program comprehension: a large-scale field study with professionals. IEEE Trans. Software Eng. **44**(10), 951–976 (2017)
24. Zanoni, M., Fontana, F.A., Stella, F.: On applying machine learning techniques for design pattern detection. J. Syst. Softw. **103**, 102–117 (2015)

LLM-Based Explainability at Design Time: Detecting Elasticity Antipatterns in Software Architectures

Floriment Klinaku(✉), Jonas Lammert, and Steffen Becker

Institute of Software Engineering, Software Quality and Architecture, University of Stuttgart, Stuttgart, Germany
floriment.klinaku@iste.uni-stuttgart.de

Abstract. As software architecture grows in complexity, understanding the implications of design decisions becomes increasingly challenging. Large Language Models (LLMs) offer new opportunities for enhancing explainability during architecture modeling and evaluation by generating natural language explanations that support comprehension, learning, and decision-making. This potential is particularly valuable in domains with increased technical complexity—such as elasticity in cloud-based systems. In this work, we integrate and evaluate LLM-based explanations in supporting design-time evaluation of software architectures, focusing on the detection of elasticity antipatterns. Elasticity antipatterns are flawed autoscaling policy configurations that potentially lead to inefficient or unreliable system behavior. We extend an existing modeling and simulation approach with a novel feature that generates contextualized, textual explanations derived from simulation data. These explanations aim to guide architects in understanding scaling behaviors, identifying design issues, and refining their models. Our contribution includes the conceptualization of explanation types relevant to elasticity modeling, the design of prompt templates to elicit effective responses from LLMs, and an evaluation of the generated explanations' usefulness and quality. Results indicate that LLM-assisted feedback enhances the interpretability of elasticity models and supports the early identification of antipatterns, albeit with some limitations in precision and conciseness with only a slight agreement between expert evaluations ($\kappa = 0.202$). The explanation quality across types of explanations differs. Even though most explanations contain factual information, a large portion was deemed as imprecise especially in explaining problem and solution, the policy and target and service level objectives.

1 Introduction

Understanding software architecture design decisions is inherently complex due to the interplay of quality attributes, constraints, and long-term impact [2]. The cognitive effort required to comprehend the implications of design decisions is further amplified in cloud-native systems, where elasticity introduces dynamic,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 141–154, 2026.
https://doi.org/10.1007/978-3-032-04403-7_14

time-sensitive behaviors. To support the evaluation of architectural decisions, design-time modeling tools—particularly those that incorporate simulation-based evaluation—have been developed to predict system behavior under varying conditions. These tools enable architects to analyze the performance, scalability, and resource utilization of architectural alternatives before deployment, thereby reducing the risk of costly design flaws.

Once architectural decisions are modeled and simulation results become available, architects may learn that certain quality objectives are not being met. However, identifying how to refine the model or adjust specific parameters to resolve these issues remains a significant challenge. Simulation tools often provide quantitative feedback—such as response times or resource utilization metrics—without clearly linking these outcomes to the architectural decisions that produced them. As a result, architects are left to interpret complex data without sufficient guidance, making it difficult to pinpoint which elements of the model require change and how to effectively adapt them.

To bridge this gap, we investigate how Large Language Models (LLMs) can be used to generate natural language explanations that contextualize simulation feedback in relation to design decisions captured in the model. The main research question we set out to investigate is as follow:

Research Question

How does a state of the art large language model perform in aiding modeling and simulating the elasticity of software architectures with explanations optimized for conciseness?

The novelty of the solution is the definition of type-specific prompts for parts of the model and guiding the prompt to produce concise explanations. The idea behind the solution is to understand whether large language models trained on large and generic datasets perform well in such a specific context and especially by tailoring explanations to model parts. The proposed solution advocates that explanations are seeked for parts of the model in order to reduce the amount of interactions required in order to identify architectural antipatterns. By translating performance indicators and policy behavior into understandable insights, LLMs offer the potential to assist architects in interpreting results, diagnosing design flaws and allow architects to better understand the implication of existing decisions and the potential of refining the model.

In this work, we focus on supporting software architects in identifying elasticity antipatterns at design time by enriching the modeling process with explainability features. We build on the Palladio [17] toolset, which allows for performance-oriented architectural modeling and simulation, and extend its capabilities to include automatically generated natural language explanations.

The main contribution of this work are: (1) we introduce a set of explanation types relevant to elasticity modeling and propose prompt templates designed to retrieve explanations from an LLM, (2) we evaluate the quality of the generated explanations through expert assessment using defined quality criteria, in order

to determine their usefulness in practice (3) we implement the concept in a prototype that integrates LLM-generated explanations into Palladio's graphical model editor for elasticity policies. The goal of the work is to demonstrate that such an AI-assisted feedback mechanism based on LLMs guide architects in detecting elasticity-related flaws more effectively at design time.

2 Background and Related Work

The work builds upon previous foundational research which we elaborate in this section. Additionally, we elaborate related works. Figure 1 shows all the parts which are neccessary to understand the context and the contribution of this work. The software architect (SA) uses the tool Slingshot that is based on the Palladio approach and supports modeling and evaluating elasticity policies. The Palladio approach has been intended for static software architectures, whereas in Slingshot a new view type is added to support the definition of dynamic software architectures based on elasticity policies.

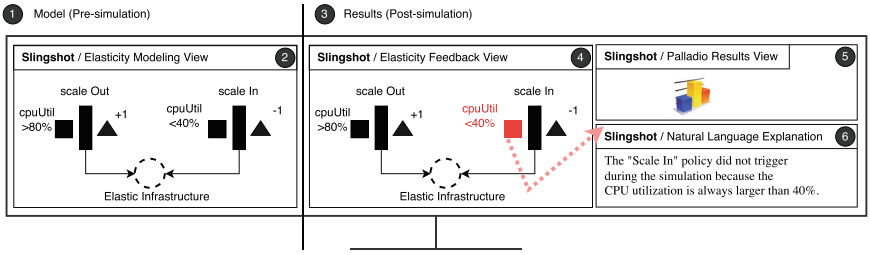

Fig. 1. Overview Example

Overview. In the modeling (pre-simulation) phase (❶), the SA usess the elasticity view to model policies that adapt the software architecture (❷). This view type is defined by an abstract syntax, graphical notation and precise simulation semantics as transformations to the initial software architecture configuration and has tool support for graphically creating models based on EMF [11]. In the figure, there exist two policies, where the one the left increases the number of infrastructure elements by one whenever the CPU utilization exceeds 80% and the one on the right decreases the number of processing elements by one when the CPU utilization is below 40%. In the results analysis (post-simulation) phase (❸), the SA uses additional views to evaluate whether decisions made when modeling achieve some given quality objectives. In a previous work, to spot problematic model elements, Hahn [7] proposes to color elements of policies that may be the root cause for bad performance. Similar visual feedback is found in other tools such as JMT for spotting performance problems. For example, in the figure, the trigger shown as a square is marked red to indicate that the trigger did not fire in the course of the simulation (❹). The approach already supports

adding monitors and probes in the model to gather quantitative insights from the architecture (❺). Mapping back the insights to refine the model is a challenging task that requires a lot of domain expertise and analytical skills. Hence, the scope of this paper focuses on providing natural language explanations that in addition to the quantitive data and the feedback view reduces the time to make a refinement or explore a new model alternative (❻).

2.1 Elasticity and Elasticity Antipatterns

Elasticity, in the context of cloud-native applications and software, is the degree to which a software system is able to adapt to workload changes by provisioning and de-provisioning resources in an autonomic manner [8]. The goal is to match available resources to the current demand as closely as possible. Several approaches have been proposed in the literature to evaluate the elasticity of cloud-native applications, ranging from formal modeling that require significant modeling expertise [5], to load testing methods that are cost-intensive and depend on the availability of resources [9]. In between these extremes are approaches that provide suitable modeling abstractions and leverage simulation to explore the behavior of a software architecture (e.g., CloudSim [3] or Palladio [17]). Both CloudSim and Palladio has been extended to support new software architecture evaluation use cases such as the evaluation of elasticity and autoscaling. Specifically, Klinaku et al. [11] proposes an elasticity-specific architectural view type in which architects can model scaling policies that apply to a software architecture model. The resulting models are simulated using discrete event simulation to produce performance indicators of interest.

Straesser et al. [18] identify a collection of antipatterns that can emerge in autoscaling policies for cloud-native systems. They distinuish seven antipatterns and discusses their underlying causes, each of which may lead to violations of Service Level Objectives (SLOs). These include: *No reaction*, where scaling fails to occur despite changes in load; *Wrong steady state provisioning*, in which the system remains static under constant load but still experiences SLO violations; *Rapid upscaling*, where an excessive number of instances are added too quickly, potentially overshooting demand; *Slow upscaling*, where the autoscaler reacts too slowly, resulting in insufficient capacity; *Rapid downscaling*, where the system aggressively removes instances following a drop in load, risking underprovisioning; and *Slow downscaling*, which delays the removal of instances, leading to unnecessary cost overhead. The presence of such antipatterns reflects a poor alignment between scaling behavior and workload dynamics, often culminating in performance degradation and cost inefficiencies. The antipatterns could be spotted and identified already at design time in modeling and simulation tools such as Palladio.

2.2 LLMs for Explainability in Software Design and Development Tools

Large Language Models (LLM) are large neural networks trained on vast amounts of text. These models adjust millions or even billions of weights and biases—referred to as parameters in this context—allowing them to generate coherent, context-aware responses [20]. These models are advanced AI systems capable of understanding, generating, and manipulating human language. Modern LLMs can generate text that is often indistinguishable from human writing. In literature, there exist several instances of work that use LLMs in various use cases in the software engineering domain. For example, Reif et al. [16] introduce a system that uses LLMs to automatically analyze and explain unstructured datasets. Lubos et al. [14] use an LLM to generate natural language explanations on why a recommender system recommends products to a user. Another well-known use case for LLMs is code explanations [15].

Fault Localization. Another well-known use case of LLM-based explanation generation is fault localization. Such a use case shares similarities with the use case of our work. There are several works that investigate the usage of LLMs to generate explanations on where and how a fault occurred and how to potentially fix it. For example, Das et al. [4] introduce a system that can generate explanations for the cause of an unexpected failure of a robot's plan execution. They are in natural language and aim to be written such that even non-experts should be able why errors happened and how the problem can potentially be fixed. There are additional works that investigate the usage of LLM-based explanation generation to explain faults in software. Leinonen et al. [12] use LLMs to generate explanations for programming error messages, which are often unclear, especially for novices. Their evaluation showed that their approach is feasible and shows promise. However, they did not find much success with suggesting correct fixes. These were only correct 47% of the time. Similarly, Liu et al. [13] use LLMs to automatically analyze online logs, which helps comprehend a program's status and take appropriate action.

One work that is of high relevance for the evaluation we chose is the work of evaluation of LLM-based explainable fault localization by Kang et al. [10]. They propose an LLM-based fault localization technique called AutoFL, that can generate explanations of a bug in a software, along with a suggested fault location. The quality of explanations was assessed using four criteria: accuracy, reflecting how well the explanation details the root cause of the issue; imprecision, indicating the presence of any inaccurate statements; conciseness, measuring the succinctness and absence of irrelevant content; and usefulness, evaluating whether the explanation provides a correct method for fixing the issue. Explanations that merely describe the test and covered method without offering deeper insight were rated as bland. Such a qualitative evaluation, originally proposed by Kang et al. [10] serves us for evaluating the quality of generate explanations in our context. In our work, we aim to produce explanations that are accurate, useful, and concise, while minimizing imprecisions.

3 Concept and Implementation

In Fig. 2, we present an overview of the solution concept. The solution involves extracting out of the design model types of explanantions that provide additional information (❶). For each explanantion type, the architectural model (❷) and simulation results (❸) may be used to build a type-specific prompt that is given to a state-of-the-art large language model. The prompt (❹) contains instructions on the expected output. The expected output is guided for conciseness, so that the explanantion is presented in conjuction to the design model and the architect is not engaged in chat.

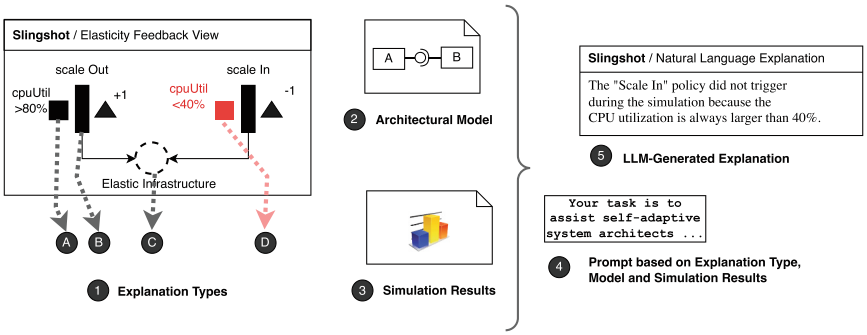

Fig. 2. Concept

3.1 Explanation Types

To guide the enhancement of the prototype, we consolidated requirements based on user feedback from two prior works by Summerer [19] and Hahn [7], on which our system is built. Both studies included empirical evaluations where participants interacted with early versions of the simulation and explanation tooling, providing valuable insights into usability, desired features, and limitations. These evaluations involved user surveys and structured feedback sessions that identified practical needs during design-time simulation-based analysis. We systematically analyzed the collected data which lead to a set of three use cases we aim to support:

- *Use Case 1: Successful Scaling.* The defined elasticity policies function as intended. Architect may interest in the purpose and functionality of the applied policies. Which specific policy was triggered, when it was activated, and why? Architects seek detailed explanations regarding scaling policies and their associated trigger behaviors.
- *Use Case 2: SLO violations occur.* In instances where an SLO violation occurs and the feedback view highlights problematic elements, architects require deeper insights beyond those outlined in Use Case 1.

– *Use Case 3: Testing individual policies during policy creation.* Architects seek support during the creation of policies with explanantions that aid the creation: e.g., what policies exist and what need to be created for fulfilling SLOs.

After analyzing the use cases, we identified four distinct types of explanations that users may request to better understand elasticity behavior. These are: (1) Trigger Behavior Explanations (A), which detail when and why a specific scaling policy trigger was activated, providing insights into the logic and timing of trigger events; (2) Scaling Policy Explanations (B), which describe the functionality of a scaling policy and the conditions under which it operates; (3) Target Group & SLO Explanations (C), which describe the service-level objectives (SLOs) expected of a target group, enumerate the scaling policies currently in place, identify any missing elements, and assess potential risks of SLO violations stemming from existing configurations; and (4) Problem & Solution Explanations (D), which clarify why a particular component of a policy has been flagged as problematic, identify the underlying antipattern and root cause, and offer concrete suggestions for resolution; and

3.2 Architectural Model and Simulation Results

The architectural model that defines the context of the modeling and simulation is represented by the Palladio Compoment Model (PCM). The Palladio development process has four essential roles: the component developer, the software architect, the system deployer, and the domain expert. All four roles have different concerns on the modeled system and use different view types to create parts of the PCM [17]. Palladio provides a set of well-defined view types tailored to different stakeholders involved in the design and evaluation of cloud-native systems. The component developer uses the Repository View to model software components in terms of performance-relevant behavior. This includes defining Service Effect Specifications (SEFFs), which capture the internal actions, external service calls, control flow, and resource demands of each component, potentially using random variables to express uncertainty. The software architect works with the System View, where these components are instantiated as assemblies and interconnected based on their required and provided interfaces, allowing the exploration of architectural alternatives. The system deployer relies on two views: the Resource Environment View, which models the execution infrastructure including processing speeds and communication links, and the Allocation View, which specifies the mapping of component assemblies to the underlying resources. Finally, the domain expert defines the expected user behavior in the Usage Model, which describes usage scenarios and workloads (open or closed), as well as user interaction sequences referencing component behaviors. Together, these views support performance prediction through simulation and enable early-stage reasoning about system quality attributes. A textual description of the Palladio model is provided as an input to the prompt. Providing the textual description for such a model and ways to automate it, is out of scope.

Simulation result data in Palladio typically consists of tuples in the form (x, y), where x represents the simulation time and y is a measured indicator,

such as the response time of a user request or the number of active component instances. For our concept and implementation, we focused on response time data, which should enable the LLM to infer how heavily the system is utilized. Additionally, we include data on the number of running instances in the target group, allowing the LLM to reason about up- and downscaling actions that occurred. Although current LLM models support input sizes up to 50MB—potentially accommodating millions of data points—this comes at the cost of high token consumption. In this work, we did not explore methods for optimizing or reducing the simulation output before use.

3.3 Prompt Design

According to Giray et al. [6], a well-structured prompt generally comprises four key components. The first is the Instruction, which provides a clear directive to guide the model's behavior and specify the task to be performed, ensuring alignment with the intended outcome. The second is the Context, which includes relevant background information or external details that enhance the model's understanding and enable more precise, contextually appropriate responses. Third is the Input Data, representing the core question or problem statement the model is expected to process—this forms the foundation of the prompt. Finally, the Output Indicator defines the expected format of the model's response, such as a short answer, a paragraph, or another structured output, helping to shape the response accordingly. In the following, we construct prompts for each of our explanation types by systematically incorporating these components.

Figure 3 shows the prompt structure for one of the explanation types, specifically for the Problem & Solution type. The preamble of the prompt, that is shared for all types, defines the assistance task for self-adaptive system architects. After the preamble, the prompt contains the description of the Palladio model and the description of all scaling policies defined in the elasticity view by the architect. Each explanation type has its own customizations. For all policies, the output is guided towards conciseness, shortness and it is limited to 400 characters. The explanation type for the scaling policy element does not require the simulation data, the prompt consist of only the model parameters. To help the LLM describe trigger behavior and identify antipatterns, we provide it with two key data types: simulation metrics—ideally those matching the policy's trigger condition (e.g., CPU utilization)—and the number of active instances over time. Even without direct metrics, the LLM can infer behavior from related indicators like response time. We also include a clear data format description, such as timestamped (x, y) tuples, to ensure correct interpretation. Given token limits, large simulation datasets may require preprocessing, posing a potential limitation of this approach.

LLM Explanation Type Task Description

Your task is to assist self-adaptive system architects in designing scaling policies for elastic cloud systems that assure that the system always has enough resources to provide its service, but never too many, which would lead to unnecessary costs, by giving explanations and/or feedback on their scaling policy designs.

`[PCM DESCRIPTION]`

The architect has defined scaling policies as follows:

The policy `[POLICYNAME]` `[increases|reduces]` the resource `[TARGETGROUP]` by `[ADJUSTMENTVALUE]` if `[TRIGGERCONDITION]` holds. It is restricted by `[CONSTRAINTTYPE]` of `[CONSTRAINTVALUE]`.
After simulating our system, we want to analyze our policy for anomalies, describing if this policy could have led to an anomaly in the simulation and, if yes, which antipattern could have occurred in the design process of the scaling policy, to which service level objective violation it led in the simulation and to what extent it was violated.

There are seven possible antipatterns:

- **No reaction** – neither up nor downscaling performed.
- **Wrong steady state provisioning** – after scaling action, no more scaling actions but SLO violations occur.
- **Jitter** – no steady state is reached; repeated scaling occurs.
- **Rapid upscaling**
- **Rapid downscaling**
- **Slow upscaling**
- **Slow downscaling**

`[DATA FORMAT DESCRIPTION]`

`[INPUT DATA]`

Structure your answer as follows: Explanation on which SLO violation occurred at which time and to what extent, explanation of which antipattern could be the cause and why the policy design led to it, suggestion on how to fix the problem. Answer in natural language, keep the answer short and concise, keep it under 400 characters.

Fig. 3. LLM prompt template for explanation of the type Problem & Solution.

4 Evaluation

To assess the quality of the generated explanations, we conducted an expert evaluation involving two members of the Software Quality and Architecture (SQA)

institute at the University of Stuttgart. Both participants are domain experts in self-adaptive system architecture and have practical experience working with the elasticity view. They were asked to evaluate a total of 47 explanations based on five predefined quality criteria.

To broaden the scope of explanation generation, we designed elasticity models that intentionally lead to one of the antipatterns described previously. This resulted in seven distinct models, each corresponding to a specific antipattern. All models include both an upscaling and a downscaling policy, both linked to a single target group. For each model, five standard explanations were generated: one Scaling Policy Explanation and one Trigger Behavior Explanation for each policy, as well as one explanation for the target group. Additionally, each model includes one or two Problem & Solution Explanations, depending on whether anomalies were observed. These are only generated when a scaling policy leads to an actual anomaly. In the cases of the Jitter and No Reaction antipatterns, both policies exhibited anomalous behavior, whereas in the remaining models, only one policy was responsible. In addition, we add 4 incomplete elasticity models so that we can observe Target Group & SLO Explanations that give tips on how to complete an unfinished elasticity model.

We chose to generate our explanations using the ChatGPT web interface, which provides access to GPT-4o. In contrast. The GPT-4o model is estimated to have around 200 billion parameters [1], although its exact size has not been officially disclosed. The web interface imposes a message size limit of 4096 tokens, which was sufficient for the experimental data. We did not explore different temperature settings.

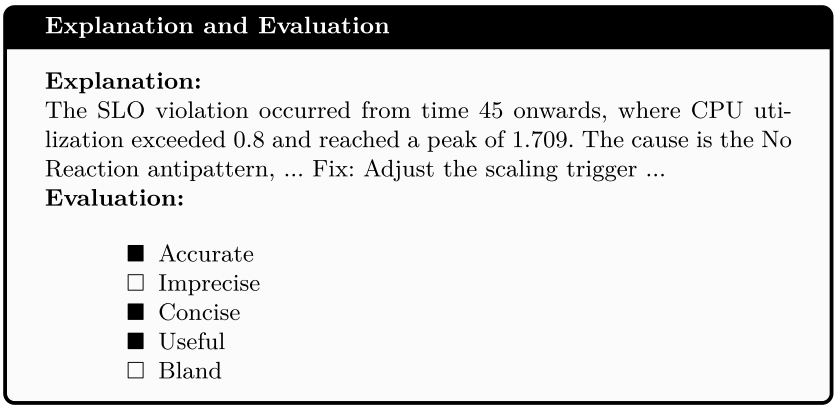

Fig. 4. Expert evaluation of an explanation with checkboxes indicating perceived qualities.

As previously stated, the evaluation criteria used in our study are based on the work of Kang et al. [10], and serve as the basis for expert assessment of the generated explanations. The criteria include: Accuracy, which assesses whether

the explanation correctly describes the relevant part of the policy and, in the case of problem explanations, correctly identifies the antipattern and its cause; Imprecision, which flags explanations containing at least one inaccurate or misleading statement; Conciseness, which evaluates whether the explanation is succinct and free of unnecessary information; Usefulness, which measures the extent to which the explanation provides actionable insights for improving the policy or resolving identified issues; and Blandness, which applies when the explanation correctly describes the policy but lacks deeper analysis or interpretation. All criterias are binary, e.g., an explanation either is accurate or not. In the evaluation form, experts are aksed with a multiple-choice question for each explanation, listing all five criteria. They can check a box if they believe the explanation meets a given criterion or leave it unchecked if they think it does not.

5 Results and Discussion

Table 1. Expert ratings of explanation types (transposed). Gray cells indicate suboptimal values: for Accurate, Concise, and Useful <70%, and for Imprecise and Bland >30%.

	Probl. & Sol.	Trigger	Policy	Target & SLO
Accurate	77.78%	69.23%	71.43%	81.81%
	55.56%	100%	71.43%	90.91%
Imprecise	33.33%	23.08%	14.29%	0%
	100%	23.08%	64.29%	54.55%
Concise	33.33%	30.77%	35.71%	72.72%
	66.67%	69.23%	50%	81.81%
Useful	77.78%	53.85%	42.86%	72.72%
	77.78%	61.54%	14.29%	81.81%
Bland	0%	15.38%	14.29%	9.09%
	55.56%	30.77%	71.43%	36.36%

We compiled every explanation and its rating in our Zenodo repository[1]. All the prompts used to generate the explanations are publicly accessible. The results of the explanation quality evaluation can be seen in Table 1, which shows the percentage of explanations that met each criterion. For example, the grayed value of 55.56% for accurate in the type Problem and Solution indicates that

[1] https://doi.org/10.5281/zenodo.15045589.

only that portion out of 47 explanation were marked as accurate by the second expert. The grayed values indicate thresholds for better visibility. For positive quality criteria such as accurate, concise and useful we use a 70% threshold. If not more than 70% of the explanations are not marked with the criterion we mark that as problematic. For negative quality criteria such as imprecise and bland we use the opposite threshold with the direction of 30%.

Explanations were deemed accurate for the Policy and Target & SLO types and less accurate explanations were marked for the Problem and Solution and Trigger Behavior type. For all explanations, we conclude that the used model was able to generate explanations that do include factual information, at least partially, for the most part. A large portion of explanations were deemed imprecise, mainly in the Problem and Solution, Policy and Target and SLO explanation types. The second expert marked more explanations as imprecise in comparison to the first expert. Related to conciseness results are poor eventhough the prompt specifically aims at concise, short and answers of limited length. Relatively low numbers in conciseness for the first expert, and relatively high numbers in blandness for the second expert imply that explanations often contain extraneous information that does not contribute to their usefulness. Usefulness ratings being around 60% for both experts implies a mediocre performance of the explanations on this criteria. Overall, results are promising in some quality criteria. However, they suggest high discrepancies and low quality in important criteria.

The two experts showed only slight agreement, with a Cohen's κ of 0.202. To clarify their differences, a post-evaluation discussion was held. It revealed differing interpretations of the criteria. Expert 1 considered an imprecise explanation inherently not useful, whereas Expert 2 still found partial usefulness in such cases. Expert 1 was also stricter about conciseness, requiring every part of the explanation to add value, while Expert 2 applied the concise label more liberally, citing word count limits in the prompts. Conversely, Expert 2 was more critical of imprecision, marking explanations as imprecise for minor errors or unusual terms, whereas Expert 1 was more forgiving unless the information was outright false. Regarding blandness, Expert 1 reserved the rating for explanations they found dull, especially shorter ones, while Expert 2 used it when explanations added unnecessary information without improving usefulness.

Discussions showed additional issues present in the explanations such as: (1) SLO violations were mistakenly judged as single points conditions, however, the SLO was defined over a time interval, (2) making up mistakenly arguments for the antipatterns, (3) inconsistent terminology, (4) trivial and non-helpful tips such as "Fine-tuning the threshold and cooldown can improve reliability".

6 Conclusion

This work explored the potential of large language models (LLMs) to enhance the explainability of simulation-based feedback in elasticity modeling at design time. By defining explanation types tailored to elasticity concerns and integrating LLM-generated explanations into the Palladio toolset, we aimed to support

software architects in interpreting performance results, identifying antipatterns, and refining design decisions more effectively. Our evaluation shows that LLMs can generate partially accurate and sometimes useful explanations; however, the overall quality was inconsistent across criteria and explanation types. Accuracy and usefulness were moderately achieved, particularly for policy- and SLO-related explanations, while issues such as imprecision, verbosity, and blandness were prominent—especially in complex explanations like Problem & Solution. The low inter-rater agreement further highlights the subjectivity and ambiguity involved in evaluating explanation quality, as experts differed significantly in interpreting and applying the criteria.

These findings indicate that, while promising, current LLM capabilities are insufficiently robust for fully reliable, domain-specific architectural feedback without further refinement. Future work should investigate strategies to improve explanation quality by incorporating domain-specific fine-tuning, developing structured reasoning chains, and enforcing terminology consistency. Additionally, interactive mechanisms for architects to query or refine explanations, combined with visualization of design-to-result traceability, could improve both usability and trust in AI-assisted modeling. Expanding the evaluation to include a larger set of domain experts and real-world case studies will be essential to validate the practical value of LLM-generated feedback in software architecture design.

References

1. Abacha, A.B., et al.: MEDEC: a benchmark for medical error detection and correction in clinical notes. arXiv preprint arXiv:2412.19260 (2024)
2. Bass, L.: Software Architecture in Practice. Pearson Education India (2012)
3. Calheiros, R.N., Ranjan, R., Beloglazov, A., De Rose, C.A., Buyya, R.: CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Software Pract. Experience **41**(1) (2011)
4. Das, D., Banerjee, S., Chernova, S.: Explainable AI for robot failures: generating explanations that improve user assistance in fault recovery. In: Proceedings of the 2021 ACM/IEEE International Conference on Human-Robot Interaction. HRI '21, New York, NY, USA, pp. 351–360. Association for Computing Machinery (2021). https://doi.org/10.1145/3434073.3444657
5. Evangelidis, A., Parker, D., Bahsoon, R.: Performance modelling and verification of cloud-based auto-scaling policies. Future Gener. Comput. Syst. **87**, 629–638 (2018). https://doi.org/10.1016/j.future.2017.12.047
6. Giray, L.: Prompt engineering with ChatGPT: a guide for academic writers. Ann. Biomed. Eng. **51**(12), 2629–2633 (2023)
7. Hahn, B.: Enriching Graphical Editors for Scaling Policy Definitions with Feedback from Simulations. Bachelor Thesis: University of Stuttgart, Institute of Software Technology, Software Quality and Architecture (2023)
8. Herbst, N.R., Kounev, S., Reussner, R.H.: Elasticity in cloud computing: what it is, and what it is not. In: Kephart, J.O., Pu, C., Zhu, X. (eds.) 10th International Conference on Autonomic Computing, ICAC'13, San Jose, CA, USA, June 26-28, 2013, pp. 23–27. USENIX Association (2013)

9. Herbst, N.R., Kounev, S., Weber, A., Groenda, H.: Bungee: an elasticity benchmark for self-adaptive IAAS cloud environments. In: IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems. IEEE (2015)
10. Kang, S., An, G., Yoo, S.: A quantitative and qualitative evaluation of LLM-based explainable fault localization. Proc. ACM Softw. Eng. **1**(FSE) (2024). https://doi.org/10.1145/3660771
11. Klinaku, F., Stieß, S.S., Hakamian, A., Becker, S.: An architectural view type for elasticity modeling and simulation–the slingshot approach. J. Syst. Softw. **228**, 112432 (2025). https://doi.org/10.1016/j.jss.2025.112432
12. Leinonen, J., et al.: Using large language models to enhance programming error messages. In: Proceedings of the 54th ACM Technical Symposium on Computer Science Education. SIGCSE 2023, New York, NY, USA, vol. 1, pp. 563–569. Association for Computing Machinery (2023). https://doi.org/10.1145/3545945.3569770
13. Liu, Y., et al.: Interpretable online log analysis using large language models with prompt strategies. In: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension. ICPC '24, New York, NY, USA, pp. 35–46. Association for Computing Machinery (2024). https://doi.org/10.1145/3643916.3644408
14. Lubos, S., Tran, T.N.T., Felfernig, A., Polat Erdeniz, S., Le, V.M.: LLM-generated explanations for recommender systems. In: Adjunct Proceedings of the 32nd ACM Conference on User Modeling, Adaptation and Personalization. UMAP Adjunct '24, New York, NY, USA, pp. 276–285. Association for Computing Machinery (2024). https://doi.org/10.1145/3631700.3665185
15. MacNeil, S., Tran, A., Mogil, D., Bernstein, S., Ross, E., Huang, Z.: Generating diverse code explanations using the GPT-3 large language model. In: Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 2, pp. 37–39 (2022)
16. Reif, E., Qian, C., Wexler, J., Kahng, M.: Automatic histograms: Leveraging language models for text dataset exploration. In: Extended Abstracts of the CHI Conference on Human Factors in Computing Systems. CHI EA '24, New York, NY, USA. Association for Computing Machinery (2024). https://doi.org/10.1145/3613905.3650798
17. Reussner, R., et al.: The palladio component model (2011)
18. Straesser, M., Eismann, S., von Kistowski, J., Bauer, A., Kounev, S.: Autoscaler evaluation and configuration: a practitioner's guideline. In: Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering. ICPE '23, New York, NY, USA, pp. 31–41. Association for Computing Machinery (2023). https://doi.org/10.1145/3578244.3583721
19. Summerer, T.: Graphical editors for defining scaling policies analysable using simulations. B.S. thesis (2022)
20. Wolfram, S.: What Is ChatGPT Doing:... and Why Does It Work? Wolfram Media (2023)

AI-Powered Architecting for Industry 4.0 Cyber-Physical Production Systems: A Novel Approach, Research Problems and Challenges

Stephen John Warnett[1,2(✉)] and Uwe Zdun[1]

[1] Research Group Software Architecture, Faculty of Computer Science, University of Vienna, Vienna, Austria
{stephen.warnett,uwe.zdun}@univie.ac.at
[2] UniVie Doctoral School Computer Science DoCS, Faculty of Computer Science, University of Vienna, Vienna, Austria

Abstract. The integration of artificial intelligence (AI) into software architecture has the potential to greatly support practitioners in architecting machine learning (ML) and reinforcement learning (RL) systems. In the context of Industry 4.0 cyber-physical production systems (CPPSs), this work presents our approach, research problems, and open challenges when designing ML and RL systems and applying AI-powered architecting to enhance automation and reduce manual effort. We describe a novel approach with six advanced contributions: organising architectural design decisions (ADDs) linked to the ML workflow and deployment, extending machine learning operations (MLOps) into reinforcement learning operations (RLOps), introducing formal architectural model development, adding automatic conformance checks, using large language models (LLMs) in RL conformance assessment and creating MLOps and RLOps pipelines using LLMs and a low-code approach. We also describe how earlier contributions in supporting the architecting process could be enhanced using AI. We discuss four further key open research challenges: RL model versioning, the development of RL communication protocols, working with agentic AI, and supporting RL within continuous integration and continuous delivery pipelines.

Keywords: artificial intelligence · MLOps · RLOps · machine learning · reinforcement learning · software architecture · software engineering · cyber-physical production systems

1 Introduction

Integrating artificial intelligence (AI) into software architecture can catalyse rapid advancements in how practitioners design, optimise, and maintain systems [1]. AI-driven techniques transform traditional practices by documenting

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 155–168, 2026.
https://doi.org/10.1007/978-3-032-04403-7_15

decision-making processes, refining architectural trade-offs, and enabling dynamically adaptive systems capable of self-evolution [2,3].

AI's role in overcoming long-standing issues in software architecture, including complexity management, up-to-date documentation, and continuous evolution of systems, is becoming increasingly significant [1]. The manual expertise, effort, and sophisticated reasoning required when applying traditional architecting practices cannot keep up with the demanding architectural needs and operational uncertainty of AI-enabled systems [1]. Combined with the highly dynamic, rapidly changing, and unpredictable nature of the domains in which architectures are applied, for instance in cyber-physical production systems (CPPSs) [4], there is an urgent need to investigate how AI can automate the architectural decision-making process that caters for such domains [1,5,6] in machine learning (ML) [7] and reinforcement learning (RL) [8]-based systems.

In this paper, we examine architecting ML and RL-based systems in the context of our recent research. We identify open research challenges encountered when architecting such systems and consider how AI may be applied to the architectural process. We also report on how we have begun to utilise large language models (LLMs) in addressing research problems to aid practitioners when architecting ML and RL-based systems, present some forthcoming work, and identify open research challenges we plan to address in future work.

We describe our overarching approach, covering topics including the cataloguing of architectural design decisions (ADDs) for the ML workflow and ML deployment, as well as RL training strategies in an Industry 4.0 [9] setting, together with the analysis and assessment of quality aspects in machine learning operations (MLOps) and reinforcement learning operations (RLOps) architectures. We discuss how associated research problems and manual techniques developed to support ML and RL practitioners could be automated by leveraging AI and how AI could replace these manual methods by generating MLOps architectures and providing continuous and self-adapting architectural guidance based on evolving needs. We also describe the problem of assessing projects for RL ADDs and our first foray into LLMs. We describe forthcoming work that utilises LLMs with a low-code approach and a human-in-the-loop to integrate AI into MLOps and RLOps pipeline architecture generation and evolution for open-source projects and an Industry 4.0 case study. Finally, we discuss open challenges and new research opportunities where AI can be applied to software architecting that we will address in the future in collaboration with our Industry 4.0 project partners.

The rest of the paper is structured as follows. In Sect. 2, we introduce key concepts and terminology integral to the topics discussed in the paper. Section 3 describes our approach in the context of research problems and our contributions. In Sect. 4, we introduce open research challenges that we plan to address in the context of our Industry 4.0 project collaboration, and in Sect. 5, we discuss specific aspects of our approach and contributions. We conclude the paper in Sect. 6.

2 Key Concepts and Terminology

This section briefly describes key relevant concepts and terminology, and how they are correlated.

Architectural design decisions (ADDs) are concerned with the series of decisions upon which software architecture is based. Making architectural decisions has a long-term impact on the system and covers decisions that are costly to change. Architects must prioritise, since making an architectural decision involves trade-offs concerning requirements. ADDs are essential for software architecting, since they capture the architectural choices made during the early design process [10,11].

MLOps [12,13] represents a foundational paradigm in contemporary AI engineering that has evolved to address the complexities of deploying and maintaining ML systems in production environments. It constitutes a holistic approach to the ML workflow, applies DevOps [14] and continuous integration/continuous delivery (CI/CD) [15] principles to ML, and involves training, deploying, and managing supervised and unsupervised ML models in production environments [16,17].

RLOps [18] has emerged as a distinct but related paradigm that still represents a critical gap in current academic literature and industrial practice [5]. The extent to which MLOps practices apply to RL, particularly due to major differences between ML and RL concerning model deployment and model training, is still not fully understood. We addressed this deficiency and demystified RLOps by investigating how MLOps principles can be extended to RL contexts through an empirical Industry 4.0 case study focusing on CPPSs [5] with our Industry 4.0 project partners.

Large language models (LLMs)[1] are trained using methods that do not require explicit labelling of the training data. Self-attention in transformer structures helps models deal with long sequences of data by processing it in bulk so that models can use a large number of parameters and access large datasets [19][2].

Industry 4.0 and cyber-physical production systems (CPPSs) [9,20] represent a combination of digital technologies symbolising a significant development in modern manufacturing. Smart factories in Industry 4.0 combine the Internet of Things, AI, and Cloud Computing and are efficient networks interacting automatically and sharing data in real time.

A systematic alignment of technical decisions and operational strategies can be achieved by utilising ADDs as a basis for the architectural generation process for MLOps [21]. Involving LLMs in the architectural generation process increases its efficiency by automating various aspects of architectural synthesis, documentation, and adaptation to changing needs [22]. Such strengths are particularly impactful when transferred to RLOps since RL requires a flexible and resilient architecture [18]. Implemented in the context of Industry 4.0 and CPPSs, these strengths can be used to architect intelligent and interconnected manufacturing

[1] https://en.wikipedia.org/wiki/Large_language_model.
[2] https://www.nvidia.com/en-us/glossary/large-language-models.

environments quickly and continuously improve the development of operational excellence and innovation in complex digital-physical ecosystems [23].

3 Approach

We followed a systematic approach to understand and support how practitioners can better design, build, and manage ML/RL systems in an automated fashion, using AI where appropriate. Our approach has evolved, starting with asking fundamental questions about practitioners' understanding of architectural decisions, and developing into implementing tools in an Industry 4.0 context for CPPSs that utilise AI and can help practitioners make better decisions and automate specific tasks.

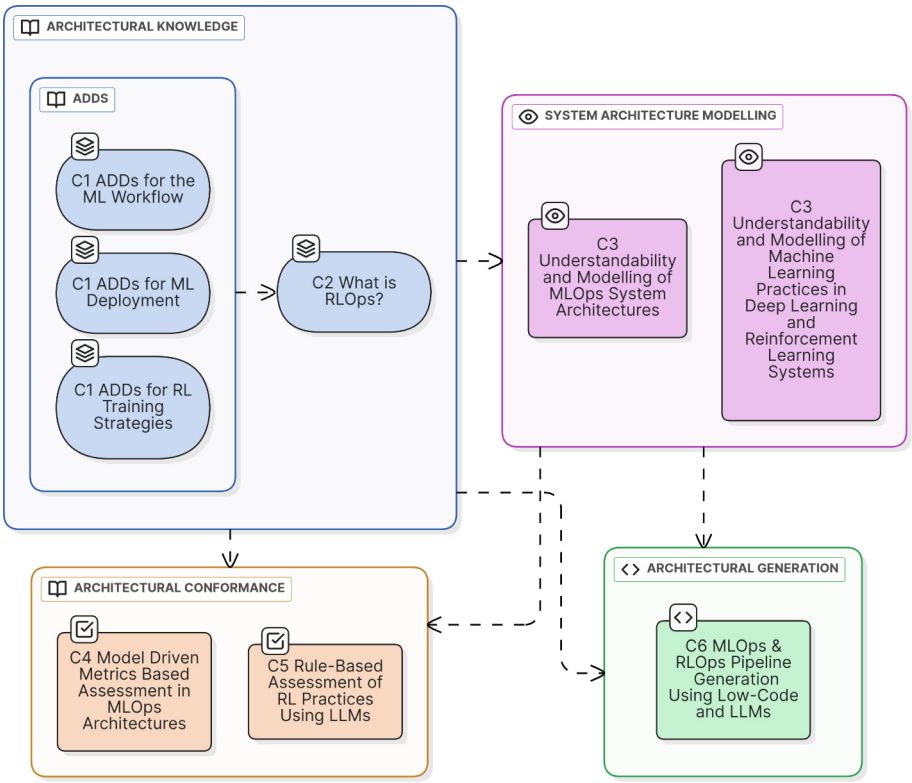

Fig. 1. Overview of Our Approach.

Figure 1 depicts our overarching approach, which consists of several contributions. Fundamental to the overall strategy was the initial consolidation of **Architectural Knowledge** in the form of ADDs for ML and RL and an understanding of emerging paradigms, such as RLOps. We then used this knowledge to

develop a **System Architecture Modelling** method, and tools for automating the assessment of **Architectural Conformance** using detectors and LLMs. **Architectural Knowledge** is also applied to an **Architectural Generation** technique that we developed, using LLMs in a low-code approach. Below, we discuss the various research problems we have addressed so far and outline the details of our contributions.

ADDs for ML and RL

Problem P1 Productionising ML models is technically challenging. This challenge is compounded by knowledge gaps on the level of the individual practitioner and the lack of formalised, reusable architectural knowledge sources. For RL, practitioners have many training, deployment, and coordination strategies to choose from.

Contribution C1 We systematically studied architectural knowledge by conducting three Straussian grounded theory [24] studies of practitioner (grey) literature in various subfields of AI, including the ML workflow, ML deployment, and RL training strategies [16,17,25]. The aim was to catalogue ADDs and current practices and provide design guidance for practitioners since existing practitioner guidelines and best practices are often informal and inconsistent. We developed a novel ADD model and established reusable architectural knowledge in the form of ADDs, which form the foundation of our overarching approach and help bridge the gap between science and practice. Our formal ADD Python models are visualised in automatically generated UML-based model diagrams.

What is RLOps?

Problem P2 The extent to which MLOps and architectural practices apply to RL is poorly understood.

Contribution C2 We conducted an exploratory, qualitative, deductive-inductive industry case study [5] on an Industry 4.0 CPPS and performed content analysis [26] of CPPS artefacts like architectural schematics and source code to understand their relation to known ADDs and associated decision options. In doing so, we fostered an initial understanding of RLOps architectures and provided a reference for practitioners that can be used as design guidance.

MLOps, ML and RL System Architecture Modelling and Understandability

Problem P3 How to model ML and RL-based systems so that they are understandable, can be reasoned about, and programmatically analysed?

Contribution C3 We developed a modelling method for ML and RL-based systems by defining a formal metamodel with appropriate stereotype extensions in Python and automatically generating UML-based visualisations of the modelled systems. We systematically developed the metamodel and modelled freely available system architectures, open source projects, and relevant aspects of our Industry 4.0 partner's CPPS. We conducted user studies in the form of controlled experiments [27,28] to validate how well UML diagrams generated from our models aid system architecture and design comprehension.

Assessing Support for Quality Aspects in MLOps System Architectures

Problem P4 Ensuring conformance to ADDs, patterns, practices, and decision options in MLOps systems is a laborious and error-prone manual task. Objective measures, like metrics, for assessing the ADD options are also lacking.

Contribution C4 To avoid manual validation of architectural conformance, we implemented detectors and novel, technology-agnostic metrics to automatically calculate conformance of modelled MLOps system architectures to known ADDs in the context of quality criteria, such as the level of support for automation practices, in given architectures [29]. We statistically validated our solution, showing that it can be used to assess MLOps architectures programmatically.

Assessing RL Practices Using LLMs

Problem P5 As RL-based systems become increasingly complex, there is a need for standardised and automated detection of conformance to best practices, notably concerning training methods.

Contribution C5 We implemented a rule-based framework that integrates LLMs and heuristic-based code detectors to ensure compliance with best practices in RL training pipelines [30]. Our architectural rules focus on best practices in RL-based architectures that are particularly relevant to CPPSs, like checkpoints, hyperparameter tuning, and agent configuration. We validated our solution via open-source projects and an Industry 4.0 CPPS case study.

Generating MLOps and RLOps Pipelines Architectures using Low-Code and LLMs

Problem P6: Automating the ML/RL workflow requires specialist expertise and considerable manual effort to write and maintain pipeline configurations in an error-prone process that hinders scalability and rapid deployment.

Contribution C6 In forthcoming work, which has been completed but not yet published, we present a new solution for generating pipelines for MLOps and RLOps. Our solution is a low-code, template-based approach that leverages LLMs and a human-in-the-loop to automate pipeline generation, validation, and deployment. We evaluated our solution against seven LLMs, four usage scenarios, three open source projects, and an Industry 4.0 CPPS case study, achieving very low error rates across several metrics. Our solution has the potential to enable rapid scaling and deployment of reliable RLOps pipelines and negates the need for practitioners to have advanced software engineering or DevOps skills.

3.1 Placement of Our Approach in a CI/CD Pipeline

Figure 2 shows how our overall approach fits into the context of CI/CD pipelines for ML and RL. The figure depicts high-level essential steps in a CI/CD pipeline common to ML and RL, which are coloured blue, orange, green, and purple, augmented by contributions from our approach, which are coloured yellow.

The basis for the entire process **C1+C2 Consolidation of Architectural Knowledge**. It is an activity that only needs to be performed rarely, as new practices and patterns are not likely to be documented in grey literature very

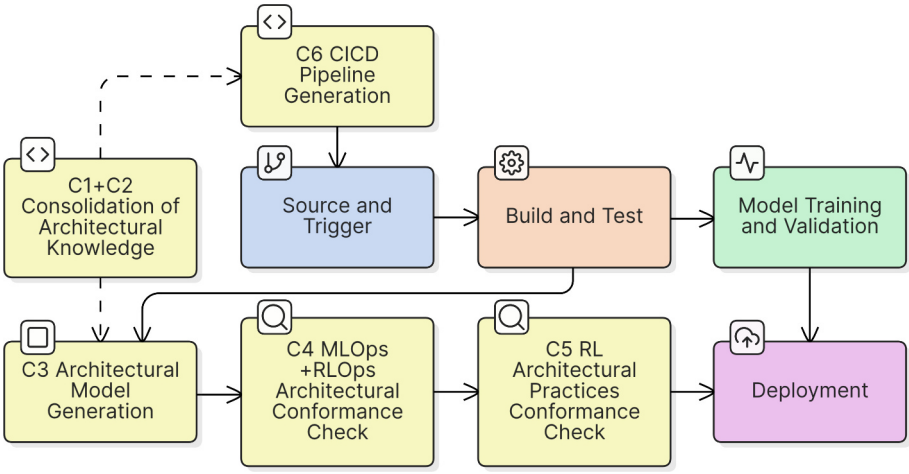

Fig. 2. Placement of Our Approach in a CI/CD Pipeline.

often. Initially, the pipeline is triggered in the usual way, such as when changes to the source code are made or an external component triggers the pipeline, for example, on model performance drift (**Source and Trigger**). In an Industry 4.0 CPPS setting, where frequent changes are made to pipeline configurations, and new pipelines need to be rapidly generated at scale, we can trigger a new pipeline with **C6 CICD Pipeline Generation** when our LLM-based low-code tool generates a new pipeline configuration and commits it, for instance, to a GitLab repository.

Following the **Build and Test** phase, both **Model Training and Validation** and **C3 Architectural Model Generation** can run in parallel. The latter phase uses **C1+C2 Consolidation of Architectural Knowledge** as a basis. Currently, **C3 Architectural Model Generation** is a manual activity due to the distributed and polyglot nature of ML/RL-enabled systems, which makes it hard to create formal models from source code or architectural schematics using traditional methods such as source code parsing and requires considerable expertise in a time-consuming process. However, with modern AI tools, we think system architecture model generation could be automated, and we discuss this further in Sect. 5.

The next step is **C4 MLOps+RLOps Architectural Conformance Check**, which is an automated task that checks for architectural conformance, based on the provided system model, to specific quality attributes under consideration of specific ADDs and associated practices and patterns. This task is manually programmed, but we envisage an enhancement using similar techniques to the next step, **C5 RL Architectural Practices Conformance Check**, where we use LLMs combined with the consolidated **Architectural Knowledge**. We describe this proposed improved process in Sect. 5. **C5 RL Architectural Practices Conformance Check** runs on RL source code and checks

for the presence of specific architectural practices. Assuming the conformance checks, model training, and validation succeeded, we reach the **Deployment** stage, and the pipeline can deploy the model.

4 Open Research Challenges

In collaboration with our Industry 4.0 partners, we have identified several open research challenges that must be addressed. Due to the combination of ML, RL, and Industry 4.0 technologies, software architecture obstacles demand new solutions. This section examines four significant challenges for building reliable and scalable architectures for intelligent manufacturing systems and how AI may help.

Model Versioning for RL in CPPSs
Challenge Traditional methods for managing model versions are not suited to RL in smart factories due to continuous learning by the agents, and this calls for more advanced tools to oversee model updates, track policy changes, and update models to fit the smart factory environment whilst maintaining operational stability. Difficulties arise when several agents are distributed across different parts of a production line and require simultaneous, jointly managed version updates and rollbacks.
Strategy In prior work, we noted the use of a hybrid MLOps-RLOps solution that updates model registries with RL policy metadata, exploration strategies, and environment interaction history [5]. The solutions involve using distributed version control to track model parameters and learned policies, and the setup of rollback mechanisms [31]. AI-powered automated version management might choose the most effective model using AI meta-learning techniques and analysing prior performance records and the current context.

Communication Protocols for RL in Industry 4.0
Challenge Current communication schemas, such as the Model Context Protocol (MCP)[3], whilst useful for AI-tool interactions, do not cover necessary architectural designs used in industrial fields where critical operations, real-time constraints, and legacy systems predominate. Protocol implementations mostly handle desktop applications and simple AI assistants, and they do not cover the needs of industry, which require reliability, rigid response times, and compatibility with current manufacturing systems.
Strategy One strategy is to work on system architectures built for industry, adding real-time guarantees, secure authentication and manufacturing-oriented MCP-like protocol architectures[4]. A solution could build federated networks that cross multiple security domains in a factory, use special tools for data transfer over industrial networks and facilitate interoperability of networks with other

[3] https://www.anthropic.com/news/model-context-protocol.
[4] https://www.linkedin.com/pulse/securing-model-context-protocol-mcp-architecture-best-srivastava-v4hgf.

popular industrial protocols. AI could also help facilitate protocol optimisation as network and production changes occur.

Agentic AI Architecture for CPPSs
Challenge Using agentic AI in CPPSs means building architectures that can handle complex decisions on their own, whilst making sure the AI is secure, reliable and works with previous control systems [32][5]. Currently, agentic systems mainly concentrate on software environments and fail to include industry-relevant error handling, latency assurances and reliability guarantees[6]. It is essential to ensure that changing behaviours of autonomous agents do not disrupt the orderly operations of manufacturing [33].
Strategy A strategy could emphasise architectures that ensure safety-related functions are under separate control from autonomous agents, using advanced verification techniques on agent systems and developing hybrid human-agent supervision models [32]. Some ways to reduce risks are to precisely define agent human responsibilities, standardise interfaces between agent systems and standard industrial controls, and to include continuous monitoring and intervention methods [31]. AI-assisted optimisation in architecture could influence agent autonomy according to risks and outcomes in production.

Support for RL in CI/CD Pipelines
Challenge Existing frameworks for CI/CD fail to adequately support RL systems since they do not support many of the unique needs of RL, like those described in the challenges above (versioning for RL models, support for RL communication protocols, and agentic AI systems). The need is for CI/CD pipelines that understand complex deployment environments and can learn system evolution [31] since traditional software deployment methods do not consider the unique demands of learning and adaptation after release [5].
Strategy One strategy is to build multi-step deployment systems for RL artefacts, create automated systems to test agent skills, and design monitoring tools that notice changes in production environments [31]. AI-supported pipeline configuration can bootstrap the appropriate deployment methods given the model type, risks, and production needs, with feedback loops that improve the system in controlled production settings [5].

5 Discussion

In Sect. 3, we described our overall approach. We noted that some of the solutions we developed to address specific challenges earlier in our research could potentially be enhanced due to recent progress that has been made in AI. This section discusses how AI could improve some residual manual aspects of our approach. Concrete details, such as pipeline examples, architectures, controlled experiment results, limitations of our approach, and threats to validity for each

[5] https://techcommunity.microsoft.com/blog/machinelearningblog/baseline-agentic-ai-systems-architecture/4207137.
[6] https://www.ibm.com/think/topics/agentic-architecture.

contribution, are discussed in the respective papers. We also reference replication packages for our prior contributions in the corresponding papers.

ML models can now scan technical documentation, find architectural patterns and decisions made in the design, and group system elements based on source code and architectural diagrams[7, 8]. Recent studies show that LLMs and generative AI can significantly improve grounded theory analysis by automating coding, finding patterns, and extracting themes without compromising the accuracy of qualitative work. For example, Übellacker [34] describes AcademiaOS, which is an initial attempt to automate grounded theory using LLMs by applying the understanding, generation, and reasoning capabilities of LLMs to augment humans in qualitative research. Yue et al. [35] apply ChatGPT-4 Turbo to grounded theory and describe how LLMs can enhance the efficiency of text coding and qualitative analysis in grounded theory. When gathering **Architectural Knowledge** (**C1** and **C2**), rather than periodically manually performing grounded theory and content analysis studies, which can be time-consuming, we could consider reducing manual work by using AI tools to implement alternative solutions. An advantage is that this knowledge-gathering step could be more easily automated within a CI/CD pipeline. The gathered knowledge is more frequently updated to use in later CI/CD steps that implement our approach and make use of this knowledge, such as **C3 Architectural Model Generation** and the **C4 MLOps+RLOps Architectural Conformance Check**.

LLMs can now build structural and behavioural models for systems without relying on human-written code. Latest findings also suggest that LLMs now allow systems to be modelled, for example, with SysML[9] diagrams automatically, helping lower the need to do this work manually. Apvrille and Sultan [36] describe how LLMs can make sense of natural text descriptions and generate content fitting predefined formats, supporting the automation of model generation. They introduce a framework where LLMs automatically construct structural and behavioural SysML diagrams from system specifications. **Contribution C3 Architectural Model Generation** in Fig. 2 is currently a manual process in which the practitioner models the system architecture in Python using our extensible MLOps/RLOps metamodel. This formal system model can then be processed in later stages of the pipeline, for instance during the **C4 MLOps+RLOps Architectural Conformance Check**. Rather than generating diagrams directly, we envisage an enhanced solution that would make use of generative AI to write Python code, based on our CodeableModels[10] metamodel, to generate the formal system architecture models. This automated stage would dramatically reduce the required modelling effort in the early stages of a project. Optionally, the tool could subsequently generate UML visualisations from the Python models using PlantUML[11] as it currently does.

[7] https://www.westat.com/machine-learning-nlp.
[8] https://mindthegraph.com/blog/automated-content-analysis.
[9] https://sysml.org.
[10] https://github.com/uzdun/CodeableModels.
[11] https://github.com/plantuml/plantuml.

Contribution C4 (MLOps+RLOps Architectural Conformance Check) substantially reduces the need for manual assessment of MLOps/RLOps architectures for conformance to known ADDs and decision options and allows for automation in a CI/CD pipeline. However, some manual effort is still involved in implementing the detectors that analyse the models, and the metrics that evaluate the support for quality aspects based on the applied decision options. An upgraded solution could use LLMs similarly to our implementation of **C5 RL Architectural Practices Conformance Check** by examining the Python models generated using **C3 Architectural Model Generation** based on the Architectural Knowledge from **C1+C2 Consolidation of Architectural Knowledge**. Recent related work describes applying LLMs to architectural conformance checking. Keshri et al. [37] present a novel system for verifying code implementations against the algorithms and methodologies from corresponding research papers. Their solution uses retrieval-augmented generation [38] to extract relevant details from both the research papers and code bases, and compares them using LLMs to reduce manual effort, enhance research credibility, and advance the state of the art in code verification.

6 Conclusion

This paper outlines our approach towards using AI for architecting ML and RL-based systems, particularly for Industry 4.0 CPPSs. We outlined six research problems and how we addressed them through Straussian grounded theory-based grey literature studies, the development of a novel ADD metamodel in Python, an exploratory industry case study on an Industry 4.0 CPPS, the development of a novel, formal metamodel for modelling ML and RL-based systems, controlled experiments testing whether providing semi-formal system and architecture representations as UML diagrams helps understanding of MLOps system architectures compared to informal system architecture diagrams, a method for automatically checking conformance of MLOps architectures to known ADDs, the assessment of conformance to RL best practices in RL code using LLMs, and the generation of MLOps and RLOps pipeline architecture using low-code and LLMs. We showed where we have already applied AI techniques, and described how we could further enhance our approach with AI.

Using our approach in CI/CD pipelines helps automate much of the process. Still, some significant open challenges remain: advanced ways to version models for distributed RL agents, effective communication protocols for industrial applications, safe integration of agents in AI systems, and RL integration into CI/CD pipelines. We described these four open challenges and possible strategies to address them. Future work will address the identified open challenges and further automate our approach in collaboration with industry partners. Our research helps to form a basis for intelligent, adaptive systems that can constantly adapt to fit the dynamic demands of modern production systems and lead to functional AI implementation in industry when architecting ML and RL-based systems.

Acknowledgements. This work was supported by the FFG (Austrian Research Promotion Agency) project MODIS (no. FO999895431). This research was funded in whole or in part by the Austrian Science Fund (FWF) project CQ4CD, Grant-DOI: 10.55776/I6510. For open access purposes, the authors have applied a CC BY public copyright licence to any author-accepted manuscript version arising from this submission.

References

1. Bucaioni, A., Weyssow, M., He, J., Lyu, Y., Lo, D.: Artificial intelligence for software architecture: literature review and the road ahead. In: 2030 Software Engineering - 2025 (2025). http://www.ipr.mdu.se/publications/7175-
2. Zhou, X., et al.: Using LLMs in generating design rationale for software architecture decisions (2025). https://arxiv.org/abs/2504.20781
3. Gheibi, O., Weyns, D., Quin, F.: Applying machine learning in self-adaptive systems: a systematic literature review. ACM Trans. Auton. Adapt. Syst. **15**(3) (2021). https://doi.org/10.1145/3469440
4. Monostori, L.: Cyber-physical production systems: roots, expectations and R&D challenges. Procedia CIRP **17**, 9–13 (2014). https://doi.org/10.1016/j.procir.2014.03.115, https://www.sciencedirect.com/science/article/pii/S2212827114003497, Variety Management in Manufacturing
5. Warnett, S.J., Zdun, U.: Bridging the gap between MLOps and RLOps: an Industry 4.0 case study on architectural design decisions in practice. In: 2025 IEEE 22nd International Conference on Software Architecture (ICSA), pp. 232–242 (2025). https://doi.org/10.1109/ICSA65012.2025.00031
6. Ramic, A., Kugele, S.: A Systematic Mapping Study on Software Architecture for AI-based Mobility Systems (2025). https://arxiv.org/abs/2506.01595
7. Jordan, M.I., Mitchell, T.M.: Machine learning: trends, perspectives, and prospects. Science **349**(6245), 255–260 (2015)
8. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. MIT Press (2018)
9. Singh, H., Singh, B.: Industry 4.0 technologies integration with lean production tools: a review. TQM J. (2024). https://doi.org/10.1108/TQM-02-2022-0065
10. Jansen, A., Bosch, J.: Software architecture as a set of architectural design decisions. In: 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05), pp. 109–120 (2005). https://doi.org/10.1109/WICSA.2005.61
11. Shahin, M., Liang, P., Li, Z.: Do architectural design decisions improve the understanding of software architecture? two controlled experiments. In: Proceedings of the 22nd International Conference on Program Comprehension. ICPC 2014, New York, NY, USA, pp. 3–13. Association for Computing Machinery (2014). https://doi.org/10.1145/2597008.2597139
12. Kreuzberger, D., Kühl, N., Hirschl, S.: Machine learning operations (MLOps): overview, definition, and architecture. arXiv:abs/2205.02302 (2022)
13. Hewage, N., Meedeniya, D.: Machine learning operations: a survey on MLOps tool support (2022). https://doi.org/10.48550/ARXIV.2202.10169
14. Bass, L., Weber, I., Zhu, L.: DevOps: A Software Architect's Perspective, 1st edn. Addison-Wesley Professional (2015)

15. Humble, J., Farley, D.: Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, 1st edn. Addison-Wesley Professional (2010)
16. Warnett, S.J., Zdun, U.: Architectural design decisions for the machine learning workflow. Computer **55**(3), 40–51 (2022). https://doi.org/10.1109/MC.2021.3134800
17. Warnett, S.J., Zdun, U.: Architectural design decisions for machine learning deployment. In: 2022 IEEE 19th International Conference on Software Architecture (ICSA), pp. 90–100 (2022). https://doi.org/10.1109/ICSA53651.2022.00017
18. Li, P., et al.: RLOps: development life-cycle of reinforcement learning aided open RAN. IEEE Access **10**, 113808–113826 (2022). https://doi.org/10.1109/ACCESS.2022.3217511
19. Vaswani, A., et al.: Attention is all you need. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS'17, Red Hook, NY, USA, pp. 6000–6010. Curran Associates Inc. (2017)
20. Khdoudi, A., Masrour, T., El Hassani, I., El Mazgualdi, C.: A deep-reinforcement-learning-based digital twin for manufacturing process optimization. Systems **12**(2) (2024). https://doi.org/10.3390/systems12020038, https://www.mdpi.com/2079-8954/12/2/38
21. Leest, J., Gerostathopoulos, I., Raibulet, C.: Evolvability of machine learning-based systems: an architectural design decision framework. In: 2023 IEEE 20th International Conference on Software Architecture Companion (ICSA-C), pp. 106–110 (2023). https://doi.org/10.1109/ICSA-C57050.2023.00033
22. Dhar, R., Vaidhyanathan, K., Varma, V.: Can LLMs generate architectural design decisions? - an exploratory empirical study. In: 2024 IEEE 21st International Conference on Software Architecture (ICSA), pp. 79–89 (2024). https://doi.org/10.1109/ICSA59870.2024.00016
23. Fischbach, A., et al.: CAAI—a cognitive architecture to introduce artificial intelligence in cyber-physical production systems. Int. J. Adv. Manuf. Technol. **111**, 1–18 (2020). https://doi.org/10.1007/s00170-020-06094-z
24. Corbin, J., Strauss, A.L.: Grounded theory research: procedures, canons, and evaluative criteria. Qual. Sociol. **13**, 3–20 (1990)
25. Ntentos, E., Warnett, S.J., Zdun, U.: supporting architectural decision making on training strategies in reinforcement learning Architectures. In: 2024 IEEE 21st International Conference on Software Architecture (ICSA), pp. 90–100 (2024). https://doi.org/10.1109/ICSA59870.2024.00017
26. DeFranco, J., Laplante, P.: A content analysis process for qualitative software engineering research. Innovations Syst. Softw. Eng. **13**, 1–13 (2017). https://doi.org/10.1007/s11334-017-0287-0
27. Warnett, S.J., Zdun, U.: On the understandability of MLOps system architectures. IEEE Trans. Software Eng. **50**(5), 1015–1039 (2024). https://doi.org/10.1109/TSE.2024.3367488
28. Ntentos, E., Warnett, S.J., Zdun, U.: On the understandability of machine learning practices in deep learning and reinforcement learning based systems. J. Syst. Software **222**, 112343 (2025). https://doi.org/10.1016/j.jss.2025.112343, https://www.sciencedirect.com/science/article/pii/S0164121225000111
29. Warnett, S.J., Ntentos, E., Zdun, U.: A model-driven, metrics-based approach to assessing support for quality aspects in MLOps system architectures. J. Syst. Software **220**, 112257 (2025). https://doi.org/10.1016/j.jss.2024.112257, https://www.sciencedirect.com/science/article/pii/S0164121224003017

30. Ntentos, E., Warnett, S.J., Zdun, U.: Rule-based assessment of reinforcement learning practices using large language models. In: 2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN), pp. 1–11 (2025). https://doi.org/10.1109/CAIN66642.2025.00009
31. Faubel, L., Schmid, K.: MLOps: a multiple case study in Industry 4.0. In: 2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA), pp. 01–08 (2024). https://doi.org/10.1109/ETFA61755.2024.10711136
32. Baldoni, M., Baroglio, C., Ditano, V., Micalizio, R., Tedeschi, S.: Agents for Industry 4.0: the case study of a production cell. In: Workshop From Objects to Agents (2023). https://api.semanticscholar.org/CorpusID:266211460
33. Kegyes, T., Süle, Z., Abonyi, J.: The applicability of reinforcement learning methods in the development of Industry 4.0 applications. Complexity **2021**(1), 7179374 (2021). https://doi.org/10.1155/2021/7179374
34. Übellacker, T.: AcademiaOS: automating grounded theory development in qualitative research with large language models (2024). https://arxiv.org/abs/2403.08844
35. Yue, Y., Liu, D., Lv, Y., Hao, J., Cui, P.: A practical guide and assessment on using ChatGPT to conduct grounded theory: tutorial. J. Med. Internet Res. **27**, e70122 (2025), https://doi.org/10.2196/70122
36. Apvrille, L., Sultan, B.: System Architects are not alone anymore: automatic system modeling with AI. In: Mayo, F.J.D., Pires, L.F., Seidewitz, E. (eds.) Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering, MODELSWARD 2024, Rome, Italy, February 21–23, 2024, pp. 27–38. SCITEPRESS (2024)
37. Keshri, R., Zachariah, A., Boone, M.: Enhancing code consistency in AI research with large language models and retrieval-augmented generation (2025). https://doi.org/10.48550/arXiv.2502.00611
38. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proceedings of the 34th International Conference on Neural Information Processing Systems. NIPS '20, Red Hook, NY, USA. Curran Associates Inc. (2020)

AMP Workshop

Assessing the Impact of Asynchronous Communication on Resilience and Robustness: A Comparative Study of Microservice and Monolithic Architectures

Nathanael Bosilia, Gerald Weinberger, and Philipp Haindl(✉)

Department of Computer Science and Security, St. Pölten University of Applied Sciences, St. Pölten, Austria
{is211308,gerald.weinberger,philipp.haindl}@fhstp.ac.at

Abstract. *Context:* Microservice architectures offer advantages such as scalability and independent deployment compared to traditional monolithic systems. Nonetheless, the specific impact of asynchronous communication patterns within microservice architectures on critical quality attributes, such as resilience and robustness, particularly when directly compared to monolithic systems under failure conditions, remains insufficiently explored empirically. *Objective:* This study empirically examines the influence of asynchronous communication mechanisms on the resilience and robustness of microservice architectures in comparison to equivalent monolithic architectures, identifying associated benefits and operational trade-offs. *Method:* Two equivalent systems representing a simplified e-commerce order process were developed: a synchronous monolith and an event-driven microservice architecture utilizing asynchronous communication. Controlled Chaos Engineering experiments simulating service crashes were conducted under load. Key metrics, including failure rate, system availability, request duration, and processed requests, were measured and compared across architectures under identical fault scenarios. *Results:* The asynchronous microservice architecture demonstrated significantly enhanced resilience and robustness under simulated failures. Compared to the monolith, it exhibited lower failure rates (up to a 12% reduction observed), maintained higher availability, and processed requests more effectively during fault conditions. Asynchronous decoupling localized failures and facilitated continued partial operation. However, the microservice architecture exhibited increased operational complexity and higher resource requirements. *Conclusion:* Asynchronous communication substantially improves the resilience and robustness of microservice architectures against failures compared to traditional monolithic approaches. While offering significant fault tolerance advantages, the adoption necessitates careful consideration of increased operational complexity, resource consumption, and the need for sophisticated monitoring and debugging infrastructures.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 171–186, 2026.
https://doi.org/10.1007/978-3-032-04403-7_16

Keywords: Microservice Architecture · Monolithic Architecture · Asynchronous Communication · Resilience · Robustness · Event-Driven Architecture · Chaos Engineering

1 Introduction

In recent years, software architecture has undergone significant transformations, primarily driven by the demands for scalability, flexibility, and maintainability. Microservice architecture has emerged as a prominent paradigm, characterized by the decomposition of applications into small, independently deployable services that communicate via lightweight protocols, often utilizing asynchronous messaging patterns [10,13]. Each microservice typically encapsulates a specific business capability and operates autonomously, enabling independent development, deployment, and scaling without impacting other services [12,16].

Newman [16] notes that effective microservice implementations usually embrace automation, decentralized governance, failure isolation, and observability to maintain service independence. Similarly, Richardson [21] advocates for each microservice to own its data store, which bolsters service isolation by facilitating independent development and scaling. Moreover, Fowler [10] defines microservices as an approach to developing a single application as a suite of small services, each running in its own process and communicating through lightweight mechanisms, with minimal centralized management and flexibility to employ diverse programming languages and data storage technologies.

The autonomy conferred by microservice architecture allows organizations to adopt heterogeneous technology stacks and deploy updates rapidly, thereby enhancing agility and resilience in dynamic environments [5]. However, this architectural approach typically requires higher computational resource allocation. In our experimental setup, the microservice architecture required three independent service instances plus message broker infrastructure, with each service allocated 1 vCPU and 512 MiB RAM, resulting in a total resource footprint that exceeded the monolithic counterpart by a factor of approximately four when including the Apache Kafka messaging infrastructure. In contrast, monolithic architectures encapsulate all application functionality within a single, tightly coupled codebase [5]. All modules share a common memory space and are executed as a single process, which simplifies development, testing, and deployment for small to moderately complex systems [7]. However, as systems expand, monoliths often encounter scalability bottlenecks, maintenance challenges, and an increased risk of cascading failures, as a fault in one module can propagate throughout the entire system [20].

1.1 Motivation and Problem Statement

While microservices offer notable benefits in terms of scalability and independent deployment [20], they also introduce complexities, particularly concerning inter-service communication, consistency management, and failure handling [23].

Asynchronous communication, frequently implemented through message brokers (e.g., Apache Kafka), has been proposed as a solution to address the challenges associated with synchronous communication, including tight coupling and cascading failures [9,19,24]. Event-driven architectures and messaging patterns play a crucial role in reducing resource consumption, enhancing fault tolerance, and decoupling service interactions [6,19].

Nevertheless, the practical implications of asynchronous communication on critical quality attributes such as resilience and robustness remain insufficiently examined, particularly in direct comparison to monolithic systems. Existing research predominantly concentrates on general performance and scalability comparisons. For instance, Blinowski *et al.* [5] and Horváth *et al.* [11] present evidence indicating that microservice architectures achieve superior scalability in high-load scenarios, albeit occasionally at the expense of increased latency due to network overhead. Rodrigues *et al.* [22] and Razzaq and Ghayyur [20] explore the broader implications of transitioning from monolithic to microservice systems, highlighting challenges such as operational complexity and inconsistent performance gains.

Berry *et al.* [3] emphasize that while microservices can enhance resource utilization through horizontal scaling, certain components may become bottlenecks if not adequately replicated. Di Francesco *et al.* [8] and Soldani *et al.* [23] further note that the industrial adoption of microservices often encounters significant challenges alongside the benefits, particularly in managing distributed failures and ensuring robustness. Early reports, such as those by Thönes [25] and Fowler [10], offered optimistic perspectives on microservice adoption; however, empirical evidence addressing fault tolerance remains limited.

1.2 Research Questions

This study seeks to fill this gap by empirically examining the impact of asynchronous communication mechanisms on the resilience and robustness of microservice architectures, in direct comparison to traditional monolithic architectures. The comparison between asynchronous microservices and synchronous monoliths is valuable as it represents the most common architectural transition scenario in practice, where organizations migrate from synchronous, tightly-coupled monolithic systems to event-driven, loosely-coupled microservice architectures. This architectural transformation fundamentally changes both the deployment model (single process vs. distributed) and the communication paradigm (synchronous vs. asynchronous), making it essential to understand the combined impact of these changes on system resilience and robustness. To this end, we have formulated the following research questions:

RQ1. What is the impact of architectural style, specifically microservices with asynchronous communication versus monolithic architecture, on system resilience in the face of failures?

RQ2. How does the choice of architectural style influence system robustness, such as error rates and response times under load, during simulated fault conditions?

RQ3. What operational challenges and trade-offs, including complexity and resource usage, are observed when comparing these two architectural approaches in this context?

Analyzing the behavior of these architectures under failure scenarios is essential for assessing their appropriateness for systems that require high availability and reliability [5,11].

1.3 Structure of the Paper

The structure of this paper is organized as follows: Sect. 2 provides a review of pertinent scholarly literature. Section 3 elaborates on the methodology, encompassing the design of the experimental systems and the application of the Chaos Engineering approach. Section 4 presents the empirical findings obtained from the experiments, addressing each research question. Section 5 interprets these findings, connecting them to existing literature and practical implications. Potential threats to validity are examined in Sect. 6. Finally, Sect. 7 offers concluding remarks and proposes directions for future research.

2 Related Work

We have identified three principal streams of related research.

2.1 Comparative Performance and Scalability Studies

Studies have compared the performance, scalability, and costs of monolithic and microservice architectures. Blinowski *et al.* [5] evaluated monolithic and microservice systems in Java and C# .NET across deployment environments. Their load tests showed monolithic systems perform better on single machines, while microservices excel in distributed environments, though scaling beyond certain instances caused performance issues. Horváth *et al.* [11] conducted load testing on file-sharing systems, confirming microservices' superior scalability under dynamic loads but noted increased complexity and latency. Rodrigues *et al.* [22] reviewed performance and costs of transitioning from monolithic to microservice systems, finding conflicting results that necessitate context-specific evaluations. Villamizar *et al.* [26] compared deployment strategies in cloud environments, concluding microservices' benefits depend heavily on workload patterns and system design. A common limitation across these studies is the insufficient focus on resilience and robustness metrics under fault conditions, with most experiments being performance-driven rather than quality-attribute-driven. Another relevant study in this area is by Bjørndal *et al.* [4], which benchmarks a case study of migration from monolith to microservices.

2.2 Impact of Communication Patterns in Microservices

Research explores the impact of communication patterns on microservice systems, with a particular focus on asynchronous communication. Srijith *et al.* [24] investigated the use of Apache Kafka Connect for inter-service communication among microservices, suggesting pipeline optimization to alleviate bottlenecks. Their findings indicated that adjusting Kafka producers and consumers reduced CPU and memory overhead. Rahmatulloh *et al.* [19] evaluated event-driven architecture (EDA) in microservices, discovering that EDA systems achieved a 19.18% faster response time and a 34.40% lower error rate compared to API-driven architectures, albeit with increased CPU usage. Cabane and Farias [6] examined the performance impact of event-driven architectures, focusing on the trade-offs between complexity and performance. Fehlinget *et al.* [9] cataloged cloud computing patterns, including event-driven resilience patterns for asynchronous microservice systems. Limitations of these studies include a focus on performance over resilience, with limited evaluation of real-world fault tolerance.

2.3 Quality Attributes and Design Patterns in Microservices

Recent research has focused on the impact of microservices on quality attributes and design patterns for addressing quality attributes. Li *et al.* [13] conducted a systematic review, identifying six key quality attributes: scalability, performance, availability, monitorability, security, and testability. They synthesized architectural tactics, noting maintainability needs further investigation. Pinciroli *et al.* [17] used queuing networks to analyze seven microservices design patterns' performance. Their evaluation quantified pattern trade-offs, though large-scale validation was limited. Meijer *et al.* [14] provided experimental validation of architectural patterns, focusing on service latency and resource utilization. Their findings showed real-world performance aligns with model predictions, highlighting evaluation complexity in heterogeneous environments. Jamshidiet *et al.* [12] addressed challenges including resilience engineering, deployment complexity, and evaluation methodologies. Limitations include difficulty quantifying complex attributes and limited empirical studies complementing theoretical modeling.

3 Methodology

In order to systematically evaluate the impact of asynchronous communication mechanisms on the resilience and robustness of microservice architectures as compared to monolithic architectures, a controlled experimental framework was established. This section outlines the system prototypes developed, the experimental design, including baseline tests and fault injection methodology, as well as the metrics employed for evaluation. To enable replication of our study, we also provide the source code and the raw experimental data[1].

[1] https://doi.org/10.5281/zenodo.15740374.

3.1 System Prototypes

Two software systems were developed as experimental artifacts: a monolithic system and a microservice-based system utilizing asynchronous communication. Both systems were designed to simulate the order processing workflow of a simplified e-commerce platform, including order creation, simulated shipment processing, and customer notification. Importantly, the core business logic was consistently maintained across both systems, and a uniform technology stack was employed where feasible to ensure a comparable baseline. A figure illustrating the high-level architecture of both applications can be found in the supplementary material to aid understanding.

Monolithic Architecture. The monolithic system was implemented as a single deployable Java (v23) application using the Spring Boot (v3.4.1) framework. All functionalities - order validation/creation, shipment initiation, and notification generation - were executed sequentially within a single process. Internal coordination between functional modules relied on synchronous, in-process method calls. The system utilized a PostgreSQL (v17.2) database. Deployment was managed as a single unit within a Kubernetes (v1.32.1) cluster using Docker (v27.4.0) containerization.

Microservice Architecture. The microservice system decomposed the application into three independent services: an *Ordering Service* (handling incoming order requests and validation), a *Shipping Service* (simulating shipment processing), and a *Notification Service* (handling user notifications). Each service was developed independently using Java (v23) and Spring Boot (v3.4.1), each connected to its own PostgreSQL (v17.2) database, and packaged as separate Docker containers. Orchestration was managed via Kubernetes (v1.32.1) using Minikube (v1.35.0) locally. Inter-service communication was implemented asynchronously through an event-driven architecture using Apache Kafka (v3.9.0), deployed via Strimzi (v0.45.0). Specific Kafka topics (*orderCreatedTopic*, *shipmentCreatedTopic*) facilitated the event flow between services.

3.2 Experiment Design

The experimental phase included three experiments. The first established baseline performance under load without failures, while the next two used Chaos Engineering [1,2]. This involves controlled experimentation by injecting failures to build confidence in a system's ability to withstand turbulence. In these experiments, faults were introduced into systems under controlled load to evaluate behavior and resilience.

– **Experiment ❶ (Baseline Load Tests):** This experiment aimed to establish the baseline performance and behavior of both systems under high load conditions without induced faults. Workload was generated using k6 (v0.54.0)

in *constant-vus* mode, maintaining 20 virtual users (VUs) for a duration of two minutes. This level of load was determined through preliminary testing to stress system resources near their limits without causing instability. Results from this experiment served as a reference point for evaluating the impact of failures in subsequent experiments.

– **Experiment ❷ and ❸ (Fault Injection Scenarios):** These experiments utilized Chaos Mesh (v2.7.0) to inject faults while the systems were under load generated by Grafana k6 (v0.54.0). The primary fault scenario involved simulating service crashes by terminating Kubernetes pods. For experiment ❷, the pod running the monolith application or the *Ordering Service* (the edge service in the microservice architecture) was terminated periodically. For experiment ❸, a pod corresponding to one of the three microservices was terminated randomly during the test runs.

A consistent workload was applied using k6 configured for a *constant-arrival-rate* of 10 requests per second over a duration of two minutes, utilizing up to 40 VUs to maintain the target rate under potential performance degradation. Each experimental run was preceded by checks to ensure service availability and followed by system restarts and database clearing to ensure consistent starting conditions for subsequent runs.

3.3 Metrics and Data Collection

Table 1 presents the systematically collected metrics during and subsequent to the experiments, aimed at evaluating the performance, resilience, and robustness of the two architectures. Data was obtained from k6 output logs and direct database queries following each test execution.

Table 1. Metrics Collected for the Evaluation.

Category	Metric Name	Description
Business Metrics	Orders Processed	Successfully processed orders.
	Shipments Processed	Simulated shipments created.
	Notifications	Generated notifications.
System Metrics	HTTP Requests	HTTP requests processed.
	HTTP Request Duration	Request duration statistics (ms).
	Failed HTTP Requests	HTTP request failure rate (%).

Results from multiple iterations were aggregated, and descriptive statistics, including mean, minimum, and maximum values, were computed using Python scripts. For experiment ❸, log data from the *Notification Service* was additionally utilized to distinguish between *Order* and *Shipment* notifications.

4 Results

This section delineates the empirical findings obtained from the comparative experiments conducted on the monolithic and microservice system prototypes. The results are systematically organized in accordance with the three research questions that guide this study. The data presented are derived from multiple iterations of each experimental scenario, including baseline load testing, targeted service failure, and random service failure.

4.1 RQ1: Impact on Resilience

In RQ1 we evaluated the systems' capacity to process requests successfully despite the presence of induced faults. The primary metrics considered were the number of successfully processed business transactions (*Orders*, *Shipments*, *Notifications*) and the HTTP request failure rate. Experiment ❷ involved simulating the failure of the primary entry point, which could be either the monolith itself or the *Ordering Service* within the microservice architecture. Table 2 presents a comparison of the business metrics and system-level request handling under this specific failure scenario.

Table 2. Targeted Failure of Entry Point (Mean Values).

Metric	Monolith	Microservices
Orders Processed	956	984
HTTP Requests	1,200	1,200
Failed HTTP Requests (%)	20.35	18.04

As illustrated, when the primary request-handling component failed, the asynchronous microservice architecture processed a slightly higher number of successful orders (approximately 3% more) and demonstrated a lower HTTP request failure rate (approximately 2.3% points lower) compared to the monolithic architecture. This indicates a marginal resilience advantage even when the main entry point is compromised, potentially attributable to faster recovery or startup times of the smaller service instance, although this difference was minor in our setup. Experiment ❸ introduced random failures among any of the three microservices, while comparing against the monolith failure data from experiment ❷, as a monolith failure inherently affects all functions. Table 3 presents the results under these random internal failures.

The results from experiment ❸ demonstrate a pronounced resilience advantage for the asynchronous microservice architecture when internal components fail randomly. The microservice architecture processed significantly more successful orders (approx. 15.7% more) compared to the baseline monolith failure scenario. More notably, the average HTTP request failure rate dropped drastically from 20.35% in the monolith failure case to just 7.88% in the random

Table 3. Random Internal Service Failure (Mean Values).

Metric	Monolith	Microservices
Orders Processed	956	1,106
HTTP Requests	1,200	1,200
Failed HTTP Requests (%)	20.35	7.88

failure case of the microservice architecture. This highlights the fault isolation capability provided by the decoupled microservices and asynchronous communication; failures in downstream services (*Shipping*, *Notification*) did not prevent the *Ordering service* from accepting and acknowledging new requests, which were queued in Kafka for later processing.

4.2 RQ2: Impact on Robustness Under Fault Conditions

The evaluation of robustness under fault conditions was primarily conducted by analyzing HTTP request duration and error rates. This involved comparing the system's behavior during experiments ❷ and ❸ with the baseline performance established in experiment ❶, which was a load test conducted without faults. Table 4 provides a summary of the baseline performance under load conditions without the introduction of failures.

Table 4. Baseline Performance Under Load.

Metric	Monolith	Microservices
Processed Orders (in 2 min)	21,421	61,124
Requests per Second (reqs/s)	178.41	509.19
HTTP Req. Duration (ms) - Mean	112.11	39.18
HTTP Req. Duration (ms) - Min	3.20	2.13
HTTP Req. Duration (ms) - Max	1,622.00	1,170.00
Failed HTTP Requests (%)	0.00	0.00

Under the specified baseline conditions, the asynchronous microservice architecture exhibited a markedly higher throughput, approximately 185% more requests per second, and a significantly reduced average request duration, approximately 65% faster than the monolithic architecture. This performance enhancement is likely attributable to the asynchronous nature of the system, which enables the edge service (*Ordering Service*) to respond promptly without necessitating the completion of the entire downstream process (*Shipping*, *Notification*) in a synchronous manner, as is required in the monolithic architecture. Table 5 provides the request duration metrics observed during the targeted failure scenario of experiment ❷.

Table 5. Request Duration with Entry Point Failure.

Metric	Monolith	Microservices
HTTP Req. Duration (ms) - Mean	59.57	33.58
HTTP Req. Duration (ms) - Min	7.42	5.48
HTTP Req. Duration (ms) - Max	2,680.00	1,925.00

As demonstrated in Table 5, during the targeted service failure (experiment ❷), the microservice architecture exhibited significantly improved response times, with an average duration approximately 43% shorter than that of the monolithic architecture. Additionally, the maximum latency was considerably reduced in the microservice architecture. This suggests enhanced robustness in terms of performance consistency, even when the primary service endpoint encounters instability.

Table 6 presents the request duration metrics during the random internal failure scenario of experiment ❸ (microservices only), compared to the monolith's performance during its failure scenario (experiment ❷).

Table 6. Request Duration with Random Internal Failure.

Metric	Monolith	Microservices
HTTP Req. Duration (ms) - Mean	59.57	24.87
HTTP Req. Duration (ms) - Min	7.42	4.99
HTTP Req. Duration (ms) - Max	2,680.00	1,333.42

When subjected to random internal failures, the asynchronous microservice architecture demonstrated superior robustness in performance relative to the monolithic architecture under similar conditions. Specifically, the average request duration was reduced by approximately 58%, and the maximum latency experienced a significant decrease. This observation underscores that decoupling through asynchronous communication effectively mitigates the impact of downstream issues on the responsiveness of the user-facing service endpoint.

4.3 RQ3: Operational Challenges and Differences

While quantitative results show advantages for the asynchronous microservices architecture in resilience and robustness under failure, the development process revealed several operational challenges.

– **Increased Complexity:** The microservice implementation required more components (services, databases, Kafka cluster, service discovery) than the monolith. Asynchronous communication setup demanded specific expertise.

- **Resource Utilization:** Baseline load testing required careful resource allocation (1 vCPU, 512 MiB RAM per pod) for both architectures. While the microservice architecture handled higher throughput, its total resource footprint (3 services + Kafka) exceeded the monolith, suggesting higher operational costs.
- **Monitoring and Debugging:** Tracing requests across distributed services and Kafka proved more challenging than monolith debugging. Message processing issues required analysis across multiple logs, necessitating distributed tracing solutions.
- **Consistency Management:** Asynchronous communication requires careful design for eventual consistency across distributed databases, adding complexity compared to monolithic consistency.

These findings indicate that while asynchronous microservices provide resilience benefits, they require greater operational maturity and infrastructure investment than monoliths.

5 Discussion

This section discusses the interpretation of these findings in relation to the research questions and existing literature, highlighting both the advantages and the inherent trade-offs.

5.1 Resilience and Robustness Findings (RQ1 and RQ2)

The findings indicate a positive influence of the asynchronous, event-driven microservice architecture on system resilience (RQ1). The significantly reduced failure rate observed during random internal service failures (experiment ❸, Table 3) compared to the monolithic scenario (experiment ❷, Table 2) highlights the advantage of fault isolation [16]. Within the microservice architecture, the failure of a downstream service (e.g., *Shipping Service* or *Notification Service*) did not impede the acceptance of new orders by the user-facing *Ordering Service*. Asynchronous communication via Apache Kafka facilitated the buffering of events (*orderCreated*), allowing downstream services to process them upon recovery. This stands in stark contrast to the monolithic architecture, where any internal processing failure within the synchronous request chain resulted in the failure of the entire transaction. Even in the event of an entry-point failure (experiment ❷), the microservice architecture demonstrated a slight resilience advantage, potentially due to the faster restart/recovery time of the smaller, independent service instance compared to the larger monolith, although this effect was less pronounced in our specific setup.

Regarding robustness (RQ2), the asynchronous microservice architecture consistently outperformed the monolith under fault conditions in terms of request duration (Table 5 and 6). The decoupling achieved through asynchronous messaging prevented latency in downstream services from directly affecting the

response time of the initial order request. This is consistent with findings suggesting performance benefits of event-driven architectures, although some studies report increased baseline latency due to network overhead [5,6,19]. Our baseline tests (experiment ❶, Table 4) actually demonstrated lower average latency for the microservice architecture, likely because the initial HTTP response could be sent much earlier without waiting for the full synchronous chain required by the monolith. The ability of the microservice architecture to maintain superior performance and lower failure rates under stress confirms its enhanced robustness compared to the monolithic counterpart in our experiments.

5.2 Operational Challenges and Trade-Offs (RQ3)

Despite the demonstrated enhancements in resilience and robustness, the implementation of an asynchronous microservice architecture introduced substantial operational challenges (RQ3), which is consistent with findings in literature.

First, the inherent complexity of a distributed system increases significantly. The management of multiple services, databases, and the Kafka messaging infrastructure necessitated more sophisticated deployment orchestration (Kubernetes) and a deeper level of specialized knowledge compared to a single-unit monolith. This complexity extends to development, debugging, and maintenance [7]. Diagnosing issues requires correlating events across multiple service logs and potentially Kafka itself, underscoring the critical need for robust distributed tracing and observability tools in production environments, which represents an additional operational overhead.

Second, resource utilization and potential operational costs were elevated. While individual microservices may be small, the aggregate resource consumption (CPU, memory) of multiple service instances, along with the messaging broker infrastructure, typically surpasses that of an equivalent monolith, as indicated by comparisons in other studies [3] and implied by the need for adequate resource allocation in our setup.

Third, maintaining data consistency across distributed services communicating asynchronously necessitates careful design, often involving complex patterns, such as Sagas [21], whereas the monolith benefits from simpler transactional consistency within a single database. Although our simple application does not require complex consistency patterns, it remains a significant challenge in real-world scenarios.

5.3 Relation to Existing Work and Implications

Our findings empirically validate the theoretically proposed resilience advantages of asynchronous microservices. While prior studies, such as [5,11], primarily concentrated on scalability and performance under normal load conditions, our research specifically quantifies the resilience and robustness enhancements under simulated failure scenarios using Chaos Engineering. The results underscore the significance of asynchronous, event-driven patterns [6,19] in constructing fault-tolerant distributed systems.

However, the operational challenges observed reflect the "pains" frequently reported alongside the "gains" of microservices [12,23]. Consequently, the decision to adopt asynchronous microservices is complex, involving significant trade-offs between resilience and robustness improvements and increased operational complexity and cost. Organizations must possess the requisite operational maturity, tools, and expertise to manage these complexities effectively. For systems where absolute resilience is critical and operational capacity is available, the benefits likely justify the costs. Conversely, for simpler applications or organizations with limited operational resources, a well-structured monolith, potentially a modular monolith [18], might remain a more pragmatic choice [7,15].

6 Threats to Validity

We recognize the following potential threats to the validity of this study.

6.1 Construct Validity

Construct validity concerns whether the operationalizations and metrics accurately represent the theoretical constructs under study, specifically resilience and robustness. Standard metrics like HTTP failure rate, request duration, and processed transactions were used based on literature, but these may not fully capture all aspects of these complex quality attributes. The user-perceived system behavior during degradation and data consistency under failure were not directly measured. Additionally, the implemented system prototypes represent a simplified e-commerce workflow. This simplification, while necessary for controlled experimentation, may limit how well the observed behaviors represent resilience and robustness in complex, real-world enterprise systems.

6.2 Internal Validity

Internal validity addresses whether outcomes can be confidently attributed to experimental manipulations rather than confounding factors. Internal validity may be threatened by differing resource allocations, as the microservice architecture used three times the CPU/RAM limits of the monolith, potentially confounding performance comparisons. Additionally, inherent differences in failure semantics between the monolithic transactional model and event-driven buffering in the asynchronous microservice architecture could influence outcome metrics. Mitigation included using identical core logic and consistent deployment environments.

6.3 External Validity

External validity concerns the generalizability of findings beyond this study's context. Generalizability may be limited by the simplified e-commerce system

prototype, which might not fully represent the complexities of real-world enterprise applications. The experiments were conducted in a controlled local Kubernetes environment, and results might vary in production conditions. The comparison was restricted to a classic synchronous monolith, limiting applicability to other monolithic patterns such as modular monoliths. The experimental apps were custom-built prototypes, which restricts the findings' broad applicability.

7 Conclusion and Future Work

This study empirically examines the impact of asynchronous communication mechanisms on the resilience and robustness of microservice architectures in comparison to traditional monolithic systems. By employing equivalent system prototypes subjected to controlled fault injection through Chaos Engineering, we systematically assessed key quality attributes under failure conditions.

The results indicate that the asynchronous, event-driven microservice architecture demonstrates significantly enhanced resilience and robustness relative to the synchronous monolithic architecture. Notably, the microservice architecture exhibited lower failure rates, particularly in the context of random internal component failures, and maintained superior performance (request duration) during fault scenarios. This confirms that the decoupling and fault isolation provided by asynchronous communication effectively mitigate the impact of failures and prevent cascading issues common in tightly coupled systems.

However, the study also identified significant operational challenges and trade-offs associated with the asynchronous microservice architecture. Increased architectural complexity, higher aggregate resource consumption (approximately 4x the resources of the monolith when including messaging infrastructure), and the necessity for sophisticated monitoring and debugging capabilities represent critical considerations. Therefore, while asynchronous communication demonstrably enhances fault tolerance, the decision to adopt this pattern must balance these benefits against the required operational maturity and potential costs. The answer to the central research question is thus nuanced: asynchronous communication significantly enhances microservice resilience and robustness compared to synchronous monoliths, but introduces substantial operational complexities that must be managed.

7.1 Future Work

Future work should validate these findings using more complex, realistic system prototypes and explore comparisons with alternative architectures, such as asynchronous modular monoliths. Further research could also investigate the nuanced impact of resource allocation strategies on resilience trade-offs and explore lightweight observability frameworks to address the monitoring challenges inherent in distributed, event-driven systems. Finally, expanding the Chaos Engineering experiments to include a broader range of fault types would provide a more comprehensive resilience assessment.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Al-Said Ahmad, A., Al-Qoran, L.F., Zayed, A.: Exploring the impact of chaos engineering with various user loads on cloud native applications: an exploratory empirical study. Computing **106**(7), 2389–2425 (2024). https://doi.org/10.1007/s00607-024-01292-z
2. Basiri, A., et al.: Chaos engineering. IEEE Softw. **33**(3), 35–41 (2016). https://doi.org/10.1109/MS.2016.60
3. Berry, V., Castelltort, A., Lange, B., Teriihoania, J., Tibermacine, C., Trubiani, C.: Is it Worth migrating a monolith to microservices? An experience report on performance, Availability and energy usage. In: 2024 IEEE International Conference on Web Services (ICWS), pp. 944–954 (2024). iSSN 2836-3868. https://doi.org/10.1109/ICWS62655.2024.00112
4. Bjørndal, N., Jonatã Pires de Araújo, L., Bucchiarone, A., Dragoni, N., Mazzara, M., Dustdar, S.: Benchmarks and performance metrics for assessing the migration to microservice-based architectures. J. Object Technol. **20**(2), 2:1–2:17 (2021). https://doi.org/10.5381/jot.2021.20.2.a3
5. Blinowski, G., Ojdowska, A., Przybyłek, A.: Monolithic vs. microservice architecture: a performance and scalability evaluation. IEEE Access **10**, 20357–20374 (2022). https://doi.org/10.1109/ACCESS.2022.3152803
6. Cabane, H., Farias, K.: On the impact of event-driven architecture on performance: an exploratory study. Futur. Gener. Comput. Syst. **153**, 52–69 (2024). https://doi.org/10.1016/j.future.2023.10.021
7. Christian, J., Steven, Kurniawan, A., Anggreainy, M.S.: Analyzing microservices and monolithic systems: key factors in architecture, development, and operations. In: 2023 6th International Conference of Computer and Informatics Engineering (IC2IE), pp. 64–69 (2023). https://doi.org/10.1109/IC2IE60547.2023.10331155
8. Di Francesco, P., Lago, P., Malavolta, I.: Architecting with microservices: a systematic mapping study. J. Syst. Softw. **150**, 77–97 (2019). https://doi.org/10.1016/j.jss.2019.01.001
9. Fehling, C., Leymann, F., Retter, R., Schupeck, W., Arbitter, P.: Cloud computing patterns: fundamentals to design, build, and manage cloud applications. Springer, Vienna (2014). https://doi.org/10.1007/978-3-7091-1568-8
10. Fowler, M.: Microservices: a definition of this new architectural term (2014). https://martinfowler.com/articles/microservices.html. Accessed 26 Jun 2025
11. Horváth, M., Sakhnenko, V., Gurbáľ, F.: Comparison of scalability and performance in Microservices and Monolithic Architectures. In: 2024 IEEE 17th International Scientific Conference on Informatics (Informatics), pp. 82–87 (2024). https://doi.org/10.1109/Informatics62280.2024.10900892
12. Jamshidi, P., Pahl, C., Mendonça, N.C., Lewis, J., Tilkov, S.: Microservices: the journey so far and challenges ahead. IEEE Softw. **35**(3), 24–35 (2018). https://doi.org/10.1109/MS.2018.2141039
13. Li, S., et al.: Understanding and addressing quality attributes of microservices architecture: a systematic literature review. Inf. Softw. Technol. **131**, 106449 (2021). https://doi.org/10.1016/j.infsof.2020.106449

14. Meijer, W., Trubiani, C., Aleti, A.: Experimental evaluation of architectural software performance design patterns in microservices. J. Syst. Softw. **218**, 112183 (2024). https://doi.org/10.1016/j.jss.2024.112183
15. Mendonça, N.C., Box, C., Manolache, C., Ryan, L.: The monolith strikes back: why Istio migrated from microservices to a monolithic architecture. IEEE Softw. **38**(5), 17–22 (2021). https://doi.org/10.1109/MS.2021.3080335
16. Newman, S.: Building Microservices: Designing Fine-Grained Systems, 1st edn. O'Reilly Media, Sebastopol, CA (2015)
17. Pinciroli, R., Aleti, A., Trubiani, C.: Performance modeling and analysis of design patterns for microservice systems. In: 2023 IEEE 20th International Conference on Software Architecture (ICSA), pp. 35–46, March 2023. https://doi.org/10.1109/ICSA56044.2023.00012
18. Prakash, C., Arora, S.: Systematic analysis of factors influencing modulith architecture adoption over microservices. In: 2024 TRON Symposium (TRONSHOW), pp. 1–8, December 2024
19. Rahmatulloh, A., Nugraha, F., Gunawan, R., Darmawan, I.: Event-driven architecture to improve performance and scalability in microservices-based systems. In: 2022 International Conference Advancement in Data Science, E-learning and Information Systems (ICADEIS), pp. 01–06 (2022). https://doi.org/10.1109/ICADEIS56544.2022.10037390
20. Razzaq, A., Ghayyur, S.A.K.: A systematic mapping study: the new age of software architecture from monolithic to microservice architecture–awareness and challenges. Comput. Appl. Eng. Educ. **31**(2), 421–451 (2023). https://doi.org/10.1002/cae.22586
21. Richardson, C.: Microservices Patterns: With examples in Java, 1st edn. Manning, Shelter Island, New York (2018)
22. Rodrigues, H., Rito Silva, A., Avritzer, A.: Performance comparison of monolith and microservice architectures. In: Tekinerdoğan, B., Spalazzese, R., Sözer, H., Bonfanti, S., Weyns, D. (eds.) Software Architecture, ECSA 2023 Tracks, Workshops, and Doctoral Symposium, pp. 185–199. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-66326-0_12
23. Soldani, J., Tamburri, D.A., Van Den Heuvel, W.J.: The pains and gains of microservices: a systematic grey literature review. J. Syst. Softw. **146**, 215–232 (2018). https://doi.org/10.1016/j.jss.2018.09.082
24. Srijith, Karan Bantia, R., Govardhan, N., Anala, M.R.: Inter-service communication among microservices using Kafka connect. In: 2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS), pp. 43–47 (2022). iSSN 2327-0594. https://doi.org/10.1109/ICSESS54813.2022.9930270
25. Thönes, J.: Microservices. IEEE Softw. **32**(1), 116–116 (2015). https://doi.org/10.1109/MS.2015.11
26. Villamizar, M., et al.: Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud. In: 2015 10th Computing Colombian Conference (10CCC), pp. 583–590, September 2015. https://doi.org/10.1109/ColumbianCC.2015.7333476

Exploring Micro Frontends: A Case Study Application in E-Commerce

Ricardo Hideki Hangai Kojo[1], Luiz Fernando Corte Real[1], Renato Cordeiro Ferreira[1,2,3,4(✉)], Thatiane de Oliveira Rosa[1,5], and Alfredo Goldman[1(✉)]

[1] Institute of Mathematics and Statistics, University of São Paulo (USP), São Paulo, Brazil
{renatocf,gold}@ime.usp.br
[2] Jheronimus Academy of Data Science (JADS), 's-Hertogenbosch, The Netherlands
[3] Technical University of Eindhoven (TUe), Eindhoven, The Netherlands
[4] Tilburg University (TiU), Tilburg, The Netherlands
[5] Federal Institute of Tocantins (IFTO), Palmas, Brazil
thatiane@ifto.edu.br

Abstract. In the micro frontends architectural style, the frontend is divided into smaller components, which can range from a simple button to an entire page. The goal is to improve scalability, resilience, and team independence, albeit at the cost of increased complexity and infrastructure demands. This paper seeks to provide insights into when the adoption of micro frontends may be worthwhile, particularly in an industry context, considering that research in this area is still evolving. To achieve this, we conducted an investigation into the state of the art of micro frontends, based on both academic and gray literature. We then implemented this architectural style in a marketplace for handcrafted products, which already used microservices. Finally, we evaluated the implementation through a semi-open questionnaire with the developers. At the studied marketplace company, the need for architectural change arose due to the tight coupling between their main system (a Java monolith) and a dedicated frontend system. Additionally, there were deprecated technologies and poor developer experience. To address these issues, the micro frontends architecture was adopted, along with the API Gateway and Backend for Frontend patterns, and technologies such as Svelte and Fastify. Although the adoption of Micro Frontends was successful, it was not strictly necessary to meet the company's needs. According to the analysis of the mixed questionnaire responses, other alternatives, such as a monolithic frontend, could have achieved comparable results. What made adopting micro frontends the most convenient choice in the company's context was the monolith strangulation and microservices adoption, which facilitated implementation through infrastructure reuse and knowledge sharing between teams.

Keywords: Micro Frontends · Software Architecture · Case Study · Experimental Software Engineering

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 187–201, 2026.
https://doi.org/10.1007/978-3-032-04403-7_17

1 Introduction

The Microservices architecture, which emerged around 2012 [8], has grown in popularity in both academia and industry, being adopted by companies such as Amazon, Netflix, and Uber [14]. This architecture consists of breaking down an application into small, independent services that work together. The loose coupling between these services brings benefits such as maintainability, scalability, and resilience, key characteristics of large systems that need to be reliable, always available, and minimally prone to failure. However, this approach focuses solely on the backend of the application, leaving the frontend aside [11].

In this context, the micro frontend architecture has emerged, aiming to apply microservices concepts to the presentation layer of applications [11]. In this approach, the frontend system is divided into small parts, which may range from a simple button to a complete page. These parts are composed either on the client side (e.g., in the user's browser), the edge side (e.g., a CDN or Content Delivery Network), or the server side (e.g., a cloud server), resulting in a unified and cohesive output [10].

There is significant overlap between the benefits and challenges of microservices and micro frontends. Therefore, it is reasonable to consider adopting both architectures, leveraging existing knowledge and infrastructure. However, the concept of micro frontends is relatively recent (introduced in 2016 [5]), and as an emerging architectural style, the number of academic studies on the topic is still limited, indicating that research in this area is ongoing and evolving. Unlike microservices, which benefit from well-established technologies and design patterns, micro frontends still have limited tool support. Consequently, many opportunities remain to be explored.

Thus, this study provides an empirical account of the application of the micro frontend architectural style in an industry context, with the aim of helping understand under which circumstances its adoption may be worthwhile.

From this, the following research questions were defined:

- ***RQ1:*** *What are the motivations and challenges involved in adopting a micro frontend architecture in the studied company, which already uses microservices?*
- ***RQ2:*** *What are the perceived benefits and drawbacks reported by developers involved in the migration from a monolithic architecture to micro frontends?*

To support the proposed objective and answer the research questions, an initial exploratory investigation was conducted to identify relevant publications and practical insights on the architecture. This involved consulting both academic sources (such as articles and books) and gray literature, including blog posts and recorded talks. The goal was not to perform a systematic review, but to gather a representative sample of materials that could inform the implementation and guide the study. This analysis helped identify recurring concepts, key authors, and gaps in the existing knowledge.

In addition to the literature review, the micro frontends architecture was implemented at Company A (a fictitious name), a marketplace platform that

already used microservices. This objective also included documenting and evaluating the implementation: understanding the reasons behind the architectural change, outlining the development history, including the new systems, and explaining the rationale for the decisions made. Finally, the impacts of adopting micro frontends were analyzed via a questionnaire applied to team members.

2 Background

This section aims to present the essential concepts necessary for a comprehensive understanding of this study, assuming a basic knowledge of software architecture and architectural styles, especially micro frontends.

2.1 Software Architecture and Architectural Styles

Software architecture can be defined as "*the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both*" [2]. It encompasses the foundations, properties, rules, and constraints that guide system design.

The purpose of defining a software architecture is to facilitate aspects such as development, deployment, operation, and maintenance [9], in addition to supporting the analysis of qualitative attributes such as scalability, resilience, security, and maintainability. Architectural decisions are therefore essential to the system's evolution. To identify the most suitable architecture for addressing specific quality attributes, various architectural styles and patterns are adopted [13].

An architectural style prescribes a constrained set of elements and relationships that can be used to define a system's structure [13]. A system that conforms to such constraints can be considered an instance of that architectural style. It is important to note that a single application may incorporate multiple styles.

When choosing a specific architectural style, it is crucial to consider the motivations behind the decision, its expected benefits, and potential challenges. Architectural decisions tend to have significant implications, and as Brooks famously stated [3], "*there are no silver bullets in software development*"; no single architectural style offers universal solutions or only advantages.

This study explores the monolithic, microservices-based, and micro frontend architectural styles, which are relevant for understanding the proposed research.

2.2 Monolithic vs. Microservices-Based Architectural Styles

In a monolithic architecture, the entire application is structured in a single layer containing all necessary logic for full system functionality, typically resulting in a single executable artifact. This architecture is simple, making it a common choice for new projects. However, as the team and codebase grow, technical and organizational issues may emerge, such as maintenance difficulties, increased bug count, slow feature development, and scalability limitations. In such contexts, a microservices architecture may serve as a viable alternative [16].

According to Lewis and Fowler [8], "*the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.*"

In the literature, microservices are often described as an alternative to monolithic systems that face organizational and technical limitations. The transition from monolith to microservices can follow the Strangler Fig pattern [4], in which existing functionalities are extracted into independent services until the monolith becomes obsolete and is replaced entirely. This process is gradual, as a full rewrite of the monolith is usually infeasible due to its size and complexity.

2.3 Micro Frontend Architecture

Micro frontends are "*an architectural style where independently deliverable frontend applications are composed into a greater whole*" [7]. The application is broken down into smaller and simpler parts that can be developed, tested, and deployed independently, while preserving a unified user experience.

According to Peltonen *et al.* [11], "*micro-frontends extends the microservice architecture idea and many principles from microservices apply to micro frontends*". Therefore, it is common to implement both architectures in parallel. Since backend and frontend are distinct domains, existing distributed system knowledge and infrastructure can be leveraged without conflict between services. For these reasons, micro frontends are considered a natural next step after the adoption of microservices [19].

Peltonen *et al.* [11] and Geers [5] identify several motivations for adopting a micro frontend architecture. These include the increasing complexity of frontend systems, the growth of the codebase, and organizational challenges. Scalability is also a recurring concern, particularly regarding the need for independent deployments and teams, faster feature delivery, innovation enablement, and reduced onboarding time for new developers. In some cases, adoption is also influenced by market trends, with companies embracing microservices and micro frontends to align with industry best practices.

Furthermore, micro frontends offer benefits similar to those of microservices. These include technological flexibility, allowing each part of the system to use the most appropriate tools, and the ability to form cross-functional teams focused on specific business domains. The architecture promotes team and project autonomy, facilitating development, testing, and continuous delivery. It also supports scalability both at the application and organizational levels. Another relevant aspect is resilience: failures in a micro frontend affect only that service, avoiding the full-system outages common in monolithic architectures.

Firstly, the most significant challenges of adopting micro frontends was the complexity introduced by the distributed nature of the architecture, which involves multiple projects and diverse technologies. Secondly, ensuring visual

consistency across services, as the final product must appear cohesive and uniform to the user. Thirdly, the governance, especially when different teams share responsibility for the same service or operate within the same business domain. Finally, the performance concerns, due to potential code duplication, shared dependencies, and the volume of data transferred during interface composition [5,11].

Before implementing micro frontends, it is necessary to define which approach will be adopted. There are two main approaches: the horizontal approach, in which each micro frontend represents a portion of the page, and the vertical approach, in which each micro frontend corresponds to a business context or domain. In the latter, a single micro frontend may contain parts of a page, an entire page, or even multiple pages [10].

Another important aspect to consider when implementing a micro frontend architecture is the adoption of architectural patterns. In this context, the most relevant patterns are API Gateway and Backend for Frontend (BFF).

The API Gateway is a service that acts as the entry point for other services. All requests made by clients – such as a mobile app or a desktop website – go through the gateway, which, in addition to routing them to the appropriate services, can also handle authorization checks and protocol translation. This pattern is particularly useful in microservices-based architectures, where multiple clients access various services. Communication with each service can occur through different protocols, such as HTTP and gRPC [15].

The Backend for Frontend (BFF) pattern is a variation of the API Gateway pattern [15], in which each client type has its own dedicated gateway. This allows for the isolation of specific client needs and enables a team to take responsibility for both the client application and its corresponding BFF.

3 Architecture Overview of Company A

Company A (a fictitious name used for confidentiality) is an online handicraft marketplace founded in 2008. By 2009, it had reached 100,000 products sold and 10,000 active sellers. As of 2022, the platform hosts over 7 million products from more than 100,000 active sellers. Operating in the e-commerce sector, **Company A** connects independent artisans with customers, enabling the sale of handmade and unique products through its digital marketplace.

The frontend plays a critical role in **Company A**'s success, as it must deliver a performant, user-friendly, and accessible experience across desktop and mobile devices, directly impacting customer satisfaction and retention. Prior to adopting micro frontends, **Company A** employed the Strangler Fig pattern to incrementally migrate its legacy monolithic system, referred to here as `Marketplace`, to a microservices-based backend architecture. Despite backend progress, the frontend remained centralized within the monolith. Additionally, a separate project, `Aquarelle`, was developed to support specific features such as a chat interface between buyers and sellers.

Launched in 2012, `Marketplace` handled both backend and frontend responsibilities, serving as the single entry point for all platform functionalities and representing a potential single point of failure. Developed with Java, JSP, Sass, and internal JavaScript libraries, the monolithic design enabled rapid early development but eventually became a bottleneck due to growing complexity, slow deployment cycles, and architectural erosion. Core functionalities like product search, shipping calculations, and category management were progressively extracted into independent microservices following the Strangler Fig pattern.

The `Aquarelle` project, built on Node.js, was introduced to implement a reactive chat feature, capable of displaying dynamic backend data such as order status and user actions. Since modern frontend frameworks were not yet widely adopted in early 2016, the team developed internal libraries to support this functionality. However, `Aquarelle` faced several technical and architectural challenges: key internal libraries were deprecated shortly after launch, maintenance suffered from poor documentation, and developer interest waned due to outdated technologies. Furthermore, limited expertise in Node.js and insufficient time to decouple features from the monolith resulted in tight coupling between `Aquarelle` and `Marketplace`.

`Marketplace` also remained responsible for critical cross-cutting concerns such as authentication, data orchestration, and formatting, requiring frontend developers to interact directly with backend code. This rigid architecture, combined with deprecated technologies and organizational constraints, ultimately motivated the adoption of a micro frontend architecture aimed at improving modularity, team autonomy, and deployment agility.

4 Implementing Micro Frontends at Company A

Due to the issues discussed in the previous section, a plan was devised for the implementation of a new frontend architecture based on the adoption of modern technologies. The chosen architecture was micro frontends, which facilitates the isolation of responsibilities, allows interfaces to be divided into smaller parts, and enables the use of different technologies. To support this, it was necessary to create a service that would act as an API Gateway, so that the monolith would no longer serve as the central entry point and would be accessed only when needed. Based on this plan, the projects `Fortelle`, `Veloura`, `Nuvelle`, and `Brindle` (fictitious names) were created, each playing a strategic role in the construction of the new architecture.

`Fortelle` is the implementation of the API Gateway pattern, serving as the entry point for **Company A**'s systems. All user requests pass through it, where routing, validations, and security functions for other services are handled. This implementation was essential to enable the micro frontends to start taking over the responsibility of serving user interfaces, previously centralized in the monolith. In parallel, the team began evaluating frameworks and libraries to ensure good performance and a positive developer experience.

`Veloura` is **Company A**'s design system, consisting of a set of standards for building user interfaces – including colors, spacing, fonts, and icons – which

reinforce the company's visual identity [6]. The `Nuvelle` project was conceived as a proof of concept for the selected frontend technologies. Its development allowed the team to validate practices such as file organization, supporting libraries, and code conventions. From `Nuvelle`, the first `Brindle` was created, with the goal of migrating the search page for a specific set of craft materials from the monolith to the new architecture. This page was selected because it was visually similar to the main search page, but had a lower volume of traffic.

After deploying the first `Brindle` to production, new clones were developed to meet internal demands. All systems derived from `Nuvelle` came to be known as `Brindle` projects, with additional names used to differentiate them, such as `Brindle Discovery`, for example. Currently, all of **Company A**'s micro frontends are `Brindle` projects. Figure 1 illustrates the basic structure of a micro frontend: user requests pass through the API Gateway and are redirected to a `Brindle` instance, where the Backend for Frontend (BFF) pattern handles internal routing, orchestrates data from microservices, and forwards it to a template rendered by an open-source library developed by **Company A**.

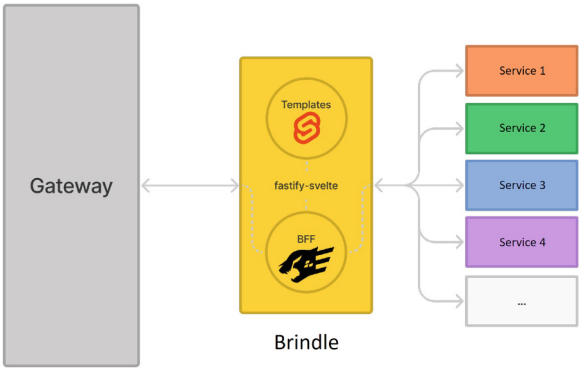

Fig. 1. Architectural schematic of a `Brindle` project.

With the creation of three `Brindle` projects, the frontend teams were able to identify common modules across services. These shared components were extracted into a new library, `Brindle Core`, an open-source framework for building `Brindle`s. One of the main advantages of `Brindle Core` is its ability to abstract internal mechanisms that rarely change, reducing code duplication, the number of files, and the need for repeated testing in common components.

5 Current Architecture of Company A

With the conception of the mentioned projects, the company's frontend architecture underwent significant changes. The introduction of the API Gateway pattern – as illustrated by Fig. 2 – enabled other services, besides the monolith,

to handle part of the user request processing, decentralizing responsibilities and enabling the adoption of micro frontends. As a result, creating new `Brindle` projects became straightforward, requiring only minor additions to the API Gateway code.

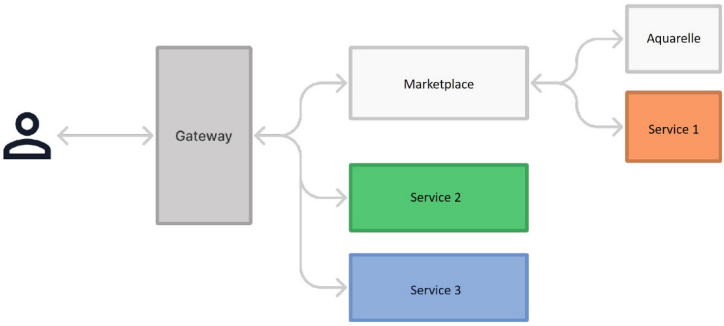

Fig. 2. Request flow to frontend after implementing micro frontends.

The architectural transition at **Company A** proved successful. The adoption of vertical micro frontends allowed the migration of pages from the monolith to the `Brindle` projects, which utilize modern technologies and provide an improved development experience. Moreover, the new structure facilitated the creation of frontend systems for the company's new products. It is worth noting that the ongoing transformation towards microservices favored this adoption by enabling the reuse of technologies and infrastructure already in place, since micro frontends also share microservices' principles.

During this research, it was observed that the natural evolution of the architecture involves migrating more pages from the monolith to the new services, as well as creating additional micro frontends. However, there are no plans to expand the number of services, as **Company A** has defined a domain division based on its products. Therefore, until new products are conceived, the teams remain focused on improving existing services.

Nonetheless, a bottleneck exists: to migrate more pages to the `Brindle` projects, functionalities from the monolith must be converted into microservices. Since the backend teams currently lack the availability to develop these services, obtaining the necessary data for the interfaces is compromised.

To mitigate this issue, backend, and frontend teams must establish contracts, allowing simulation of calls to future services and advancing development. Despite adopting microservices and micro frontends, **Company A** is still evaluating the implementation of multidomain teams, considering organizational impacts and questioning whether this practice is always beneficial, as discussed in the literature.

6 Survey with Developers

To better understand the impacts of the architectural change, a semi-open questionnaire was conducted on **Company A**'s employees with knowledge or involvement in frontend projects. This initiative aimed to identify perceived problems, expectations about the change, and observed impacts, particularly regarding the adoption of micro frontends.

The target audience consisted of all active employees during the research period who were directly or indirectly involved in frontend development or decisions. Based on internal information from the engineering team, this included frontend software developers, technical leads, engineering managers, and architecture team members. Invitations were formally sent via institutional email, and the questionnaire was made available through Google Forms for approximately one month, with at least one reminder issued. All eight invited employees responded, yielding a 100% participation rate. Informed consent was obtained, and responses were anonymized for academic analysis. Among the respondents, only one was a woman. Most participants (62.5%) had over ten years of experience in the technology field, and half worked specifically as frontend developers.

The questionnaire[1] consisted of 19 questions, 15 open-ended and 4 multiple-choice, designed to distinguish, as much as possible, the impacts of adopting new technologies from those stemming from architectural changes. A Likert scale was used for multiple-choice questions. The questions were organized into five sections: (1) respondent profile (professional experience and involvement in frontend activities), (2) architecture prior to micro frontends, (3) adoption of new technologies, (4) architectural change, and (5) understanding of the new architecture.

Participants were asked to evaluate both technological and architectural impacts across seven key aspects: hiring and onboarding, deployment, development across teams, project understanding, implementation of new features, testability, and development speed. These aspects were selected due to their relevance to the company and frequent recurrence in related literature.

Regarding involvement in frontend activities, whether in development or architectural decisions, respondents rated themselves between medium and high levels. This result was expected, considering that developers are directly involved in implementation while managers typically contribute at a more strategic level.

When asked about the impacts of the new architecture, Fig. 3 shows the emergence of responses such as "*I don't know*" and the lack of responses like "*No impact*", indicating a lack of clarity regarding the architectural effects. Among the predefined aspects, testability and hiring and onboarding processes were reported as the most negatively affected. In terms of testability, although tests were no longer coupled to the monolith, end-to-end testing in distributed systems, such as those based on microservices and micro frontends, proved to be inherently more complex. Regarding hiring and onboarding, the architectural fragmentation enabled a gradual learning path but made it more difficult to understand the system as a whole.

[1] The link is available at: https://doi.org/10.5281/zenodo.16915366.

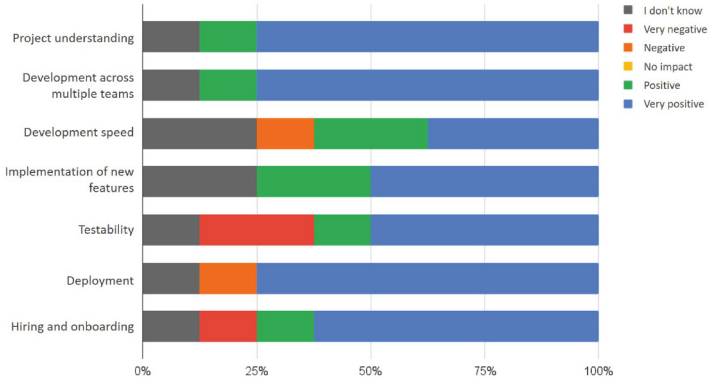

Fig. 3. Impacts brought about by new architecture.

Open-ended responses were individually analyzed, and relevant excerpts were coded. These codes were then grouped into three overarching themes: old architecture, new architecture, and micro frontends. Each theme was subdivided to better organize the emerging subtopics.

Regarding the old architecture (Fig. 4), the analysis focused on identifying its strengths and weaknesses, as well as the expectations for improvement. For the new architecture (Fig. 5), the goal was to understand perceived advantages and disadvantages, especially those affecting participants' daily work. Finally, participants were asked to describe their understanding of micro frontends and to evaluate whether the new architecture could be considered a legitimate implementation of that concept.

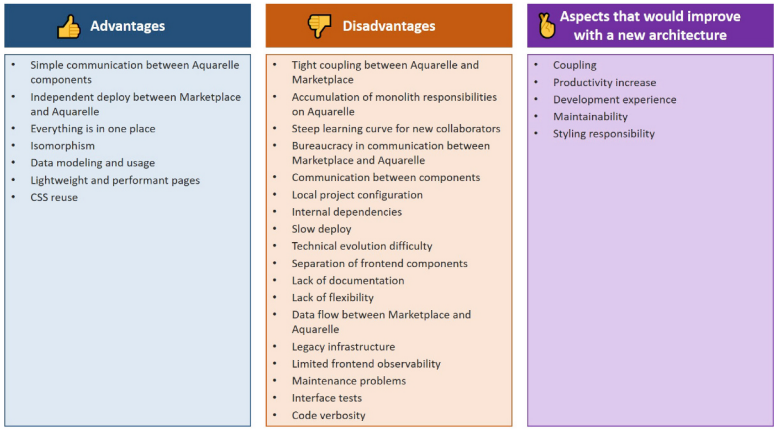

Fig. 4. Advantages and disadvantages of old architecture, according to respondents.

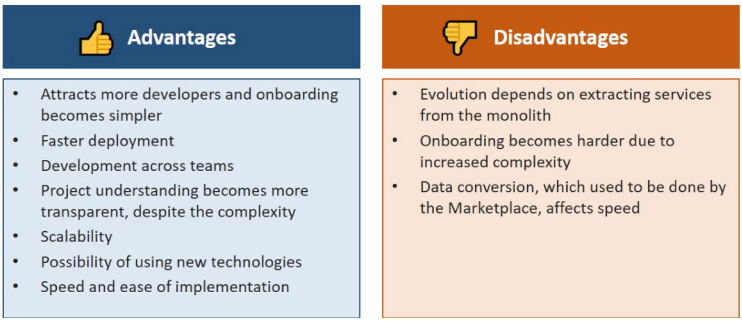

Fig. 5. Advantages and disadvantages of new architecture, according to respondents.

Several disadvantages were mentioned. However, some points raised may be more closely related to the technologies used than to the architectural approach itself, as some confusion between technological and architectural impacts was anticipated. Most advantages reported were associated with `Aquarelle`, which, despite meeting its objectives, still suffers from technological and architectural limitations. The most frequently cited disadvantage was the strong coupling between frontend systems, precisely the key issue the architectural change aimed to resolve.

Overall, there were more mentions of advantages than disadvantages regarding the new architecture. Some aspects were viewed ambivalently: the use of an emerging architecture helps attract developers, and decomposing the system into smaller services can ease onboarding. Nonetheless, the complexity of distributed systems tends to prolong the onboarding process. Furthermore, although implementation speed increased, it was accompanied by codebase expansion.

A noteworthy finding is that half of the participants did not recognize the new architecture as a micro frontend implementation (Fig. 6). This perception may stem from two main factors: the dominant definitions in the literature and the absence of explicit references to the vertical slicing approach. Many definitions suggest that a page must be split into multiple visual components to qualify as a micro frontend, which is not necessarily true. As with microservices, there is no strict definition of what constitutes a *service*. Thus, micro frontends may exist even without direct visual interface decomposition.

Finally, the analysis indicated that, despite the successful implementation of micro frontends, this was not necessarily the only viable architectural solution for **Company A**'s context. A monolithic frontend, for instance, could have been feasible, provided that decoupling and modern technologies were adequately applied. One finding that supports this argument is that only one participant mentioned scalability, which is commonly cited as one of the main benefits of microservice-based architectures, and by extension, of micro frontends.

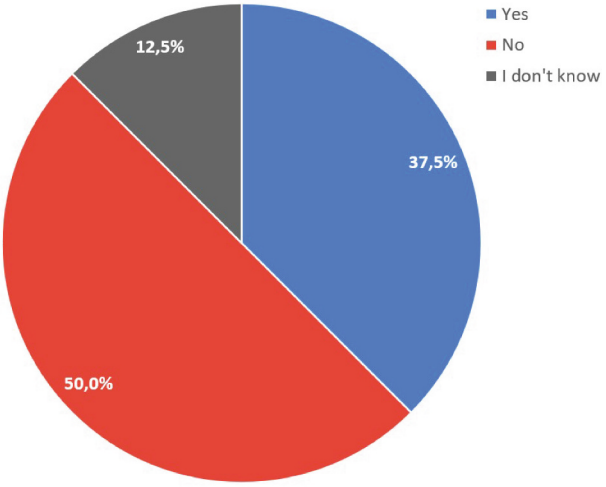

Fig. 6. Respondents' perception of whether the new architecture qualifies as a micro frontend.

7 Related Work

Some related work to this study has been identified in the literature, investigating relevant aspects of micro frontend architecture adoption and complementing the findings of the present research.

The work by Taibi and Mezzalira [18] presented the fundamental principles, implementation approaches, and common pitfalls associated with the use of micro frontends. The methodology combined a literature review with practical experience in a sports streaming company. The authors analyzed composition models such as iFrames, Web Components, and Module Federation, as well as communication strategies between micro-applications. The results indicate that, although the architecture fosters team autonomy and scalability in large-scale projects, it also introduces challenges such as increased complexity, the need for robust governance, and significant investments in automation and monitoring. These findings corroborate the technical and organizational challenges identified in this study, especially concerning architectural complexity and governance.

Amorim and Canedo [1] conducted a systematic mapping study investigating the impacts, challenges, and architectural patterns across 24 scientific studies on micro frontends. Among the most recurrent patterns are Single-SPA, Web Components, Module Federation, and Application Shell. Reported benefits include scalability, flexibility, and efficiency in continuous delivery, while drawbacks involve operational complexity, code redundancy, and difficulties in standardizing user experience. Our study reinforces these results, especially concerning developers' perceptions of system complexity and challenges in main-

taining interface consistency, reflected in onboarding difficulties and conceptual ambiguities.

Sutharsica et al. [17] explored the adoption of micro frontend architecture in startups through a mixed-methods approach combining a systematic literature review, quantitative survey, and qualitative interviews. Their findings reveal that despite common startup constraints such as limited resources, perceived advantages include scalability, modularity, and code reusability. Nevertheless, persistent challenges relate to integration, UI/UX inconsistency, and performance overhead. The emphasis on incremental planning and technical upskilling highlights aspects also observed in our industrial context, where migration required careful alignment among architecture, infrastructure, and teams.

Finally, Peltonen et al. [12] performed a multivocal literature review integrating academic sources and gray literature, focusing on motivations, benefits, and challenges of micro frontend adoption. The results show that adoption is primarily driven by the need to address growing complexity in monolithic frontends and the pursuit of team autonomy and scalability. Key benefits include technological independence and faster feature delivery. On the other hand, challenges such as increased system complexity, user experience inconsistency, code duplication, and governance issues were also identified. These conclusions align with the perceptions reported by developers in our study, who highlighted both advantages related to flexibility and modernization and difficulties concerning complexity, onboarding, and conceptual clarity of the architecture.

Overall, when considering the related work, our study contributes by providing an empirical analysis of micro frontend implementation in a specific industrial setting, emphasizing motivations, challenges, and developer perceptions, and reinforcing the importance of a contextualized understanding for successful adoption of the micro frontend architecture.

8 Conclusion

This paper provides an empirical account of the application of the micro frontend architectural style in an industry context, with the aim of helping understand under which circumstances its adoption may be worthwhile. Initially, a state-of-the-art study on the architecture was conducted through both academic literature and gray literature sources, such as blog posts and conference talks. Next, micro frontends were implemented at **Company A**, and finally, the adoption was evaluated through surveys with its developers.

The study was guided by two research questions: ***RQ1****: what are the motivations and challenges involved in adopting micro frontend architecture in a company that already uses microservices?* and ***RQ2****: what are the perceived benefits and drawbacks reported by developers involved in the migration?*

The academic literature on micro frontends is still evolving, especially regarding the concept of vertical, or domain-based, micro frontends as described by Mezzalira [10], whose simplicity facilitates implementation. Furthermore, existing definitions lack flexibility and generally tie the architecture to the decomposition of a page into components, which caused confusion within the company.

Although a vertical approach was adopted, developers hesitated to classify it as a micro frontend architecture due to the absence of composition, a central focus in works such as by Geers [5]. Therefore, there is a need to align the definition of the architecture more closely with that of its predecessor, microservices, and to document alternative forms such as micro frontends without user interfaces.

At **Company A**, all prerequisites were met, and the adoption was considered successful. The API Gateway pattern enabled a distribution of frontend responsibilities, while the Backend for Frontend (BFF) pattern supported communication with backend services. Still, adopting micro frontends was not strictly necessary, as other architectures, such as a monolithic frontend, could have achieved comparable results. What made the transition more feasible was the company's ongoing migration to microservices, which allowed teams to reuse infrastructure and share existing knowledge.

Regarding ***RQ1***, the main motivations for adopting micro frontends included the need to modernize the frontend stack, overcome the limitations of the monolithic architecture, and align with the microservices-based backend already in place. Key challenges involved technical complexity, the coexistence with legacy systems, and conceptual misunderstandings about the architecture, particularly due to restrictive definitions found in the literature.

In relation to ***RQ2***, developers reported both benefits and drawbacks. Among the advantages were faster implementation, alignment with modern practices, and greater flexibility. On the other hand, difficulties included increased system complexity, onboarding challenges due to service fragmentation, and ambiguities in defining whether the solution qualifies as a micro frontend architecture.

Thus, while not the only possible solution, micro frontends turned out to be the most convenient within that specific context. It is believed that this article may support researchers and practitioners alike by combining theory and practice, offering a historical account of implementation in an e-commerce setting, and providing a critical perspective on the literature. Additionally, this study contributed to architectural discussions at **Company A** and serves as documentation for current and future developers.

References

1. Cunha de Amorim, G., Canedo, E.D.: Micro-frontend architecture in software development: a systematic mapping study. In: Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS 2025), vol. 2, pp. 105–116. SCITEPRESS (2025). https://doi.org/10.5220/0013195800003929
2. Bass, L., Clements, P., Kazman, R.: Software Architecture in Practice, 3rd edn. Addison Wesley (2013)
3. Brooks, F.: The Mythical Man-Month: Essays on Software Engineering. Addison Wesley (1995)
4. Fowler, M.: Stranglerfigapplication (2004). https://martinfowler.com/bliki/StranglerFigApplication.html. Accessed 14 Aug 2021

5. Geers, M.: Micro Frontends in Action. Manning, 1st edn. (2020)
6. Hacq, A.: Everything you need to know about design systems (2018). https://uxdesign.cc/everything-you-need-to-know-about-design-systems-54b109851969. Accessed 3 Dec 2021
7. Jackson, C.: Micro frontends, June 2019. https://martinfowler.com/articles/micro-frontends.html. Accessed 21 Jun 2021
8. Lewis, J., Fowler, M.: Microservices, March 2014. https://www.martinfowler.com/articles/microservices.html. 21 Jun 2021
9. Martin, R.C.: Arquitetura Limpa: O Guia do Artesão para Estrutura e Design de Software. Alta Books Editora (2019)
10. Mezzalira, L.: Building Micro-Frontends. O'Reilley Media, Inc. (2021)
11. Peltonen, S., Mezzalira, L., Taibi, D.: Motivations, benefits, and issues for adopting micro-frontends: a multivocal literature review. Inf. Softw. Technol. **136**, 106571 (2021). https://doi.org/10.1016/j.infsof.2021.106571. https://www.sciencedirect.com/science/article/pii/S0950584921000549
12. Peltonen, S., Mezzalira, L., Taibi, D.: Motivations, benefits, and issues for adopting micro-frontends: a multivocal literature review. Inf. Softw. Technol. **136**, 106571 (2021). https://doi.org/10.1016/j.infsof.2021.106571
13. Richards, M.: Software Architecture Patterns, 1st edn. O'Reilley Media, Inc. (2015)
14. Richardson, C.: Microservices Patterns, 1st edn. Manning (2018)
15. Richardson, C.: Pattern: API gateway/backends for frontends (2018). https://microservices.io/patterns/apigateway.html. Accessed 27 Oct 2021
16. Richardson, C.: Pattern: monolithic architecture (2019). https://microservices.io/patterns/monolithic.html. Accessed 23 Dec 2021
17. Sutharsica, A., Arambepola, N.: Micro-frontend architecture: a comparative study of startups and large established companies–suitability, benefits, challenges, and practical insights. In: Proceedings of the 2025 International Conference on Smart Computing and Software Engineering (SCSE). IEEE (2025). https://doi.org/10.1109/SCSE65633.2025.11030972. https://www.researchgate.net/publication/392684781
18. Taibi, D., Mezzalira, L.: Micro-frontends: principles, implementations, and pitfalls. ACM SIGSOFT Softw. Eng. Notes **47**(4), 25–29 (2022). https://doi.org/10.1145/3561846.3561853
19. Yang, C., Liu, C., Su, Z.: Research and application of micro frontends. IOP Conf. Ser. Mater. Sci. Eng. **490**, 062082 (2019)

Industrial Views on DevOps Adoption Before and After Implementation: A Qualitative Comparison

Sebastian Copei[1,3(✉)], Oliver Hohlfeld[1], and Jens Kosiol[2]

[1] Distributed Systems, University of Kassel, Kassel, Germany
sco@uni-kassel.de,sebastian.copei@iee.fraunhofer.de,
oliver.hohlfeld@uni-kassel.de
[2] Philipps-Universität Marburg, Marburg, Germany
kosiolje@mathematik.uni-marburg.de
[3] Innovationfield Digital Ecosystems, Fraunhofer IEE, Kassel, Germany

Abstract. DevOps is establishing itself as an important software development principle, pushing more organizations to adopt the trend for their projects. Several studies have investigated the benefits and challenges of DevOps adoption, but a comparison of practitioners who already use DevOps with those who are about to adopt is still missing. By comparing these two groups, we want to find possible differences in expectations for adopting DevOps and how planned adoption processes deviate from actual performed adoption processes. Moreover, seeing if the industrial assessments and understanding of DevOps change over time is interesting. To close this gap, we conducted interviews about DevOps adoption with nine representatives of five companies. Our study includes subjects who use DevOps and those who plan to do so soon. We compare their assessments with each other and with the literature. Among other things, we find a high agreement between expected and actual benefits (which are also consistent with those reported in the literature) but a potential underestimation of cultural difficulties among subjects who plan to adopt DevOps. Furthermore, we find that customer expectations and demands are decisive factors in *why* and *how* DevOps is used.

Keywords: DevOps · Qualitative Study · Adoption

1 Introduction

DevOps is becoming an established paradigm for software development, deployment, and maintenance. There might be a hype around it – but studies on DevOps adoption consistently find certain benefits associated with its use like fast release cycles, high automation, and many more (see Sect. 2). Such studies, however, also report major challenges in the adoption process – from complex tooling with a steep learning curve to strong cultural changes for the involved persons. DevOps adoption is depicted as a complex and demanding process,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 202–211, 2026.
https://doi.org/10.1007/978-3-032-04403-7_18

which accords to anecdotal evidence and our own experience [2]. The literature seems to lack studies that compare the expectations of people about to adopt DevOps with the expectations and experiences of those already using it. By separating our participants into these two groups, we hope to better understand how the adoption process of DevOps may differ from planned and reality. If there are deviations, what are these explicitly? Does DevOps fulfill the expected benefits? What are the reported and expected obstacles during the adoption process? How do the reasons for using DevOps differ between adopters and non-adopters? Furthermore, existing studies on DevOps adoption are often small and local and DevOps is expanding, which might lead to changes in its perception. All this suggests to further study the process of DevOps adoption. Understanding it better could help to devise tools that make it easier. For example, a better understanding of non-adopters suggested expectations and the adopters' actual learning could help develop tools that support the adoption process.

In this paper, we report on a small study we conducted about DevOps adoption. Our focus was on comparing the perspectives of adopters and non-adopters who are about to start using DevOps. The results presented in this paper are part of a larger interview series initially conducted to evaluate a tool for supporting the technology-choosing process for large-scale systems. The findings in this paper were not related to the overall study's intention but, from our perspective, are still relevant to the research on the DevOps adoption process. Therefore, we decided to report them in this paper. Concretely, we aim to answer the following research questions (RQs):

RQ1 How do the expectations of DevOps differ from reality?
- **RQ1.1** How do expected benefits compare to experienced ones?
- **RQ1.2** How do reasons for not adopting DevOps compare to experienced obstacles?

RQ2 Are the reasons for adopting DevOps in change?
- **RQ2.1** What specific requirements do customers have in terms of the tools that have to be used?

For this, we performed interviews with 9 representatives from 5 different companies belonging to the two different groups (about to adopt and adopters). Our main findings are the following: We find a realistic assessment of DevOps benefits, but our results also indicate that participants about to adopt might underrate the cultural challenges ahead of them. Furthermore, we find that using DevOps increasingly becomes a customer requirement – evidence that DevOps is on the march and a result we could not find in the literature so far.

Structure. We describe related work in Sect. 2. In Sect. 3, we report our study, including methods (Sect. 3.1) and results (Sect. 3.2). Finally, we discuss and interpret our results in Sect. 4 and summarize our work and further research steps in Sect. 5.

2 Related Work

Several publications address different aspects of the DevOps adoption process from a technical or organizational perspective. From the organizational perspective, recent publications have examined different models to support the adoption of DevOps [4,13]; others collect best practices for it [8,9,14].

Different qualitative studies investigate a wide range of questions about DevOps in the industry, such as practitioners' perceptions of its benefits and challenges [1,11,15,16], observation [5,7,17], improvement of the whole adoption process of DevOps [10], or detection critical success factors of DevOps [6]. There is also a quantitative case study, measuring the effects of DevOps adoption [12].

For the purposes of this paper, we extracted the following from the literature cited above: (1) reasons to adopt DevOps; (2) benefits experienced through DevOps; (3) challenges of DevOps adoption; and (4) implemented DevOps practices. In each case, we present our summary without any particular order. Regarding reasons (1) we extracted: Fast release cycles, high grade of automation, reducing technical debts, improving communication, feedback loops, availability, and observability. Regarding benefits (2) we found: Improved software quality, fast release cycles, high grade of automation, improved scalability, and reduced costs. Mentioned challenges (3) are: Scepticism of developers, scepticism of management, missing knowledge, complex technology stack, teams need to be reorganized, steep learning curves for adopters, and security concerns through cloud usage. Finally, implemented practices (4) consist of: Source control, CI/CD pipelines, culture shifts with team reorganization, usage of cloud from hyperscaler, continuous monitoring.

While some of the cited studies accompany DevOps adoption processes (i.e., they are longitudinal), we could not find a comparison of the views of practitioners who already use DevOps with the views of practitioners who want to use DevOps but are not doing so yet.

3 A Qualitative Study on DevOps Adoption

In order to answer our questions about the possibly changing reasons for the use of DevOps and the different expectations of future users, we conducted a qualitative study, which we present in this section.

3.1 Methodology

We interviewed participants to compare their views on DevOps adoption before and after implementation. In the first group, participants had not adopted DevOps but concretely planned to do so for a near-term project (however, they did not specify a timeframe for their planned adoption). Participants in the second group should be actively using DevOps for their projects. We recruited the participants from the first author's network and ensured that the resulting participants were distributed among multiple companies across Germany. We used a

mix of structured and semi-structured interviews for our qualitative study. The questions were predefined in a fixed order, but the participants could answer freely. The first author performed the interviews online (individually with each participant) and recorded them with Microsoft Teams. Microsoft Teams also created the initial transcript of the interviews. Afterward, we refined the transcripts by hand to improve their readability. During the first screening of the interview results, we decided to clarify our findings through follow-up questions. These were sent and answered by e-mail. During the data collection phase, we extracted the core statements from the participants' answers using the refined transcripts and compiled them into a spreadsheet. The refined transcripts were in German, so the data extraction and compiling into the spreadsheet were also done in German.

For the sake of transparency, the transcripts of the interviews, as well as our data collection files, are available at Zenodo [3]. All files were automatically translated from German to English using DeepL. As previously mentioned, the presented study is part of a bigger interview study. Therefore, the transcript may appear short, but the full interviews were longer.

3.2 Results

We interviewed nine participants from five companies in Germany, which consist of software development service providers and consulting firms. Participants from the same company were interviewed regarding different projects. Four of the nine participants want to use DevOps in their projects in the near future (Group 1 (**G1**)), and five are already using it (Group 2 (**G2**)). Table 1 provides an overview of our interview partners and their assignment to study groups.

Table 1. Overview of survey participants

Participant ID	Company ID	Industry Sector	Position	Group
P1	C1	Heat Industry	Software Engineer	2
P2	C1	Heat Industry	Software Engineer	2
P3	C2	Energy Economics	Software Engineer	2
P4	C3	Energy Economics	Software Engineer	1
P5	C4	SAP	Technical Consultant	1
P6	C4	SAP	Technical Consultant	1
P7	C5	Multiple	Cloud Solution Architect	2
P8	C5	Multiple	Cloud Engineer	2
P9	C5	Multiple	Lead DevOps Engineer	1

Table 2 provides the results of our interviews with G1 (about to use DevOps) and Table 3 the ones for G2 (already using). We asked participants from G1

the following questions. **Q1**: Why do you want to use DevOps in your project? **Q2**: Are there any reasons why you did not use it earlier? **Q3**: Were there any obstacles beforehand that prevented you from applying DevOps? We also asked participants from G2 about their reasons for adopting DevOps and experienced obstacles but additionally asked for their assessment of the experience. The questions are as follows. **Q4**: Why did you use DevOps for your project? **Q5**: Was the introduction worthwhile? **Q6**: Are there any reasons that might prompt you to migrate back? **Q7**: Have you encountered any obstacles during the introduction of DevOps?

Regarding reasons for adoption (Q1, Q4), G1 (about to adopt) named time savings through automation (P5, P6, P9), outside pressure *"Yes the software is 20 years old. I do not know if it would benefit from DevOps, but it definitely would not happen in the near future, and my points of contact with DevOps now actually only com mainly through this new project, in which we will need it at all."* (P4), and expected improved communication within the team (P6). G2 (already using) also mentioned automation (P3, P8) and, similarly to outside pressure *"That is because we work as software developers on custom software. Over the years, as the industry and our customers have transformed, we have basically taken over DevOps operations because they were getting closer and closer to us ... I would say it's a bit driven by the industry and also by the market and development."* (P7), the industry trend to use DevOps (P7), and additionally the customer requirement (P1, P2, P8), or necessity by high project complexity (P1).

Table 2. Interview results for Group 1 (about to adopt)

Question	Answers	Participant
Q1	Pressure from outside, we have to move with the times	P4
	Automation and the associated time savings through pipelines	P5, P6, P9
	We hope to improve communication within the team	P6
Q2	Time-consuming migration due to the age of the project	P4, P5
	We do not have the needed knowledge in our team	P5, P6
	It was not a requirement of our client	P4, P9
Q3	Our client has not defined DevOps as a requirement	P4, P5, P9
	We do not have the needed knowledge in our team	P6

Asked for their reasons to not adopt DevOps earlier and the obstacles that hold them back in the past (Q2 and Q3), participants from G1 (about to use DevOps) reported that they do not yet use DevOps because it hadn't been required by the client before (P4, P5, P9), because of the complex migration process (P4, P5), and because of missing knowledge (P5, P6). Regarding the actually experienced obstacles (Q7), G2 (already using) reported the lack of knowledge and personnel and the steep learning curve (P1, P3, P7, P8), conflicts, where individual persons had to be convinced of the benefits of using DevOps

(P3, P7, P8), and missing insight into the customer's pipeline (P1). However, a single participant reported not having experienced any real obstacles (P2).

Regarding the assessment, G2 (already using) unequivocally agrees that the adoption of DevOps was worthwhile (Q6) and lists advantages like the rescue of the project (P1, P2), time savings (P3, P8), better resilience, observability, and team/client communication (P7) when answering Q5.

Table 3. Interview results for Group 2 (adopters)

Question	Answers	Participant
Q4	It was a requirement of our client, they want it because of the automation possibilities	P1, P2, P8
	Without DevOps, the project cannot be built due to its complexity	P1
	We have saved time through a high degree of automation	P3, P8
	The general industry is moving to DevOps, so we followed the trend	P7
Q5	Yes, without DevOps we could no longer develop the project	P1, P2
	Yes, the time saving factor is huge	P3, P8
	Yes, our benefits are better resilience, observability, and team/client communication	P7
Q6	No, the drawbacks of losing the benefits are too critical	P1, P2, P3, P7, P8
Q7	No direct obstacles; we see the overhead of DevOps as the prize to pay	P2
	No insight into the customer's pipeline	P1
	Lack of knowledge/trained personnel; steep learning curve	P1, P3, P7, P8
	Needed to convince individual persons of the benefits of DevOps	P3, P7, P8

Surprised by how frequently the adoption of DevOps was mentioned as a customer requirement, we asked participants in both study groups in follow-up emails which technologies had been explicitly specified by the customer in their project (**Q8**) and which technologies they had explicitly been free to choose (**Q9**). They reported the following tools and technologies as specified by their customers: CI/CD with GitLab, GitHub or Jenkins (P1, P2, P6, P9), Jira (P2, P6), Grafana (P1, P6), Docker and Visual Studio Code (P6), managed Kubernetes from AWS, GCP or Azure (P7, P9), Nexus (P1, P2), and Confluence, Gradle, SonarQube and Firebase (P2). For the tools they were free to choose they stated the following: they were only free to specify the development workflow (P6), they could only choose the communication channel (P1, P2), they got a list of authorized cloud services (P7, P9), they could choose the programming language and frameworks as long as these do not affect the infrastructure as code system (P9).

4 Key Findings

In this section, we interpret the obtained results to answer our research questions.

4.1 The Cultural Aspect of DevOps Still Seems to Be Underestimated

In our study, the technical reasons of G1 (about to use DevOps) for starting to adopt DevOps are congruent with the reasons and the experienced benefits

reported by G2 (already using), namely expected benefits by automation and improved communication. These benefits are also well-documented in scientific literature (see Sect. 2). In contrast, the reasons reported by G1 (about to use DevOps) to not adopt earlier are not entirely congruent with the actually experienced obstacles reported by G2 (already using). Missing knowledge and high costs are reported by both groups. However, the adopters reported additional non-technical issues – in particular, conflicts in convincing individual persons of the benefits of DevOps have been reported by 3 of 5 subjects of G2 (already using).

Answering RQ1, we interpret these findings as indicating that adopting DevOps is associated with benefits that adopters know about and can realistically expect to experience (RQ1.1). The strong focus on potential technical obstacles in the reasons of G1 (about to use DevOps) to not adopt DevOps earlier might indicate that industrial representatives still underestimate the complexity of the cultural shift needed to adopt DevOps (RQ1.2). This is surprising as this issue is well-reported in the literature on DevOps adoption (see Sect. 2).

4.2 DevOps as a Requirement of the Customers

As seen in Sect. 2, important reasons for adopting DevOps and benefits experienced by that have been reported in the literature include increased automation and improved communication. As just worked out above, we also find this in our study. But we also saw an important new trend that, to the best of our knowledge, has not been reported in the literature so far: In our study, 3 of 5 participants from G2 (already using) mention an explicit customer requirement as one of the reasons for adoption. While no participant from G1 (about to use DevOps) answers this when directly asked for reasons (Q1), 3 of 4 mention a missing customer requirement as reason for not adopting earlier (Q2 and Q3). This shows that also their adoption is accompanied by a now raised customer requirement. Additionally, in both groups outside pressure resp. a wish to be part of the trend is mentioned.

When examining the *concrete customer requirements* from our follow-up questions to understand their understanding of DevOps, we see that the tools requested and specified by the customers are congruent with the DevOps practices described by the literature and cover the DevOps cycle (consisting of planning, coding, building, testing, releasing, deploying, operating, and monitoring). Specifically, tools for source control with the ability to define CI/CD pipelines like GitLab or Github (P1, P2, P6, P9), containerization and Cloud Provider Platforms with Kubernetes (P6, P7, P9), for agile methods like Jira and Confluence (P2, P6), and monitoring like Grafana (P6) are mentioned.

To summarize and answer RQ2, the reasons for adopting DevOps are shifting as, beyond technical reasons, customer requirements are becoming a critical reason for not only *why* but also *how* to adopt (including stringent requirements on tooling). In that, for both the adopters of DevOps and the clients who desire DevOps in their projects, the primary association with DevOps seems to be a high grade of automation through CI/CD pipelines and deployment in a

containerized/cloud platform environment. While further research is needed to substantiate that claim, we interpret these results cautiously as showing increasing acceptance of DevOps and maybe even indicating a move to more modern software development practices in German industry. As our questioned companies operate in different industry sectors, this could be a hint that results may also apply to the broader digitization movement in Germany.

4.3 Threats to Validity

The main threats to the *external validity* of our study are the small sample size, the recruiting method for our subjects, and the study method being interviews, where the obtained results represent opinions and viewpoints. To tackle these threats, we tried to recruit our subjects as heterogenous as was possible to us, having representatives from different companies working in different branches, and ensuring that participants from the same company work at least in different projects. The main threat to *internal validity* is that the first author individually performed the interviews and data extraction, increasing the threat of bias and misinterpretation. To mitigate this threat, all three authors together discussed examples for data extraction and the whole interpretation of the obtained data. Furthermore, as the participants were recruited from the first author's network, social desirability could have led to untrustworthy answers during the interviews. However, this threat was mitigated through the design of the questions by directly asking about negative aspects.

5 Conclusion

This paper presents the results of a qualitative study which involved standardized interviews with nine participants from five different companies across Germany, belonging to two different groups: participants about to start adopting DevOps and participants already using it. The key findings of our work are: i) Expectations for and experiences of DevOps benefits nearly overlap in both groups. In particular regarding benefits, both match and furthermore agree with benefits reported in scientific literature. However, it might be possible that the non-adopters underestimate the cultural challenges of DevOps ahead of them. This finding is somewhat surprising because cultural challenges are a well-reported obstacle in the DevOps adoption process. ii) Besides the benefits of DevOps, the primary reason the interviewed participants (want to) use DevOps is the requirement of their clients. We could not find this insight being reported in the literature so far and cautiously interpret it as showing that awareness of DevOps and its benefits increases (in Germany). iii) Industrial practitioners perceive DevOps as associated with a high degree of automation through CI/CD pipelines and deployment in a cloud environment.

Nevertheless, we still have open questions that need further investigation. Why is there a noticeable gap between expected and experienced challenges?

How can the stated challenges be mitigated without adding more complexity? Can our findings also be observed in the European Union or even worldwide?

We will conduct further investigations to close these gaps.

Data Availability. The transcripts of the interviews, as well as our data collection files, are available at Zenodo [3].

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aouni, F.E., Moumane, K., Idri, A., Najib, M., Jan, S.U.: A systematic literature review on agile, cloud, and DevOps integration: challenges, benefits. Inf. Softw. Technol. **177**, 107569 (2025)
2. Copei, S.: A practical experience report on moving from VM to Kubernetes in an academic research group. In: International Conference on Microservices (2023)
3. Copei, S., Hohlfeld, O., Kosiol, J.: Interview results for "Industrial views on DevOps adoption before and after implementation: a qualitative comparison, January 2025, https://doi.org/10.5281/zenodo.14732141
4. Díaz, J., et al.: Harmonizing devops taxonomies - a grounded theory study. J. Syst. Softw. **208**, 111908 (2024)
5. Erich, F.M.A., Amrit, C., Daneva, M.: A qualitative study of DevOps usage in practice. J. Softw. Evol. Process **29**(6), e1885 (2017), e1885 smr.1885
6. Jayakody, V., Wijayanayake, J.: Critical success factors for DevOps adoption in information systems development. Int. J. Inf. Syst. Proj. Manag. **11**(3) (2023)
7. Jones, S., Noppen, J., Lettice, F.: Management challenges for DevOps adoption within UK SMEs. In: Proceedings of the 2nd International Workshop on Quality-Aware DevOps, QUDOS 2016, pp. 7–11. ACM, New York, NY, USA (2016)
8. Kumar, A., Nadeem, M., Shameem, M.: Assessment of DevOps lifecycle phases and their role in DevOps implementation using best–worst MCDM. Int. J. Inf. Technol. **16** (2024)
9. Kumar, A., Nadeem, M., Shameem, M.: Multicriteria decision-making-based framework for implementing DevOps practices: a fuzzy best-worst approach. J. Softw. Evol. Process **36**(6), e2631 (2024)
10. Luz, W.P., Pinto, G., Bonifácio, R.: Adopting DevOps in the real world: a theory, a model, and a case study. J. Syst. Softw. **157**, 110384 (2019)
11. Lwakatare, L.E., et al.: DevOps in practice: a multiple case study of five companies. Inf. Softw. Technol. **114**, 217–230 (2019)
12. Port, D., Taber, B., Emkani, P.: Investigating effectiveness and compliance to devops policies and practices for managing productivity and quality variability. J. Syst. Softw. **213**, 112030 (2024)
13. Pérez-Sánchez, J., et al.: A theory on human factors in DevOps adoption. Comput. Stand. Interf. **92**, 103907 (2025)
14. Rafi, S., Akbar, M.A., Mahmood, S.: Aligning academic with industrial needs: investigating DevOps. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, EASE 2024, pp. 516–519. ACM, New York, NY, USA (2024)

15. Riungu-Kalliosaari, L., Mäkinen, S., Lwakatare, L.E., Tiihonen, J., Männistö, T.: DevOps adoption benefits and challenges in practice: a case study. In: Abrahamsson, P., Jedlitschka, A., Nguyen Duc, A., Felderer, M., Amasaki, S., Mikkonen, T. (eds.) PROFES 2016. LNCS, vol. 10027, pp. 590–597. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-49094-6_44
16. Smeds, J., Nybom, K., Porres, I.: Devops: A definition and perceived adoption impediments. In: Lassenius, C., Dingsøyr, T., Paasivaara, M. (eds.) Agile Processes in Software Engineering and Extreme Programming, pp. 166–177. Springer International Publishing, Cham (2015)
17. Trigo, A., Varajão, J., Sousa, L.: Devops adoption: insights from a large European Telco. Cogent Eng. **9**(1), 2083474 (2022)

The (C)omprehensive (A)rchitecture (P)attern (I)ntegration Method: Navigating the Sea of Technology

Sebastian Copei[1,3(✉)], Oliver Hohlfeld[1], and Jens Kosiol[2]

[1] Distributed Systems, University of Kassel, Kassel, Germany
{sco,oliver.hohlfeld}@uni-kassel.de
[2] Philipps-Universität Marburg, Marburg, Germany
kosiolje@mathematik.uni-marburg.de
[3] Innovationfield Digital Ecosystems, Fraunhofer IEE, Kassel, Germany
sebastian.copei@iee.fraunhofer.de

Abstract. The technological landscape changes daily, making it nearly impossible for a single person to be aware of all trends or available tools that may or may not be suitable for their software project. This makes tool selection and architectural design decisions a complex problem, especially for large-scale software systems. To tackle this issue, we introduce CAPI, the Comprehensive Architecture Pattern Integration method that uses a diagnostic decision tree to suggest architectural patterns depending on user needs. By suggesting patterns instead of tools, the overall complexity for further decisions is lower as there are fewer architectural patterns than tools due to the abstract nature of patterns. Moreover, since tools implement patterns, each non-proposed pattern reduces the number of tools to choose from, reducing complexity. We iteratively developed CAPI, evaluating its understandability and usability in small studies with academic participants. When satisfied with the outcome, we performed a user-study with industry representatives to investigate the state-of-the-art in technology selection and the effectiveness of our proposed method. We find that technology selection is largely performed via trial and error, that CAPI is uniformly perceived as helpful, and that CAPI is able to reproduce the productive architectural environments of our participants.

Keywords: Architectural Patterns · Technology Selection · DevOps

1 Introduction

The trend towards developing software as a (web) service promotes the creation of (large-scale) distributed systems, thereby increasing the complexity of software architectures. Due to the complexity and scale of these systems, they are often implemented as microservice architecture and by using agile methods and DevOps organizational practices, such as CI pipelines, that enable teams to

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 212–228, 2026.
https://doi.org/10.1007/978-3-032-04403-7_19

organize efficiently. To facilitate the development of these systems, a wide range of software frameworks has emerged (e.g., as illustrated in the Cloud Native Landscape atlas). These frameworks address the implementation of individual design patterns, such as API gateways, databases, and message brokers, to help simplify the developers' work in building their systems. Despite the chosen software development methods, previous work highlights significant challenges and barriers in developing large-scale systems. Commonly cited issues include a lack of collaboration and communication, insufficient skills and knowledge, and ineffective criticism practices [6,9,10]. Additionally, as we will show in this paper, developers face challenges in selecting software frameworks. This issue primarily arises from the overwhelming number of frameworks that implement similar design patterns (e.g., message brokers) and the rapidly evolving software landscape.

In this paper, we tackle the challenge of navigating the complex landscape of software development by simplifying the process for developers to choose design patterns for large-scale software as a service projects. Rather than recommending tools that frequently change, our approach focuses on selecting the fundamental architectural design patterns that these tools implement. This results in a more manageable set of tools, making it easier for developers to make selections in a subsequent step. These design patterns are foundational for building scalable software architectures, and making an informed choice is a crucial aspect of software architecture design. Therefore, selecting the appropriate architectural design pattern is a critical step in the software architecture process. To assist in this selection, this paper introduces the Comprehensive Architecture Pattern Integration (CAPI) method, driven by a decision tree. To better address the needs of developers and architects, we will *i)* survey their requirements and *ii)* assess our CAPI method through interviews with industry practitioners. This way, we aim to answer the following research questions:

RQ1 How do practitioners select software architectures for their projects?
RQ2 What problems do the practitioners face during this process?
RQ3 How does the CAPI method help industry practitioners select better software architectural designs more easily?

We evaluate the CAPI method in interviews with industry practitioners and show that it addresses a need. Our contributions are as follows.

- We show that there is lack of approaches that guide the selection of software architectural designs in modern software projects.
- We contribute the CAPI method, which enables software architects to select the appropriate architectural design pattern for realizing a given software. The CAPI method is realized as a decision tree with questions that guide the design pattern selection process. The CAPI method covers the current architecture designs, with a focus on microservices as the emerging implementation approach.
- We evaluate the CAPI method with industry practitioners and show that it solves a technology selection problem that industry practitioners face. Our

interviews highlight that our method is perceived as a useful approach in situations where software design patterns need to be selected.

Structure In Sect. 2, we describe the related work. In Sect. 3, we introduce the CAPI method. In Sect. 4, we review our qualitative study, and in Sect. 5, we discuss the results. Finally, Sect. 6 summarizes our work and briefly describes our next steps.

2 Related Work

Architectural Design Selection. Regarding architecture selection, there is a range of studies that investigate the relation between architectural patterns and quality attributes of the resulting system [5,7,8,12,13]; Haoues et al. [5] even derive a decision model where a foundational architectural design decision is suggested depending on quality attributes one wants to focus on. However, their systems addresses the foundational style of architecture to be used in a system (model-driven architecture, component-based architecture, service-oriented architecture, event-driven architecture, aspect-oriented architecture), whereas we are interested in helping users to decide which of the plethora of patterns to use that have been suggested in connection with modern, often cloud-based development. In particular for component-based architectures, approaches exist to automatically optimize an architectural design, given as a Palladio component model [1,11]. Razzaq extracts patterns for microservice development from the literature and maps these to common problems in IoT development they can help to solve [14]. Decision support systems (DSS) for technology selection have also been developed in other contexts, e.g., for database technologies [15]. Finally, in the paper most similar to ours, Farshidi and Jansen develop a DSS that supports the decision-making process for pattern-driven software architecture [4]. Whereas our method relies on a decision tree fueled by diagnostic questions, the DSS mentioned in the publication uses a list of definable requirements prioritized by the Must have, Should have, Could have, and Won't have (MoSCoW) technique to determine fitting patterns. The authors use an inference engine to generate a list of patterns that fulfill the requirements. Similar to our approach, the authors extract the patterns used in their method from a literature review. In total, the authors used 29 patterns with 40 features. From this set, the inference engine can list fitting patterns depending on the features that will fulfill a requirement. Our method uses a set of 47 patterns and creates a result set due to a decision tree instead of an inference engine. In comparison, the DSS and our method share four patterns, meaning there are 25 patterns we are not including and 44 they do not include. We explain these deviations with the methodology of the literature reviews performed beforehand. Nevertheless, from this deviation, we are confident that our method can cover a broader set of target architectures as the number of patterns it can suggest is significantly higher. Moreover, the extendability of an inference engine, if they want to add more patterns, is questionable as the complexity rises for each pattern added. In contrast, the change

or the extension of our decision tree is easily possible by adding further questions or even extra paths.

We observe that none of the surveyed works address the problem of architectural decision-making for current software stacks in software as a service projects. Other methods do not build on the same pool of patterns as ours and address different levels in the design space. We aim at closing this gap with the CAPI method.

3 The CAPI Method

In this section, we describe the decision tree (Sect. 3.1) that constitutes the heart of the CAPI method and present its iterative development process (Sect. 3.2).

3.1 CAPI's Decision Tree

CAPI's overall approach involves asking users a series of questions, organized in a decision tree format. Based on their answers, it provides a set of architectural design patterns to implement the envisioned system.

Assumptions and Entry Point of CAPI. *In our approach, we assume that a decision for a* foundational architectural style *has already been made.* That is, it has been decided to implement the software as monolithic, client-server or microservices. The process of making this decision is beyond the scope of this paper. Depending on the selected foundational architectural style, a user is guided through a series of questions about the envisioned system; the given answers determine a set of *architectural design patterns* that are suitable to implement the envisioned system in the selected foundational architectural style. The overall abstract structure of the decision tree is depicted in Fig. 1.

The initial question of our decision tree asks for the foundational architectural style, with "monolithic", "client-server" and "microservices" being possible answers. We opted to not support users regarding that fundamental decision for two reasons: First, as seen in Sect. 2, such fundamental decisions have at least partially been addressed in prior work. And, more importantly, in our experience these fundamental decisions are not only influenced by technical considerations but also heavily by cultural (e.g., knowledge and openness of development team) or economical (e.g., "Which resource can we afford to use for this system?") ones. Thus, the foundational decision deserves its own, multi-faceted process that can be distinguished from a process that helps with an appropriate technical implementation, on which we want to focus in this work. We support "monolithic", "client-server" and "microservices" as foundational architectural styles because, in our experience (and also suggested by the results of Kassab et al. in [8]), these are widespread and considerably different ways in which to develop a modern service-oriented system.

Structure of CAPI's Decision Tree. Generally, the flow of the decision tree is structured by six *categories*, into which architectural design patterns can be

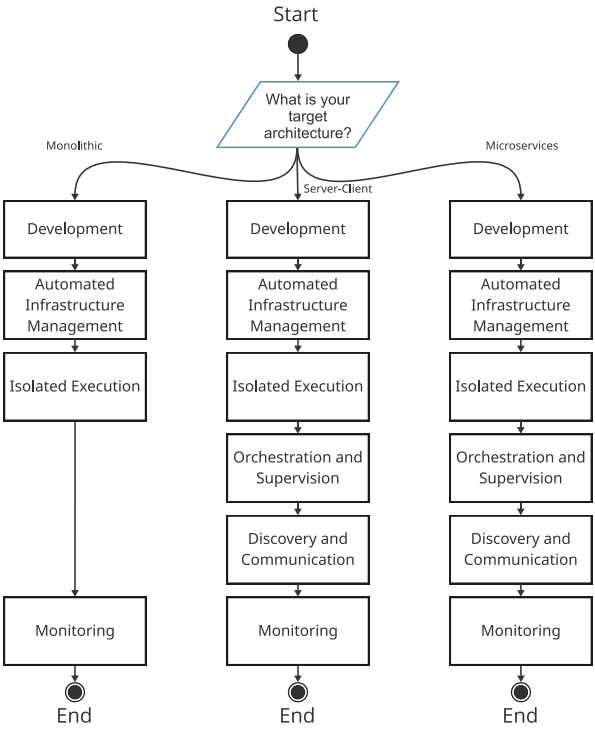

Fig. 1. Abstract structure of CAPI's decision tree using categories

grouped (compare again Fig. 1). Iteratively, for each category and for each pattern that belongs to it, a series of questions is asked to determine whether or not the respective pattern should belong to the result set. Currently, the tree consists of three paths: Each possible answer to the initial question about the fundamental architectural style leads to a distinct path in the tree because in each case, there is a different set of applicable patterns to ask for.

Our selection of design patterns is based on those identified from our recent survey study [2], where we systematically collected architectural design patterns for DevOps and microservices. From the 52 reported patterns, we use 47 to design the decision tree and group these into six different categories, namely *Development*, *Automated Infrastructure Management*, *Isolated Execution*, *Orchestration and Supervision*, *Discovery and Communication* and *Monitoring*. Our grouping of patterns is inspired by the well-known DevOps cycle. An overview of the categories and examples for patterns belonging to these is provided in Fig. 2.

For each pattern of each category that is relevant for the selected architectural style, a series of two to three questions decides whether the decision tree suggests to use that pattern; sometimes such a series of questions also checks for a small group of patterns that should be used together. The first question(s) always check whether there are positive reasons to use the pattern(s) to develop the envisioned

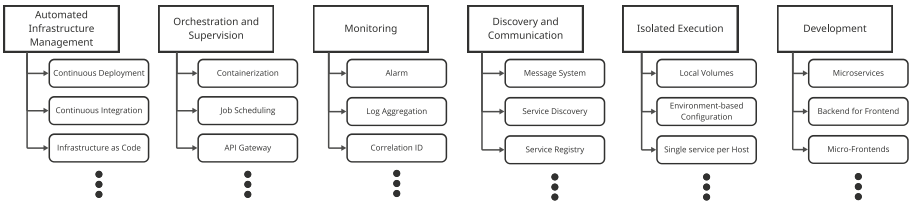

Fig. 2. Categories and Patterns used to create the decision tree

system. The last question in such a series is a *contraindication* question to ensure that there is no reason to not use the pattern(s). For simplicity, the questions can only be answered with "yes" or "no". If a user's answer to the first question(s) already indicates that a pattern is not applicable, the remaining question(s) for that pattern are skipped and the process immediately continues with the questions for the next pattern. To not influence a user in one way or other, they are only presented the questions, without being explicitly told for which pattern the current question checks. The relevant patterns are collected in the background and only presented to the user as result when they traversed the whole tree.

Illustration of Questions and Usage. We use the following scenario to illustrate a snippet of the tree's usage.

Assume you work in a large software engineering team at Streamland*, a newly founded company that has acquired most streaming licenses for movies and series worldwide. Streamland wants to build a new platform that combines all the current streaming services so customers only need a single service to stream all movies and series. The application should be available for all devices, like TVs, smartphones, or PCs.*

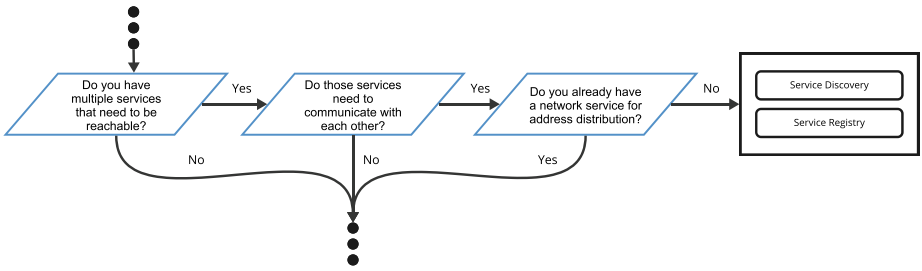

Fig. 3. Example question flow

As shown in Fig. 1, the first decision is to choose the overall architectural style. In this example, we choose a microservice architecture. Given that decision, the user is asked questions for patterns that belong to the "Discovery and Communication" category (which is not the case for a monolithic architecture),

amongst which there are "Service Discovery" and "Service Registry". Figure 3 shows the question flow for these two patterns. Here, the first two questions ask whether the patterns are needed, and the third question is the contraindication question. Only answering the first two questions with "yes" and the third one with "no" will lead to the result that the two patterns are added to the result set of suggested patterns. Otherwise, we will continue to the next question flow. This concept is applied to all question flows for each path in the tree. In the case of our example, as we want to implement a microservice-based architecture we need to use multiple services that need to communicate with each other. If we already have infrastructure that hosts mechanics for service discovery, like Kubernetes, the decision tree will send us to the next question flow. Otherwise, it would suggest the patterns for *Service Discovery* and *Service Registry*. CAPI's full decision tree is available in our research bundle [3].

Implementation. To evaluate the CAPI method, we developed a simple online tool, in which a user can click through the questions. In the end, the tool displays the suggested patterns. The tool is available online [3].

Discussion. The CAPI method is intentionally focusing on selecting the architectural design patterns that are relevant in realizing the intended software. This way, we intentionally do not focus on selecting the foundational architectural style (e.g., monolithic vs. microservice), since this decision is bound to other factors (e.g., current practices in the software team, etc.) and already addressed by prior work. Furthermore, we intentionally focus the decision tree on the current three dominant architectural styles (i.e., monolithic, client-server, and microservice). The approach described in this work can be used to extend the decision tree in future work to cover a broader set of styles. Extending the tree can be done by adding more branches. In that, it is even possible to gain (large parts of) the new branch by taking the most comprehensive branch of our tree, the one for microservices, and dropping all questions that check for patterns irrelevant to the considered style. In fact, we partly developed our tree in that way, highlighting that extensions are easily possible.

3.2 Iterative Development Process and First Quantitative Analysis

We developed our decision tree in two iterations and concluded each of the two phases with a small-scale quantitative study to determine the intelligibility of our questions and to get a first impression of the quality of architectural designs users obtained with it. The two studies were performed with disjoint groups of participants recruited from academia. In each case, the participants should apply the tree in its form at that time in four artificial scenarios of different complexities (including, e.g., the Streamland scenario from above as most complex system). For both these studies, the participants were provided with the questions of the tree printed on paper. In the end, they were presented the sets of patterns that resulted from their answers for each scenario and then asked to answer a questionnaire. There we asked questions (some open, some answerable on a Likert-scale) to determine how the participants used the tree, as how intelligible

the tree's questions and as how helpful its usage was perceived to arrive at a design, and we asked for general feedback. Moreover, we compared the resulting architectures that the participants obtained to architectures that we perceived as appropriate for the scenarios.

In our initial draft, we already used the overall structure of the tree as described above and the same set of patterns. However, per pattern, we asked a single question that checked for existence of a reason to use that pattern. We performed our first study with a group of seven academics, consisting of PhD students, PostDocs and professors. The participants perceived the tree as helpful and rated the intelligibility of the asked questions as high. However, when we compared the resulting designs to our intended ones, we observed that the participants tended to answer with "yes", leading to unnecessary large sets of patterns for many scenarios.

We explained that phenomenon with the first draft of the tree not associating any kind of "cost" with selecting a pattern. Besides minor rephrasing of individual questions, we tackled that issue with (i) asking for several reasons to use a pattern in the case of complex ones and (ii) introducing our contraindication questions that check for the existence of a reason to not use a pattern. We performed a second user study with exactly the same setup as the first, now using the revised tree. We also recruited our second set of participants (ten) from academia, now consisting of advanced undergraduates and master and PhD students. Despite being far less experienced (academically and in terms of DevOps and cloud-based development), the phenomenon of selecting a large amount of unnecessary patterns was noticeably reduced. The decision tree was still largely perceived as intelligibly and helpful. Therefore, we just used the additional feedback of that study to clarify a few further questions and used the resulting design to evaluate our approach with industrial practitioners, as presented next.

4 CAPI in Action: An Industrial User Study

We are now interested in examining the satisfaction levels of industry professionals with CAPI. To do so, we have conducted interviews with industrial representatives to evaluate whether CAPI can help select better software architectures. We are further interested to understand how technology selection is performed in industrial practice, which challenges practitioners face, and if the CAPI method helps in dealing with these challenges (RQs 1–3).

4.1 Methodology

We base our study on using a mix of structured and semi-structured interviews. We used predefined questions in a fixed order but allowed participants to answer freely. The interviews ended with a case study in which the participants had to use CAPI via a web interface. All interviews were performed by the first author (individually with each participant) and recorded with Microsoft Teams. Microsoft Teams also created the initial transcript of the interviews. Afterward,

we refined the transcripts by hand to improve their readability. During the data collection phase, we extracted the core statements from the participants' answers using the refined transcripts and compiled them into a spreadsheet. The transcript of the interviews and our data collection files are available at Zenodo [3].

Participant Selection. As DevOps is an emerging technology for developing large-scale applications, we focused on participants who use DevOps for their application development. Furthermore, we separated the participants into two groups. In the first group, participants had not adopted DevOps but concretely planned to do so for a near-term project (Group 1 (**G1**)). Participants from the second group are actively using DevOps for their projects (Group 2 (**G2**)). By separating the participants into these two groups, we aim to better understand the impact of decisions made in the initial phase of projects. These insights could affect the positioning of the CAPI method from a timing perspective. For example, is it enough to only use the method once in the beginning, or may it be better to recheck the decisions with the method in an iterative manner? We recruited the participants from the first author's network and country and ensured that the resulting participants were distributed among multiple companies and different branches of business.

Study Procedure. The procedure of the interviews consisted of the following steps: *i)* Initial questions about the technology choice process and potential problems; *ii)* Participants need to determine the technologies they need using the CAPI method; **G1** does this for the project they want to realize, and **G2** for the project they already have implemented; and *iii)* Questions about the usage of the CAPI method. We asked participants from **G1** the following questions before they used our method (**Q1**, **Q2**) and after they used our method (**Q3**, **Q4**). **Q1**: Do you have a process for identifying necessary technologies? **Q2**: Could there be problems with this process? **Q3**: Would the tree have helped you with your problems in identifying necessary technologies? **Q4**: Would the result of the tree now help you implement DevOps? We also asked participants from **G2** about their technology choice process and their experiences with problems. Additionally, we asked how strongly the results of our method differ from their actual architecture. The questions are as follows. **Q5**: How did you select the necessary technologies? **Q6**: Have you encountered any problems with this approach? **Q7**: Would the tree have helped you with your problems in identifying necessary technologies? **Q8**: How much does the result of the tree differ from the stack you are actually using?

4.2 Results

We interviewed ten participants from six companies in Germany, which consist of software development service providers and consultant firms. We interviewed participants from the same company regarding different projects. The participants are evenly distributed across the two groups. Five participants plan to use DevOps, and five are actively using it. Table 1 overviews our interview partners' demographics and assignment to study groups.

Table 1. Overview of survey participants

Participant ID	Company ID	Branch	Position	Group
P1	C1	Heat Industry	Software Engineer	2
P2	C1	Heat Industry	Software Engineer	2
P3	C2	Energy Economics	Software Engineer	2
P4	C3	Energy Economics	Software Engineer	1
P5	C4	SAP	Technical Consultant	1
P6	C4	SAP	Technical Consultant	1
P7	C5	Multiple	Cloud Solution Architect	2
P8	C6	Consultant	Lead Architect	1
P9	C5	Multiple	Cloud Engineer	2
P10	C5	Multiple	Lead DevOps Engineer	1

Table 2 summarizes the results of the interviews with the participants from G1 (about to adopt), and Table 3 the ones for G2 (already using).

Lack of a Architecture Selection Process. Both the adopters and non-adopters stated they do not have a process to identify needed technology (Q1, Q5). In detail, G1 reported that the technology was specified by their clients (P6, P10) or that they relied on experience and iterated their decision using this experience (P4, P5, P8, P10). G2 also reported that their clients specified technology (P1, P2, P7, P8). Further, they named trial-and-error (P3, P9) and self-motivated employees (P9) as drivers for their technological decisions. This highlights the need for a guided technology selection approach such as CAPI.

Table 2. Interview results for Group 1 (about to adopt)

Question	Answers	Participant
Q1	No	P4, P5, P6, P8, P10
	Technologies were specified by client	P6, P10
	We rely on our experience and do it iteratively	P4, P5, P8, P10
Q2	No, we do not encounter any problems	P4
	By using the already known, we miss new things	P5, P10
	As the knowledge is widely distributed, it is cost-intensive to collect it	P10
Q3	Yes	P4, P5, P6, P8, P10
	It hints in the correct direction	P4, P6, P8
	But as our projects are inflexible, the results need to be complete	P5
	But the method should also suggest tools for the patterns	P6, P10
	But the method should show the reasoning for a pattern	P10
Q4	Yes, the method would have helped us	P5, P6, P8, P10
	The method is a good checklist	P5, P8
	It would be even better if the method showed the relationship between patterns	P8, P10
	The method should be used multiple times during an iterative process	P8
	No, as we do not know all of the patterns	P4

CAPI is Not for Everyone. Not every industry practitioner *directly* faced technology selection problems, e.g., since the technology selection was made by the client. Regarding the problems encountered during the technology selection process (Q2, Q6), G1 reported that they did not encounter any problems (P4), they missed out on new things (P5, P10), and high costs for knowledge collection (P10). G2 reports similar experiences; they also did not encounter any problems as their clients specified everything so there was no choice (P1, P2), high costs were a problem (P3, P9), and finding the correct point in time to move to the next technology if the currently selected one proves to be unsuitable. This highlights that not every industry practitioner needs to use CAPI, in particular when the technology selection is provided by clients. In these cases, however, the problem is simply shifted since the clients need to select the technology.

Does CAPI Solve a Problem? After using the CAPI method and asking if it could help them solve their problems (Q3, Q7), all participants from G1 and G2 agreed that it could and would help them during their technology selection process. Pointing to the right direction (P4, P6, P8) and helping to focus on the actual requirements (P3, P7) are the highlighted positive aspects of the CAPI method. Additionally, participants from G1 also reported that the method should be nearly complete as their project is very inflexible (P5). After suggesting patterns, the method should suggest tools that implement the patterns (P6, P10), and it should show the reasoning for a selected pattern (P10). G2 reported additionally that the method should suggest tools for the patterns (P1, P2, P3, P7), the method should be used multiple times in an iterative process (P3), and it would be helpful if, besides the patterns, the relationship between them should also be highlighted (P9).

Table 3. Interview results for Group 2 (adopters)

Question	Answers	Participant
Q5	There was no process, as our client specified many factors	P1, P2, P7, P8
	We just tried out the technologies that were not specified	P3, P9
	The knowledge comes from self-motivated employees	P9
Q6	No, as everything was specified	P1, P2
	The process of trying out was very cost-intensive	P3, P9
	It is hard to find the correct timing to try the next technology if there are problems	P7
Q7	Yes	P1, P2, P3, P7, P9
	If the method also would suggest tools for the patterns	P1, P2, P3, P7
	It helps focus on what is needed	P3, P7
	But only for greenfield projects without client specifications	P9
	The method should be used multiple times during an iterative process	P3
	prioritization and viewing of the relationship between the patterns would help even more	P9
Q8	On the first view, the results match with our stack	P1, P2, P3, P7, P9
	Some patterns that we are using are not highlighted	P7, P9
	Methods highlight even patterns that we plan to implement	P3

When asked if the method would be helpful for their current project (Q4), four participants from G1 answered "yes" (P5, P6, P8, 10), and one from G1

answered "no" (P4). The participant who answered "no" explained that he did not know all the patterns listed by the method. Furthermore, the participants from G1 stated that the method is a good checklist (P5, P8), the relationship between patterns should be highlighted (P8, P10), and that the method should be used multiple times during an iterative process (P8).

CAPI Provides a Sound Technology Selection. Finally, we asked the participants from G2 about the differences between the patterns selected using the CAPI method and the actual patterns used in their project (Q8). All five participants agreed that the patterns suggested by our CAPI method match the actually used patterns (P1, P2, P3, P7, P9). Complementary, two participants (P7, P9) stated that a few patterns were not selected. P9 concretized that the pattern *Infrastructure as Service* was not suggested by the CAPI method. For one project, our method suggested three more patterns (*Fault Injection*, *System-wide Resiliency*, *Log Aggregation*) that were not implemented yet but should be in the future (P3).

5 Discussion

In this section, we interpret the obtained results to answer our research questions.

5.1 Teams Have No Dedicated Structural Process to Select Technology for Their Projects (RQ1)

All ten participants from the two groups stated they do not have a dedicated, structured process for selecting technologies for their projects. However, two participants of G1 and four of G2 explained that they cannot select technology (completely) independently, as their clients specify (at least a partial) technology stack. We interpret this finding as the participants not needing to select (all) technology. Thus, they have no process. On the other hand, four members of G1 and three of G2 also stated that they relied on experienced employees (P4, P5, P8, P9, 10) or just tried technology until they found one that was suitable for a specified requirement (P3, P9). These answers indicate that if there are no external regulations, like a client's specifications, the actual process of technology selection relies on experienced employees. Although six of ten participants stated that they did not have a process for technology selection as their clients specified the technology, only three participants (P1, P2, P4) reported that they did not encounter any problems under this circumstance. As the clients do not fully specify the whole technology stack and often work with allow-lists, the companies can still choose a certain amount of technology, which again leads to problems. Even if the whole stack is specified, the actual problem of technology selection then shifts from the implementing company to the specifying client.

All in all, for RQ1, we extracted from our interview that there is often no process for selecting technology because the clients specified these technologies beforehand. Regardless of whether the client regulates the technology to use, the companies rely on their experience and reuse technology already known or use trial-and-error to determine the well-suited technology.

5.2 Selecting Technology Without a Process Can Lead to Time and Money-Consuming Trial-and-Error (RQ2)

Both groups reported that a trial-and-error-based process is very costly (P2, P9, P10). Furthermore, these costs may increase if the knowledge is spread over multiple persons as the knowledge needs to be collected (P10). Moreover, G1 reported that they tend to reuse already known technology instead of trying new ones, leading to missing out on new technology and trends (P5, P10). As Participant P10 reports, on the one hand, they miss out on new technology because they stick to the already known, and on the other hand, the process of knowledge collection is cost-intense. In our opinion, these two problems could be interconnected. As technology collection and discovery of new ones is a cost-intensive investment, this problem could automatically lead to sticking to the technology that the company's employees already know because, through this behavior, there is no need to invest money into a technology selection process. Additionally, G2 reports that if there is a new technology, it will only be introduced by employees with a certain self-motivation to encourage a specific technology (P9). We interpret this also as a symptom of the high costs of a technology selection process, so companies instead rely on self-motivated employees to reduce costs.

In conclusion, the answer to RQ2 is that if companies have no dedicated structured process for selecting needed technology, they tend to do trial-and-error or, as this is very costly, rely on self-motivated employees who bring this knowledge without extra costs. However, in both ways, either the companies have to pay for a costly trial-and-error process, or they need to allocate valuable, knowledgeable employees to the process, leading to a higher workload for these specific people.

5.3 CAPI, the Map for the Technology Labyrinth (RQ3)

We organize this research question in multiple blocks of key takeaways and summarize in the final block our conclusion for RQ3.

From a First Impression, the Method Supports Practitioners During the Technology-Choosing Process. Nine of the ten participants concluded that the CAPI method would be helpful as it closes the gap of the missing process of selecting technology for a project. Only participant P4 declared that the CAPI method would not help him as neither he nor his team were fully aware of all patterns that were suggested by the method. As reported in Sect. 5.2, the main issue with a missing process is that the needed technology is selected through a trial-and-error process, which leads to increased costs. Although the participants do not directly address this issue, they reported that the method is a good checklist (P4, P8), helps focus on the things that are needed (P3, P7), and helps by hinting the focus in the correct direction (P4, P5, P8). These findings imply that the CAPI method provides direction in the technology landscape. With a navigation tool, the technology-choosing process is less time-consuming and more structured than the previously mentioned trial-and-error process, leading implicitly to reduced cost. We conclude that the CAPI method hit the right

nerve through our approach of suggesting patterns instead of tools and is indeed helpful for the technology selection process.

Software Engineers Prefer to get Tool Suggestions in Addition to the Patterns. From G1, the reported potential improvements are that the method should also suggest tools that implement the patterns (P6, P10), the method should show the reasoning for a selected or not-selected pattern (P10), the method should also highlight the relationship between selected patterns (P8, P10), and the method should be rather used multiple times during an agile development process instead of once in the beginning (P10). Also, from G2, participants report that the capability of suggesting tools that implement the selected patterns would be a desirable improvement (P1, P2, P3, P7). Furthermore, like G1, highlighting the relationship between patterns (P9) and the usage of the method multiple times during an agile process (P3), G2 saw these improvements, too. Additionally to G1, G2 also mentioned that in addition to highlighting the relationship of the patterns, a prioritization of the patterns would be helpful to determine which patterns should be implemented first (P9). Surprisingly, six out of ten participants mentioned that the method should also suggest tools in addition to patterns. This finding seems to conflict with our thought that the strength of our approach was to suggest patterns instead of tools, as the landscape of tools is far broader than the available patterns. However, P1, P2, and P3 declared themselves as Software Engineers who develop software for their projects, so for them, the choice of tools is more important as they will directly use them. The other participants working on a more abstract layer have not mentioned the necessity of suggested tools besides P7. From this perspective, the result does not conflict with our approach. We conclude that the desired improvements for the CAPI methods are: i) for selected and non-selected patterns there should be a reasoning, ii) selected patterns should be prioritized, and their relationship should be visualized, iii) the method should be used multiple times during the development instead of just in the beginning, iv) additionally to the selected patterns, tools should be suggested that implement the patterns.

Regardless of the role in the project, the CAPI method is able to reproduce a productive environment. All G2 participants reported that the set of patterns resulting from applying the CAPI method roughly matches their actual implemented stack (P1, P2, P3, P7, P9). Additionally, one participant mentioned that the resulting set of suggested patterns contains patterns they have not used but want to implement in the future (P3). However, the CAPI method could not reproduce the whole stack of two participants because it missed some patterns (P7, P9). Nevertheless, these findings make us confident that our approach is a valuable solution to fill the gap if there is no dedicated process to select the needed technology in a project. Interestingly, both Software Engineers (P1, P2, P3) and Architects (P7, P9) stated that the results nearly match from their point of view. We interpret these as, whether a more tool-driven software engineer or an abstract architect, both groups were satisfied with our method on the baseline. From our interviews, we derive that the expected output of such a technology choice method is tools for software engineers. Nevertheless, they

do not reject the approach of suggesting patterns; instead, they are able to find their used tools in the patterns.

Finally, we Answer RQ3 as Follows. The reported problems in our interviews are high costs due to an unorganized trial-and-error process for determining suitable technology for their requirements. Our method is able to reproduce the participants' production environments, so the suggested patterns are trustworthy. Therefore, the method could lower the initially mentioned cost due to time savings, as the method suggests patterns that are trustworthy and implicitly press the user in a specific direction, which reduces the number of tools to be investigated. By following the concerns of the participants and suggesting tools that implement the patterns additionally, the cost reduction may be even higher. Equipped with the CAPI method as a map, both architects and software engineers can benefit from the structure it offers. As mentioned by the participants, this is not fully limited to greenfield projects; the method can be used multiple times in an agile process, making it also suitable for brownfield projects.

5.4 Threats to Validity

The main threat to the *external validity* of our study is the small sample size and the recruiting method for our subjects. We cannot be sure about the completeness of received answers (e.g., on reasons for the (non-)helpfulness of our method) and how well our results generalize to other settings. Still, we recruited participants in considerably different branches and roles and, for some questions, received very homogenous answers. Therefore, we are confident that our most important findings (regular lack of technology selection process and perceived helpfulness of the CAPI method) will also hold in other settings (although possibly to a different extent). The main threat to *internal validity* is that the first author individually performed the interviews and data extraction, increasing the threat of bias and misinterpretation. To mitigate this threat, all authors together discussed examples for data extraction and the whole interpretation of the obtained data. Moreover, for lack of a closely comparable alternative process, our industrial study focused on the usage of the CAPI method. In this way, we cannot (and do not) claim superiority over other methods or draw conclusions about which aspects of our method affect its perceived helpfulness and how. In any case, the widespread lack of methodological approaches to technology selection in industry highlights the importance of developing such.

6 Conclusion

This paper introduces the CAPI method, which suggests architectural patterns using a decision tree with diagnostic questions about the user's different requirements. We conducted qualitative interviews to investigate the need for such a method and its effectiveness. From these interviews, we conclude that the method hits the correct spot and is a possible solution for a missing process for selecting

technology in a project. Besides the positive feedback from our interview participants, we also got valuable improvements that could increase the effectiveness of our method. From these improvements, we derive our next steps, which consist of investigating relationships between our patterns, including a possible prioritization derived from these relationships, and developing a method to map tools onto patterns efficiently. In addition, a revaluation of the improved method would provide us with meaningful insight to determine the effectiveness of the changes.

Data Availability Statement. Our research bundle is available at Zenodo [3]. The online tool used during the interviews is available at https://capi.uniks.de. The tool's authentication is part of the research bundle, follow the readme for instructions.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Busch, A., Fuchss, D., Koziolek, A.: PerOpteryx: automated improvement of software architectures. In: IEEE International Conference on Software Architecture Companion, pp. 162–165 (2019)
2. Copei, S., Kosiol, J.: DevOps patterns: a rapid review. In: Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium, pp. 33–50 (2024)
3. Copei, S., Kosiol, J., Hohlfeld, O.: Research bundle for "The (C)omprehensive (A)rchitecture (P)attern (I)ntegration method: Navigating the sea of technology" March 2025, https://doi.org/10.5281/zenodo.15105538
4. Farshidi, S., Jansen, S.: A decision support system for pattern-driven software architecture. In: Software Architecture, pp. 68–81 (2020)
5. Haoues, M., Sellami, A., Ben-Abdallah, H., Cheikhi, L.: A guideline for software architecture selection based on ISO 25010 quality related characteristics. Int. J. Syst. Assur. Eng. Manag. **8**(2s), 886–909 (2017)
6. Kalantari, R., Lethbridge, T.C.: Unveiling Developers' mindset barriers to software modeling adoption. In: 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion, pp. 737–746 (2023)
7. Kassab, M., El-Boussaidi, G., Mili, H.: A quantitative evaluation of the impact of architectural patterns on quality requirements. In: Software Engineering Research, Management and Applications, vol. 377, pp. 173–184. Springer (2011)
8. Kassab, M., Mazzara, M., Lee, J.Y., Succi, G.: Software architectural patterns in practice: an empirical study. Innov. Syst. Softw. Eng. **14**(4), 263–271 (2018). https://doi.org/10.1007/s11334-018-0319-4
9. Khan, M.S., Khan, A.W., Khan, F., Khan, M.A., Whangbo, T.K.: Critical challenges to adopt DevOps culture in software organizations: a systematic review. IEEE Access **10**, 14339–14349 (2022)
10. Khattak, K.N., Qayyum, F., Naqvi, S.S.A., Mehmood, A., Kim, J.: A systematic framework for addressing critical challenges in adopting DevOps culture in software development: a PLS-SEM perspective. IEEE Access **11** (2023)
11. Koziolek, A., Koziolek, H., Reussner, R.H.: PerOpteryx: automated application of tactics in multi-objective software architecture optimization. In: 7th International Conference on the Quality of Software Architectures, pp. 33–42. ACM (2011)

12. Me, G., Procaccianti, G., Lago, P.: Challenges on the relationship between architectural patterns and quality attributes. In: IEEE International Conference on Software Architecture, pp. 141–144. IEEE Computer Society (2017)
13. Meijer, W., Trubiani, C., Aleti, A.: Experimental evaluation of architectural software performance design patterns in microservices. J. Syst. Softw. **218** (2024)
14. Razzaq, A.: A systematic review on software architectures for IoT systems and future direction to the adoption of microservices architecture. SN Computer Science **1**(6), 1–30 (2020). https://doi.org/10.1007/s42979-020-00359-w
15. Siamak Farshidi, Slinger Jansen, R.D.J., Brinkkemper, S.: A decision support system for software technology selection. J. Decis. Syst. **27**(sup1), 98–110 (2018)

ArchDTO Workshop

Requirements for a Digital Twin of Organizations for Advancing Cybersecurity Risk Management

Bennet Puthuparambil and Claudio Giovanoli(✉)

School of Business, University of Applied Sciences and Arts Northwestern Switzerland FHNW, Riggenbachstrasse 16, 4900 Olten, Switzerland
claudio.giovanoli@fhn.ch

Abstract. As organizations digitize, their exposure to cyber threats increases, while traditional cybersecurity tools struggle to provide real-time, adaptive protection. Digital Twins of an Organization (DTOs) offer a promising solution by enabling dynamic simulations of systems and processes, allowing for proactive threat detection and strategic response. In this paper we explore requirements for the integration of DTOs with advanced analytics to support cybersecurity leadership, particularly Chief Information Security Officers (CISOs).

Keywords: Digital Twin of an Organization (DTO) · Architecture · Information Security · Cybersecurity · Chief Information Security Officer (CISO) · Digital Transformation

1 Introduction

While the digitization of organizations has demonstrated advantages, these digitization aspirations have also created a wider target for cyberattacks (Kaplan et al. 2019). Existing cybersecurity tools often lack providing comprehensive, real-time insights required for proactive threat management and strategic decision-making. Moreover, they are constrained by their inability to effectively detect in-network threats, to manage sophisticated techniques such as credential harvesting and lateral movement, and by the high level of complexity, the occurrence of false positives, and the limitation of their proactive capabilities (Steingartner et al. 2021). Consequently, leaving organizations vulnerable to evolving attacks. This growing complexity highlights the inadequacy of traditional tools in providing a comprehensive defense.

At this stage, DTOs emerge as a promising solution. By offering dynamic, real-time simulations of organizational processes and systems, DTOs enable a more holistic approach to identifying vulnerabilities, anticipating threats, and strategizing responses. This shift from static to dynamic modeling not only addresses the current gaps in cybersecurity management but also aligns with the broader digital transformation efforts of modern enterprises. Though DTOs showed transformative potential in other domains, their application in cybersecurity remains limited and underdeveloped (Holmes et al.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 231–239, 2026.
https://doi.org/10.1007/978-3-032-04403-7_20

2021). According to Srivastava et al. (2024), DTOs can help to enhance cybersecurity through real-time monitoring and simulation.

This gap highlights the need for a solution that combines the real-time modeling capabilities of DTOs with advanced analysis systems to support risk identification, evaluation, and mitigation. As highlighted in the literature review, traditional cybersecurity tools struggle to provide a comprehensive, adaptive framework for addressing complex and rapidly evolving threats. DTOs, by offering dynamic modeling and real-time threat simulations, stand out as a promising solution for bridging this gap (Holmes et al. 2021). The literature further underscores the transformative potential of combining DTOs with AI-driven analytics and predictive modeling, which enhance their ability to detect vulnerabilities and enable preemptive actions (Coppolino et al. 2023; Dietz et al. 2022). This foundation establishes the relevance of a DTO-based approach to advancing organizational resilience and shaping future cybersecurity frameworks.

By leveraging the real-time modeling capabilities of DTOs integrated with advanced analytical systems, this research aims to bridge the gap in proactive threat management.

In this paper we answer the following research question based on the Awareness Phase of Design Science Research: *What are the necessary requirements to develop an architecture for a Digital Twin of an Organization with a focus strategic decision-making in information security.*

2 Background

A Digital Twin of an Organization (DTO), in some cases as well referred to as Organizational Digital Twin (ODT) can be defined as a comprehensive representation of an organization that integrates data from organizational assets, individuals, activities, and their interactions (Parmar et al. 2020). For the purpose of this research, the authors use the term Digital Twin of an Organization (DTO). As virtual representations of physical systems, Digital Twins (DTs) provide the unique ability to simulate, monitor, and respond to cybersecurity threats in real time, offering enhanced capabilities in protecting critical systems across various domains.

DTs enable continuous monitoring and threat detection by replicating the physical systems in a virtual environment. This replication allows cybersecurity professionals to identify vulnerabilities and anomalous activities without disrupting actual operations. For instance, Holmes et al. (2021) highlight that cyber digital twins (CDTs) allow security assessments and simulations of potential attack scenarios, enabling organizations to anticipate and mitigate threats effectively.

The World Economic Forum (WEF) has acknowledged cybercrime as the third-largest global economy (Muggah and Margolis, 2023). This is underscored by estimated annual cost of $10.5 trillion globally by 2025 (Ferreira et al. 2024). The increase in cyber-attacks can largely be attributed to existing system errors, including vulnerabilities, deficiencies, and misconfigurations in hardware, software, and computer networks (Aslan et al. 2023). The role of the CISOs therefore has evolved to encompass not just technical oversight but also strategic decision-making and external stakeholder engagement. CISOs are tasked with integrating cybersecurity into broader organizational objectives, ensuring that risk management aligns with business priorities (Hooper and McKissack, 2016).

Emerging technologies are rapidly reshaping the field of cybersecurity, offering new capabilities for addressing increasingly sophisticated threats while introducing novel challenges. DTs, which create virtual replicas of physical systems, are increasingly being used in cybersecurity for real-time monitoring, vulnerability assessment, and threat simulation. In IoT environments, DTs can simulate attack scenarios, enabling organizations to preemptively address vulnerabilities without compromising actual systems (Pirbhulal et al. 2022). Their application in healthcare and industrial systems demonstrates their potential to improve situational awareness and optimize incident response strategies. However, their effectiveness depends on the accuracy and timeliness of input data, which can be a challenge in dynamic and complex environments (Pirbhulal et al. 2022).

DTOs represent an extension of the DT paradigm, traditionally used for physical systems, into the domain of organizational operations and processes. A DTO provides a real-time, virtual replica of an organization's structures, processes, and dynamics, enabling continuous monitoring, optimization, and strategic decision-making (Lyytinen et al. 2023; Parmar et al. 2020). Furthermore, DTOs enhance process compliance and risk management by continuously monitoring processes and identifying deviations from standard protocols. Using tools such as action-oriented process mining, DTOs provide actionable insights and auto-mated interventions to ensure compliance and mitigate risks (Park and Van Der Aalst, 2021).

3 Research Method

This study follows the Awareness Phase of the Design Science Research (DSR) methodology (Hevner et al. 2004), which is well-established in the design of IT artefacts. The goal in this phase is to develop a clear understanding of the problem space and the requirements for a Digital Twin of Organizations (DTO) in cybersecurity risk management. To achieve this, we combined a systematic literature review with semi-structured expert interviews.

The literature review was conducted systematically to ensure comprehensiveness and reproducibility.

Search Strategy: We identified relevant literature using the following keywords: "CISO," "Challenges," "Risk," "Strategy," "Cybersecurity," and "Digital Twins." These were chosen based on their relevance to the intersection of digital twins, cybersecurity, and organizational leadership.

Databases Searched: The search was executed across three major academic databases: Springer Link, Web of Science Core Collection, and Scopus.

Inclusion and Exclusion Criteria: We limited our search to peer-reviewed journal articles and review articles to ensure quality and relevance.

The search targeted terms in titles, abstracts, and keywords. Articles outside the fields of cybersecurity, digital twins, or strategic IT management were excluded using a pre-defined set of exclusion keywords.

Screening and Selection: The initial search returned 564 articles. After removing duplicates and screening for relevance based on titles and abstracts, 96 articles remained. A full-text review was then performed, resulting in 71 articles deemed directly relevant to the research questions. Thematic content analysis was conducted on this final set,

following the guidelines of Denyer and Tranfield (2009) and Page et al. (2021) for systematic reviews.

Expert Interviews: To complement the literature and capture current challenges and requirements from the field, we conducted semi-structured qualitative interviews:

Sampling and Recruitment: We used purposive sampling to identify participants with extensive experience in cybersecurity leadership, focusing on Chief Information Security Officers (CISOs) and senior cybersecurity professionals. Participants were recruited from organizations in sectors with high cybersecurity requirements (e.g., finance, healthcare, IT services), primarily based in Switzerland and Germany.

Interview Procedure: A total of six interviews were conducted between February and March 2024. Interviews followed a semi-structured guide covering topics such as DTO requirements, data needs, user interactions, simulation capabilities, and adoption challenges. Each interview lasted 45–60 min and was conducted via secure video conferencing tools. With participants' consent, all interviews were audio-recorded and subsequently transcribed verbatim.

Data Analysis: Transcripts were analyzed using qualitative coding, supported by MAXQDA software. An inductive approach was used to identify recurring themes, with codes grouped into categories corresponding to functional requirements, data needs, user interaction, simulation, and challenges. Triangulation between interview insights and literature findings was performed to ensure robustness and comprehensive coverage of identified requirements.

Research Ethics: Informed consent was obtained from all interview participants. Data was anonymized during analysis to protect confidentiality.

4 Requirement Analysis

4.1 Functional Requirements

The expert interviews underscored the necessity for a DTO-based support system to act as an integrative layer across strategic, operational, and tactical cybersecurity domains. Rather than focusing solely on technical indicators, all experts emphasized the need for a system that enables CISOs to align strategic goals with operational implementation and monitor the progress of cybersecurity initiatives.

A central theme across interviews was the demand for a structured interface where strategic goals can be defined, broken down into action areas, and linked to measurable control implementation. According to the experts, traditional systems often lack transparency and provide a fragmented view when linking implemented controls and projects to overarching strategic objectives. The system must therefore provide a capability to visualize control maturity, assess project progress, and link both to defined strategic targets. All experts agreed that the system must support common frameworks. Modularity also emerged as a key requirement—the interviewees emphasized that the DTO system must be adaptable to the specific regulatory or organizational needs of different companies.

To enable meaningful monitoring, the system should provide a dashboard that visualizes the implementation and maturity of controls in relation to strategic objectives, map

cybersecurity projects and technical controls to strategic action fields, support role-based access for different cybersecurity stakeholders, and include a repository for documenting, updating, and reviewing strategic goals alongside their corresponding technical implementations.

4.2 Data Requirements and Availability

One expert emphasized that the system requires a well-structured data architecture capable of capturing the current organizational state and linking it with knowledge from past cases. This includes a central repository for storing real-time and historical data, a conceptual model for interpreting and learning from comparable scenarios, and a mechanism for generating actionable recommendations. He also highlighted the need for a simulation environment, ideally integrated with workflow representations, and stressed that the system must be able to map internal conditions to appropriate responses through a structured matching process.

Interview findings consistently emphasized that high-quality data is the foundation of a functional DTO. A second expert stated that while real-time telemetry has some value for operational visibility, it is of limited use for higher-level strategic planning. Instead, data must reflect the presence and maturity of organizational controls, such as the enforcement of security policies and the coverage of essential processes.

Rather than relying solely on status fields, the system must gather data on the scope and depth of control application. The third expert pointed out that coverage metrics are more informative than technical existence. Another expert added that data on review cycles, employee awareness training, and cross-departmental control ownership are critical indicators of organizational readiness. The system must integrate diverse internal data sources, such as asset management systems for infrastructure and endpoint visibility, identity and access management (IAM) systems to monitor user-level control effectiveness, ticketing and workflow tools for project and process tracking, and policy management systems to assess the currency and completeness of internal security directives.

The experts also called for traceability mechanisms that allow users to understand the origin and transformation of data within the DTO. The data within the DTO must not only be comprehensive and reliable, but also explainable. Its role is not to replicate technical telemetry but to support higher-order reflection on the effectiveness, coverage, and evolution of cybersecurity capabilities over time.

4.3 User Interaction, Monitoring and Compliance Support

All three interviewees agreed that the main benefit of a DTO is improving communication and monitoring of cybersecurity performance and compliance, highlighting the need for intuitive user interaction, especially for time-constrained CISOs.

One expert pointed out that the system must support mapping between internal controls and multiple regulatory frameworks. He further stressed the need for cross-referenced mappings so that organizations can maintain alignment across frameworks without redundant effort. The ability to generate audit-ready documentation was seen

as essential—especially revision-secure logs and automated evidence collection were repeatedly mentioned.

Interviewees further discussed the necessity of role-based access and tailored visibility. For example, while a CISO may require strategic summaries, operational staff should be able to access the status of specific controls or systems.

Another recurring theme was the value of integrating monitoring with alerting mechanisms. Rather than operating as a passive repository, the DTO should proactively flag deteriorations or missed review cycles. However, interviewees warned against over-automation. One expert emphasized highlights the importance of threat prioritization to reduce noise and ensure that the system communicates only what is contextually relevant.

Hence the DTO must be an accessible and effective interface for monitoring cybersecurity and compliance performance. It should enhance the CISO's situational aware-ness and provide targeted, role-specific insights. Furthermore, it must facilitate audit preparation and reduce manual documentation tasks—ultimately increasing confidence in the organization's security posture and regulatory readiness.

4.4 Simulation and Strategic Decision Support

One of the most frequently emphasized benefits of a DTO in the interviews was its ability to support strategic foresight through scenario simulation. A purely retrospective or real-time view is insufficient for modern cybersecurity management, according to the experts. Instead, systems must enable CISOs to assess the potential consequences of inaction, resource limitations, or delayed implementations through forward-looking simulations. Scenario-based simulations are seen as essential for prioritizing cybersecurity efforts, training teams, and evaluating decision-making, as they help assess the impact of attacks, model critical variables, and enhance strategic readiness beyond static compliance.

The simulations require to be both technically accurate and strategically relevant. One expert pointed out that many simulations fall short because they fail to link risk values to concrete business impacts, which limits their relevance for strategic decision-making. Therefore, simulations should incorporate contextual factors such as the business value of affected assets, the reputational consequences of failure, and the financial exposure resulting from prolonged vulnerabilities. Additionally, the experts emphasized that simulations should not be limited to a single run. Instead, the DTO should support rolling scenario planning, allowing CISOs to update risk forecasts as the organizational landscape evolves.

To be useful for decision-making, simulation results must be presented in a comprehensible and actionable format. Visual scenario summaries, projected risk curves, and scenario comparison tools were all mentioned as desirable features.

The simulation capabilities of the DTO therefore are central to its value proposition as a strategic support tool. They enable CISOs to move beyond reactive posture and into anticipatory planning—aligning cybersecurity efforts with broader organizational resilience objectives and executive-level expectations.

4.5 Concerns and Challenges

While the experts recognized the potential benefits of a DTO-based support system, they also raised several concerns that must be addressed to ensure practical applicability. These challenges relate to data integrity, adoption barriers, and the complexity of integrating strategic decision-making tools into dynamic organizational environments.

A recurring concern was the risk of inaccurate or incomplete data undermining the DTO's credibility. This highlights the need for robust data validation and traceability mechanisms. Without confidence in the underlying data, stakeholders are unlikely to rely on DTO-generated outputs. Another challenge is the operational effort required to implement and maintain the DTO. One expert pointed out that successful adoption requires the system to integrate smoothly with existing tools and organizational structures, noting that disconnected teams and systems often hinder effective implementation. Furthermore, all three experts raised the issue of complexity. While the DTO must support strategic alignment, there is a danger of overengineering. Another expert remarked that if a system is too complex or requires excessive configuration, it risks being ignored or poorly used, emphasizing that its value must clearly outweigh the effort needed to maintain it. Finally, concerns were raised about overreliance on automation. While simulations and control recommendations can support decision-making, experts emphasized that CISOs must retain control.

Conclusion and Research Gap

Recent advancements in cybersecurity and DT technologies have garnered significant attention for their transformative potential. Cybersecurity remains one of the most critical challenges faced by organizations today, with threats becoming increasingly complex and dynamic. Existing frameworks, such as NIST CSF and ISO/IEC 27001, provide strong foundations for risk management and compliance but often lack the agility to address sophisticated threats. This challenge is compounded by decision-making bottlenecks that stem from inadequate integration of technical tools with human-driven processes.

DTs, and specifically DTOs, offer a promising avenue to address some of these issues. As described by Caporuscio et al. (2020) and (Kerremans, 2017), DTOs provide dynamic digital representations of organizational systems, enabling continuous assessment, simulation, and optimization. These tools have already proven their value in domains like manufacturing and logistics, where they support operational efficiency and predictive maintenance. In the context of cybersecurity, DT use cases such as anomaly detection, predictive threat modeling, and real-time monitoring have demonstrated the potential in improving organizational resilience against cyberattacks. However, despite these advances, key challenges persist including the complexity of integrating systems, the prevalence of data privacy concerns, and the limited scalability of current solutions. The dynamic and unpredictable nature of cybersecurity threats often exceeds the capabilities of current DTO frameworks, which predominantly focus on static simulations or narrowly scoped operational applications. This creates a gap in leveraging DTOs as real-time, intelligent support systems for cybersecurity management. The lack of a comprehensive, integrated framework that incorporates socio-technical factors—balancing technical performance with human decision-making processes—further exacerbates this issue. Additionally, as Van Der Horn and Mahadevan, (2021) highlight, organizational

challenges like standardization, usability, and cultural barriers hinder broader adoption of DTOs in cybersecurity contexts.

References

Aslan, Ö., Aktuğ, S.S., Ozkan-Okay, M., Yilmaz, A.A., Akin, E.: A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. Electronics **12**(6), 1333 (2023). https://doi.org/10.3390/electronics12061333

Ceipek, R., Hautz, J., Mayer, M.C.J., Matzler, K.: Technological diversification: a systematic review of antecedents, outcomes and moderating effects. Int. J. Manag. Rev. **21**(4), 466–497 (2019). https://doi.org/10.1111/ijmr.12205

Coppolino, L., Nardone, R., Petruolo, A., Romano, L.: Building cyber-resilient smart grids with digital twins and data spaces. Appl. Sci. **13**(24), 13060 (2023). https://doi.org/10.3390/app132413060

Dagnino, G.B., Picone, P.M., Ferrigno, G.: Temporary competitive advantage: a state-of-the-art literature review and research directions. Int. J. Manag. Rev. **23**(1), 85–115 (2021). https://doi.org/10.1111/ijmr.12242

Dietz, M., Hageman, L., Von Hornung, C., Pernul, G.: Employing digital twins for security-by-design system testing. Proceedings of the 2022 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems, pp. 97–106 (2022). https://doi.org/10.1145/3510547.3517929

Denyer, D., Tranfield, D.: Producing a systematic review. In The Sage handbook of organizational research methods, pp. 671–689 (2009). Sage Publications Ltd.

Ferreira, C., et al.: Cybercrime: understanding the current state of literature and issues facing CISOs. IT Professional **26**(2), 83–89 (2024). https://doi.org/10.1109/MITP.2024.3375571

Hevner, A., March, S., Park, J., Ram, S.: Design science in information systems research. MIS Q. **28**(1), 75–105 (2004)

Holmes, D., Papathanasaki, M., Maglaras, L., Ferrag, M.A., Nepal, S., Janicke, H.: Digital twins and cyber security – solution or challenge? 2021 6th South-East Europe Design Automation, Computer Engineering, Computer Net-works and Social Media Conference (SEEDA-CECNSM), pp. 1–8 (2021). https://doi.org/10.1109/SEEDA-CECNSM53056.2021.9566277

Hooper, V., McKissack, J.: The emerging role of the CISO. Business Hori-zons **59**(6), 585–591 (2016). https://doi.org/10.1016/j.bushor.2016.07.004

Justice, C., Sample, C.: Future needs of the cybersecurity workforce. International Conference on Cyber Warfare and Security **17**(1), 81–91 (2022). https://doi.org/10.34190/iccws.17.1.33

Kaplan, J., Richter, W., Ware, D.: The Digital Enterprises' Cybersecurity. McKinsey. McKinsey & Company (2019)

Lyytinen, K., Weber, B., Becker, M.C., Pentland, B.T.: Digital twins of organization: implications for organization design. J. Organization Design **13**(3), 77–93 (2023). https://doi.org/10.1007/s41469-023-00151-z

Muggah, R., Margolis, M.: Why we need global rules to crack down on cybercrime. World Economic Forum (2023). https://www.weforum.org/stories/2023/01/global-rules-crack-down-cybercrime/

Page, M.J., et al.: The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. BMJ **71** (2021). https://doi.org/10.1136/bmj.n71

Park, G., Van Der Aalst, W.M.P.: Realizing a digital twin of an organiza-tion using action-oriented process mining. 2021 3rd International Conference on Process Mining (ICPM), pp. 104–111 (2021). https://doi.org/10.1109/ICPM53251.2021.9576846

Parmar, R., Leiponen, A., Thomas, L.D.W.: Building an organizational digital twin. Bus. Horiz. **63**(6), 725–736 (2020). https://doi.org/10.1016/j.bushor.2020.08.001

Pirbhulal, S., Abie, H., Shukla, A.: Towards a novel framework for reinforcing cybersecurity using digital twins in iot-based healthcare applications. 2022 IEEE 95th Vehicular Technology Conference: (VTC2022-Spring), pp. 1–5 (2022). https://doi.org/10.1109/VTC2022-Spring54318.2022.9860581

Srivastava, A., et al.: Digital twins serving cybersecurity: more than a model: cybersecurity as a future benefit of digital twins 2. IEEE Power Energ. Mag. **22**(1), 61–71 (2024). https://doi.org/10.1109/MPE.2023.3325196

Steingartner, W., Galinec, D., Kozina, A.: Threat defense: cyber deception approach and education for resilience in hybrid threats model. Symmetry **13**(4), 597 (2021). https://doi.org/10.3390/sym13040597

Stoiber, K., Matzler, K., Hautz, J.: Ambidextrous structures paving the way for disruptive business models: a conceptual framework. RMS **17**(4), 1439–1485 (2023). https://doi.org/10.1007/s11846-022-00589-7

Tranfield, D., Denyer, D., Smart, P.: Towards a methodology for developing evidence-informed management knowledge by means of systematic review. Br. J. Manag. **14**(3), 207–222 (2003). https://doi.org/10.1111/1467-8551.00375

AROSA Workshop

Comprehensive Exploration of Resilience and Adaptability in IoT-Based System-of-Systems: Bibliometric and Full-Text Literature Analysis

Aymen Abdelmoumen[1(✉)], Zakaria Benzadri[1], and Ismael Bouassida Rodriguez[2]

[1] University of Constantine 2 – Abdelhamid Mehri, LIRE Laboratory, Ali Mendjeli B.P. 67A, Constantine 25016, Algeria
aymen.abdelmoumen@univ-constantine2.dz

[2] ReDCAD, ENIS, University of Sfax, Sfax, Tunisia

Abstract. System-of-systems (SoS) are a class of large-scale distributed structures across domains such as defense, transportation, and healthcare, characterized by inherent complexity, operational autonomy, and emergent behavior, thereby posing substantial challenges in their design, specifically in maintaining resilience, adaptability, and dynamic reconfiguration. This paper presents an in-depth investigation into resilience and adaptability within the context of cyber-physical (CPS) and IoT-based SoS, combining qualitative narrative analyses with systematic bibliometric methods. The first phase consists of a critical synthesis of the most relevant and prominent literature, including a detailed, manually-done interpretation of full-text documentation examining recurring definitions, underlying concepts, adopted taxonomies. These insights inform the second phase, which applies a quantitative approach to a curated corpus of research, aiming to identify trends, influential works, and emerging themes in said field. The study covers 1,517 publications from 2010 to 2025, a time span selected to capture both the historical evolution and the current state of the art. Data was collected from four major scientific databases: IEEE Xplore, ACM Digital Library, CrossRef, and Semantic Scholar. Particular attention is given to resilience-oriented properties such as dynamic reconfiguration and adaptability as novel SoS engineering methodologies, in this regard, are discussed. Through this dual-phase analysis, the study identifies existing research gaps and challenges related to complexity management, interoperability, and the development of evolvable architectures in IoT-driven SoS. The findings contribute to a clearer conceptual framing of resilient IoT-SoS and provide future directions for research in self-adaptive and fault-tolerant SoS design.

Keywords: System of Systems (SoS) · Internet of Things (IoT) · Cyber-Physical Systems (CPS) · Bibliometric Analysis · Literature Review · Resilience · Adaptability · Dynamic Reconfiguration

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 243–258, 2026.
https://doi.org/10.1007/978-3-032-04403-7_21

1 Introduction

The convergence of the Internet-of-Things and system-of-systems paradigms has introduced a new class of highly dynamic, distributed, and interdependent architectures whose design and operation defy traditional engineering approaches. Unlike conventional monolithic systems, IoT-based SoS are characterized by a collection of independently developed and managed, yet naturally heterogenous, subsystems, that interact collaboratively to deliver emergent capabilities. These architectures operate in complex, unpredictable environments where runtime variability, partial failures, and evolving requirements are the norm rather than the exception. In such contexts, ensuring system-wide resilience and adaptability is not only a desirable property but a necessity. Robustness in IoT-SoS encompasses the capacity to tolerate faults, withstand cyber and physical disruptions, and maintain essential functions despite degraded conditions. Moreover, the absence of centralized authority, coupled with dynamic topologies and asynchronous behavior, demands the adoption of flexible, self-organizing mechanisms capable of real-time adaptation and coordinated reconfiguration.

Recent research underscores the importance of knowledge-driven architectures, real-time control adaptation, and systemic coordination strategies to address these challenges. Dynamic reconfiguration techniques have evolved to support automated, real-time adaptation of devices and control mechanisms, reducing configuration time and error rates while improving system reliability [1,2]. Self-healing mechanisms and fault-tolerant designs, such as anomaly detection and automated recovery, are employed to ensure dependability [3,4]. Real-time reconfiguration of control algorithms has been shown to preserve performance under network degradation or cyber intrusion [5].

In [6], the authors suggest the enhancement of resilience by enabling real-time validation, visualization, and assessment of adaptive strategies in complex CPS environments through the integration of digital twins and simulation models. To support large-scale coordination, the work of [7] has adopted a hierarchical management framework combining bottom-up monitoring, based on the fault detection and isolation (FDI) algorithm, with top-down reconfiguration. From a security perspective, model-based methodologies increasingly incorporate resilience into the design process through techniques such as Moving Target Defense (MTD) [8].

While IoT-based systems and CPS often exhibit SoS characteristics, they are not inherently SoS. The scope of this paper is narrowed to those IoT and CPS architectures that align with the operational definition of SoS, as this work investigates the literature regarding resilience and adaptability of IoT-based system-of-systems and cyber-physical systems in a dual-phase approach. Starting off from an in-depth interpretation of SoS theory, including recurring definitions and concepts, taxonomic classifications, and core challenges. Particular emphasis is placed on resilience-enabling properties such as dynamic reconfiguration and runtime adaptability, with the aim of identifying gaps and informing the second phase, which consists of a systematic analysis of the bibliography on a

meta-level, then a presentation of state of the art solutions targeting resilience and robustness.

The structure of the paper is organized into 5 sections. Section 1 introduces the research context, outlining the novelty of resilience in IoT-based SoS. Section 2 presents a condensed historical review of foundational literature on SoS, with a specific focus on inherent SoS-CPS properties threatening resilience. Section 3 provides a detailed bibliometric overview, subdivided into 3 parts: *(a)* the search strategy; *(b)* the design of search inputs including databases and queries; and *(c)* the outputs and results, which visualize analyses of relevance, keywords, and publication trends. Section 4 highlights some relevant state of the art engineering approaches and practical implementations for SoS. Finally, Sect. 5 concludes the study by a summary of findings and future directions for research.

2 Historical Overview of SoS Theory

This section outlines the historical evolution of system-of-systems literature, tracing key developments in definitions and theoretical foundations over time.

In Kotov's work [9], SoS are described as "large-scale concurrent and distributed systems the components of which are complex systems themselves," while Maier [10] introduces a foundational operational definition based on two key criteria: (a) *Operational independence*, where each constituent system (CS) can operate usefully on its own; and (b) *Managerial independence*, where each CS is managed independently while collaborating within the SoS. Maier critiques overly broad definitions, such as those that might classify any subsystem-based structure (e.g., a personal computer) as SoS, and instead stresses the need to distinguish "very large, complex, and distributed systems" through autonomy and emergent collaboration.

Earlier views like Shenhar's "array" systems [11] and Eisner's [12] centrally directed SoS integration efforts emphasize geographic distribution under centralized models. In contrast, Maier reorients SoS thinking toward collaborative, loosely coupled integration paradigms, and proposes a taxonomy based on control structure: Directed (central authority), Collaborative (voluntary coordination), and Virtual (emergent coordination without governance). Keating et al. [13] extends this by defining System-of-Systems Engineering (SoSE) as the discipline of designing and managing metasystems composed of autonomous and diverse systems with pluralistic goals and flexible boundaries.

Delaurentis et al. [14] shifts the discourse toward a dynamic paradigm where emergent behavior becomes central, framing SoS as evolving systems with porous boundaries. Whereas, Abbott [15] defines SoS as "open at the top," "open at the bottom," and "continually (but slowly) evolving," emphasizing adaptability and layered scalability. In [16], SoS traits are consolidated under the acronym ABCDE (Autonomy, Belonging, Connectivity, Diversity, Emergence), reinforcing themes such as independence, distribution, and interoperability. As for [17], the semantic integration challenges of SoS are highlighted, emphasizing autonomy, heterogeneity, and distribution as essential dimensions of their complexity.

Dahmann et al. [18] aligns strongly with Maier's taxonomy and definition yet introduce the "Acknowledged" SoS type, where constituent systems retain independent control but operate under an agreed coordination framework, blending features of Directed and Collaborative SoS.

International standards, namely; the INCOSE definition of SoS [19] and ISO/IEC/IEEE 21839:2019 [20] characterize SoS as systems whose constituents interact to provide unique capabilities not achievable independently.

Survey papers such as [21,22] trace definitional evolution across viewpoints (e.g. industrial, academic, structural) also referencing classification frameworks like [23] and Shenhar's uncertainty taxonomy [11]. They consistently frame SoS as "physically and functionally heterogeneous networks" with distinct lifecycles and goals. Figure 1 summarizes work redefining SoS, organized historically.

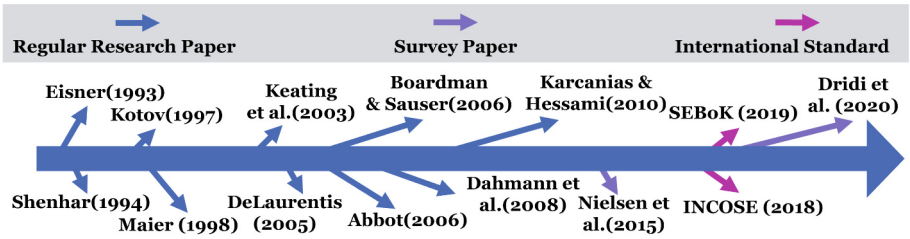

Fig. 1. Summary of Contemporary Evolution of SoS Literature.

Across all sources, the following foundational concepts of SoS are present in one wording or another:

- **Emergent behavior** is a defining trait suggesting that overall system behavior is not predictable from the individual constituent's behavior alone.
- **Autonomy and heterogeneity** requires that the architectures support asynchronism, loose coupling, diverse yet interoperable entities.
- **Lack of centralized control** calls for negotiation-based architectures and runtime governance strategies (such is the case with Acknowledged types, e.g.: concept of Service-Level Agreement [24]).

Among the recurring issues that stem from inherent SoS properties, there is:

- **Dynamic complexity and boundary ambiguity:** The evolving behavior of constituent systems within an SoS introduces dynamic complexity, which in turn makes it difficult to define and maintain clear boundaries between subsystems. As functionalities shift and interactions change over time, ambiguities in system roles and responsibilities emerge, leading to coordination challenges and misaligned objectives [25].
- **Dependability and Security:** Ensuring the dependability and security of SoS is complex due to the diversity of constituent systems and their varying levels of trustworthiness. Threats such as faults, errors, and failures can propagate through the system, affecting overall reliability [26].

– **Resilience and Adaptability:** In highly dynamic environments, such as SoS, the capacity for adaptive reconfiguration becomes essential to sustaining functionality. Zhang et al. [27] argues that resilience is not merely a functional trait but an emergent structural property that arises from the interrelation of its constituents. Evaluating resilience requires consideration of the system's overall structure, not just its reaction to discrete disruptions. This structural perspective inherently ties resilience to the system's ability to reorganize and adjust in response to evolving operational contexts.

3 Bibliometric Overview

This section presents a bibliometric analysis aimed at uncovering key trends, concepts, and influential works in the intersection of Internet-of-Things, system-of-systems, and cyber-physical systems, with particular attention to resilience-oriented attributes. Although this study does not formally follow a systematic literature review protocol, the approach follows semi-systematic review principles, including well-defined search queries, inclusion/exclusion criteria, deduplication, and metric-based analysis. The dual-phase strategy was chosen to balance depth (via narrative analysis) and breadth (via quantitative exploration).

3.1 Search Strategy

To construct a representative and thematically-focused set of data, a 2-stage search strategy was employed. The initial broad query targeted the general domain terms: "IoT", "SoS", "CPS", followed by a more specific query incorporating "resilience," "adaptability," and "dynamic reconfiguration" to isolate literature addressing system robustness and evolution. Results from both searches were merged, de-duplicated, and analyzed through numerous interpretations and visualizations (see Fig. 2).

3.2 Search Inputs

The design of the bibliometric search is structured around 5 core elements:

1. **Target:** The goal is to ensure ample data coverage by setting a threshold of publications that is deemed sufficient for meaningful statistical analysis. The data must be relevant (or adjacent) to the domains of computer science, system architecture, the Internet of Things, or Systems of Systems.
2. **Sources:** Data was collected from 4 major digital libraries and indexing platforms, namely; CrossRef, Semantic Scholar, ACM Digital Library, and IEEE Xplore. These sources were chosen for their coverage of high-quality, peer-reviewed publications, of which the latter 2 are particularly specialized in computer science and the relevant search topics.
3. **Query:** The search is performed via 2 sequential and complementary queries:

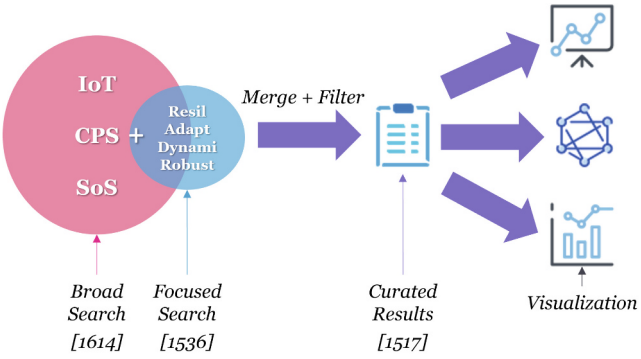

Fig. 2. Bibliometric Data Collection, Screening and Visualization.

(a) The first query aims for a broad investigation. The search string was crafted with the intent of including papers that discuss both IoT and SoS-adjacent works simultaneously, regardless of the variation in terminology, by including common abbreviations and full phrases. The boolean query used is semantically equivalent to the following: *"(IoT OR Internet-of-Things OR CPS OR Cyber-Physical Systems) AND (SoS OR System-of-Systems)"*.
(b) The second query is a much more focused investigation. The search string includes the aforementioned terms in addition to ones relating to robustness. Namely; *Query 1 AND ("Resilience" OR "Adaptability" OR "Dynamic Reconfiguration)"*.

4. **Filters:** To focus the search on scholarly contributions, results were limited to documents classified as research papers or conference papers, restricted to the range from 2010 to 2025, in order to express both contemporary relevance and hindsight.
5. **Metrics:** The relevance and influence of the retrieved publications were assessed using 3 key metrics:
   (a) *Citation Prominence:* is measured by the number of times a paper or term is cited, only partially reflecting visibility and influence.
   (b) *Term Prevalence:* is the number of times a contribution or term is mentioned across the dataset, not its scientific validity.
   (c) *Temporal Activity:* is calculated based on times mentioned in recent memory, implying short-term trends.

These indicators are used to identify patterns and prominence in literature discourse, rather than as direct proxies for research quality or domain relevance.

3.3 Search Outputs

Data was collected through automated means, incorporating a python script with the use of API calls for querying "CrossRef" and "Semantic Scholar" databases.

As for "ACM Digital Library" and "IEEE Xplore", their web interfaces allows for mass download of search results after applying the corresponding filters. The data retrieved and retained from the search, as shown in Table 1, excludes duplicated entries in the final set. Further curation was needed to apply the designated filters manually, resulting in a cull of 65 entries, making the final pool of bibliometric data exactly equal to 1517. Metric evaluation and visualizations were performed using python code (Check Data Availability Statement).

Table 1. Distribution of Results across Bibliometric Sources.

Source	IEEEXplore	Semantic Scholar	ACM DL	CrossRef	**Total**
Initial Set	968	171	352	91	**1582**
Duplicates	32	28	0	5	**65**
Final Set	936	143	352	86	**1517**

Relevance Figure 3 illustrates the frequency of "IoT", "CPS", "SoS", "adaptability" and "resilience" search topics along the past 15 years.

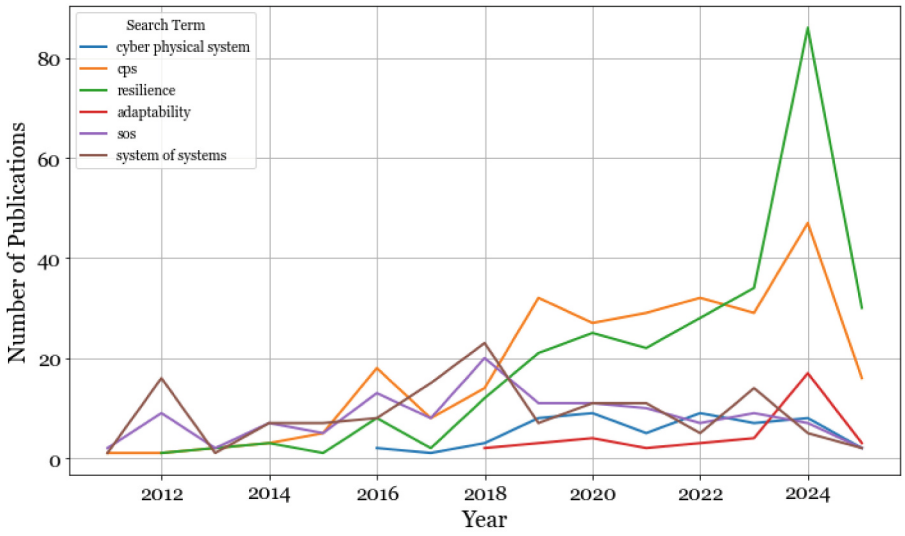

Fig. 3. Search Term Frequency Over the Years.

Over the span of 15 years, ranging from 2010 until 2025, a steady rise marks the period between 2014 and 2020, peaking in the short span of the 2 forthcoming years. This suggests that research strongly converged to IoT and SoS topics

during those years. However, it is followed by a sudden dip after the year 2022 for all search terms, which indicates that it may not be due to a loss of relevance, but can be rather attributed to the lengthy process of reviewing, indexing and publishing. On another note, IoT/Internet-of-Things and SoS/System-of-Systems search terms noticeably fluctuate together in a consistent manner, which suggests a potential correlation between said fields.

Contrary to previous trends, keywords "resilience", "adaptability", and "cps" have surged in popularity over the years, particularly in 2024, which may suggest a genuine rise in interest for these properties, whether they are directly associated with "IoT" and "SoS", or not.

The most cited works retrieved from the results are presented in Table 2.

Table 2. Top Cited Papers in IoT and SoS Research

Reference	Citations	Year	Source
[28]	2026	2017	IEEExplore
[29]	382	2019	Semantic Scholar
[30]	344	2016	Semantic Scholar
[31]	253	2019	IEEExplore
[32]	192	2021	IEEExplore
[33]	186	2020	Semantic Scholar
[34]	169	2017	Semantic Scholar

Notably, 4 of these publications are surveys or reviews focused on IoT-based platforms and enabling technologies, while 5 of them are primarily concerned with cybersecurity in IoT and CPS contexts. Despite the targeted search strategy, none of the top-cited articles explicitly address System-of-Systems (SoS) or resilience, indicating a disparity between citation impact and the intended thematic focus of SoS and/or resilience-oriented research.

Keywords Figure 4 illustrates the co-occurrence network of keywords extracted from the pool of bibliometric data, where term frequency is coded by node size and closeness to the center, whereas edge width reflects co-occurrence frequency. Measures were taken to maximize readability, such as the excluding rarely-mentioned words from the nodes set, and neglecting loosely-interconnected ones.

As expected, core structural terms such as "system", "IoT", "cyber-physical", and "model" appear centrally interconnected, highlighting their foundational role in the literature. Notably, "resilience" exhibits strong ties to "system" and "SoS" or "IoT" alike, affirming its thematic relevance.

However, it is surprising that there is no significant mention of "resilient"-adjacent terms such as "adaptability" and "dynamic reconfiguration" despite its explicit inclusion in the search query. This absence underscores a potential

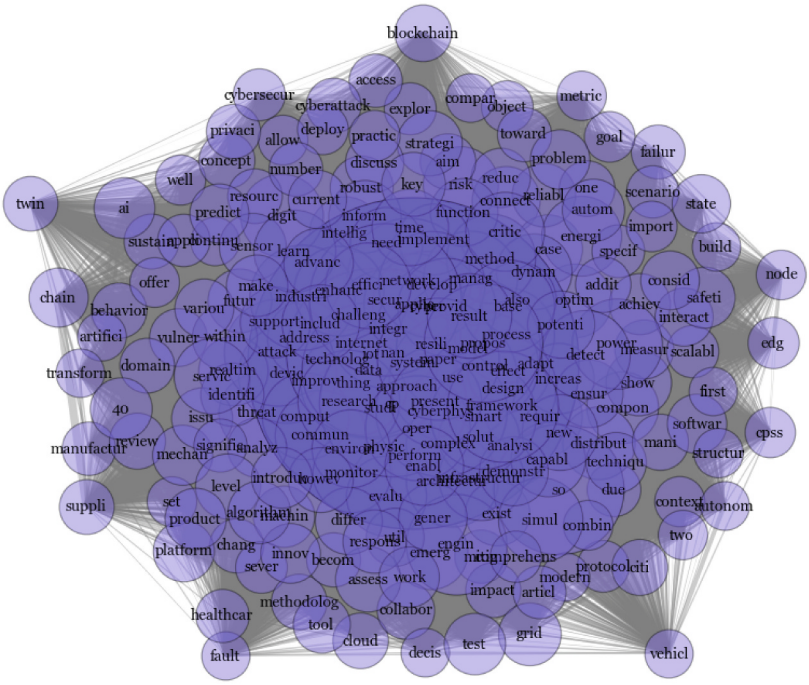

Fig. 4. Keyword Co-occurrence Network.

research gap in addressing runtime evolution and self-adaptive properties in IoT-based SoS. In contrast, terminology related to security is prominently featured and densely linked, suggesting that security concerns have received considerably more attention than architectural resilience or adaptability in current literature.

Figure 5 highlights the most recurring methodological keywords within the results, excluding terms related to IoT, SoS, resilience, and CPS.

The prominence of terms such as "model", "data", "network", and "framework" reflects the strong emphasis on the methodological and architectural dimension of such systems. Interestingly, the high frequency of "security"-related terms (as mentioned before), compared to the near absence of keywords like "adaptability" or "dynamic reconfiguration" suggests what the research is leaning towards, while adaptability and self-management remain relatively underrepresented in mainstream discourse, potentially hinting at a research gap.

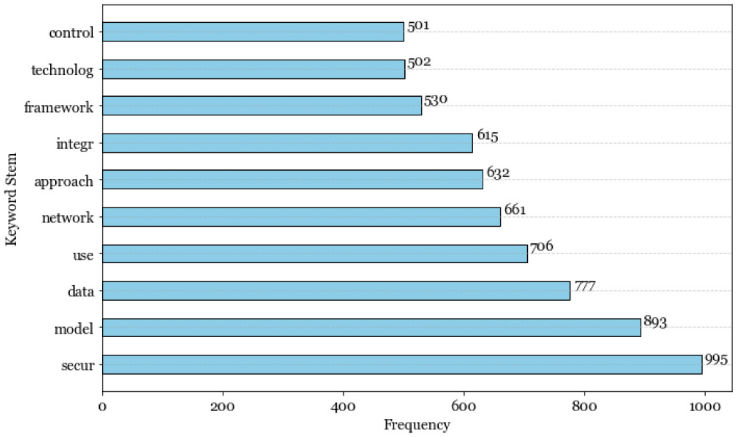

Fig. 5. Most Frequent Keywords.

4 Emphasis on Resilience, Adaptability, and Dynamic Reconfiguration

Resilience, adaptability, and dynamic reconfiguration are closely related yet conceptually distinct concepts essential for the dependable operation of SoS:

- **Resilience (or robustness)** is generally understood as a system's capacity to withstand disruptions, maintain a degraded but functional level of performance, and recover to a stable operational state. This involves both: **(a)** the ability to retain core functionality during extreme working conditions, be it traffic overload, unavailability of resources, or adverse events. **(b)** recovery, which pertains to regaining nominal performance over time [35].
- In contrast, **adaptability** refers to a system's inherent ability to respond to evolving conditions, or threats by modifying its structure, behavior, or operational logic, dynamically and independently. While resilience emphasizes survival and recovery, adaptability focuses on learning and evolution [36].
- **Dynamic reconfiguration**, on the other hand, is the "reaction" itself. It represents a concrete mechanism through-which adaptability is enacted, involving the system's capacity to alter its internal configuration or rewire its components at runtime.

These are all close yet fundamentally different properties. Together, they enable SoS to remain operational, well-performing, and efficient under uncertainty.

4.1 Resilience and Its Link to Interoperability

In this work [37], resilience is defined as a system's capacity to recover from disturbances while maintaining dynamic stability, emphasizing that interoper-

ability is not resilience per se, but a prerequisite, as in: inadequate interoperability increases the risk of cascading failures and obstructs recovery pathways at interface points. As for the work of [38], it explicitly notes that interoperability limitations (e.g., device heterogeneity, standard misalignment) undermine resilience implementation, especially for black-box or COTS components.

4.2 Adaptability and Dynamic Reconfiguration

Adaptability is inherently embedded within most resilience frameworks, manifesting through various structural and behavioral mechanisms. Uday et al. [37] conceptualizes adaptability in terms of service layer reallocation and functional reassignment, allowing systems to repurpose or redistribute tasks in response to disturbances. Whereas Ratasich et al. [38] expands on this by detailing dynamic reconfiguration techniques such as reinstantiation, rewiring, and migration, which are enabled through semantic models and real-time monitoring. Similarly, Chen et al. [39] emphasizes dynamic resource reallocation, strategy adaptation, and component integration as essential for ensuring responsive behavior.

Dynamic reconfiguration is typically presented as a runtime property, implemented through model-based feedback loops (e.g., SHSA, Kalman Filters), formal runtime verification (e.g., LTL/CTL monitors), or self-healing architectures leveraging agent-based detection and substitution.

4.3 Practical Techniques for Resilience in SoS

Across the reviewed papers, a variety of resilience-enabling techniques have been identified and classified into 3 main categories.

Traditional Risk-Based Methods: Failure Modes, Effects and Criticality Analysis (FMECA), Failure Modes and Effects Analysis (FMEA), Fault Trees, and Event Trees are commonly applied to evaluate resilience. While these techniques are widely used in reliability engineering and provide systematic ways to identify failure points and their consequences, their applicability in large-scale, distributed SoS is limited due to their poor scalability. Probabilistic models like Bayesian Networks, Markov Models, and Petri Nets offer more flexibility by modeling uncertainty and capturing the dynamic nature of failure propagation. These formal methods enable the quantification of risks and the probabilistic behavior of system states over time, which is particularly useful in assessing the resilience of evolving or loosely-coupled systems.

SoS-Specific Techniques: Functional redundancy, for instance, allows unmanned aerial vehicles (UAVs) or neighboring nodes to assume the tasks of failing components, thus ensuring continuity of service [37]. Human-in-the-loop decision-making is another critical mechanism, particularly in contexts where adaptability and creativity are needed in unforeseen crisis situations. Localized capacity mechanisms, such as fog computing nodes or transportation hubs, are

used to absorb failure loads locally, minimizing the risk of cascading failures across the system. Structural adaptation, such as Substitutable Heterogeneous System Architectures (SHSA), dynamically replaces components using variable relationship graphs to ensure system integrity under failure conditions [38]. In addition to these, recent contributions have highlighted the role of hierarchical monitoring and reconfiguration frameworks [7,40], real-time control adaptation mechanisms [5], and simulation-based techniques using digital twins [6] to support resilient behavior at runtime. Security-aware approaches such as Moving Target Defense (MTD) also introduce runtime variability in attack surfaces, enhancing cyber-resilience [8].

Optimization Methods: Evolutionary algorithms such as NSGA-II have been used to derive Pareto-optimal configurations that balance resilience requirements with resource constraints, providing system designers with multiple viable trade-offs [39]. Graph-theoretic approaches inspired by ecological metrics, as introduced in [41], help evaluate resilience structurally, during early-stage design.

5 Related Work

In addition to prior reviews ([21] and the more recent [22]) that were brought up in Sect. 2, there appears other relevant IoT-based SoS reviews (such is summarized in Table 3), notably:

- The work of [32] offers a detailed classification of methodologies, frameworks, platforms, and tools for engineering IoT systems viewed through the lens of SoS. This survey contributes practical guidance for developers by organizing 70 engineering products according to interoperability, scalability, smartness, and autonomy. However, their work does not include a bibliometric dimension, nor does it focus on resilience or adaptability as systemic properties.
- This bibliometric study [42] provides trend insight on recent IoT-based SoS work, using IEEE Xplore as the exclusive data source. The analysis centers on publication frequency, co-authorship networks, and term co-occurrence. Nevertheless, the study remains purely statistical and does not include a narrative or qualitative synthesis of resilience-related engineering concepts.

Table 3. Summary of Related Work

Reference	Year	Review Type	Scope	Data Source
[21]	2015	Narrative	SoS	/
[22]	2020	Narrative + Proposal	SoS, Model-Driven SoS	/
[32]	2021	Narrative	IoT as SoS	/
[42]	2023	Bibliometric	IoT-based SoS	IEEE Xplore
This Work	2025	Narrative + Bibliometric	IoT-based SoS + Resilience in IoT-SoS	IEEE Xplore, Semantic Scholar, CrossRef, ACM

By contrast, the present work complements these previous efforts through a dual-mode approach: combining bibliometric analysis with in-depth narrative synthesis. Our focus is distinct in targeting resilience and adaptability, which are pivotal yet underexplored properties in IoT-based and CPS-oriented SoS.

6 Threats to Validity

The potential threats to the validity of this study and the measures taken to mitigate them are:

1. **Selection Bias:** The exclusion of other large repositories such as Scopus and Web of Science, due to access limitations, may result in a partial view of the relevant literature. To mitigate this bias, the dataset was constructed from 4 major and complementary digital libraries that, together, ensure substantial coverage across computer science and systems engineering domains.
2. **Query Formulation Bias:** The search strategy relies on carefully designed boolean queries, which, while comprehensive, may still fail to capture papers using alternative terminologies or domain-specific language. To address this, we used a two-stage query formulation: one broad and inclusive, and one specifically targeted at resilience-related terms. Nevertheless, terminology variance remains a potential source of omission.
3. **Temporal Bias:** Bibliometric trends were analyzed over a time range extending to 2025. However, recent works (particularly from 2023–2025) may be underrepresented due to indexing delays, resulting in an apparent drop in publication frequency. This temporal lag may affect the interpretation of recent trends, which is a limitation acknowledged in the analysis.
4. **Metric Interpretation Limitations:** The study uses citation count (impact), term frequency, and recency as indicators of relevance. These metrics, while useful for detecting patterns and influence, do not always correlate with conceptual significance. Highly cited works may be peripheral to the core topic, and less cited papers may offer critical insights. For which reason, we complemented the analysis with full-text review and narrative synthesis of a selected few papers to ensure thematic alignment.
5. **Generalizability:** The scope of this study is limited to IoT-based and CPS-driven systems-of-systems. As such, generalizations to other SoS domains (e.g. socio-technical SoS) should be made cautiously.

7 Conclusion

The study offers a balanced investigation into resilience and adaptability in IoT-based SoS. However, the bibliometric findings may be inconclusive, as the narrow range of data sources, reliance on quantitative indexes like citation prominence, and absence of a rigorous systematic review methodology constrain the generalizability of observed patterns. Nevertheless, the current interpretations indicate

that while SoS principles are well established, their integration with resilience-oriented properties remains underexplored, particularly given that emergent behavior is under-represented in most state of the art models. The data also reveal a notable imbalance: terms related to security and connectivity dominate, whereas resilience-specific concepts like "adaptability" and "dynamic reconfiguration" are comparatively scarce despite their importance. This underscores the need for interoperable, scalable mechanisms and context-aware implementations that can manage complexity and support adaptive behavior under uncertainty, such as toolchains and runtime infrastructures. Methodologically, broader data coverage, formal SLR protocols, and expert validation are recommended to triangulate bibliometric trends with practitioner priorities.

Acknowledgments. This work was partially supported by the LABEX-TA project MeFoGL: *"Méthodes Formelles pour le Génie Logiciel"*.

Data Availability Statement. The datasets generated and analyzed during the current study are available on the following link [https://github.com/MuTheMule/ECSA2025_paper108]. Alternatively, all bibliometric data used in this study were collected from public databases (IEEEXplore, ACM Digital Library, CrossRef, Semantic Scholar).

References

1. Banerjee, A., Choppella, V.: A knowledge-driven approach for dynamic reconfiguration of control design in internet of things and cyber-physical systems. IEEE Internet Things J. **12**, 5615–5641 (2025)
2. Alhozaimy, S., Menascé, D., Albanese, M.: Resilience and performance quantification of dynamic reconfiguration. Futur. Gener. Comput. Syst. **160**, 120–130 (2024)
3. Ratasich, D., Khalid, F., Geissler, F., Grosu, R., Shafique, M., Bartocci, E.: A roadmap toward the resilient internet of things for cyber-physical systems. IEEE Access **7**, 13260–13283 (2018)
4. Bagula, A., Ajayi, O., Maluleke, H.: Cyber physical systems dependability using CPS-IOT monitoring. Sensors **21**(8), 2761 (2021)
5. Kim, S., Lee, S., Park, K.: Real-time controller reconfiguration for delay-resilient cyber-physical systems. IEEE Access **10**, 101220–101228 (2022)
6. Epiphaniou, G., Hammoudeh, M., Yuan, H., Maple, C., Ani, U.: Digital twins in cyber effects modelling of IoT/CPS points of low resilience. Simul. Model. Pract. Theory **125**, 102744 (2023)
7. Jiang, J., Lakhal, O., Merzouki, R.: A Resilience implementation framework of system-of-systems based on hypergraph model. In: 2022 17th Annual System of Systems Engineering Conference (SoSE), pp. 487–492 (2022)
8. Casola, V., De Benedictis, A., Mazzocca, C., Montanari, R.: Designing secure and resilient cyber-physical systems: a model-based moving target defense approach. IEEE Trans. Emerg. Top. Comput. **12**, 631–642 (2024)
9. Kotov, V.: Systems of systems as communicating structures. Object-oriented technology and computing systems re-engineering, pp. 141–154 (1997)
10. Maier, M.W.: Architecting principles for systems-of-systems. Syst. Eng. **1**(4), 267–284 (1998)

11. Shenhar, A.J.: A new systems engineering taxonomy. INCOSE Int. Symp. **5**, 723–732 (1995)
12. Eisner, H., McMillan, R., Marciniak, J., Pragluski, W.: RCASSE: rapid computer-aided system of systems (S2) engineering. INCOSE Int. Symp. **3**, 267–273 (1993)
13. Keating, C.B., et al.: System of systems engineering. Eng. Manag. J. **15**(3), 36–45 (2003)
14. DeLaurentis, D.: Understanding transportation as a system-of-systems design problem. In: 43rd AIAA Aerospace Sciences Meeting and Exhibit. AIAA (2005)
15. Abbott, R.: Open at the top; open at the bottom; and continually (but slowly) evolving. In: System of Systems Engineering, p. 6. IEEE (2006)
16. Boardman, J., Sauser, B.: System of Systems - the meaning of of. In: IEEE/SMC International Conference on System of Systems Engineering, p. 6. IEEE (2006)
17. Karcanias, N., Hessami, A.G.: Complexity and the notion of system of systems: part (II): defining the notion of system of systems. In: 2010 World Automation Congress, pp. 1–7. IEEE (2010)
18. Dahmann, J.S., Baldwin, K.J.: Understanding the current state of US defense systems of systems and the implications for systems engineering. In: 2nd Annual IEEE Systems Conference, pp. 1–7. IEEE (2008)
19. INCOSE: Systems of Systems Primer, Version 1.0. International Council on Systems Engineering (2018)
20. ISO/IEC/IEEE: ISO/IEC/IEEE 21839:2019: Systems and software engineering—System of systems (SoS) considerations in life cycle stages of a system. ISO, Geneva (2019)
21. Nielsen, C.B., Larsen, P.G., Fitzgerald, J., Woodcock, J., Peleska, J.: Systems of systems engineering: Basic concepts, model-based techniques, and research directions. ACM Comput. Surv. **48**(2), 18:1-18:41 (2015)
22. Dridi, C.E., Benzadri, Z., Belala, F.: System of systems modelling: recent work review and a path forward. In: 2020 International Conference on Advanced Aspects of Software Engineering (ICAASE), pp. 1–8. IEEE (2020)
23. DeLaurentis, D.A.: A taxonomy-based perspective for systems of systems design methods. In: IEEE International Conference on Systems, Man and Cybernetics, vol. 1, pp. 86–91. IEEE (2005)
24. Henshaw, M., et al.: The Systems of Systems Engineering Strategic Research Agenda (2013)
25. Watson, M., Anway, R., McKinney, D., Rosser, L.A., MacCarthy, J.: Appreciative methods applied to the assessment of complex systems. INCOSE Int. Symp. **29**, 448–477 (2019)
26. Ceccarelli, A., Bondavalli, A., Froemel, B., Hoeftberger, O., Kopetz, H.: Basic concepts on systems of systems. In: Bondavalli, A., Bouchenak, S., Kopetz, H. (eds.) Cyber-Physical Systems of Systems. LNCS, vol. 10099, pp. 1–23. Springer, Cham (2016)
27. Zhang, H., Matsubara, Y.: Archimate-based system of systems resilience evaluation approach. Systems **13**(5), 327 (2025)
28. Lin, J., Yu, W., Zhang, N., Yang, X., Zhang, H., Zhao, W.: A survey on internet of things: architecture, enabling technologies, security and privacy, and applications. IEEE Internet Things J. **4**(5), 1125–1142 (2017). https://doi.org/10.1109/JIOT.2017.2683200
29. Huang, J., Kong, L., Chen, G., Wu, M.-Y., Liu, X., Zeng, P.: Towards secure industrial IoT: blockchain system with credit-based consensus mechanism. IEEE Trans. Industr. Inf. **15**(6), 3680–3689 (2019). https://doi.org/10.1109/TII.2019.2903342

30. Sajid, A., Abbas, H., Saleem, K.: Cloud-assisted IoT-based SCADA systems security: a review of the state of the art and future challenges. IEEE Access **4**, 1375–1384 (2016). https://doi.org/10.1109/ACCESS.2016.2549047
31. Makhdoom, I., Abolhasan, M., Lipman, J., Liu, R.P., Ni, W.: Anatomy of threats to the Internet of Things. IEEE Commun. Surv. Tutor. **21**(2), 1636–1675 (2019). https://doi.org/10.1109/COMST.2018.2874978
32. Fortino, G., Savaglio, C., Spezzano, G., Zhou, M.: Internet of things as system of systems: a review of methodologies, frameworks, platforms, and tools. IEEE Trans. Syst. Man Cybern. Syst. **51**(1), 223–236 (2021). https://doi.org/10.1109/TSMC.2020.3042898
33. Farivar, F., Haghighi, M.S., Jolfaei, A., Alazab, M.: Artificial intelligence for detection, estimation, and compensation of malicious attacks in nonlinear cyber-physical systems and industrial IoT. IEEE Trans. Industr. Inf. **16**(4), 2716–2725 (2020). https://doi.org/10.1109/TII.2019.2956474
34. Kim, J.H.: A review of cyber-physical system research relevant to the emerging IT trends: industry 4.0, IoT, big data, and cloud computing. J. Indus. Integr. Manage. **2**(3), 1750011 (2017). https://doi.org/10.1142/S2424862217500117
35. ISO: ISO 22300:2021: Security and Resilience—Vocabulary. International Organization for Standardization, Geneva (2021)
36. Hosseini, S., Barker, K., Ramirez-Marquez, J.E.: A review of definitions and measures of system resilience. Reliabil. Eng. Syst. Saf. **145**, 47–61 (2016)
37. Uday, P., Marais, K.: Designing resilient systems-of-systems: a survey of metrics, methods, and challenges. Syst. Eng. **18**(5), 491–510 (2015)
38. Ratasich, D., Khalid, F., Geissler, F., Grosu, R., Shafique, M., Bartocci, E.: A roadmap toward the resilient internet of things for cyber-physical systems. IEEE Access **7**, 13260–13283 (2019)
39. Chen, Z., Hong, D., Cui, W., Xue, W., Wang, Y., Zhong, J.: Resilience evaluation and optimal design for weapon system of systems with dynamic reconfiguration. Reliab. Eng. Syst. Saf. **237**, 109409 (2023)
40. Colabianchi, S., Costantino, F., Gravio, G.D., Nonino, F., Patriarca, R.: Discussing resilience in the context of cyber physical systems. Comput. Indus. Eng. **160**, 107534 (2021). https://doi.org/10.1016/j.cie.2021.107534
41. Chatterjee, A., Helbig, C., Malak, R., Layton, A.: A comparison of graph-theoretic approaches for resilient system of systems design. ASME J. Comput. Inf. Sci. Eng. **23**(3), 030906 (2023)
42. Abdelmoumen, A., Benzadri Z., Bouassida Rodriguez I.: Reviewing recent literature on IoT-based system-of-systems: a bibliometric analysis. TACC. 120–137 (2023)

Recent Literature Review on Modeling Explainable Medical Cyber-Physical Systems: A Bibliometric Analysis

Benina Feryel[1(✉)], Zakaria Benzadri[2], Faiza Belala[2], and Ahmed Hadj Kacem[1]

[1] University of Sfax, ReDCAD Laboratory, Sfax, Tunisia
{feriel.benina,ahmed.hadjkacem}@fsegs.usf.tn
[2] University of Constantine 2, LIRE Laboratory, Constantine, Algeria
{zakaria.benzadri,faiza.belala}@univ-constantine2.dz

Abstract. This paper provides a bibliometric analysis study of recent developments in Medical Cyber-Physical Systems (MCPS), with a particular emphasis on explainable artificial intelligence (XAI) and formal modeling approaches. MCPS are crucial for advancing healthcare by seamlessly integrating physical and computational components to enhance patient care. However, ensuring that these systems are transparent and interpretable is vital, especially as they increasingly rely on AI-driven decision-making. Through a comprehensive bibliometric analysis, this study identifies publication trends, leading authors, prominent institutions, and emerging themes within the field of explainable MCPS. Additionally, we examine how formal modeling languages like SysML and AADL contribute to strengthening the reliability and interpretability of MCPS. The paper concludes by identifying key directions for future research, aiming to foster deeper insights and further innovation in this transformative area.

Keywords: Medical Cyber-Physical Systems · Explainable AI · SysML · AADL · Model-Based Systems Engineering · Bibliometric Analysis

1 Introduction

Medical Cyber-Physical Systems (MCPS) represent a new generation of intelligent healthcare infrastructures that integrate physical medical devices (e.g., sensors, actuators) with computational and communication capabilities to improve clinical care and operational efficiency [1]. These systems support real-time monitoring, data-driven diagnosis, and autonomous therapeutic actions, making them essential for the advancement of personalized medicine and intensive care [2].

However, the growing reliance on Artificial Intelligence (AI) within MCPS introduces critical challenges related to system reliability, safety, and most importantly, transparency [3,4]. In high-stakes medical settings, AI-driven decisions must be not only accurate but also understandable by healthcare professionals.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 259–272, 2026.
https://doi.org/10.1007/978-3-032-04403-7_22

This requirement has led to the emergence of Explainable Artificial Intelligence (XAI), which seeks to make AI behavior transparent, traceable, and justifiable [5–7].

In parallel, model-based engineering approaches using languages such as SysML and AADL are gaining traction for their ability to manage complexity and ensure traceability throughout the system lifecycle [8]. While both XAI and formal modeling play critical roles in the development of trustworthy MCPS, there remains a lack of comprehensive reviews that bridge these two dimensions.

This article aims to fill this gap by conducting a bibliometric analysis concerning the research on Explainable Medical Cyber-Physical Systems (ExMCPS). By combining quantitative mapping with qualitative interpretation, we seek to identify key trends, leading contributors, and methodological priorities in the field.

Our investigation is guided by the following research questions:

- **RQ1: Evolution of Research** – How has research on ExMCPS evolved in terms of scientific output and thematic focus from 2015 to 2025?
- **RQ2: Key Contributors** – Who are the most influential authors and institutions in the field?
- **RQ3: Geographical Distribution** – Which countries are most active in ExMCPS research?
- **RQ4: Conceptual Landscape** – What are the most prominent concepts and trends in the literature?

2 Key Concepts

To better understand the scope and foundations of this research, it is essential to clarify the core concepts that underpin the development of explainable medical cyber-physical systems.

2.1 Medical Cyber-Physical Systems (MCPS)

Medical Cyber-Physical Systems (MCPS) represent a transformative evolution in healthcare by tightly integrating physical medical devices (e.g., wearable sensors, implantable monitors) with computational intelligence and communication networks [1,9]. These systems establish a continuous feedback loop between the patient's physiological state and therapeutic responses, enabling personalized, real-time interventions [2,10]. An illustrative example of a Medical Cyber-Physical System (MCPS) is the **Continuous Glucose Monitoring System (CGMS)**. This system combines both physical and cyber components to manage diabetes in real-time.

- **Physical Layer (Hardware):** A *subcutaneous glucose sensor* continuously monitors the patient's glucose level. The data is transmitted to an *insulin pump*, which acts as an actuator to deliver insulin.

– **Cyber Layer (Software):** Integrated *AI algorithms and decision-making applications* analyze glucose trends, *predict hypo- or hyperglycemic episodes*, and autonomously recommend or trigger insulin delivery actions [11,12].

This seamless integration improves treatment outcomes, reduces manual intervention, and is particularly effective for managing **Type 1 Diabetes** [13].

Beyond CGMS, MCPS applications also extend to *cardiac monitoring*, *intelligent intensive care units (ICUs)*, and *surgical robotics* [14,15], confirming their critical role in the evolution of smart healthcare (Fig. 1).

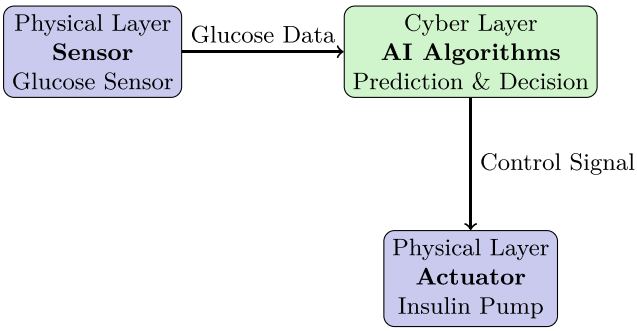

Fig. 1. Simplified architecture of CGMS as a Medical Cyber-Physical System

2.2 Explainable AI in Healthcare

AI Incorporation into healthcare decision-making amplifies the need for transparency and accountability. Explainable AI (XAI) aims to render AI models interpretable, helping clinicians understand the rationale behind automated decisions [3,5,6]. In high-stakes settings such as diagnostics or emergency care, black-box models raise regulatory and ethical concerns, making explainability imperative [4,16].

For example, in CGMS applications enhanced by AI, explainability tools such as SHAP and LIME can illustrate how features (e.g., glucose trend, meal intake) influence insulin recommendations. This improves clinicians' trust and allows them to validate AI outputs, especially in edge cases [17,18].

XAI also facilitates compliance with medical regulations (e.g., GDPR, FDA) and supports informed consent by clarifying algorithmic logic to patients and stakeholders.

2.3 Model-Based Systems Engineering (MBSE)

Model-Based Systems Engineering (MBSE) provides a structured methodology to manage the complexity of Medical Cyber-Physical Systems (MCPS) through

the use of formal and semi-formal modeling techniques. MBSE supports a systematic process from requirements engineering to system design, verification, and validation [19].

Modeling languages such as SysML (Systems Modeling Language) and AADL (Architecture Analysis and Design Language) are central to MBSE. SysML enables the specification of functional and behavioral aspects of systems through diagrams such as requirement, activity, state machine, and block definition diagrams [20]. This makes it suitable for modeling high-level interactions between the CGMS components and patient use scenarios. In contrast, AADL focuses on modeling and analyzing the non-functional aspects of embedded systems, particularly in terms of timing, resource usage, and fault tolerance [8].

In the context of CGMS, SysML can represent system functionalities like, glucose sensing, insulin dosage calculation, and patient alerts through behavioral and structural diagrams [21]. AADL complements this by providing precise architectural models that enable early validation of safety, real-time constraints, and reliability through schedulability and error propagation analysis [22].

MBSE toolchains also play a crucial role. Tools such as SysML modeling, allow integration with requirement traceability features. OSATE, the Open Source AADL Tool Environment, is specifically designed to support AADL-based analysis of real-time embedded systems [23].

Finally, MBSE fosters interoperability and shared understanding between engineering teams, healthcare professionals, and regulatory agencies by making system assumptions, operational constraints, and design decisions explicit and traceable [24]. This is particularly crucial in medical systems where safety, accountability, and certification are paramount [25].

3 Data Analysis Process

The data analysis process is conducted using a rigorous and structured methodology to ensure the reliability and relevance of the findings. As illustrated in Fig. 2, this process is divided into two main phases: the *Designing Research* phase and the *Realizing Research* phase. Each phase comprises several key steps, from the formulation of research questions to the final analysis of the collected data. The components of this process are detailed in the following subsections.

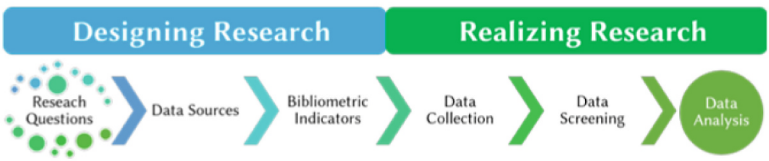

Fig. 2. Structured phases of the bibliometric analysis process, adapted from [26].

3.1 Data Source Selection

This study uses Scopus, a leading bibliographic database recognized for its wide coverage of peer-reviewed literature across engineering, computer science, and healthcare [27]. Scopus is chosen for its rich metadata (keywords, affiliations, citations) and compatibility with bibliometric tools like `Bibliometrix` and Python. Compared to other databases such as Web of Science or IEEE Xplore, Scopus provides broader and more consistent coverage relevant to ExMCPS.

3.2 Search Strategy and Data Processing

Following bibliometric principles [28], a targeted search is conducted in Scopus using the Boolean query: ("modeling" OR "formal method" OR "SysML" OR "AADL*") AND ("medical cyber-physical systems" OR "MCPS" OR "health*") AND ("explainable AI" OR "XAI" OR "interpretable AI")**.

The search focused on peer-reviewed journal articles and conference papers from 2015 to 2025, excluding non-academic sources and Grey literature. Only English-language and thematically relevant publications addressing modeling, explainability, or MCPS in healthcare were retained.

The final data set is exported in formats compatible with `Bibliometrix` (R), VOSviewer, and Python for cleaning, standardization, and visualization.

3.3 Integrated Analysis Methodology

To ensure a robust bibliometric analysis, this study employs a combined R Studio and Python framework. R Studio, via bibliometrix and biblioshiny, supports descriptive and visual tasks such as, publication trends, keyword networks, and productivity maps [12] Python complements this with advanced preprocessing deduplication, affiliation cleaning, and thematic clustering using NLP techniques [23]. This integration balances statistical rigor with semantic depth in exploring the research on explainable medical cyber-physical systems.

4 Findings

During this bibliometric analysis study, our focus is on a selected set of references related to explainable Cyber-Physical Systems (CPS) in the medical domain, in order to address the research questions previously outlined (RQ1–RQ4). This interpretation section is essential for contextualizing the results derived from Scopus data, which were processed and analyzed using both R (via bibliometrix/biblioshiny) and Python. By linking the quantitative patterns with qualitative insights, we aim to extract meaningful trends, identify research gaps, and provide informed answers to the core questions guiding this study.

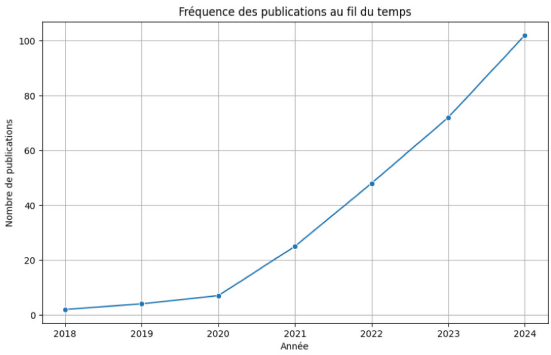

Fig. 3. Publication trends on ExMCPS (2018–2024)

4.1 Publication Trends

This subsection analyzes the evolution of scientific output on explainable medical cyber-physical systems (ExMCPS) from 2015 to 2025. The bibliometric data indicate a steady increase in publication volume, particularly from 2018 onward. As shown in Fig. 3, the number of publications rose from approximately 100 in 2018 to 177 in 2024, reflecting sustained research interest. This upward trend likely stems from increased funding, international collaboration, and regulatory interest in system transparency. Minor fluctuations, such as the dip in 2020 and spike in 2022, may correspond to the COVID-19 impact and the launch of targeted research initiatives, respectively.

4.2 Thematic Trends

Thematic trend analysis (Fig. 4) from 2018 to 2025 highlights the rising prominence of the term "explainable", reflecting growing interest in AI transparency within medical contexts. Persistent keywords like, "learning", "AI", and "intelligence" underscore the field's AI-centric nature, while the steady presence of "model" and "prediction" confirms the ongoing focus on predictive modeling. The recent emergence of "deep" signals increasing reliance on deep learning. This temporal evolution sheds light on current dynamics and suggests future directions for Ex-MCPS research.

4.3 Key Contributors and Collaboration Networks

This section highlights the main contributors to the research landscape of explainable medical cyber-physical systems (Ex-MCPS), both at the individual and institutional levels. The identification of these contributors is based on a bibliometric analysis of the Scopus dataset, specifically filtering authors with a minimum threshold of five publications related to Ex-MCPS during the period

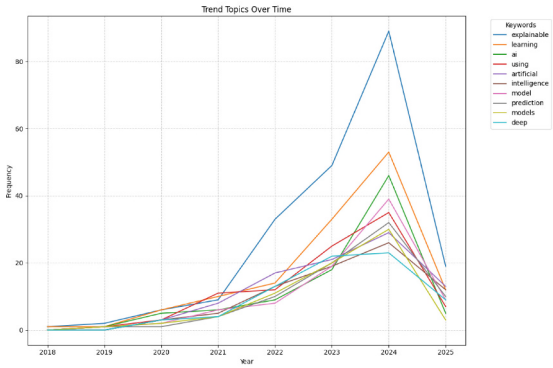

Fig. 4. Trend topics over time

2015–2025. This criterion ensures that the analysis focuses on consistently active researchers whose work has significantly shaped the field.

The examination of the most prolific authors (Fig. 5) reveals that **Andreas Holzinger** and **Majid Ali Khan** emerge as leading figures, frequently cited and highly productive. Holzinger's work is foundational in bridging the gap between explainable artificial intelligence (XAI) and biomedical informatics, notably through interdisciplinary research combining machine learning with human-centric interpretation in medical decision-making contexts [16,29]. Similarly, Khan has contributed significantly to the intersection of AI, healthcare, and secure cyber-physical infrastructures, with a particular emphasis on medical data classification, interpretability, and edge intelligence [30,31].

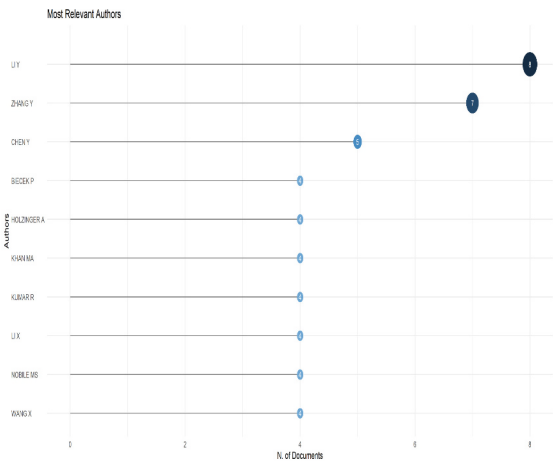

Fig. 5. Major Contributors to the Ex-MCPS Field

A closer look at the temporal distribution of their publications reveals diverse engagement patterns. HOLZINGER A, for example, exhibits a marked productivity peak in 2022, which may correspond to the completion of significant research projects or the dissemination of impactful results. In contrast, authors such as ZHANG Y and CHEN Y maintain steady and sustained research output across the years, reflecting continuous involvement in the domain. Meanwhile, KUJIAR R demonstrates a progressive increase in scholarly activity, suggesting a growing presence and influence within the Ex-MCPS research community.

4.4 Geographical Distribution and International Collaboration

Figure 6 presents the geographical distribution of publications on explainable medical cyber-physical systems. The United States, China, and India lead the field, each contributing over 60 publications, driven by robust research infrastructures and substantial investments in healthcare technologies. European countries such as Germany, the United Kingdom, and France also show strong engagement, underscoring Europe's prominent role as a research hub. Some countries demonstrate a tendency toward national collaborations, likely influenced by linguistic, policy-related, or systemic research factors. These patterns highlight the concentration of scientific activity in a few key regions and emphasize the importance of fostering international collaboration in this interdisciplinary field.

Country	Articles
USA	75
INDIA	59
CHINA	44
ITALY	38
UNITED KINGDOM	34
KOREA	21
GERMANY	20
SPAIN	19
AUSTRALIA	15
CANADA	10

Fig. 6. Geographical distribution of publications

4.5 Most Frequent Terms

In this study, we identified and analyzed the most frequently occurring terms within the collected literature. Core concepts such as "learning," "AI," "prediction," and "explainable" highlight the prevailing influence of machine learning and the increasing focus on interpretability. The presence of keywords like "deep" further suggests a strong adoption of deep learning techniques. Meanwhile, terms such as "SysML" and "AADL" indicate a growing, albeit still limited, interest in formal modeling approaches and system-level reliability (Fig. 7).

Word	Frequency
explainable	208
learning	137
ai	100
using	94
artificial	91
intelligence	91
model	87
prediction	82
models	76
deep	74

Fig. 7. Most frequent terms

5 Discussion

Building on the bibliometric landscape established in the earlier sections, this discussion aims to synthesize our findings and outline a path forward. This synthesis is articulated around two core objectives: (i) identifying key methodological and structural limitations in existing research on explainable medical cyber-physical systems (ExMCPS), and (ii) presenting the foundational architecture of our proposed EXMCPS framework.

5.1 Limits of Existing Works

To evaluate the current state of the field, we conducted a comparative analysis of representative works (Table 1), using three analytical dimensions derived from both our bibliometric analysis and domain-specific requirements:

- **Formal methods**, for ensuring safety and correctness in critical systems,
- **Explainability (XAI)**, necessary for clinical transparency and trust,
- **Model-Based Systems Engineering (MBSE)**, for system-wide traceability and integration.

These dimensions were selected because they address the unique challenges of ExMCPS, where systems must be not only functionally reliable but also interpretable and comprehensively modeled.

From this comparative analysis, three main limitations emerge:

1. **Limited system-level explainability**: While interest in XAI has surged since 2018—largely in response to regulatory frameworks such as the GDPR's "right to explanation" [38]—most existing research focuses on algorithmic transparency. As seen in [39], system-level explainability, which is essential for clinical validation and end-to-end traceability, remains largely underexplored.

Table 1. Comparative Analysis of Key Works in Explainable Medical CPS

Study	Technical Approach			Key Contribution
	Formal	XAI	MBSE	
Multimodal Explainable AI [32]	No	Yes	No	Unified framework for multimodal explainability in complex systems
Explainable AI for Medical Decisions [33]	No	Yes	No	Clinical interpretability requirements for high-stakes decisions
Time-Aware Medical CPS [34]	Yes	No	No	Formal modeling of temporal constraints in healthcare systems
Integrated AADL Verification [35]	Yes	No	Yes	Co-modeling approach combining AADL and Simulink
Safety-Critical Medical AI [36]	Yes	Yes	No	Formal verification framework for AI-driven clinical systems
SysML for Healthcare [37]	No	No	Yes	MBSE adaptation for medical device interoperability

2. **Underutilization of formal methods**: Despite their proven effectiveness in safety-critical applications, only 32% of the analyzed studies integrate formal techniques. Formal frameworks such as AADL [40] and modeling approaches like SysML [37] are rarely employed in combination with explainability objectives.
3. **Fragmented integration of key dimensions**: Many contributions focus on either explainability, safety, or system modeling in isolation. However, as safety-critical clinical environments require all three, the absence of integrative design frameworks presents a barrier to real-world deployment and scalability.

5.2 The EXMCPS Framework Proposal

In light of these observations and building upon our previous contributions to this domain we propose an integrated framework for the development of explainable medical cyber-physical systems: **EXMCPS**. This framework responds directly to the gaps identified in our bibliometric and comparative analysis and aims to unify formal methods, explainability, and system-level modeling into a coherent methodology.

The EXMCPS framework consists of three interconnected layers:

– **SysML Extension**: This layer augments conventional MBSE practices by introducing XAI-aware constructs into system models. By incorporating

traceability links, decision annotations, and model rationale, it facilitates transparency and supports regulatory documentation requirements [41].
- **XAI Integration**: Here, explainability is embedded at both the component and system levels. This includes not only the integration of interpretable models but also contextualization mechanisms that support clinical understanding and trust [42].
- **Formal Verification**: The final layer applies AADL's formal semantics [40] to verify safety properties and behavioral constraints. This ensures that system behavior remains predictable and verifiable, even when integrating complex AI components.

Together, these layers offer a comprehensive framework that addresses the methodological fragmentation currently observed in the literature. EXMCPS provides a structured, extensible, and standards-aligned foundation for the development of safe, explainable, and traceable medical CPS (Fig. 8).

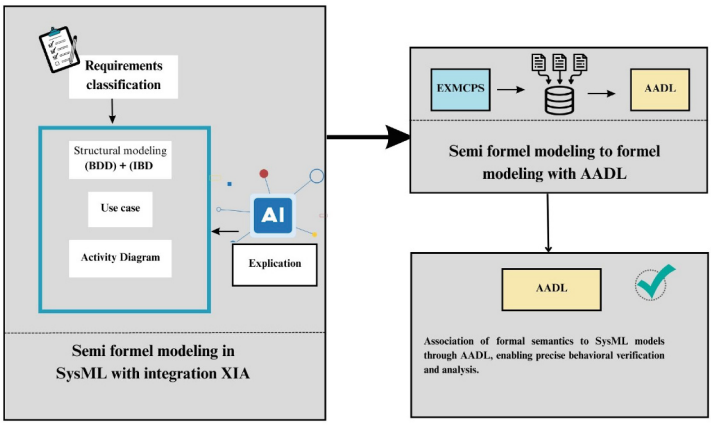

Fig. 8. Proposed conceptual framework for explainable medical cyber-physical systems (EXMCPS)

6 Conclusion

Through this bibliometric analysis, we have addressed the research questions formulated at the outset of our study, shedding light on the development, trends, and methodological directions of Explainable Medical Cyber-Physical Systems (ExMCPS). Then, we have observed a consistent increase in research activity since 2018, driven by the AI advancements and the growing need for transparency and accountability in clinical decision making (RQ1). This trend reflected both

technological progress and the influence of regulatory frameworks such as the GDPR.

Our analysis has highlighted the leading contributors, including HOLZINGER A [16], and KHAN M.A. [30], who play a central role in shaping the field. However, interdisciplinary collaboration remains fragmented, this limits the integration of diverse expertise essential for conceiving robust and context-aware ExMCPS (RQ2).

The geographical distribution of publications has shown dominance by the United States, China, and India, with strong contributions from Europe and emerging regions. Nonetheless, global collaboration networks show imbalances, suggesting the need to strengthen international research partnerships (RQ3).

Thematic analysis has revealed a strong focus on learning algorithms and explainability techniques. In contrast, formal methods and Model-Based Systems Engineering (MBSE) received relatively little attention, despite their value in enhancing system-level safety, reliability, and regulatory compliance (RQ4).

Beyond answering these research questions, this study provided a conceptual framework EXMCPS—designed to address current methodological fragmentation. The framework has been divided in three interconnected layers: a SysML extension that embeds explainability into system models, multi-level XAI integration supporting interpretability at both component and system scales, and formal verification using AADL to ensure safety and traceability across system behavior.

Future research should prioritize the operationalization of the EXMCPS framework through real-world case studies, such as Continuous Glucose Monitoring Systems (CGMS) and other safety-critical healthcare technologies. Advancing co-design practices involving clinicians, AI experts, system engineers, and regulatory authorities will be key to building ExMCPS that are not only high-performing but also trustworthy, explainable, and clinically meaningful. Additionally, greater integration of formal verification and MBSE with explainability mechanisms will help close current methodological gaps, contributing to the development of safer and more interpretable Medical Cyber-Physical Systems.

Acknowledgments. This work was partially supported by the LABEX-TA project MeFoGL: “Formal Methods for Software Engineering”.

References

1. Zhang, R., Bai, K.L., Bai, Y.: Medical cyber-physical systems. Proc. IEEE **101**(6), 1124–1131 (2013)
2. Mishra, A., Maheswari, U., Tripathy, B.: Cyber-physical systems: the next computing revolution. Int. J. Comput. Appl. **113**(1), 17–21 (2015)
3. Doshi-Velez, F., Kim, B.: Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608 (2017)
4. Samek, W., Wiegand, T., Müller, K.R.: Explainable artificial intelligence: understanding, visualizing and interpreting deep learning models. arXiv preprint arXiv:1708.08296 (2017)

5. Barredo Arrieta, A., et al.: Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible ai. Inf. Fusion **58**, 82–115 (2020)
6. Gunning, D., Aha, D.: XAI–explainable artificial intelligence. AI Mag. **40**(2), 44–58 (2019)
7. Ras, G., van Gerven, M., Haselager, P.: Explainable machine learning for medical applications. NPJ Digit. Med. **5**(1), 1–15 (2022)
8. Feiler, P.H., Gluch, D.P., Hudak, J.J.: The architecture analysis and design language (AADL): An introduction. Technical Report, Software Engineering Institute, Carnegie Mellon University (2006)
9. Lee, E.A.: Cyber physical systems: design challenges. In: 2010 11th IEEE International Symposium on Object Oriented Real-Time Distributed Computing, pp. 363–369. IEEE (2010)
10. Yang, G.Z., et al.: An integrated medical cyber-physical system for automatic diagnosis and real-time monitoring. Comput. Methods Programs Biomed. **145**, 45–56 (2017)
11. Zhu, T., et al.: A closed-loop artificial pancreas based on deep reinforcement learning. IEEE Trans. Cybernet. **50**(2), 448–460 (2020)
12. Khan, O., et al.: Real-time glucose prediction in CGMs using adaptive algorithms. IEEE Trans. Biomed. Eng. **66**(8), 2343–2352 (2019)
13. Benina, F., Benlahrache, N., Belala, F., Kacem, A.H.: Modélisation et analyse des systèmes cyber-physiques cas du système de surveillance continue du glucose. In: TACC, pp. 148–158 (2023)
14. Chen, M., et al.: Wearable sensors for reliable long-term health monitoring. IEEE Commun. Mag. **56**(1), 64–71 (2018)
15. Li, S., et al.: A survey on medical cyber-physical systems for personalized healthcare. ACM Comput. Surv. **52**(1), 1–31 (2019)
16. Holzinger, A., Biemann, C., Pattichis, C.S., Kell, D.B.: What do we need to build explainable AI systems for the medical domain? arXiv preprint arXiv:1712.09923 (2017)
17. Holzinger, A., et al.: Knowledge-aware and explainable machine learning in biomedicine. Nat. Rev. Methods Primers **2**(1), 1–18 (2022)
18. Topol, E.: Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again. Basic Books (2019)
19. Estefan, J.A.: Survey of model-based systems engineering (MBSE) methodologies. Technical report, 2007-006-01, INCOSE MBSE Initiative (2007)
20. Friedenthal, S., et al.: A Practical Guide to SysML: The Systems Modeling Language. Morgan Kaufmann (2014)
21. Elkhodr, M., et al.: Model-based engineering in the development of healthcare systems: a review. Healthcare Technol. Lett. **7**(3), 73–82 (2020)
22. Liu, Y., et al.: Safety assurance of medical cyber-physical systems through model checking and simulation. ACM Trans. Cyber-Phys. Syst. **4**(2), 1–27 (2020)
23. Medvidovic, N., et al.: Modeling and analysis of cyber-physical systems: a comprehensive survey. IEEE Trans. Software Eng. (2022)
24. Bramberger, S., et al.: Model-based engineering for certification of healthcare systems: opportunities and challenges. Softw. Syst. Model. **21**, 945–962 (2022)
25. Bouheroum, A., Benmerzoug, D., Hemam, S.M., Belala, F.: A multidimensional approach for formal modeling and analyzing medical cyber-physical systems. Eng. Technol. Appl. Sci. Res. **15**(1), 20211–20221 (2025)

26. Abdelmoumen, A., Benzadri, Z., Bouassida Rodriguez, I.: Reviewing recent literature on IoT-based system-of-systems: a bibliometric analysis. Informatica (Slovenia) **46**(3), 389–405 (2022). https://doi.org/10.31449/inf.v46i3.3816
27. Mongeon, P., Paul-Hus, A.: Journal coverage in web of science and scopus: a comparative analysis. Scientometrics **106**(1), 213–228 (2016)
28. Zupic, I., Čater, T.: Bibliometric methods in management and organization. Organ. Res. Methods **18**(3), 429–472 (2015)
29. Holzinger, A., Langs, G., Denk, H., Zatloukal, K., Muller, H.: Causability and explainability of artificial intelligence in medicine. Wiley Interdiscip. Rev. Data Min. Knowl. Discov. **9**(4), e1312 (2019)
30. Khan, M.A., Javaid, N., Imran, M., Guizani, M.: Ai-enabled solutions for healthcare using edge computing and federated learning. IEEE Internet Things J. **7**(10), 10300–10309 (2020)
31. Khan, M.A., Javaid, N., Zafar, A., Qasim, U.: IoT-based healthcare systems: a review of machine learning and deep learning applications, architectures and challenges. Sensors **21**(22), 7759 (2021)
32. Haldorai, A., et al.: Multimodal explainable AI for complex systems. J. Artif. Intell. Res. **72**, 123–145 (2021). https://doi.org/10.1613/jair.1.12345
33. Amann, J., et al.: Explainability for artificial intelligence in healthcare: a multidisciplinary perspective. BMC Med. Inform. Decis. Mak. **20**(1), 1–9 (2020)
34. Graja, M., et al.: Time-aware medical cyber-physical systems: formal modeling and analysis. In: Proceedings of the IEEE Real-Time Systems Symposium (RTSS), pp. 200–210 (2018). https://doi.org/10.1109/RTSS.2018.00023
35. Xu, L., et al.: Integrated AADL verification with Simulink for medical cps. IEEE Trans. Software Eng. **46**(4), 350–364 (2020). https://doi.org/10.1109/TSE.2019.2912412
36. Johnson, D., et al.: Safety-critical medical AI: formal verification framework. ACM Trans. Cyber-Phys. Syst. **4**(2), 15:1–15:25 (2020). https://doi.org/10.1145/3399723
37. Friedenthal, S., et al.: A Practical Guide to SysML: The Systems Modeling Language, 3 edn. Morgan Kaufmann (2014)
38. Arrieta, A., et al.: Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges. Inf. Fusion **58**, 82–115 (2020)
39. Zhang, Y., et al.: A survey on explainable AI in healthcare. IEEE Trans. Neural Networks Learn. Syst. **32**(11), 4793–4813 (2021)
40. Feiler, P., et al.: System architecture modeling with AADL. Carnegie Mellon University, Technical report (2006)
41. Chen, J., et al.: Explainable AI in medical cyber-physical systems: a review. J. Biomed. Inform. **128**, 104012 (2023)
42. Ghassemi, M., et al.: The false hope of current approaches to explainable AI in healthcare. Lancet Digital Health **3**(11), e745–e750 (2021)

Silver Box Testing for Scaling Software Development Fidelity: Enhancing Quality and Efficiency

Natali Levi-Soskin and Naomi Unkelos-Shpigel(✉)

Department of Software Engineering, Braude College of Engineering, Karmiel, Israel
{nataly1,naomius}@braude.ac.il

Abstract. This study presents Silver Box, a novel software development process that blends white-box, black-box, and gray-box testing into a unified, iterative framework. Designed for cloud-based projects, Silver Box emphasizes continuous testing alongside empathy-driven design thinking to improve software quality. The method was applied in a Cloud Computing course at Braude College with 158 third-year Software Engineering students. Teams developed large-scale systems, beginning with persona interviews and empathy mapping, followed by functional requirements and use case modelling. Despite identical tasks, teams produced diverse use case diagrams, highlighting inconsistent interpretation of requirements. Silver Box integrates early code review and acceptance testing strategies, leading to the detection of new and surprising types of errors—both in students' own code and while executing acceptance tests for other teams' projects. These findings show that cross-project testing fosters deeper understanding and better error identification. Silver Box offers an effective framework for enhancing cloud-native software development and education.

Keywords: software engineering · error detection · software testing · software fidelity

1 Introduction

The growing complexity of software systems necessitates robust testing methodologies that can effectively identify defects and ensure quality. Traditional testing approaches such as Black Box, White Box, and Gray Box each offer distinct advantages but also possess inherent limitations. Black box testing focuses solely on functional requirements without knowledge of the internal code, while White Box testing relies on complete access to the source code to validate internal logic. Gray Box testing attempts to balance these perspectives by leveraging partial knowledge of the system. However, these approaches may overlook critical discrepancies arising from differences in requirement interpretation, creativity in implementation, and edge case handling. To address these gaps, we propose a novel testing methodology termed Silver Box testing. This method involves independent groups developing the same feature or functionality separately, followed by cross-testing, including acceptance tests and code reviews. By introducing

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 273–281, 2026.
https://doi.org/10.1007/978-3-032-04403-7_23

this layer of peer evaluation from an independently developed perspective, Silver Box testing aims to uncover errors and omissions that traditional methods might miss, ultimately improving software quality and aligning development more closely with user expectations.

2 Related Work

As the software industry has evolved, numerous methodological approaches have been formulated. One of the earliest and most traditional approaches, the Waterfall model follows a sequential, linear process with distinct phases such as requirements gathering, design, implementation, testing, and maintenance. Each phase must be completed before moving to the next [1].

To define and develop systems across various domains, the International Council on Systems Engineering (INCOSE) has adopted the Vee model. This model's left branch focuses on decomposing requirements and creating system specifications, whereas the right branch concentrates on integrating parts and validating the system's performance [2]. In addition to the Vee model, another structural mode was defined – the Spiral model. This model involves a cyclical process of planning, risk analysis, development, and evaluation, with each cycle building upon the previous one [3].

Agile methodologies, such as Scrum, Kanban, Extreme Programming (XP), and Lean Software Development, have become widely embraced in recent years. These methodologies emphasize collaboration, iterative and incremental development, frequent feedback loops, and the ability to adapt to changing requirements [4].

While the above methods are software-focused, Model-Driven Development (MDD) uses models and domain-specific languages to automatically generate code, documentation, and other artifacts, aiming to boost productivity and reduce manual work. [5, 6]. Through the application of the Model Fidelity Hierarchy [7], the expectation is that the developed software will be precise. At the lowest level of this hierarchy lies spoken language, followed by informal written text, conceptual models, their enhancement with computational functionalities, and ultimately, an executable iteration of the model. These four levels reveal four types of errors: unexpected values errors, formulae and parameters errors, logical errors, and incomplete information or contradiction errors.

All mentioned approaches share a common thread: the development process [8] involves gathering requirements, modelling the prospective system, developing the software, and ultimately testing it. Despite the variety of testing methods, they all share the same goal: running a program to find errors. [9].

According to the international standard ISO 29119 [10], the testers can either be (1) the developers of the system for removing defects and increasing code quality, (2) the testing group that will create better test cases, (3) the managers for deciding when to stop testing, or (4) the users that will benefit from higher product quality [10]. All types of testers share a common objective: to enhance the system's quality and realize the benefits associated with its effective implementation. In addition, ISO 29119 defines four main types of testing: unit testing, integration testing, system testing, and acceptance testing. All tests are conducted following the code review process, which affects significantly on the defect removal effectiveness [11].

During various stages of software testing and review, different types of errors can be identified. The main motivation of modern code review is improving the quality of the software [12, 13], in addition to an early fix of defects during the development cycle [14]. Unit testing primarily focuses on verifying the correctness of individual units of code [15]. Errors commonly revealed during unit testing include logic errors within the code, boundary condition errors and interface errors between units [16]. Integration tests verify interactions between different units or components of the software [17, 18]. Errors typically revealed during integration tests include interface mismatches, incompatible dependencies between components and data flow errors across integrated modules [19]. When the integration tests are completed, system tests should be performed for assessing the behavior and functionality of the entire software system [20, 21]. Errors commonly revealed during system testing are discrepancies between system requirements and implementation, performance bottlenecks and security vulnerabilities [22]. Finally, after the software development process is completed, acceptance tests validate whether the software meets the requirements and expectations of end-users [23–25]. At this stage, typical errors are usability issues or understanding that some requirements were misinterpreted [26, 34].

In practical terms, the complete collection of test cases is often deemed infinite, making it theoretically impossible to cover all scenarios, even for the most straightforward programs. Consequently, executing such exhaustive testing could demand an extensive amount of time. Therefore, the challenge becomes selecting the most appropriate subset of test cases. Variety of techniques are employed for this purpose, some rooted in risk analysis and others relying on the expertise of test engineers [27].

Three methods of testing applied on a developed software: White box testing, employed to uncover bugs throughout the development phase [27], Black box testing, utilized after the development process is finalized and the product is ready, relying solely on output requirements [28], and Gray box testing, represents a fusion of Whitebox and Black box methodologies, occurs once the product is prepared, incorporating partial knowledge of the internal structure and code [29].

3 Method

In this section, we introduce the research participants, setting, tools, and the testing process.

3.1 Research Participants and Course Description

The research was performed in a third-year course “Introduction to Cloud Computing”, in a bachelor’s degree program in Software Engineering at a College of Engineering. This study included 158 students, in two different semesters, having the exact same task. At the first semester the task was building a game for learning Python while at the second semester the task was building a basic search engine with manager desktop and some statistics. As part of this course, students were required to develop a high-scale software project during the semester, in groups of four to six participants. The project was divided into five stages, each with its own specific purpose.

At stage one, each group had to perform a design thinking, which is used among software engineering practitioners [33], starting with a client interview. After the interview, the rest of the group had to build an empathy map [30, 33] in which they described what the persona does, feels, thinks and says during the interview (see Fig. 1).

DOES:	THINKS:
The persona uses popular search engines (like Google). The persona relies on forums and relevant internet communities. The persona relies on his friends for work.	The persona hopes that the search engine will understand his phrasing and language and provide accurate results. The persona hopes that the interface will be intuitive and personalized. The persona wonders if the engine will be suitable for his specific needs. The persona wonders if the results will remain up-to-date over time. Samuel is concerned about the reliability of the engine.
SAYS:	FEELS:
The persona says, "It is important for me to get relevant search results and an intuitive interface to navigate different topics." "I often struggle to find specific information in guides regarding the requirements of my project." "I would love to receive recommendations and customizations based on my previous searches; it would make me work faster."	The persona feels frustrated when the information he finds is not helpful or relevant to his task. The persona fears that he was not understood in presenting the problems and difficulties he is experiencing. The persona is curious about the search engine we intend to develop. The persona feels that if he does not know his job well enough, he will get fired.

Fig. 1. An example of an empathy map

Having the final solution the group members wrote a list of functional and non-functional requirements [31]. Finally, based on the requirements lists and the design thinking process, the groups built a use case diagram [32] for the proposed system.

Considering that all groups had the same task description and underwent the same design process simultaneously, the expectation was to produce similar use case diagrams. However, in practice, the diagrams exhibited differences, some of which were significant.

Figure 2 and Fig. 3 show an example of the differences produced by different groups of students in the search engine and python learning game project. These differ in the number of actors and the purpose of the system.

At stage two the groups defined the future acceptance tests to be performed on their own developed software. The third stage was the software development process. The

fourth stage focused on conducting a code review and performing software testing, and the final, fifth stage was dedicated to performing acceptance tests for their program.

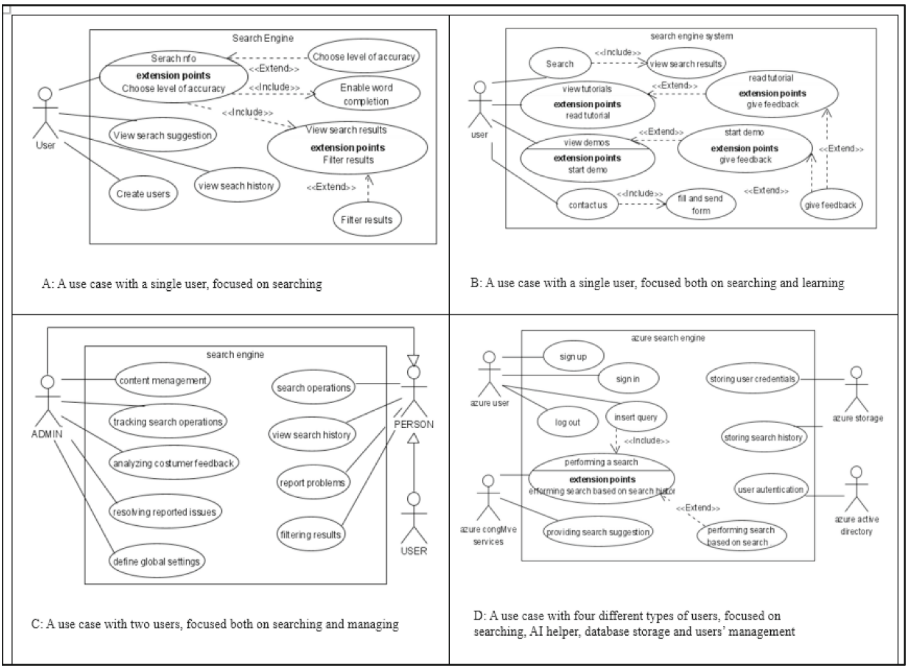

Fig. 2. Four different use cases of the basic search engine task

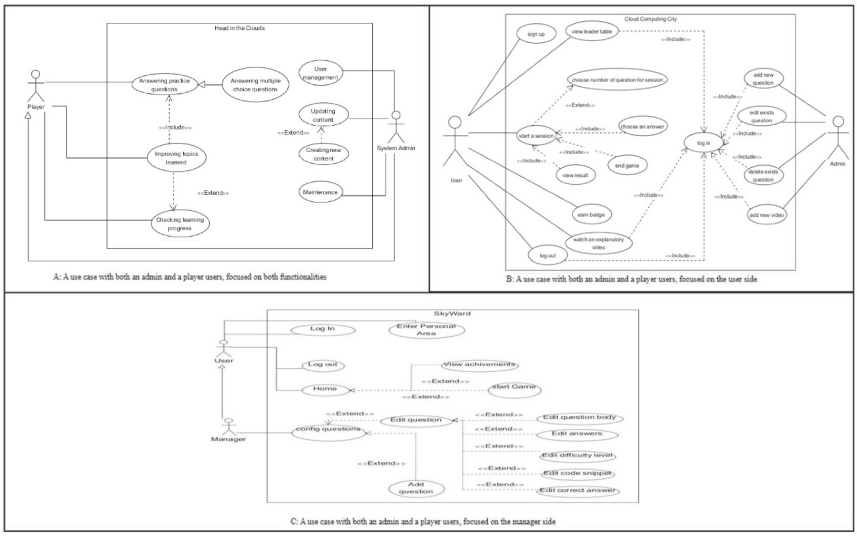

Fig. 3. Use case of the basic Python learning game task

3.2 Special Characteristics of the Second Semester

In the second semester, two additional stages were incorporated into the process. The sixth stage entailed each group executing an acceptance test originally authored by a different group, while the seventh stage involved conducting a code review of another group's implementation.

The integration of these stages facilitated the identification of further errors and contributed to the refinement of the user interface design.

4 Results

4.1 Acceptance Tests

Each group conducted acceptance tests on the system developed by another group. Despite differences in project domains and implementations, most tests passed successfully, indicating that core functionalities were maintained throughout the development lifecycle. In total, 131 acceptance tests were performed, of which 98 passed, resulting in a success rate of approximately 74.8%. While this relatively high success rate reflects a shared understanding of functional requirements and consistent implementation, the fact that 25.2% of the tests failed highlights the added value of the peer testing approach. These failures reveal a non-negligible number of errors that may not have been identified otherwise, emphasizing the importance of structured cross-group validation in enhancing system robustness and quality.

4.2 Code Review

In addition to acceptance testing, each group conducted a code review of a project developed by another team. The reviews focused on code readability, adherence to design principles, modularity, and maintainability. Most reviews identified a range of issue - such as unclear variable naming, duplicate logic, or lack of separation of concerns - which were not detected through functional testing alone. Several groups implemented improvements following the reviews, which led to more structured, maintainable, and consistent codebases. The process also enhanced students' awareness of good programming practices and design considerations. Overall, the code reviews served as an effective tool not only for error detection but also for promoting higher coding standards and mutual learning across groups.

5 Silver Box Testing

In this testing approach, the software is evaluated by an individual or a group assigned to perform the exact same task. Since different individuals may interpret customer requirements differently, vary in their level of creativity in user interface design, or consider different edge cases, testing conducted by someone who completed the task independently can reveal errors or omissions that might not have been identified otherwise. Table 1 gives a summary of the difference between three existing testing approaches and the novel silver box approach.

Table 1. Difference between testing approaches

Criterion	Black Box	White Box	Gray Box	Silver Box
Knowledge of Code	None	Full	Partial	Partial – the tester implemented the same functionality independently. Reviews another group's code
Performed By	External tester (QA)	The developer who wrote the code	Typically, another developer or tester with partial access to the code	A group that developed the same feature independently, testing the work of another group
Focused of Testing	Functionality and user requirements	Functionality and user requirements	Understanding of user requirements, implementation differences, edge cases, and UI design choices	Understanding of user requirements, implementation differences, edge cases, and UI design choices
Perspective	Neutral perspective; simulates end-user behavior	Neutral perspective; simulates end-user behavior	Balanced coverage of functional and structural aspects	Reveals gaps in interpretation, creativity, and hard-to-spot edge cases
Risks	May not cover all code paths	May overlook usability or requirements-related issues	Dependent on limited internal knowledge	Requires redundant development efforts (two groups per feature); needs coordination between groups
Phase in Development Cycle	Post-implementationtion	During or immediately after coding	Usually later stages, such as integration testing	Post-implementation, after each group has independently completed the feature

6 Summary and Conclusion

Silver Box testing represents an innovative extension to conventional software testing paradigms. By leveraging parallel independent development and subsequent cross-validation, it effectively uncovers subtle bugs and design flaws that arise from varying interpretations of user requirements and creative differences in implementation. This approach also facilitates a more comprehensive evaluation of the user interface and edge case scenarios, areas where other methods may lack depth. Although Silver Box testing requires additional development resources due to the need for duplicate implementations, the benefits include improved defect detection and enhanced collaboration among

development teams. The method's capacity to expose discrepancies and promote rigorous code review can lead to higher software reliability and user satisfaction. Future work should explore optimizing Silver Box testing to balance its resource demands with its quality assurance benefits, potentially integrating automated tools to streamline acceptance testing and code review processes. Overall, Silver Box testing offers a promising complement to existing methodologies, particularly for projects demanding high robustness and nuanced understanding of requirements.

References

1. Royce, W.W.: Managing the Development of Large Software Systems (1970) (2021)
2. Li, L., Levi Soskin, N., Jbara, A., Karpel, M., Dori, D.: Model-based systems engineering for aircraft design with dynamic landing constraints using object-process methodology. IEEE Access (2019)
3. Boehm, B.W.: A spiral model of software development and enhancement. Computer (Long Beach Calif) **21**(5), 61–72 (1988)
4. Ambler, S.W., Lines, M.: Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise. IBM press (2012)
5. Hailpern, B., Tarr, P.: Model-driven development: the good, the bad, and the ugly. IBM Syst. J. **45**(3), 451–461 (2006)
6. Selic, B.: The pragmatics of model-driven development. IEEE Softw. **20**(5), 19–25 (2003). https://doi.org/10.1109/MS.2003.1231146
7. Levi-Soskin, N., Jbara, A., Dori, D.: The model fidelity hierarchy: from text to conceptual, computational, and executable model. IEEE Syst. J. **15**(1), 1287–1298 (2020)
8. Parezanovic, N.: Racunarstvo i informatika. Zavod za udzbenike i nastavna sredstva–Beograd (1996)
9. Abran, A., Buglione, L., Sellami, A.: Software Measurement Body of Knowledge œ Initial Validation using Vincenti's Classification of Engineering Knowledge types (2004)
10. Iso. Software and Systems Engineering-Software Testing-Part 1: General Concepts (2022). Accessed: May 13, 2024. https://www.iso.org/standard/81291.html
11. Kemerer, C.F., Paulk, M.C.: The impact of design and code reviews on software quality: an empirical study based on psp data. IEEE Trans. Software Eng. **35**(4), 534–550 (2009)
12. Bacchelli, A., Bird, C.: Expectations, outcomes, and challenges of modern code review. In: 2013 35th International Conference on Software Engineering (ICSE), pp. 712–721 (2013)
13. McIntosh, S., Kamei, Y., Adams, B., Hassan, A.E.: An empirical study of the impact of modern code review practices on software quality. Empir. Softw. Eng. **21**, 2146–2189 (2016)
14. Fagan, M.E.: Design and code inspections to reduce errors in program development. IBM Systems Journal **38**(2.3), 258–287 (1999)
15. Daka, E., Fraser, G.: A survey on unit testing practices and problems. In: 2014 IEEE 25th International Symposium on Software Reliability Engineering, pp. 201–211 (2014)
16. Binder, R.: Testing Object-Oriented Systems: Models, Patterns, and Tools. Addison-Wesley Professional (2000)
17. Leung, H.K.N., White, L.: A study of integration testing and software regression at the integration level. Proceedings. Conference on Software Maintenance **1990**, 290–301 (1990)
18. Jorgensen, P.C., Erickson, C.: Object-oriented integration testing. Commun. ACM **37**(9), 30–38 (1994)
19. Kaner, C.: Software engineering metrics: what do they measure and how do we know?, In: Proc. Int'l Software Metrics Symposium, Chicago, IL, USA, Sept. 2004, pp. 1–12 (2004)

20. Hooda, I., Chhillar, R.S.: Software test process, testing types and techniques. Int J Comput Appl. **111**(13) (2015)
21. Simpson, W.R., Sheppard, J.W.: System Test and Diagnosis. Springer Science & Business Media (2012)
22. Pressman, R.S., et al.: A practitioner's approach. Software Engineering **2**, pp. 41–42 (2010)
23. Davis, F.D., Venkatesh, V.: Toward preprototype user acceptance testing of new information systems: implications for software project management. IEEE Trans. Eng. Manag. **51**(1), 31–46 (2004)
24. Ricca, F., Torchiano, M., Di Penta, M., Ceccato, M., Tonella, P.: Using acceptance tests as a support for clarifying requirements: a series of experiments. Inf. Softw. Technol. **51**(2), 270–283 (2009)
25. Leung, H.K.N., Wong, P.W.L.: A study of user acceptance tests. Software Qual. J. **6**, 137–149 (1997)
26. Cohn, M.: User Stories Applied: For Agile Software Development. Addison-Wesley Professional (2004)
27. Jovanović, I.: Software testing methods and techniques. The IPSI BgD Transactions on Internet Research **30** (2006)
28. Steegmans, E., et al.: Black & white testing: Bridging black box testing and white box testing. Software Testing: Beheers Optimaal de Risico's van IT in uw Business, pp. 1–12 (2004)
29. Tonella, P., Ricca, F.: A 2-layer model for the white-box testing of web applications. In: Proceedings. Sixth IEEE International Workshop on Web Site Evolution, pp. 11–19 (2004)
30. Ferreira, B., Silva, W., Oliveira, E., Conte, T.: Designing personas with empathy map. In: SEKE (2015)
31. Zhou, J.Y.: Functional Requirements and Non-Functional Requirements: A Survey. Concordia University (2004)
32. U. M. L. R. T. Force. OMG Unified Modeling Language Specification, version 1.3, June 1999 (1999)
33. Unkelos-Shpigel, N.: Heights of glory: design thinking to enhance battle monuments visits via augmented reality. Proceedings of AVICH (2024)
34. Unkelos-Shpigel, N., Sheidin, J., Kupfer, M.: Climb your way to the model: Teaching UML to software engineering students. In: International Conference on Advanced Information Systems Engineering, pp. 40–46. Springer (2019)

CASA Workshop

The Lifecycle Workbench - A Configurable Framework for Digitized Product Maintenance Services

Dominique Briechle(✉), Mohammed Fahad Ali, Marit Briechle-Mathiszig, Tobias Geger, Robert Werner, and Andreas Rausch

Clausthal University of Technology, 38678 Clausthal-Zellerfeld, Germany
{dominique.fabio.briechle,mohammed.fahad.ali,marit.elke.anke.mathiszig,
thomas.tobias.marcello.geger,robert.werner,
andreas.rausch}@tu-clausthal.de

Abstract. The global production of electric goods is at an all-time high, resulting in negative environmental and health impacts, as well as the continued depletion of natural resources. Considering the worsening global climate change, a transition of current industrial processes is necessary to tackle the factors mentioned above. Socio-economic systems like the Circular Economy (CE) provide options for a more sustainable society. Especially in terms of product lifecycle-prolonging, this system provides suitable approaches to prolong the use of products. Although the benefits of sustainable services enabling these options are known, they are not applied on a larger scale. The reasons for this are located at both the clients and service providers alike, and are associated with the lack of reliability in the expected pricing of a service and the monetary risk rooted in the inability to assess the condition of a product. To address these weak points in current service execution, the authors propose the *Lifecycle Workbench (LCW)*-ecosystem, which features digital representations to enhance the reliability of service pricing as well as the assessment of the condition of items, assemblies, and parts in the Circular Economy domain.

Keywords: Adaptive Lifecycle Management · Maintenance Service System · Digital Twin · System Architecture · Circular Economy

1 Introduction

Global production of electric and electronic goods is rising every year and is expected to exceed 9000 million products by 2029, which is 1500 million more than back in 2018 [1]. This continuously rising production causes severe environmental and health impacts [2] and depletion of natural resources [3]. In addition, the global amount of electric and electronic waste (WEEE) is rising with every passing year, exceeding the 60 million metric tons mark in 2022 [4].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 285–294, 2026.
https://doi.org/10.1007/978-3-032-04403-7_24

In contrast to the unsustainable linear economy, the CE, as an economic model, aims to prolong the use of products and resources alike. For products, it leverages certain options that are directed at enhancing their longevity, including reuse, repair, remanufacturing, and repurposing [5].

However, the CE faces several challenges when it comes to its implementation, especially in the area of reliability and risk connected to the determination of the cost of a certain service [6]. This becomes apparent by looking at the two main stakeholder types: the user/owner of a product, called in the following *ProductAdministrator*, and the provider of a CE-Service, called in the following *ServiceProvider*.

For example, one reason why *ProductAdministrators* do not have their devices repaired is the lack of reliability of the cost determination of such services, caused by the inability of *ServiceProviders* to assess the products before they receive them. This lack of uncertainty can be extended as well to the to-be-expected repair time, which depends on several factors like free capacity on the *ServiceProvider*-side, availability of spare parts, and level of knowledge of the *ServiceProvider*.

Therefore, on the *ServiceProvider*-side, the main barrier for offering services is the high amount of risk caused by the uncertainty of actual cost on his side [7], whereas the *ProductAdministrators* are disincentivized because of either high or unknown costs they are not willing to pay or an absence of economical feasibility.

A potential solution to increase the reliability for both stakeholders could be the deployment of digital ecosystems, which make use of digital representations as stakeholder proxies, that are outfitted to manage the offer generation and acceptance of the same.

In the scope of the paper, the authors are presenting a possible approach for such a digital ecosystem concept: the Lifecycle Workbench. To address the highlighted shortcomings of modern approaches and point out the interactions in the proposed *LCW*-ecosystem, the authors are focusing on a framework proposal for a digital service system model and its process conduction flow, shown in a scenario in Sect. 5. Thereby, the following two research questions are addressed:

- ***RQ1****: How can digitization ensure the reliability of service offers in the product maintenance service domain?*
- ***RQ2****: How can the requirements of different stakeholder types in product maintenance services be determined?*

The paper is structured in the following way: The related work is pointed out in Sect. 2. The description of the motivation is given in Sect. 3, whereas Sect. 4 presents the proposed framework. In Sect. 5, the theoretical application of the framework is shown. The section is followed by Sect. 6, in which the research questions and future research directions are discussed. Finally, the paper summarizes with the conclusion in Sect. 7.

2 Related Work

The use of Product Service Systems (PSS) to maintain products and their adjunct functionality in the context of sustainability has already been the subject of various studies. In terms of findings related to consumers' reactions to Sustainable Product Service Systems (SPSS), Hobson et al. [8] investigated these effects towards a hypothetical mobile phone SPSS. The requirements from the customer side of a Circular Economy-centred PSS are investigated as well by Fernandez et al. [9] by utilizing value proposition design to improve the design phase of PSS. Kjaer et al. [10] have investigated the interrelation between the Circular Economy and PSS as a way to decouple resource consumption from economic growth. This framework proposition was taken up by Matschewsky [11], who expanded it with the idea of circular support activities, which are measures to further increase resource decoupling. Similar in terms of the influencing factors, efficiency, longevity, and sufficiency, whereas different in the overall domain, Ghafoor et al. [12] proposed a PSS framework for housing, considering the applications and characteristics relevant in a Circular Economy. To help organizations in designing the corresponding CE-related business model, Pieroni et al. [13] identified two key factors, which they have considered in the conception of their proposed framework. In terms of design recommendations for PSS, McAloone and Pigosso [14] have presented an approach that takes the different dimensions of PSS into account and points out the design phases of the same.

3 Motivation and Scenario

To highlight the identified issues, stated in Sect. 1, the authors designed a scenario, acting as an initial problem description, which visualizes the necessity of the development of the anticipated *LCW*-framework for sustainable service design.

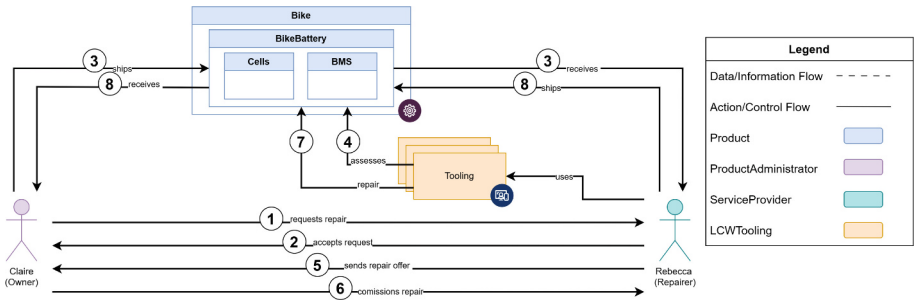

Fig. 1. Initial Bike battery repair scenario

E-bikes currently experience an all-time high in terms of urban and suburban mobility, especially in larger urban areas, for example, Dublin [15], and

were, therefore, selected as an example use case. The scenario, shown in Fig. 1, describes the interaction of two stakeholders, our E-bike *ProductAdministrator* Claire, as well as Rebecca, the *ServiceProvider*. Claire recognizes that her E-bike battery is depleted and is no longer capable of storing enough energy for her daily commute to her job. To change this, she is in search of somebody who can repair the battery. Luckily for Claire, after a while, she finds on the Internet a suitable *ServiceProvider*, Rebecca, who is able to repair her battery. She is, therefore, sending Rebecca a request for the repair of the battery **(1)**. After a day, Rebecca has time to look up her emails and review Claire's request. She accepts the repair job **(2)** but is, for now, not able to assess the potential cost of the operation, a circumstance she draws Claire's attention to. Claire sends the battery in the absence of options to Rebecca in the hope that the repair won't be too expensive **(3)**. Rebecca receives the battery and, after an additional day, which she had to take care of other clients of hers, she starts with the measurements of the battery cells in order to determine the damage **(4)**. After the assessment of the damage, Rebecca has now determined the root of the malfunction and calculates the repair cost. She is writing a repair offer and sends it to Claire **(5)** via E-Mail. Claire reviews the offer and is shocked at how costly the repair of her battery is. Unfortunately, she is in dire need of her battery and accepts contritely the offer after a day of reflection time **(6)**. Unfortunately, Rebecca has to wait until the spare parts are delivered to conduct the repair. After five more days, the spare parts are finally delivered to her and she can start with the repairing of the battery **(7)**. After a total of 8 days, the battery's functionality has been reestablished, and it is shipped back to Claire **(8)**. Claire receives her battery after 4 more days and can use her E-bike again. Although Claire is happy that she can now use her bike again, she is also frustrated because of the way the repair process was handled.

4 *LCW*-framework Conception

The scenario in Sect. 3 has pointed out the two major barriers that prevent both the stakeholders from using or offering CE services for the prolonging of product lifecycles. The proposed *LCW*-framework, shown in Fig. 2, has, therefore, to ensure both the generation of precise price estimations on the part of the *ProductAdministrator* as well as the mitigation of financial risk on the part of the *ServiceProvider*. Further, the *LCW*-framework needs to ensure that the identified temporal delays, caused by the stakeholders' interaction with one another as well as the selected business models, are addressed.

As will be shown in Sect. 5, the functionality for a *ProductAdministrator* of an *Item, Assembly, or Part* as such can also be reinstated through measures of exchange. Thereby, the *ProductAdministrator* will receive a spare product instead of his previously owned one, which will offer the same functionality. The proposed framework is designed to accompany different business models for CE-Services targeted at the prolonging of product lifecycles.

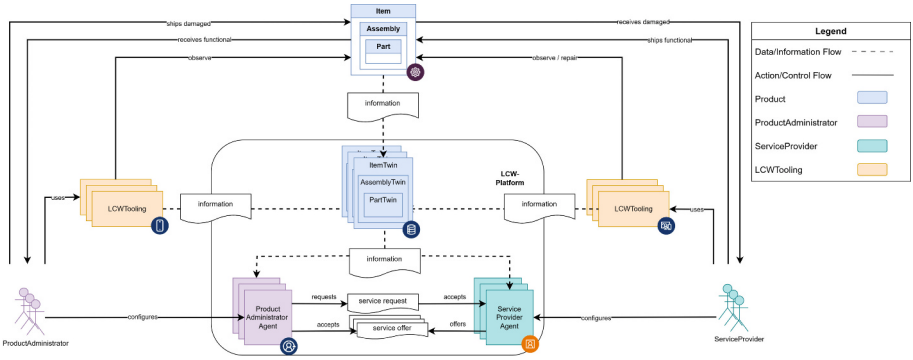

Fig. 2. *LCW*-framework describing the interaction between the different stakeholders and system components

Both stakeholders have at their disposal tools that enable the assessment of the product, the *LCWTooling*. For the *ServiceProvider*, the *LCWTooling* can take on many shapes and features a manifold of characteristics, spinning from simple tools for repairing to automated repair facilities and cloud-backed Artificial Intelligence (AI)-based tools for the evaluation of a large variety of different products. On the *ServiceProvider*-side, the tooling, therefore, includes both tools for the assessment and tools for the functionality reinstatement, hence the repair. For the *ProductAdministrator*, the *LCWTooling* will most likely be an application, which is executable on mobile phones and home computers. In contrast to the tools of the *ServiceProvider*, the purpose is solely the assessment of the product's condition. Both *LCWToolings* are, in addition, used to distribute their collected information to the Digital Twin of the corresponding *Item, Assembly, or Part*.

The central element of the proposed *LCW*-ecosystem is the *LCW*-platform. The *LCW-Platform* provides a frame for the integration of the digital representations of both the stakeholders as well as the *Item, Assembly or Part*. For both the stakeholders, these representations act as digital proxies, outfitted to take over the management of the service conduct. In contrast, the Digital Twin of the product acts as the core information hub for the received transmission streams, providing the information for the stakeholders and their agents.

As described, the stakeholders are represented by active agents. This means that they can actively manipulate other entities in the ecosystem based on their configuration. Stakeholders can, therefore, use these agents and define the rules upon which they will initiate certain actions. As depicted in Fig. 2, the *ProductAdministratorAgent* has two fundamental actions: it can request and accept service offers. The *ProductAdministrator*, therefore, configures the boundary conditions on whom the *ProductAdministratorAgent* can act to find an according *ServiceProvider*. These boundary conditions can be, for example, financial or time constraints. The *ServiceProviderAgent* on the other part crawls the active service requests and matches those to its configuration. With the active search

of suitable service requests, the *ServiceProviderAgent* is granted full access to the information about the product. The *ServiceProviderAgent* features as well conditions that lay down the rules of interaction for themselves. These can be, for example, the price and time constraints of a service or a specific product type. If a service request matches its conditions, the *ServiceProviderAgent* will act by sending a service offer to the *ProductAdministratorAgent*. The *ProductAdministratorAgent* will collect the incoming offers and will then decide based on its constraints, which one will be selected for the conduct of the service.

5 *LCW*-framework Scenario

In this section, the defined framework is now applied to the scenario introduced in Sect. 3. Our *ProductAdministrator* is again Claire, who owns the defective *Bikebattery*. Our *ServiceProvider*, Rebecca, is also part of the scenario and offers a service for the reinstatement of the functionality. However, Rebecca is no longer the only *ServiceProvider*. Together with her, Robert, and Reese offer services that reinstate the functionality of the product.

Before the actual process is started, the different stakeholders configure their respective agents **(0)**. Claire, on her part, configures her *ProductAdministratorAgent* with the boundaries of the later-generated service request. For her *BikeBattery* she is willing to pay 400 € and requests the functionality reinstatement in 6 days. The three *ServiceProvider* featured in the scenario have configured their corresponding *ServiceProviderAgents* to match the conditions of the services they offer, featuring the following configurations (Fig. 3):

- Rebecca's Agent: Duration and cost for functionality reinstation: 14 days & 350 €
- Robert's Agent: Duration and cost for functionality reinstation: 5 days & 450 €
- Reese's Agent: Duration and cost for functionality reinstation: 4 days & 400 €

The process is initially triggered by Claire through the assessment and damage detection of the *BikeBattery*, using her *LCWTooling* device. In this scenario, Claire uses an application on her mobile phone as *LCWTooling*, which allows the optical recording of the product's features. With the conclusion of the assessment of the *BikeBattery*, the *BikeBatteryTwin* is updated with the recorded information **(1)**. According to recorded data, the type and manufacturer of the battery are determined, as well as damage to the outside of the battery casing on the main connection plug of the *BikeBattery*.

With the information, the *ProductAdministratorAgent* is now generating a service request **(2)**, which is based on its configuration as well as the information stored in the *BikeBatteryTwin* **(3)**. Now the actively searching *ServiceProviderAgents* have access to the offer request information and can decide if they want to send a service offer to the *ProductAdministratorAgent* **(4)**. Since all the *ServiceProviderAgents* are configured to address the given product, all of them are

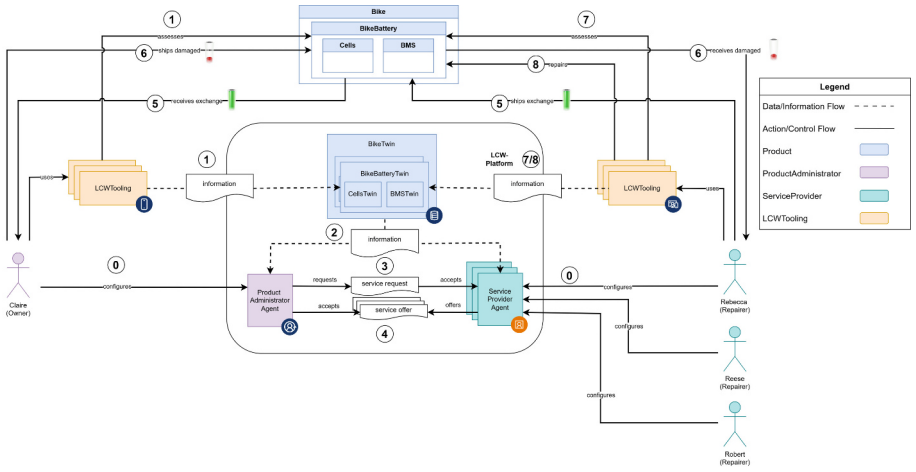

Fig. 3. Scenario description applying the *LCW*-framework to maintain a battery repair service for clients

sending a service offer with the previously introduced parameters. Since Reese's service offer is based on a product exchange model, her agent can offer the reinstatement of functionality within the shortest service duration while still being within Claire's defined price margin. Therefore, the *ProductAdministratorAgent* accepts the offer of Reese's *ServiceProviderAgent*.

Reese is acknowledged by her *ProductAdministratorAgent* of the acceptance of Claire's agent and sends her a refurbished *BikeBattery* of the same model **(5)**. Claire is happy with the easy and uncomplicated way this process is handled and sends her damaged *BikeBattery* back to Reese **(6)**.

However, for Reese, the process continues after receiving the *BikeBattery* **(7)** with the assessment of the same in detail by using the *LCWTooling*. The tool assesses the condition, in this case, measuring the capacity of cells and the functionality of the BMS, etc. and transmits the information to the *BikeBatteryTwin*. Following the assessment, Reese uses again her *LCWTooling* to conduct the repair of the *BikeBattery*, namely the exchange of certain cells and the BMS as well as the exchange of the damaged plug by disassembly and dissolving the connections with tools, like a Soldering Tool, in **(8)**. After finishing the repair, Reese stores the *BikeBattery* in her warehouse for future service requests.

6 Discussion and Future Work

The depicted scenario and the proposed framework describe how digital ecosystems can support the dissemination, optimization of existing services, and the conception of new business models enabled by digital technologies. This is the case for mechanics like the presented model in sect. 5. The scenario and framework can provide insights for the discussion.

RQ1: The two core issues highlighted in the motivation are addressed by the framework through digital representations: the assurance of the reliability of an offer for the *ProductAdministrator*, as well as the risk mitigation on part of the *ServiceProvider* due to the information provided by the *LCWTooling*. The reliability, paired with a decrease in service cost or a reduction in service time, can be an alternative to classical service offers. However, the effectiveness of the system is highly dependent on the *LCWTooling* providing the necessary information for the agents. For comprehensibility's sake, only rather simple *LCWTooling* applications were introduced. However, to increase the processing speed of such services, more complex tooling could be integrated, like an automated product disassembly system, as proposed by the authors in [16]. To determine the impact and potential effect on cost saving and productivity, the framework needs to be implemented. Future research should focus on how to specify the interaction mechanics between the stakeholders to increase the evaluation.

RQ2: Frameworks in general are proven to be capable of greatly supporting the development process of digital systems [17,18]. However, to increase acceptance, the determination of the requirements for the stakeholders is crucial. These are mostly dependent on the product itself as well as the corresponding business model of the stakeholders. The presented framework can support the stakeholders in the process of clarifying those requirements and highlighting their effect on the configuration of digital, agent-based representations. Future research should lie in breaking down the requirements of the stakeholders and investigating real-world applications to define their scope and specify the configuration of the digital representations.

In general, future research should focus on how to further enhance the presented framework and expand its functionalities. A potentially interesting field could hereby be the integration of a suitable logistics system to accompany the dynamic nature of such ecosystems, similar to a concept presented by Ali et al. in 2024 [19].

7 Conclusion

The paper presents an approach for designing a product maintenance service ecosystem with the support of the *LCW*-framework, which helps define the configurations and interactions of the different stakeholders and digital entities within *LCW*-platform. An initial scenario was presented to motivate the research questions raised. The presented framework was defined, and the main stakeholders and interactions within the ecosystem were described. In addition, an application of the framework was described. However, although the framework provides a possible foundation for such systems, it opens up several possible connection points for subsequent research. Especially in preparation for a real-world application of the framework, further research regarding the *LCWTooling* and the different digital representations is crucial.

Acknowledgment. This work has been developed in the project "Life_TWIN" (Research Grant Number 03EI5014A) and is funded by the German Federal Ministry for Economic Affairs and Climate Action (BMWK). The following partners were involved in this project: Clausthal University of Technology, Robert Bosch GmbH, Hellmann Process Management GmbH & Co. KG and Bernhard Olbrich Elektroinstallationen-Industrieanlagen GmbH.

References

1. *Consumer electronics - worldwide, , n.d.. [online]*, https://www.statista.com/outlook/cmo/consumer-electronics/worldwide. Accessed 30 Apr 2025
2. Noh, Y., Odimayomi, T., Sendesi, S.M.T., Youngblood, J.P., Whelton, A.J.: Environmental and human health risks of plastic composites can be reduced by optimizing manufacturing conditions. J. Cleaner Prod. **356**, 131803 (2022)
3. zu Castell-Rudenhausen, M., Wahlström, M., Nelen, D., et al.: Accelerating the circular economy in Europe - state and outlook 2024 (2023)
4. Baldé, C.P., Kuehr, R., Yamamoto, T., et al.: Global e-waste monitor 2024 (2024)
5. Kirchherr, J., Reike, D., Hekkert, M.: Conceptualizing the circular economy: an analysis of 114 definitions. Resour. Conserv. Recycl. **127**, 221–232 (2017)
6. Grafström, J., Aasma, S.: Breaking circular economy barriers. J. Clean. Prod. **292**, 126002 (2021)
7. Oh, S., Rhodes, J., Strong, R.: Impact of cost uncertainty on pricing decisions under risk aversion. Eur. J. Oper. Res. **253**(1), 144–153 (2016)
8. Hobson, K., Lynch, N., Lilley, D., Smalley, G.: Systems of practice and the circular economy: transforming mobile phone product service systems. Environ. Innov. Soc. Trans. **26**, 147–157 (2018)
9. da Costa Fernandes, S., Pigosso, D.C., McAloone, T.C., Rozenfeld, H.: Towards product-service system oriented to circular economy: a systematic review of value proposition design approaches. J. Clean. Prod. **257**, 120507 (2020)
10. Kjaer, L.L., Pigosso, D.C.A., Niero, M., Bech, N.M., McAloone, T.C.: Product/service-systems for a circular economy: the route to decoupling economic growth from resource consumption? J. Ind. Ecol. **23**(1), 22–35 (2018). https://doi.org/10.1111/jiec.12747
11. Matschewsky, J.: Unintended circularity?—assessing a product-service system for its potential contribution to a circular economy. Sustainability, 11(10) (2019)
12. Ghafoor, S., Hosseini, M.R., Kocaturk, T., Weiss, M., Barnett, M.: The product-service system approach for housing in a circular economy: an integrative literature review. J. Clean. Prod. **403**, 136845 (2023)
13. PP Pieroni, M., McAloone, T.C., CA Pigosso, D.: Configuring new business models for circular economy through product–service systems. Sustainability, **11**(13), 3727 (2019)
14. McAloone, T.C., Pigosso, D.C.: Designing product service systems for a circular economy. In: Designing for the Circular Economy, pp. 102–112. Routledge (2018)
15. Hosseini, K., Choudhari, T.P., Stefaniec, A., O'Mahony, M., Caulfield, B.: E-bike to the future: scalability, emission-saving, and eco-efficiency assessment of shared electric mobility hubs. Transp. Res. Part D: Transp. Environ. **133**, 104275 (2024)
16. Briechle, D., Rausch, A.: You've got a plan? A domain modelling approach for collaborative product disassembly planning with PDDL. In: ADVCOMP 2024, The Eighteenth International Conference on Advanced Engineering Computing and Applications in Sciences (2024)

17. Morisio, M., Romano, D., Moiso, C.: Framework based software development: investigating the learning effect. In: Proceedings Sixth International Software Metrics Symposium, pp. 260–268. IEEE (1999). (Cat. No. PR00403)
18. Valerio, A., Succi, G., Fenaroli, M.: Domain analysis and frameworkbased software development. ACM SIGAPP Appl. Comput. Rev. **5**(2), 4–15 (1997)
19. Ali, M.F., Briechle, D., Briechle-Mathiszig, M., Geger, T., Rausch, A.: Party without a cake? onto an inter-modal hitchhike logistics platform for passengers and products transportation. In: European Conference on Software Architectur, pp. 100–114. Springer (2024)

Exploring Static Analysis for Understanding Self-Adaptive Systems

Claudia Raibulet[1(✉)] and Michele Pugno[2]

[1] DISCo - Dipartimento di Informatica, Sistemistica e Comunicazione, Università degli Studi di Milano-Bicocca, Viale Sarca, 336, Milan, Italy
claudia.raibulet@unimib.it

[2] Faculty of Applied Engineering, Antwerpen Universiteit, Groenenborgerlaan 171, 2020 Antwerpen, Belgium
michele.pugno@uantwerpen.be

Abstract. Self-adaptive systems (SAS) detect environmental changes, prevent or react to them, and adjust their own configuration and behavior to ensure their quality. Understanding how SAS are designed and work/function, as well as their complexity and quality features, is necessary for their utilization, reuse, maintenance, and further evolution/development. A key role in understanding SAS is played by the analysis tools. Currently, there are no available tools developed specifically for SAS analysis, i.e., tools which focus on the adaptivity aspects including design and implementation features, or performance achievements. Therefore, the research question we address in this paper is: *How to analyze/understand SAS and their adaptive features with available general-purpose tools?*. Adaptivity can be analyzed (1) statically, starting from the source code, and generating higher abstraction related models concerning design and implementation solutions, and (2) dynamically, starting from software execution and focusing on the performances obtained due to the SAS adaptivity. In this paper, we perform static analysis of two SAS examples with the general purpose Understand tool. We use the Understand features to generate various graphs of the software architecture/structure, to analyze its complexity, and to compute various metrics. Furthermore, we propose metrics specific to self-adaptation. We compute these metrics using data gathered from Understand. The defined metrics are implemented as a plugin for Understand. We conclude that the Understand tool is suitable to perform static analysis on SAS. It may be also a candidate tool to be further extended and personalized to become also a SAS specific analysis tool.

Keywords: Self-Adaptive System · SAS · Evaluation · Software Engineering · Understand Tool

1 Introduction

Self-adaptivity [8,9,11] is gathering an increasing interest from researchers and practitioners due to its objectives of addressing variability and uncertainty at

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 295–310, 2026.
https://doi.org/10.1007/978-3-032-04403-7_25

runtime in dynamic and variable environments without human intervention. Software systems which are self-adaptive are more complex than the non-self-adaptive ones due to the presence of a managing part (implementing the self-adaptive mechanisms) and its interaction with the managed part (implementing the functionality of a system) (see Fig. 1). Understanding (part of) this design complexity may help practitioners use and re-use self-adaptive solutions and extend, personalize, and integrate them in other (even more) complex software systems.

In this context, it is fundamental to analyze available self-adaptive software with the objective to understand their design solutions, i.e., architecture and its components, their interaction mechanisms, as well as to identify personalization and extension points. This will enable the use and re-use of self-adaptive solutions and the further spread and growth of self-adaptive software systems.

The Main Idea of our Paper: As far as concerns our knowledge there is no tool support specific for the static analysis, quality assessment, and evaluation of SAS. Therefore, in this paper we investigate the use a general purpose tool to understand SAS, e.g., their design. Further, to personalize this tool for SAS we extend it with software metrics which provide an insight into self-adaptation specific design features. We explore static analysis of SAS to get an insight on their design and gather some aspects concerning their complexity. In this paper, we do not consider dynamic analysis which underline mostly the runtime benefits, e.g., in terms of performances, achieved through self-adaptation.

What We Have Done: First, we have decided to focus on the static analysis of SAS because the authors of SAS point out the performance results achieved through self-adaptation, while providing limited details on the design aspects, which are equality important for the SAS use, re-use, maintenance, and evolution. After considering some of the available tools for static analysis, e.g., SonarQube, UML lab, Enterprise Architect, we have chosen Understand because it is easy to use, easy to extend, freely available for academia, and it generates various graphs and views, which are easily navigable, of the analyzed software.

Second, we analyzed two examples of SAS made available by the SEAMS community, i.e., DeltaIot [5] and DingNet [4]. Both examples address issues in the IoT application domain and are developed by the same research group. They are implemented in the Java programming language. Third, we extended Understand with a plug-in which computes self-adaptation specific metrics. Understand calculates various metrics not tailored to self-adaptation. We broaden this set of metrics by computing metrics specific to self-adaptation based on the general purpose metrics calculated by Understand.

Fig. 1. SAS Managing and Managed Parts.

The Main Contributions of This Paper are two: (1) static analysis of SAS examples using a general-purpose tool and a discussion towards analyzing larger systems with intrinsic adaptation mechanisms, and (2) extension of Understand with self-adaptation specific metrics for static analysis.

Related Work: The approaches of SAS analysis are mostly focused on the performance evaluations at runtime. For example, one of the objectives is to evaluate the utility function in terms of costs (e.g., development effort) and benefits (e.g., performance) [1,10]. Available static analysis of SAS assesses their design quality through design patterns, code and architectural smells [7].

The rest of the paper is organized as follows. Section 2 introduces our study design. Section 3 describes the analysis of two SAS. Section 4 introduces the self-adaptive specific metrics. A discussion on the performed analysis is dealt in Sect. 5. The paper ends with the conclusions and further steps in Sect. 6.

2 Study Design

This section provides a brief overview on the motivation of our study and on the main steps performed.

Objective of the Static Analysis of SAS: Static analysis creates abstractions, models of software that aid in its comprehension and its design solutions. For the analysis of SAS, we are interested to comprehend the design of the managing part and its interaction with the managed part. Architecturally, there may be two types of implementations: internal and external. An internal implementation blends the adaptation logic and the application itself, hence the managed and managing parts are integrated. In an external implementation the two components are separated, i.e., the application becomes the managed part and the adaptation logic is the managing part. Hence, we are interested in the design solutions of the managing part with the objective to analyze their complexity and the development effort required [1,3].

A Brief Overview of Understand Tool: Understand[1] is a static analysis tool that allows visualization of the source code using graphs, charts, metrics, and reports. It can function as an integrated development environment (IDE). In addition, it is capable of standard testing, instant search and has plug-in support. It generates hierarchy graphs, cluster graphs, structure graphs, UML diagrams (e.g., class, sequence), dependency graphs, metrics treemaps, comparison graphs. It computes various metrics including complexity metrics (e.g., McCabe Cyclomatic Complexity, Path Count, Knots), count metrics (e.g., SLOC, Number of Functions, Statements), object oriented metrics (e.g., Coupling, Depth of Inheritance Tree, Lack of Cohesion). The types of graphs and metrics available for generation are dependent on the programming language. Understand supports various languages such as Ada, Assembly, C/C++, C#, Cobol, Java, JavaScript, PHP, Python, .NET], and XML.

[1] Understand Tool: https://www.scitools.com.

SAS Analysis with Understand: To analyze the self-adaptivness of a system, the components implementing its mechanisms must be identified. In an internal implementation, these components are mixed in the system. In an external implementation, they are separated into managed subsystem and self-adaptive managing subsystem. With Understand, we propose the search and identification of the managing subsystem in the following ways:

- Architecture Browser: lists all the components (e.g., directory structure). It does not identify SAS automatically, however SAS structures can be identified by name.
- Searching for keywords in an Entity Filter: SAS related keywords can be used to search for entities. Such keywords might be: "monitor", "analyze", "plan", "execute", "probe", "effector", "controller". Searched entities are: Files, Code Files, Header Files, Classes, Functions, Enumerators, Objects, Types, Macros, Subprograms, Packages, Modules, Blocks, Methods, Interfaces, SQL Tables. It should be noted that the discovered entity may or may not be a component of the self-adaptive subsystem, depending on the nomenclature provided by the developers.
- Metrics Treemap: is a useful tool for visualizing the overall structure of a project. It gives an overview of all the directories and files within the project. The rectangle's size and color intensity can be mapped to various metrics like, Cyclomatic Complexity, Lines of Code, Number of Files or Classes. In this study, we map the size to Lines of Code and the color to Average Cyclomatic Complexity. The managing subsystem can be identified by directory/files names. By examining the rectangles size and color intensity, the discovered subsystem can be directly compared to the rest of the system.

If managing subsystem cannot be identified, it is assumed that it has an internal implementation. The challenge is to identify a loop which implements the adaptation process.

Metrics in Understand: Understand provides various Complexity, Count and Object Oriented metrics. These metrics depend on the programming language used. We consider the following metrics to be relevant for SAS static analysis. These are their API names and their definitions, as described in Understand metrics browser:

- AvgCyclomatic - Average Cyclomatic Complexity for all nested functions or methods.
- CountDeclClass - Number of Classes.
- CountDeclFile - Number of Files.
- CountLine - Number of Physical Lines.
- CountLineCode - Number of Lines that contain source code. Note that a line can contain source and a comment and thus count towards multiple metrics.
- RatioCommentToCode - Ratio of number of comment lines to number of code lines. Note that because some lines are both code and comment, this could easily yield percentages higher than 100.

3 SAS Static Analysis with Understand

In this section we analyze two well-known exemplars: DeltaIoT[2] and DingNet.[3]

3.1 DeltaIoT Analysis with Understand

System Overview: Understand provides a visual overview of a project when loaded as shown in Fig. 2. It consists of line breakdown, directory structure, most complex functions, largest functions, most complex classes, largest classes, most complex files and largest files charts. These charts and graphs make it possible to understand the scope of the project and identify key components. The various colors in all charts (except for Line Breakdown) represent different directories. These charts immediately show whether any of the self-adaptive components play a significant role in the system. In DeltaIoT, we notice that `FeedbackLoop.planning` (McCabe Cyclomatic Comlexity (MCC) is 11) and `FeedbackLoop.execution` (MCC is 9) are the most complex methods.

The whole project has 7155 lines of code, out of which 3846 are code, 2315 are comments and 1072 are blank. It has 85 files, 105 classes and 551 functions.

Fig. 2. DeltaIoT Overview - Static Analysis.

[2] DeltaIoT: https://www.hpi.uni-potsdam.de/giese/public/selfadapt/exemplars/deltaiot/.

[3] DingNet: https://www.hpi.uni-potsdam.de/giese/public/selfadapt/exemplars/dingnet/.

Managing Subsystem Identification: DeltaIoT is a system that has an external adaptation. Its self-adaptive components have been identified as follows:

- Architecture Browser displays Directory Structure, where two directories named *SimpleAdaptationWithDeltaIoTClient* and *SimpleAdaptationWithSimulation* are present.
- Searching for keywords in an Entity Filter: We used keywords "monitor", "analyze", "plan", "execute", "probe" and "effector" to search for entities. Search results for these keywords return entities that are either in */Simulator/src/deltaiot/client* or */DeltaIoTClient/src/deltaiot/client*.
- Metrics Treemap is generated (see Fig. 3). Rectangle size is mapped to Lines of Code, and color is mapped to Average Cyclomatic Complexity. Directories *SimpleAdaptationWithDeltaIoTClient* and *SimpleAdaptationWithSimulation* have a yellow outline. Three files with the highest Average Cyclomatic Complexity are */DeltaIoTClient/src/deltaiot/client/Configurations.java* and the two feedback loop files *FeedbackLoop.java* from *SimpleAdaptationWithDeltaIoTClient* and *SimpleAdaptationWithSimulation*. The three largest files, in terms of Lines of Code, are *DeltaIoTSimulator.java* part of */Simulator/src/deltaiot/*, then *DeltaIoTClientMain.java* and *DeltaIoTEmulatorMain.java* part of */SimulatorGUI/src/deltaiot/gui/*.

Using these three approaches together, the self-adaptive elements are located. In DeltaIoT, two directories have these elements - 'SimpleAdaptationWithDeltaIoTClient' and 'SimpleAdaptationWithSimulation'. The two directories have identical files and classes. DeltaIoT User Guide that comes with this artifact, states that DeltaIoTClient software allows you to create your own adaptation logic and remotely interact with an actual IoT system, while Simulator is a simulation of the DeltaIoT network for offline experiments.

Metrics Computation: The metrics relevant to self-adaptation analysis along with their API names (see Sect. 2) from Understand for the entire DeltaIot project are listed in Table 1. Two self-adaptive directories *SimpleAdaptationWithDeltaIoTClient* and *SimpleAdaptationWithSimulation* have the highest Cyclomatic Complexity compared to other directories. However, they have the least amount of classes and files. The number of Lines of Code is on the lower end for these two directories.

Table 1. DeltatIoT Directory Structure Metrics

Directory	Avg Cyclomatic	Count DeclClass	Count DeclFile	Count Line	Count LineCode	RatioComment ToCode
DeltaIoTClient	1	45	46	3954	1510	1,33
DeltaIoTGUI	1,13	8	2	225	167	0,12
SimpleAdaptationWithDeltaIoTClient	2,15	4	4	278	220	0,05
SimpleAdaptationWithSimulation	2,13	4	4	250	190	0,07
Simulator	1,25	28	26	1966	1373	0,17
SimulatorGUI	1,35	16	3	482	386	0,05

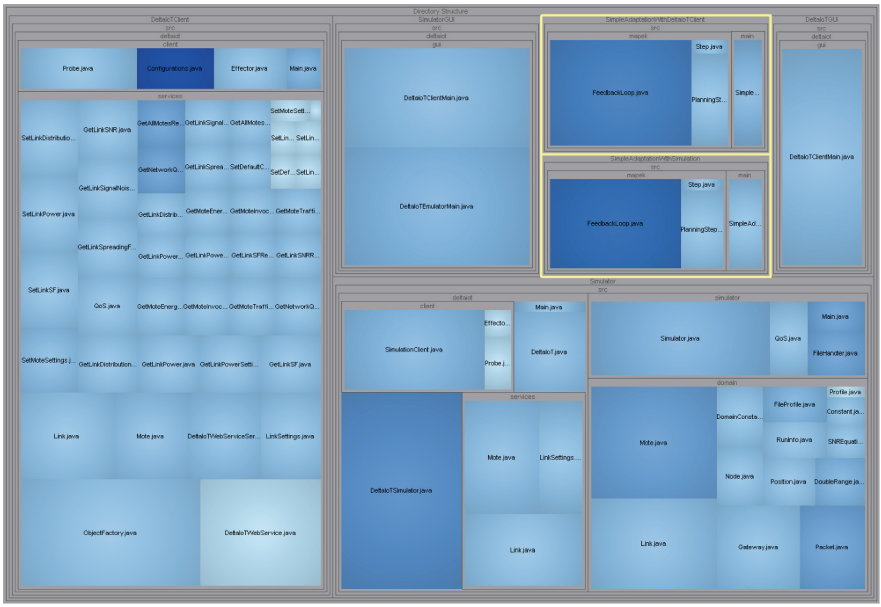

Fig. 3. Treemap of DeltaIoT directory structure. Blue rectangles represent files, grey outline around them represents directories. Directories *SimpleAdaptationWithDeltaIoTClient* and *SimpleAdaptationWithSimulatio* have a yellow outline. (Color figure online)

Graphs: The information that can be extracted depends on the type of a graph. There are many types of graphs that can be generated using Understand, such as hierarchy graphs, cluster graphs, structure graphs, UML graphs, and dependency graphs.

In DeltaIot, two directories are part of self-adaptation structure: *SimpleAdaptationWithSimulation* and *SimpleAdaptationWithDeltaIoTClient*.
Both have identically named files and classes. In this subsection, all the graphs are part of *SimpleAdaptationWithSimulation*.

The directory structure dependencies are depicted in Fig. 4. The blue lines between directories indicate dependence, while the red line indicates mutual dependence. Numbers indicate the amount of dependencies, for example, *SimpleAdaptationWithDeltaIoTClient* has 35 dependencies on *SimpleAdaptationWithSimulation*, 71 on *DeltaIoTClient* and 22 on *Simulator*. On the red line, *DeltaIoTClient* has 9 dependencies on *Simulator*, which has 26 dependencies on *DeltaIoTClient*. Directory *SimulatorGUI* has 3 dependencies on *SimpleAdaptationWithSimulation*. Directory *SimpleAdaptationWithSimulation* has 54 dependencies on *DeltaIoTClient* and 22 on *Simulator*.

Dependency graphs can also be made for classes. Such graph is depicted in Fig. 5. Various settings can be changed to customize the graph and/or get additional information. In our case, everything is on default, except for "Depends

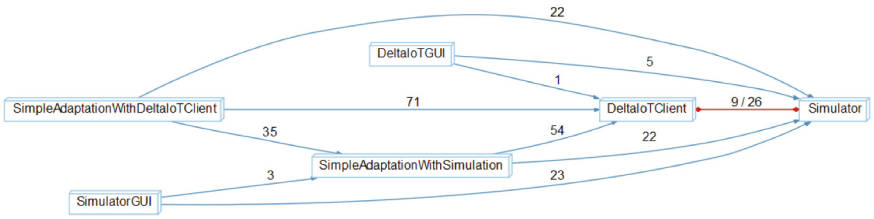

Fig. 4. DeltaIoT dependencies: directory structure.

on Depth" which is set to 2. Figure 5 shows `FeedbackLoop` class's dependencies. The class box in this graph can be expanded to reveal entities within it. Among entities within it, are `monitor` and `probe` methods which have a dependency on `deltaiot.clientProbe`; method `planning` has dependencies on `deltaiot.services.Mote`, `deltaiot.services.Link`, `mapek.Step`, and `mapek.PlanningStep`; `execution` has dependencies on `mapek.PlanningStep`, `mapek.Step`, `deltaiot.services.Link`, `deltaiot.services.LinkSettings`, and `deltaiot.client.Effector`.

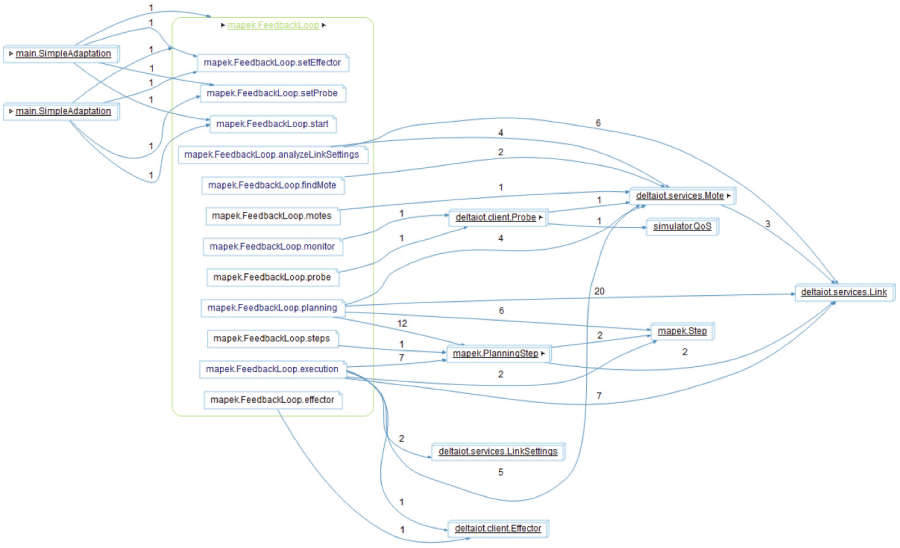

Fig. 5. `FeedbackLoop` class dependencies. Class box is expanded, Depends on Depth set to 2.

Understand provides "Calls & Called by" graphs for Java methods. Due to the dimensions of the graphs and the page limit of thus paper, please refer to the following document for details [N/A for double blind review].

3.2 DingNet Analysis with Understand:

System Overview:
In DingNet, the most complex method related to self-adaptation is `adapt` with McCabe Cyclomatic Complexity (MCC) of 19 from class `SelfAdaptation.FeedbackLoop.ReliableEfficientDistanceGateway` with MCC of 4.. This is the only class related to self-adaptation that is present in the depicted circles. The most complex files related to self-adaptation are *ReliableEfficientDistanceGateway.java* (MCC is 4) and *FeedbackLoopGatewayBuffer.java* (MCC is 2). None of the self-adaptation components are present in Largest Functions, Largest Classes or Largest Files. DingNet project has 12710 lines of code, out of which 8923 are code, 2345 are comments and 1536 are blank (Fig. 6).

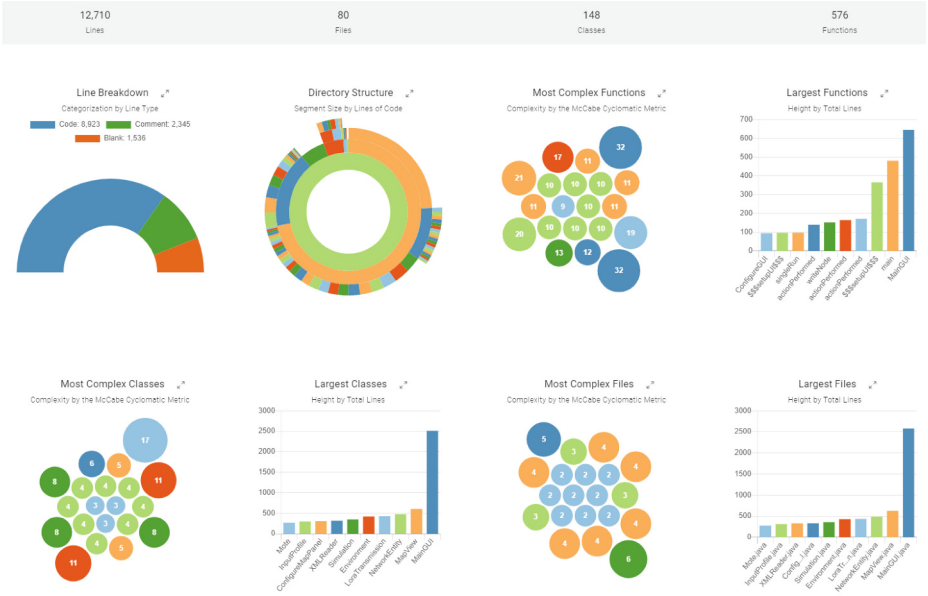

Fig. 6. DingNet Overview - Static Analysis.

Managing Subsystem Identification: DingNet is a system which uses probes and effectors to implement self-adaptation and its related feedback control loop. Its self-adaptive components have been identified as follows:

- **Architecture Browser** reveals the presence of the *SelfAdaptation* directory.
- **By searching for keywords in an Entity Filter**
  Keywords "probe" and "effector" are used. Directories *SelfAdaptation*, *IotDomain* and *GUI* contain "probe". Directories *SelfAdaptation* and *GUI* contain "effector".
- **Metrics treemap** is generated (see Fig. 7). Directory *SelfAdaptation* has yellow outline and is relatively small in comparison with the whole project.
  Three files with the highest Average Cyclomatic Complexity are *MapView.java*, *XMLReader.java* and *ReliableEfficientDistanceGateway.java* (part of *SelfAdaptation* directory).
  The three largest files, in terms of Lines of Code, are *MainGUI.java*, *MapView.java* and *XMLReader.java*.

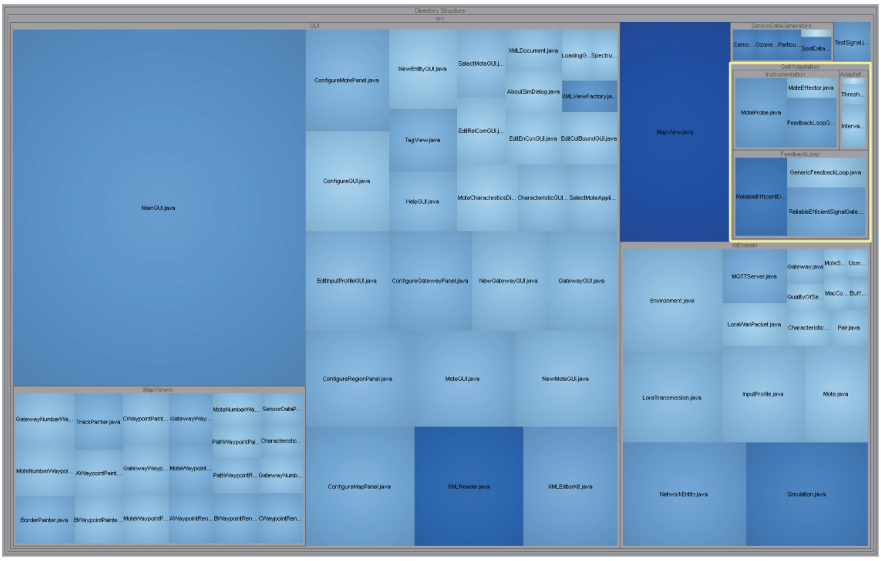

Fig. 7. Treemap of DingNet directory structure. Blue rectangles represent files, grey outline around them represents directories. Directory ' SelfAdaptation' has a yellow outline. (Color figure online)

Metrics Computation: Table 2 lists the metrics for the entire DingNet project. Directory *SelfAdaptation* has high (by comparison to other directories) Cyclomatic Complexity, low number of classes and files, as well as low amount of lines of code.

Graphs: Directory structure dependencies of the entire DingNet project is shown in Fig. 8. Directory *SelfAdaptation* has 22 dependencies on *IotDomain* while *IotDomain* has 114 dependencies on *SelfAdaptation*. Directory *GUI* has 61 dependencies on *SelfAdaptation*.

Figure 9 shows the internal dependencies of *SelfAdaptation* directory. Directory *FeedbackLoop* has 28 dependencies on *Instrumentation* and 2 dependencies

Table 2. DingNet Directory Structure Metrics

Directory	Avg Cyclomatic	Count DeclClass	Count DeclFile	Count Line	Count LineCode	RatioComment ToCode
GUI	2,11	115	48	8180	6415	0,13
IotDomain	1,86	17	16	2964	1509	0,74
SelfAdaptation	2,04	9	9	772	399	0,68
SensorData-Generators	3,2	5	5	132	69	0,72

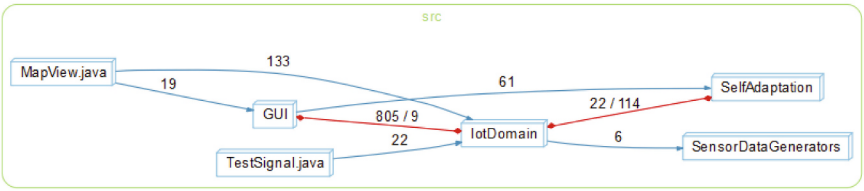

Fig. 8. DingNet dependencies: directory structure.

on *AdaptationGoals*. Directory *Instrumentation* has 2 dependencies on *FeedbackLoop*. Figure 10 shows `GenericFeedbackLoop` class's dependencies. The class has `getMoteProbe`, `setMoteProbe` and `getMoteEffector`, `setMoteEffector` methods. In comparison to DeltaIoT feedback loops, this one does not have monitor, plan or execution methods.

4 Self-Adaptation Specific Metrics

Understand computes various general-purpose metrics. However, they are not tailored to self-adaptation. We broaden the set of metrics by proposing metrics specific to self-adaptation and by computing them based on the general purpose metrics calculated by Understand. Similar or identical self-adaptive metrics have

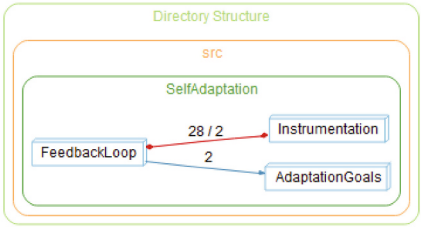

Fig. 9. Internal dependencies of 'SelfAdaptation' directory.

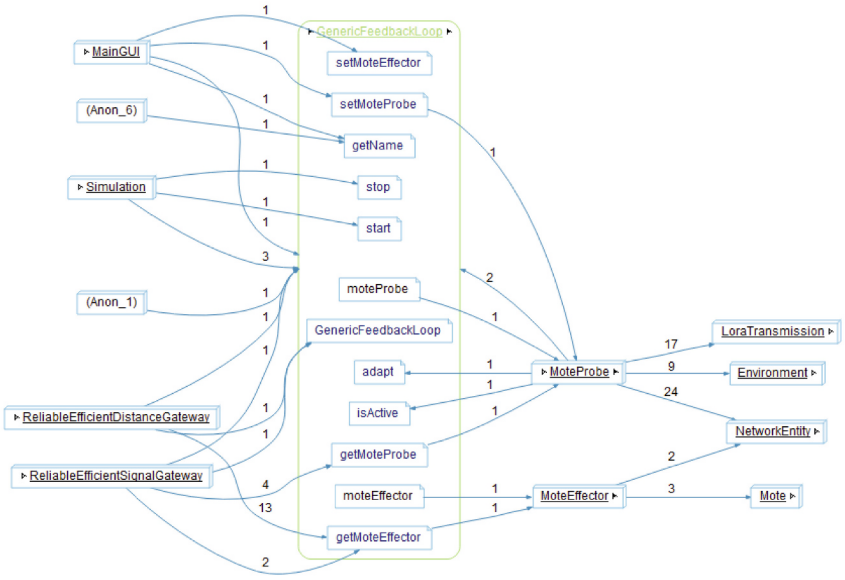

Fig. 10. `GenericFeedbackLoop` class dependencies. Class box is expanded, Depends on Depth set to 2.

been proposed in [3,6]. However, they have been theoretically defined. As far as concerns our knowledge there is no tool which supports their computation. We have implemented the following metrics as a plug-in for Understand (Table 3).

1. **Adaptation Absolute Dimension Increase (AADI):** defined as the number of Lines of Code in the managing part, i.e., for the adaptation loop.
2. **Adaptation Percentage Dimension Increase (APDI):** computed as AADI * 100 / Total number of Lines of Code.
3. **Adaptation Number of Classes (ANoC):** defined as the Number of Classes used to implement the adaptation loop.
4. **Domain Factors Influencing Adaptivity (DFIA):** indicating the context awareness of the managing part. It is calculated as the number of domain specific variables that are monitored and that influence the adaptation loop.
5. **Complexity of the Adaptation Loop (CAL):** expressed as the Average Cyclomatic Complexity, Cyclomatic Complexity, Maximum Cyclomatic Complexity and computed directly by Understand.
6. **Complexity of the Adaptation Strategies (CAS):** indicating the complexity of the analysis and planning steps of an adaptation loop.
7. **Number of Changes performed by Effectors (NCE):** indicating the impact of the managing part on the managed part. It is defined as the number of changes (e.g., on variables, services, hardware) performed by the managing part on the managed part.

8. **Adaptation Trigger (AT):** indicating whether adaptation is triggered by a change or performed continuously. It has two possible values: triggered or continuous.
9. **Architectural Adaptation (AA):** indicating whether adaptation is designed at the architectural level, i.e., it indicates the existence of a managing part separately of the managed part.

Table 3. DeltatIoT and DingNet metrics summary

Exemplar	AADI	APDI	ANoC	DFIA	CAL	CAS	NCE	AT	AA
DeltaIoT	528	7,38%	4	7	2,13/49/11	2/11	2	continuous	MAPE-K
DingNet	772	6,07%	9	4	2,04/94/19	5/19	5	continuous	none

5 Discussion

In this paper, attention is focused on the static analysis of SAS, i.e., on the identification of the managing part implementing the self-adaptive mechanisms. We aim to understand their design and complexity. This is a first step of the use and reuse of the available SAS approaches and towards a more complex estimation of their extendibility, maintainability, and sustainability over time.

We considered as case studies two SAS, DeltaIoT and DingNet, in the IoT domain, both provided by the SEAMS community. They have been developed following a consistent architectural model which highlights the role of each piece of code (e.g., MAPE-K components are clearly labeled and easily identifiable).

The use of static analysis tool appears to be valuable especially if results are compared over time: being able to monitor complexity (e.g., Cyclomatic complexity), level of coupling, number of dependencies and overall growth of the parts relevant to the SAS implementation may avoid architectural problems and constraints which might arise later during the software life cycle. Results provided by the analysis of the two well-known artefacts of small size show this approach could lead to valuable results useful during their further development.

5.1 Experimenting with Production Quality Software

Due to the limited size and complexity of the provided artefacts, the analysis of larger software is desirable to assess the usefulness of the static analysis through Understand. Moreover, taking into account a software not specifically designed as a proof of concept and with an architecture not completely known to the authors was due in order to spot difficulties in the process in a more realistic scenario. SAS are common in different fields as Resource Management, Robotics and many more; the former provided a suitable candidate.

Slurm[4] is a resource manager used by the 60% of the largest High Performance Clusters around the world. It provides a framework for starting, executing, and monitoring work on a set of nodes contested by different users [12].

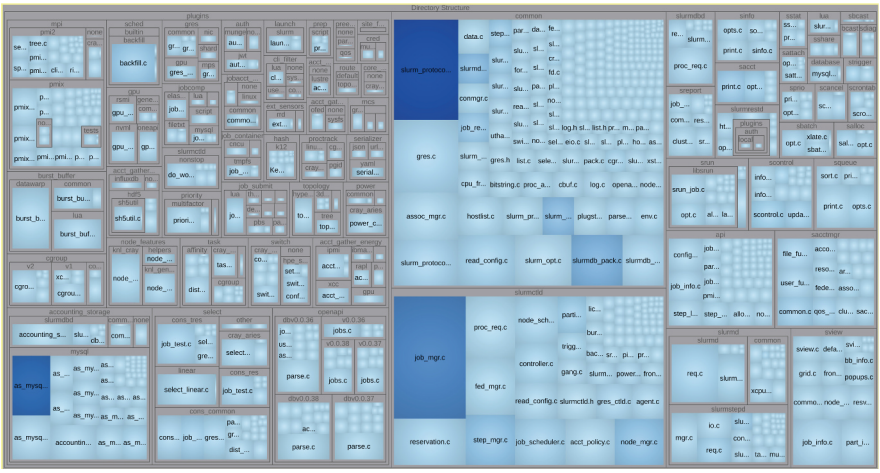

Fig. 11. Treemap of Slurm directory structure.

The internal organisation of the code-base appears greatly different and complex when compared to the previous artefacts mainly due to the use of an imperative language such as C and to the large amount of code (624,056 lines including blanks and comments). In order to extract meaningful information, one has to localize the relevant piece of code, not a trivial task considering the sheer size of the repository. For further details about the organisation of the code we refer to the documentation. Key areas and control loops have been identified in specific files and functions as shown in Fig. 11 and particular focus has been given to the `slumrctld` folder which contains the implementation of the deamon running on the master node, orchestrating the resources of the entire cluster. The deamon runs multiple threads, one of which dedicated to the scheduling duties.

As far as the authors were able to evaluate, the manager may be classified as a MAPE loop based self-adaptive software, providing a fault tolerant abstraction layer and CLI/API interface for resources/jobs scheduling. Although, the automatic extraction of relevant metrics resulted in partial or not trustworthy output due to the complex relationship network interleaving the various components of the system.

6 Conclusions and Further Work

To answer the research question in the abstract *How to analyze/understand SAS and their adaptive features with available general-purpose tools?*, we performed

[4] https://slurm.schedmd.com/.

a static analysis of SAS with the general-purpose tool Understand. The static analysis allowed us to identify the adaptation mechanisms and have an insight on their design and complexity through various graphs and metrics. Understand proved to be a very useful tool for easily navigating and comprehending the source code. Additionally, it is easily extendable through the plug-in mechanisms, and hence, adaptable for specific analysis such as for SAS. The plug-in code is described in [13].

Our next steps will concern the use of static analysis of SAS with the objective to extend, maintain, and enable sustainability over time [2]. Furthermore, analysing SAS core mechanisms from a static perspective may provide higher quality implementations over time by monitoring their natural evolution and integration within the general architecture of the target system. Furthermore, we aim to complement the static analysis with the dynamic analysis in order to investigate the benefits and the costs of self-adaptation at runtime.

Acknowledgment. We would like to thank Deividas Aksomaitis for the collaboration on the implementation of the Understand plug-in for self-adaptive specific metrics.

References

1. Gerostathopoulos, I., Raibulet, C., Alberts, E.: Assessing self-adaptation strategies using cost-benefit analysis. In: 44th International Conference on Software Engineering, Companion (2022)
2. Gerostathopoulos, I., Raibulet, C., Lago, P.: Expressing the adaptation intent as a sustainability goal. In: 44th International Conference on Software Engineering - New Ideas and Emerging Results (2022)
3. Kaddoum, E., Raibulet, C., Georgé, J-P., Picard, G., Gleizes, M-P.: Criteria for the evaluation of self-* systems. In: Software Engineering on Adaptive and Self-Managing System Workshop co-located with ICSE, pp. 29–38 (2010) https://doi.org/10.1145/1808984.1808988conf/icse/2010seamsdb/conf/icse/seams2010.html#KaddoumRGPG10
4. Provoost, M., Weyns, D.: DingNet: a self-adaptive internet-of-things exemplar. In: 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pp. 195–201 (2019). https://doi.org/10.1109/SEAMS.2019.00033
5. Iftikhar, M.U., Ramachandran, G.S., Bollansee, P., Weyns, D., Hughes, D.: DeltaIoT: a self-adaptive internet of things exemplar. In: 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, pp. 76–82 (2017). https://doi.org/10.1109/SEAMS.2017.21
6. Raibulet, C., Fontana, F.A.: Evaluation of self-adaptive systems: a women perspective. In: 11th European Conference on Software Architecture (Companion), pp. 23–30 (2017). https://doi.org/10.1145/3129790.3129825%20conf/ecsa/2017cdb/conf/ecsa/ecsa2017c.html#RaibuletF17
7. Raibulet, C., Fontana, F.A., Carettoni, S.: A preliminary analysis of self-adaptive systems according to different issues. Softw. Qual. J. **28**(3), 1213–1243 (2020). https://doi.org/10.1007/s11219-020-09502-5

8. Salehie, M., Tahvildari, L.: Self-adaptive software: landscape and research challenges. ACM Trans. Auton. Adapt. Syst. J. **4**(2), 14:1–14:42 (2009). https://doi.org/10.1145/1516533.1516538
9. Wong, T., Wagner, M., Treude, C.: Self-adaptive systems: a systematic literature review across categories and domains, Inf. Softw. Technol. J. **148** (2022). https://doi.org/10.1016/j.infsof.2022.106934
10. Jeroen Van Der Donckt, M.., Weyns, D., Usman Iftikhar, M.., Buttar, S.S.: Effective decision making in self-adaptive systems using cost-benefit analysis at runtime and online learning of adaptation spaces. In: Evaluation of Novel Approaches for Software Engineering (ENASE) (Selected Papers), pp. 373–403 (2018). https://doi.org/10.1007/978-3-030-22559-9_17conf/enase/2018s
11. Weyns, D.: An introduction to self-adaptive systems: a contemporary software engineering perspective. Wiley-IEEE Computer Society Press (2020). ISBN: 978-1-119-57494-1
12. Yoo, A.B., Jette, M.A., Grondona, M.: SLURM: simple linux utility for resource management. In: Feitelson, D., Rudolph, L., Schwiegelshohn, U. (eds.) JSSPP 2003. LNCS, vol. 2862, pp. 44–60. Springer, Heidelberg (2003). https://doi.org/10.1007/10968987_3
13. Aksomaitis, D.: Self-Adaptive Software Evaluation with Understand Tool. Bachelor Thesis Vrije Universiteit Amsterdam, The Netherlands (2022)

SADIS Workshop

AutoML: A Tertiary Study of Phases, Methods, Tools, and Frameworks

Keerthiga Rajenthiram[1(✉)], Pauline Delporte[2], and Patricia Lago[1]

[1] Vrije Universiteit Amsterdam, Amsterdam, The Netherlands
{k.rajenthiram,p.lago}@vu.nl
[2] CS Group, Le Plessis-Robinson, France
pauline.delporte@cs-sopratseria.com

Abstract. Automated Machine Learning (AutoML) plays a pivotal role in making machine learning more accessible by automating key steps in the model development process. Over the past decade, an increasing number of literature reviews (LRs) have examined specific components of AutoML, including data preparation, feature engineering, model generation, neural architecture search, hyperparameter optimization, and evaluation. However, a unified synthesis that consolidates findings across these reviews is missing. This paper presents a tertiary review of 32 LRs to provide a comprehensive and up-to-date overview of the AutoML landscape. We systematically analyze the core AutoML phases, categorize AutoML methods used across different stages of the machine learning (ML) pipeline, and compile a set of AutoML frameworks and tools. The synthesis offers a panoramic view of the techniques and tools supporting automation across ML phases. Our findings aim to serve as a reference for researchers and practitioners seeking to understand the current state of AutoML, the extent of automation achieved across pipeline stages, and the tools and platforms that support these capabilities.

Keywords: Automated Machine Learning (AutoML) · Tertiary Study · Literature Review (LR) · AutoML Tools and Frameworks · Automation in Machine Learning (ML)

1 Introduction

Artificial Intelligence (AI) and Machine Learning (ML) have revolutionized a wide range of domains, from healthcare to autonomous systems, by enabling data-driven decision-making through complex ML pipelines [19,20,22]. However, building effective ML models remains a complex, iterative, and resource-intensive process, often requiring expert intervention across multiple phases such as data preprocessing, algorithm selection, and hyperparameter tuning [13,50]. To reduce the expertise barrier and accelerate ML development, Automated Machine Learning (AutoML) has emerged as a paradigm to automate ML workflow design and optimization [1]. AutoML systems aim to minimize human intervention by automatically selecting models, tuning hyperparameters, and even

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 313–328, 2026.
https://doi.org/10.1007/978-3-032-04403-7_26

composing entire ML pipelines. Early efforts focused on tasks such as Hyperparameter Optimization (HPO) [35] and algorithm selection [13], but the field has since expanded to encompass Neural Architecture Search (NAS) [31], pipeline composition, and meta-learning [1,52].

AutoML has evolved rapidly in the past decade, with growing interest from both academia and industry. Major cloud providers like Google (Cloud AutoML [7]), Amazon (SageMaker Autopilot [4]), and Oracle AutoML [9] now offer AutoML services designed to empower non-experts with end-to-end ML capabilities [21,61]. Meanwhile, academic frameworks such as Auto-WEKA [6], auto-sklearn [5], and TPOT [11] have demonstrated AutoML's potential to match or exceed expert-tuned pipelines [52]. Despite these successes, several challenges persist: current systems vary in their degree of automation, generalization across tasks, scalability, and user interaction models [47].

While many literature reviews (LRs) explore specific aspects, such as HPO, NAS, or tool evaluations, no unified synthesis exists that consolidates their insights across the full ML lifecycle. As a result, researchers and practitioners lack a comprehensive understanding of how AutoML phases, methods, and tools interrelate, and a central reference point to select suitable methods and tools for their specific use cases.

This paper addresses this gap through a tertiary review of 32 LRs on AutoML. Our contribution is threefold: (i) synthesizing conceptual definitions of AutoML; (ii) identifying and organizing key phases and components (e.g., search space, strategy, evaluation); and (iii) analyzing dominant methods and mapping the most cited tools and frameworks to specific phases of the AutoML lifecycle. Through this synthesis, we aim to provide a structured overview of AutoML, serving as a reference for novice AutoML practitioners and experienced ML researchers, and domain experts applying AutoML in real-world contexts.

2 Related Work

Numerous LRs have explored individual aspects of AutoML over the past decade, yet most focus on specific phases, methods, or domains rather than offering a unified synthesis. Hyperparameter Optimization (HPO), for example, is widely studied using techniques like Bayesian optimization and random/grid search [3,19,35], while Neural Architecture Search (NAS) is the focus of works such as [31,32,41]. Some reviews contribute taxonomic insights. [18] proposed a unified terminology framework but did not explore tools or techniques in detail. Similarly, [27] classified AutoML into automated feature engineering (AutoFE), automated model and hyperparameter tuning (AutoMHT), and automated deep learning (AutoDL), focusing primarily on optimization strategies, yet without cross-phase integration.

Benchmark-oriented surveys like [67] evaluate AutoML systems based on performance but rarely address methodological structure or pipeline orchestration. Broader overviews, such as [34] and [36], outline core components of AutoML pipelines but lack integration of methods with tools or domains. Domain-specific

reviews add contextual richness yet remain narrow in scope. For instance, several works examine AutoML applications in healthcare [63,66]. Others address spatial decision support [59], graph-based learning [64], and recommender systems [26,65]. This underscores the need for comprehensive syntheses that span across domain boundaries.

To date, only one tertiary review [43] exists, which examines AutoML from a human-centered perspective—and focusing on visualization, user control, and associated risks—yet it does not provide a comprehensive synthesis of AutoML phases, methods, or tools. To the best of our knowledge, this study is the first tertiary review to comprehensively classify AutoML phases, components, methods, and tools across the ML lifecycle, offering a structured and integrative overview.

3 Study Design

We conducted a tertiary review [39] to investigate the current state of AutoML research, synthesizing evidence from existing systematic and multivocal literature reviews, which are treated here as primary sources. To efficiently explore and identify appropriate methods and tools for practitioner use cases, we adopted a rapid review methodology [24], involving a domain practitioner (second author) to ensure relevance and accuracy. The complete research process, along with the number of papers retained at each stage, is illustrated in Fig. 1.

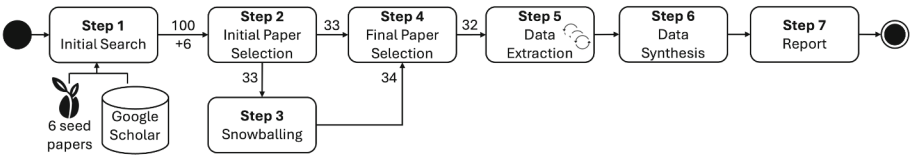

Fig. 1. Overview of the research process.

3.1 Research Questions

This study synthesizes definitions, phases, components, methods, and tools of AutoML from existing LRs through the following research questions (RQs).

RQ1: How is AutoML defined in the literature? Rationale: Despite the growing literature, AutoML lacks a unified definition, making it difficult to design, compare, and assess systems effectively. Clarifying its conceptualization across reviews can help standardize scope and terminology.

RQ2: What are the key phases and components of AutoML systems? Rationale: AutoML workflows span phases like preprocessing, feature engineering, model selection, tuning, and evaluation, but their structure varies across reviews. Synthesizing these differences is essential to define a unified pipeline for AutoML research and practice.

RQ3: Which methods and tools are most frequently used within each phase of the AutoML systems? Rationale: AutoML leverages diverse techniques and a range of tools and frameworks to automate ML workflows. Synthesizing their use across phases helps identify common practices, support tool selection, and guide future improvements.

3.2 Research Process

Step 1: Initial Search. The review began by selecting six seed papers (LRs on AutoML), which helped identify relevant keywords and formulate a structured search query. After iterative refinement, the final query was:

> ("AutoML" OR "Automated Machine Learning" OR "Auto Machine Learning") AND ("methods" OR "techniques" OR "tools" OR "frameworks" OR "practices" OR "guidelines" OR "tactic" OR "strategy" OR "challenges" OR "pitfalls" OR "limitations" OR "problem" OR "issue")

The query was run on Google Scholar (April 7, 2025) for papers since 2020. The first ten pages and six seed papers yielded 106 records; no additional LRs were found beyond page 10.

Step 2: Initial Paper Selection. After screening the titles and abstracts of the 106 papers using the inclusion(I)/exclusion (E) criteria (Table 1), 40 secondary studies were identified; removing preprints and book sections left 33 for further analysis.

Step 3: Snowballing. To expand coverage, we applied one round of backward and forward snowballing [2], retrieving 809 records. After removing duplicates, 496 unique papers remained. Screening them with the same selection criteria yielded 54 relevant LRs. Excluding 20 already in the initial set, 34 additional papers were retained.

Table 1. Inclusion and Exclusion Criteria for Study Selection

Inclusion Criteria	Exclusion Criteria
I1 Clearly identified as systematic literature reviews (SLRs) or multivocal literature reviews (MLRs)	E1 Primary studies
I2 Focused on AutoML as a central topic, addressing its phases, components, methods, or tools	E2 Reviews limited to narrow subtopics (e.g., only HPO or NAS) without broader insights into AutoML
I3 Published in English between 2020 and 2025	E3 Grey literature, unless included in a multivocal review
	E4 Book chapters, preprints, or non-peer-reviewed works

Step 4: Final Paper Selection. Following full-text screening and the application of selection criteria, 32 out of 67 literature reviews identified through both the initial search and snowballing were retained for data extraction.

Step 5: Data Extraction. Prior to full-scale extraction, a pilot study was conducted on a small subset of papers (3) to ensure consistency between authors and to refine the extraction guidelines. A standardized extraction guideline was used to collect metadata and AutoML-specific content, including publication details, research method, definitions, phases, components, methods, tools, application domains, challenges, best practices, human-in-the-loop (HITL) concepts, and table/figure captions. The extraction then proceeded in three allocation rounds, focusing solely on in-text content, without the author's interpretations. A total of 32 papers were fully extracted, organized in spreadsheets, and validated by the second author to ensure alignment with study goals.

Step 6: Data Synthesis. We applied open coding to all the segments of the extracted data, labeling each with descriptive codes for AutoML definitions, tools, methods, components, phases (e.g., NAS, CASH, HPO, Evaluation), and broader categories (e.g., Abstract, Challenges, HITL, Practices). These codes were iteratively synthesized into higher-order themes through bottom-up thematic synthesis. For instance, codes on search space, optimization, and evaluation were grouped under the corresponding AutoML phases. All paper IDs and in-text citations (e.g., [P1:111]) were retained for traceability. Although additional topics such as reported challenges, practices, and HITL concepts were coded, this paper focuses solely on synthesizing AutoML definitions, phases, components, methods, and tools. All extraction and synthesis artifacts are publicly available for reuse and transparency [10].

4 Results

This section presents the results extracted and synthesized from 32 LRs published between 2018 and 2025, with a notable increase in output from 2021 onwards, accounting for 85% (27/32) of the reviews. The majority are journal articles (22/32), with the remainder comprising conference papers (10/32). Although the selection focused on domain-agnostic LRs, at least 10/32 (29%) also examine specific domains, including healthcare, industrial systems, medical imaging, clustering, and time series forecasting. Thematically, HPO ($\geq$10/32) and NAS ($\geq$8/32) dominate the literature, with consistent attention given to the evaluation and comparison of AutoML tools. Several reviews also explore meta-learning, feature engineering, and AutoML tool benchmarking. This distribution reflects both the growing maturity of the field and its increasing relevance across diverse application areas and ML subfields. In the following subsections, we present the results structured around the three research questions guiding the study.

4.1 RQ1: Definition of AutoML Across Existing LRs

Definition and Scope. AutoML is broadly defined in the literature as a paradigm that automates the design and optimization of end-to-end ML pipelines to maximize model performance with minimal human effort [15,36,46]. This includes automating essential tasks such as data preprocessing, feature engineering, algorithm selection, hyperparameter optimization, and performance evaluation [28,29,34]. At its core, AutoML reframes traditional machine learning as a higher-level optimization problem, often described as "learning to learn" [60].

A key goal of AutoML, as emphasized across multiple reviews, is to democratize machine learning by enabling non-experts to develop high-performing models without in-depth technical knowledge [29,33,49]. This objective extends to diverse application contexts, from classical machine learning to deep learning and domain-specific pipelines (e.g., in healthcare and robotics) [12,51,53].

While the central aim – reducing human effort – is consistent, definitions differ in scope. Some papers define AutoML narrowly, focusing on optimizing individual components such as hyperparameters [12], while others emphasize full-pipeline automation extending to deployment and monitoring [49]. Despite this variation, the literature converges on AutoML as a tool to streamline ML adoption by minimizing expert dependency [28,44].

A central theme across definitions is that AutoML does not imply removing humans from the loop entirely. Instead, it shifts their role—from actively designing pipelines to configuring, supervising, or interpreting automated systems. This aligns with broader visions such as human-in-the-loop and interactive ML [23,29].

Taxonomies of AutoML. Several reviews go further to formalize the AutoML landscape through taxonomies. One of the most widely adopted structures is the triad of search space, search strategy, and performance estimation [34,62]. These dimensions define how an AutoML system explores different pipeline configurations, how it optimizes among them, and how it evaluates the results. This taxonomy is particularly helpful for comparing AutoML systems at a technical level. In contrast, other reviews employ a pipeline-phase taxonomy, organizing AutoML into sequential stages such as data preparation, feature engineering, model generation, and evaluation [12,49]. This offers a more functional view and is useful for practitioners mapping AutoML tools to real-world workflows. Some reviews also introduce level-based taxonomies, classifying systems by their automation coverage and depth—for instance, ranging from hyperparameter tuning to full pipeline synthesis [44]. A notable example is the seven-level classification proposed by Karmaker et al. [36], which organizes systems based on the automation of subtasks—ranging from basic model implementation to full pipeline formulation, feature engineering, and result summarization. These levels reflect both the extent of automation and the accessibility of tools to non-experts, enabling a structured comparison of AutoML capabilities. Some reviews introduce a task-type taxonomy that classifies AutoML approaches as supervised, unsupervised, or semi-supervised, based on the type of learning problem they address [16]. This

classification highlights AutoML's applicability across varying data regimes and underscores limited support for label-scarce scenarios like clustering and anomaly detection.

4.2 RQ2: Phases and Components of AutoML Systems

This section synthesizes existing literature on AutoML systems, focusing on the key phases of the ML pipeline that are commonly automated, as well as the structural components that enable this automation. An overview of these key phases is shown in Fig. 2, and each phase is discussed in detail below.

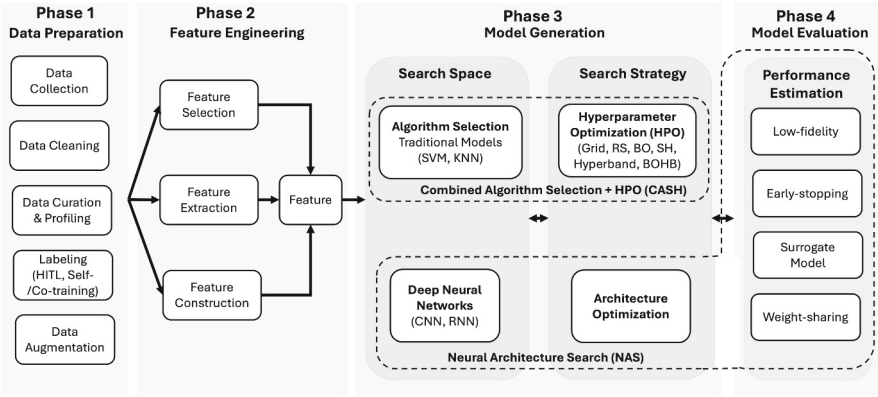

Fig. 2. Overview of key AutoML pipeline phases, extended from [34].

Phase 1: Data Preparation encompasses tasks required to transform raw data into a usable format for downstream learning. These include *visualization, cleaning, augmentation, labeling, and transformation* [34,49]. This phase is inherently iterative and often revisited throughout the pipeline to refine hypotheses and address evolving data issues [36]. Data is typically sourced from public repositories, web scraping, simulators, or generative models, and often requires filtering, deduplication, or relabeling to resolve noise and inconsistencies [34,49].

Phase 2: Feature Engineering (FE) transforms raw input data into a set of informative features that improve model accuracy and generalization [25]. The quality of features often has a greater impact on performance than the algorithm choice itself [34,36]. This phase comprises three subtasks: *Feature construction* creates new variables via mathematical or logical operations to reveal hidden patterns [29,67]; *Feature extraction* reduces dimensionality by mapping input features to a new space that retains essential information [34,49]; *Feature selection* identifies the most relevant subset of original features to minimize redundancy and improve interpretability [49,67].

Phase 3: Model Generation encompasses algorithm selection and hyperparameter optimization, often jointly addressed through Combined Algorithm Selection and Hyperparameter optimization (CASH) techniques [12,34].

Algorithm Selection identifies the most suitable learning algorithm for a given dataset using characteristics like distributional properties or meta-features to guide the choice of algorithm before or during hyperparameter tuning. In many AutoML systems, this is implemented as part of a joint optimization routine where algorithm selection is embedded within the search space traversal alongside hyperparameter configurations [30,67]. This strategy enables more efficient and targeted search compared to exhaustively tuning all algorithms in isolation.

Hyperparameter Optimization (HPO) focuses on selecting the best hyperparameter configurations to improve model performance. Unlike model parameters learned during training, hyperparameters are predefined and shape aspects like model architecture and learning dynamics [48]. In AutoML systems, HPO is commonly formulated as a black-box optimization problem, where configurations are evaluated based on successive trials without explicit gradient information [45,67].

Combined Algorithm Selection and Hyperparameter Optimization (CASH) jointly addresses the selection of the most suitable machine learning algorithm and the tuning of its hyperparameters to optimize performance [16,55]. By formulating these tasks as a single problem, CASH captures the interdependence between model choice and its parametrization, thereby enhancing the construction of end-to-end ML pipelines and influencing stages such as preprocessing and evaluation [55]. It is particularly critical in complex scenarios like multi-label or multi-target learning, where the search space is hierarchical and requires structured representations (e.g., context-free grammars, task decompositions) to explore valid algorithm-hyperparameter configurations [29].

Neural Architecture Search (NAS): In the context of deep learning, the model generation phase may also involve NAS, which automates the design of neural network architectures, reducing reliance on manual tuning. A typical NAS system comprises three core components: the search space, which defines allowable architectures; the search strategy (also considered Architecture Optimization); and the performance estimation method used to evaluate candidates [34,57]. The NAS phase continues to evolve with trends such as resource-aware design, joint optimization with hyperparameters, and benchmarking frameworks (e.g., NAS-Bench) that support reproducible evaluation and comparative analysis [34,49].

Phase 4: Model Evaluation is essential for validating the performance of models generated through automated search processes like CASH or NAS. Traditional evaluation methods, which involve fully training and testing each candidate model, are computationally expensive and time-consuming. For instance, some NAS strategies have required over 800 GPUs and weeks of computation

time to evaluate architectures [34]. Model selection automates the identification of the most suitable model or architecture for a dataset, exploring a predefined search space of algorithms and configurations [14,29]. These include both classical ML models and deep neural networks, often within CASH or NAS formulations [29,49]. The phase jointly identifies the most promising solution under computational and time constraints [29,48].

AutoML Components: AutoML systems rely on three foundational components—search space, search strategy, and performance estimation—to automate model development with minimal human intervention [12,17].

Search Space defines the range of choices an AutoML system can explore, including algorithms, hyperparameters, and neural architectures. These hyperparameters can be categorical or numerical, and often have dependencies that create conditional or hierarchical structures [34,57]. In NAS, search spaces are organized at macro and micro levels, with micro-level patterns (cells) reused across architectures, especially in deep learning for medical imaging [12].

Search Strategy determines how the space is explored. Traditional approaches have evolved into more advanced techniques like reinforcement learning, gradient-based optimization, evolutionary algorithms, and Bayesian optimization. These strategies aim to efficiently balance exploration and exploitation during model and hyperparameter selection [12].

Performance Estimation ensures reliable evaluation of candidate models. AutoML systems often use nested validation to avoid overfitting, and may incorporate fast approximations such as one-shot evaluation, surrogate models, early stopping, or zero-cost proxies to reduce computational overhead while maintaining accuracy [17].

4.3 Methods and Tools Across AutoML Phases (RQ3)

To answer RQ3, we synthesize evidence from LRs on the common methods, strategies, and tools used across AutoML phases and components. For each, we highlight the most frequently reported automation techniques and the tools or frameworks that support them.

Data Preparation involves key preprocessing tasks such as data cleaning, transformation, handling of missing values, and dataset profiling [34,67]. Preprocessing strategies span from rule-based heuristics and statistical imputers to meta-learningâĂŞbased feedback loops, adapting dynamically to dataset characteristics [34,57]. Cleaning operations are often divided into error detection and error repair [67]. Advanced approaches frame cleaning as a pipeline structure search problem, using greedy algorithms to compose operators based on data quality objectives [67]. Domain knowledge via integrity constraints or logic rules

(e.g., Katara) further enhances rule-based cleaning [67]. Pipeline-aware validation mechanisms consider downstream model constraints (e.g., encoding compatibility) during preprocessing [67]. Interactive and active cleaning strategies allow human feedback during data curation, supported by visualizations and hypothesis testing [36]. Labeling uses HITL and self-training methods, while transformations like encoding and scaling are selected based on model requirements. Data augmentation—via affine transforms, mixup, back-translation, or reinforcement-learned policies is increasingly optimized through frameworks such as AutoAugment [12,49]. Tools like Auto-sklearn, BoostClean, AlphaClean, DataRobot, and TransmogrifAI provide comprehensive support for these functions. Online Appendix A.1 [8] summarizes key techniques and tools used for data preparation.

Feature Engineering in AutoML is typically automated through three main subtasks—*selection, construction, and extraction* [34,67]. Feature selection is widely supported via filter, wrapper, and embedded models, with additional techniques like stability selection, regression-based filtering (e.g., in AutoLearn), and meta-feature ranking (e.g., in ExploreKit) [57]. Construction methods span arithmetic/logical transformations, graph-based search (e.g., Cognito, deep feature synthesis (Featuretools), and pipeline schemes like expand-reduce (DSM) or those without intermediate evaluations (OBM) [57]. Feature extraction focuses on dimensionality reduction (e.g., PCA, autoencoders) and domain-specific encodings (e.g., TF-IDF, HOG) [34,49]. Meta-learning (LFE), reinforcement learning, and genetic programming (AutoFE) are increasingly used to recommend or evolve transformations across phases [27]. Online Appendix A.2 [8] provides an overview of these techniques and their tool support.

Model Generation spans multiple phases (HPO, CASH, and NAS)—each formulated as a high-dimensional search problem over algorithm configurations, parameters, and pipeline structures [42]. A range of black-box optimization techniques such as Grid Search [30], Random Search [29,38], and Bayesian Optimization (e.g., SMAC, TPE) [17,67] support efficient search in HPO, with frameworks like HyperOpt, Optuna, and Syne Tune providing practical implementations [17]. Multi-fidelity techniques such as Hyperband and BOHB [29] improve search efficiency by allocating resources adaptively. For CASH, joint selection of algorithms (e.g., SVM, kNN, Decision Trees) and hyperparameters is supported using Sequential Model-Based Optimization (SMBO), as in Auto-sklearn [34,38] and Auto-WEKA [34]. Planning-based optimizers such as ML2-Plan use Hierarchical Task Networks to compose classifiers for multi-label tasks [60].

The *Architecture Optimization (AO)* core of NAS automates the discovery of neural network structures from search spaces such as entire networks, cells, or hierarchies [34]. AO strategies include: (1) Evolutionary Algorithms (EA) using direct or indirect encodings (e.g., NEAT, CGP) to evolve architectures via mutation and crossover; (2) Reinforcement Learning (RL) with controllers like RNNs (e.g., MetaQNN, ENAS, BlockQNN) learning architecture policies from performance feedback; (3) Gradient-Based Search, introduced in DARTS,

that relaxes the search space into a differentiable form to enable bilevel optimization, later refined in P-DARTS and ProxylessNAS to address memory and convergence challenges; (4) Surrogate Model-Based Optimization (SMBO) using predictors like GPs, Random Forests, or LSTMs (e.g., PNAS, NAO) to guide architecture selection; and (5) Hybrid Methods, such as Evo-NAS or NAO, combining strengths from EA, RL, GD, and SMBO for improved convergence and efficiency [34]. Recent advances in NAS include one-shot models, decoupled optimization of weights and topology, and resource-aware methods (e.g., MONAS, FBNetv3) that consider constraints like latency and FLOPs [29,34]. Online Appendix A.3 [8] provides a phase-wise mapping of optimization strategies.

Beyond model and architecture selection, AutoML systems aim to optimize complete end-to-end machine learning workflows—covering preprocessing, feature selection, modeling, and evaluation—by jointly optimizing structure and configuration through *Pipeline Optimization* [57,67]. This extends the CASH problem to a global level and involves balancing predictive performance with computational efficiency [55,62]. Two main design approaches exist for pipeline optimization: *fixed structure pipelines*, such as those in Auto-WEKA, define a linear sequence of steps (e.g., preprocessing → feature selection → modeling) and offer efficiency at the cost of flexibility [67]; and *variable structure pipelines*, which enable dynamic topologies via branching, feature unions, or dataset duplication. The latter approach leverages genetic programming (TPOT), hierarchical task networks (ML-Plan), reinforcement learning (AlphaD3M), and Monte Carlo Tree Search [67]. Multiple frameworks extend these approaches with notable design choices. Auto-WEKA uses Bayesian optimization via SMAC; Auto-sklearn introduces meta-learning warm starts and model ensembling [55,57], while Auto-sklearn 2.0 adds policy-based budget control [62]. TPOT leverages evolutionary algorithms to evolve scikit-learn-based pipelines. H2O AutoML adopts stacked ensembles and random search, whereas AutoFrankensteining and ATM combine bandit strategies with Bayesian search for distributed tuning [56]. AlphaD3M and Autostacker focus on interpretability and hierarchical model stacking, respectively [57]. Other notable methods include Portfolio Hyperband, which blends meta-learning and bandit optimization, TuPAQ's query optimization formulation, and probabilistic matrix factorization to model pipeline [57]. Grammar-based constraints are sometimes applied to ensure syntactic validity of generated pipelines [67]. Online Appendix A.4 [8] summarizes key pipeline optimization frameworks and their underlying optimization strategies.

Model Evaluation plays a central role in guiding algorithm selection and search efficiency, especially under computational constraints. To reduce overhead, various low-fidelity methods (e.g., FABOLAS [34]) simulate evaluations by training on subsamples or lower-resolution data. Neural Architecture Search (NAS) frameworks such as ENAS [49] and Single-Path NAS exploit weight sharing for faster evaluation. Surrogate models—like Gaussian Processes or neural predictors (e.g., PNAS, SemiNAS)—approximate performance and accelerate selection without full training [67]. Techniques such as early stopping

and learning-curve extrapolation also prune weak configurations efficiently [62]. Multi-fidelity strategies like Hyperband and SuccessiveHalving dynamically allocate resources based on intermediate results [40,56,67]. To improve robustness, ensemble learning is often employed, combining high-performing pipelines instead of selecting a single model (e.g., Auto-sklearn, H2O AutoML). Stacked ensembles further boost performance. Interpretability tools such as LIME and PDP are now integrated into platforms like Vertex AI to support explainable evaluation. For NAS-specific evaluation, One-shot NAS uses shared weights, while Zero-shot NAS predicts performance via proxies, and Resource-aware NAS adjusts for latency constraints. Across all methods, evaluation metrics (e.g., AUC-ROC, F1-score, RMSE) guide deployment decisions and continuous monitoring [28,29]. Online Appendix A.5 [8] summarizes widely used model evaluation methods and their application across various AutoML systems.

5 Threats to Validity

This study adopts a rapid, tertiary review approach, supported by practitioner collaboration, though several limitations remain important to reflect on. While the initial search was limited to Google Scholar, backward and forward snowballing helped capture relevant literature beyond the target period and outside the initial source. However, restricting the search to a single database may have excluded studies indexed elsewhere. We ensured adherence to rapid review principles by involving the second author, a practitioner, for regular input throughout the synthesis process. Still, the primary author conducted most of the data extraction and synthesis, which may introduce subjective bias. Lastly, heterogeneity in terminology, structure, and scope across the included literature reviews may have affected the consistency and completeness of the synthesis.

6 Discussion and Conclusion

AutoML has grown rapidly over the past decade, targeting diverse stages of the ML pipeline—from data preparation to model evaluation. This review synthesizes findings from 32 literature reviews, offering a structured view of the core phases, methods, and supporting tools and frameworks that shape the AutoML landscape. A key observation is the lack of a unified conceptual or taxonomic framework. AutoML definitions remain fragmented, often scoped to supervised learning, while unsupervised learning, multi-target regression, and online adaptation are underexplored. This inconsistency limits benchmarking, reproducibility, and comparability [49].

Despite progress, significant gaps remain in automation depth and generalizability. Many systems hard-code cleaning routines, treat feature engineering and evaluation as static, and provide limited transparency in model search. Although meta-learning, reinforcement learning, and evolutionary methods are increasingly adopted, most remain tuned for academic benchmarks rather than real-world constraints like cost, fairness, or interpretability [67]. Several open challenges stand out. First, AutoML for unsupervised learning—especially anomaly

detection and clustering—requires label-free evaluation and robust estimation. Second, task formulation remains a bottleneck due to its interactive, domain-specific nature. Third, frameworks underutilize domain knowledge, limiting their effectiveness in high-stakes areas like healthcare or finance [37].

Future Work. Looking ahead, promising directions include human-in-the-loop and trustworthy AutoML to balance automation with user control [47]; adaptive AutoML for non-stationary and streaming data; cross-domain integration using foundation models (e.g., LLMs) [54]; and benchmarking beyond accuracy to include efficiency, fairness, and socio-technical metrics [58]. Ultimately, AutoML is not a replacement for human expertise, but an assistive framework to elevate productivity, democratize ML development, and accelerate experimentation. By addressing the current limitations, the next generation of AutoML systems can move closer to the vision of fully autonomous, context-aware, and trustworthy machine learning.

Acknowledgments. This research is supported by ExtremeXP, a project co-funded by the European Union Horizon Programme under Grant Agreement No. 101093164.

Data Availability. The data and artifacts of this study are available as a replication package [10].

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Automated machine learning: methods, systems, challenges. https://doi.org/10.1007/978-3-030-05318-5
2. Contemporary empirical methods in software engineering. https://doi.org/10.1007/978-3-030-32489-6
3. Gradient-based optimization of hyperparameters | neural computation | MIT Press
4. Amazon SageMaker Autopilot (2025). https://aws.amazon.com/sagemaker/. Accessed 28 May 2025
5. Auto-sklearn (2025). https://automl.github.io/auto-sklearn/master/. Accessed 28 May 2025
6. Auto-WEKAt (2025). https://www.automl.org/automl-for-x/tabular-data/autoweka/. Accessed 28 May 2025
7. Google Cloud AutoML (2025). https://cloud.google.com/automl. Accessed 28 May 2025
8. Online Appendix for "AutoML: A Tertiary Study of Phases, Methods, Tools, and Frameworks" (2025). https://doi.org/10.5281/zenodo.15754103
9. Oracle AutoML (2025). https://www.oracle.com/nl/heatwave/automl/. Accessed 28 May 2025
10. Replication package for "AutoML: A Tertiary Study of Phases, Methods, Tools, and Frameworks" (2025). https://doi.org/10.5281/zenodo.15583132
11. TPOT (2025). https://automl.info/tpot/. Accessed 28 May 2025

12. Ali, M.J., Essaid, M., Moalic, L., Idoumghar, L.: A review of AutoML optimization techniques for medical image applications. Comput. Med. Imaging Graph. **118**, 102441 (2024). https://doi.org/10.1016/j.compmedimag.2024.102441
13. Ali, S., Smith, K.A.: On learning algorithm selection for classification **6**(2), 119–138. https://doi.org/10.1016/j.asoc.2004.12.002
14. Alsharef, A., Aggarwal, K., Sonia, Kumar, M., Mishra, A.: Review of ML and AutoML solutions to forecast time-series data **29**(7), 5297–5311. https://doi.org/10.1007/s11831-022-09765-0
15. Bachinger, F., Zenisek, J., Affenzeller, M.: Automated machine learning for industrial applications – challenges and opportunities **232**, 1701–1710. https://doi.org/10.1016/j.procs.2024.01.168
16. Bahri, M., Salutari, F., Putina, A., Sozio, M.: AutoML: state of the art with a focus on anomaly detection, challenges, and research directions **14**(2), 113–126. https://doi.org/10.1007/s41060-022-00309-0
17. Baratchi, M., Wang, C., Limmer, S., van Rijn, J.N., Hoos, H., Bäck, T., Olhofer, M.: Automated machine learning: past, present and future **57**(5), 122. https://doi.org/10.1007/s10462-024-10726-1
18. Barbudo, R., Ventura, S., Romero, J.R.: Eight years of AutoML: categorisation, review and trends **65**(12), 5097–5149. https://doi.org/10.1007/s10115-023-01935-1
19. Bergstra, J., Bengio, Y.: Random search for hyper-parameter optimization
20. Bernstein, A., Provost, F., Hill, S.: Toward intelligent assistance for a data mining process: an ontology-based approach for cost-sensitive classification **17**(4), 503–518. https://doi.org/10.1109/TKDE.2005.67
21. Bisong, E.: Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners. https://doi.org/10.1007/978-1-4842-4470-8
22. Bughin, J., et al.: Artificial intelligence: the next digital frontier?
23. Cao, L.: Beyond AutoML: Mindful and actionable AI and AutoAI with mind and action **37**(5), 6–18. https://doi.org/10.1109/MIS.2022.3207860
24. Cartaxo, B., Pinto, G., Soares, S.: Rapid reviews in software engineering. In: Felderer, M., Travassos, G. (eds.) Contemporary Empirical Methods in Software Engineering, pp. 357–384. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-32489-6_13
25. Chauhan, K., et al.: Automated machine learning: the new wave of machine learning. In: 2020 2nd International Conference on Innovative Mechanisms for Industry Applications (ICIMIA), pp. 205–212. IEEE
26. Chen, B., Zhao, X., Wang, Y., Fan, W., Guo, H., Tang, R.: A comprehensive survey on automated machine learning for recommendations. https://doi.org/10.48550/arXiv.2204.01390
27. Chen, Y.W., Song, Q., Hu, X.: Techniques for automated machine learning **22**(2), 35–50. https://doi.org/10.1145/3447556.3447567
28. Connor, L., Kelly, R., Murphy, S., McCarthy, E., Murray, E.: Analysis of AutoML tools in the world of automated deep learning **5**(1), 541–555
29. Del Valle, A.M., Mantovani, R.G., Cerri, R.: A systematic literature review on AutoML for multi-target learning tasks **56**(2), 2013–2052. https://doi.org/10.1007/s10462-023-10569-2
30. El Shawi, R., Maher, M., Sakr, S.: Automated Machine Learning: State-of-The-Art and Open Challenges. https://doi.org/10.48550/arXiv.1906.02287
31. Elsken, T., Metzen, J.H., Hutter, F.: Neural architecture search: A survey. https://doi.org/10.48550/arXiv.1808.05377

32. Fu, J., Cong, P., Xu, S., Chang, J., Liu, X., Sun, Y.: Neural architecture search with deep radon prior for sparse-view CT image reconstruction **52**(5), 3044–3058. https://doi.org/10.1002/mp.17685
33. Gosai, D., Patel, M.: Advancing the machine learning workflow: a comprehensive analysis of auto machine learning (auto-ML) techniques and future investigations. In: Luhach, A.K., Jat, D.S., Ghosh, U., Gao, X.Z., Lingras, P. (eds.) Advanced Informatics for Computing Research, pp. 308–323. Springer. https://doi.org/10.1007/978-3-031-84062-3_25
34. He, X., Zhao, K., Chu, X.: AutoML: a survey of the state-of-the-art **212**, 106622. https://doi.org/10.1016/j.knosys.2020.106622
35. Hutter, F., Lücke, J., Schmidt-Thieme, L.: Beyond manual tuning of hyperparameters. KI Künstliche Intelligenz **29**(4), 329–337 (2015). https://doi.org/10.1007/s13218-015-0381-0
36. Karmaker ("Santu"), S.K., Hassan, M.M., Smith, M.J., Xu, L., Zhai, C., Veeramachaneni, K.: AutoML to date and beyond: Challenges and opportunities **54**(8), 175:1–175:36. https://doi.org/10.1145/3470918
37. Khalid, E.T., Aldarwish, A.J., Yassin, A.A.: Challenges in automl and declarative studies using systematic literature review **2023**, 118–125
38. Kim, D., Koo, J., Kim, U.M.: A survey on automated machine learning: problems, methods and frameworks. In: Kurosu, M. (ed.) Human-Computer Interaction. Theoretical Approaches and Design Methods, pp. 57–70. Springer. https://doi.org/10.1007/978-3-031-05311-5_4
39. Kitchenham, B., et al.: Systematic literature reviews in software engineering – a tertiary study **52**(8), 792–805. https://doi.org/10.1016/j.infsof.2010.03.006
40. Li, Y., Wang, Z., Xie, Y., Ding, B., Zeng, K., Zhang, C.: AutoML: from methodology to application. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, CIKM 2021, pp. 4853–4856. Association for Computing Machinery. https://doi.org/10.1145/3459637.3483279
41. Liu, C., et al.: Progressive neural architecture search. https://doi.org/10.48550/arXiv.1712.00559
42. O'Leary, C., Toosi, F., Lynch, C.: A review of AutoML software tools for time series forecasting and anomaly detection:. In: Proceedings of the 15th International Conference on Agents and Artificial Intelligence, pp. 421–433. SCITEPRESS - Science and Technology Publications. https://doi.org/10.5220/0011683000003393
43. Palazuelos, C., Duque, R., Tirnauca, C., Pérez, A., Casas, A.: Non-expert AI users building machine learning models: a short survey of AutoML systems. https://doi.org/10.36227/techrxiv.170619255.52099254/v1
44. Patibandla, R.L., Srinivas, V.S., Mohanty, S.N., Pattanaik, C.R.: Automatic machine learning: an exploratory review. In: 2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), pp. 1–9. IEEE
45. Poulakis, Y., Doulkeridis, C., Kyriazis, D.: A survey on AutoML methods and systems for clustering **18**(5), 1–30. https://doi.org/10.1145/3643564
46. Quaranta, L., Azevedo, K., Calefato, F., Kalinowski, M.: A multivocal literature review on the benefits and limitations of industry-leading AutoML tools **178**, 107608. https://doi.org/10.1016/j.infsof.2024.107608
47. Rajenthiram, K., et al.: Towards continuous experiment-driven MLOps. https://doi.org/10.48550/arXiv.2503.03455
48. Ravishankar, S., Battineni, G.: A survey on recent advancements in auto-machine learning with a focus on feature engineering **4**(1), 56–63. https://doi.org/10.47852/bonviewJCCE3202720

49. Salehin, I., et al.: AutoML: a systematic review on automated machine learning with neural architecture search **2**. https://doi.org/10.1016/j.jiixd.2023.10.002
50. Shin, R., Packer, C., Song, D.: Differentiable neural network architecture search
51. Shujaat, S.: Automated machine learning in dentistry: a narrative review of applications, challenges, and future directions **15**(3), 273. https://doi.org/10.3390/diagnostics15030273
52. Thornton, C., Hutter, F., Hoos, H.H., Leyton-Brown, K.: Auto-WEKA: combined selection and hyperparameter optimization of classification algorithms. https://doi.org/10.48550/arXiv.1208.3719, issue: arXiv:1208.3719
53. Tian, J., Che, C.: Automated machine learning: a survey of tools and techniques **2**(6), 71–76. https://doi.org/10.70393/6a69656173.323336
54. Tornede, A., et al.: AutoML in the age of large language models: Current challenges, future opportunities and risks
55. Tuggener, L., et al.: Automated machine learning in practice: state of the art and recent results. In: 2019 6th Swiss Conference on Data Science (SDS), pp. 31–36. https://doi.org/10.1109/SDS.2019.00-11
56. Vazquez, H.C.: A general recipe for automated machine learning in practice. In: Bicharra Garcia, A.C., Ferro, M., Rodríguez Ribón, J.C. (eds.) Advances in Artificial Intelligence – IBERAMIA 2022, pp. 243–254. Springer. https://doi.org/10.1007/978-3-031-22419-5_21
57. Waring, J., Lindvall, C., Umeton, R.: Automated machine learning: review of the state-of-the-art and opportunities for healthcare **104**, 101822. https://doi.org/10.1016/j.artmed.2020.101822
58. Weerts, H., et al.: Can fairness be automated? Guidelines and opportunities for fairness-aware AutoML **79**, 639–677. https://doi.org/10.1613/jair.1.14747
59. Wen, R., Li, S.: Spatial decision support systems with automated machine learning: a review **12**(1), 12. https://doi.org/10.3390/ijgi12010012
60. Wever, M., Tornede, A., Mohr, F., Hüllermeier, E.: AutoML for multi-label classification: overview and empirical evaluation **43**(9), 3037–3054. https://doi.org/10.1109/TPAMI.2021.3051276
61. Yakovlev, A., et al.: Oracle AutoML: a fast and predictive AutoML pipeline **13**, 3166–3180. https://doi.org/10.14778/3415478.3415542
62. Yao, Q., et al.: Automated machine learning: from principles to practices
63. Yuan, H., Yu, K., Xie, F., Liu, M., Sun, S.: Automated machine learning with interpretation: a systematic review of methodologies and applications in healthcare **2**(3), 205–237. https://doi.org/10.1002/med4.75, _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/med4.75
64. Zhang, Z., Wang, X., Zhu, W.: Automated machine learning on graphs: a survey. In: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, pp. 4704–4712. https://doi.org/10.24963/ijcai.2021/637
65. Zheng, R., Qu, L., Cui, B., Shi, Y., Yin, H.: AutoML for deep recommender systems: a survey. https://doi.org/10.48550/arXiv.2203.13922
66. Zhuhadar, L.P., Lytras, M.D.: The application of AutoML techniques in diabetes diagnosis: current approaches, performance, and future directions **15**(18), 13484. https://doi.org/10.3390/su151813484
67. Zöller, M.A., Huber, M.F.: Benchmark and survey of automated machine learning frameworks **70**, 409–472. https://doi.org/10.1613/jair.1.11854

FAIR-ER Data: Proposing a Data Model for Data Cooperatives

Francisco Mendonça[1,2(✉)], Nabil Abdennadher[2], and Giovanna Di Marzo Serugendo[1]

[1] University of Geneva, Geneva, Switzerland
francisco.mendonca@hesge.ch
[2] University of Applied Sciences and Arts of Western Switzerland, Delémont, Switzerland

Abstract. Data cooperatives empower communities to collectively govern data, but existing frameworks like FAIR (Findable, Accessible, Interoperable, Reusable) lack mechanisms for equitable and democratic control of data. We present FAIR-ER, extending FAIR with two dimensions: *Equitability* - ensuring fair access and benefit-sharing regardless of individual contributions, and *Regulability* - enforcing member sovereignty through governance rules. The framework bridges critical gaps between FAIR's technical focus and cooperative values, demonstrated through both its operational building blocks and governance mechanisms. As a conceptual contribution, FAIR-ER provides a foundation for both technical implementation and future empirical validation in cooperative environments, advancing the development of ethically-grounded data ecosystems.

Keywords: Data Cooperatives · FAIR Principles · Data Governance · Data Management

1 Introduction

The concentration of data among a few corporations has created a demand for democratic alternatives. Data cooperatives - member-owned, democratically governed organizations - address this need by pooling data to achieve shared goals (e.g., economic development, scientific research, or community innovation) [1,2]. Unlike traditional governance models, data cooperatives return control to data generators [3,4], leveraging collective potential while prioritizing equity and sovereignty. However, three persistent integration challenges remain: policy enforcement of collective decisions, interoperable systems with member-controlled access, and verifiable benefit tracking [3].

To truly realize their potential, data cooperatives must follow a set of principles that guide their governance, operations, and member relations. To be classed as a true *cooperative*, a data cooperative must aspire to uphold the

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 329–336, 2026.
https://doi.org/10.1007/978-3-032-04403-7_27

seven principles of cooperatives [5], which include *open participation*, *member-led decision-making*, *shared financial stakes*, *operational independence*, *ongoing learning*, *collaboration between groups*, and *community focus*. These principles are essential for building trust among members, ensuring equitable distribution of benefits, and fostering long-term sustainability [6]. Yet, these principles lack technical frameworks to operationalize data-specific challenges like interoperability or access control - a gap FAIR-ER addresses.

Effective data governance in a cooperative requires adopting structured best practices. The FAIR principles (Findable, Accessible, Interoperable, Reusable) provide a proven stewardship framework for collaborative environments [7,8]. By ensuring data is discoverable, conditionally accessible, interoperable across systems, and reusable for multiple purposes, FAIR promotes transparency, collaboration, and value maximization [9]. However, as shown in [10], standard FAIR implementations assume some sort of centralized control rather than democratic governance, and neutral data sharing rather than equitable benefit models - limitations that prevent direct application in cooperatives.

This paper examines how FAIR data principles can be adapted to meet data cooperatives' unique requirements. While FAIR provides a foundational framework, we demonstrate the necessity of augmenting it with two critical dimensions: (1) Equitability and (2) Regulability. After analyzing standard FAIR principles in cooperative contexts, we introduce FAIR-ER—our extended framework addressing ethical and social governance gaps.

2 FAIR Data in the Context of Data Cooperatives

In data cooperatives, FAIR principles (Findable, Accessible, Interoperable, and Reusable) serve as essential pillars for maximizing shared data value [7]. While these principles provide a technical foundation for data stewardship, their conventional implementation requires careful adaptation to address two core cooperative requirements: (1) preserving member sovereignty, and (2) enabling collective governance. We first examine how Findability and Accessibility must be re-imagined to meet these needs, before later discussing their integration with cooperative-specific dimensions.

2.1 Findability

Within a data cooperative, Findability serves a fundamental requirement for effective data sharing and collaboration. When data assets lack systematic discoverability, they risk becoming isolated in organizational silos, which undermines cooperative efficiency and limits the collective value derived from shared resources [3]. To address this challenge, three key mechanisms must be implemented. First, comprehensive metadata standards must be established, capturing essential attributes such as data provenance, content characteristics, and usage context. Second, persistent unique identifiers must be assigned to both data and its associated metadata to ensure reliable identification and

retrieval. Third, a centralized metadata registry must be maintained, enabling cooperative-wide searchability while preserving appropriate access controls.

A key distinction emerges in the data cooperative context. While the original Findability principle often emphasizes open access to data, cooperatives must operationalize this concept differently. Here metadata availability to all members must be mandatory, whereas access to the underlying data remains at the discretion of individual members. This architectural separation provides dual benefits: it maintains the cooperative's discoverability requirements while upholding member data sovereignty. Members retain autonomy over their data assets while contributing to the collective knowledge base through shared metadata. This approach not only facilitates resource discovery but also aligns with the cooperative's fundamental principles of equitable participation and member-controlled governance.

2.2 Accessibility

In data cooperatives, Accessibility represents a complicated equilibrium between collaborative data sharing and individual sovereignty. Unlike open-access models, it ensures authorized members can retrieve necessary data while adhering to governance policies [11]. This requires not only technical retrieval mechanisms but also transparent procedures for access requests and approvals, fostering trust and enabling data driven-collaboration [12].

As presented in the previous Sect. 2.1, a key distinction lies in the separation between metadata and data access. While metadata must remain openly available to facilitate discovery, the access to underlying dataset may be restricted based on member-defined condition, legal constraints, or sensitivity. This framework respects data sovereignty while maintaining collective utility.

Implementation demands three core components: A centralized access system with authentication and authorization protocols, governed either by data owners or a stewardship committee; Granular access tier differentiating between data categories and use cases (discussed later); Persistent metadata preservation, ensuring institutional knowledge persists even when datasets are modified or deprecated.

This approach maximizes cooperative value by balancing discoverability with controlled utilization. Members contribute to shared resources without relinquishing autonomy, and the cooperative maintains an auditable, scalable access framework. By design, it avoids the pitfalls of either excessive restriction (which stifles collaboration) or unregulated openness (which risks sovereignty), instead enabling ethical, purposeful data exchange.

2.3 Interoperability

Interoperability enables seamless data exchange and utilization across diverse systems within a cooperative framework. It requires standardized formats and structures that allow data to be meaningfully interpreted, combined, and repurposed beyond its original context [13]. For data cooperatives, this capability is

fundamental - it ensures all members can effectively use shared data regardless of its source, thereby maximizing collaborative potential and collective value.

The basis for interoperability lies in establishing a common semantic framework, typically implemented through shared ontologies or data dictionaries developed and maintained cooperatively [14]. These reference systems provide consistent definitions for data types, fields, and relationships, creating a unified understanding across the cooperative. Importantly, such frameworks can be selectively shared with external partners when appropriate, extending interoperability benefits beyond the cooperative itself.

The advantages of interoperability are threefold. First it significantly enhances collaborative potential by enabling members to share insights and combine datasets with minimal friction. Second, it actively prevents data siloing by ensuring compatibility across different systems and platforms. Finally, it increases the economic and analytical value of cooperative data by making it more accessible and usable to both members and authorized external stakeholders. This creates an environment where data retains its meaning and utility across various applications and users, while maintaining the cooperative's governance structure and member control.

2.4 Reusability

Reusability maximizes data utility across multiple contexts by ensuring comprehensive documentation of both data and metadata [7]. As established in Sect. 2.3, this builds on interoperability standards like shared data dictionaries, while adding critical dimensions of provenance and lifecycle management. For cooperatives, effective reuse requires robust provenance tracking, documenting data from creation to archiving. This transparency enhances trust while enabling members to assess data quality and suitability for new applications. Metadata plays an equally vital role - detailed descriptions, usage licenses, and contextual attributes ensure proper categorization and governance.

Practical implementation requires balancing accessibility with sustainability. Smaller cooperatives should establish clear data lifecycles, archiving older datasets to preserve value without maintaining costly active storage. Secure destruction protocols for obsolete data further optimize resource allocation.

Together, these measures create an efficient cycle: standardized metadata and provenance information increase reuse potential, while lifecycle management ensures cost-effective stewardship. The result is a cooperative ecosystem where data maintains long-term utility without imposing unsustainable maintenance burdens.

3 FAIR-ER Data Model for Data Cooperatives

Data cooperatives face unique governance challenges in balancing open collaboration with member sovereignty. While FAIR principles provide technical foundations for data sharing, they insufficiently address the socioeconomic realities of

cooperative models [7]. True data cooperatives must operationalize seven principles: voluntary participation, democratic control, equitable benefits, autonomy, education, inter-cooperation, and community focus [15].

This creates inherent tensions. For instance, FAIR's reusability principle - while valuable for knowledge growth - requires careful adaptation to protect member privacy and control. Effective implementations demand: (1) granular consent mechanisms, (2) transparent usage policies, and (3) participatory decision-making around data governance. Furthermore, cooperative principles necessitate governance systems accessible to non-technical members through education and intuitive tools [16].

We address these gaps through the FAIR-ER framework, extending FAIR with two critical dimensions: *Equitability*: ensuring fair benefit distribution and democratic governance; and *Regulatable*: enabling member-controlled data sovereignty. The following sections detail how FAIR-ER reconciles data utility with member rights through structured yet flexible mechanisms.

3.1 Equitability

Equitability extends beyond basic cooperation to ensure fair distribution of both benefits and governance responsibilities in data cooperatives [16]. This principle addresses two critical dimensions: balancing unequal member contributions with universal benefit, and embedding democratic control in data governance.

Three core principles operationalize this concept. First, *Universal Metadata Access* ensures all members discover data resources through standardized metadata, implemented via a federated catalog combined with schema-on-read adapters to coordinate heterogeneous member datasets without imposing upfront standardization, and OAuth2-scoped GraphQL endpoints that enforce role-based metadata visibility. This decouples discovery from access, preventing silos while enforcing fine-grained dataset policies. Second, *Fair Benefit Distribution* guarantees proportionally value allocation to members based on contributions. A blockchain-based tracking system with smart contracts [17] automatically calculates and distributes benefits according to predefined rules, preventing disproportionate advantages for high contributors. Third, *Democratic Governance* ensures collective oversight of data policies - from access rules to usage terms - through two key mechanisms: policy-as-code automation to enforce member-approved rules, and threshold-weighted voting (typically requiring 60–75% supermajorities) for policy changes. Cryptographic audit trails provide transparency, preventing unilateral control by any individual or subgroup.

This principle complements technical Accessibility by adding ethical constraints. Where Accessibility establishes how data can be retrieved, Equitability governs *who* may access it and *how benefits are allocated*. This dual structure ensures cooperatives maintain both operational efficiency and adherence to their socio-economic values - technical systems enabling sharing, while equitable principles ensure sharing occurs justly.

3.2 Regulability

The Regulability principle provides the enforcement infrastructure to realize Equitability's fairness goals through procedural safeguards. Where Equitability defines what constitutes fair outcomes, Regulability specifies how these outcomes are technically and institutionally guaranteed - particularly through member-controlled governance and adaptive compliance mechanisms. To achieve this, a tripartite governance architecture comprising democratic oversight bodies, graduated access controls, and adaptive compliance verification systems should be employed.

The regulatory framework's five principles are implemented through specific technical and governance mechanisms that work in concert. *Sovereignty Preservation* ensures members retain ultimate veto power over data's usage conditions. This is implemented through a decentralized identity management system, where each member holds cryptographic keys enabling individual veto rights, which could be enforced via smart contracts that require explicit consent for policy changes affecting their data. *Transparent Governance* mandates immutable audit trails for all access decisions and policy modifications. Current implementations use blockchain-based ledgers that create tamper-proof transparent records of all governance actions. *Purpose Limitation* technically enforces strict boundaries on data utilization to member-approved applications, technically enforced through attribute-based access control systems that embed approved usage parameters directly into data access protocols. *Dynamic Rule Evolution* institutionalizes periodic policy reviews through participatory member referendums, which should be supported by an automated governance platform, which could schedule regular policy review cycles. Finally, *Contextual Permissions* implement granular access controls using a role-based access matrices with real-time context evaluation engines that respect organizational roles while maintaining collective oversight [18].

These principles work synergistically through the Data Stewardship Committee's operations, which translate cooperative values into executable rules. For instance, while an interoperable system might technically enable broad dataset linkages, *Purpose Limitation* and *Sovereignty Preservation* collectively determine whether such combination server equitable outcomes. Similarly, metadata designed for reusability becomes actionable only when *Contextual Permission* and *Transparent Governance* verify the requesting member's legitimate need and the cooperative's collective benefit.

The regulability framework establishes a systematic approach to governance where technical systems are explicitly designed to enforce cooperative principles. By embedding these safeguards into both institutional processes and technical architectures, data cooperatives can ensure their operational infrastructure maintains fidelity to member-defined values. This creates verifiable accountability in data sharing practices while preserving the flexibility needed to adapt to evolving requirements.

4 Conclusion

Data cooperatives have emerged as significant mechanisms for addressing growing societal concerns about data sovereignty, equitable value distribution, and democratic governance in an increasingly data-driven economy. While structurally innovative, these cooperatives face fundamental challenges in reconciling the technical requirements of data utility with the principles of cooperative organization. The proposed FAIR-ER framework addresses this tension by augmenting the established FAIR principles with two critical dimensions essential for cooperative contexts. First, Equitability ensures distributive justice via baseline access rights and proportional benefit allocation. Second, Regulability enforces member sovereignty through transparent, adaptive technical controls and institutional systems.

These extensions represent advances in three key respects. First, they bridge the gap between FAIR's technical focus on data characteristics and the social contracts required for cooperative governance. Second, they provide operational solutions to the redistribution challenges that undermine many data trust models.

Future research should investigate the implementation challenges arising from the inherent tension between granular member control and cooperative scalability. This includes examining technological architectures capable of enforcing dynamic governance policies while remaining accessible to non-technical members. Equally important will be developing pedagogical frameworks to facilitate meaningful member participation in complex technical governance decisions.

FAIR-ER establishes both a theoretical foundation and practical pathway for implementing values-driven data governance, offering an alternative to extractive data economies while maintaining research rigor.

References

1. Hardjono, T., Pentland, A.: Data cooperatives: towards a foundation for decentralized personal data management (2019)
2. Data co-ops: How cooperative structures can support women's empowerment — brookings.edu. https://www.brookings.edu/articles/data-co-ops-how-cooperative-structures-can-support-womens-empowerment/. Accessed 18 Feb 2025
3. Bühler, M.M., Calzada, I., Cane: unlocking the power of digital commons: data cooperatives as a pathway for data sovereign. Innovative Equitable Dig. Commun. Digital **3**(3), 146–171 (2023)
4. Girish, S, Avery, M.: Data cooperative: Enabling meaningful collective negotiation of data rights for communities. SSRN Electron. J. (2023)
5. Mendonca, F., DiMarzo, G., Abdennadher, N.: Democratic models for ethical data stewardship. Data Cooperatives (2025)
6. Linåker, J., Runeson, P:. Sustaining open data as a digital common – design principles for common pool resources applied to open data ecosystems. In: Proceedings of the 18th International Symposium on Open Collaboration, OpenSym 2022. ACM (2022)

7. Wilkinson, M.D., et al.: The fair guiding principles for scientific data management and stewardship. Sci. Data **3**(1) (2016)
8. FAIR Principles - GO FAIR — go-fair.org. https://www.go-fair.org/fair-principles/. Accessed 18 Feb 2025
9. Data with principles: How to be FAIR, and why you should CARE — SFU Library — lib.sfu.ca. https://www.lib.sfu.ca/help/publish/scholarly-publishing/radical-access/fair-and-care-data. Accessed 18 Feb 2025
10. Ugochukwu, A.I., Phillips, P.W.B.: Open data ownership and sharing: challenges and opportunities for application of fair principles and a checklist for data managers. J. Agric. Food Res. **16**, 101157 (2024)
11. Shanahan, H., Bezuidenhout, L.: Rethinking the a in fair data: issues of data access and accessibility in research. Front. Res. Metrics Analytics **7** (2022)
12. Petreski, D., Cheong, M.: Data cooperatives: a conceptual review. ICIS 2024 (2024)
13. da Silva, G., Leal, S., Guédria, W., Panetto, H.: Interoperability assessment: a systematic literature review. Comput. Ind. **106**, 111–132 (2019)
14. Menzel, K., Törmä, S., Markku, K., Tsatsakis, K., Hryshchenko, A., Lucky, M.N.: Linked data and ontologies for semantic interoperability. In: Daniotti, B., Lupica Spagnolo, S., Pavan, A., Bolognesi, C.M. (eds.) Innovative Tools and Methods Using BIM for an Efficient Renovation in Buildings, pp. 17–28. Springer (2022). https://doi.org/10.1007/978-3-031-04670-4_2
15. Bruno Carballa Smichowski: Alternative data governance models: moving beyond one-size-fits-all solutions. Intereconomics **54**(4), 222–227 (2019)
16. Cooper, C., Martin, V., Wilson, O., Rasmussen, L.: Equitable data governance models for the participatory sciences. Commun. Sci. **2**(2) (2023)
17. Nairi, C., Cicioğlu, M., Undefinedalhan, A.: Smart blockchain networks: revolutionizing donation tracking in the web 3.0. Comput. Commun. **228**, 107972 (2024)
18. Davis, S.: How pomerium enforces real-time, context-based access

GLOVES 2.0: Global Counterfactual-Based Visual Explanations

Nikolas Theologitis(✉), Panagiotis Gidarakos, Stavros Maroulis, Loukas Kavouras, Giorgos Giannopoulos, and George Papastefanatos

Athena Research Center, 15125 Athens, Greece
n.theologitis@athenarc.gr

Abstract. We present an updated version of Global Counterfactual-based Visual Explanations (GLOVES), a visualization platform for exploring global counterfactual explanations (GCE), designed to enhance the interpretability of machine learning models at the population level. GLOVES enables users to upload datasets and machine learning models or choose from preloaded options to interactively generate, analyze, and compare counterfactual strategies using diverse GCE algorithms. In this extended version, GLOVES includes two additional GCE algorithms, GLOBE-CE and Group-CF, which broaden the spectrum of supported methods for generating global recourse. We also introduce a comparative view that allows users to visually compare the three supported algorithms based on how they affect the population, in regards to the number of global counterfactual actions generated. Finally, we have refined the look and feel of the interface, improving usability and making key visualizations more accessible and engaging. With these enhancements, GLOVES continues to offer an effective and extensible platform to experiment with and understand global counterfactual explanations in AI-driven decision systems.

Keywords: Visualization · Global Counterfactuals · Explainability

1 Introduction

Machine learning models are increasingly deployed in high-stakes domains such as loan approvals, hiring, and healthcare. As these systems influence real-world outcomes, users and regulators need to understand how input features influence predictions and how they can be changed to obtain more favorable outcomes—an idea known as recourse [14]. Counterfactual explanations have become a widely studied approach for supporting recourse due to their interpretability [18], actionability [17], usefulness in fairness auditing [9] and model debugging.

A counterfactual explanation answers the "what-if" question: how would a prediction change if specific features were different? These are typically expressed as counterfactual actions, or feature changes that flip an unfavorable decision to a favorable one.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 337–344, 2026.
https://doi.org/10.1007/978-3-032-04403-7_28

Counterfactual methods are typically divided into local and global approaches. Local methods provide personalized recourse for individuals, while global methods define generalizable actions for subgroups. Effective global explanations aim to maximize recourse coverage, minimize the cost of feature changes, and remain interpretable through compact action sets.

While considerable progress has been made in algorithmic methods for local counterfactual explanations and their visualization, the importance of global counterfactuals has only recently gained attention in the literature [6,8–11,15].

In this work, we present an extended version of GLOVES[3]. What sets GLOVES apart is its experimentation-driven interface, which allows users to interactively test different GCE configurations, examine counterfactual actions, and evaluate their population-wide effects through interactive visualizations. This version introduces support for three state-of-the-art global counterfactual algorithms, **GLANCE** [8] which was included in the first version of the platform and the two recent additions: **GLOBE-CE** [12] and **Group-CF** [19]. To aid comparison, we introduce a new visual component that highlights trade-offs between the GCE algorithms in terms of cost, effectiveness, and the number of actions required. These enhancements make GLOVES a flexible and powerful tool for analyzing and refining global counterfactual explanations. In this version, we support models from scikit-learn version 1.1.3 with Python 3.9.20, requiring them to be in a pickle format (version 4.0) for compatibility.

Related Work. Visualization techniques are extensively utilized in explainable AI (XAI) to intuitively communicate complex model explanations. Local counterfactual explanations, which identify minimal changes to an instance's input to alter a model's prediction, have been widely explored through interactive visual systems. These tools integrate counterfactual methods with visual analytics frameworks to enhance interpretability and improve user understanding of predictive outcomes [1]. Other tools, offer a visual analytics interface for exploring aggregated local counterfactual explanations [4], unlike GLOVES, which focuses on global strategies that apply across groups or the entire population.

Recent research has emphasized the utility of counterfactual visualizations in exploratory data analysis. These visualizations help users avoid incorrect assumptions, identify confounding variables, improve causal inference, and support decision-making processes [7]. Additionally, counterfactual reasoning has been applied to specialized domains such as education [2] and recommendation systems, where feature manipulation enhances user engagement and outcomes [5]. Furthermore, visualizing decision boundaries has enabled "what-if" analyses, offering deeper insights into model behavior [16].

2 Global Counterfactual Methods

Global counterfactual explanation (GCE) methods provide population-level recourse by identifying generalizable actions that can shift unfavorable model predictions toward desired outcomes. Rather than offering individualized adjustments, these methods define strategies applicable across subgroups or entire

populations. The design of GCE methods often requires balancing three key objectives: effectiveness, cost, and size. **Effectiveness:** Measures how well the counterfactual actions achieve the desired outcomes for the population, indicating the breadth of recourse provided. **Cost:** Assesses the magnitude of changes required to implement the recourse, reflecting the practicality and feasibility of the solutions. **Size (Interpretability):** Refers to the number of distinct actions or rules generated, with smaller sets being more interpretable and easier to communicate to stakeholders.

GCE methods vary in how they generate and structure their actions. Rule-based approaches, such as AReS [15] and Fast AReS [11], construct hierarchical rules that assign minimal changes to subgroups. Partition-based methods like CET [6] divide the feature space into disjoint regions, each assigned a single counterfactual action to ensure consistency.

Clustering-based methods, such as GLANCE [8], group individuals by similarity in both feature and action spaces. For each cluster, GLANCE generates a representative counterfactual action that optimizes effectiveness, cost (using the ℓ_1 norm), and interpretability. This results in a compact action set that reflects model behavior while remaining comprehensible.

Group-CF [19] generates high-coverage actions for small groups of similar instances. It identifies common feature changes from individual counterfactuals and composes a single group-level action that is likely to be valid for all members.

GLOBE-CE [12] defines counterfactual actions as movement along shared directions in the feature space. Instead of assigning the same transformation to everyone, GLOBE-CE computes a global direction vector and allows individual-specific magnitudes along that direction, enabling a more flexible yet consistent transformation.

To ensure fair comparison, all three algorithms—GLANCE, Group-CF, and GLOBE-CE—are evaluated using consistent metrics: cost (measured via ℓ_1-norm) and effectiveness (proportion of the affected population receiving valid recourse). This alignment isolates performance differences to the algorithms themselves, enabling clear, side-by-side trade-off comparisons and helping users select the most suitable method for their needs.

It is important to note that the performance of these global counterfactual methods varies with dataset size and dimensionality. As data grows, all methods incur higher computational costs, with Group-CF and GLOBE-CE being particularly sensitive to scale. Among them, GLANCE demonstrates the most favorable trade-off between runtime and explanation quality, making it the most scalable of the three in practice.

3 GLOVES Architecture

Figure 1 illustrates the architecture of the GLOVES system, which consists of a backend API for executing explainability methods and a web-based user interface for conducting counterfactual analyses and visualizing results. The frontend provides users with tools such as the *Dataset and Model Selector* for uploading or

choosing datasets and machine learning models, and the *Explainability Parameters Configuration* module for defining analysis scenarios and parameters. In this extended version, users can also select from multiple global counterfactual explanation (GCE) algorithms to apply to the selected data and model.

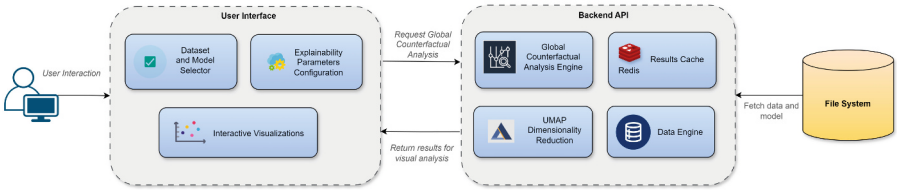

Fig. 1. GLOVES: Framework Architecture

Once a dataset and model are selected, the frontend sends a request to the backend API, which reads the data and interacts with the model to compute predicted labels and identify the affected population. The system also offers optional dimensionality reduction via UMAP [13], providing a low-dimensional embedding for visual exploration.

Users can configure and compare multiple GCE scenarios through the interface. These are processed by the *Global Counterfactual Analysis Engine*, which applies the selected algorithm to compute explanations. A lightweight *Results Cache* stores previously computed outputs to improve responsiveness. Computed results are visualized through the *Interactive Visualization and Insights* module, enabling users to explore suggested actions, evaluate impact, and compare explanations across different algorithms.

4 GLOVES User Interface

This section presents the visual interface of GLOVES, which supports the interactive exploration and analysis of global counterfactual explanations. The interface is organized into four main views—Select Dataset and Model, Explore Dataset, Analyze Counterfactuals and Compare Methods—each corresponding to a specific stage in the analysis workflow.

The Explore Dataset view (Fig. 2) enables users to examine the dataset and its feature distributions. It includes an interactive table with filtering and sorting, and highlights individuals with adverse outcomes. Users can switch between affected and non-affected subgroups for focused exploration. A scatter plot visualization supports feature analysis and includes UMAP-based dimensionality reduction for high-dimensional data.

The *Analyze Counterfactuals* view allows users to run different global counterfactual algorithms. Users can select a method, configure parameters (e.g., number of actions, feature subsets), and compute results. Key metrics such

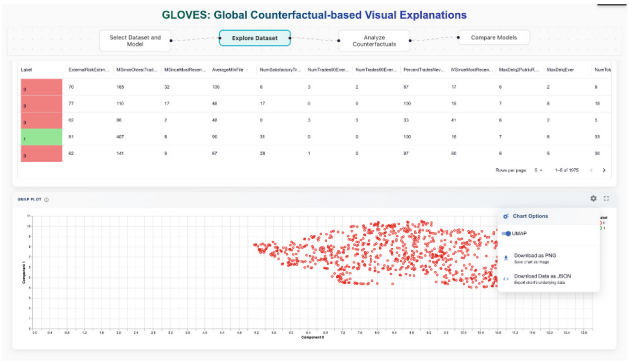

Fig. 2. The *Explore Dataset* view in GLOVES, allowing users to preview the dataset, examine feature relationships via scatter plots, and analyze the affected population.

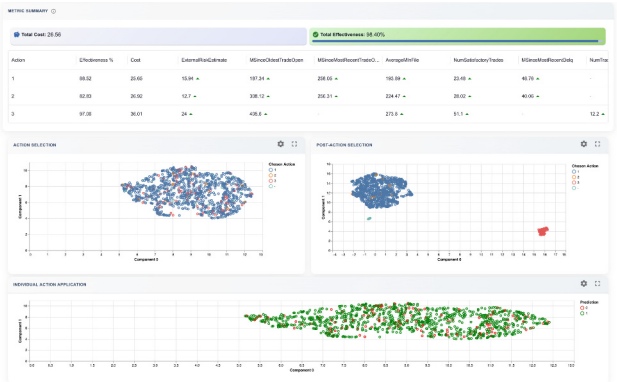

Fig. 3. The *Analyze Counterfactuals* view in GLOVES, showcasing a single configuration's details, including generated counterfactual actions, their impact on the affected population, and visual insights.

as effectiveness and cost are shown for each configuration in both tabular and visual form. Interactive plots (scatter and bar charts) visualize population shifts and trade-offs between strategies. Users can also apply individual actions across affected instances to isolate their impact and refine strategies.

Finally, *Compare Methods* view supports visual benchmarking of algorithms under identical configurations (e.g., fixed number of actions). In Fig. 4, each method is run with six actions, and results are plotted as effectiveness (y-axis) versus cost (x-axis). Each point represents a single action, and the cumulative curve shows how additional actions improve coverage and cost. Ideally, strong methods appear toward the upper-left, indicating high effectiveness with low cost. In the example, GLANCE (blue) shows robust, consistent improvement and achieves 100% coverage at moderate cost. Group-CF (red) excels at low-cost actions but caps effectiveness around 60%, suggesting limited reach. GLOBE-CE

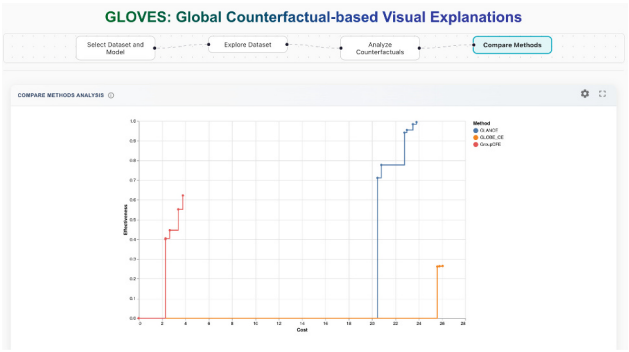

Fig. 4. The *Compare Methods* view in GLOVES, visualizing the cost-effectiveness trade-offs across different global counterfactual algorithms.

(orange) generates few actions and struggles with both cost and effectiveness, possibly due to its direction-based formulation.

By aligning evaluation metrics and presenting them through a unified visual comparison, this view helps users understand trade-offs in cost and coverage, making it easier to select the algorithm that best aligns with their goals—whether maximizing effectiveness, minimizing cost, or balancing both. The tool and its functionalities can be accessed online at http://gloves.imsi.athenarc.gr. The open source code can be accessed at https://github.com/PanosGidarakos/GLoVE/tree/main. A demonstration video of the tool is available here.

5 Demonstration Outline

In this section, we outline the interactive demonstration scenario. Attendees will engage with GLOVES to explore and analyze datasets from various domains using global counterfactual explanations. For instance, they may examine the HELOC (Home Equity Line of Credit) dataset, which includes financial attributes used to predict loan repayment likelihood.

Initially, attendees will be introduced to the platform and its key components. They will then interact with the tool to perform the following operations:

- Select a dataset and a trained ML model (e.g., XGBoost) or upload their own data and models.
- Preview the dataset in a tabular format, explore relationships between features using scatter plots, and apply dimensionality reduction techniques (e.g., UMAP) to gain insights into the affected and non-affected populations.
- Configure global counterfactual analysis by selecting parameter values, such as the number of actions or feature subsets to include in the analysis.
- Generate global counterfactual explanations and compare configurations based on metrics such as cost, effectiveness, and interpretability.

– Examine detailed counterfactual actions, assess their impact on the affected population, and visually inspect transformations in the feature space (Fig. 3).
– Perform fine-grained analysis by applying individual actions to the dataset to observe their isolated effects and evaluate their effectiveness.
– Use the dedicated comparison view to analyze the trade-offs between multiple algorithms based on shared metrics, helping identify which method best fits specific goals or constraints.

6 Conclusion

In this work, we presented an enhanced version of GLOVES, a visualization platform for exploring global counterfactual explanations (GCE). By integrating interactive visualizations with explainability algorithms, GLOVES helps users analyze model behavior and understand recourse strategies at the population level. This update introduces support for two additional GCE methods—GLOBE-CE and Group-CF—and adds a comparative view for comparing the algorithms in terms of cost, effectiveness, and interpretability. Future work will focus on customizable cost functions, enabling users to assign weights to features based on context-specific factors, improved scalability, and expanded visualization features to support more complex data and models, further establishing GLOVES as a flexible and practical tool for explainable AI. Also, we plan to address system-level scalability within the GLOVES platform. This includes optimizing performance for large, high-dimensional datasets and benchmarking responsiveness to ensure smooth user interaction in data-intensive, real-world settings.

Acknowledgments. This work was partially supported by AutoFair project (EU Horizon program, GA 101070568) and ExtremeXP project (EU Horizon program, GA 101093164).

References

1. Cheng, F., Ming, Y., Qu, H.: DECE: decision explorer with counterfactual explanations for machine learning models. IEEE Trans. Vis. Comput. Graph. **27**(2), 1438–1447 (2021). https://doi.org/10.1109/TVCG.2020.3030342. https://ieeexplore.ieee.org/document/9229232
2. Garcia-Zanabria, G., Gutierrez-Pachas, D.A., Camara-Chavez, G., Poco, J., Gomez-Nieto, E.: SDA-Vis: a visualization system for student dropout analysis based on counterfactual exploration
3. Gidarakos, P., Theologitis, N., Maroulis, S., Kavouras, L., Giannopoulos, G., Papastefanatos, G.: Gloves: global counterfactual-based visual explanations (2025)
4. Gomez, O., Holter, S., Yuan, J., Bertini, E.: Advice: aggregated visual counterfactual explanations for machine learning model validation (2021). https://arxiv.org/abs/2109.05629
5. Jeon, G., Kim, S., Lee, S.: Interactive feedback loop with counterfactual data modification for serendipity in a recommendation system. Int. Hum.–Comput. Interact. **40**(19), 5585–5601 (2024). https://doi.org/10.1080/10447318.2023.2238369

6. Kanamori, K., Takagi, T., Kobayashi, K., Ike, Y.: Counterfactual explanation trees: transparent and consistent actionable recourse with decision trees. In: International Conference on Artificial Intelligence and Statistics, pp. 1846–1870. PMLR (2022)
7. Kaul, S., Borland, D., Cao, N., Gotz, D.: Improving visualization interpretation using counterfactuals. IEEE Trans. Vis. Comput. Graph. **28**(1), 998–1008 (2022). https://doi.org/10.1109/TVCG.2021.3114779. https://ieeexplore.ieee.org/abstract/document/9552190
8. Kavouras, L., et al.: Glance: global actions in a nutshell for counterfactual explainability (2024). https://arxiv.org/abs/2405.18921
9. Kavouras, L., et al.: Fairness aware counterfactuals for subgroups. arXiv preprint arXiv:2306.14978 (2023)
10. Lakkaraju, H., Kamar, E., Caruana, R., Leskovec, J.: Faithful and customizable explanations of black box models. In: Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society, pp. 131–138 (2019)
11. Ley, D., Mishra, S., Magazzeni, D.: Global counterfactual explanations: investigations, implementations and improvements. In: ICLR Workshop on Privacy, Accountability, Interpretability, Robustness, Reasoning on Structured Data (2022)
12. Ley, D., Mishra, S., Magazzeni, D.: GLOBE-CE: atranslation based approach for global counterfactual explanations. In: Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., Scarlett, J. (eds.) Proceedings of the 40th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 202, pp. 19315–19342. PMLR (2023). https://proceedings.mlr.press/v202/ley23a.html
13. McInnes, L., Healy, J., Melville, J.: UMAP: uniform manifold approximation and projection for dimension reduction (2020). https://arxiv.org/abs/1802.03426
14. Miller, T.: Explanation in artificial intelligence: insights from the social sciences. Artif. Intell. **267**, 1–38 (2019)
15. Rawal, K., Lakkaraju, H.: Beyond individualized recourse: interpretable and interactive summaries of actionable recourses. Adv. Neural. Inf. Process. Syst. **33**, 12187–12198 (2020)
16. Sohns, J.T., Garth, C., Leitte, H.: Decision boundary visualization for counterfactual reasoning. Comput. Graph. Forum **42**(1), 7–20 (2023) https://doi.org/10.1111/cgf.14650, https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.14650, _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14650
17. Ustun, B., Spangher, A., Liu, Y.: Actionable recourse in linear classification. In: Proceedings of the Conference on Fairness, Accountability, and Transparency, pp. 10–19 (2019)
18. Wachter, S., Mittelstadt, B., Russell, C.: Counterfactual explanations without opening the black box: automated decisions and the GDPR. Harv. JL Tech. **31**, 841 (2017)
19. Warren, G., Keane, M.T., Gueret, C., Delaney, E.: Explaining groups of instances counterfactually for XAI: a use case, algorithm and user study for group-counterfactuals. arXiv preprint arXiv:2303.09297 (2023)

Temporal Harmonization of Heterogeneous Software Logs: A Unified Model for Time-Series Analysis

Milad Abdullah[1(✉)], Petr Hnětynka[1], and Jiyan Salim Mahmud[2]

[1] Charles University, Prague, Czech Republic
{milad.abdullah,petr.hnetynka}@matfyz.cuni.cz
[2] Eötvös Loránd University, Budapest, Hungary
jiyan@inf.elte.hu

Abstract. Modern software systems continuously generate vast amounts of heterogeneous logs and measurement data, typically collected for monitoring, debugging, and diagnostics. These non-functioning logs often exist in disparate formats and lack temporal alignment, making them challenging to utilize directly for modeling and analysis. This work proposes a unified model that reshapes such raw software data into a time-series structure with minimal or no preprocessing required. By treating all logged information—regardless of source, format, or granularity—as temporal events, the model preserves the chronological context while normalizing irregularities in structure and frequency. The resulting time-series representation enables direct ingestion by time-aware machine learning models and facilitates downstream tasks such as anomaly detection, predictive maintenance, and system behavior modeling.

Keywords: Data Intensive Systems · Software Logs · Time Series Dataset Model

1 Introduction

Modern software systems generate vast and continuous streams of operational data ranging from logs to metrics originating from diverse sources and in inconsistent formats. While this data holds significant potential for understanding system behavior, it is often difficult to leverage effectively due to its heterogeneity and lack of temporal alignment.

As mentioned in the survey about automated log analysis [10], existing approaches "*only support common log types such as Apache logs*" and/or "*rely on regexs and ad-hoc scripts provided by developers*". However, this limits scalability and impedes broader adoption in real-world settings and generally makes the log usage unsustainable. Thus, there is a growing need [12,16] for a unified and sustainable way to make this data more accessible for analysis.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 345–353, 2026.
https://doi.org/10.1007/978-3-032-04403-7_29

In this work, we propose a conceptual model that treats all logged and measured information as temporal events, regardless of their original format or source. This approach reshapes disparate data into a coherent time-series structure with little to no preprocessing, preserving chronological context and normalizing inconsistencies in frequency and structure.

As a short paper, our focus is on introducing the core idea rather than presenting a full implementation or evaluation. We aim to outline a foundational concept that can support downstream tasks such as anomaly detection and predictive maintenance, while encouraging further exploration into scalable, time-aware machine learning solutions for software system analysis.

2 Examples and Motivation

To illustrate the situation, we describe several public datasets commonly used to train models for time-series analytics.

The **Mozilla performance time series dataset** [14] is a collection of the Mozilla browser performance data collected over a one-year period (May 2023 to May 2024). The dataset includes performance time series, alerts, alert summaries, and bug reports. Each performance time series is organized by its corresponding repository, with one CSV file per series. In contrast, all alerts and all bugs are stored in a single CSV file.

However, for sustainable use, the dataset would benefit from additional contextual information to enhance its long-term utility and reliability. While timestamps are present and can be extracted, the dataset is not structured for direct use as a time series. Furthermore, there is limited information about the system configuration or operating conditions under which the data was collected. Including such metadata would improve interpretability, reproducibility, and ease of integration into time-series analysis workflows.

The **GraalVM Performance Dataset** [7] includes the measurements collected by benchmarking the GraalVM compiler. The measurements are taken when the GraalVM compiler receives new commits and reports the changes in the performance of the compiler to the developers as it is essential to know about performance changes caused by any commit. The dataset is prepared in various configurations and setups in which the compiler is installed and benchmarked. More than 100 benchmarks are used to cover the compiler components as much as possible. The measurements are collected in various software performance-related metrics such as time, memory usage, instruction counts, etc.

We argue that, although the raw data is not explicitly structured as a time series, it inherently represents a time-series problem. Organizing it accordingly would enhance its utility for analysis and modeling. This time-series structure can be observed at several distinct levels:

1. **Release Level:** Software releases follow a chronological sequence, and performance measurements collected at each release point naturally form a time series reflecting high-level trends over time.

2. **Commit Level:** Between releases, each merge commit introduces updates to the codebase. Performance metrics captured at the commit level offer finer-grained insights and also constitute a sequential time series.
3. **Warmup Behavior in JIT Compilers:** In systems such as the GraalVM compiler, just-in-time (JIT) compilation triggers only after certain methods are invoked multiple times. This creates a warmup phase where performance readings are volatile. Capturing this behavior in a time-series format enables automatic detection and filtering of the warmup period, leading to more accurate performance analysis.

Anomaly Detection Datasets: Time series data is widely used for anomaly and fault detection in Critical Infrastructure (CIS) and Industrial Control Systems (ICS) such as power grids, water treatment facilities, and manufacturing systems. In these environments, continuous streams of sensor and actuator data enable early detection of system faults or malicious intrusions. Two widely adopted benchmark datasets are described below:
Secure Water Treatment (SWaT) Dataset [8]**:** This dataset was collected from a fully operational water treatment testbed at the Singapore University of Technology and Design. It simulates a realistic six-stage purification process controlled by sensors, actuators, and programmable logic controllers (PLCs) within a SCADA system. The dataset is provided in two CSV files containing time-stamped readings from 51 sensors and actuators, with a `Normal/Attack` label during the attack phase. It includes over 945,000 records and is widely used for evaluating intrusion and anomaly detection models in ICS.
Skoltech Anomaly Benchmark (SKAB) Dataset [11]**:** Released by the Skolkovo Institute in 2020, SKAB is a multivariate time series dataset designed for anomaly detection in sensor-driven industrial environments. It consists of 39 CSV files, each representing a separate experimental run on a fluid control test rig. These files contain time-indexed sensor readings (e.g., pressure, flow, temperature) and a binary `target` label indicating anomalies. Faults were introduced through controlled manipulation of valves and flow rates, making the dataset useful for developing and benchmarking detection algorithms under varied operating conditions.

ICS systems typically monitored via SCADA systems [18], and they share a similar structure, often involving common components like sensors and actuators. As a result, they tend to generate comparable logs and time-series metric readings. Developing a unified model that standardizes both structured and unstructured data into time series can significantly accelerate real-world deployment for monitoring and data analytics across diverse systems. From a research standpoint, such a model would also simplify and accelerate experimentation with multiple similar datasets, promoting faster iteration and more consistent benchmarking.

Our experience with large-scale time-series data in telecommunications [4,13] further highlights this need. Session logs from diverse sources vary in structure, frequency, and feature types, making integration and preprocessing complex and

time-consuming. A unified model would simplify this by applying a consistent structure and metadata schema, easing data integration, validation, and analysis.

3 Unified Model

To overcome the issues mentioned in the section above, we propose harmonizing multiple heterogeneous logs into a unified model suitable for further analysis. Figure 1 shows an overview of the main model's elements (technically, it is the model's meta-model—its complete version created using EMF[1] is available here[2].

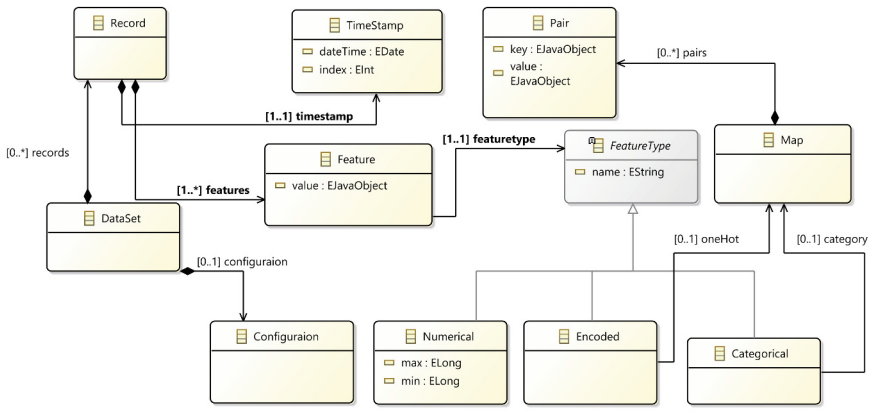

Fig. 1. Unified Model to represent time-series datasets

3.1 Model Components

In this section, we describe the model and its components.

DataSet A `DataSet` is the container for all time-series data. It aggregates multiple Records and optionally holds a Configuration. Engineers should think of a `DataSet` as a single logical time-series set, e.g., all logs for a particular application or a performance trace for one run.
Record A `Record` represents a single observation in time. It has a mandatory timestamp and contains one or more features. Each record corresponds to a snapshot in time (e.g., a log entry or performance reading). Engineers must ensure that every entry has a valid timestamp for alignment.
TimeStamp A `TimeStamp` defines the time context for a Record using:
- `dateTime`: the actual time (e.g., "2025-05-29T10:15:00Z"),

[1] Eclipse Modeling Framework: https://eclipse.dev/emf/.
[2] https://github.com/smartarch/unified-model-for-timeseries.

– `index`: an optional integer index, useful for synthetic time or ordering.

Data providers should use precise and consistent timestamping, preferring ISO date formats. If actual time is not available, they must use the index to establish order.

Feature A `Feature` represents an individual measurement associated with a `Record`, and it contains:

– `value`: actual data,
– `FeatureType`: a description of the data meaning and encoding.

Features should be normalized to one of the supported data types. Raw values such as strings or numbers should be accompanied by metadata that explicitly defines their type (e.g., categorical, numerical, or encoded). This ensures consistent interpretation and processing by the model.

FeatureType A `FeatureType` describes the nature and semantics of a Feature. It is an abstract element, and there are three particular specializations:

– `Numerical`: has min and max values for range normalization.
– `Categorical`: has a category map (e.g., "error": 1, "info": 2).
– `Encoded`: supports oneHot encodings via a map.

The `FeatureType` must be explicitly defined. This provides a clear path to prepare the data to be trained by ML models. Before training machine learning models, it is common practice to convert all features to numerical values and to scale them to a uniform range, such as $[0, 1]$ or $[-1, 1]$. This helps reduce modeling bias and ensures that all features contribute equally to the learning process. Knowing the feature type assists in the conversion and normalization process.

Configuration A `Configuration` represents settings used during dataset creation. `Configuration` can include details like sampling rate, source system info, or pre-processing rules. While optional, it helps in reproducibility and transparency.

The description above (the meta-model) represents a conceptual model (i.e., the abstract syntax [6] in the terms of model-driven engineering). From the general point of view, its particular representation (i.e., the concrete syntax) is not important if it follows the abstract syntax, then the different representations can be easily transformed between themselves.

3.2 Reasoning Behind the Model Design

The model is designed to ensure a strict one-to-one relationship between a dataset and its corresponding configuration. Configuration parameters are intentionally excluded from the feature set because we argue that the model should not learn the configuration—it should be trained within it. In real-world scenarios, a configuration represents the static environment in which the system operates.

For instance, in the case of GraalVM, a configuration includes the version family (e.g., Community or Enterprise Edition), the compiler setup, and the hardware profile. Prior research [1,2] performed on the GraalVM Dataset has

shown that statistical methods perform best when models are trained independently for each configuration. Including configuration as part of the training data would risk the model learning to associate performance changes with particular configurations, thereby reducing its generalizability.

4 Preliminary Evaluation of the Model

The proposed model serves multiple practical purposes, both in direct and indirect ways. Direct use involves adopting the model as a standardized format for representing time-series datasets. Indirect use treats the model as a validation tool, where datasets created in alternative formats are converted to or evaluated against our model to ensure consistency and compliance.

4.1 Model as a Concept

Using the model as a shared concept helps data providers and data scientists stay aligned. It provides a clear structure and common language for building and using time-series datasets, reducing confusion and the need for repeated communication or custom formatting.

Because the model is standardized, the data preparation steps can also follow a standard process. This saves time, reduces complexity, and makes it easier to reuse tools and pipelines across different projects.

For example, with the Mozilla and GraalVM performance datasets, data scientists currently have to build two different data preparation pipelines to handle each dataset's unique format. These steps are not typical machine learning preprocessing (like scaling or encoding) but involve handling raw data and structuring it. By using a common model format, this extra effort is avoided, and the same tools can be used across datasets.

The model can also serve as documentation. It helps teams understand what kind of data is expected, what each field means, and how it fits into a time series. This is especially helpful when datasets are shared between organizations or teams.

Additionally, tools built around the model such as data visualization dashboards or analysis scripts can be reused more easily. Since the structure is consistent, users can apply the same tools to new datasets without major changes.

4.2 Model as a Validator

The model can also be used to check the quality and structure of datasets, even if they are created in different formats. Data providers can use a model-to-model conversion step to translate their custom formats into the standard format, and then validate that the result meets all the necessary requirements.

Using the model as a validator helps catch missing or incorrectly formatted data early. This reduces errors in later steps, like training machine learning models or running analyses.

It also improves interoperability. If datasets from different sources follow the same standard, they can be compared or merged more easily. This is especially important in research or large-scale data projects involving many contributors.

Finally, validators based on the model can be automated. This allows for continuous checking of new data as it is collected, helping maintain high-quality time-series datasets over time.

5 Related Work

Extensive research compares various time-series representations and similarity measures across domains, showing that no single method is universally optimal. In this work, we propose that a unified model offers a general, structured view of time-series data, serving as an intermediary between data providers and data scientists. Other representations can be transformed to and from this model for specific use cases. Related works [5,15,17] explore alternative representations. For example, [5] combines clustering, LLM summarization, and dimensionality reduction (e.g., Time Curves) to analyze large-scale software logs. While innovative, such approaches are not considered general-purpose models.

Several techniques, such as Piecewise Aggregate Approximation (PAA), Symbolic Aggregate approXimation (SAX), and pattern-based methods, convert continuous time-series data into compact, interpretable forms [9]. More advanced methods, like Learned Pattern Similarity (LPS), leverage local patterns and tree-based ensembles for unsupervised tasks such as clustering and anomaly detection [3], generalizing to multivariate time series without requiring manual feature design.

Open-source frameworks like OpenTelemetry[3] focus on collecting traces, metrics, and logs for runtime observability. Our model complements these efforts by standardizing time-series data post-collection, enhancing consistency and simplifying downstream analysis and integration.

6 Conclusion and Future Work

In this work, we proposed a conceptual model for unifying heterogeneous logs into a structured time-series format with minimal preprocessing. By treating all logged data as temporal events, the model provides a scalable foundation for tasks such as real time system monitoring, anomaly detection and predictive maintenance.

Our initial evaluation shows the model can serve as both a data preparation concept and a bridge between data providers and data scientists, while also acting as documentation. It can support indirect use by enabling conversions from legacy formats, offering a modern, standardized representation.

Future work includes developing a framework for automatic conversion between common time-series formats and this meta-model, as well as designing a domain-specific language (DSL) for concise, expressive dataset descriptions.

[3] https://opentelemetry.io/docs/what-is-opentelemetry/.

Acknowledgment. This work has been partially supported by the EU project ExtremeXP grant agreement 101093164, partially by the Charles University Grant Agency project 74824, and partially by Charles University institutional funding SVV 260821.

References

1. Abdullah, M., Bulej, L., Bureš, T., Horký, V., Tůma, P.: Early Stopping of Non-productive Performance Testing Experiments Using Measurement Mutations. In: SEAA'2023, pp. 86–93 (2023)
2. Abdullah, M., Reichelt, D.G., Horký, V., Bulej, L., Bureš, T., Tůma, P.: A Combined Approach to Performance Regression Testing Resource Usage Reduction. In: PROMISE'2025, New York, NY, USA, pp. 75–84. ACM (2025)
3. Baydogan, M.G., Runger, G.: Time series representation and similarity based on local autopatterns. Data Min. Knowl. Disc. **30**(2), 476–509 (2016)
4. Birihanu, E., et al.: Client error clustering approaches in content delivery networks (CDN) (2022). https://doi.org/10.48550/arXiv.2210.05314
5. Borysenkov, D., Vogel, A., Henning, S., Perez-Wohlfeil, E.: Analyzing logs of large-scale software systems using time curves visualization (2024)
6. Brambilla, M., Cabot, J., Wimmer, M.: Model-Driven Software Engineering in Practice. Synthesis Lectures on Software Engineering. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-031-02549-5
7. Bulej, L., et al.: GraalVM compiler benchmark results dataset (data artifact). In: ICPE-C'2023, New York, NY, USA, pp. 65–69. ACM (2023)
8. Goh, J., Adepu, S., Junejo, K.N., Mathur, A.: A dataset to support research in the design of secure water treatment systems. In: Havarneanu, G., Setola, R., Nassopoulos, H., Wolthusen, S. (eds.) CRITIS 2016. LNCS, vol. 10242, pp. 88–99. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-71368-7_8
9. Guo, C., Li, H., Pan, D.: An improved piecewise aggregate approximation based on statistical features for time series mining. In: Bi, Y., Williams, M.-A. (eds.) KSEM 2010. LNCS (LNAI), vol. 6291, pp. 234–244. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15280-1_23
10. He, S., He, P., Chen, Z., Yang, T., Su, Y., Lyu, M.R.: A survey on automated log analysis for reliability engineering. ACM Comput. Surv. **54**(6), 130:1-130:37 (2021)
11. Katser, I.D., Kozitsin, V.O.: SKAB - Skoltech Anomaly Benchmark (2020). https://doi.org/10.34740/KAGGLE/DSV/1693952
12. Kratzke, N.: Cloud-native observability: the many-faceted benefits of structured and unified logging–a multi-case study. Future Internet **14**(10), 274 (2022)
13. Mahmud, J.S., Jursonovics, T., Lendak, I.: How BBR improves the quality of live TV streaming experience. In: 2025 IEEE 23rd World Symposium on Applied Machine Intelligence and Informatics (SAMI), pp. 000283–000288 (2025)
14. Mohaed Bilel, B., Diego Elias, D.C., Suhaib, M., Marco, C., Gregory, M.: Mozilla performance time series dataset (2025). https://doi.org/10.5281/ZENODO.14642239
15. Rezvani, R., Barnaghi, P., Enshaeifar, S.: A new pattern representation method for time-series data. IEEE Trans. Knowl. Data Eng. **33**(7), 2818–2832 (2021)
16. Tzanettis, I., Androna, C.M., Zafeiropoulos, A., Fotopoulou, E., Papavassiliou, S.: Data fusion of observability signals for assisting orchestration of distributed applications. Sensors **22**(5), 2061 (2022)

17. Wu, X., Li, H., Khomh, F.: On the effectiveness of log representation for log-based anomaly detection. Empir. Softw. Eng. **28**(6), 137 (2023)
18. Yadav, G., Paul, K.: Architecture and security of SCADA systems: a review. Int. J. Crit. Infrastruct. Prot. **34**, 100433 (2021)

Making a Pipeline Production-Ready: Challenges and Lessons Learned in the Healthcare Domain

Daniel Angelo Esteves Lawand[1], Lucas Quaresma Medina Lam[1], Roberto Oliveira Bolgheroni[1], Renato Cordeiro Ferreira[1,2,3,4(✉)], Alfredo Goldman[1(✉)], and Marcelo Finger[1(✉)]

[1] Instituto de Matemática e Estatística (IME), University of São Paulo (USP), São Paulo, Brazil
{renatocf,gold,mfinger}@ime.usp.br
[2] Jheronimus Academy of Data Science (JADS), 's-Hertogenbosch, The Netherlands
[3] Technical University of Eindhoven (TUe), Eindhoven, The Netherlands
[4] Tilburg University (TiU), Tilburg, The Netherlands

Abstract. Deploying a Machine Learning (ML) training pipeline into production requires good software engineering practices. Unfortunately, the typical data science workflow often leads to code that lacks critical software quality attributes. This experience report investigates this problem in SPIRA, a project whose goal is to create an ML-Enabled System (MLES) to pre-diagnose insufficiency respiratory via speech analysis. This paper presents an overview of the architecture of the MLES, then compares three versions of its Continuous Training subsystem: from a proof of concept Big Ball of Mud (v1), to a design pattern-based Modular Monolith (v2), to a test-driven set of Microservices (v3). Each version improved its overall *extensibility*, *maintainability*, *robustness*, and *resiliency*. The paper shares challenges and lessons learned in this process, offering insights for researchers and practitioners seeking to productionize their pipelines.

Keywords: Code Quality · MLOps · Software Architecture · Machine Learning Enabled Systems · Healthcare Domain · Experience Report

1 Introduction

In 2020, amidst the COVID-19 pandemic, a multidisciplinary team of researchers from the University of São Paulo (USP) created SPIRA[1]: a project to detect respiratory insufficiency via speech analysis, using Machine Learning (ML) [7].

Since then, the scope of the SPIRA project has evolved to detect respiratory insufficiency of different origins, including many sicknesses that can cause this symptom: smoking side effects, flu, severe asthma, and heart conditions [7].

[1] https://github.com/spirabr.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 354–362, 2026.
https://doi.org/10.1007/978-3-032-04403-7_30

To create a tool that could assist physicians, the team proposed to develop the SPIRA ML-Enabled System (MLES). Since 2020, different components have been incrementally developed by bachelor students working with the project.

2 The SPIRA ML-Enabled System

This section describes the architecture of the SPIRA ML-Enabled System, as illustrated by Fig. 1. Sections 2.1 to 2.6 showcase its six subsystems according to the reference architecture extended by Ferreira *et al.* [5].

2.1 Data Collection

Creating an ML model that can detect respiratory insufficiency via voice is a supervised machine learning classification problem [4]. As such, it requires *labeled data*. The SPIRA team supports a Data Collection App (1) provided to volunteer data collectors to collect voices inside hospitals, where people with respiratory insufficiency may be found. The Data Collection App (1) sends data to a Data Collection API (2), which in turn stores audio in an Audio Key-Value Database (A), and saves patients' info in a Document Database (B).

After each data collection, data collectors register a unique identifier for each participant. This ID can be cross-referenced with data shared by partner hospitals via 3rd-party Hospital APIs (3). This way, researchers can create a Label Store (C) that provides a *ground truth* for training models.

2.2 Continuous Training

Casanova *et al.* proposed a Deep Learning (DL) architecture based on a Convolutional Neural Network (CNN) to detect respiratory insufficiency from audio signals [4]. As more data becomes available as *ground truth*, there is potential to retrain the model and improve its accuracy [10].

The proof-of-concept shared by Casanova *et al.* [4] can be divided into two components: a Feature Engineering Pipeline (I), to process audio signals for the training algorithm; and a Training Pipeline (II), to train and validate the CNN.

The development of these two pipelines is an opportunity to introduce standard machine learning design patterns [5,12], such as a Feature Store (D), a Metadata Store (E), and a Model Registry (I). For the SPIRA project, the goal is to use MLFlow to make the two latter roles. On the other hand, for the features, the proposal is to use a simple key-value database like MinIO.

2.3 Development

As more data becomes available as *ground truth*, there is potential to redesign the architecture of the model used by SPIRA to improve its accuracy [10]. To make experimentation easy, it is desirable to provide a standardized environment for data scientists, compatible with production environments [21].

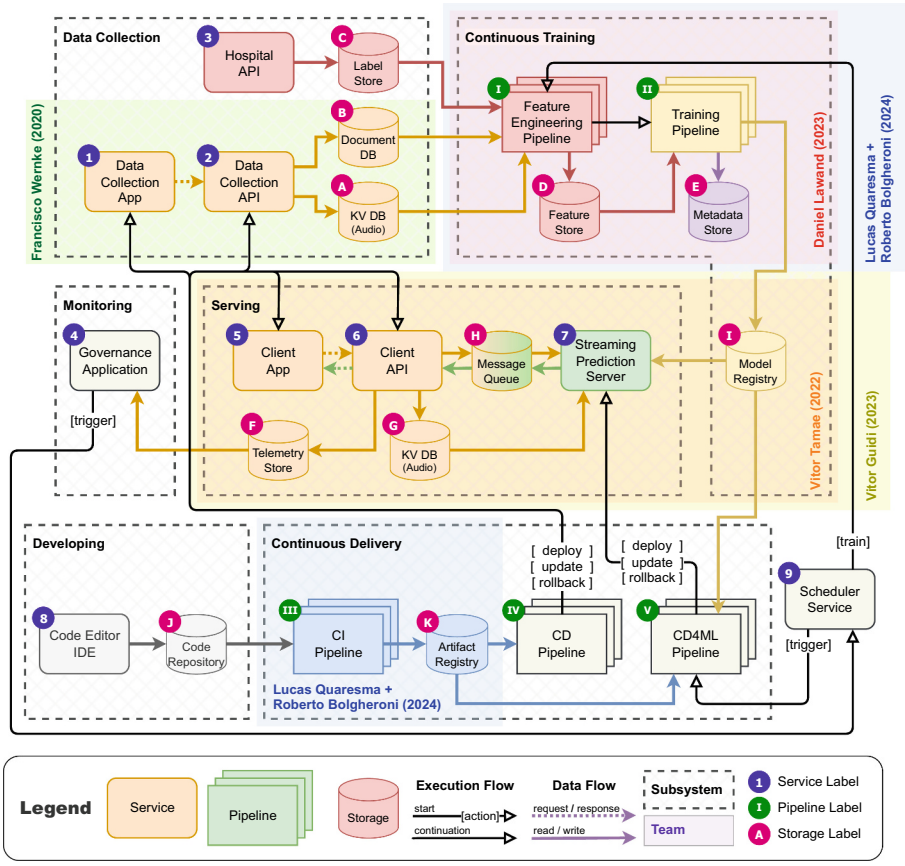

Fig. 1. *Architecture of the SPIRA ML-Enabled System.* The architecture is described with the same notation of the reference architecture described in Ferreira *et al.* [5]. Rectangles represent **services**, which execute continuously. Stacked rectangles represent **pipelines**, which execute a task on demand. Lastly, cylinders represent **data storage**, which may be databases of any type. Components are connected by arrows. Black arrows with a hollow tip illustrate the **execution flow**. They start and end in a component. Labeled arrows represent the trigger that starts a workflow, whereas unlabeled arrows represent the continuation of an existing workflow. Colored arrows with a filled tip illustrate the **data flow**. They appear in two types: solid arrows going to and from a data storage represent write and read operations, respectively; dotted arrows represent a sync or async request-response communication between components. Components are colored according to the data they produce: raw data ■, source code ■, executable artifacts ■, ML-specific data ■, ML models ■, ML training metadata ■, ML model predictions ■, and ML model metrics ■. The remaining standalone components □ orchestrate the execution of others. Components are also grouped into **subsystems**, with their background colored according to the **students** responsible for their development. Numbers ●, roman numerals ● and letters ● are used as labels throughout Sect. 2.

Currently, all SPIRA source code is open source in a Code Repository (J) at GitHub. Following the principles of *Infrastructure as Code (IaC)* [17], the SPIRA organization provides standard configurations for popular development tools used by data scientists and machine learning engineers. In particular, this includes a basic setup for Code Editors (8) (such as VSCode) or IDEs (8) (such as PyCharm), but can also include other utility scripts helpful for the developers.

2.4 Continuous Delivery

To support the creation of new versions of the MLES – after retraining or redesigning a model – automation is essential to make deployment seamless.

Taking advantage of GitHub, a Continuous Integration Pipeline (III) uses GitHub Actions to generate Docker containers for components, which are then deployed into the GitHub Container Registry, the de facto Artifact Store (K) for SPIRA. Conversely, the Continuous Delivery Pipeline (IV) and Continuous Delivery for Machine Learning Pipeline (V) use infrastructure configuration to deploy and rollback services in a Kubernetes-managed cluster.

2.5 Serving

Similarly to the Data Collection App (1), the SPIRA Client App (5) is meant to be used inside hospitals. However, it has a different goal: to provide (indirect) access to the SPIRA model, pre-diagnosing respiratory insufficiency via speech analysis. Therefore, its data collection is more critical.

Hospitals can have poor, unreliable internet reception. Multiple audios can be sent in a single burst whenever a user gets internet access. To prevent data loss, the Client App (5) must store all data collected locally before sending it to its corresponding Client API (6). Moreover, to prevent data inconsistency, the Client API (6) must validate all data received with the corresponding Client App (5) before storing it in its Audio Key-Value Database (G).

Deep Learning models such as the one proposed by Casanova *et al.* often require accelerator hardware (GPUs) to run efficiently [10]. Since executing the model is a costly operation, handling arbitrarily bursts of request is difficult.

To avoid this bottleneck, the SPIRA ML model can be executed in a separate Streaming Prediction Service (7), decoupled from the Client API (6). A Message Queue (H) stores prediction requests and corresponding results. Thanks to this separation, the Streaming Prediction Service (7) may be run in machines with GPUs, whereas other components are deployed in cheaper hardware.

2.6 Monitoring

Once the SPIRA MLES reaches production, it needs to be maintained in operation. The Telemetry Store (F) stores useful logs and data about its usage, while the Governance Application (4) summarizes statistics of its working status based on them. Combined, these two components help the SPIRA team to decide to retrain the ML model, or consider redesigning the ML model, or update and redeploy a component by triggering the Scheduler Service (9).

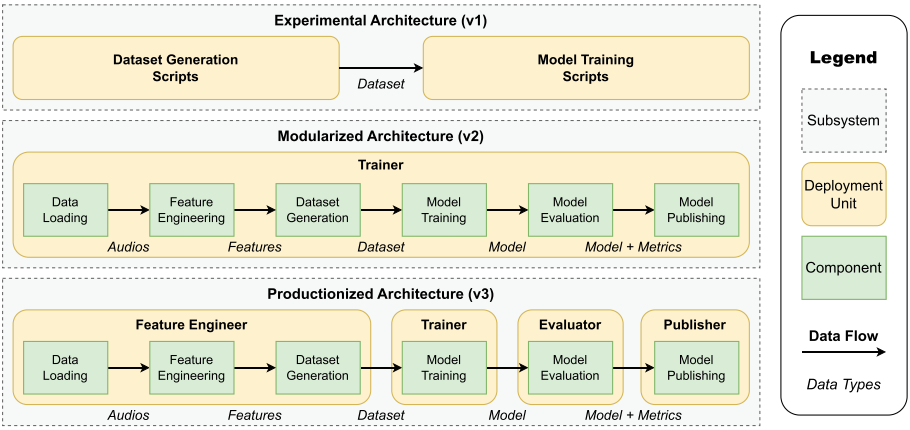

Fig. 2. *Incremental Development of the SPIRA* Continuous Training *Subsystem.* Dashed boxes ▢ represent the versions of the subsystem. Rounded boxes ▢ represent deployment units. Squared boxes ▢ represent components. Black arrows with a filled tip illustrate the **data flow**, while the labels in *italic* represent the **data types**.

3 Incremental Implementation

The SPIRA MLES has been incrementally developed since 2021. This paper focuses on the Continuous Training subsystem, whose development process is summarized in Fig. 2. For challenges and lessons learned while developing the Data Collection and Serving, subsystems, please refer to our paper *"SPIRA: Building an Intelligent System for Respiratory Insufficiency Detection"* [6].

4 Experimental Architecture (v1)

The first version of the Continuous Training pipeline was created as a proof-of-concept to showcase the SPIRA model. The code was published as part of the reproduction package by Casanova *et al.* in 2020 [4].

The package has two main sets of scripts: one to create datasets (available in the `scripts` folder), and one to train the CNN model (available in the `models` folder). The code presents an unplanned, organically developed structure, as it often occurs in the experimentation phase by data scientists [12,21].

Figure 3 (left) illustrates an example of the Misplaced Responsibility bad smell [15] in the code, evidence of high coupling and low cohesion in the code. In this way, this architecture can be classified as a Big Ball of Mud [8].

5 Modularized Architecture (v2)

The second version of the Continuous Training pipeline was created during the bachelor thesis of Daniel Lawand [14]. Its code is open source at GitHub.

(v1 architecture)

```
random.seed(c.train_config["seed"])
torch.manual_seed(c.train_config["seed"])
torch.cuda.manual_seed(c.train_config["seed"])
np.random.seed(c.train_config["seed"])
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

self.c = c
self.ap = ap
self.train = train
self.test = test
self.test_insert_noise = test_insert_noise
self.num_test_additive_noise = num_test_additive_noise
self.num_test_specaug = num_test_specaug
self.dataset_csv = \
  c.dataset["train_csv"] if train else c.dataset["eval_csv"]

assert os.path.isfile(self.dataset_csv), \
  "Test or Train CSV file don't exists! Fix it in config.json")

accepted_tc = [ 'overlapping', 'padding', 'one_window' ]
assert self.c.dataset['temporal_control'] in accepted_tc), \
  "You cannot use the padding_with_max_length option with the \
   split_wav_using_overlapping option, disable one of them !!")

self.control_class = c.dataset['control_class']
self.patient_class = c.dataset['patient_class']

self.dataset_list = \
  pd.read_csv(self.dataset_csv, sep=',') \
  .replace({'?': -1}) \
  .replace({'negative': self.control_class}, regex=True) \
  .replace({'positive': self.patient_class}, regex=True) \
  .values
```

(v2 architecture)

```
# Setup

config_path = ValidPath.from_str("/app/spira/spira.json")
config = load_config(config_path)

operation_mode = OperationMode.TRAIN
randomizer = initialize_random(config, operation_mode)

# Data Loading

patients_paths = read_valid_paths_from_csv(config.patients_csv)
controls_paths = read_valid_paths_from_csv(config.controls_csv)
noises_paths = read_valid_paths_from_csv(config.noises_csv)

patients_inputs = Audios.load(
  patients_paths, config.audio, config.dataset
)
controls_inputs = Audios.load(
  controls_paths, config.audio, config.dataset
)
noises = Audios.load(noises_paths, config)
  noises_path, config.audio, config.dataset
)

# Feature Engineering

audio_processor = create_audio_processor(config.audio)

patient_feature_transformer = create_audio_feature_transformer(
    randomizer, audio_processor, config, noises,
)
control_feature_transformer = create_audio_feature_transformer(
    randomizer, audio_processor, config, noises,
)
```

Fig. 3. *Example of improving modularization between* v1 *and* v2 *architectures.* The v1 snippet (left) shows a Misplaced Responsibility bad smell [15] at the class `Dataset`. Lines 1–7 handle random number generation. Lines 8–16 assign values to attributes. Lines 18–19 and 22–24 handle assertions. Line 30 handles data loading. The v2 snippet (right) shows the application of design patterns at the module `pipeline`. Line 15–23 handles data loading using the `Audio` Adapter. Line 27 builds an `audio_processor` via a Chain of Responsibility pattern. Line 29–36 build `feature_transformer`s via the Strategy pattern. Improving the modularization makes the intention of the code more explicit: allow multiple experiments depending on the configuration.

This version applied multiple design patterns [9] to modularize the code. Some examples include: Chain of Responsibility, to dynamically choose the pipeline steps to be executed; Strategy, to dynamically choose techniques for data preprocessing, feature engineering, and model evaluation; and Template Method, to reuse generic code snippets. In another level of abstraction, the business logic was decoupled from external dependencies by using the Ports and Adapters architectural pattern [16], using Dependency Injection to connect different layers of the application.

Figure 3 (right) illustrates the use of design patterns. This reimplementation made the code more *maintainable* and *extensible* [19]. The v2 architecture can be classified as a Modular Monolith [20].

6 Productionized Architecture (v3)

The third version of the Continuous Training pipeline was created during the bachelor thesis of Lucas Quaresma and Roberto Bolgheroni [13]. Its code is open source at GitHub.

This version expands on the v2 architecture by splitting the single monolithic application into multiple deployment units, as described by the Workflow Pipeline ML design pattern [12]. The goal was to enable executing the pipeline with a Scheduler Service (9). In this way, if a failure occurs in one stage of the pipeline, it does not require re-executing it from the start, a costly operation.

Testability was also a key priority in this version. It was only possible because the v2 architecture made clearer the expected behaviors from the v1 architecture. Following testing practices for microservices [20], the business logic received unit tests via *Test-Driven Development* (TDD) [1], while the Ports and Adapters received integration tests around external dependencies [16].

This reimplementation made the code more *robust* and *resilient* [19]. The v3 architecture follows a Microservices architectural style [18].

7 Challenges

The Experimental Architecture (v1) was built following a typical CRISP-DM process [3,21]. The goal was to create a proof-of-concept model. Serving it, i.e., going into production, was the last step of the workflow.

This development process brought a series of drawbacks for the Continuous Training subsystem's architectural characteristics, in particular its *extensibility*, *maintainability*, *robustness*, and *resiliency*. As a consequence, there were two main challenges to migrating the subsystem toward production.

Separation of Concerns. The v1 architecture had many examples of Misplaced Responsibility bad smell [15], which affected its overall *maintainability* and *extensibility* [19]. Building the v2 architecture became an exercise in *Software Archeology* [11]: it was reimplemented from the v1 code by carefully reading it line by line to decipher its intentions, applying design patterns to decrease coupling and increase cohesion in its abstractions.

Automated Testing. The v2 architecture had improved *modularity*, but it still lacked *robustness* and *resiliency* [19]. Building the v3 architecture became an exercise in *Test-Driven Development* (TDD) [1]: it was reimplemented from v2 code by carefully interpreting intentions around its design, creating automated tests to document behaviors that should be maintained for the long term.

8 Lessons Learned

The development of MLES is inherently difficult, and developing them without adequate planning makes their complexity even greater. As a consequence, there were two lessons learned that could have improved the migration of the Continuous Training subsystems toward production.

Collaboration between Data Scientists and ML Engineers. Machine Learning Engineering (MLE) is a new subarea of software engineering whose goal if to help to productionize ML models [3,21]. This discipline helps the data scientist to think about how it will fit into an MLES. By bringing these two roles together since the beginning, it is easier to make code *maintainable* and *extensible*.

Testing as a First-Class Concern. Creating automated tests is recognized as a good practice in software engineering [2]. This discipline helps developers to think about the long-term maintenance of a system. By designing tests and validation since the beginning, it is easier to make the code *robust* and *resilient*.

9 Conclusion

This paper addressed the incremental development of SPIRA, focusing on its Continuous Training subsystem. Its architecture evolved in three stages: from a proof of concept Big Ball of Mud (v1), to a design pattern-based Modular Monolith (v2), to a test-driven set of Microservices (v3). Each step helped the subsystem's *extensibility*, *maintainability*, *robustness*, and *resiliency*.

By learning from this experience, similar projects may employ the above lessons learned to avoid similar challenges, thus reaching production sooner.

Acknowledgements.. This research was funded by FAPESP (São Paulo Research Foundation), grant number 2023/00488-5.

References

1. Beck, K.: Test Driven Development: By Example, 1st edn. Addison-Wesley Professional (2002)
2. Beck, K., Andres, C.: Extreme Programming Explained, 2nd edn. Addison-Wesley Professional (2004)
3. Burkov, A.: Machine Learning Engineering, 1st edn. True Positive Inc. (2020)
4. Casanova, E., et al.: Deep learning against covid-19: respiratory insufficiency detection in Brazilian Portuguese speech. In: Findings of the Association for Computational Linguistics, pp. 625–633 (2021)
5. Ferreira, R.C.: A Metrics-Oriented Architectural Model to Characterize Complexity on Machine Learning-Enabled Systems. Proceedings - 2025 IEEE/ACM 4th International Conference on AI Engineering - Software Engineering for AI, CAIN 2025 (2025)
6. Ferreira, R.C., Gomes, D., Tamae, V., Wernke, F., Goldman, A.: SPIRA: building an intelligent system for respiratory insufficiency detection. In; Workshop Brasileiro de Engenharia de Software Inteligente (ISE), vol. 1, pp. 19–22 (2022)
7. Finger, M., et al.: Detecting respiratory insufficiency by voice analysis: the SPIRA project. In: Acoustic Communication: An Interdisciplinary Approach (2021)
8. Foote, B., Yoder, J.: Big ball of mud. In: PLoP '97 / EutoPLoP '97. Monticello (1999)

9. Gamma, E., Helm, R., Johnson, R., Vlissides, J.: Design Patterns: Elements of Reusable Object-Oriented Software, 1st edn. Addison-Wesley Professional (1994)
10. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning, 1st edn. The MIT Press (2016)
11. Hunt, A., Thomas, D.: Software archaeology. IEEE Software **19**(2) (2002)
12. Lakshmanan, V., Robinson, S., Munn, M.: Machine Learning Design Patterns, 1st edn. O'Reilly Media (2020)
13. Lam, L.Q.M., Bolgheroni, R.O., Ferreira, R.C., Goldman, A.: Productionizing SPIRA's Model Trainer System. Ph.D. thesis, University of São Paulo, São Paulo (2024)
14. Lawand, D.A.E., Ferreira, R.C., Goldman, A.: Enabling MLOps in the SPIRA Training Pipeline. Ph.D. thesis, University of São Paulo, São Paulo (2023)
15. Martin, R.C.: Clean Code, 1 edn. Pearson (2008)
16. Martin, R.C.: Clean Architecture, 1st edn. Pearson (2017)
17. Morris, K.: Infrastructure as Code, 2nd edn. O'Reilly Media (2025)
18. Newman, S.: Building Microservices, 2nd edn.. O'Reilly Media (2021)
19. Richards, M., Ford, N.: Fundamentals of Software Architecture, 1st edn. O'Reilly Media (2020)
20. Richardson, C.: Microservices Patterns, 1st edn. Manning Publications (2018)
21. Wilson, B.: Machine Learning Engineering in Action, 1st edn. Manning Publications (2022)

DQGen: Scalable Metadata-Driven Automation for Data Quality Validation in Data-Intensive Applications

Moamin Abughazala[1,2] and Henry Muccini[1(✉)]

[1] University of L'Aquila, L'Aquila, Italy
{moamin.abughazala1,henry.muccini}@univaq.it
[2] An-Najah National University, Nablus, Palestine
m.abughazaleh@najah.edu

Abstract. Data-intensive applications (DIAs) increasingly rely on trustworthy, high-quality data to support reliable analytics, machine learning, and automated decision-making. However, ensuring data quality at scale remains a major challenge due to schema evolution, ingestion inconsistencies, and the manual effort required to define validation rules. This paper presents **DQGen**, a framework for automating the generation of data quality validation scripts tailored to complex, evolving datasets. By leveraging dataset metadata and systematically mapping data quality dimensions (e.g., completeness, uniqueness, validity) to Great Expectations (GE) rules, DQGen produces executable validation code adaptable to any schema.

We evaluate DQGen on a real-world dataset from a large-scale Internet Service Provider (ISP), comprising over 26 million records across multiple relational tables. Results show that DQGen reduces validation setup time by over 90%, improves rule coverage and consistency, and enables continuous integration of data quality checks in batch or CI/CD workflows. The proposed framework contributes to the reliability and governance of modern DIAs by ensuring scalable, transparent, and automated validation.

Keywords: Data Quality · Data-Intensive Applications · Metadata-Driven Code Generation

1 Introduction

Modern data-intensive applications (DIAs) are increasingly central to critical business operations, scientific research, and intelligent decision-making. These systems, which span domains such as telecommunications, healthcare, finance, and software analytics, process large volumes of heterogeneous and evolving data. The quality of this data is crucial—poor data quality, including missing values, inconsistent formats, schema drift, and duplicate records, can compromise analytics, degrade machine learning (ML) model performance [1], and introduce risks in automated pipelines [2].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 363–380, 2026.
https://doi.org/10.1007/978-3-032-04403-7_31

Ensuring data quality in DIAs remains a non-trivial challenge. Traditional validation processes often rely on manual data inspection or hard-coded rules, which are labor-intensive, error-prone, and difficult to scale [3]. While libraries such as *Great Expectations* (GE) [4], Deequ [5], and DBT [6] provide rule-based validation, their configuration typically requires manual definition of expectations, limiting reuse and adaptability in dynamic, schema-evolving environments.

To overcome these limitations, we propose a metadata-driven approach that automatically creates validation rules based on schema information. This method links data quality dimensions to validation code using templates. It reduces manual work, enhances consistency of rules across datasets, and facilitates the development of quality assurance processes that are scalable and easy to maintain.

In this paper, we introduce **DQGen**[1] **(Data Quality Code Generation)**, a metadata-driven framework that automates the generation of validation scripts for data-intensive systems. DQGen bridges dataset schema metadata with a configurable set of quality dimensions to dynamically generate executable GE validation code [7]. By integrating seamlessly into CI/CD workflows or batch pipelines [8], the framework enables scalable and maintainable data quality enforcement in large-scale, schema-rich environments.

We evaluate DQGen using a real-world dataset from a major Internet Service Provider (ISP), comprising over 26 million records across multiple tables. The framework demonstrates significant improvements in both efficiency and consistency, reducing validation development time while improving consistency through broader and more uniform rule coverage compared to manually developed validation scripts.

This paper makes the following key contributions:

- We present a metadata-driven framework for automated validation code generation in data-intensive applications, leveraging quality dimensions mapped to GE expectations.
- A real-world case study in the telecommunications sector demonstrates the framework's ability to address missing values and inconsistencies across large-scale ISP datasets.
- We provide an empirical evaluation showing improved efficiency and consistency in validation compared to manual scripting, contributing to maintainable and trustworthy data workflows in modern DIAs.

The remainder of this paper is organized as follows: Sect. 2 offers a review of related work concerning automated data quality assessment and its influence on software engineering tasks. Section 3 introduces the proposed framework, detailing its architecture, mapping process, and code generation pipeline. Section 4 provides a case study that uses ISP data. In Sect. 5, we provide a quantitative evaluation that compares our approach to traditional methods. Section 6 covers the findings and their implications. Section 7 addresses potential threats

[1] **DQGen** Source Code & Data: https://anonymous.4open.science/r/DQGenSE-22B3.

to validity, while Sect. 8 concludes the paper by outlining directions for future research.

2 Background and Related Work

Data quality has become a foundational concern DIAs, as inaccuracies, inconsistencies, and missing data can directly impair the performance of downstream analytics and AI/ML systems [9,10]. This section reviews three key areas of related work: (1) rule-based validation tools, (2) automated data quality assessment frameworks, and (3) the impact of poor-quality data on ML pipelines. We conclude by identifying the gap our work addresses.

2.1 Rule-Based Validation Tools

Tools like **Great Expectations (GE)** [4], **Apache Deequ** [5], and **DBT** [6] enable users to define validation rules for data profiling and testing. GE offers a library of expectations, integration with CI/CD, and human-readable data documentation. Deequ, built by AWS, supports large-scale checks on Spark-based data, while DBT focuses on SQL-based pipeline transformations and tests. Notable tools like Soda SQL [11] and AWS Glue DataBrew [12] offer useful features but have limitations. Soda SQL allows data quality checks through YAML and integrates with CI/CD pipelines, yet requires manual validation logic and doesn't automate rule derivation. AWS Glue DataBrew provides a no-code interface for data profiling and cleaning, but lacks reusable validation scripts and doesn't fit well in rule-driven pipelines. Compared to DQGen, which offers automated rule mapping, these tools are less scalable in fast-changing data environments.

However, these tools *rely on hand configuration of validation rules*, which can result in uneven coverage across changing datasets and create scale restrictions. They also lack automation in rule discovery, metadata, or new schema adaptation. For instance, validating that all necessary fields across 50 tables are non-null using SQL, for example, calls for writing individual "Select... WHERE column IS NULL" searches for each column, usually by hand. Ensuring uniqueness or type conformity also means creating custom scripts that struggle as schemas change. Any changes in the schema require manual script updates, which require maintenance overhead and the risk of inconsistency. Tools like DBT or GE help standardize these rules, but still rely heavily on manual definition. In contrast, DQGen abstracts this process through metadata-driven rule generation, making it feasible to enforce comprehensive quality checks across large and changing datasets.

2.2 Automated Data Quality Assessment Frameworks

Several attempts aimed to automate rule execution and data profiling. **BIGQA** [13] enables declarative specification of quality checks and supports

distributed execution. **PyDaQu** [14] focuses on the validation of the data lifecycle using configurable profiles tied to ingestion, storage, and access stages. Schelter et al. [3] introduce an industrial-scale metadata-aware validator that integrates with batch pipelines.

Many of these systems are either closely tied to specific platforms or lack the ability to generate extensible code for use in various infrastructure contexts. Additionally, *few of them provide seamless integration with validation libraries like GE*, missing the chance to combine rule standardization with flexible automation.

2.3 Data Quality and Analytics Integrity

Data quality directly affects the effectiveness of machine learning and decision systems. Herbold et al. [10] found that label noise in defect prediction datasets reduces classifier accuracy. Krishna et al. [15] show that duplicate or redundant data entries can bias models and diminish reliability. This highlights the need for validation frameworks that not only detect surface-level issues but also ensure quality dimensions are enforced consistently.

2.4 Identified Gap

While existing tools offer valuable capabilities for rule-based validation or large-scale profiling, they lack a general-purpose, metadata-driven mechanism to automatically generate executable validation scripts. Currently, there is no reusable, domain-agnostic framework that automates the mapping of high-level quality dimensions to low-level validation rules, adapts to evolving schemas and metadata, generates modular and deployable validation code, integrates directly with existing tools like Great Expectations (GE), and addresses the scalability and maintainability requirements of modern data-intensive applications (DIAs).

Unlike existing tools, which largely ignore schema metadata during rule configuration, DQGen explicitly maps schema information (e.g., column types, constraints) to validation templates. This design enables scalable, reusable automation across changing datasets. To further contextualize the gap identified in current approaches.

The comparative metrics in Table 1 were derived through hands-on evaluations of each tool using standard validation tasks (e.g., null checks, type constraints, range checks) applied to real-world ISP tables. Automation is rated as Low, Medium, or High. A tool was considered “High” if greater than 80% of rules were auto-generated from metadata; “Medium” if templates or APIs were available but required manual configuration; and “Low” if most rules had to be written manually. Adaptability is rated based on how easily a tool handles evolving schemas. DQGen, for example, re-extracts schema metadata and regenerates rules upon detecting structural changes, requiring no manual updates. Other tools that lacked schema-awareness were rated lower. Scalability is evaluated by applying the same validation logic to datasets with varying table sizes and column counts. Tools that maintained consistent performance and rule generation

Table 1. Comparison of Existing Data Quality Frameworks vs. DQGen

Feature	DBT [6]	Apache Deequ [5]	Great Expectations [4]	DQGen (Proposed)
Validation Approach	SQL-based transformation checks	Rule-based quality validation	Python-based validation	Automated code generation via metadata-driven mapping
Automation Level	Low (manual rule definition)	Medium (some automation)	Medium (scripted automation)	High (automatic generation of validation scripts)
Adaptability	Manual updates required	Limited adaptability	Requires human-defined rules	Dynamically adapts to data schemas
Scalability	Low	Medium	Medium	High (code generation scales across large datasets)

without degradation were rated "High". Tools with significant manual effort increase per table or schema complexity scored lower. While these tools offer foundational capabilities for rule-based validation, they often lack automation, adaptability to schema evolution, and reusable code generation pipelines. DQGen addresses these shortcomings by offering high automation through metadata-driven rule mapping, dynamic adaptability to evolving datasets, and scalable script generation, thus positioning itself as a more comprehensive and maintainable solution for data-intensive applications.

3 Proposed Framework

3.1 High-level Overview of the DQGen Framework

We propose **DQGen**, a metadata-driven framework designed to automate the generation of data validation scripts for DIAs. The framework systematically maps high-level data quality dimensions—such as completeness, uniqueness, validity, timeliness, and consistency—to concrete validation rules using the Great Expectations (GE) library [4]. The output is a set of executable Python scripts that enforce quality checks on tabular datasets at scale.

DQGen supports structured and repeatable validation across large, heterogeneous, and evolving datasets. It is designed to integrate with CI/CD pipelines or batch ETL workflows, enabling continuous quality assurance with minimal manual intervention. ETL (Extract, Transform, Load) workflows are crucial for moving data from raw sources to structured storage systems. They involve extracting data, transforming it (e.g., cleaning, formatting), and loading it into databases or data warehouses. By using DQGen-generated scripts, you can validate data quality during transformation and loading, ensuring high standards. For CI/CD (Continuous Integration/Continuous Deployment), DQGen produces

standalone Python scripts compatible with automation tools like Jenkins. These scripts automatically trigger updates in code or data pipelines, checking quality and generating reports to maintain accurate data in production environments. Figure 1 provides a high-level view of the DQGen architecture, illustrating the flow from metadata extraction to validation script generation.

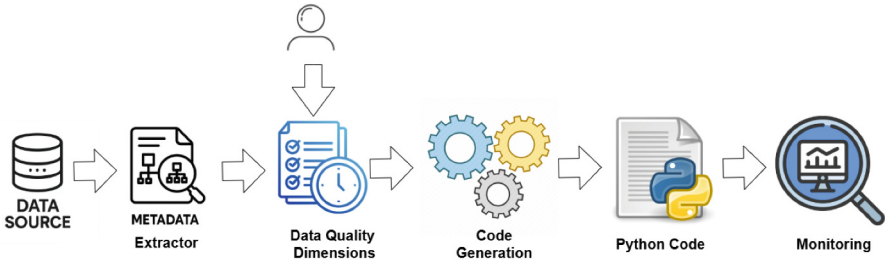

Fig. 1. Code generation framework

3.2 Framework Components

A detailed examination of the framework's components is essential for understanding its design and functionality. Figure 2 provides a comprehensive overview of the tool's architecture, outlining the relationships and interactions between its key modules. Each component's purpose and operation will be explored to illustrate how they collectively enable efficient data quality monitoring and code generation within the framework.

The framework consists of the following key components:

Metadata Extraction and Quality Dimension Selection. The process begins by identifying the target data source, typically a relational database or data warehouse. Schema metadata (tables, columns, data types) is extracted programmatically. The schema extraction module connects to data sources using JDBC connectors to retrieve the metadata. The metadata extraction process is fully automated, eliminating the need for users to manually define table structures. While users may need to take a moment to select the relevant data quality dimension. Users can easily identify critical dimensions for each table by adjusting the dimension list. Once these dimensions are set, DQGen seamlessly automates the rule mapping and script generation, ensuring an efficient and reliable workflow.

The selected data quality dimensions encompass a comprehensive set of validation criteria essential for ensuring dataset integrity. **Completeness** Completeness ensures that all mandatory fields are populated, especially in records critical to downstream processes [16]. **Uniqueness** enforces the distinctness of

identifiers such as primary keys or user IDs to prevent duplication [17]. **Validity** ensures that data entries adhere to expected types or formats, such as enforcing correct date formats or numeric constraints [18]. **Timeliness** evaluates whether temporal fields align with expected chronological ranges, supporting correct event sequencing and latency analysis [19]. **Consistency** involves checking logical coherence between related fields, such as matching formats across columns or enforcing referential integrity [20]. Finally, **Accuracy** validates whether field values belong to a defined set of permissible values, typically enforced using whitelists or enumerated domains to prevent invalid entries [18].

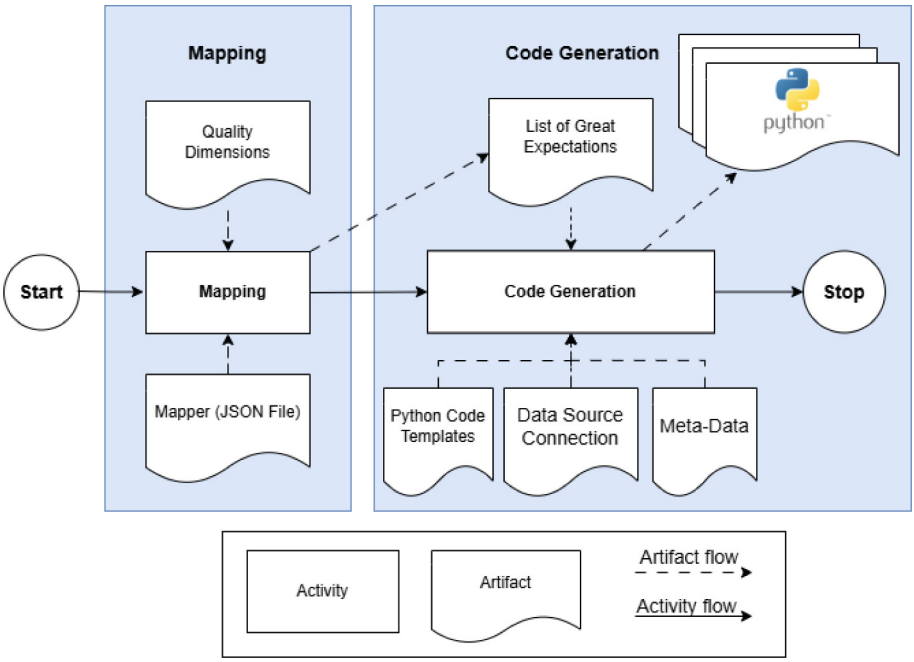

Fig. 2. Component-level architecture of DQGen showing the interaction between Metadata Extractor, Mapper, Quality Rule Selector, and Python Code Generator

A core contribution of DQGen is its systematic mapping of quality dimensions to GE expectations. This mapping is based on metadata and configurable templates.

Table 2 outlines the part of the list of quality dimensions and their corresponding GEs validation functions. The framework allows users to extend quality dimensions and templates through configuration files. Developers can implement custom checks for schema drift, business rule violations, or referential integrity by adding new mappings and templates to the rule engine.

This abstraction allows the same high-level quality requirements to be enforced across diverse schemas and data domains, ensuring consistency and reducing human error.

Code Generation Pipeline. DQGen uses templated Python scripts to produce runnable Great Expectations (GE) validation code [21]. The generation process combines the schema metadata, the quality dimension mappings, and a set of reusable Python code templates

Table 2. Part Of Mapper

Quality Dimensions	Great Expectations
Uniqueness	expect_column_values_to_be_unique
Completeness	expect_column_values_to_not_be_null
	expect_column_values_to_be_null
Validity	expect_column_values_to_be_of_type
	expect_column_values_to_be_in_type_list
	expect_column_values_to_be_increasing
Consistency	expect_column_value_lengths_to_be_between
	expect_column_value_lengths_to_equal
	expect_column_values_to_match_regex
	expect_column_values_to_match_regex_list
Timeliness	expect_column_min_to_be_between
	expect_column_max_to_be_between
Accuracy	expect_column_values_to_be_in_set
	expect_column_values_to_not_be_in_set

Each script connects to the target data source and applies the relevant expectations per table and column. Templates use placeholders for dynamic substitution of table names, column types, and expectation logic. This allows the framework to scale across hundreds of columns without rewriting validation logic manually.

To illustrate the output of DQGen, Fig. 3 presents an example of an auto-generated validation script for the userinfo table. In this case, the username column is subject to two checks, a completeness check using expect_column_values_to_not_be_null and a uniqueness check using expect_column_values_to_be_unique. These expectations were selected automatically based on schema metadata and applied using the GEs API. By templating these rules into executable Python scripts, DQGen ensures consistent enforcement across datasets with minimal manual coding. Table 3 outlines the key template elements used for defining and generating automated data validation rules.

```python
import great_expectations as ge
from Entities.batches import Batch
from Entities.checkpoints import CheckPointConfig
from Entities.datasources import DataSource
from Entities.expectationUtiles import ExpectationUtil
from Entities.expectationsuites import ExpectationSuite
# Get Context
context = ge.get_context()
if __name__ == '__main__':

    # Data Source Code
    dataSource=DataSource("root","root","127.0.0.1","3306","testdb",DataBaseType.MySQL)
    context.add_datasource(**dataSource.getMySQLDataCourceConfig())

    condition =""
    list_of_expectation_configuration=[]
    # Table Name
    tableName='userinfo'
#---------------------------------
    # List of expectations for each column
    #Expectations For Column [username]
    expectation_configuration = ExpectationUtil.expect_column_values_to_be_unique("username")
    list_of_expectation_configuration.append(expectation_configuration)

    expectation_configuration = ExpectationUtil.expect_column_values_to_not_be_null("username")
    list_of_expectation_configuration.append(expectation_configuration)

    # add list of expectation configuration to test suite
    suite = ExpectationSuite.create_expectation_suite(tableName,context ,list_of_expectation_configuration)
    context.save_expectation_suite(suite)
    list_of_expectation_configuration.clear()

    # getting batch requests for the tables
    batch_request = Batch.get_batch(dataSource.dataSchema, tableName, condition)

    # running tests on tables
    print(tableName, " ", batch_request)
    CheckPointConfig.run_check_point_config(context,batch_request, tableName)
```

Fig. 3. Example of a DQGen-generated GE validation script for the userinfo table. It demonstrates completeness and uniqueness checks on the username field

Each element represents a configurable placeholder in the test suite or validation script, supporting metadata such as connection details, table and column attributes, data constraints (e.g., min/max values, regex patterns), and expectation logic. This structured vocabulary enables consistent and flexible specification of data quality checks across different data sources and validation scenarios.

Validation and Accuracy of Code Generation Templates. The templates were developed and refined in collaboration with the ISP Company, ensuring they are based on practical industry experience. Practitioners from these companies provided real-world code, forming the foundation for creating templates that adhere to industry coding standards and best practices.

The testing and validation process involves a multi-tiered approach to validate the generated code, ensuring it meets high standards of accuracy and performance:

– The generated code is systematically evaluated to ensure it performs as intended and accurately implements the data quality rules defined in the

Table 3. Template Elements and Their Descriptions

Elements	Description
{UserName}{Password}	Data source Credentials
{IPAddress}{Port}	IP Address and Port Number
{DatabaseName}{DatabaseType}	Database Name and Type
{tableCode}	List of Tables
{expectationCode}	List of expectations for each column
{expectationMethod}	Expectation Name
{expectationParameter}	Expectation Parameters
{suite_name}	Suite Name that contains all test cases
{column}	Column Name
{dataType}	Column data Type
{valueSet}	Value set
{minValue}	Minimum Value
{maxValue}	Maximum Value
{regex}	Regular Expression
{regex_list}	Regular Expression list
{like_pattern}	Pattern
{strftime_format}	Time format
{json_schema}	JSON Schema
{schema_name}	Schema Name
{table_name}	Table Name
{condition}	Condition

model. This evaluation checks whether the code meets the specific requirements of real-world scenarios provided by practitioners and reliably executes the expected validation tasks.

- Manual code reviews were integral to the case, ensuring the generated code accurately processed error data from diverse sources like JSON files and transformed them into formats such as Parquet and CSV. These reviews addressed discrepancies and inefficiencies, ensuring the code met the unique demands of the application. This process maintained the robustness and quality of the codebase.
- The generated code is compared with manually created versions provided by practitioners to verify performance and functionality equivalence. This comparison validates the accuracy and alignment of the generated code with real-world coding standards and practices used within the companies.

Examples and the Case Study: The effectiveness of the templates is demonstrated through a case study that involves implementing the generated code in real-world situations. This case includes using the code in operational settings

within organizations. The code consistently adheres to the necessary performance and accuracy standards, as demonstrated by successful implementations and practitioner feedback. Python script templates are generic and reusable across datasets, parameterized by table schema and mapped dimensions. Users can adapt them for domain-specific quality checks or compliance needs.

The framework is designed to be schema-agnostic, meaning it does not depend on pre-defined table structures. Instead, it dynamically extracts schema metadata at runtime. This flexibility allows DQGen to adapt seamlessly to changes in schema, such as the addition or modification of columns, without needing manual updates to rule definitions. This capability is essential for maintaining quality assurance in evolving data environments, such as continuous ingestion pipelines or iterative schema designs.

At present, DQGen is optimized for flat, relational tables. It does not yet support validation rule generation for nested data structures such as JSON fields. However, future versions of the framework are planned to extend schema extraction and rule mapping to handle hierarchical data by applying recursive parsing and custom rule templates.

4 Case Study

To evaluate the effectiveness and practical applicability of DQGen, we conducted a case study in the telecommunications sector, focusing on data quality challenges encountered by a major Internet Service Provider (ISP). This environment is particularly relevant due to the volume, complexity, and business-critical nature of the data used in performance monitoring, customer analytics, billing, and compliance reporting.

Table 4. Summary of ISP Dataset Tables and Their Sizes

Table Name	Rows	Data Size	Description
`radacct`	26.25M	6.74 GB	User session logs
`radcheck`	118.5K	8.5 MB	User credential
`radgroupreply`	228	16 KB	Group-level settings
`radreply`	2.2K	208 KB	User reply attributes
`radusergroup`	118.9K	7.5 MB	Group-user associations
`userbillinfo`	372	6 KB	Billing info
`userinfo`	372	3 KB	User profile data

4.1 Dataset Overview

The dataset was collected from operational systems within the ISP and stored in a relational database over six months. It contains structured data from customer

interactions, authentication logs, billing records, and service configurations. Ultimately, the database contains seven core tables with a cumulative volume of over 26 million records and approximately 7 GB of data. The schema is accessible online via the project's supplementary materials repository[2]. Table 4 summarizes the dataset used in our case study, showing the size of each table, the number of records, and its primary function. This illustrates the diversity and scale of the dataset, serving as a useful benchmark for evaluating DQGen's automation capabilities.

4.2 Data Quality Challenges

The dataset includes a wide range of common, real-world data quality challenges that significantly impact the reliability of downstream analytics, reporting accuracy, and the robustness of AI/ML model outputs. These challenges, which were identified in multiple relational tables, are common in operational settings at the enterprise level:

- **Missing Values**: The *radreply* and *userbillinfo* tables often have missing values, especially in specific fields like *alt_email* and *promo_plan*. While these fields are not required, they often contain important information for client segmentation and personalized services. In supervised learning tasks, missing data can limit the quality of features and lead to biased analytics.
- **Inconsistent Formats**: The lack of standardized formatting rules and differences in data, such as date and time, could cause complexity in feature extraction for machine learning and require additional preprocessing, which increases the risk of missed anomalies.
- **Schema Drift**: After six months of observation, both the `radacct` and `radcheck` tables underwent schema changes, including new columns or changes in the data types. This caused problems in ETL workflows and made previously valid validation scripts no longer work. Without automation, maintaining quality rules across such evolving schemas is error-prone and labor-intensive.
- **Duplicate and Redundant Records**: The `radusergroup` table contained sporadic duplicate entries, largely due to retry failures in the ETL pipeline. Such redundancy can distort aggregate statistics (e.g., group counts), inflate usage metrics, and introduce label noise in supervised learning models.
- **Invalid References**: Referential integrity violations were observed between foreign keys in `radcheck` and the corresponding primary keys in `userinfo`. These mismatches could lead to orphaned records and unreliable join operations, degrading the quality of integrated datasets used for cross-table analysis or training entity resolution models.

[2] Database schema:https://anonymous.4open.science/r/DQGenSE-22B3/data/radddb_schema.sql.

These data quality defects not only illustrate the operational complexity of managing large-scale datasets but also underline the necessity for scalable, automated validation solutions. In data-intensive applications, undetected quality issues can propagate through analytic pipelines, leading to incorrect business insights, misinformed decision-making, and degraded AI system performance. The DQGen framework was evaluated in this context specifically to demonstrate its capability to detect, validate, and report such issues in a repeatable and systematic manner.

4.3 Application of DQGen

Using DQGen, we configured and automatically generated validation scripts for all seven tables. The mapping process dynamically associated relevant quality dimensions with validation rules in Great Expectations. For example, uniqueness checks were applied to primary keys such as the session ID "acctsessionid" and the username. Completeness checks targeted mandatory fields like login_time, usages, and billing_amount. Timeliness and consistency checks ensured that the session end time "acctstoptime" always occurred after the session start time "acctstarttime" and that the recorded duration matched the time difference. Additionally, format checks verified that phone numbers conformed to a standard regular expression and that date fields maintained consistent formatting. The generated scripts were then deployed in the ISP's staging environment, running nightly batch validations. Alerts were triggered for failed expectations, and detailed reports were produced for the data engineering team.

Figure 4 shows the analysis of the "radacct" table, which contains over 26 million rows. Within this table, the "acctstoptime" column had 452 null values, resulting in a data completeness rate of 99.9%. This high percentage demonstrates that the majority of the data in the column is intact and reliable, with only a minimal portion requiring further attention for completeness.

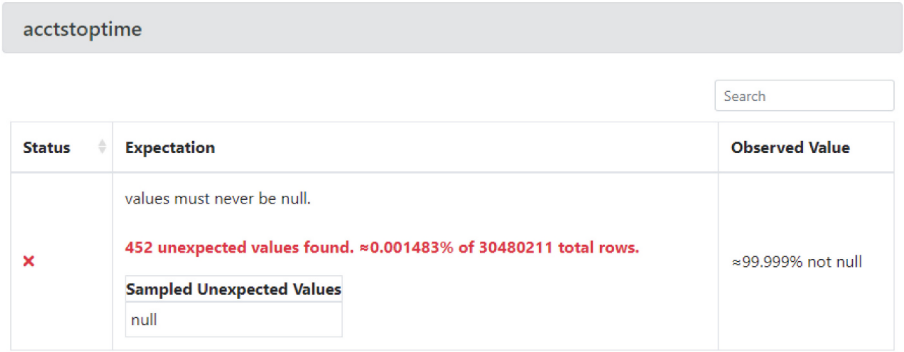

Fig. 4. Result of Checking completeness of values for Specific column

5 Evaluation

This section evaluates the DQGen framework based on two key dimensions: **efficiency** in reducing manual effort and development time, and **effectiveness** in accurately identifying data quality issues. We conducted an empirical study using a real-world dataset from a major Internet Service Provider (ISP) to understand how automated metadata-driven validation supports scalable and maintainable data governance in real-world DIA pipelines.

5.1 Research Questions

To assess the efficiency and effectiveness of the framework, this study is guided by the following research question:

- **RQ1**: *How efficiently does DQGen reduce the time required to define and apply data quality checks compared to manual scripting?*
- **RQ2**: *How accurately does the framework identify data quality issues compared to manual validation?*

5.2 Efficiency in Time and Effort

This section assesses the time required to generate data quality validation code using the framework compared to manual implementation. The evaluation highlights the significant time savings achieved through automated code generation and reinforces the framework's efficiency in supporting development tasks.

We conducted an experiment involving three software engineers with varying experience levels, one recent graduate, and two senior engineers. Each participant was tasked with writing data validation scripts for two tables ('userinfo' and 'radacct') using the Great Expectations Python library, without automation support. We measured three key metrics: total development time, rule coverage, and consistency of rule application.

These metrics were influenced by a sequence of standard in-house development tasks that participants performed, including: (1) exploring database schemas, (2) identifying critical fields based on domain knowledge, (3) manually implementing GE validation rules, (4) configuring test suites, and (5) debugging scripts to ensure correctness and full coverage. Participants used commonly available tooling such as SQL clients, Python IDEs (e.g., PyCharm), and the official GE documentation—replicating realistic project conditions found in data engineering workflows.

The senior developers had over five years of experience in data quality engineering and had previously worked on complex ETL validation pipelines. Reported durations, such as one full working day, represent actual effort tracked during the experiment and reflect the full cycle of manual script development, from schema review to execution validation.

Table 5. Time comparison between manual and DQGen-based validation code generation

Engineer Level	Task Type	Manual Time	DQGen Time
Fresh Graduate	Validation Script Setup	3 days [a]	15 min
Senior	Validation Script Setup	1 day	15 min
Senior	Validation Script Setup	1 day	15 min

[a] Includes data exploration, rule definition, and script testing.

Table 5 summarizes the time taken for manual vs. automated validation code development. It shows that manual implementation of data validation logic could take between 1 to 3 days, depending on the engineer's experience level. In contrast, the framework consistently generated the necessary validation code in under 15 min, regardless of the use case.

This experiment shows a time savings of over 90%, significantly reducing development overhead. In addition, manual validation effort in the ISP case study decreased from 30 to 12 h per week, while issue detection time dropped from 48 to 14 h—demonstrating notable gains in operational efficiency.

5.3 Coverage and Consistency of Validation

To assess the impact of DQGen beyond development speed, we investigate its ability to improve the coverage and consistency of data quality rule implementation when compared to manual validation scripts developed by engineers.

We analyzed validation scripts created manually by experienced engineers for a subset of the ISP dataset and compared them against the scripts generated by DQGen. We evaluated:

- Coverage: The proportion of relevant data quality dimensions (e.g., completeness, uniqueness, validity) applied to applicable columns in the dataset.
- Consistency: The uniformity of validation logic applied to similar types of columns across different tables (e.g., applying not_null checks to all required fields, not just a few).
- Rule Redundancy/Omission: The extent to which manual scripts omitted rules or applied them redundantly.

We focused on three critical tables (userinfo, radacct, radcheck) and conducted a detailed comparison of the validation logic across six quality dimensions supported by GEs.

Table 6 shows that the manual approach led to partial and inconsistent application of data validation rules. In contrast, the framework ensured full and uniform enforcement of rules by systematically mapping quality dimensions to appropriate expectations using metadata.

DQGen significantly improves the breadth of validation by ensuring that each applicable column is evaluated across all relevant dimensions, and eliminates

Table 6. Comparison between Manual Scripts and DQGen Scripts

Metric	Manual Scripts	DQGen Scripts
Avrage Quality Dimensions per Table	3.2	**5.7**
Percentage of Required Fields with `not_null` Checks	68%	**100%**
Consistency in Rule Application (across columns)	Medium	**High**
Redundant or Duplicated Rules	Present	**None**

human bias or oversight in rule selection. It also enhances consistency, applying identical rules to similar data types across tables without variation. These qualities are essential for ensuring trustworthiness and governance in large-scale data-intensive software engineering environments.

6 Discussion

The evaluation highlights DQGen's effectiveness in improving efficiency and consistency in data quality validation for data-intensive applications. Regarding efficiency (RQ1), DQGen significantly shortens the time and effort required to implement rules, enabling engineers to generate validation scripts in under 15 min, compared to the traditional multi-day process. For RQ2, the results show that DQGen enforces more consistent and comprehensive checks, avoiding the common pitfalls of incomplete manual validation by leveraging metadata-driven mappings.

While not all validated fields are used directly in machine learning (e.g., phone numbers as identifiers), validating them remains essential for data integrity and regulatory compliance. DQGen enables users to tailor validation rules according to the specific needs of downstream applications, whether for ML readiness, compliance, or operational accuracy.

Beyond time savings, DQGen enhances trust in data by reducing human error, supporting long-term scalability, and enabling reuse across domains. It integrates into CI/CD pipelines for continuous monitoring, promotes governance through traceable validation, and facilitates collaboration via readable scripts compatible with existing tools. In essence, DQGen supports automation, transparency, and quality-centric practices in modern data engineering workflows.

7 Threats to Validity

Several potential threats to validity, categorized as internal, external, construct, and conclusion validity threats, may affect the evaluation of the automated data quality framework and this study's findings.

7.1 Internal Validity

A limitation is that our case study relies on data from a single ISP. While insightful, its characteristics may not fully represent data quality challenges across industries. This may limit the generalizability of our findings to broader contexts.

To address this threat, we intentionally selected diverse domains within the ISP, including Customer Analytics, Network Performance, and Billing. Each area presents unique data quality challenges, enabling us to assess the framework's effectiveness across various situations. By considering different requirements, we aimed to enhance the robustness of our findings within the ISP context.

7.2 External Validity

The external validity of this study is influenced by its specific focus. While the framework was developed with a particular context in mind, its applicability may vary across organizations due to differences in governance structures, data quality challenges, and domain-specific requirements. To address these concerns, we are now extending our evaluation to include the healthcare sector, encompassing areas such as patient records, clinical trials, and medical billing systems. This additional assessment aimed to test the framework's adaptability in a significantly different setting. Early results from the healthcare sector demonstrated its flexibility, suggesting that it can be tailored to meet the unique demands of other industries.

8 Conclusion

This paper presents DQGen, a metadata-driven framework for automated data quality validation using the Great Expectations (GEs) library. By bridging high-level quality dimensions with executable rules, DQGen reduces manual effort and enhances consistency in data-intensive workflows.

A case study with a major ISP showed that issue detection time dropped by 70% and validation effort was reduced by 60%, highlighting both efficiency and operational gains. The framework's adaptability to schema changes and integration with CI/CD pipelines positions it as a scalable solution for modern data governance.

As future work, we plan to apply DQGen in additional domains such as healthcare, finance, and public services to further validate its flexibility and robustness. Broader adoption across diverse data ecosystems will help refine its capabilities and demonstrate its generalizability for enterprise-scale data quality assurance.

Acknowledgments. We wishes to acknowledge the Behavior-enabled IoT (BeT) PRIN 2022 project (2022TEPX4R) for supporting this research. This work includes language refined with generative AI tools (e.g., Grammarly, ChatGPT), reviewed and edited by the first author for clarity and accuracy.

References

1. Bosch, J., Olsson, H.H., Crnkovic, I.: Engineering AI systems: a research agenda. In: Artificial Intelligence Paradigms for Smart Cyber-Physical Systems, pp. 1–19 (2021)
2. Schelter, S., Boese, J.-H., Kirschnick, J., Klein, T., Seufert, S.: Automatically tracking metadata and provenance of machine learning experiments (2017)
3. Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., Grafberger, A.: Automating large-scale data quality verification. Proc. VLDB Endowment **11**(12), 1781–1794 (2018)
4. Expectations, G.: Great expectations (2021). https://greatexpectations.io/
5. A. Labs: Deequ - unit tests for data (2018). https://github.com/awslabs/deequ
6. DBT. Data build tool (2022). https://www.getdbt.com
7. Syriani, E., Luhunu, L., Sahraoui, H.: Systematic mapping study of template-based code generation. Comput. Lang. Syst. Struct. **52**, 43–62 (2018)
8. Breck, E., Cai, S., Nielsen, E., Salib, M., Sculley, D.: The ml test score: a rubric for ml production readiness and technical debt reduction. In: IEEE International Conference on Big Data (Big Data). IEEE 2017, pp. 1123–1132 (2017)
9. Pipino, L.L., Lee, Y.W., Wang, R.Y.: Data quality assessment. Commun. ACM **45**(4), 211–218 (2002)
10. Herbold, S., Trautsch, A., Ledel, B., Aghamohammadi, A., Ghaleb, T.A., Chahal, K.K., Bossenmaier, T., Nagaria, B., Makedonski, P., Ahmadabadi, M.N.: A fine-grained data set and analysis of tangling in bug fixing commits. Empir. Softw. Eng. **27**(6), 125 (2022)
11. NV S.D.: Soda SQL: open-source data quality tool for data engineers (2023). https://docs.soda.io/soda-sql/. Accessed 21 Jun 2025
12. Amazon Web Services. Aws glue databrew: visually clean and normalize data without writing code (2023). https://aws.amazon.com/glue/features/databrew/. Accessed 21 Jun 2025
13. Fadlallah, H., et al.: BigQA: declarative big data quality assessment. ACM J. Data Inf. Qual. (2023)
14. Abughazala, M., Muccini, H., Qadri, K.: PYDAQU: python data quality code generation based on data architecture. In: 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), pp. 60–64 (2023)
15. Gopalan, R.P., Krishna, A.: Duplicate bug report detection using clustering. In: 23rd Australian Software Engineering Conference, pp. 104–109. IEEE (2014)
16. Batini, C., Scannapieco, M., et al.: Data and Information Quality, vol. 63. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-24106-7
17. Berti-Equille, L.: Measuring and modelling data quality for quality-awareness in data mining. In: Guillet, F.J., Hamilton, H.J. (eds.) Quality Measures in Data Mining, pp. 101–126. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-44918-8_5
18. Wang, R.Y., Strong, D.M.: Beyond accuracy: what data quality means to data consumers. J. Manag. Inf. Syst. **12**(4), 5–33 (1996)
19. Karkouch, A., Mousannif, H., AlMoatassime, H., Noel, T.: Data quality in internet of things: a state-of-the-art survey. J. Network Comput. Appl. **73**, 57–81 (2016)
20. Elouataoui, W., Alaoui, I.E., Gahi, Y.: Data quality in the era of big data: A global review. Big Data Intell. Smart Appli. 1–25 (2022)
21. Voelter, M.: A catalog of patterns for program generation. In: EuroPLoP, pp. 285–320 (2003)

A Visionary Architecture for Adaptive Data Contracts in Behavior-Driven IoT Systems

Moamin Abughazala[1,2] and Henry Muccini[1(✉)]

[1] University of L'Aquila, L'Aquila, Italy
{moamin.abughazala1,henry.muccini}@univaq.it
[2] An Najah National University, Nablus, Palestine
m.abughazaleh@najah.edu

Abstract. Static data contracts in IoT systems are inadequate for dynamic, behavior-driven environments where real-time adaptation is required to maintain data quality, responsiveness, and resource efficiency. This paper is presented as a visionary architecture paper to outline foundational directions for future AI-integrated data governance in adaptive IoT systems. It introduces two core components: (1) an AI Engine responsible for analyzing behavioral data and predicting system conditions such as user presence, preferences, or load patterns, and (2) a Generative AI Engine based on Large Language Models (LLMs), which automatically produces adaptive data contracts based on insights derived from the AI Engine. These contracts define expected data schema, validation constraints, and quality expectations. The architecture is motivated by smart environment use cases, such as museums, where adaptation to user behavior can improve experience, efficiency, and personalization. We present five integrated layers: IoT Data Collection, Data Processing, AI Prediction, Generative AI Contract Generation, and Contract Enforcement with Feedback. Each layer contributes to enabling runtime adaptation in data-intensive IoT systems. This early-stage architecture sets the foundation for future implementations that aim to automate data governance using AI-driven decision logic and generation mechanisms.

Keywords: IoT Systems · Generative AI · LLMs · Behavior-Driven Architecture · Adaptive Systems · Data Quality · AI for IoT · Data Contracts

1 Introduction

Modern data-intensive systems, especially those in IoT domains, are entering a new era of contextual complexity, where human behavior, environmental conditions, and runtime variability demand **adaptive, intelligent architectural responses**. Traditional approaches to data governance, which rely heavily on *static data contracts*, are ill-equipped for such environments. These contracts

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 381–388, 2026.
https://doi.org/10.1007/978-3-032-04403-7_32

are often manually defined and rigidly enforced, making them unsuitable for systems that must *dynamically adapt to behavioral or contextual changes in real time* [1] [2].

The limitations of static governance models have become particularly evident in *behavior-rich IoT environments*, such as smart buildings, personalized healthcare, and interactive museums. In these domains, system responsiveness and data quality depend on the ability to interpret, react to, and evolve with **user behavior**. Recent work on *Behavior-enabled IoT (Be-IoT)* [3] [4] and the *Internet of Behaviors (IoB)* [5] underscores the growing importance of behavior as a first-class architectural concern. Yet, despite advances in adaptive control and context-aware middleware [6], most systems continue to treat *data validation and schema logic as immutable artifacts*—failing to reflect behavioral insights or support real-time adaptation.

This paper proposes a **visionary, layered architecture** designed to close this gap by integrating *predictive and generative AI components* directly into the data governance layer. At the core of the architecture are two decoupled modules:

- An **AI Analysis Engine**, which applies machine learning techniques to forecast behavioral patterns and system conditions, and
- A **Generative AI Engine**, which leverages *Large Language Models (LLMs)* to dynamically synthesize data contracts that define schema, validation rules, latency expectations, and data completeness thresholds.

This architectural separation aligns with modern modular design principles [7] [8], enabling *scalable, explainable, and independently evolvable* components across distributed edge-cloud systems. The Generative AI Engine facilitates *prompt-based contract creation*, supporting low-code and domain-adaptive governance logic, while the feedback loop ensures *continuous refinement based on performance metrics* and contract violations. We investigate *smart museum environments* where user engagement, occupancy trends, and interaction preferences shape data collection and interpretation. We show that static contracts are inadequate in dynamic contexts and that *adaptive contracts* based on behavioral signals can enhance system efficiency, user experience, and personalization.

This paper provides a **forward-looking blueprint** for designing autonomous, behavior-aware governance frameworks in IoT. It outlines five conceptual layers of the architecture, scenario-to-layer mappings, a prototype workflow for contract generation, and evaluation criteria for future implementations. This work uniquely contributes a dual-AI architectural loop that merges predictive modeling with LLM-based contract generation, creating a feedback-driven governance mechanism that adapts data policies in real time. Unlike existing solutions that treat data validation as static or decoupled from system behavior, our approach positions behavior itself as the central trigger for autonomous governance.

The Rest of this Paper is Structured as Follows: Section 2 reviews related work on behavior-aware IoT systems, smart contracts, and data validation tools. Section 3 details the proposed layered architecture, including its design rationale

and workflow. Section 4 presents motivating scenarios and maps them to system components. Section 5 discusses the architecture's strengths and limitations. Section 6 outlines its visionary scope and long-term impact. Section 7 concludes the paper, and Sect. 8 outlines future directions including feasibility validation and prompt engineering for LLMs.

2 Background and Related Work

Adaptive IoT systems have gained attention in domains requiring context-awareness and real-time responsiveness. Behavior-driven paradigms like the BeT architecture [3] integrate behavioral insights into software systems but do not support dynamic generation of governance artifacts. The Internet of Behaviors (IoB) emphasizes behavioral data for adaptive services [5], yet lacks integration with schema evolution or runtime validation [9]. Blockchain-based governance models offer immutability but lack behavioral adaptability [2]. ML-based platforms such as SOLAR [4] optimize control but don't extend to dynamic data validation. Tools like Great Expectations [10] and Apache Deequ [11] enforce data rules but require manual definition.

Our architecture addresses these gaps by combining behavioral prediction with generative AI to automate adaptive data contracts, linking AI analysis and generation through a feedback loop that supports dynamic, real-time schema and validation updates.

3 Proposed Layered Architecture

This section presents a conceptual layered architecture designed to enable adaptive data contracts in behavior-driven IoT systems. The architecture separates responsibilities across distinct layers to support modularity, scalability, and real-time responsiveness. At its core, the architecture integrates two AI-driven components: an AI Engine responsible for behavior analysis and prediction, and a Generative AI Engine for synthesizing adaptive data contracts.

3.1 Overview

The proposed architecture consists of five conceptual layers designed to support the dynamic generation and enforcement of data contracts in behavior-driven IoT systems. Each layer plays a distinct role in enabling system adaptability, data quality assurance, and continuous behavioral alignment.

1. IoT Device Layer: This layer consists of diverse sensors and actuators that collect environmental and user interaction data, such as temperature, motion, or touch input, forming a behavioral footprint of system usage. Devices support edge processing and communicate via standard protocols (e.g., MQTT, BLE). Their real-time data streams serve as the foundation for context inference and adaptive contract generation.

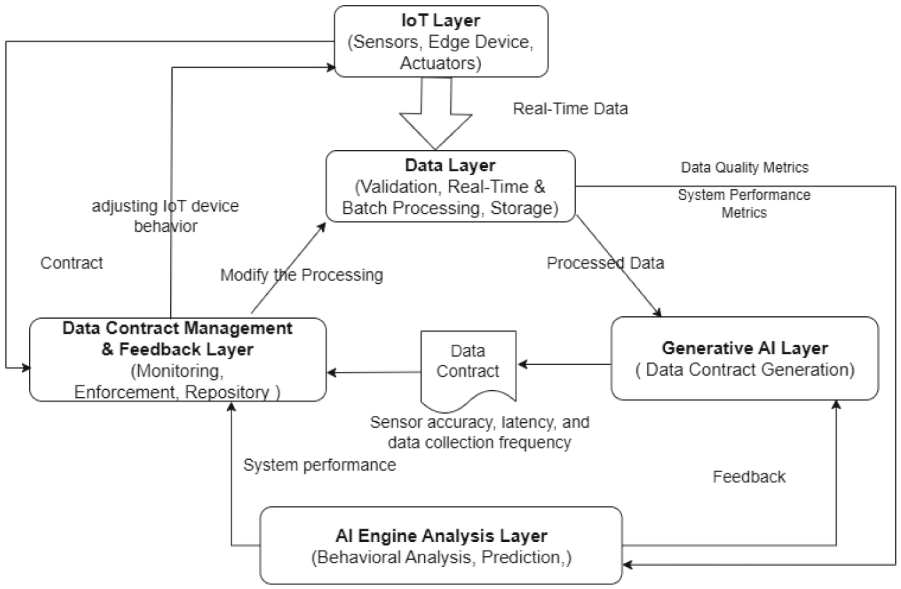

Fig. 1. Layered architecture enabling adaptive data contracts in behavior-driven IoT systems via real-time analysis, contract generation, and feedback loops.

2. Data Processing Layer: This layer converts raw sensor inputs into clean, structured, and context-rich data. It handles tasks such as data cleaning, normalization, semantic tagging, and feature extraction, ensuring that only high-integrity, relevant data flows into downstream AI components for robust analysis and contract generation.
3. AI Analysis Engine Layer: This layer applies machine learning and time-series models to identify behavioral patterns, forecast usage trends, and infer contextual conditions. Insights such as peak activity periods or anomalies trigger adaptive processes. Its modular design supports domain-specific models, online learning, and flexible deployment across edge or cloud environments.
4. Generative AI Engine Layer: This layer dynamically generates structured data contracts using LLMs, translating behavioral insights into specifications like validation rules, sampling rates, and latency thresholds. Prompt engineering guides the model output, while template-based generation ensures consistency and compliance with predefined schemas.
5. Contract Management & Feedback Layer: This layer enforces data contracts across system components and monitors performance metrics like latency, compliance, and data quality. It triggers feedback loops upon violations, enabling iterative refinement by the AI and Generative layers. It also supports versioning, rollback, and logging to ensure traceability and system reliability.

Figure 1 illustrates the proposed layered architecture for behavior-driven IoT systems, highlighting the flow from sensor data collection to behavioral analysis, adaptive contract generation, enforcement, and feedback.

3.2 Architectural Flow and Design Rationale

The architecture operates as a continuous loop of sensing, analysis, adaptation, and enforcement. Data is first collected by IoT devices and preprocessed to ensure quality and context. This structured input feeds into the AI Analysis Engine, which applies predictive models to infer patterns—such as visitor density or anomalous behavior—serving as triggers for adaptation. The Generative AI Engine then translates these insights into data contracts (e.g., YAML schemas) specifying validation logic and sampling requirements. These contracts are enforced by the Contract Management Layer, which monitors system performance and feeds violations back into the AI layers for refinement.

A central design principle is the separation between behavior analysis and contract generation. Predictive models identify dynamic patterns, while generative models—especially LLMs—create structured, domain-specific governance artifacts. This modular approach enhances scalability, explainability, and deployment flexibility across edge-cloud infrastructures, aligning with cloud orchestration and federated analytics paradigms [7,8]. The architecture offers a generalizable blueprint for domains like healthcare, energy, and mobility, where adaptive governance must align with changing behavioral contexts.

4 Motivating Scenarios

To illustrate the need for adaptive data contracts in behavior-driven IoT systems, we originally considered three motivating scenarios from the domain of smart museums. However, due to space constraints, we focus on one representative example—managing visitor flow—which effectively demonstrates how dynamic behavioral contexts influence data governance requirements and reveal the limitations of static contracts.

4.1 Scenario: Managing Visitor Flow

During peak hours, certain museum galleries may experience overcrowding, leading to long wait times and reduced visitor satisfaction. A static data contract might define a fixed sampling rate for occupancy sensors, regardless of real-time conditions. In contrast, the proposed architecture enables the **AI Analysis Engine** to predict peak visitation periods using behavioral trends, such as entry time clusters or seasonal traffic patterns. The **Generative AI Engine** then dynamically adjusts the data contract for occupancy sensors by increasing the data collection frequency and enforcing stricter latency and completeness constraints. The **Contract Management Layer** ensures these adjustments are deployed and enforced, allowing the system to proactively manage crowd flow via digital signage or interactive guidance.

4.2 Scenario-to-Layer Mapping

Each scenario engages a different combination of layers, illustrating the modularity of the architecture. Managing Visitor Flow involves all five layers: IoT devices track occupancy, Processing cleans sensor streams, AI predicts peak times, Generative AI adjusts sampling contracts, and Contract Management enforces feedback. Personalized Visitor Experience emphasizes user-level adaptation by combining IoT interaction logs, Processing for contextual tagging, AI profiling, and Generative AI contracts for tailored content, with Contract Management ensuring continuous adjustment. Energy Optimization demonstrates efficiency by relying on IoT presence data and AI forecasting, where Generative AI generates energy-saving contracts that are dynamically enforced through the Contract Management layer.

Together, these scenarios show how behavioral complexity is managed through clear separation between sensing and inference, while the consistent role of Contract Management guarantees coherence and adaptive feedback across the system.

5 Discussion

The proposed layered architecture offers a modular and scalable solution for managing adaptive data contracts in behavior-driven IoT systems. By decoupling behavioral analysis from data contract generation, it provides flexible data governance that adapts to evolving user contexts and operational needs. This approach highlights the distinct roles of predictive and generative AI technologies and embraces principles of loose coupling and component specialization.

Its generalizability is a key strength; while it is inspired by smart museum scenarios, it is also applicable to smart energy systems and healthcare, where user behavior can influence contract adaptations. The layered design facilitates independent evolution and customization for different domains.

Additionally, the architecture aligns with trends in MLOps and AI orchestration, allowing for integration with standard ML pipelines and low-code contract creation, simplifying governance in complex IoT environments. However, challenges remain, such as the need for a working prototype, the careful selection of ML models, and concerns about interpretability and reproducibility when integrating LLMs into safety-critical systems [12] [13].

Despite these limitations, this architecture lays a foundation for advanced, AI-integrated governance frameworks in data-intensive systems, bridging the gap between behavior analysis and data validation.

6 Visionary Scope and Long-Term Impact

This paper presents a forward-looking architecture at the intersection of behavior-aware systems and autonomous data governance. As IoT environments grow more dynamic and user-centric, traditional static validation mechanisms

no longer suffice for ensuring real-time responsiveness, personalization, and efficiency.

By decoupling behavioral analysis from data contract generation, the architecture introduces a modular and AI-driven blueprint for systems that evolve with usage patterns and environmental feedback. This marks a shift from reactive, rule-based governance to proactive, generative adaptation.

The vision supports self-adaptive infrastructures capable of monitoring behavioral dynamics, autonomously generating and updating data governance rules, and refining decisions through continuous feedback. These capabilities, such as runtime behavioral monitoring, autonomous contract generation via LLMs, and low-code adaptation, enable the architecture to generalize across domains like energy, healthcare, and mobility.

Ultimately, by outlining this vision now, we aim to encourage future research and practical implementations that bring this adaptive governance framework to life.

7 Conclusion

This paper presented a layered architecture for adaptive data contracts in behavior-driven IoT systems. By decoupling behavioral prediction from contract generation, the approach leverages the complementary strengths of predictive and generative AI models. Through smart museum scenarios, we demonstrated the potential to enhance personalization, responsiveness, and resource efficiency. The architecture lays a conceptual foundation for AI-integrated data governance in dynamic IoT environments.

As an early-stage architectural vision, our work contributes to the growing body of research exploring how AI can reshape system-level governance, from behavior-aware analytics to self-updating validation layers. Its generality invites further exploration and domain-specific refinement.

8 Future Work

To develop the proposed architecture into a functional system, we outline several key directions:

- Feasibility & Implementation: Initial simulations are underway to validate adaptive contract generation using behavioral triggers and LLM-based prompt synthesis.
- Prototype Development: A working prototype will integrate behavior detection, contract generation, and feedback loops using real or synthetic IoT data.
- Prompt Engineering: Research will explore optimal prompt formats and enrichment strategies to ensure valid, domain-aligned contract generation.
- Evaluation Metrics: System performance will be assessed via prediction accuracy, contract compliance, adaptation latency, resource usage, and data quality improvements.

– Domain Adaptation: The architecture will be customized for diverse sectors (e.g., energy, healthcare, mobility) by adapting device layers, processing logic, and prompt templates.

Acknowledgments. We wishes to acknowledge the Behavior-enabled IoT (BeT) PRIN 2022 project (2022TEPX4R) for supporting this research. This work includes language refined with generative AI tools (e.g., Grammarly, ChatGPT), reviewed and edited by the first author for clarity and accuracy.

References

1. Balint, F.B.: Negotiating, monitoring and recommending data contracts in IoT dataspaces. Ph.D. dissertation, Wien (2016)
2. Sultana, T., Almogren, A., Akbar, M., Zuair, M., Ullah, I., Javaid, N.: Data sharing system integrating access control mechanism using blockchain-based smart contracts for IoT devices. Appl. Sci. **10**(2), 488 (2020)
3. Muccini, H., Russo, B., Zimeo, E.: The bet project: behavior-enabled IoT. arXiv preprint arXiv:2307.13186 (2023)
4. De Caro, A., Zimeo, E.: Behavior enabled IoT: a software architecture for self-adapting a renewable energy community. Internet Things **27**, 101294 (2024)
5. Moghaddam, M.T., Muccini, H., Dugdale, J., Kjægaard, M.B.: Designing internet of behaviors systems. In: 2022 IEEE 19th International Conference on Software Architecture (ICSA), pp. 124–134. IEEE (2022)
6. Satyanarayanan, M.: The emergence of edge computing. Computer **50**(1), 30–39 (2017)
7. Abbas, N., Zhang, Y., Taherkordi, A., Skeie, T.: Mobile edge computing: a survey. IEEE Internet Things J. **5**(1), 450–465 (2017)
8. Singh, P., Singh, M.K., Singh, R., Singh, N.: Federated learning: Challenges, methods, and future directions. In:Yadav, S.P., Bhati, B.S., Mahato, D.P., Kumar, S. (eds.) Federated Learning for IoT Applications, pp. 199–214. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-85559-8_13
9. Raza, S., Wallgren, L., Voigt, T.: Svelte: Real-time intrusion detection in the internet of things. Ad Hoc Netw. **11**(8), 2661–2674 (2013)
10. G. Expectations. Great expectations (2021). https://greatexpectations.io/
11. A. Labs. Deequ - unit tests for data (2018). https://github.com/awslabs/deequ
12. Wang, Q., et al.: Schemaagent: a multi-agents framework for generating relational database schema. arXiv preprint arXiv:2503.23886 (2025)
13. Liu, X., et al.: A survey of nl2sql with large language models: where are we, and where are we going?" arXiv preprint arXiv:2408.05109 (2024)

Author Index

A
Abdelmoumen, Aymen 243
Abdennadher, Nabil 329
Abdullah, Milad 345
Abughazala, Moamin 363, 381
Ahmad, Noman 105
Ali, Mohammed Fahad 285
Amasanti, Giorgio 89
Aneggi, Alessandro 62

B
Bakhtin, Alexander 78, 105
Becker, Steffen 141
Belala, Faiza 259
Benzadri, Zakaria 243, 259
Bolgheroni, Roberto Oliveira 354
Bosilia, Nathanael 171
Briechle, Dominique 285
Briechle-Mathiszig, Marit 285

C
Caporuscio, Mauro 119
Cerny, Tomas 3
Codabux, Zadia 21
Copei, Sebastian 202, 212
Correia, Filipe F. 12

D
de Oliveira Rosa, Thatiane 187
Delporte, Pauline 313

E
Esposito, Matteo 105

F
Ferreira, Renato Cordeiro 187, 354
Ferreira, Ricardo 12
Feryel, Benina 259
Finger, Marcelo 354

G
Geger, Tobias 285
Giannopoulos, Giorgos 337
Gidarakos, Panagiotis 337
Giovanoli, Claudio 231
Giussani, Samuele 70
Goldman, Alfredo 187, 354
Goulis, Gabriel Elijah 3

H
Haindl, Philipp 171
Hnětynka, Petr 345
Hohlfeld, Oliver 202, 212

J
Jahić, Jasmin 89
Johansson, Nils 119

K
Kacem, Ahmed Hadj 259
Kavouras, Loukas 337
Kehrer, Timo 29
Klinaku, Floriment 141
Kojo, Ricardo Hideki Hangai 187
Kosiol, Jens 202, 212
Kozák, David 3

L
Lago, Patricia 313
Lam, Lucas Quaresma Medina 354
Lammert, Jonas 141
Lawand, Daniel Angelo Esteves 354
Lenarduzzi, Valentina 105
Levi-Soskin, Natali 273

M
Mahmud, Jiyan Salim 345
Maroulis, Stavros 337

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 389–390, 2026.
https://doi.org/10.1007/978-3-032-04403-7

Mendonça, Francisco 329
Motsios, Angelos 29
Muccini, Henry 41, 363, 381

P
Palomba, Fabio 21
Papastefanatos, George 337
Perera, Hasini Sumalee 21
Pugno, Michele 295
Puthuparambil, Bennet 231

Q
Queiroz, Paulo G. G. 12

R
Rahmani, Adam Kattan 3
Raibulet, Claudia 295
Rajenthiram, Keerthiga 313
Rausch, Andreas 127, 285
Real, Luiz Fernando Corte 187
Robredo, Mikel 105
Rodriguez, Ismael Bouassida 243

S
Schmalriede, Florian 53
Serugendo, Giovanna Di Marzo 329
Shenoy, Manthan Venkataramana 127
Su, Ruoyu 105

T
Taibi, Davide 105
Theologitis, Nikolas 337
Tsigkanos, Christos 29

U
Unkelos-Shpigel, Naomi 273

V
Vaidhyanathan, Karthik 41
Vojnar, Tomáš 3

W
Warnett, Stephen John 155
Weinberger, Gerald 171
Werner, Robert 285

Z
Zdun, Uwe 155